Onlinevarsity

Storing and Manipulating Data with MySQL

Are you registered with **Onlinevarsity.com**?	Yes ●	No ●
Did you download this book from **Onlinevarsity.com**?	Yes ●	No ●

Scores

For each **YES** you score **50**
For each **NO** you score **0**
If you score less than 100 this book is illegal.
Register on **www.onlinevarsity.com**

Storing and Manipulating Data with MySQL

Learner's Guide

© 2016 Aptech Limited

All rights reserved.

No part of this book may be reproduced or copied in any form or by any means – graphic, electronic or mechanical, including photocopying, recording, taping, or storing in information retrieval system or sent or transferred without the prior written permission of copyright owner Aptech Limited.

All trademarks acknowledged.

APTECH LIMITED
Contact E-mail: ov-support@onlinevarsity.com

First Edition - 2016

Dear Learner,

We congratulate you on your decision to pursue an Aptech Worldwide course.

Aptech Ltd. designs its courses using a sound instructional design model – from conceptualization to execution, incorporating the following key aspects:

- Scanning the user system and needs assessment

Needs assessment is carried out to find the educational and training needs of the learner

Technology trends are regularly scanned and tracked by core teams at Aptech Ltd. TAG* analyzes these on a monthly basis to understand the emerging technology training needs for the Industry.

An annual Industry Recruitment Profile Survey# is conducted during August - October to understand the technologies that Industries would be adapting in the next 2 to 3 years. An analysis of these trends & recruitment needs is then carried out to understand the skill requirements for different roles & career opportunities.

The skill requirements are then mapped with the learner profile (user system) to derive the Learning objectives for the different roles.

- Needs analysis and design of curriculum

The Learning objectives are then analyzed and translated into learning tasks. Each learning task or activity is analyzed in terms of knowledge, skills and attitudes that are required to perform that task. Teachers and domain experts do this jointly. These are then grouped in clusters to form the subjects to be covered by the curriculum.

In addition, the society, the teachers, and the industry expect certain knowledge and skills that are related to abilities such as *learning-to-learn, thinking, adaptability, problem solving, positive attitude etc*. These competencies would cover both cognitive and affective domains.

A precedence diagram for the subjects is drawn where the prerequisites for each subject are graphically illustrated. The number of levels in this diagram is determined by the duration of the course in terms of number of semesters etc. Using the precedence diagram and the time duration for each subject, the curriculum is organized.

- Design & development of instructional materials

The content outlines are developed by including additional topics that are required for the completion of the domain and for the logical development of the competencies identified. Evaluation strategy and scheme is developed for the subject. The topics are arranged/organized in a meaningful sequence.

The detailed instructional material – Training aids, Learner material, reference material, project guidelines, etc.- are then developed. Rigorous quality checks are conducted at every stage.

- Strategies for delivery of instruction

  Careful consideration is given for the integral development of abilities like thinking, problem solving, learning-to-learn etc. by selecting appropriate instructional strategies (training methodology), instructional activities and instructional materials.

  The area of IT is fast changing and nebulous. Hence considerable flexibility is provided in the instructional process by specially including creative activities with group interaction between the students and the trainer. The positive aspects of web based learning –acquiring information, organizing information and acting on the basis of insufficient information are some of the aspects, which are incorporated, in the instructional process.

- Assessment of learning

  The learning is assessed through different modes – tests, assignments & projects. The assessment system is designed to evaluate the level of knowledge & skills as defined by the learning objectives.

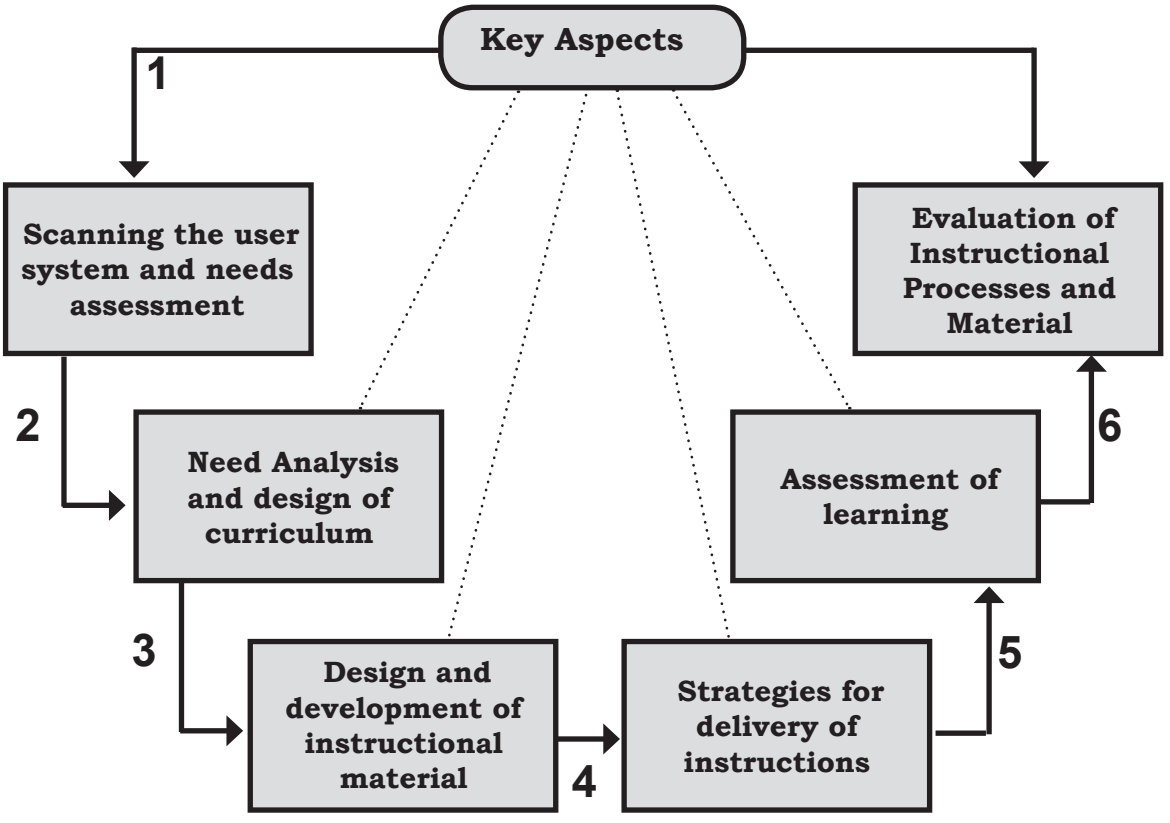
- Evaluation of instructional process and instructional materials

  The instructional process is backed by an elaborate monitoring system to evaluate - on-time delivery, understanding of a subject module, ability of the instructor to impart learning. As an integral part of this process, we request you to kindly send us your feedback in the reply pre-paid form appended at the end of each module.

*TAG – Technology & Academics Group comprises members from Aptech Ltd., professors from reputed Academic Institutions, Senior Managers from Industry, Technical gurus from Software Majors & representatives from regulatory organizations/forums.

Technology heads of Aptech Ltd. meet on a monthly basis to share and evaluate the technology trends. The group interfaces with the representatives of the TAG thrice a year to review and validate the technology and academic directions and endeavors of Aptech Ltd.

Aptech New Products Design Model



BE AHEAD OF EVERYONE ELSE

READ ARTICLES

Onlinevarsity

Preface

MySQL 5.7 is a Database Management System that enables us to enter, organize, and select data from a database. MySQL is open source software. It is available for free download on the Internet. The advantage of MySQL is that it provides flexibility of modifying its source code. It has almost all the features of the commercial databases in the market. It also provides with different tools that facilitates the user to work on databases.

This book explains how to create and manage databases and tables in MySQL. The book also describes queries, joins, functions, and stored procedures. New enhancements in MySQL 5.7 such as those made to server security, MySQL database maintenance, backup, replication, transaction management, performance optimization, and storage systems are also explored in this book. New features such as the new schemas, spatial data types, and so on are also discussed.

This book is the result of a concentrated effort of the Design Team, which is continuously striving to bring you the best and the latest in Information Technology. The process of design has been a part of the ISO 9001 certification for Aptech-IT Division, Education Support Services. As part of Aptech's quality drive, this team does intensive research and curriculum enrichment to keep it in line with industry trends. We will be glad to receive your suggestions.

We will be glad to receive your suggestions.

Design Team

Onlinevarsity

BLOGS

WHERE THE EXPERTS SPEAK THE EXPERIENCE

Aptech™
COMPUTER EDUCATION
Unleash your potential

Table of Contents

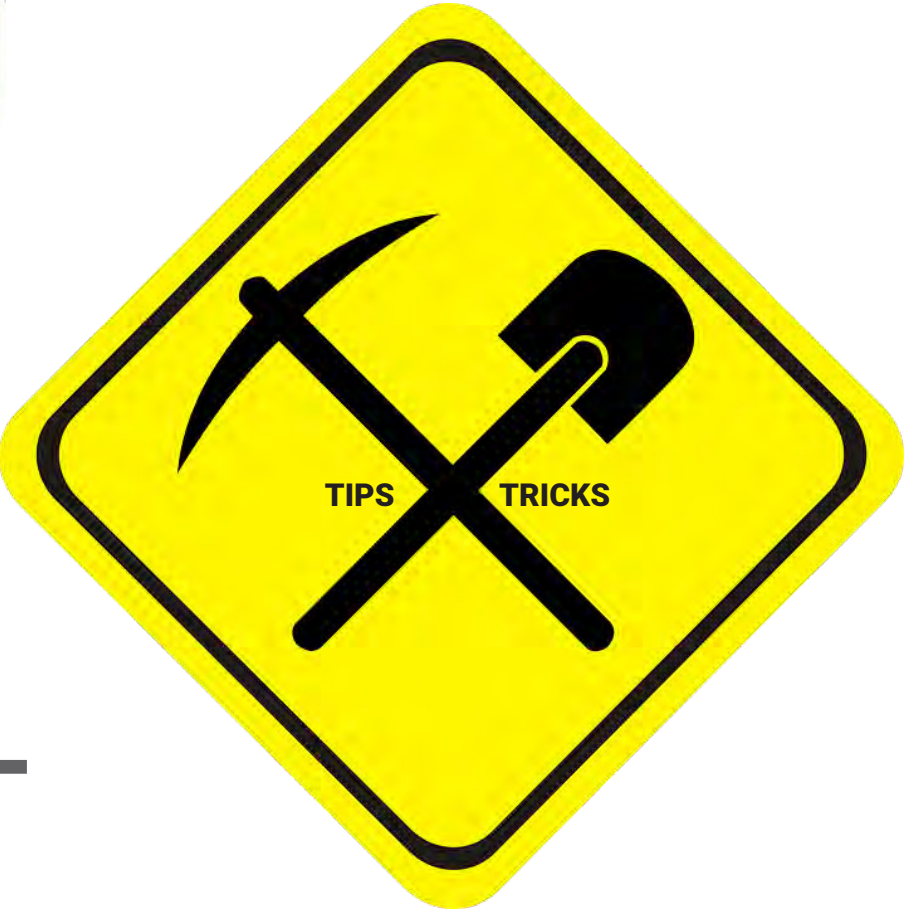
Sessions

1. Introduction to MySQL as an Open Source Database
2. Installing and Configuring MySQL
3. Installing and Configuring MySQL (Lab)
4. Using MySQL
5. Using MySQL (Lab)
6. Implementing SQL Queries Using MySQL - I
7. Implementing SQL Queries Using MySQL - I (Lab)
8. Implementing SQL Queries Using MySQL - II
9. Implementing SQL Queries Using MySQL - II (Lab)
10. Using Joins
11. Using Joins (Lab)
12. Using Basic Functions in MySQL - I
13. Using Basic Functions in MySQL - I (Lab)
14. Using Basic Functions in MySQL - II
15. Using Basic Functions in MySQL - II (Lab)
16. Controlling and Managing MySQL Database
17. Controlling and Managing MySQL Database (Lab)
18. An Overview of MySQL 5.7
19. Transaction Management and Globalization
20. Performance Optimization and Scalability
21. Replication, Data Backup, Partitioning, and Storage Systems

Onlinevarsity

TIPS & TRICKS

BECOME A ~~HARD~~ SMART WORKING PROFESSIONAL



Session 1

Introduction to MySQL as an Open Source Database

Concepts

Objectives

At the end of this session, the student will be able to:

- *List the features of MySQL.*
- *State the advantages of MySQL over other RDBMS.*
- *Compare MySQL with other RDBMS.*
- *State the advantages of PHP in MySQL.*
- *Explain open source software licenses.*

1.1 Introduction

A database is a systematic collection of data. A Database Management System (DBMS) is required to store, access, delete, or otherwise organize data in a database.

MySQL is an open source DBMS. You can freely download, modify, and use open source software without having to pay any fees or royalty to the original author.

In this session, you will learn about MySQL, its features, and its advantages over other Relational Database Management Systems (RDBMS). A RDBMS is based on relational model as specified by E.F. Codd. In addition, you will learn about the advantages of MySQL as an open source database and use of HyperText PreProcessor (PHP) with MySQL.

1.2 Overview of MySQL

Consider a library which lends books to its members. Traditionally, the details of books, members, and lending are maintained manually using ledgers. As the number of books increases, managing and searching for books, members, and lending details becomes difficult. This information can be stored in rows and columns in a table. A database can be created to store these tables. In addition, a DBMS can be used to manage the databases.

A DBMS can be defined as a software program that stores and manages databases. A database is a system used to store the data in a structured format. In other words, database can be defined as an organized collection of data. DBMS is responsible for managing the various database operations such as adding, accessing, and processing of data. A DBMS helps you to manage data in two ways:

- It provides an interface to manage data

- It supports connectivity to other applications that can be used to manage data

Both, DBMS and RDBMS, perform the same task of storing and managing data. One of the key differences between DBMS and RDBMS is that RDBMS splits large amount of data into smaller tables and establishes relationship between the tables. DBMS stores large amount of data in a single table. Also, the RDBMS is based on a relational model whereas DBMS is not.

MySQL is an open source RDBMS. MySQL uses the standardized Structured Query Language (SQL) to manage the database. MySQL is developed and distributed by MySQL AB, a company founded by the MySQL developers. In 2008, Sun Microsystems acquired MySQL AB. In 2010, Oracle acquired Sun Microsystems and hence, MySQL is now owned by Oracle Corp.

1.2.1 Features of MySQL

MySQL was designed to achieve speed, robustness, and ease of use. The features of MySQL are as follows:

- Technical Features:
  - Is written in C and C++
  - Is tested with different compilers
  - Is compatible with multiple operating systems
  - Has support for multiple storage engines; both transactional and non-transactional
  - Has Application Programming Interfaces (APIs) for accessing MySQL databases available in many languages, including C, C++, Eiffel, Java, Perl, PHP, Python, Ruby, and Tcl
  - Is using multiple kernel threads or processing units, if available, for data processing
  - Is using multiple processors where available, boosting performance
  - Has ability to divert memory resources from inactive threads to active threads for faster processing
  - Has commands and features to retrieve, update, and delete data from several tables
  - Has support for compatibility to be used as a separate application or as an embedded library

- Column Types:
  - Includes multiple column or data types including numeric, date and time, and string
  - Includes mapping of data types from other databases to MySQL data types
  - Includes fixed-length and variable-length strings
- Commands and Functions:
  - Has support for all MySQL operators and functions in the `SELECT` statement and the `WHERE` clause
  - Has support for tables from different databases in one statement
  - Has support for table and column aliases
  - Has support for displaying information about databases, tables, and indexes using the `SHOW` command
  - Has support for displaying query resolution information using the `EXPLAIN` command
  - Has full support for `SQL GROUP BY` and `ORDER BY` clauses, group functions and left and right outer joins
  - Has support for the use of function names as table or column name
- Security:
  - Has support for in-built data encryption and decryption
  - Has support for user account privileges
  - Has support for password encryption
- Scalability and Limits:
  - Handles large databases that have upto 5 billion rows
  - Allows upto 64 indexes per table
  - Allows upto 16 keys per table

- Connectivity:
  - Supports connectivity on any platform to MySQL server using Transmission Control Protocol/Internet Protocol (TCP/IP) sockets
  - Supports connectivity on Windows NT, 2000, XP, 2003, and Vista using named pipes or shared-memory connections
  - Supports connectivity on UNIX systems using UNIX domain socket files
- Localization:
  - Displays error messages in languages, such as Czech, French, German, Japanese, Korean, Norwegian, Polish, and Russian
  - Supports Unicode and various character sets
  - Allows data to be stored, sorted, and compared using the chosen character set
- Clients and tools:
  - Provides built-in support to check, optimize, and repair tables
  - Provides "`mysql`" tool to execute individual SQL commands or SQL commands stored in a file
  - Provides "`mysqlaccess`" tool to check host, user, and database privileges
  - Provides "`mysqladmin`" tool to manage the database server
  - Provides "`mysqldump`" tool to backup the contents of one or more MySQL databases to a file
  - Provides "`mysqlhotcopy`" tool to backup a single database or table on to the same computer
  - Provides "`mysqlimport`" tool to import data into a MySQL table from a file
  - Provides "`mysqlshow`" tool to display information about the databases, tables, and columns
  - Provides "`mysqld _ safe`" tool that enables safe start up of the MySQL server

1.3 Advantages of MySQL Over Other RDBMS

There are many commercial Database Management Systems such as Oracle, Microsoft SQL Server, and Sybase available in the market. These Database Management Systems are robust, reliable, and support most of the features that a user wants. However, it is impossible for these databases to compete with MySQL with regards to price, as MySQL is available for free. In addition, for commercial Database Management Systems, the initial setup cost is more expensive, resource intensive, and time consuming, whereas with MySQL, this is not the case. This is one of the key advantages of MySQL.

Also, as the source for MySQL is fully available, you can customize MySQL as required. There are many troubleshooting techniques, command help, and syntax help that are available. This information is available in blogs, forums, and lists that do not require paid subscriptions. However, troubleshooting techniques, command help, and syntax help for commercial databases may require a paid subscription.

Typically, open source software tends to be updated more frequently than commercial software because many users contribute to its development. As a result, new features are available more often than for commercial databases.

MySQL provides different versions that work on different versions of Linux, UNIX, Microsoft, Windows, and other operating systems. MySQL also supports various built-in and third-party GUI tools for faster and easier design, implementation, and administration.

Following are the other advantages that MySQL offers over other RDBMS:

- **Reliable:** Supports tables that can store and handle large number of records.
- **Ease of Use:** Provides a modular and flexible architecture that makes it easy to manage and customize.
- **Cross Platform Support:** Supports different operating systems, such as Linux, UNIX, and Microsoft Windows.
- **Views:** Supports views where data is copied into temporary or virtual tables during processing. This feature ensures data security.
- **Stored Procedures:** Supports stored procedures and functions. This allows you to implement business logic at the database level.
- **Triggers:** Supports triggers. This feature also enables you to implement business logic during data processing.

Concepts

1.4 Comparing MySQL As An Open Source Database With Other RDBMSes

There are many open source and commercial databases available today. Some of the popular open source databases include mSQL, PostgreSQL, and InstantDB. As mentioned earlier, Oracle, MS SQL Server, and Sybase are some popular commercial databases.

The early editions of MySQL did not support all the SQL features. For example, transaction support and stored procedures were not available in the older versions of MySQL. The latest versions of MySQL, however, provide full SQL and transaction support.

The commercial databases support almost all the features that are present in MySQL, but the performance of MySQL is better.

One drawback in MySQL is that it does not support advanced SQL3 features such as object oriented data types. PostgresSQL supports advanced SQL3 features and is a better choice as an open-source DBMS. However, PostgresSQL has a major disadvantage in its hidden limit of 8K of data per row.

PostgreSQL is more powerful but MySQL is faster. MySQL does not need a vacuum procedure as PostgreSQL. Vacuum procedure refers to the process of optimizing the data stored in the database. PostgreSQL withstands higher loads. The latest versions of PostgreSQL and MySQL support features, such as sub-selects, stored procedures, triggers, unions, and views. However, older versions of MySQL did not support all these features. In addition, because these features are new to MySQL, there are some performance issues.

However, MySQL provides more user-friendly command interface so it is popular among Web developers. Also, MySQL supports more data types and functions as compared to mSQL.

InstantDB competes well with MySQL when you consider the different features. The only feature of MySQL that InstantDB is unable to compete with is performance. MySQL is faster as compared to InstantDB.

1.5 Advantages of PHP In MySQL Environment

A scripting tool enables you to control one or more applications when executed. PHP is a scripting tool designed for Web development. PHP supports embedding scripts into HTML code. Developers can use PHP scripts to create HTML Web pages that can read and write data from a database.

PHP is a scripting language that is executed at run-time. It enables interaction of the application with the database. You can use PHP and MySQL together to manage data on the Web. PHP is compatible with MySQL. You can also store and manage information from the database. Figure 1.1 displays the interaction between the client, server, and database.

Session 1

Introduction to MySQL as an Open Source Database

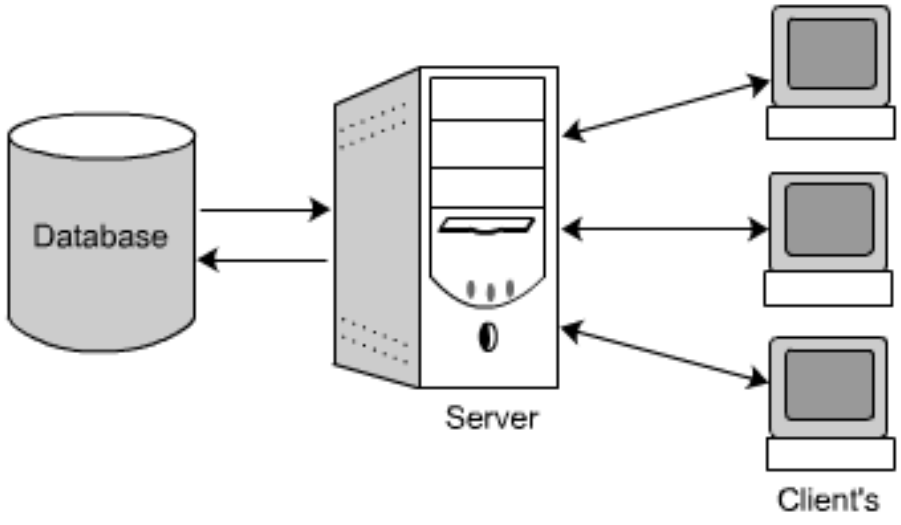


Figure 1.1: Interaction between Client, Server, and the Database

Consider a database connected to a server as shown in figure 1.1. The server is connected to several clients. The database, server, and client maintain a two-way communication. This system appears to be simple; however, it has certain limitations. For example, when a client requests for data, the browser sends a request to the server. The server locates the data from the database, and returns it to the browser. If several clients make a request to the server for the same data, then the server will return the data to all the clients. This results in slow performance of the system.

The advantage of using PHP is that the database can be accessed directly through a Web page. In this scenario, the client will request for a PHP file. The PHP preprocessor will connect to the database, retrieve the data, convert the data into HTML format, and send it to the browser.

Following are some examples of real-world Websites where databases are used:

- **Online Ticket Reservation:** In an online reservation system you can reserve a seat using the Internet. Your action updates the backend or the database of this booking system. You can access several parts of the database by changing the Uniform Resource Locator (URL).

- **Message Boards:** On the Internet, message boards are widely present that run on MySQL and PHP. It is an online discussion site where messages can be posted. Message boards running on PHP and MySQL are more efficient because you need to update only one page and the changes are automatically reflected in others.

- **Marketing Websites:** Consider that a large Website is required to be updated. The Website can be updated using few PHP scripts as the information related to these pages is stored in MySQL database. The PHP scripts accesses the MySQL database to obtain the information about the pages.

- **Advertising Banners:** Consider a Website where several advertisement banners are present on the site. These banners are stored in a database on the server. You can call a PHP script to display each banner. In order to insert, modify, or delete a banner you have to access the database. A PHP script can be written to select the database from which the banner will be displayed. The PHP script would select and display the correct banners for the pages on the site.

To run a PHP script, you will need to install the following software:

- Web server
- PHP
- MySQL

PHP supports different operating systems such as Mac OS X, Linux, UNIX, and Windows.

Both PHP and MySQL are open source. This feature makes them cost effective as compared to other software products.

MySQL also supports the command line interface. This interface enables the PHP page to access the database and display the query results.

You can use PHP scripts to control the administration activities of the database. However, it is better to install a PHPMyAdmin on the server. PHPMyAdmin is an administrative interface for MySQL databases. It consists of a set of free scripts for administration of the database.

1.6 Open Source Software Licensing

There are many types of software licenses available. An open-source software license permits users to read, access, change, and reuse the source code of a software product. Open-source software does not necessarily mean free software.

The advantage of using open-source software is that the developer can customize as per requirements. There is no limit on the customization. Also, there are various troubleshooting and performance tips freely available on the Internet for open-source software. However, open-source license does not allow you to sell customized software. In addition, you will have to provide the modified source code when you distribute the software.

However, open-source software must comply with certain conditions and some of them are as follows:

- The source code must be distributed along with the binary.
- The software can be modified as required and redistributed under the same terms as the original software.

- When the software is used as a part of development of other applications, the software must be redistributed with the application without any royalty or fee.

Note: For a detailed description of the open-source license terms and conditions, visit: `http://www.opensource.org/docs/osd`

1.6.1 Licensing of MySQL

MySQL is available under two licensing schemes. They are:

- **General Public License (GPL)** – Applies to developers who use, and/or distribute open source software under the GPL.

- **Commercial License** – Applies to developers who only use MySQL to develop their own executables and not the source code.

Summary

- A database stores data in a structured format. A Database Management System is responsible to store, access, and delete data from a database.
- MySQL is an open source RDBMS. It was developed and distributed by MySQL AB, which is now owned by Oracle.
- There are many commercial Relational Database Management Systems available, such as Oracle, Microsoft SQL, and Sybase that support most of the data management features.
- MySQL works on different operating systems, such as Mac OS X, Linux, UNIX, and Windows.
- Unlike in traditional Database Management Systems that are proprietary, in open source software, you can modify the source code to customize the features.
- PHP is a scripting language that enables interaction with a database. You can use PHP and MySQL to store and manage data on the Web.
- In order to run a PHP script, you will need to install a Web server, PHP and MySQL.
- Open-source software licenses allow you to read, access, change, and reuse the source code of a software product.
- MySQL is available under General Public License and Commercial License.

Check Your Progress

1. The `mysqlshow` command displays information about the _____.

   a. Databases and tables on the server

   b. Tables and columns in a database

   c. Columns and rows in a table

   d. Version of MySQL Server

2. _____ is an administrative interface for MySQL databases.

   a. mysqladmin

   b. PHPMyAdmin

   c. admin

   d. SQL

3. To execute a PHP script, you will need to install _____.

   a. Web Server, PHP, MySQL

   b. PHP, MySQL

   c. MySQL, Web Server

   d. PHP, Web Server

4. MySQL allows upto _____ indexes per table.

   a. 16

   b. 32

   c. 46

   d. 64

Check Your Progress

5. _____ tool manages users of MySQL.

a. mysqladmin

b. mysqlaccess

c. mysql

d. mysqlshow

Session 2

Installing and Configuring MySQL

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain the various distribution options of MySQL.*
- *Explain the installation process of MySQL on Microsoft Windows.*
- *Explain the installation process of MySQL on Red Hat Enterprise Linux.*
- *Explain the configuration process of MySQL using Scripts.*
- *Explain initialization of MySQL at startup.*

2.1 Introduction

MySQL is an Open Source software that enables you to use and modify the source code. It is available in binary and source distribution formats. You can use MySQL on different operating systems such as Microsoft Windows, Linux, or UNIX. It is easy to install, maintain, and configure.

In this session, you will learn to install MySQL on different platforms and initialize MySQL at the startup. You will also learn to configure MySQL installation using scripts and my.cnf configuration file.

2.2 Evaluating the Various Distribution Options of MySQL

You can install MySQL using either the binary distribution or the source distribution. The binary distribution contains a setup program that performs the installation. If you use the source distribution that contains the source code of MySQL, you will have to compile the code before installation.

Normally, you use the binary distribution for installation. This is because the binary distribution is a pre-compiled, ready-to-install distribution. Binary distribution is available in different formats that are compatible with a specific operating system. For example, RPM files or compressed ZIP or tar files are available for installing MySQL on Linux. Thus, binary installations are fast and easy to install.

The installation, using the binary distribution under Linux or UNIX, creates the following directories, as shown in figure 2.1.

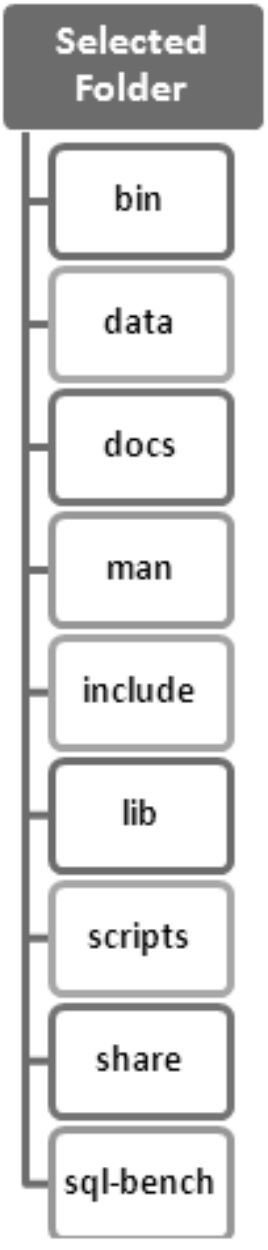


Figure 2.1: Default Directories under Source Directory for a Linux/Unix Binary Installation

Table 2.1 lists the default folders under source directory and its contents.

Directory	Content
bin	Contains client utilities and mysqld daemon
data	Contains databases and log files
docs	Contains documentation in the Info format

Directory	Content
man	Contains UNIX manual pages
include	Contains header files
lib	Contains library files
scripts	Contains mysql_install_db file
share	Contains support files such as error messages
sql-bench	Contains benchmarks of MySQL database

Table 2.1: Default Directories Under Source Directories

The only disadvantage of a binary distribution is that customization in terms of directory locations, and so forth is not possible. If you need a custom MySQL installation, then you may prefer to use the source distribution of MySQL.

You can use source distribution to create a distribution:

- Having customized directory structure
- Having a custom set of features by enabling or disabling features
- From the latest development source (binaries are available only for release candidates)

If you do not customize the installation folder structure, the distribution generated by compiling the source uses the same folder structure as the binary distribution.

2.3 Installing and Configuring MySQL on Different Platforms

You must ensure that the platform or the Operating System (OS) will support MySQL before starting the installation process of MySQL. The OS or platforms that support MySQL are as follows:

- Solaris
- Linux 6 or higher
- Microsoft Windows 7/8 or higher
- Microsoft Windows Server 2003/2008
- Mac OS X or higher
- FreeBSD
- AIX

Concepts

Note: Visit `http://www.mysql.com/support/supportedplatforms/database.html` for a detailed list of supported platforms.

2.3.1 Installing and Configuring MySQL on Microsoft Windows

The Microsoft Windows installer files are required to install MySQL on Microsoft Windows platform. You will now see the procedure to install MySQL 5.1 on Windows. Later, in Session 18, you will explore MySQL 5.7 and the steps to migrate from MySQL 5.1 to MySQL 5.7.

Following are the steps to install MySQL 5.1:

1. Browse and locate the downloaded installation package, and double click the file. The **Open File Security Warning** dialog box is displayed.

2. Click **Run**. The Windows Installer prepares the installation process and **MySQL Server 5.1- Setup Wizard** dialog box is displayed, as shown in figure 2.2.

Figure 2.2: MySQL Server 5.1 - Setup Wizard

3. Click **Next** to display the **License Agreement** pane in the **MySQL Server 5.1 – Setup Wizard** dialog box, as shown in figure 2.3.

Figure 2.3: End User License Agreement Dialog Box

4. Select **'I accept the terms in the license agreement'**.

5. Click **Next** to display the **Setup Type** pane in the **MySQL Server 5.1 – Setup Wizard** dialog box, as shown in figure 2.4.

Figure 2.4: Choose Setup Type Dialog Box

6. Click **Typical**.

7. Click **Next** to display the **Ready to Install the Program** pane in the **MySQL Server 5.1 – Setup Wizard** dialog box, as shown in figure 2.5.

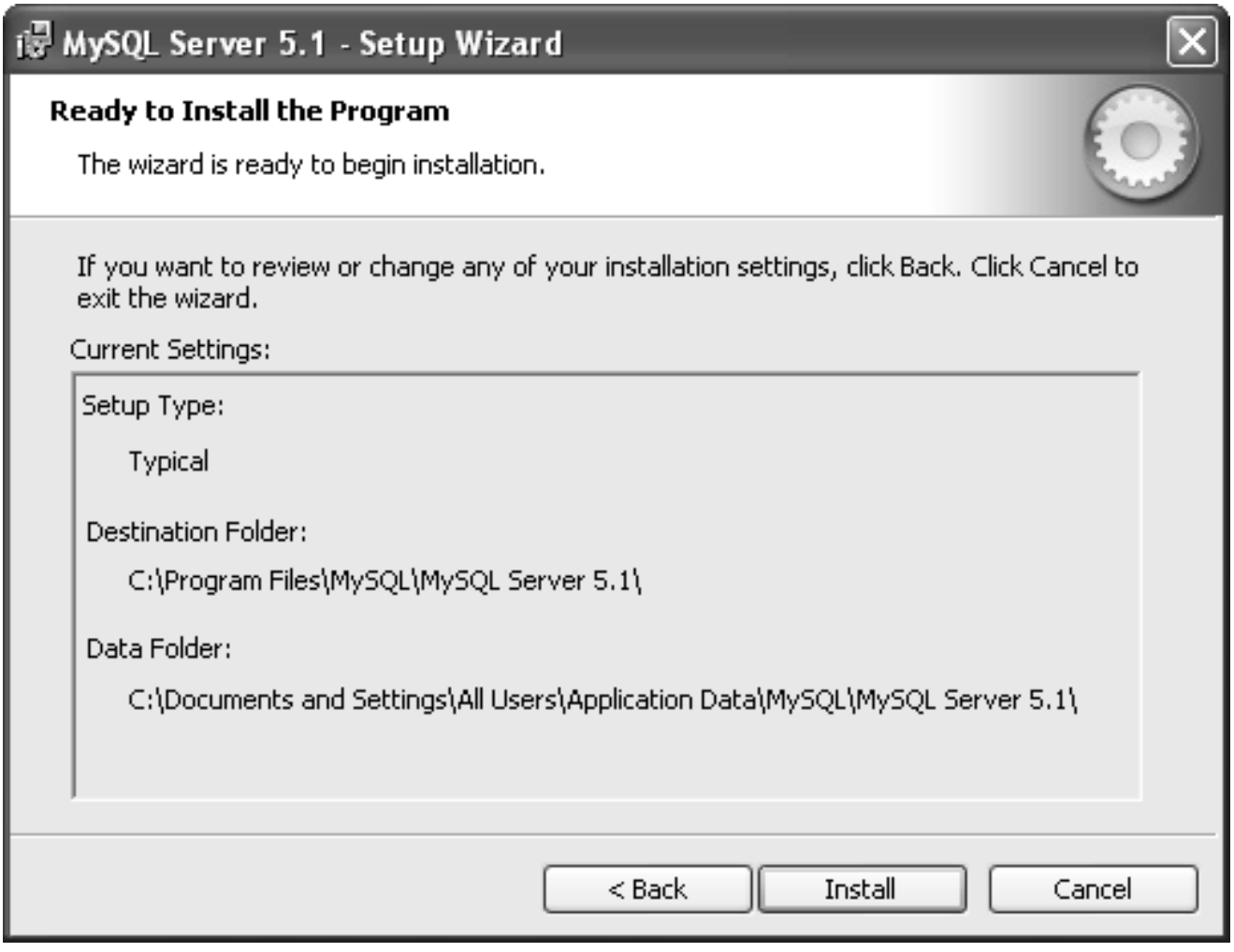


Figure 2.5: Ready to Install the Program Dialog Box

8. Click **Install**. The Setup Wizard completes the installation process and the **MySQL Enterprise** dialog box is displayed, as shown in figure 2.6.

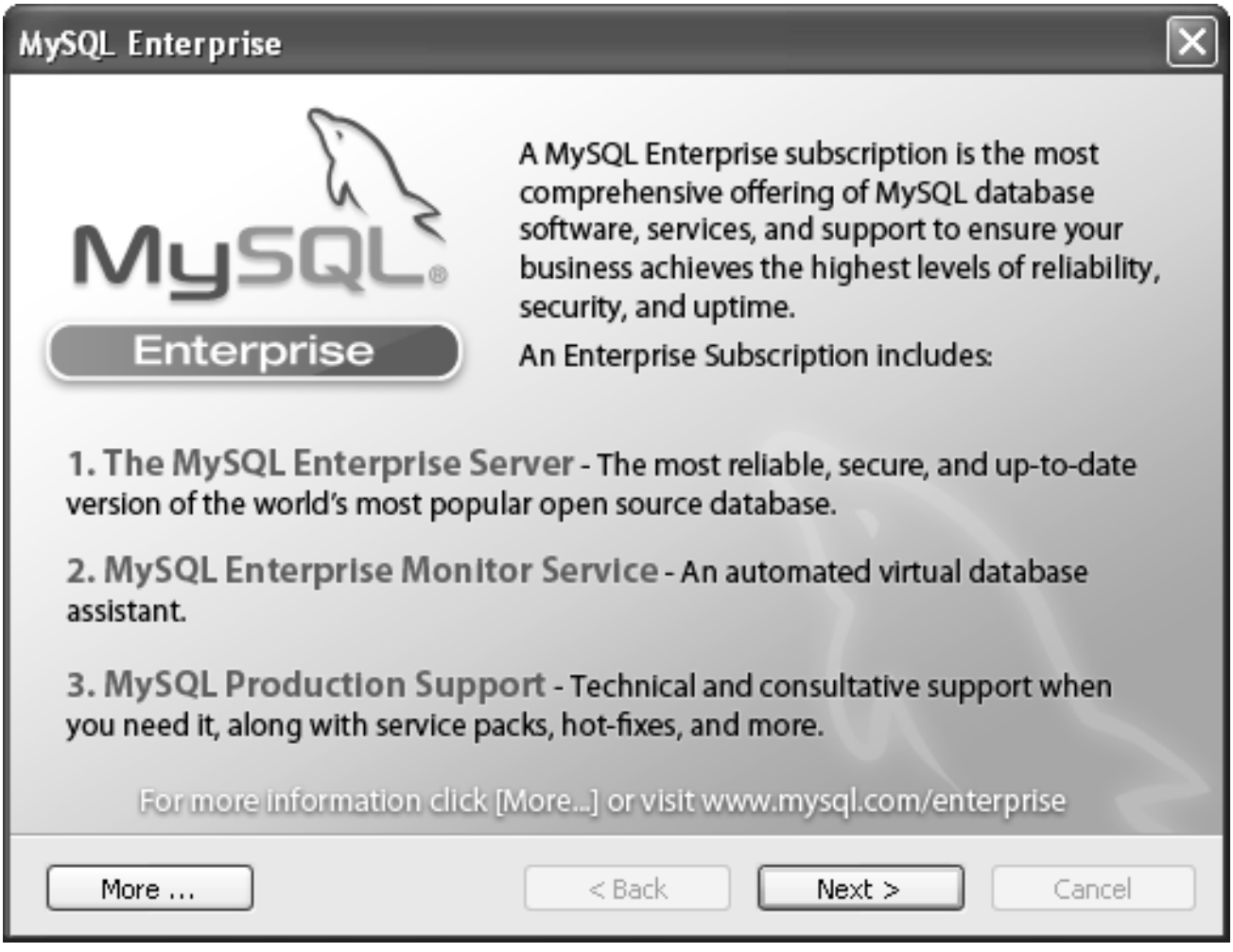


Figure 2.6: MySQL Enterprise Dialog Box

9. Click **Next**. The **MySQL Enterprise Monitor Service** pane of the **MySQL Enterprise** dialog box is displayed, as shown in figure 2.7.

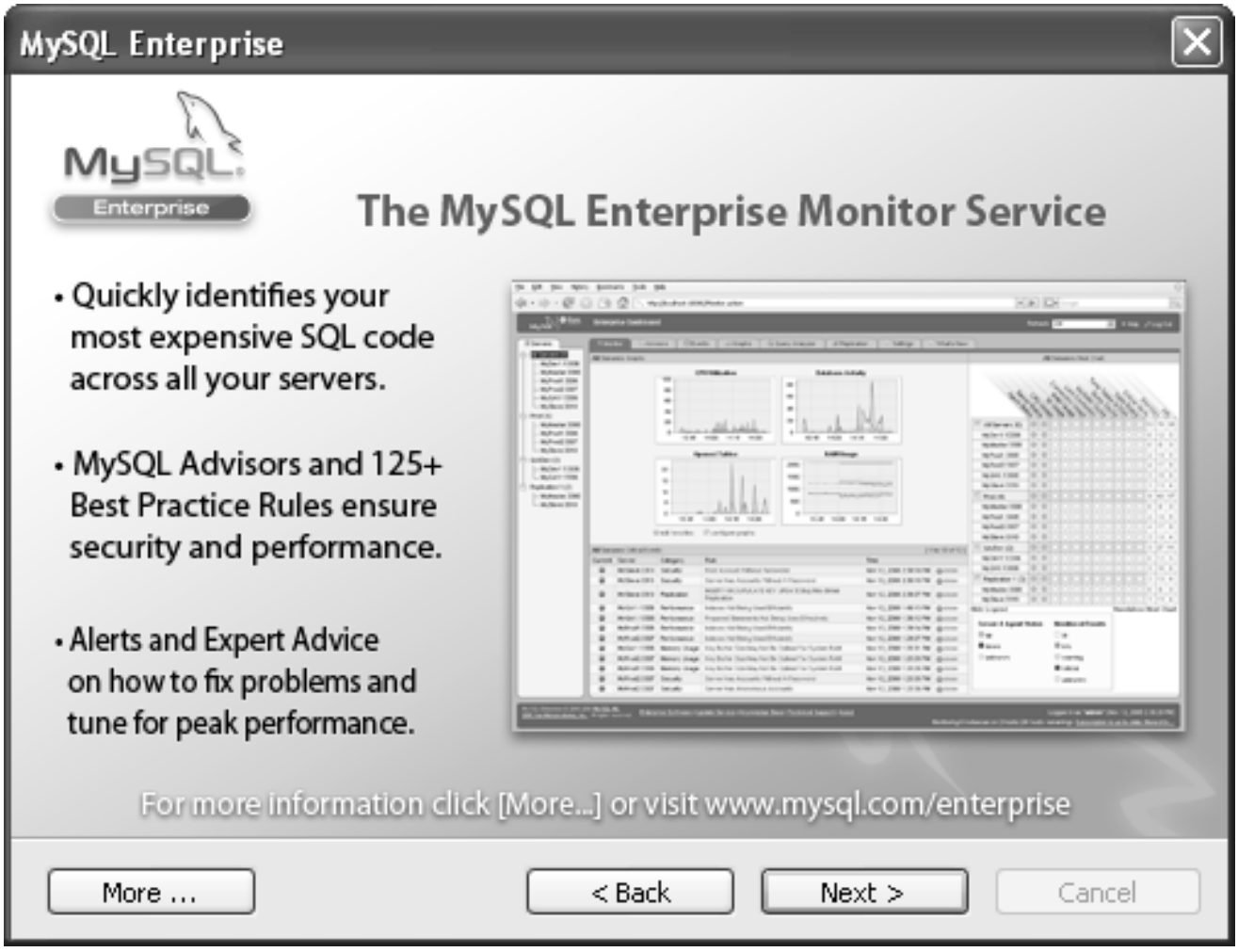


Figure 2.7: MySQL Enterprise Monitor Service Dialog Box

10. Click **Next**. The **MySQL Server 5.1 - Setup Wizard** dialog box displays the completion of the installation process by displaying the **Wizard Completed** pane, as shown in figure 2.8.

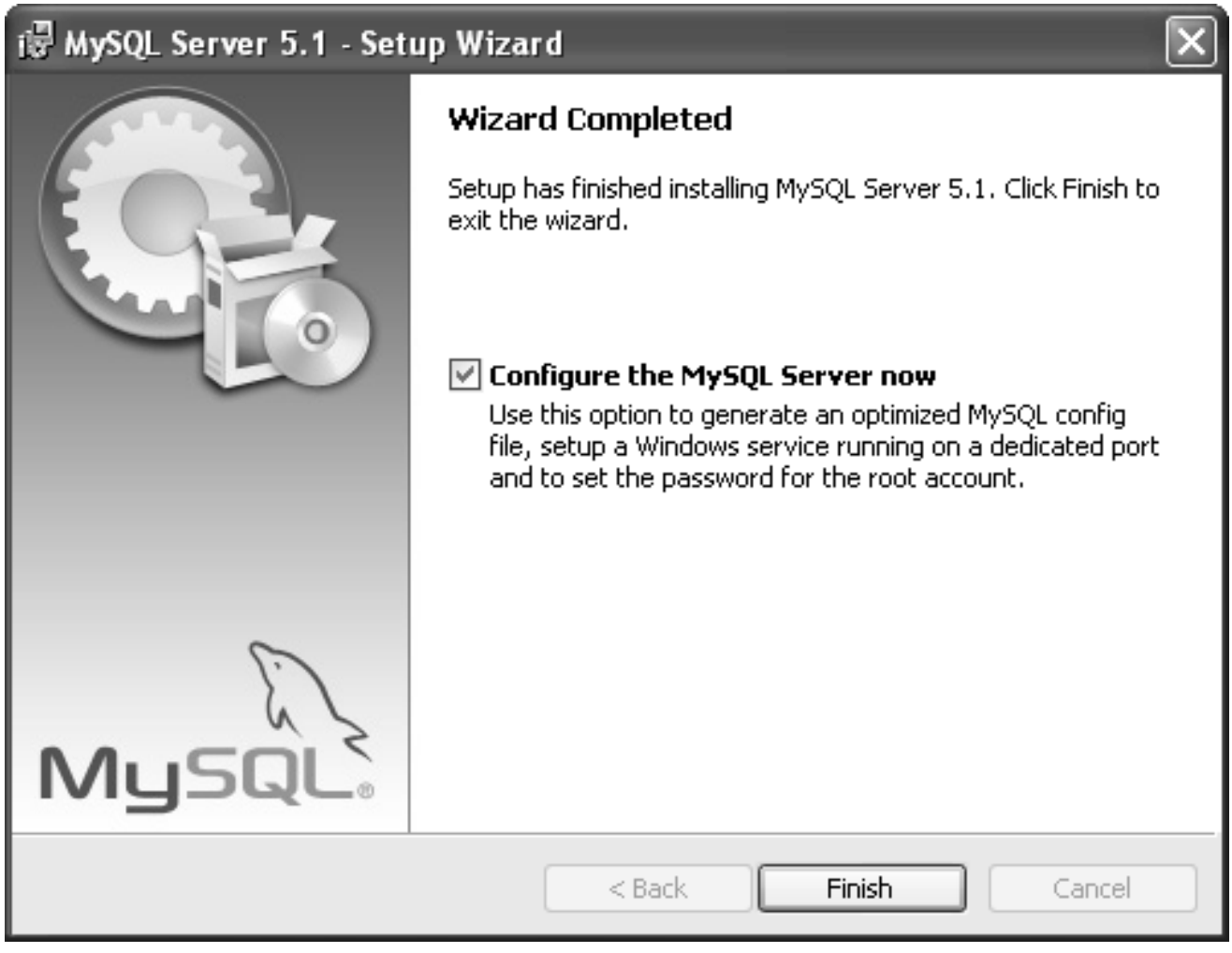


Figure 2.8: Wizard Completed Dialog Box

11. Click **Finish**. The **MySQL Server Instance Configuration Wizard** pane is displayed, as shown in figure 2.9.

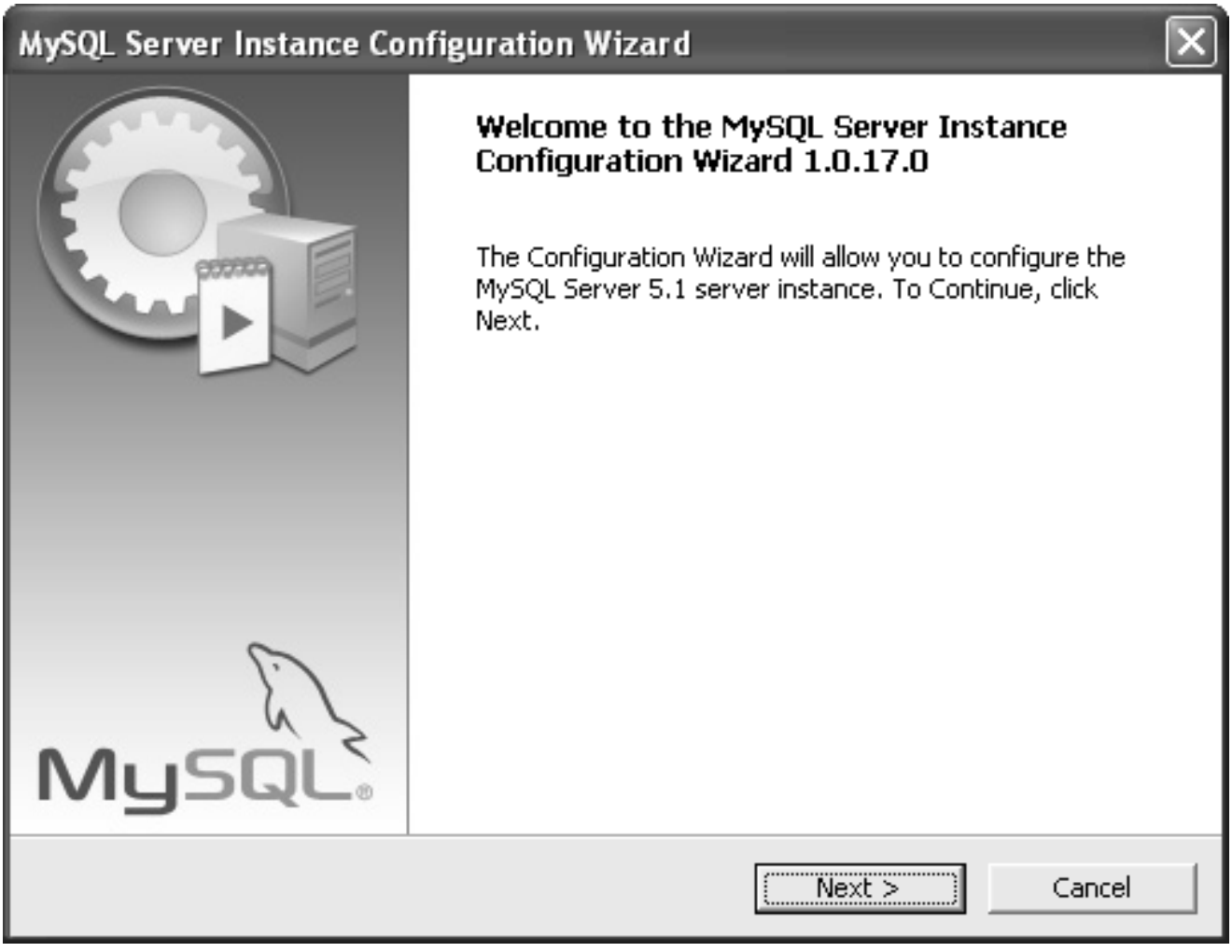


Figure 2.9: MySQL Server Instance Configuration Wizard

12. Click **Next** to display the **MySQL Server Instance Configuration** pane of the **MySQL Server Instance Configuration Wizard** dialog box, as shown in figure 2.10.

Figure 2.10: MySQL Server Instance Configuration Wizard - Configuration Type

13. Click **Standard Configuration**.

14. Click **Next** to display the **Windows Options** pane of the **MySQL Server Instance Configuration Wizard**, as shown in figure 2.11.

Figure 2.11: Windows Options Dialog Box

Options available include:

- **Install As Windows Service** – starts MySQL server at system startup
- **Launch the MySQL Server automatically** – starts MySQL with Windows
- **Include Bin Directory in Windows PATH** – copies the server and client executables in the Windows PATH variable. These variables can be invoked from the command line

15. Select the **Install As Windows Service** check box.

16. Select the **Include Bin Directory in Windows PATH** check box.

17. Click **Next**. A dialog box appears displaying the different security options, as shown in figure 2.12.

Figure 2.12: Security Options Dialog Box

The different options available include:

- **Modify Security Settings** – changes the default security settings
- **New root password** – accepts password for the root user
- **Confirm** – accepts the password for the root user
- **Enable root access from remote machines** – allows root users to connect to MySQL from other computers
- **Create An Anonymous Account** – specifies to generate a hidden login account

18. Clear the **Modify Security Settings** check box.

Note: You can assign a password for the root account by entering a password in the text boxes.

19. Click **Next** to display configuration tasks in the **MySQL Server Instance Configuration Wizard** as shown in figure 2.13.

Figure 2.13: MySQL Server Instance Configuration Wizard Displaying Configuration

20. Click **Execute**. The wizard configures the settings and a confirmation message box is displayed as shown in figure 2.14.

Figure 2.14: Confirmation Message Box

21. Click **Finish**.

The following steps helps to start MySQL:

1. Start the `MS-DOS` command prompt.
2. Change the directory to `C:\Program Files\MySQL\MySQL Server 5.1` folder.
3. Enter the following command at the command prompt:

```
mysql
```

MySQL starts and displays the `'mysql'` prompt, as shown in figure 2.15.

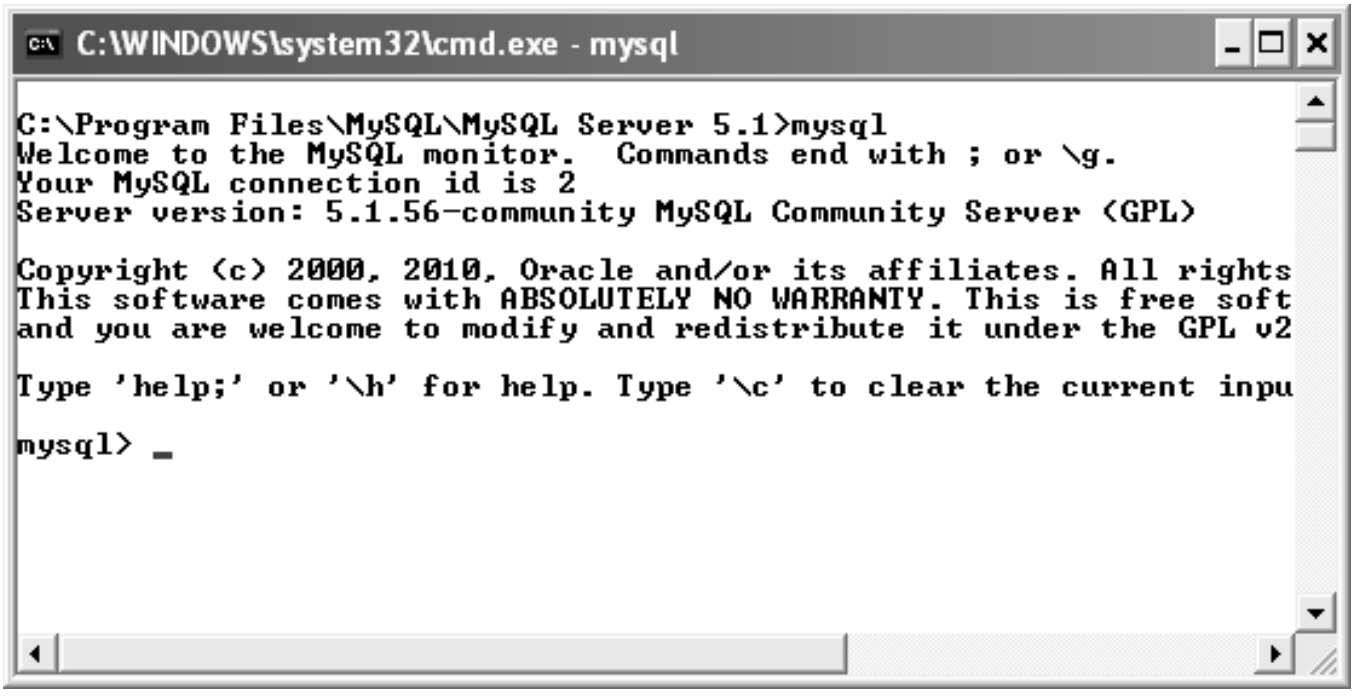


Figure 2.15: MySQL Prompt

4. To view the default databases after installation, enter the following command at the prompt:

```
SHOW DATABASES;
```

Figure 2.16 displays all the databases present on the server.

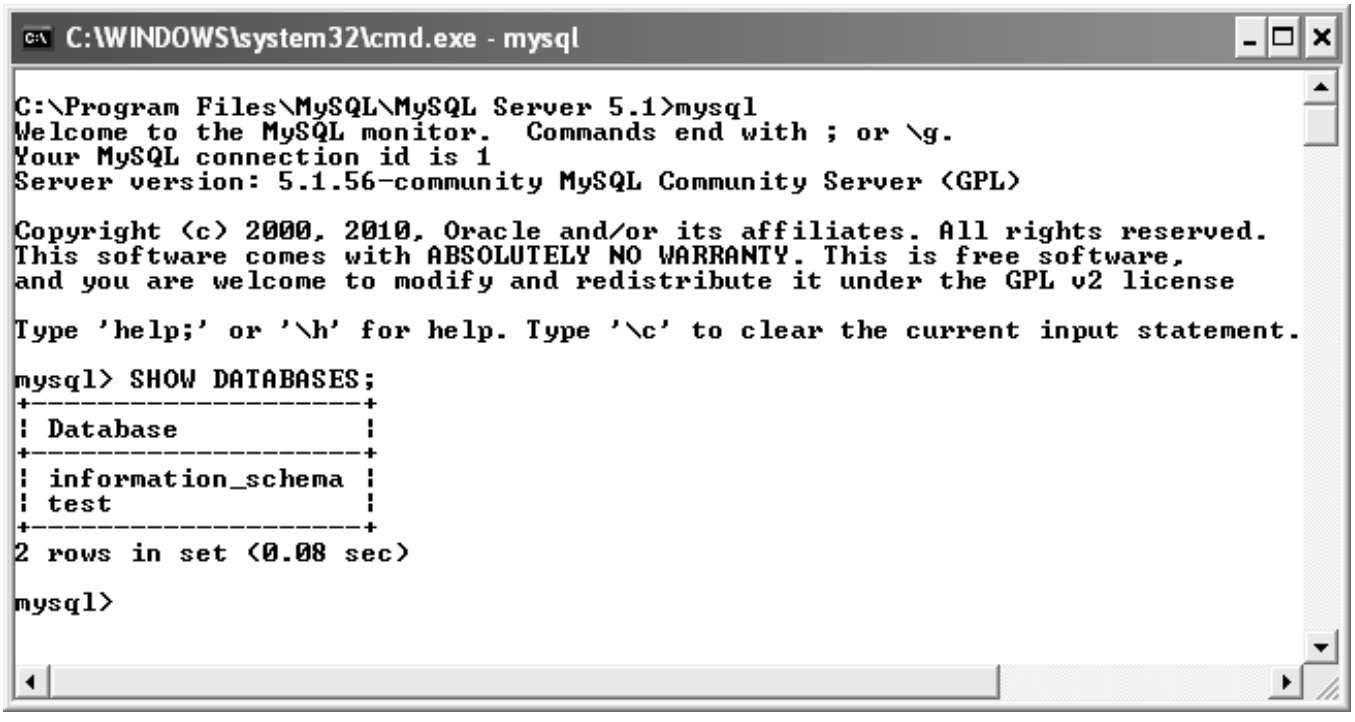


Figure 2.16: Default Databases After MySQL Installation

2.3.2 Installing MySQL on Red Hat Enterprise Linux

You will use the RPM packages to install MySQL on Red Hat Enterprise Linux. The different RPM packages required are as follows:

```
MySQL-client-community-5.1.56-1.rhel5.i386.rpm
MySQL-server-community-5.1.56-1.rhel5.i386.rpm
```

These packages can be downloaded from the Downloads section of the MySQL Website. You must select **Red Hat and Oracle Enterprise Linux** to view the list of packages that can be used to install MySQL.

You must check the existence of previous versions of MySQL before installation. If a previous version of MySQL is already present in the system, you will need to first uninstall the previous version and then install the new version.

Note: You must have root privileges to install MySQL. You can also login as root to execute the installation of MySQL.

The client and server RPM packages downloaded from the MySQL server Website will be executed to install MySQL server.

1. To execute the server RPMs, double click the **MySQL-server-community-5.1.56-1.rhel5.i386.rpm** package. The installation prompts for confirmation, as shown in figure 2.17.

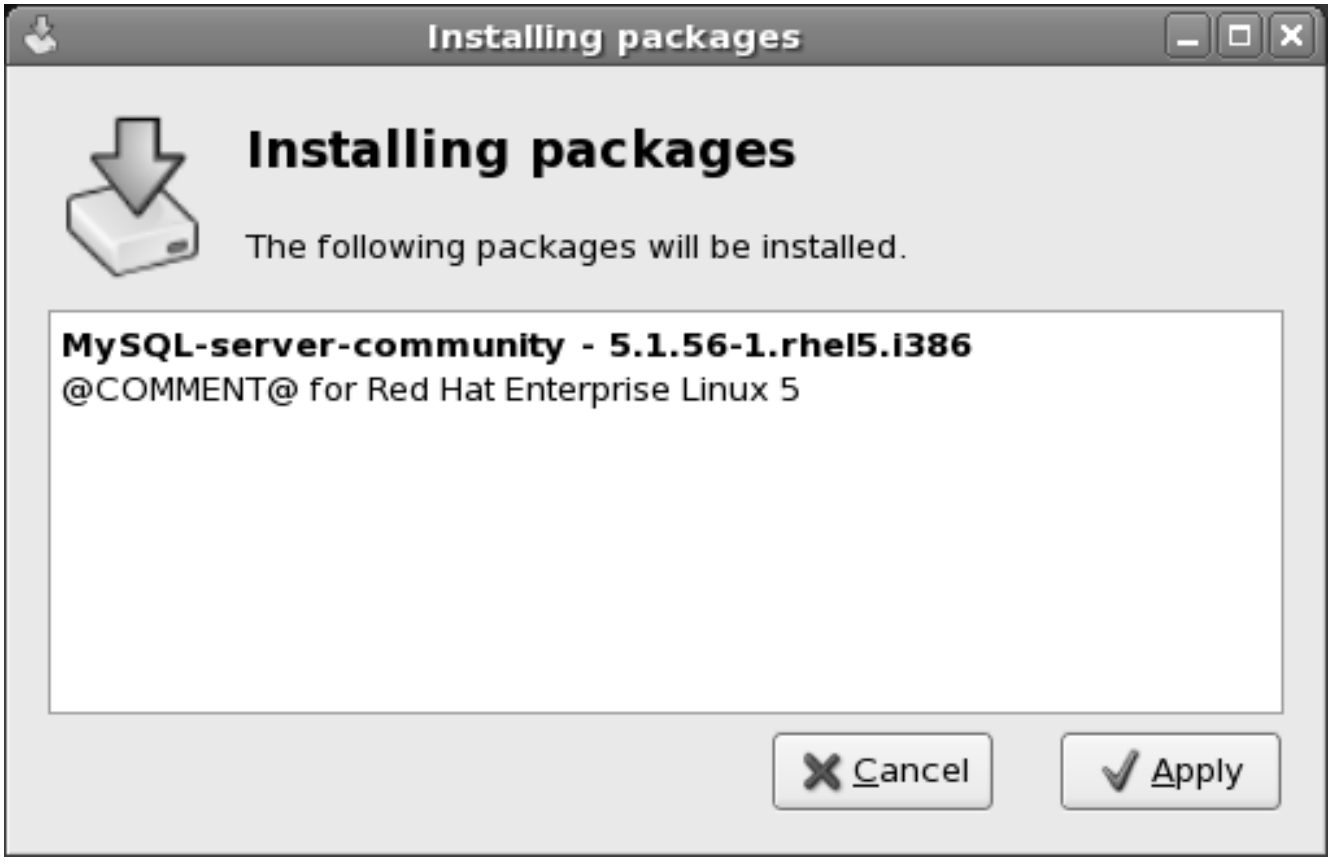


Figure 2.17: MySQL-Server-RPM Installation

2. Click **Apply**. The installation process completes and a confirmation message is displayed.

3. To execute the client RPMs, double click the **MySQL-client-community - 5.1.56-1.rhel5.i386.rpm** package. The installation prompts for confirmation, as shown in figure 2.18.

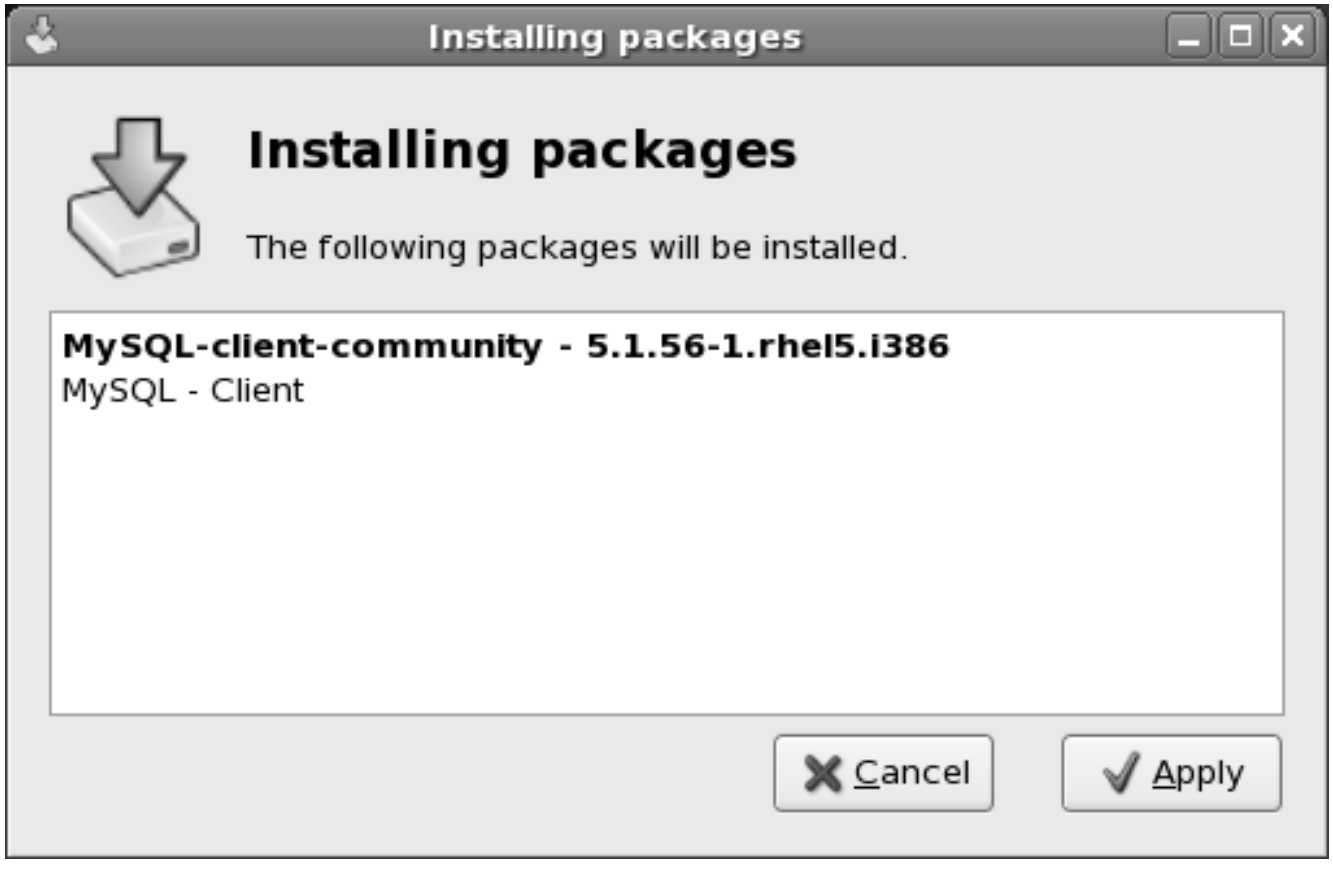


Figure 2.18: MySQL Client RPM Installation

4. Click **Apply**. The installation process completes and a confirmation message is displayed.

5. Start the terminal in Linux.

6. To start the MySQL service, enter the following command at the command prompt:

```
/etc/init.d/mysql start
```

Note: This command also adds MySQL service to the startup. It means that MySQL service automatically starts during system boot.

Figure 2.19 displays the output of the command.

Concepts

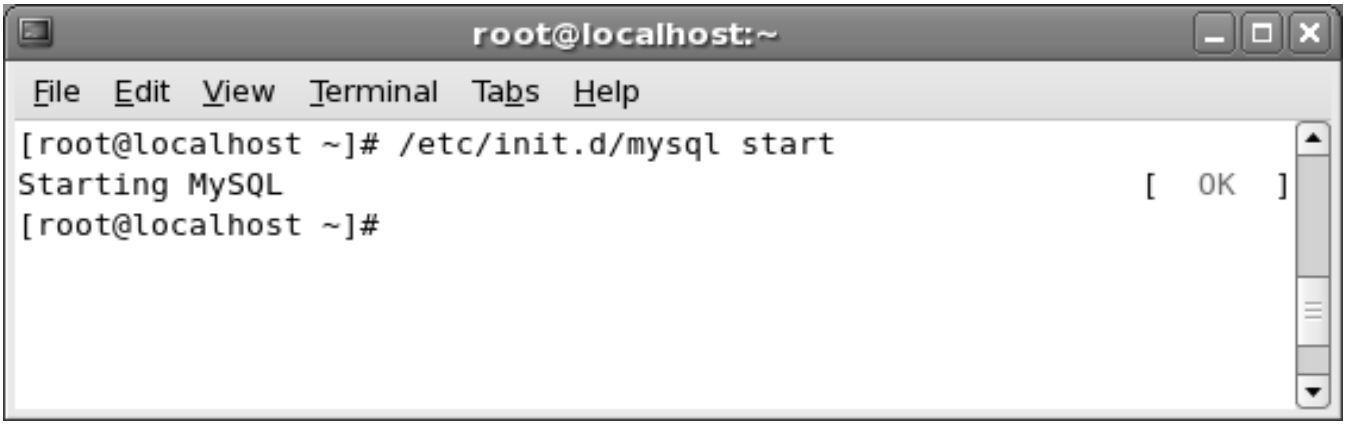


Figure 2.19: Initializing and Starting MySQL Service

7. To start MySQL, enter the following command at the command prompt:

```
mysql –u root
```

Figure 2.20 displays the output of the command.

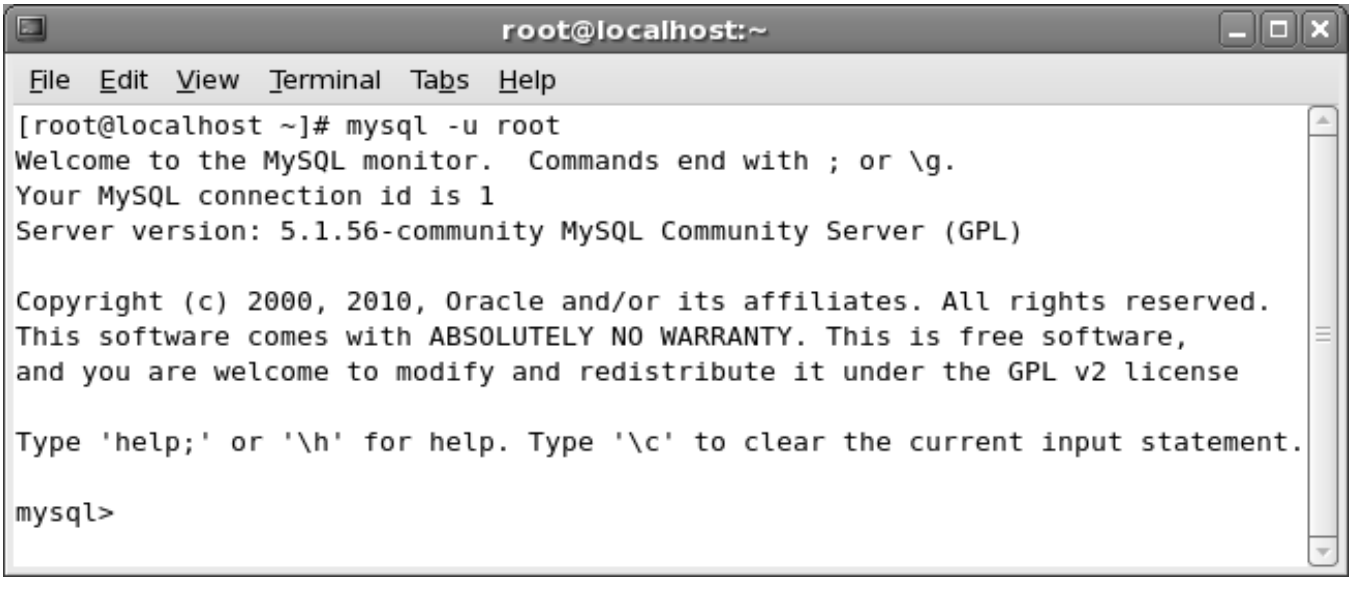


Figure 2.20: MySQL Prompt - Startup Screen

Note: If you login to Linux as the root user, you can connect to the MySQL prompt by entering the following command at the command prompt:

```
mysql
```

To view the list of default databases present on the server after installation, enter the following command at the command prompt:

```
SHOW DATABASES;
```

Figure 2.21 displays the output of the command.

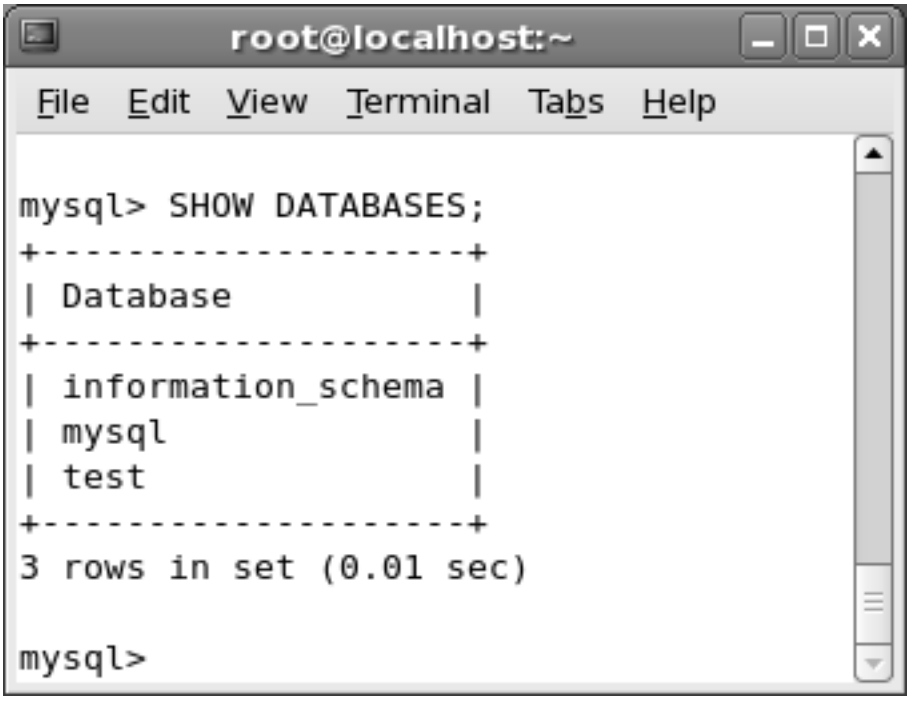


Figure 2.21: SHOW DATABASES Command

2.4 Using Scripts to Customize the Configuration of MySQL

The configure script enables you to control the MySQL distribution. You can build MySQL by using the various options of configure on the command line. You can configure MySQL using the combination of command-line options, configuration files, and environment variables.

The environment variables used by MySQL are listed in table 2.2.

Variable	Description
CXX	specifies the name of C++ compiler
CC	specifies the name of C compiler
CFLAGS	specifies instructions for the C compiler
CXXFLAGS	specifies instructions for the C++ compiler
DBI_USER	specifies the default user name for Perl DBI

Variable	Description
DBI_TRACE	specifies trace options for Perl DBI
HOME	specifies the default path for the MySQL history file. The default path is $HOME/.mysql_history
LD_RUN_PATH	specifies the directories to be searched by the dynamic linker
MYSQL_DEBUG	enables debugging options
MYSQL_GROUP_SUFFIX	specifies to read groups with a suffix
MYSQL_HISTFILE	specifies the default path for the MySQL history file. The default path is $HOME/.mysql_history
MYSQL_HOME	specifies the path of the directory where my.cnf file is located
MYSQL_HOST	specifies the default host name for the command line client
MYSQL_PS1	specifies the command prompt for the command line client
MYSQL_PWD	specifies the default password while connecting to MySQL
MYSQL_TCP_PORT	specifies the default TCP/IP port number
MYSQL_UNIX_PORT	specifies the default UNIX socket file name
PATH	enables the shell to search for MySQL programs
TMPDIR	specifies the directory to create temporary files
TZ	specifies the local time zone
UMASK	specifies the mode to create files
UMASK_DIR	specifies the mode to create directories
USER	specifies the user name on Windows when connecting to MySQL

Table: 2.2: Environment Variables in MySQL

The configuration options are stored in configuration files. These configuration files are available in the binary source code. You will be required to download the source code and extract the files to a folder before making changes to the configuration files. You must download the **MySQL-community-5.1.56-1.rhel5.src.rpm** package to extract the source code.

Note: The source code for MySQL is also available in RPM packages under the Source Code section of the Downloads page on the MySQL Website.

To extract the RPM package containing the source code:

1. Create a folder named **mysql** under the root directory.

2. Select **Applications → Accessories → Archive Manager**. The Archive Manager screen is displayed as shown in figure 2.22.

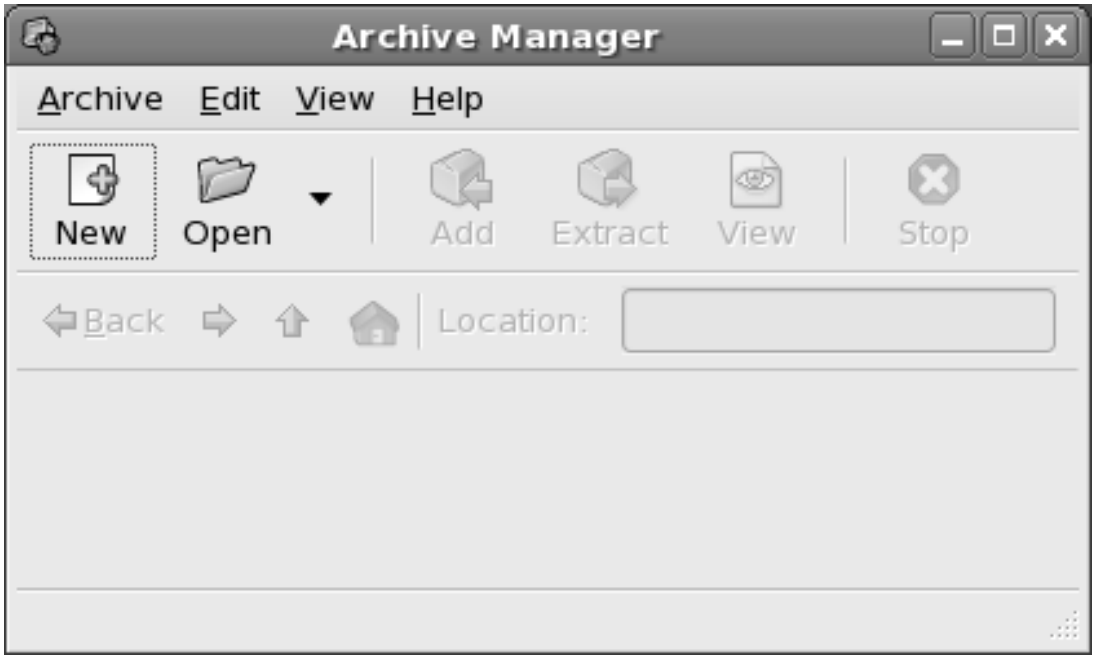


Figure 2.22: Archive Manager

3. Select **Archive** → **Open**. The Open dialog box is displayed as shown in figure 2.23.

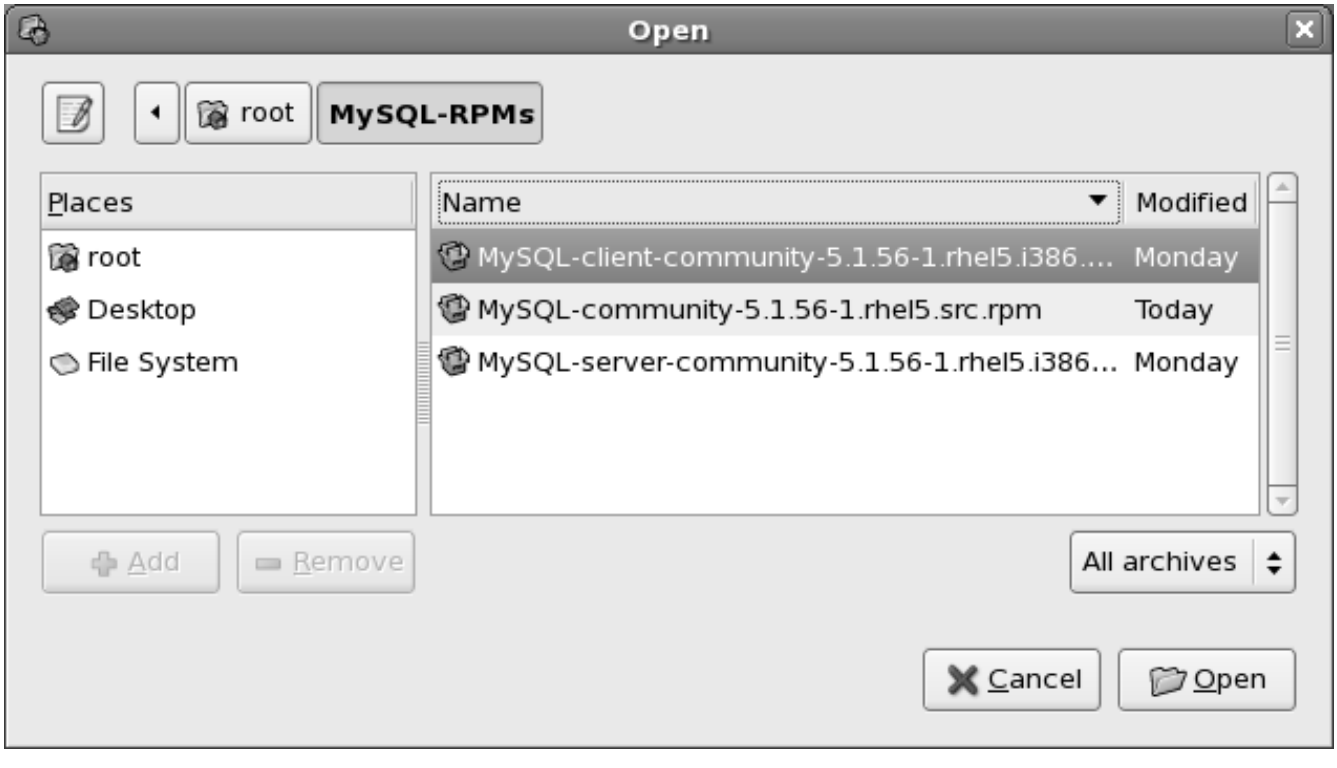


Figure 2.23: Open Dialog Box

4. Browse to the folder containing the **MySQL-community-5.1.56-1.rhel5.src.rpm** file.

5. Click **Open**. The contents of the RPM package are displayed as shown in figure 2.24.

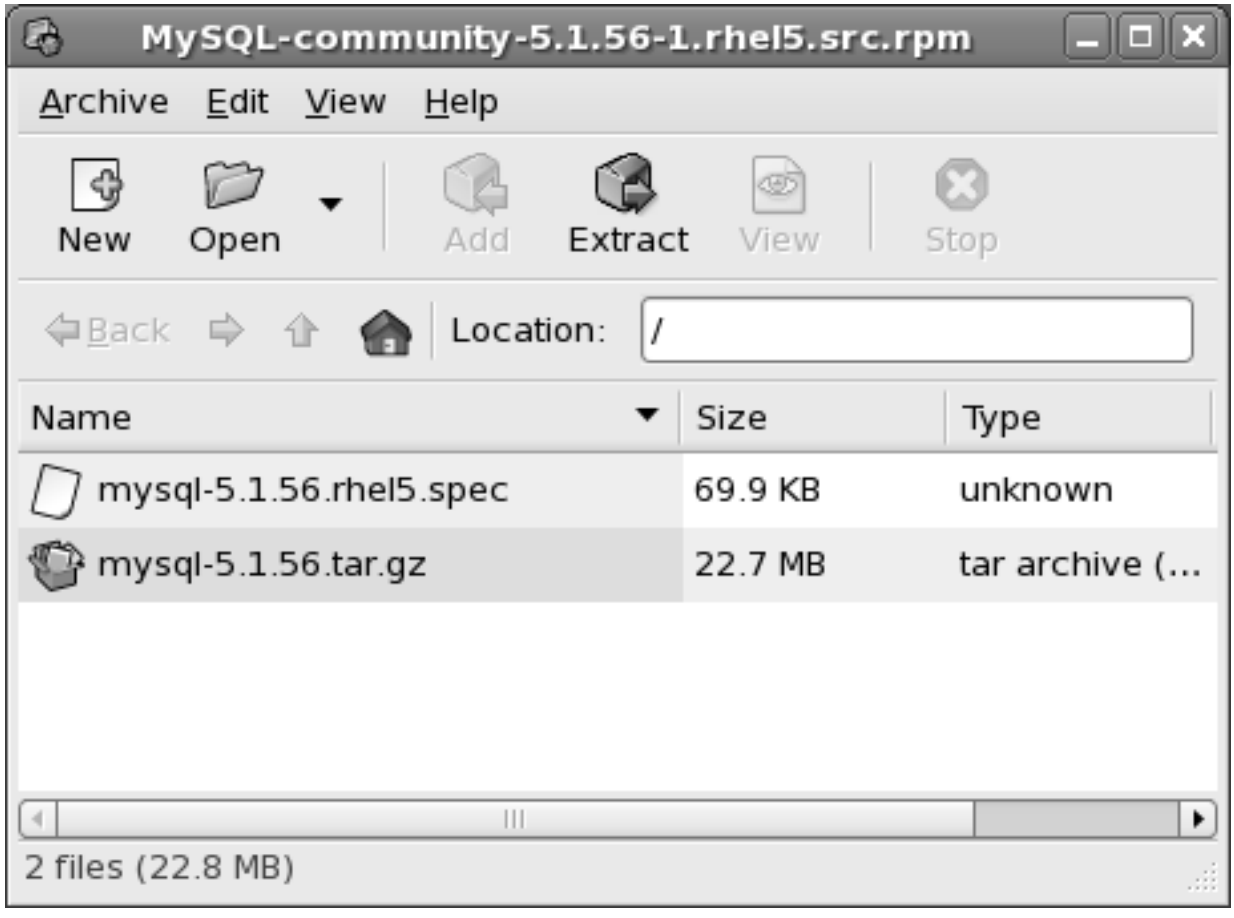


Figure 2.24: RPM Package Contents

6. Select **Archive** → **Extract**. The Extract dialog box is displayed, as shown in figure 2.25.

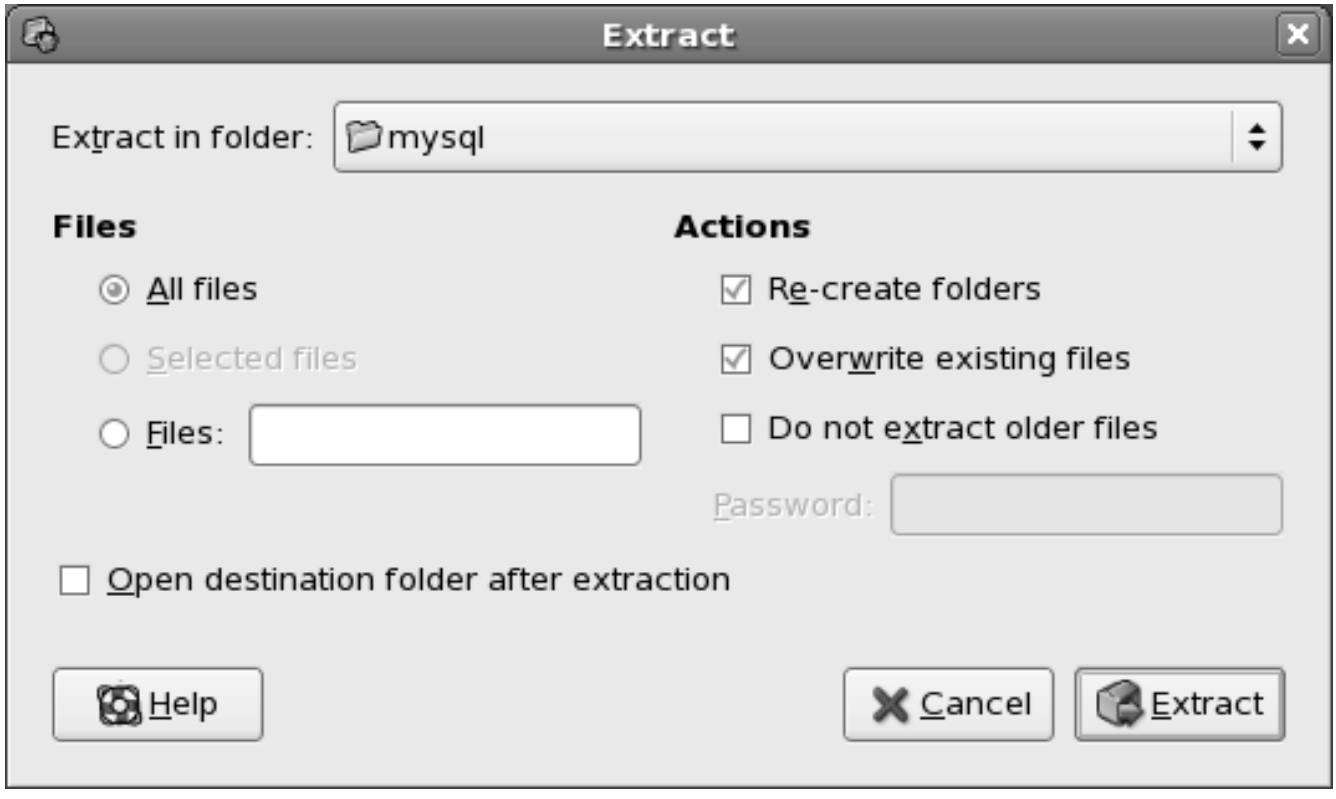


Figure 2.25: Extract Dialog Box

7. Browse to the **mysql** folder in the **Extract in folder** drop-down box.

8. Click **Extract**. The application extracts the package contents.

9. Right-click the **mysql-5.1.56.tar.gz** file.

10. Select **Extract Here** from the popup menu. The file contents are extracted as shown in figure 2.26.

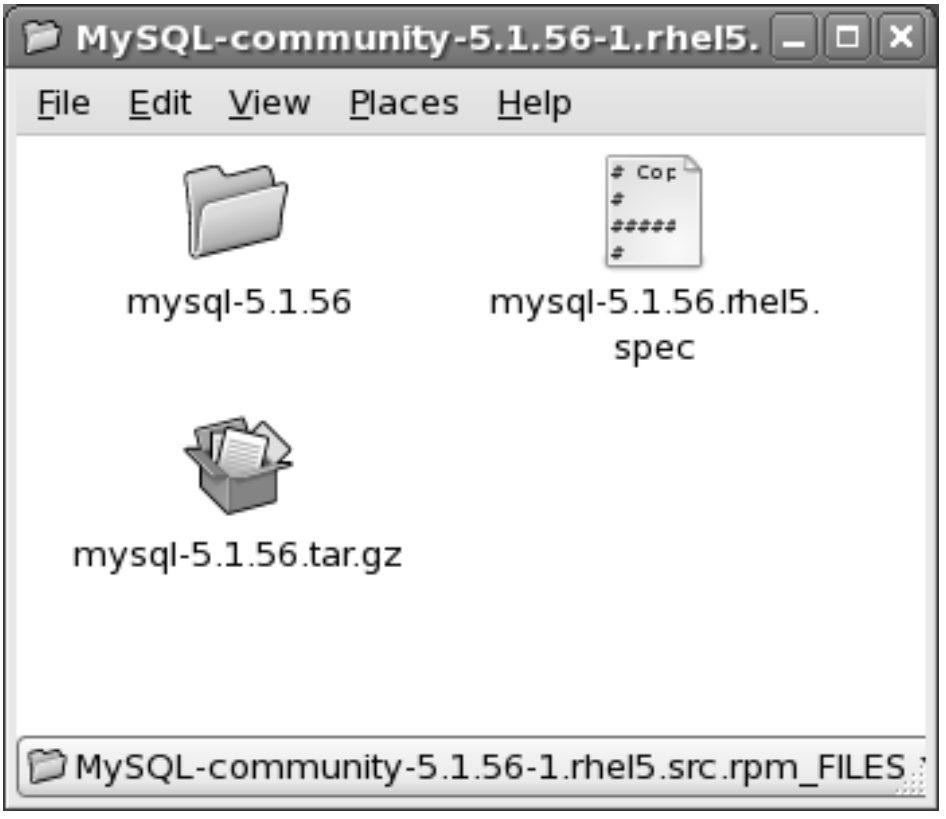


Figure 2.26: Extracted Files

After extracting the package contents, you will create a symbolic link. A symbolic link enables you to move a database directory on a specific disk. You will name the link as root and use it to start the MySQL client.

11. Right click the **mysql-5.1.56** folder and select **Open In Terminal**. The Terminal window is displayed as shown in figure 2.27.

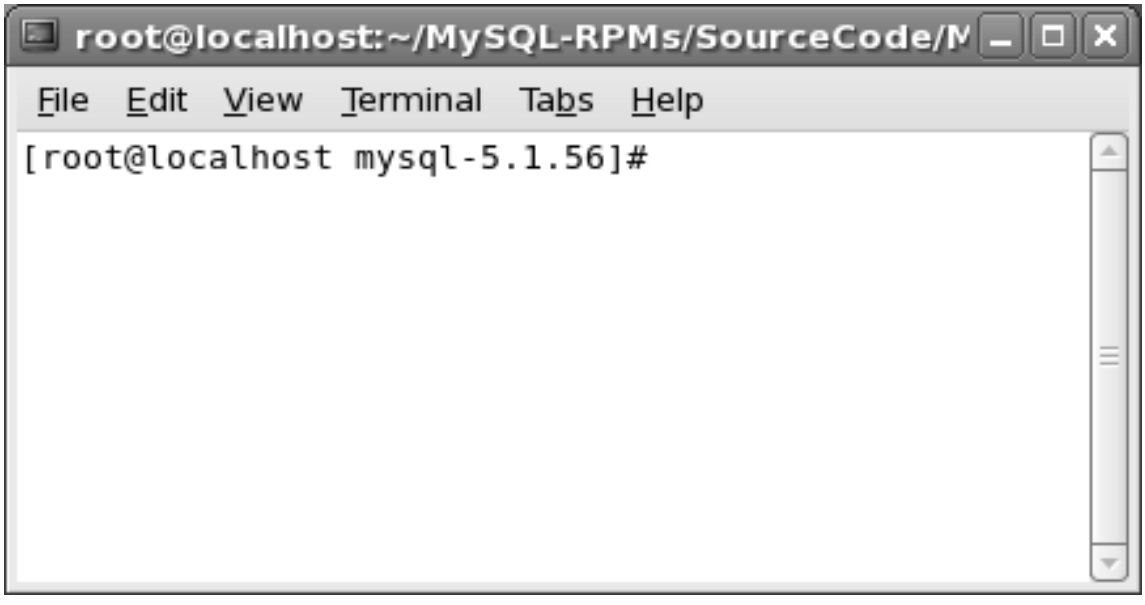


Figure 2.27: Terminal Window

12. Enter the following command at the command prompt:

```
ln –s MySQL-community-5.1.56.rhel5.src root
```

Figure 2.28 displays the output of the command.

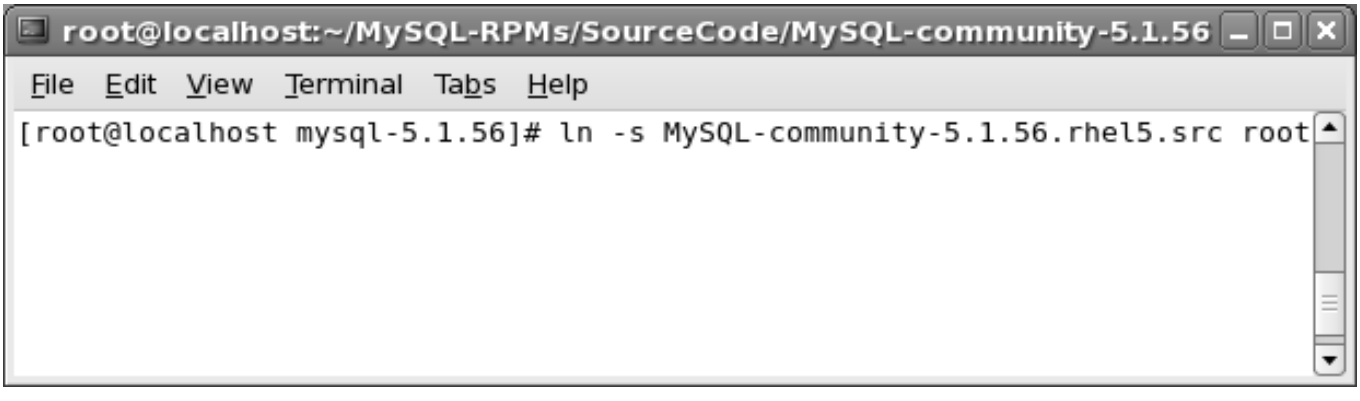


Figure 2.28: Creating a Symbolic Link

13. To view the options for configuration, enter the following command at the command prompt:

```
./configure –help
```

Concepts

Figure 2.29 displays the output of the command.

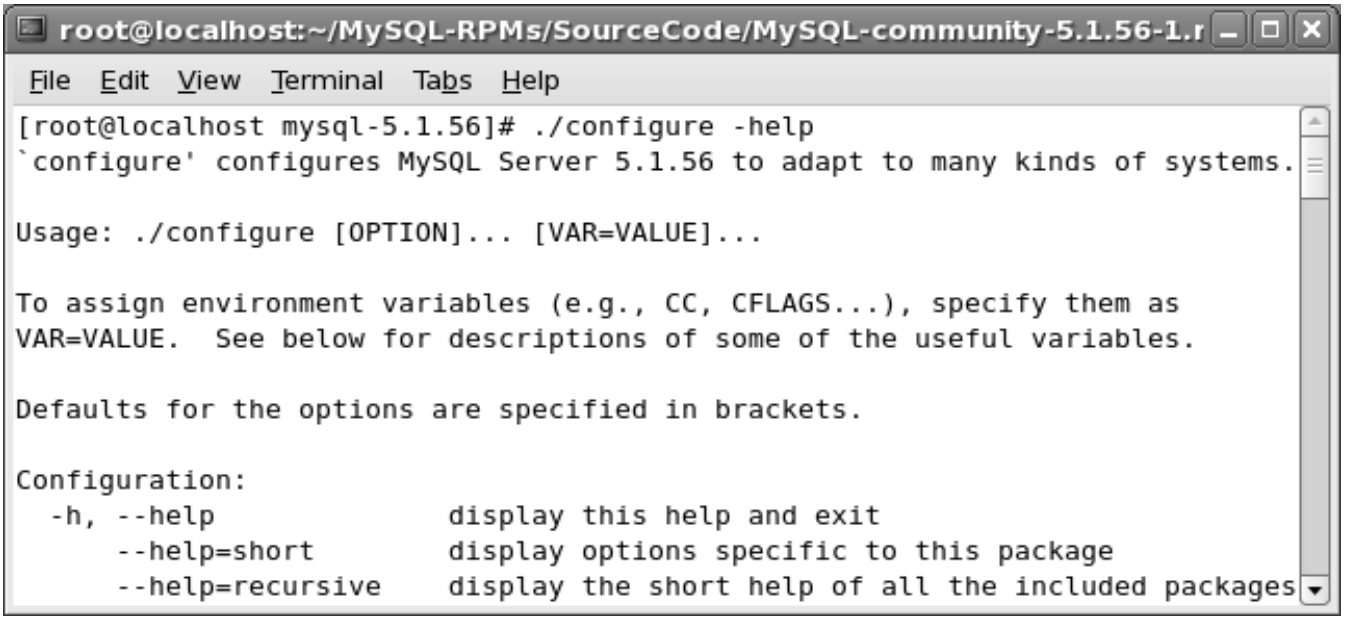


Figure 2.29: Help Options of Configure

14. To open the configuration file, enter the following command at the command prompt:

```
vi configure
```

Figure 2.30 displays the configuration file in vi editor.

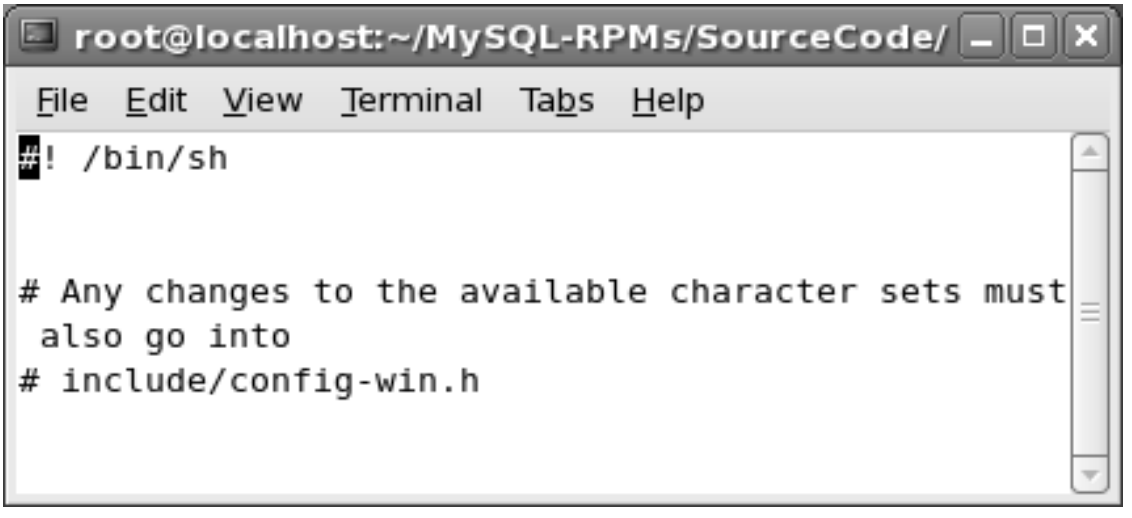


Figure 2.30: Configure file in vi Editor

Following are commonly used options with the configure command to customize the distribution of MySQL:

- To compile only MySQL client and library programs without the server, use configure with the --without-server option as shown:

```
./configure --without-server
```

- To relocate log files and database directories from `/usr/local/var` location, use configure with the --prefix option.

```
./configure --prefix=/usr/local/mysql
./configure --prefix=/usr/local\--localstatedir= /usr/local/mysql/data
```

The first command changes the installation default location from `/usr/local` to `/usr/local/mysql`. The second command maintains the chosen default installation location, and places the database directories from `/usr/local/var` to `/usr/local /mysql/` data.

- To build binary distributions, to increase the speed, or to work with the bugs, use configure with the `–with-client-Idflags` option.

```
./configure   –with-client-Idflags=-all-static\--with-mysqld-Idflags=-all-static
```

- To configure MySQL for not using DEFAULT value in the columns that cannot contain NULL values. Use configure with the option DDONT_USE_DEFAULT_FILEDS as shown:

```
CXXFLAGS=-DDONT_USE_DEFAULT_FIELDS./configure
```

`CXXFLAGS` is a flag used for C++ compiler. This is one of the environment variables. With the help of this command, it is specified that no columns will be accepting `NULL` value. This will help the `INSERT` command to avoid errors while inserting values in the tables.

- To change the default character set, use configure with the `--with-charset` option as shown:

```
./configure --with-charset=CHARSET
```

You can use the character sets, such as `big5`, `dec8`, `cp850`, `hp8`, `koi8r`, `latin1`, `latin2`, `swe7`, `ascii`, `ujis`, `sjis`, `hebrew`, `tis620`, `euckr`, `koi8u`, `gb2312`, `greek`, `cp1250`, `gbk`, `latin5`, `armscii8`, `utf8`, `ucs2`, `cp866`, `keybcs2`, `macce`, `macroman`, `cp852`, `latin7`, `utf8mb4`, `cp1251`, `utf16`, `cp1256`, `cp1257`, `utf32`, `binary`, `geostd8`, `cp932`, and `eucjpms` as the default character set in MySQL.

- To configure MySQL with codes for debugging, use configure with the `--with-debug` option as shown:

```
./configure --with-debug
```

You can debug errors and get the output with the help of this command.

2.5 Controlling MySQL Options Using my.cnf file

MySQL reads the default startup options for the server and the client. These default options are stored in the my.cnf file.

On Microsoft Windows OS, MySQL reads the default options from my.ini file. The MySQL Server Instance Configuration wizard generates this file and stores it in the MySQL Server 5.1 folder under the **C:\Program Files** folder.

On the Red Hat Enterprise Linux platform, the files from which MySQL reads default options are listed in table 2.3.

File	Purpose
`/etc/my.cnf`	Defines global options
`/etc/mysql/my.cnf`	Defines global options
`$MYSQL_HOME/my.cnf`	Defines server specific options
`~/.my.cnf`	Defines user specific options

Table 2.3: Default Option Files For MySQL in Red Hat Enterprise Linux

The programs that support option files are `mysql`, `mysqladmin`, `mysqld`, `mysqld_safe`, `mysql.server`, `mysqldump`, `mysqlimport`, `mysqlshow`, `mysqlcheck`, `myisamchk`, and `myisampack`.

An option file consists of the following details:

- **Comments** - starts lines with symbols, such as '#' or ';'. A comment is non-executable statement in a file that is added to provide extra information to the user
- **Group** - specifies the name for which the user wants to set the options
- **Option** - specifies the `--option` on the command line
- **option = value** - specifies the value for the option. It is the same as specifying `--option=value` on the command line

- **set-variable = variable value** - assigns a variable. It is the same as `--set-variable variable=value` on the command line

To open the **my.cnf** file:

1. Browse to the **mysql-test** folder under the mysql source code folder.
2. Right click the include folder and select **Open In Terminal**. The Terminal window appears.
3. Enter the following command at the command prompt:

```
vi default_my.cnf
```

Figure 2.31 displays the output of the command.

Figure 2.31: default_my.cnf in vi editor

In figure 2.31, the lines beginning with the # symbol are comments. These lines are not executable and provide information about this file. The file contains:

- Socket
- Port Number
- User Name
- Password

Summary

- You can use binary distribution format instead of source distribution format while installing MySQL. After the installation of binary distribution, default directories are created in the source directory.
- Binary distribution format contains setup program that installs every component for the server from the start. Source distribution format contains all the codes and support files for making the files executable.
- Solaris, Linux, Windows XP/2003/Vista/2007, Mac OS X, FreeBSD, and AIX are the different operating systems that support MySQL.
- You can customize the configuration of MySQL using `./configure` command. You can also use various options of the configure command to configure MySQL on the command line.
- The environment variables enable you to edit or alter the configure file.
- MySQL can be configured using the combination of command-line options, configuration files, and environment variables.
- The programs like `mysql`, `mysqladmin`, `mysqld`, `mysqld_safe`, `mysql.server`, `mysqldump`, `mysqlimport`, `mysqlshow`, `mysqlcheck`, `myisamchk`, and `myisampack` supports option files of `my.cnf` file.
- The OS can be configured to start MySQL at startup.

Check Your Progress

1. To start MySQL client session in MS-DOS prompt, type _____.

   a. mysql –u root

   b. mysql

   c. mysqld-shareware –standalone

   d. mysql -u

2. Which of the following environment variable locates the MySQL history file?

   a. HOME

   b. MYSQL_PSL

   c. MYSQL_HOST

   d. MYSQL_HISTFILE

3. Which of the following options for MySQL can be specified in the **my.cnf** file?

   a. Global

   b. User

   c. Server

   d. Comment

4. In binary distribution format, the installation is configured using _____.

   a. System Variables

   b. Environment variables

   c. Server Variables

   d. Scripts

Check Your Progress

5. Which command is used to configure the installation of MySQL in Red Hat Enterprise Linux?

a. ./configure

b. ./configure -help

c. configure

d. vi configure

Onlinevarsity

GROUPS

COLLABORATIVE LEARNING

Session 3

Installing and Configuring MySQL (Lab)

Objectives

At the end of this session, the student will be able to:

- *Install MySQL on Microsoft Windows.*
- *Configure MySQL on Microsoft Windows.*
- *Install MySQL on Red Hat Linux.*
- *Configure MySQL on Red Hat Enterprise Linux.*

The steps given in the session are detailed, comprehensive and carefully thought through. This has been done so that the learning objectives are met and the understanding of the software is complete. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Install MySQL

You can install MySQL on the following platforms:

- Microsoft Windows
- Red Hat Enterprise Linux

Note: You can download the mysql-5.1.56-win32.msi file to install MySQL from their official Website `http://dev.mysql.com/downloads/mysql/5.1.html`

Installing and Configuring MySQL on Microsoft Windows

Download the MySQL executable file in a temporary folder named mysqltmp. To install MySQL on Microsoft Windows XP platform, perform the following steps:

Note: You will require administrative rights on Microsoft Windows XP to install MySQL. Contact the system administrator for administrative permissions to install MySQL.

1. **To install MySQL on Microsoft Windows XP, run `mysql-5.1.56-win32.msi` from `mysqltmp` folder. The Open File – Security Warning dialog box is displayed as shown in figure 3.1.**

Figure 3.1: Open File – Security Warning Dialog Box

2. **Click Run. The Microsoft Windows installer prepares the installation process and the MySQL Server 5.1 - Setup Wizard dialog box is displayed as shown in figure 3.2.**

Figure 3.2: MySQL Server 5.1 Setup Wizard

3. **Click Next. The License Agreement pane of the MySQL Server 5.1 - Setup Wizard dialog box is displayed as shown in figure 3.3.**

Figure 3.3: License Agreement Dialog Box

4. **Click 'I accept the terms in the license agreement' option and the MySQL Server 5.1 - Setup Wizard activates the Next button as shown in figure 3.4.**

Figure 3.4: License Agreement Dialog Box after Selection of Terms

5. **Click Next. The 'Setup Type' pane of the MySQL Server 5.1 - Setup Wizard dialog box is displayed as shown in figure 3.5.**

Figure 3.5: Setup Type Dialog Box

6. **Click Typical.**

7. **Click Next The 'Ready to Install the Program' pane of the MySQL Server 5.1 - Setup Wizard dialog box is displayed as shown in figure 3.6.**

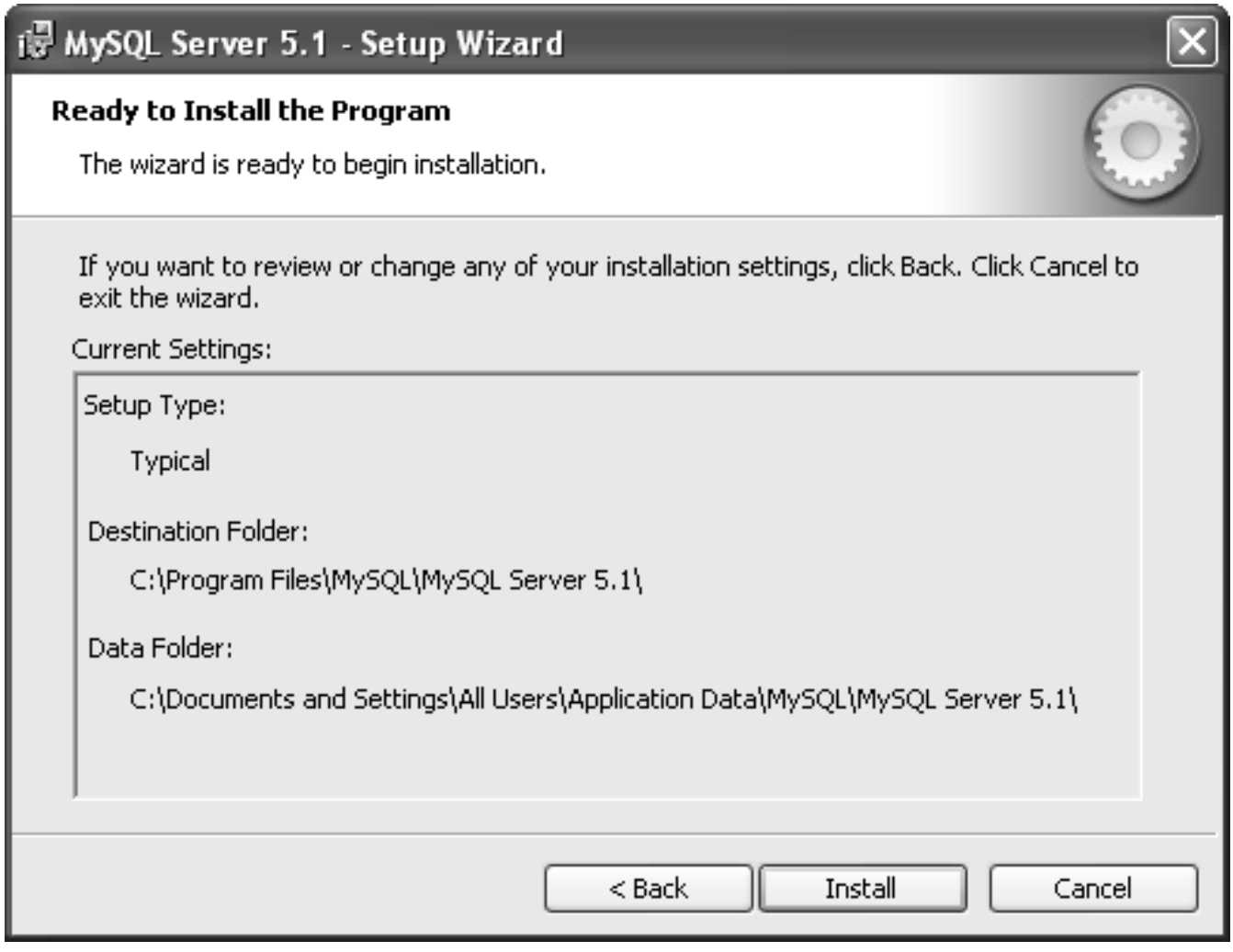


Figure 3.6: Reviewing Installation Settings

8. **Click Install. The Setup Wizard completes the installation process and the MySQL Enterprise dialog box is displayed as shown in figure 3.7.**

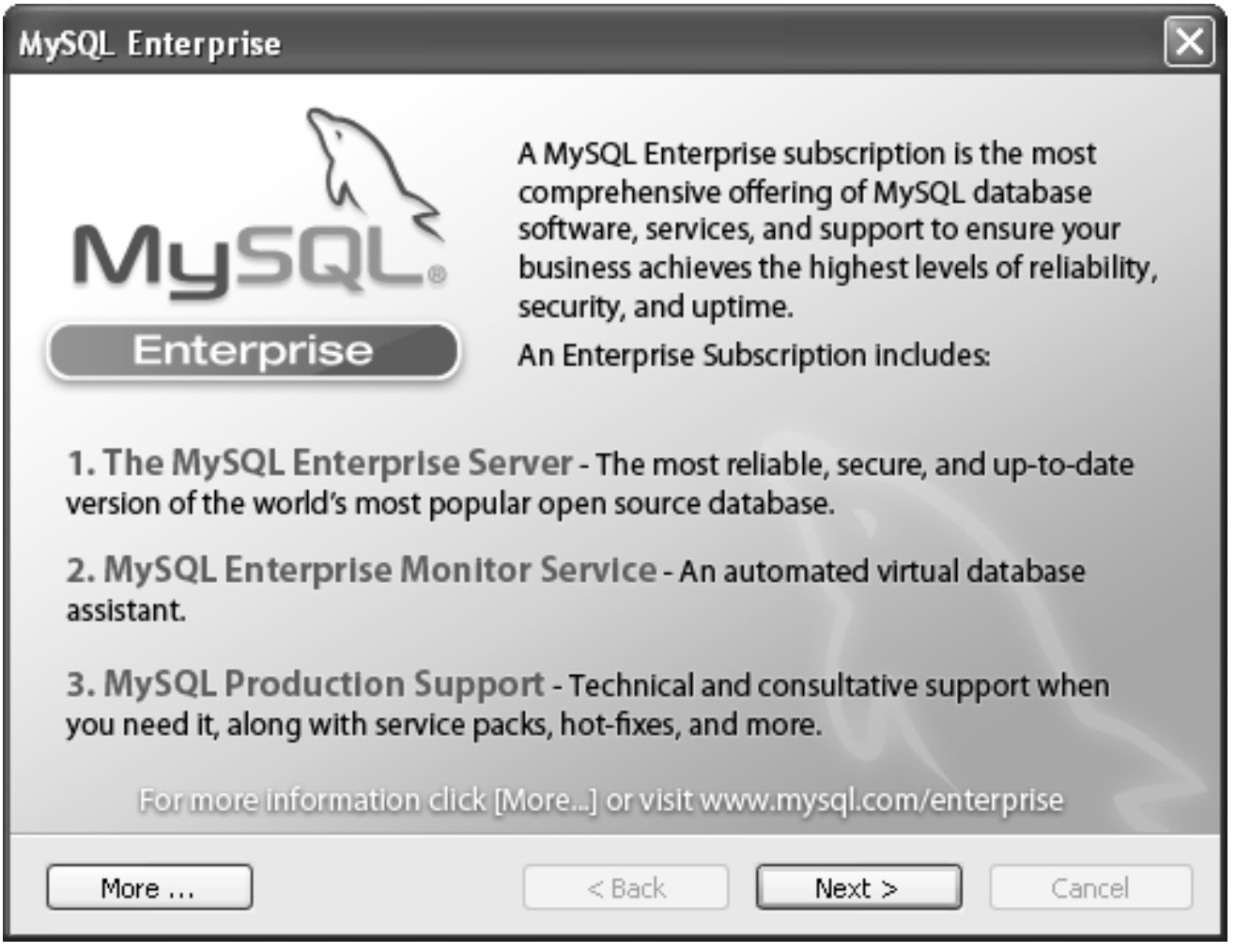


Figure 3.7: MySQL Enterprise Dialog Box

9. **Click Next. The MySQL Enterprise Monitor Service pane is displayed as shown in figure 3.8.**

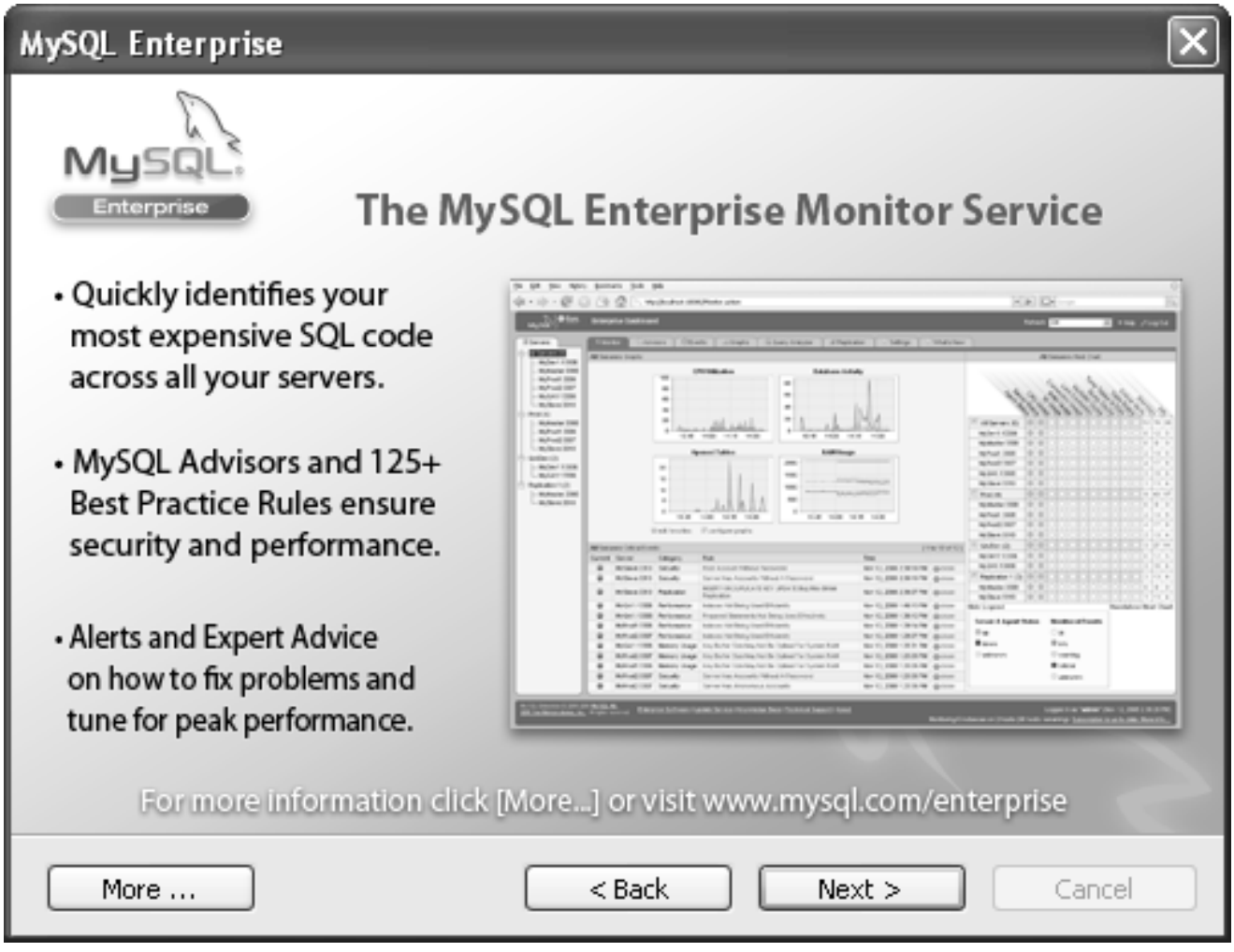


Figure 3.8: MySQL Enterprise Monitor Service Dialog Box

10. **Click Next. The Setup Wizard completes the installation process and displays the Wizard Completed pane as shown in figure 3.9.**

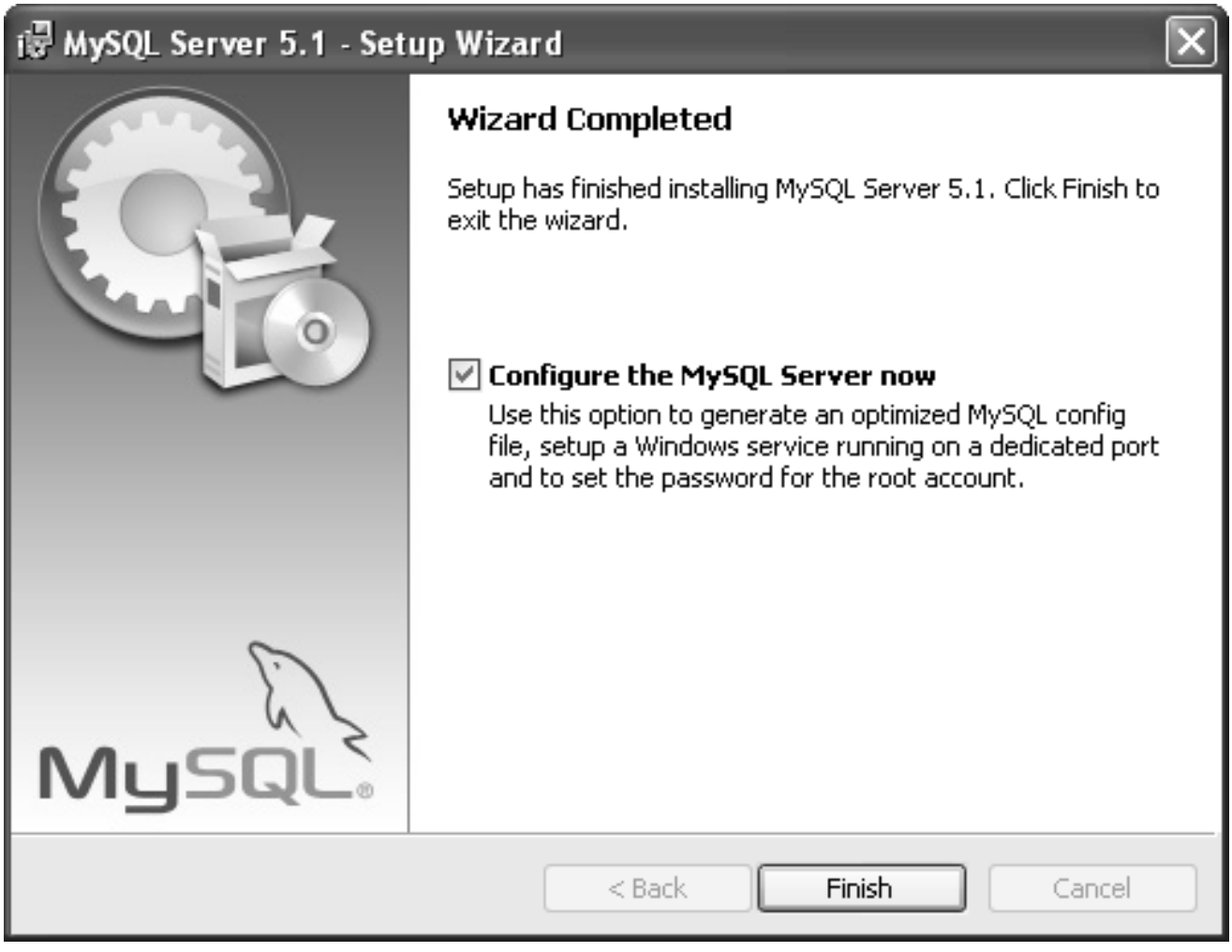


Figure 3.9: MySQL Installation Complete

11. **Click Finish. The MySQL Server Instance Configuration Wizard dialog box is displayed as shown in figure 3.10.**

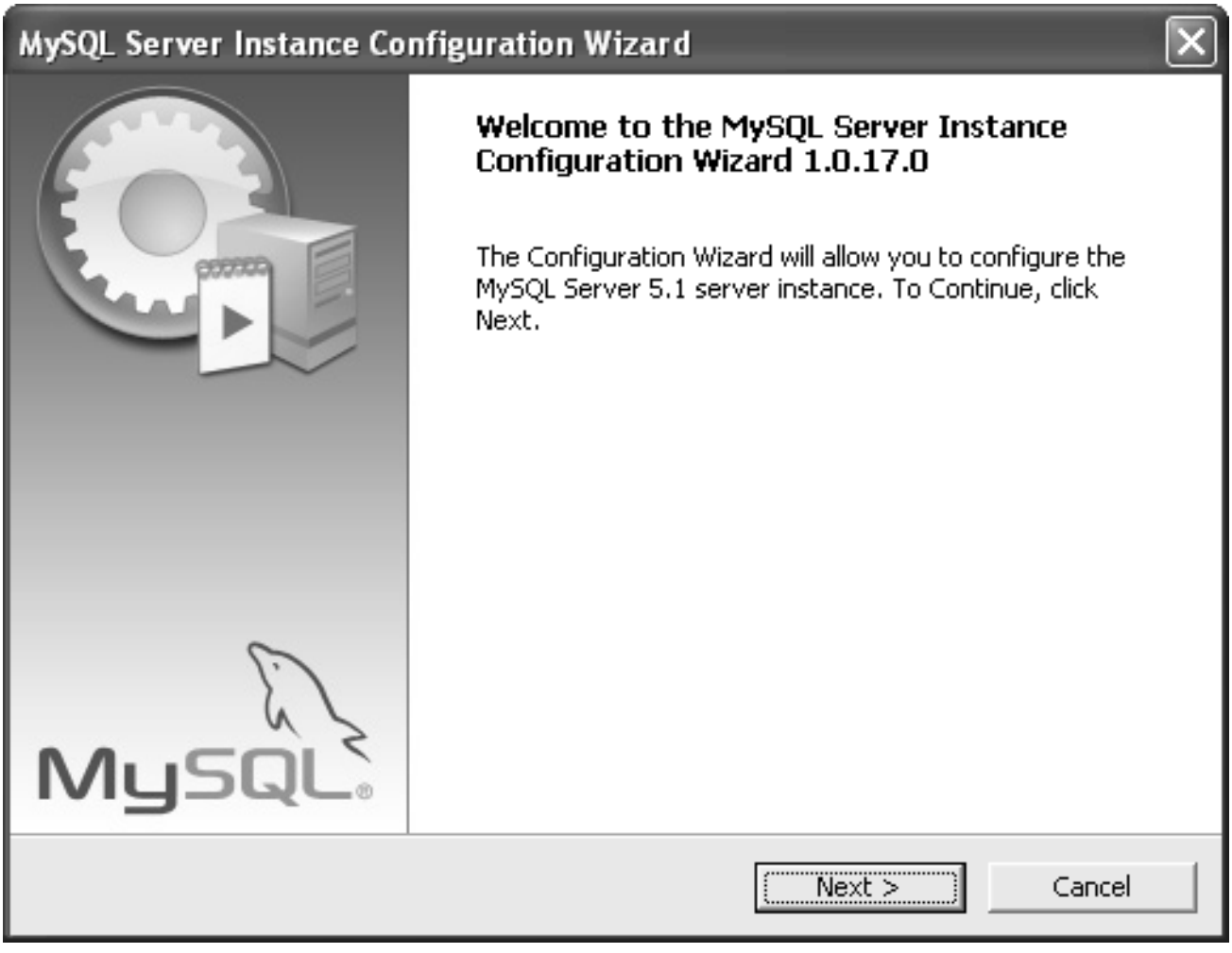


Figure 3.10: MySQL Server Instance Configuration Wizard

12. **Click Next. The MySQL Server Instance Configuration pane is displayed as shown in figure 3.11.**

Figure 3.11: MySQL Configuration Type Dialog Box

13. **Click Standard Configuration.**

14. **Click Next. The Windows Options pane is displayed as shown in figure 3.12.**

Figure 3.12: Windows Options Dialog Box

15. **Select 'Install As Windows Service' check box.**

16. **Select 'Include Bin Directory in Windows PATH' check box.**

17. **Click Next. The Security Options pane is displayed as shown in figure 3.13.**

Figure 3.13: Security Options Dialog Box

18. **Clear the 'Modify Security Settings' check box.**

19. **Click Next. The MySQL Server Instance Configuration pane is displayed for configuration of the tasks. Figure 3.14 displays the MySQL Server Configuration Tasks.**

Note: You can specify a password for the root user in MySQL by entering the password in the text boxes.

Figure 3.14: MySQL Server Configuration Tasks

20. **Click Execute. The wizard configures the settings and displays a confirmation message box, as shown in figure 3.15.**

Figure 3.15: Confirmation Message Box

21. **Click Finish.**

22. **Start the command prompt.**

23. **Change the directory to C:\Program Files\MySQL Server 5.1.**

24. **Type mysql.**

25. **Press Enter. The MySQL prompt appears as shown in figure 3.16.**

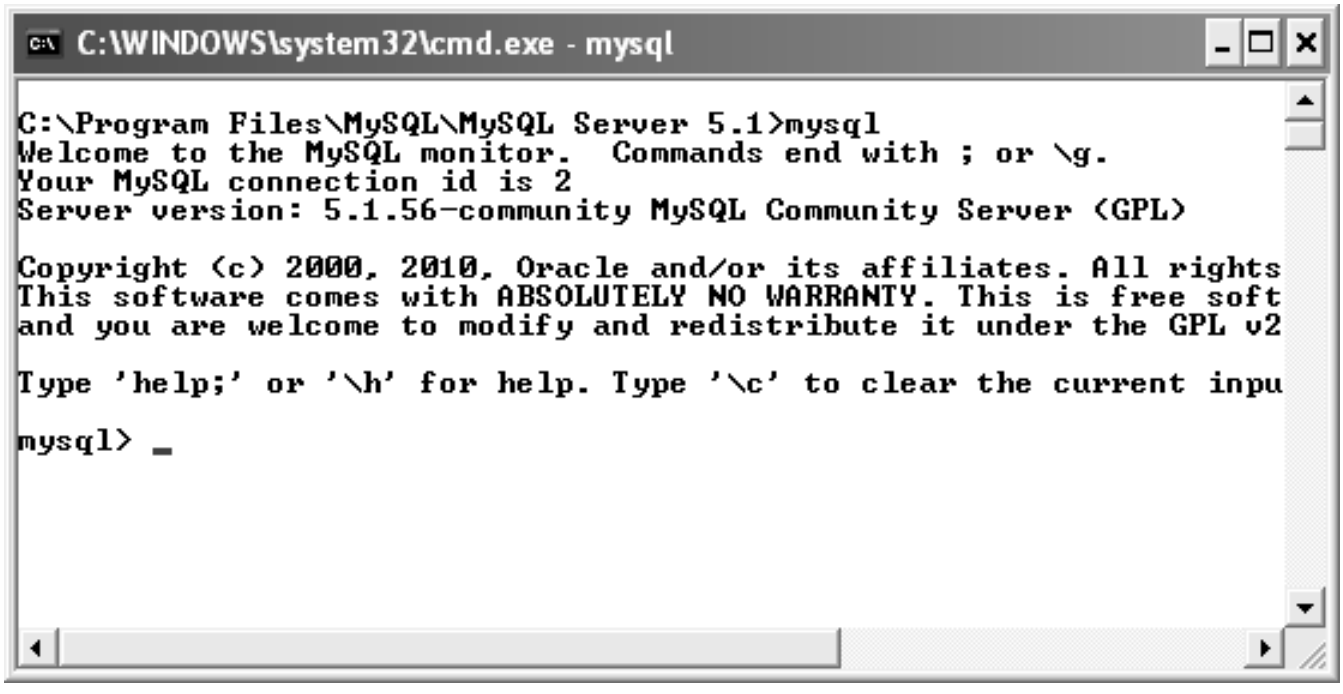


Figure 3.16: MySQL Prompt

Installing MySQL on Linux

Before installing MySQL on Red Hat Linux, create a group and a user with the GUI environment.

Note: You must login as root or have root privileges to create a group and user account.

1. **Click System → Administration → Users and Groups to create a group and a user for accessing MySQL. The User Manager dialog box is displayed as shown in figure 3.17.**

Note: If you are not a root user, Red Hat Enterprise Linux will prompt you to enter the root password in step 1. You cannot create a group or user account if you do not specify the root password.

Figure 3.17: User Manager Dialog Box

2. **Click File → Add Group to create a new group. The Create New Group dialog box is displayed.**

3. **Type mysql in the Group Name box as shown in figure 3.18.**

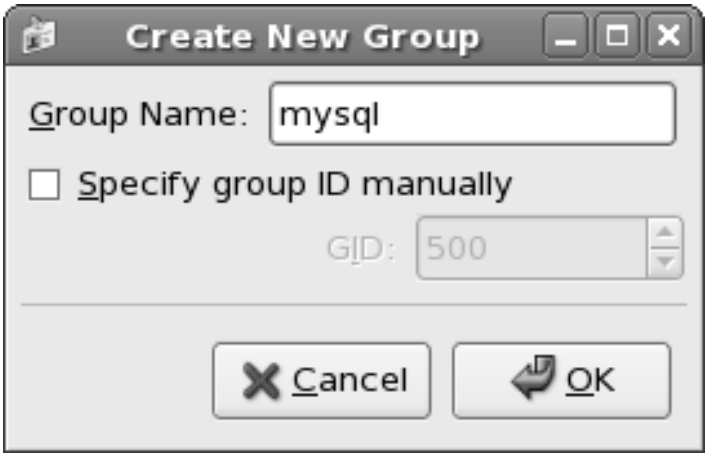


Figure 3.18: Create New Group Dialog Box

4. **Click OK.**

5. **Click the Groups tab in the User manager dialog box as shown in figure 3.19.**

Figure 3.19: Groups Tab

6. **Select mysql.**

7. **Click File → Properties. The Group Properties dialog box is displayed as shown in figure 3.20.**

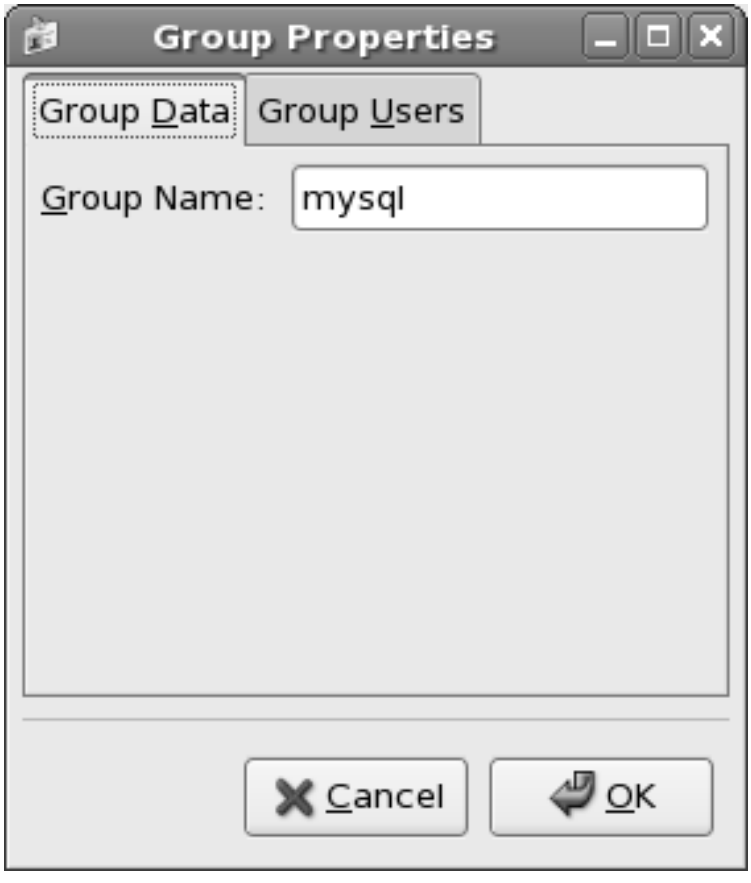


Figure 3.20: Group Properties Dialog Box

8. **Click Group Users tab as shown in figure 3.21.**

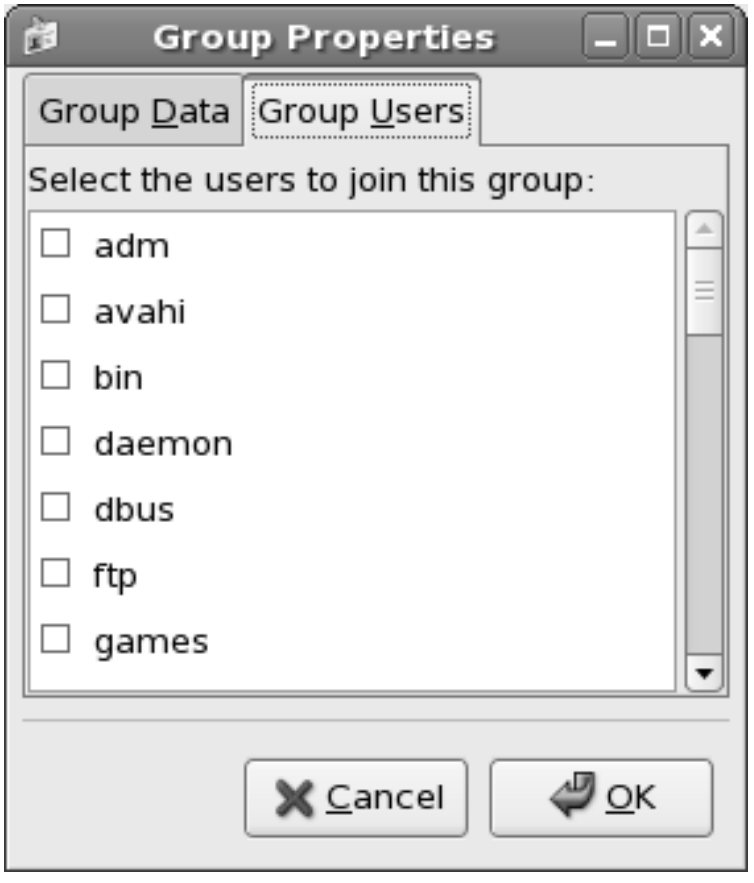


Figure 3.21: Group Users Tab

9.	**Select bin.**
10.	**Click OK.**
11.	**Click File → Add User to add a new user to the mysql group. The Create New User dialog box is displayed.**
12.	**Enter the User Name and the Password as shown in figure 3.22.**

Figure 3.22: Create New User Dialog Box

13. **Click OK.**

You will install MySQL on Linux using RPM packages. RPM packages are the binary distribution format of MySQL. The following RPM packages are required to install MySQL:

```
MySQL-client-community-5.1.56-1.rhel5.i386.rpm;

MySQL-server-community-5.1.56-1.rhel5.i386.rpm;
```

Note: You can download these RPM packages from the MySQL Website: `http://www.mysql.com/downloads/mysql/5.1.html#downloads`. You must select Red Hat & Oracle Enterprise Linux from the list of operating systems to view the RPM packages to download.

To install MySQL server 5.1.56 on Red Hat Enterprise Linux 5:

1. **Double-click `MySQL-server-community-5.1.56-1.rhel5.i386.rpm` to install MySQL Server package. The Installing packages dialog box is displayed as shown in figure 3.23.**

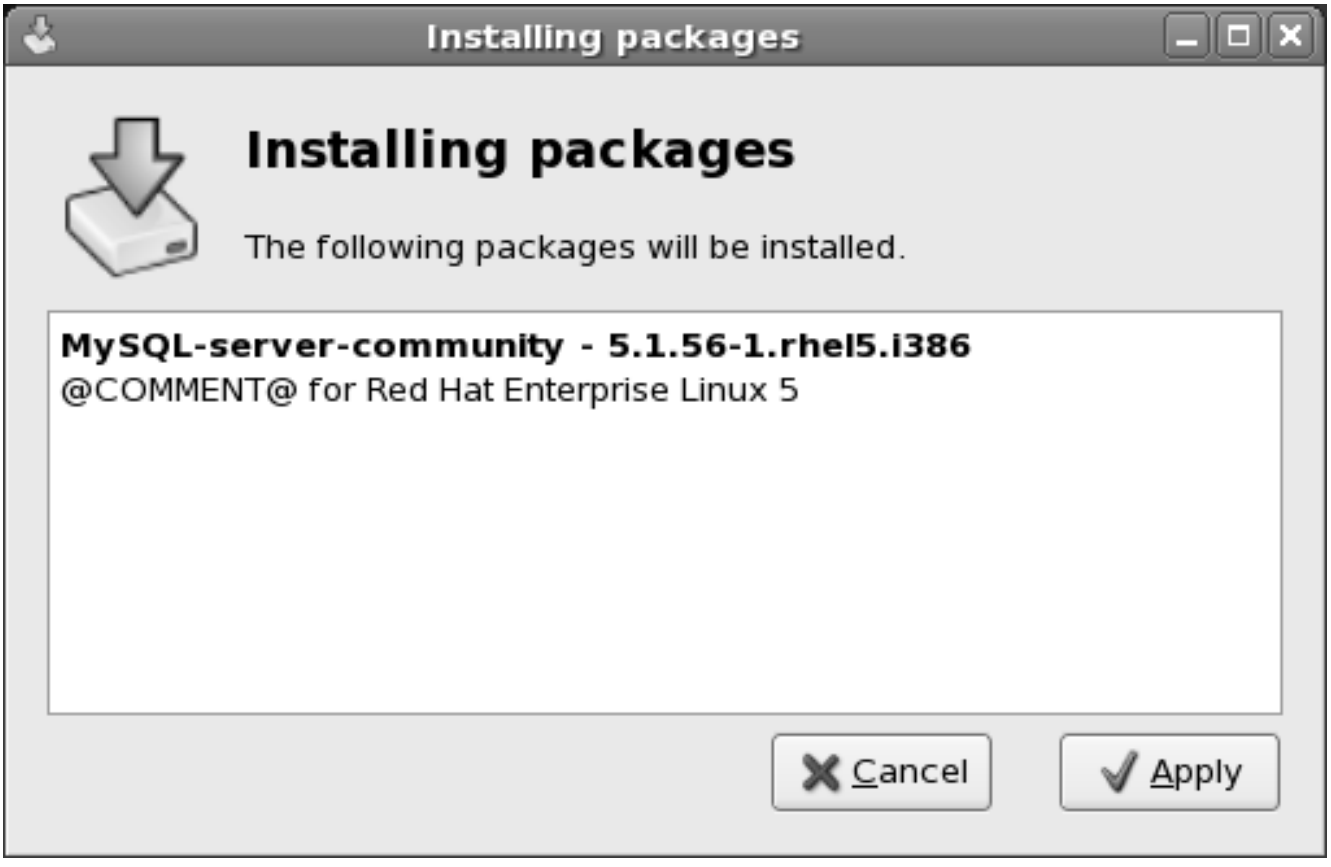


Figure 3.23: Installing Server Packages Dialog Box

2. **Click Apply. The installation completes.**

3. **Double-click `MySQL-client-community-5.1.56-1.rhel5.i386.rpm` to install MySQL client package,. The Installing packages dialog box is displayed as shown in figure 3.24.**

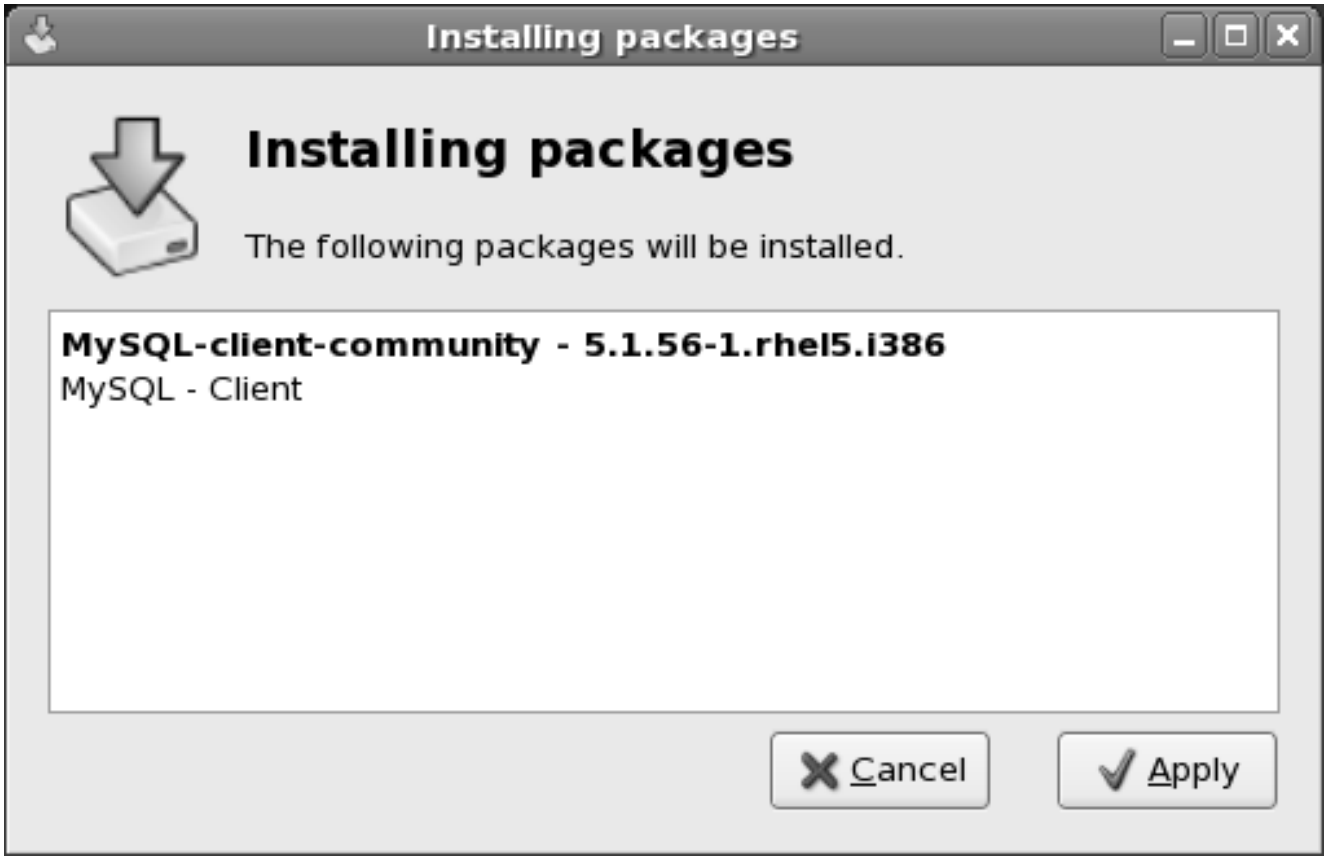


Figure 3.24: Installing Client Packages Dialog Box

4. **Click Apply. The installation completes.**

5. **To configure the MySQL service to start automatically with the operating system, open the Linux terminal and enter the following command at the command prompt:**

```
/etc/init.d/mysql start;
```

Figure 3.25 displays the output of the command.

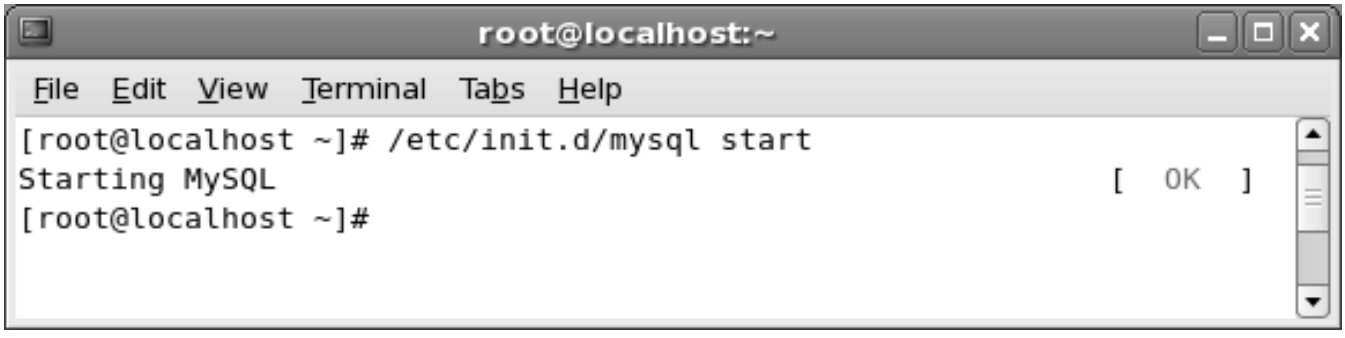


Figure 3.25: Initializing MySQL

6. **To start the MySQL client, enter the following command at the command prompt:**

```
mysql –u root;
```

Figure 3.26 displays the output of the command.

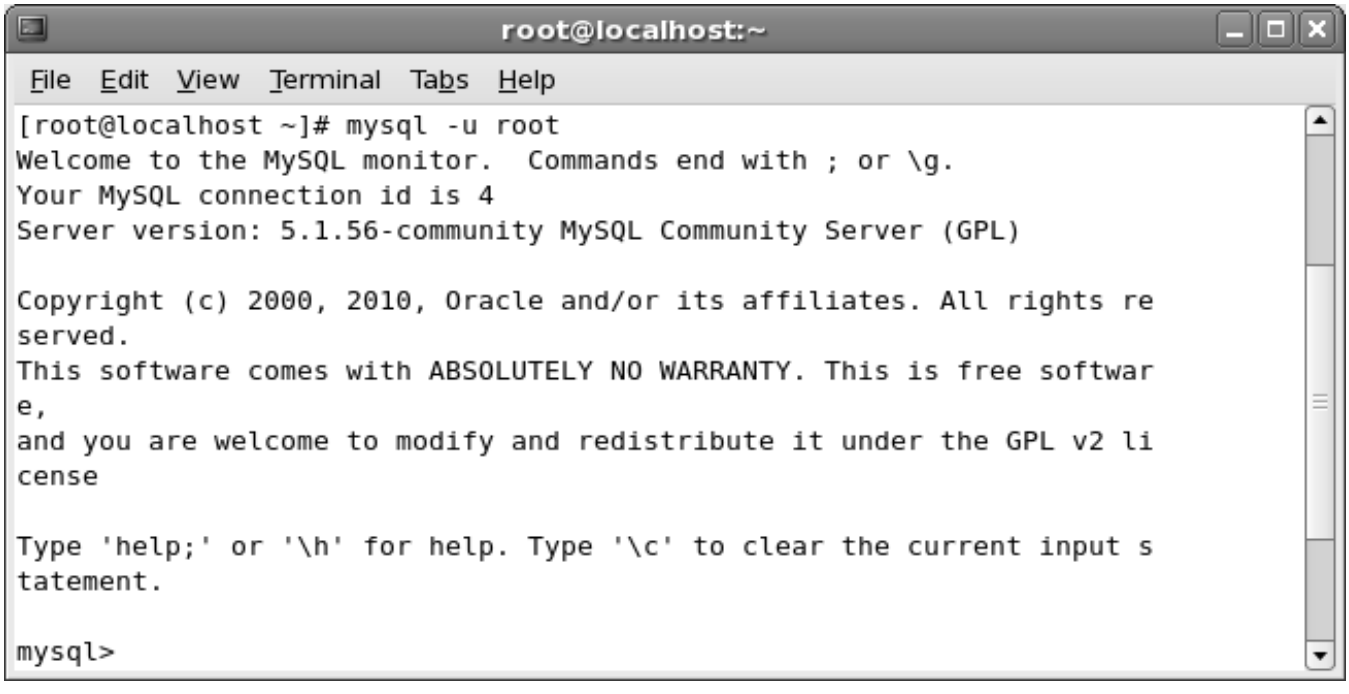


Figure 3.26: Starting MySQL Client

7. **To view the list of default databases, enter the following command at the command prompt:**

```
SHOW DATABASES;
```

Figure 3.27 displays the output of the command.

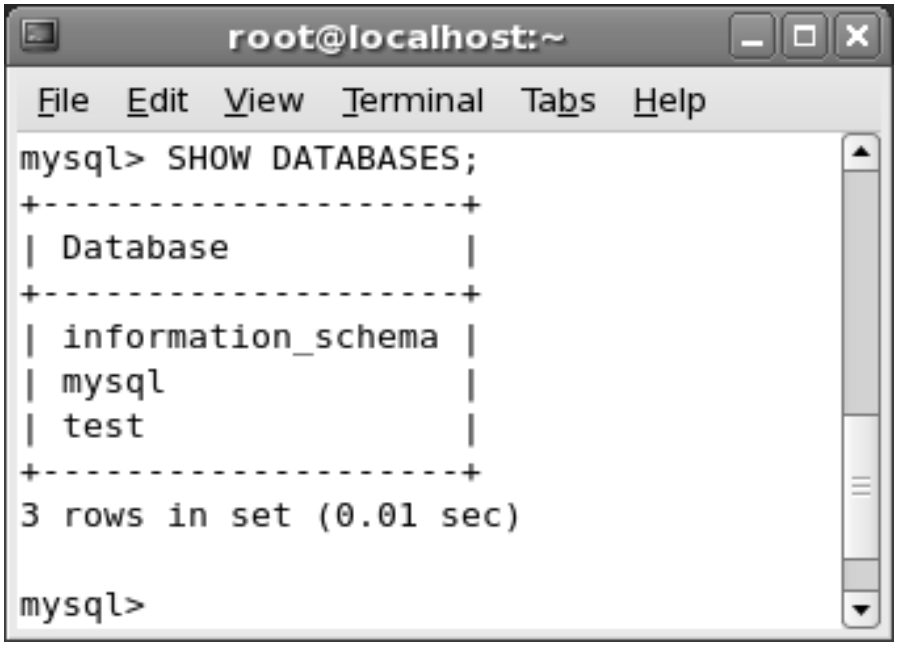


Figure 3.27: SHOW DATABASES Command

Configure MySQL Installation Using Configuration Scripts

The source code must be downloaded and extracted to a folder before making changes to the configuration files. You will be required to download the source code for MySQL to configure the installation. The source code is available in the Downloads section of MySQL Website under Source Code category. The file to be downloaded is:

- `MySQL-community-5.1.56-1.rhel5.src.rpm`

1. **Select Applications → Accessories → Archive Manager. The Archive Manager dialog box is displayed as shown in figure 3.28.**

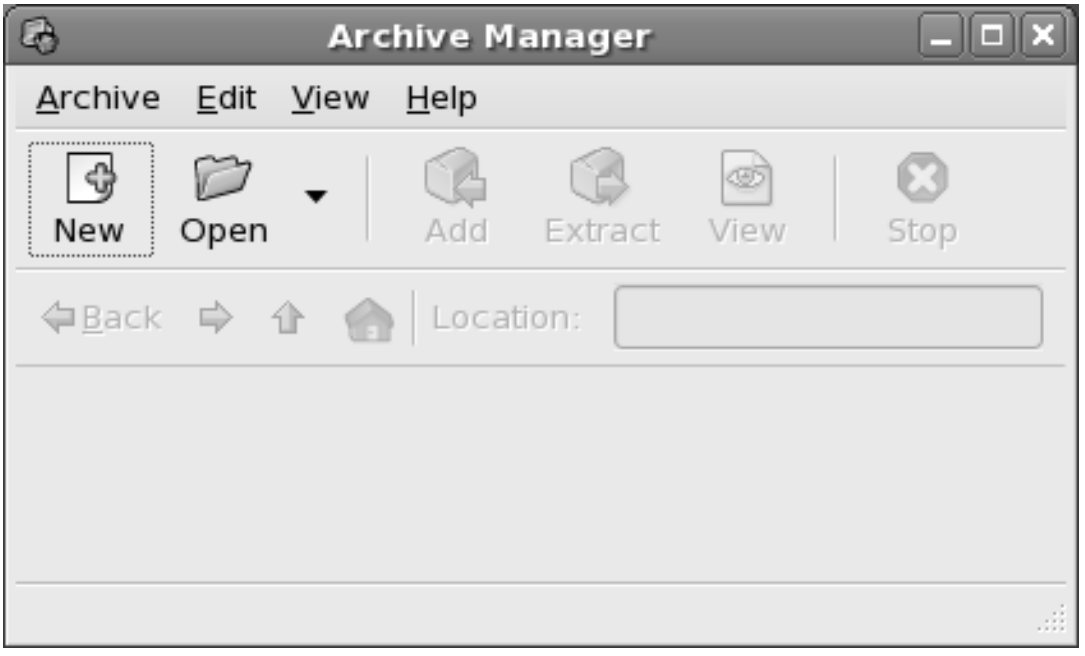


Figure 3.28: Archive Manager Dialog Box

2. **Select Archive → Open. The Open dialog box is displayed as shown in figure 3.29.**

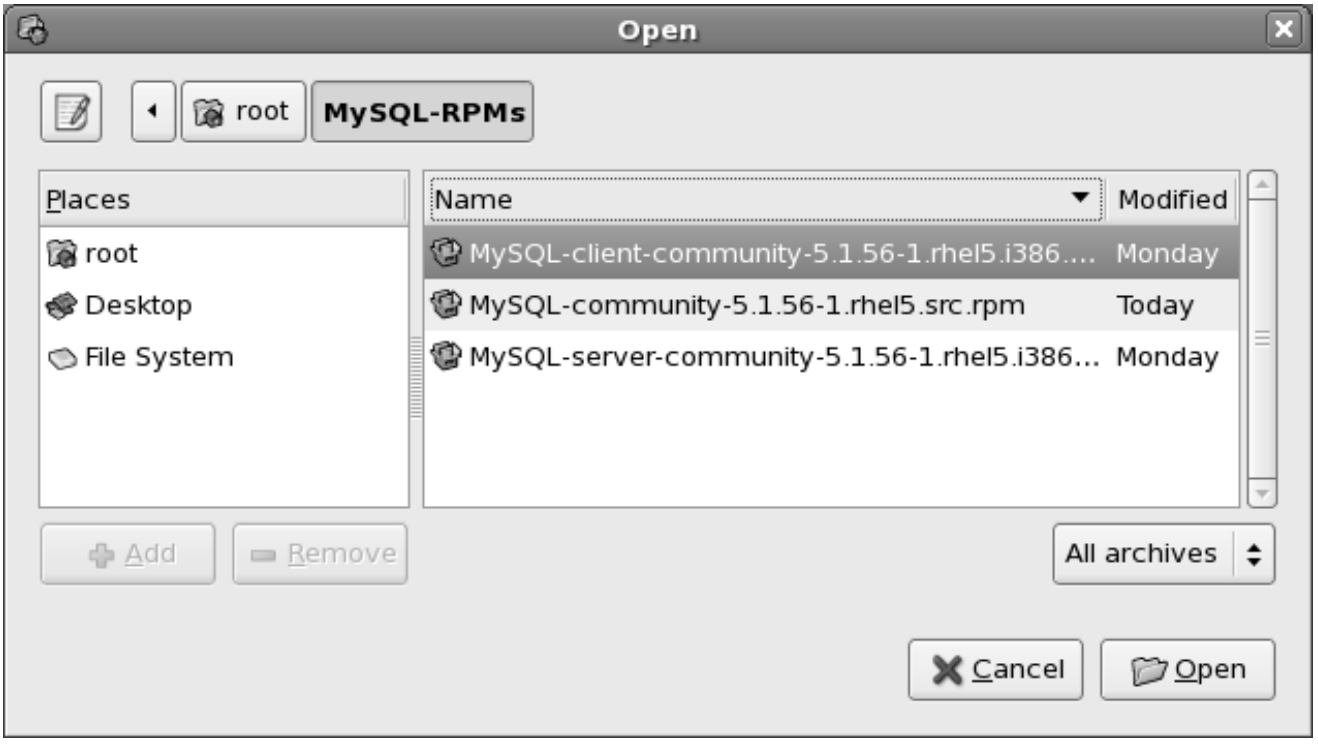


Figure 3.29: Open Dialog Box

3. **Browse to the folder that contains MySQL-community-5.1.56-1.rhel5.src.rpm.**

4. **Select MySQL-community-5.1.56-1.rhel5.src.rpm.**

5. **Click Open as shown in figure 3.30 to display the contents of the RPM package.**

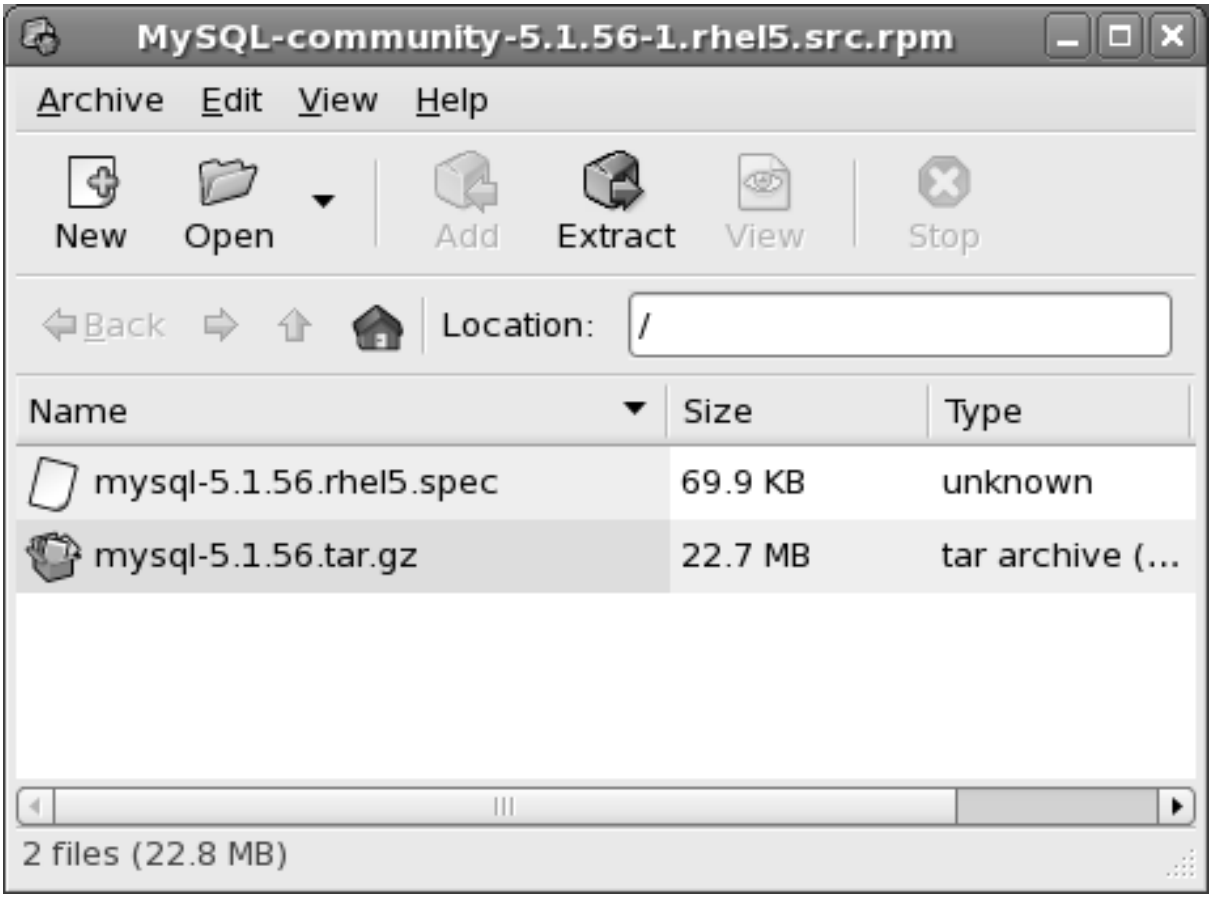


Figure 3.30: RPM Package Contents

6. **Minimize the Archive Manager window and create a folder named mysql under the root directory.**

7. **Select Archive → Extract. The Extract dialog box is displayed as shown in figure 3.31.**

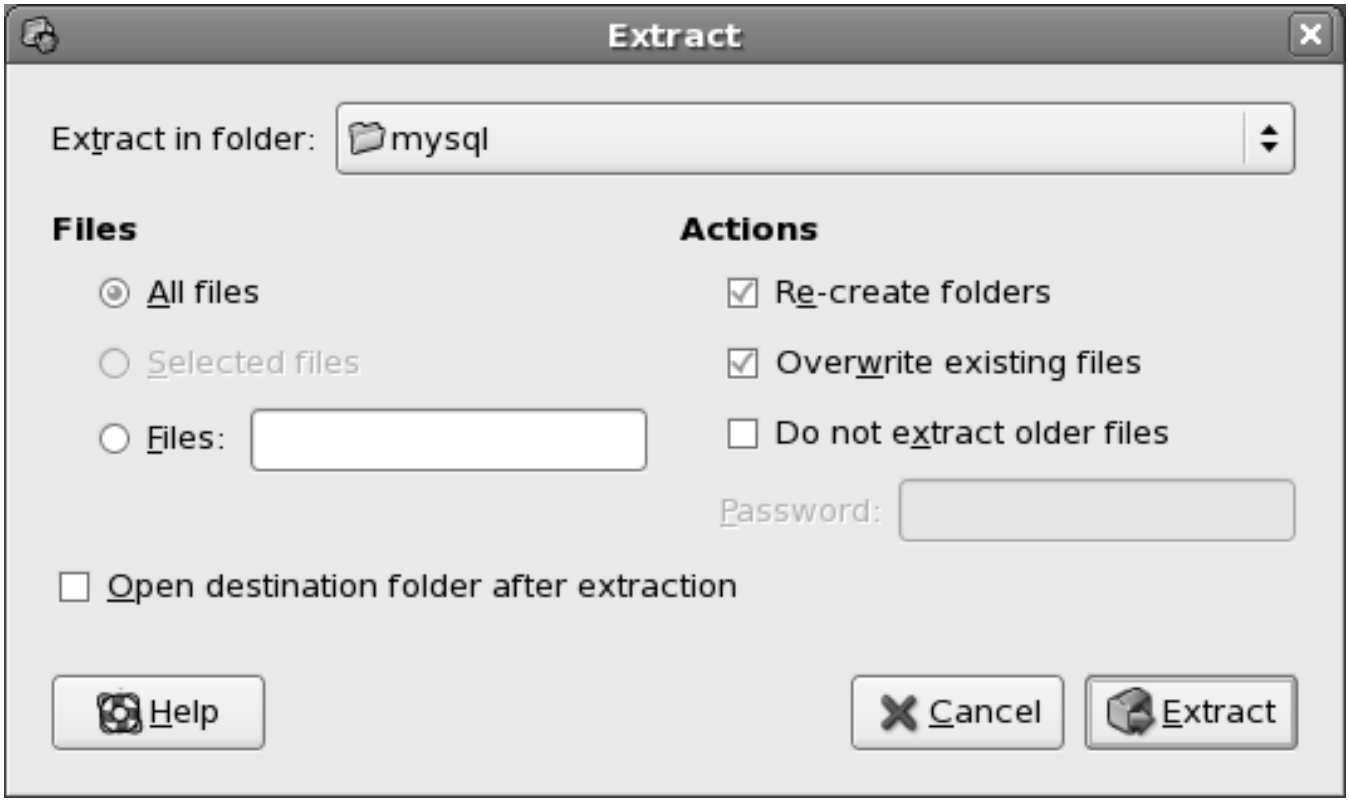


Figure 3.31: Extract Dialog Box

8. **Browse to the mysql folder in the 'Extract in folder' drop-down box.**

9. **Click Extract. The application extracts the package contents to the specified location.**

10. **Right click the `mysql-5.1.56.tar.gz` file and select Extract Here. The file contents are extracted as shown in figure 3.32.**

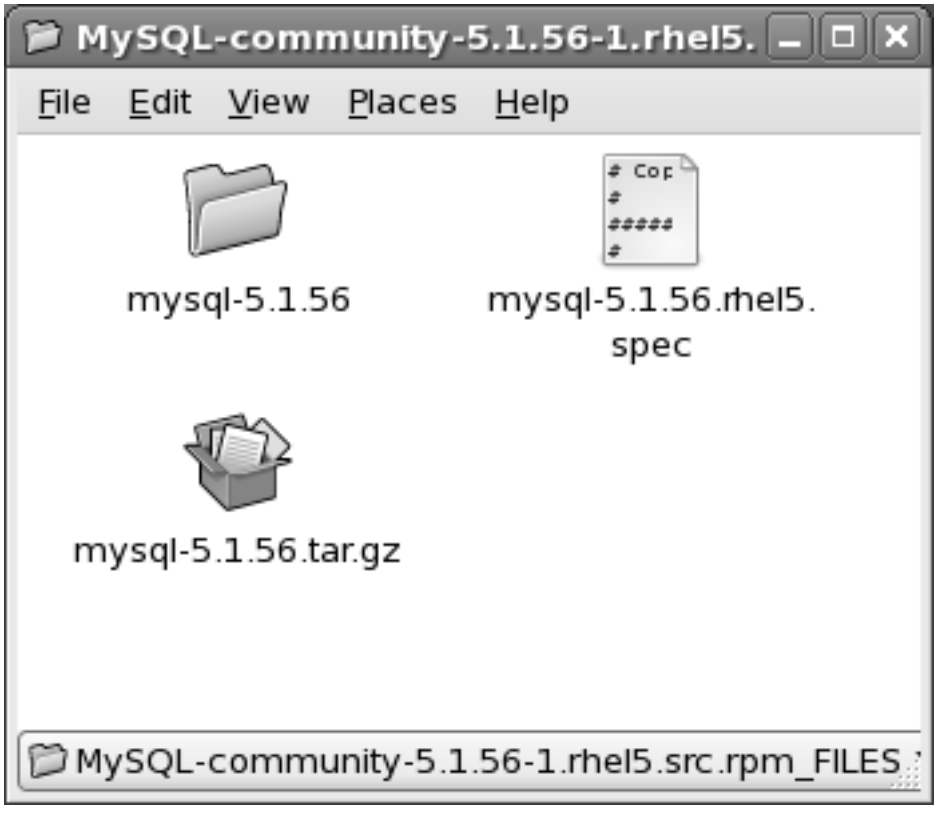


Figure 3.32: Extracted Package Contents

11. **Right click the `mysql-5.1.56` folder to display the context menu and select Open In Terminal. The Terminal window is displayed as shown in figure 3.33.**

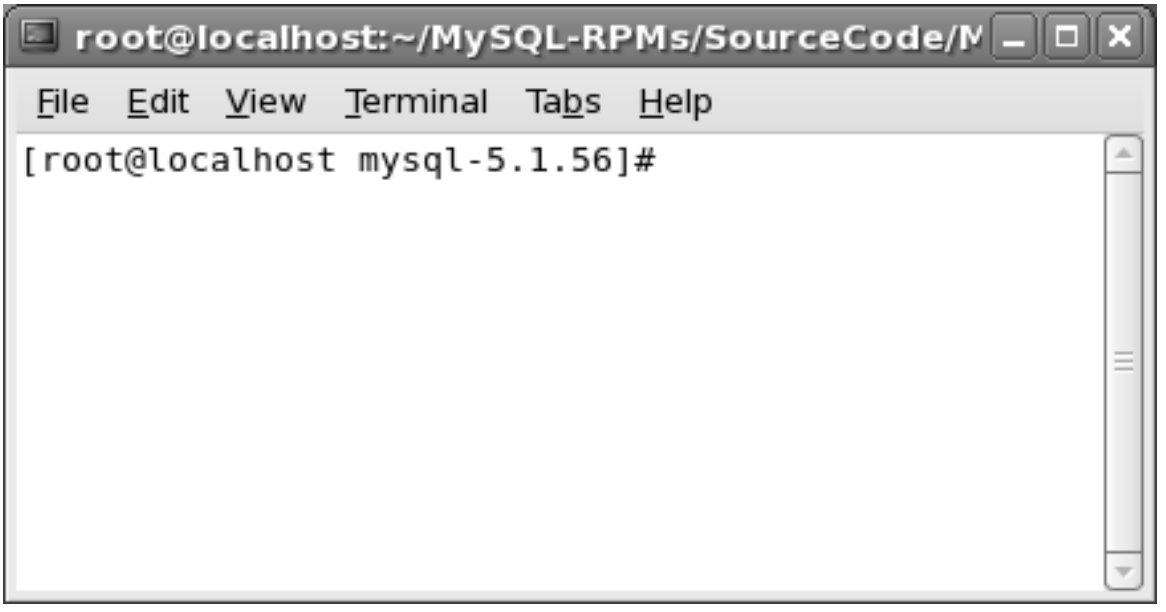


Figure 3.33: Terminal Window

12. **To create a symbolic link, enter the following command at the command prompt:**

```
ln –s MySQL-community-5.1.56.rhel5.src root;
```

Figure 3.34 displays the output of the command.

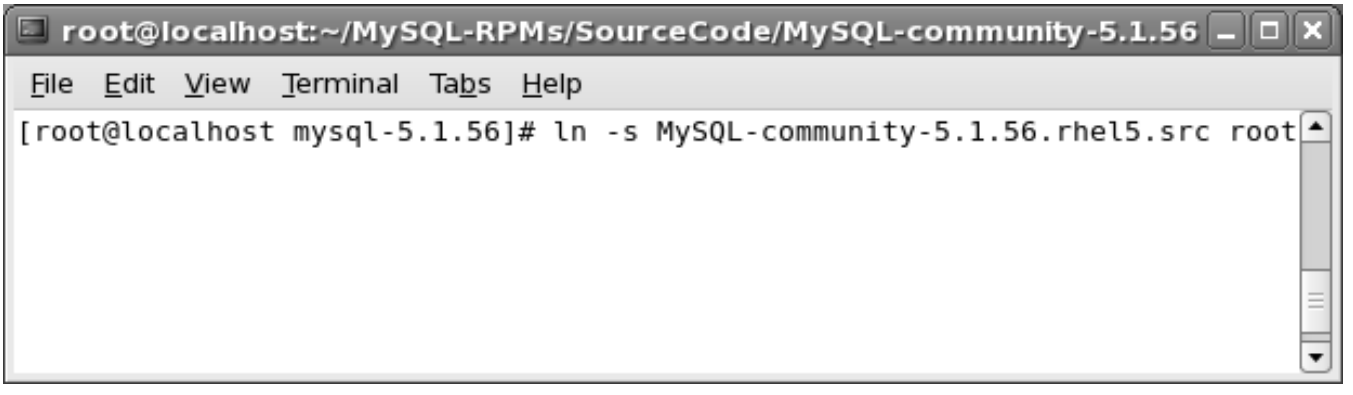


Figure 3.34: Creating a Symbolic Link

13. **To view the options for configuration, enter the following command at the command prompt:**

```
./configure –help;
```

Figure 3.35 displays the output of the command.

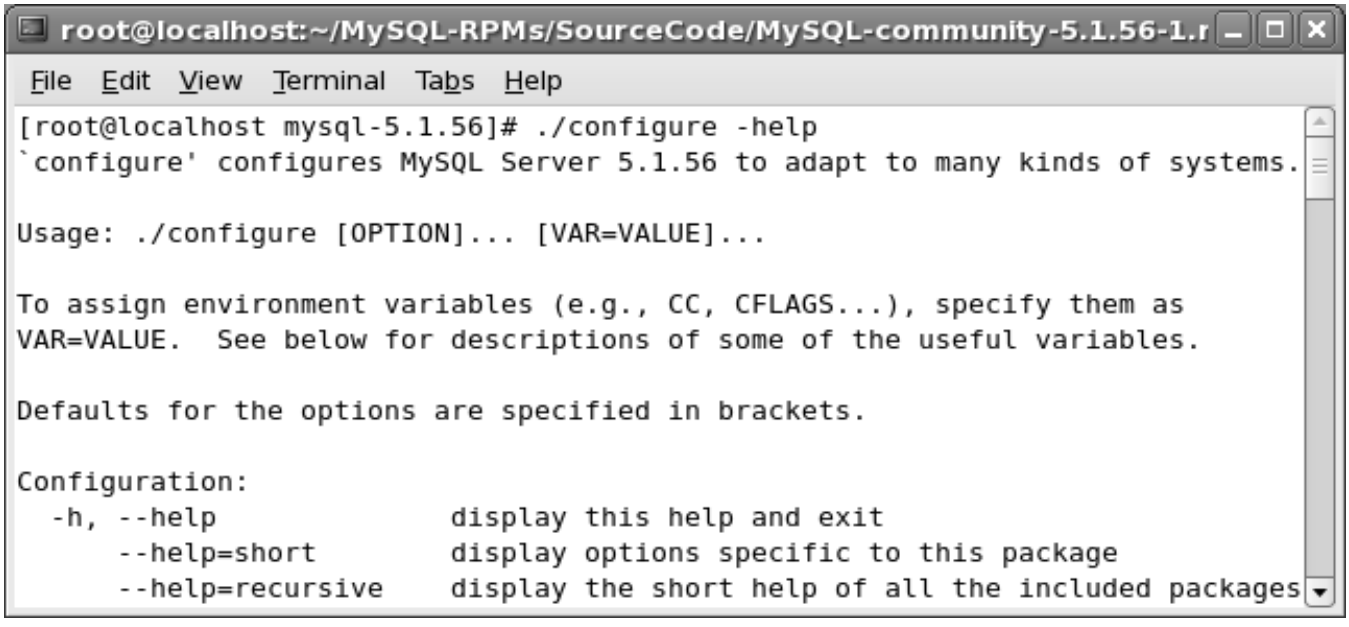


Figure 3.35: Help Options of Configure

14. **To open the configuration file, enter the following command at the command prompt:**

```
vi configure;
```

Figure 3.36 displays the output of the command.

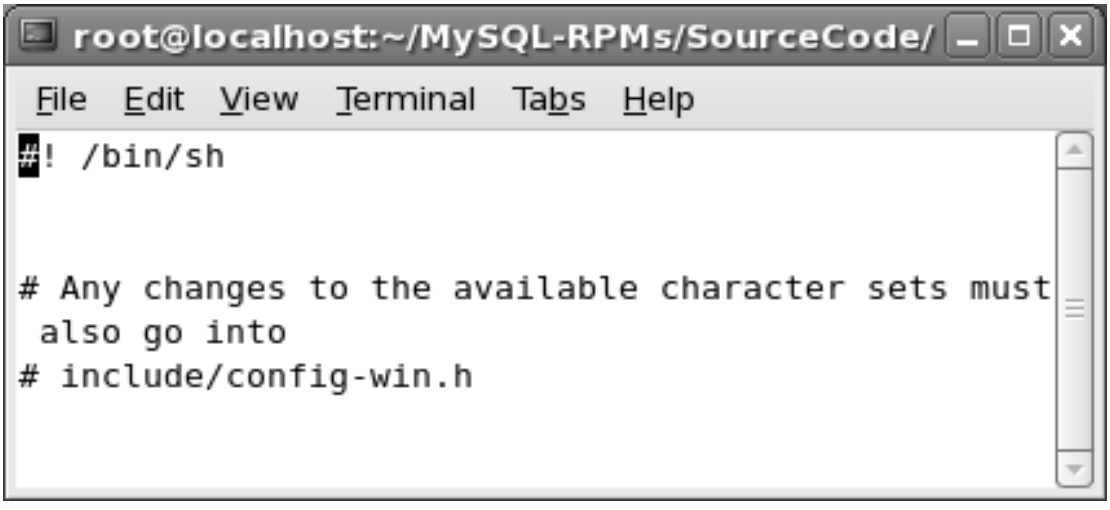


Figure 3.36: Configure File in vi Editor

15. **To open the my.cnf file, browse to the /mysql/mysql-5.1.56/include folder.**

16. **Right click the include folder and select Open In Terminal. The Terminal window is displayed.**

17. **Enter the following command at the command prompt:**

```
vi default_my.cnf;
```

Figure 3.37 displays the output of the command.

Figure 3.37: default_my.cnf File

Do It Yourself

1. Configure the installation of MySQL using the `--without-server` option.
2. Configure the installation of MySQL using the `--prefix` option.
3. Configure the installation of MySQL using the `--with-client-Idflags` option.
4. Configure the installation of MySQL using the `DDONT_USE_DEFAULT_FIELDS` option.
5. Configure the installation of MySQL using the `--with-charset` option.
6. Configure the installation of MySQL using the `--with-debug` option.

Session

4

Using MySQL

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain database.*
- *Explain the data types.*
- *Identify the different types of data.*
- *Explain the creation of a table.*
- *Explain Normalization.*
- *Identify the different forms of normalization.*
- *Explain Indexes and Referential Integrity.*

4.1 Introduction

A database is a collection of organized and related data. Databases store information in an organized manner in the form of a table. The table structure consists of rows and columns which holds data. A database management system can be used to handle the data in a database.

A database can contain duplicate records in tables if the database is not normalized and tables are not indexed. This can increase redundancy of the data in the database. You can normalize the database and index the tables to eliminate redundancy and enhance the speed of data search and retrieval operations.

In this session, you will learn how to create a database. You will identify the different types of data and how to create a table using different data types. In addition, you will learn how to normalize a database, and implement indexes and referential integrity in a database.

4.2 Basic Concepts in MySQL

Before you can use MySQL, there are certain rules that must be adhered to by the developer. This will help you to understand the naming conventions in MySQL. It will also help you to understand, how commands are used in MySQL.

Concepts

4.2.1 Naming Conventions

While naming a database and its components, such as tables, columns, and so forth you will have to follow certain rules. For example, you will have to ensure that the name does not exceed the specified length or use illegal characters.

Note: MySQL allows reserved words to be used as names. However, these words have to be enclosed in quotes when used as object names.

The following naming conventions are recommended for MySQL:

- **Legal Characters** - You can use any alphabet (a-z or A-Z) or digits (0-9) in the name. You can also use characters, such as '_' or '$'. A name can start with a digit but cannot contain only digits because you cannot distinguish it from a number. You cannot use '.' within the name because it acts as the separator. For example, db_name.table_name where db_name is the database name and table_name is the name of the table in the database. You also cannot use the separator characters such as '/' or '\'.

- **Length of names** - MySQL restricts the length of the name of the databases, indexes, or columns. It can have a maximum length of 64 characters. However, MySQL allows alias names to be 256 characters in length and compound statement labels are restricted to 16 characters. Examples of compound statement labels include join and conditional operators, such as `INNER JOIN`, `OUTER JOIN`, `LEFT INNER JOIN`, `RIGHT OUTER JOIN`, `FROM`, and `WHERE`.

Note: When you create an alias for a column name in a `CREATE VIEW` statement, the length must not exceed 64 characters.

4.2.2 MySQL Commands

You can enter SQL commands on a single line or split them across multiple lines. All MySQL commands should be terminated by a semicolon. MySQL waits for a semicolon (;) to execute a command. Alternatively, you can use the '\g' command.

4.2.3 Case Sensitive Conventions

In MySQL, case sensitivity varies with the object in question.

Note: Read more about using reserved words as identifiers at `http://dev.mysql.com/doc/refman/5.1/en/reserved-words.html`

- **SQL Statements and Keywords**

  The SQL statements and keywords are not case sensitive. A user can write SQL statements in up-

per or lower case. Consider the statements as shown:

```
SELECT VERSION ( );
select version( );
```

Both the statements are equivalent. Both the commands will return the version of MySQL being used. The following combinations are also valid:

```
Select VERSION ( );
SeLecT version ( );
```

- **Database and Table names**

Case sensitivity of databases, tables, and trigger names depend on the operating system. If the underlying operating system is case sensitive, then the names of databases, tables, and triggers. For example, Microsoft Windows is not case sensitive and so databases, tables, and triggers names in MySQL running on Microsoft Windows are not case-sensitive. However, Linux is case sensitive and so databases, tables, and triggers names under a Linux installation of MySQL are case-sensitive. This is because these objects are stored as files on the file system.

Note: You cannot use different cases while referring to the same object in the same SQL statement. For example, if you refer to a database as 'MyDB', then you cannot refer to it as 'mydb' or 'MYDB' elsewhere in the same statement.

- **Columns, Column Aliases, Index, Stored Routines, and Events Names**

Names of columns, column aliases, index, stored routines, and events are not case sensitive. You can use one of the following SQL statements to display the column 'FNAME' from a SAMPLE table:

```
SELECT FNAME from SAMPLE ( );
SELECT fname from SAMPLE ( );
```

- **Log Files**

Log file names are case sensitive.

4.3 Creating a MySQL database

MySQL is a Relational Database Management System. You can use MySQL to manage relational databases. A relational database consists of different tables in which data is stored such that there is minimal redundancy. A relational database also contains relationships or links to related tables.

Concepts

You can create a database if you have proper privileges.

The syntax for creating a database is:

```
CREATE DATABASE [IF NOT EXISTS] <dbname>;
```

where,

`CREATE DATABASE` – adds a new database to the MySQL instance

`IF NOT EXISTS` – checks for the existence of databases with the same name before adding the new database. MySQL does not allow you to create databases with same names

`dbname` – specifies a name for the new database

For example, to create a database named `EMPLOYEE` for storing employee information, such as personal details, qualifications, salary details, and so on, enter the following command at the command prompt:

```
CREATE DATABASE EMPLOYEE;
```

Figure 4.1 displays the output of the command.

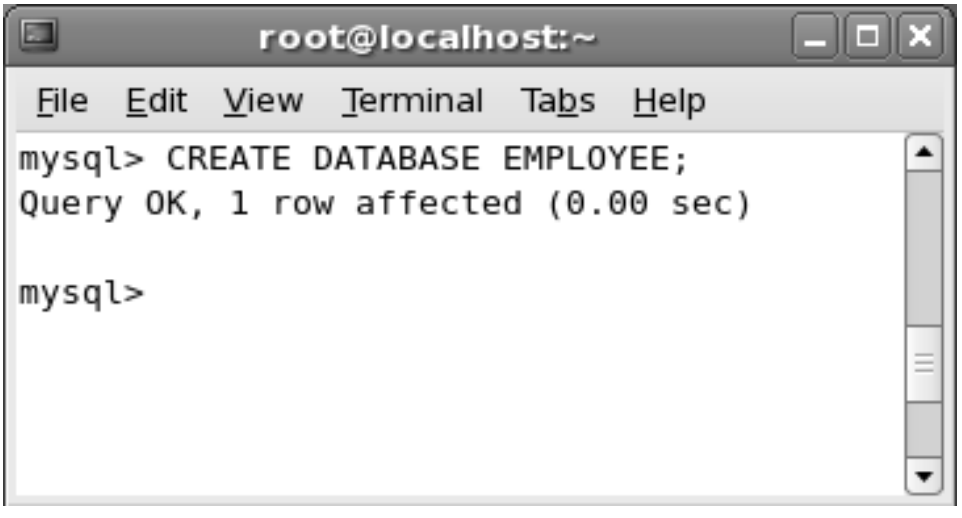


Figure 4.1: Creating a Database

Figure 4.1 displays the number of rows affected, and the time taken to execute the command.

Concepts

4.4 Working with MySQL Tables

As compared to a DBMS, RDBMS stores data in different tables to ensure consistency and faster search and retrieval operations. A table stores data in rows and columns. A field in a table is called a column and row is called a record. A record is also defined as collection of fields. The columns in a table contain different types of data.

4.4.1 Describing Data Types

Data types define the type of the data that will be stored in a column. Data types also define the size and the type of data that can be stored in the column.

MySQL supports different data types to store values. Following are the different categories under which the data can be categorized:

- Numeric data types
- Date and time data types
- String data types
- Complex data types

Numeric Data Type

A numeric data type stores data in number form. For example, a column called Product_Id containing information on the product number can be defined as numeric. There are several numeric data types available in MySQL.

Note: Numbers can be classified as signed and unsigned. A signed number is preceded with negative or positive sign. An unsigned number does not have any sign and is assumed to be positive.

Table 4.1 lists the numeric data types in MySQL.

Numeric Data Type	Storage In Bytes	Description	Synonyms
`BIGINT(length)`	8	Stores unsigned numbers in the range of 0 to 18,446,744,073,709,551,615, and signed numbers in the range of -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	DEC NUMERIC

Concepts

Numeric Data Type	Storage In Bytes	Description	Synonyms
DECIMAL(length, decimal)	Precision +2	Stores floating point numbers and cannot be unsigned	
DOUBLE	8	Stores double precision floating point numbers. Range for a negative number is between -1.7976931348623157E+308 and -2.2250738585072014E-308,0. Range for a positive number is between 2.2250738585072014E-308 and 1.7976931348623157E+308	DOUBLE PRECISION REAL
FLOAT(length, decimal)	4	Stores single precision floating point numbers. Range for a negative number is between -3.402823466E+38 and -1.175494351E-38,0. Range for a positive number is between 1.175494351E-38 and 3.402823466E+38	
INT(length)	4	Stores integer numbers. Range for a signed number is from -2,147,483,648 to 2,147,483,647. Range for an unsigned number is from 0 to 4,294,967,295	INTEGER
MEDIUMINT(length)	3	Stores integer numbers. Range for a signed number is from -8,388,608 to 8,388,607. Range for an unsigned number is from 0 to 16,777,215	
SMALLINT(length)	2	Stores integer numbers. Range for a signed number is from -32,768 to 32,767. Range for an unsigned number is from 0 to 65,535	
TINYINT	1	Stores integer numbers. Range for a signed number is from -128 to 127. Range for an unsigned number is from 0 to 255	BOOLEAN

Table 4.1: Numeric Data Types in MySQL

String Data Type

A string represents a sequence of characters. A string data type is enclosed in single quotes or double quotes. It enables you to enter alphabets from a-z, A-Z, and numbers from 0-9. For example to store employee names in a column called Ename, you can use the string data type. The number of characters specified defines the length of a character string.

Table 4.2 lists the string data types supported by MySQL.

Data Type	Storage In Bytes	Description
CHAR (Length)	Length	Stores character data set and can have a maximum length of 255 characters.
BINARY (Length)	Length	Stores binary byte string upto 255 characters.
VARCHAR ()	Length+1	Stores variable length string values and can have a length from 0 to 65,535. Uses one byte to store 255 characters. Uses additional two bytes to if values require more than 255 characters.
VARBINARY	Length+2	Stores binary strings and has a maximum length same as that of VARCHAR.
BLOB	Length+2	Stores large amount of variable binary data such as images and can have a maximum length of 65,535. Uses an additional byte of storage if values exceed 65,536.
TINYBLOB	Length+1	Stores upto 255 bytes.
MEDIUMBLOB	Length+3	Stores text in size upto 16 MB. Uses additional three bytes of storage if values exceed the specified length.
LONGBLOB	Length+4	Stores text values whose size exceeds 4 GB. Uses additional four bytes of storage if values exceed the specified length.
TEXT	Length+2	Stores characters and can be upto 65,535 characters in length. Uses additional two bytes of storage if values exceed the specified length.
TINYTEXT	Length+1	Stores short text values and can have a length upto 255 characters.
MEDIUMTEXT	Length+3	Stores medium-sized text whose size is upto 16 MB. Uses additional three bytes of storage if values exceed the specified length.
LONGTEXT	Length+4	Stores large text values whose size exceeds 4 GB. Uses additional four bytes of storage if values exceed the specified length.

Table 4.2: String Data Types in MySQL

Date Data Type

A date data type stores date and time information. For instance, you can store the date of birth of employees in date type column. The Date data type column stores date as string value in the default format, 'YYYY-MM-DD'.

Table 4.3 lists the date data types in MySQL.

Data Type	Format	Storage In Bytes	Description
DATE	YYYY-MM-DD	3	Stores a date type of data in the range of January 1,1000 to December 31,9999
DATETIME	YYYY-MM-DD hh:mm:ss	8	Stores date and time. The range for it is '1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIME	hh:mm:ss	3	Stores time
TIMESTAMP	YYYY-MM-DD hh:mm:ss	4	Stores timestamp
YEAR	YYYY	1	Stores a year in the range of 1901 to 2155

Table 4.3: Date Data Types in MySQL

Complex Data Type

Complex data type is a special data type. It defines the list of possible values that can be inserted in a column. For example, you can assign the SET or ENUM data type to a column where a distinct string value will be chosen from a list of predefined values.

Table 4.4 lists the complex data types in MySQL.

Complex Data Type	Storage In Bytes	Description
ENUM	1 ,2	Stores one value out of a possible 65535 number of options in a predefined list of possible values. This data type can contain only one distinct string value from the predefined list of values. The values are represented internally as integers. Eg: ENUM('abc','def','ghi')
SET	1,2, 3, 4 or 8	Stores a list of values from a predefined set of values. Maximum of 64 values can be stored in a SET data type column. The SET data type can contain zero or more values that can be specified in the list.

Table 4.4: Complex Data Types in MySQL

The SET data type is similar to ENUM. While an ENUM data type restricts you to selecting one of many predefined values, SET data type allows you to choose more than one of the predefined values.

Consider the following example:

```
CREATE TABLE MY_CITY
(CITY_ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
CITY_NAME ENUM('CHICAGO','DALLAS','COLUMBUS'));
```

The code will create a table MY_CITY with two columns, CITY_ID and CITY_NAME. The data type for CITY_NAME has been defined as ENUM. When you add records to the table using the INSERT command, this data type will allow you to use one of the city names specified in the ENUM list as the column value when you add rows to the table.

For example, you can insert CHICAGO, DALLAS, or COLUMBUS as values for the column CITY_NAME, but cannot insert NEW YORK as a value because it is not a part of the ENUM list. When you use an invalid value for the column CITY_NAME in the INSERT query, the query is ignored.

Consider the same code with SET as the data type for the CITY_NAME field.

```
CREATE TABLE MY_CITY
(CITY_ID INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
CITY_NAME SET('CHICAGO','DALLAS','COLUMBUS'));
```

When you assign SET data type to the CITY_NAME field, you can use one or more city names from the SET list as a column value when you add rows to the table.

For example, you can insert ('CHICAGO','DALLAS'), ('CHICAGO','COLUMBUS') or ('CHICAGO','COLUMBUS','DALLAS') as values for the column CITY_NAME. If you specify an invalid value, such as NEW YORK, in the INSERT query for the CITY_NAME column, MySQL ignores the value and displays a warning. The command to insert values would be:

```
INSERT INTO MY_CITY (CITY_NAME) VALUES('CHICAGO','DALLAS');
```

The SET data type uses storage space in the form of bytes to store the number of elements defined in the data type. Table 4.5 lists the storage structure of SET data type depending upon the number of elements.

Number Of Elements	Storage In Bytes
1-8	1
9-16	2
17-24	3
25-32	4
33-64	8

Table 4.5: Storage Structure of SET Data Type

4.4.2 Creating Tables

A relational database contains several related tables. The table name can be upto 64 characters in length. The maximum length of a field name is 64 characters. Also, you can specify the table type or the

storage engine while creating the table.

MySQL supports two types of tables:

- **Transaction-safe tables:** enables you to execute transactions or data manipulation commands without losing data. You can undo the changes made to transaction-safe tables.
- **Non-transaction safe tables:** also allows you to execute transaction or data manipulation commands. However, you cannot undo the changes made to non-transaction safe tables because the changes are permanent.

Following are the features of transaction-safe tables:

- Provides safety from data loss and allows MySQL to recover data, from a backup and the transaction log, in case of a crash or hardware failure
- Enables to combine and execute statements using the `COMMIT` command
- Enables to undo the changes made to the data using the `ROLLBACK` command
- Enables concurrency while simultaneously reading data from tables
- Examples of transaction-safe tables are InnoDB

Following are the features of non-transaction safe tables:

- Faster than the transaction-safe tables as no transaction overhead is required
- Require less disk space
- Require less memory for updates

Storage Engines

MySQL supports storage engines for different table types. Storage engines handle both transaction-safe and non-transaction safe tables. Table 4.6 lists the storage engines supported in MySQL:

Storage Engine	Feature
`MyISAM`	Is the default storage engine to handle tables
`InnoDB`	Provides commit, rollback, and crash recovery characteristics to secure data
`MERGE`	Allows developers to create tables with identical column and index information

Storage Engine	Feature
MEMORY (HEAP)	Allows developers to create tables that store contents in the memory. MEMORY tables can have upto 64 indexes per table, 16 columns per index, and a maximum key length of 3072 bytes
EXAMPLE	Allows developers to create dummy tables where no data will be stored
FEDERATED	Provides access to data from tables in remote databases without copying data to the local server. This engine does not create a replica or a copy of the database or tables to the local server while executing a query on the remote server
ARCHIVE	Allows developers to store large amount of data without indexing
CSV	Allows developers to store data in text files using comma-separated values
BLACKHOLE	Allows developers to accept data without storing. Tables that have this storage engine will always return an empty result. This storage engine can be used in distributed database design where data is automatically replicated, but not stored locally

Table 4.6: Storage Engines in MySQL

You will use the CREATE command to add a new table to a database.

The syntax for CREATE command is:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name (create_clause,...) [table_
options] [IGNORE | REPLACE] [AS] Select ...];
```

where,

CREATE TABLE – adds a new table to the database

tbl_name – specifies a name for the table

Table 4.7 lists the description for the options in the CREATE TABLE syntax:

Option	Description
TEMPORARY	Table exists as long as the current user is connected
IF NOT EXISTS	Creates table only if it does not exists in the database
create_clause	Defines column
table_options	Creates different types of tables such as ISAM, INNODB, and so on
IGNORE	If table already exists system ignores and does not replace the previous table
REPLACE	Replaces the table if a table with the same name exists in the database
Select	Enables to copy records from an existing table

Table 4.7: CREATE TABLE Options

Concepts

The syntax for create_clause in the CREATE TABLE command is:

```
column type [NOT NULL | NULL][DEFAULT value][AUTO_INCREMENT][PRIMARY
KEY][REFERENCE];
```

where,

`column type` – defines the data type for the column

Table 4.8 lists the options in the `create_clause`.

Option	Description
`AUTO_INCREMENT`	Specifies that the column value must be auto incremented. This option works only if the columns contain positive values.
`DEFAULT value`	Inserts the specified value when no value for that column is entered. The default value has to be a constant. The values cannot be a function or expression.
`NOT NULL`	Specifies that a column cannot contain a null value. When a null value is inserted, the system prompts with an error message.
`NULL`	Specifies that the column can contain null values.
`PRIMARY KEY`	Defines a column as the primary key while creating a table.
`REFERENCES`	Assigns the InnoDB storage engine.

Table 4.8: Options in the create_clause

The syntax for `REFERENCE` options is:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name
REFERENCE tbl_name[(index_col_name,….)][MATCH FULL |MATCH PARTIAL| MATCH
SIMPLE][ON DELETE reference_option][ON UPDATE reference_option];
```

where,

`CREATE TABLE` – adds a new table to the database

`tbl_name` – specifies a name for the table

`REFERENCE` – defines a relationship between the index and another table

`tbl_name` – specifies the name for the referenced table

`index_col_name` – specifies the column to be indexed

Table 4.9 lists the options in the REFERENCE clause.

Option	Description
index_col_name	Defines an index on the column
ON DELETE	Used for cascading tables
ON UPDATE	Used for cascading tables

Table 4.9: Options in the REFERENCE clause

While creating a table, you can also specify the storage engine.

Consider an example of a customer table with columns, such as customer_id and name and the table is assigned to a storage engine, INNODB.

```
CREATE TABLE customer (customer_id INT NOT NULL,name CHAR (15)) ENGINE =
INNODB;
```

Similarly, you can create a table of the required type.

You will create three tables namely EMP_DETAILS, EMP_DEPARTMENT, and SALARY_DETAILS in the EMPLOYEE database.

The data stored in the three respective tables are as follows:

- **EMP_DETAILS:** Stores the personal details of the employees, such as identification number, name, address, and phone number.
- **EMP_DEPARTMENT:** Stores the department information, such as identification number, department, date of joining, and designation of the employee.
- **SALARY_DETAILS:** Stores the salary information of the employee, such as identification number, basic salary, house rent allowance, traveling allowance, and gross salary.

You will first have to select the database to create the tables. You will select the EMPLOYEE database with the help of USE command.

The syntax for USE command is:

```
USE DATABASE;
```

where,

USE – switches to the specified database

Concepts

`DATABASE` - specifies the name of the database

The column `E_ID` contains the employee code, so you will use the `INT` data type. Similarly, you will define suitable data types for each column, depending on the data.

Accordingly, create an **`EMP_DETAILS`** table with the structure listed in table 4.10.

Column Name	Data Type	Length
E_ID	INT	3
E_FNAME	CHAR	10
E_LNAME	CHAR	15
E_ADDRESS	CHAR	15
E_PHONE_NO	INT	6

Table 4.10: EMP_DETAILS Table Structure

To create the `EMP_DETAILS` table, enter the following command at the command prompt:

```
CREATE TABLE EMP_DETAILS(E_ID INT(3),E_FNAME CHAR(10), E_LNAME CHAR(15),
E_ADDRESS CHAR(15),E_PHONE_NO INT(6));
```

Figure 4.2 displays the output of the command.

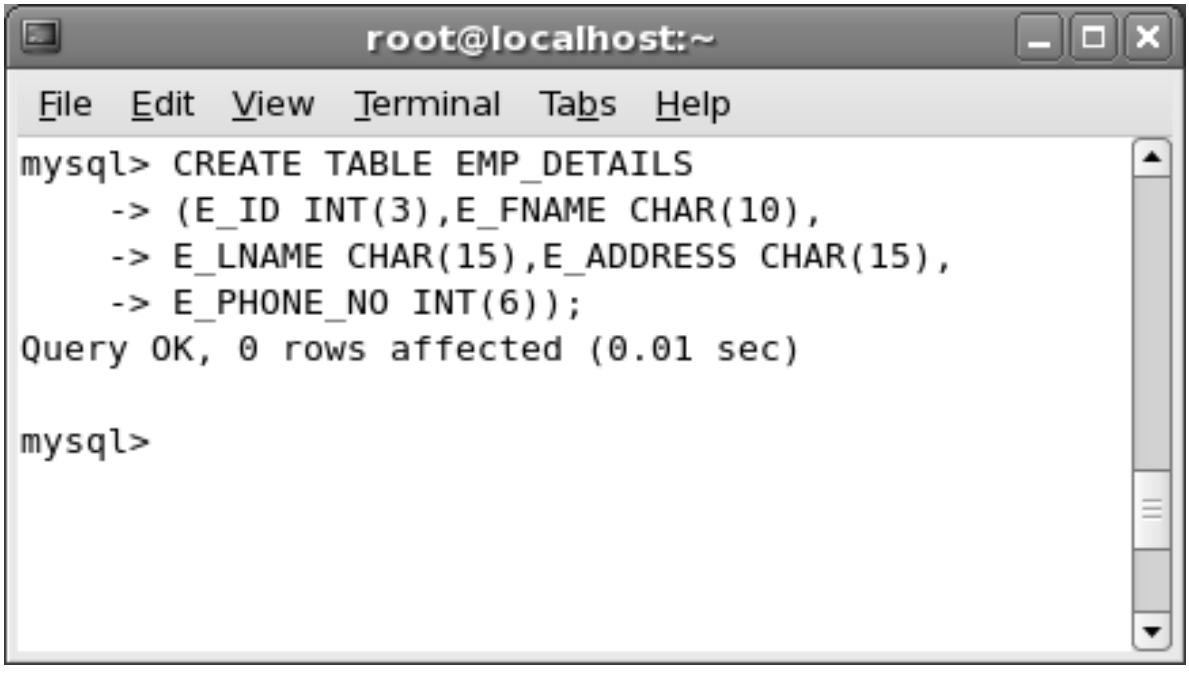


Figure 4.2: Creating a Table

MySQL provides the `DESCRIBE` command to view the structure of a table.

The syntax for the DESCRIBE command is:

```
DESCRIBE table [column];
```

Or

```
DESC table [column];
```

where,

`DESCRIBE` – displays the field structure of the table

`table` – specifies the name of the table to display the structure

`column` – displays information for a specific column of the table

To view the structure of the `EMP_DETAILS` table, enter the following command at the prompt:

```
DESC EMP_DETAILS;
```

Concepts

Figure 4.3 displays the output of the DESCRIBE command.

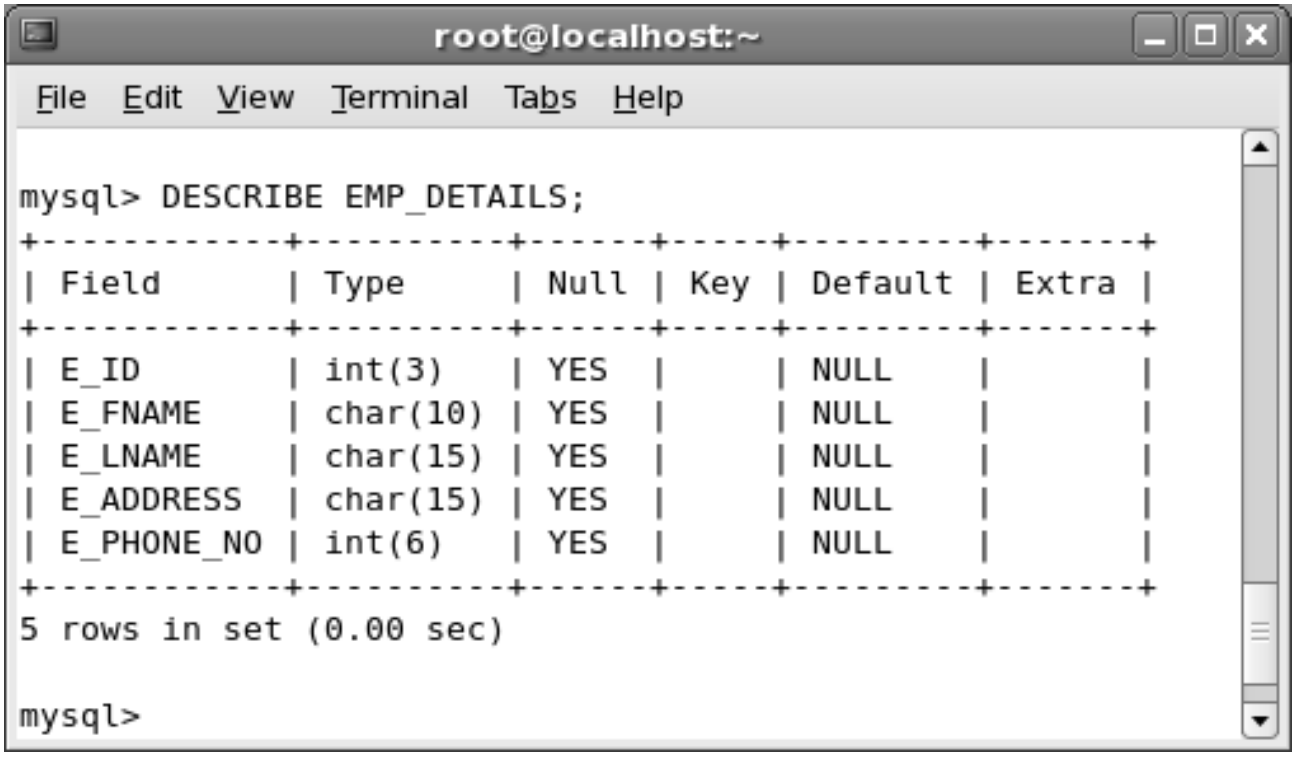


Figure 4.3: DESCRIBE Command

Figure 4.3 displays the following information for a table:

- Columns in the table
- Data type used for a column
- Acceptance of null values for a column. A column having a NULL value indicates that the value is not available and is not known
- Type of key such as Primary key
- Default value for a column
- Extra information or unique characteristics of a column
- Number of rows in the table

Now, you will create an EMP_DEPARTMENT table with the structure listed in table 4.11.

Column Name	Data Type	Length
E_ID	INT	3
D_NAME	VARCHAR	15
DATE_OF_JOIN	DATE	
DESIGNATION	CHAR	6

Table 4.11: Structure of EMP_DEPARTMENT Table

To create the EMP_DEPARTMENT table, enter the following command at the command prompt:

```
CREATE TABLE EMP_DEPARTMENT(E_ID INT(3),D_NAME VARCHAR(15), DATE_OF_JOIN
DATE, DESIGNATION CHAR(6));
```

Figure 4.4 displays the output of the command.

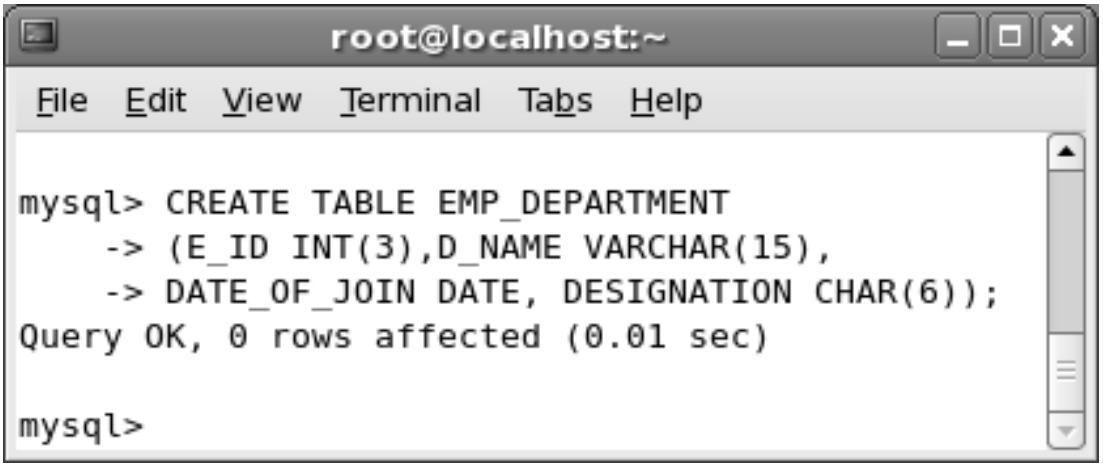


Figure 4.4: Creating an EMP_DEPARTMENT Table

Similarly, create a SALARY_DETAILS table with the structure listed in table 4.12.

Column Name	Data Type	Length
E_ID	INT	3
BASIC_SAL	DECIMAL	(12,2)
HRA	DECIMAL	(7,2)
TA	DECIMAL	(7,2)
GROSS_SAL	REAL	(15,2)

Table 4.12: Structure of SALARY_DETAILS Table

In table 4.12, the data type of BASIC_SAL field is DECIMAL.

The length of the field has been specified as (12, 2). In this instance, the value before the comma in the bracket defines the total length of the column including the decimal point. The value after the comma specifies the number of digits after the decimal point.

To create a `SALARY_DETAILS` table, enter the following command at the command prompt:

```
CREATE TABLE SALARY_DETAILS(E_ID INT(3),BASIC_SALARY DECIMAL(12,5), HRA
DECIMAL(7,2), TA DECIMAL(7,2), GROSS_SAL DECIMAL(15,2));
```

Figure 4.5 displays the output of the command.

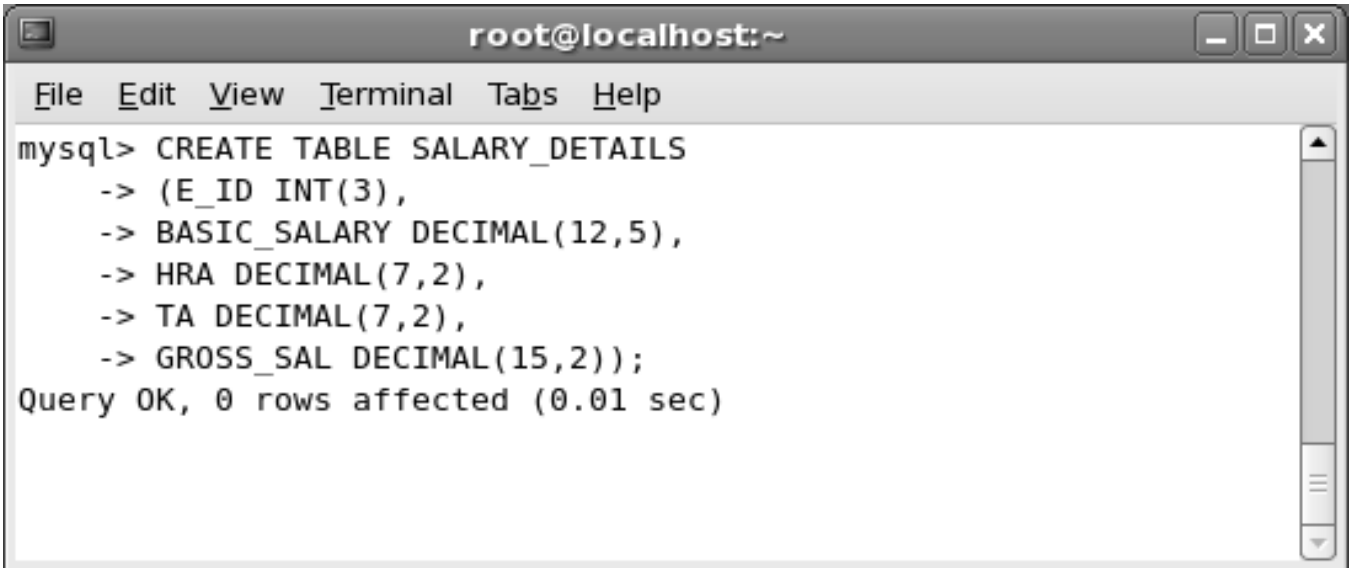


Figure 4.5: SALARY_DETAILS Table

4.5 Normalization

Normalization is the process of structuring the data in a table to minimize duplication and inconsistency. Normalization requires breaking down of a larger table into two or more smaller tables. The tables are linked by defining a relationship between them. Normalization improves the clarity in a database. You can normalize a database by using a set of rules while creating the database.

You must understand the following terms before normalizing a database:

- **Entity** - represents an object about which you want to store the information
- **Attributes** - represents the component of entity type or identifying qualities
- **Domain** - defines the range for the attributes

- **Relationship** - represents association between entities

4.5.1 Defining Keys

Keys help to establish a relationship between columns. You can define a field as a key. These keys uniquely identify a record. The different types of keys include:

- **Primary Key** - Is used as the unique identifier and has only one attribute
- **Composite Key** - Is a primary key having more than one attribute
- **Foreign Key** - Is a column or a set of column in a table which refers to a column or a set of column in another table

To normalize a database, you will use a sample table where you store department number, department name, employee identification number, employee name, employee category, and wages of the employee per hour.

Table 4.13 lists the data stored in a sample table for normalization.

D_No	D_Name	E_Id	E_Name	Emp_Category	Wages
10	Research	41	Charles	I	100
		46	Jack	II	60
		50	Peter	III	40
20	Marketing	51	Mike	I	100
		70	George	III	40

Table 4.13: Sample Table Data for Normalization

4.5.2 Different Forms of Normalization

You can apply different forms of normalization on a database to remove redundancy and improve clarity. You must follow certain set of rules while normalizing the database so that the database contains a minimum number of duplicate records.

- **First Normal Form or 1NF**

  An entity is said to be in the first normal form, if all the attributes are single valued. In other words, there should not be any repeating attributes.

The following are the rules for a table to be in the first normal form:

- **All attributes are atomic:** Field holds a value for one object.
- **All columns have only one instance:** Columns must be defined to contain a specific type of record. For example, you can store the first and the last name of employees in different columns, such as `F _ Name` and `L _ Name`. This makes the data retrieval process easier.

Table 4.14 displays the sample table structure after applying the First Normal Form.

D_No	D_Name	E_Id	E_Name	Emp_Category	Wages
10	Research	41	Charles	I	100
10	Research	50	Peter	II	60
10	Research	46	Jack	III	40
20	Marketing	51	Mike	I	100
20	Marketing	70	George	III	40

Table 4.14: First Normal Form

In Table 4.14, the `D_NO` does not uniquely identify a record, so the primary key must be a combination of `D_NO` and `E_ID`. These two fields are used to uniquely identify the records.

➢ **Second Normal Form or 2NF**

An entity is said to be in the second normal form, if it is in the first normal form and all non key attributes entirely depend on the unique identifier of the entity.

After normalizing the sample table using the first normal form, the value for `D_NO` repeats several times. Also, the field `D_NAME` depends only on the `D_NO` field which is part of the primary key and not the entire primary key. Therefore, you will apply the second normal form and split the tables, as shown in tables 4.15, 4.16, and 4.17.

EMP_DEPARTMENT TABLE

D_No	E_Id
10	41
10	50
10	46
20	51
20	70

Table 4.15: EMP_DEPARTMENT Table in Second Normal Form

EMPLOYEE TABLE:

E_Id	E_Name	Emp_Category	Wages
41	Charles	I	100
50	Peter	II	60
46	Jack	III	40
51	Mike	I	100
70	George	III	40

Table 4.16: EMPLOYEE Table in Second Normal Form

DEPARTMENT TABLE:

D_No	D_Name
10	Marketing
20	Research

Table 4.17: DEPARTMENT Table in Second Normal Form

- **Third Normal Form or 3NF**

An entity is said to be in the third normal form, if it is already in the second normal form and no non-identifying attributes are dependent on any other non-identifying attributes.

Consider the following data in the `EMPLOYEE` table, as shown in table 4.18.

E_Id	E_Name	Emp_Category	Wages
41	Charles	I	100
50	Peter	II	60
46	Jack	III	60
51	Mike	I	60
70	George	III	40

Table 4.18: EMPLOYEE Table

In table 4.18, the employee Mike is of `EMP_CATEGORY` I and the wages should be `100` instead of `60`. You are redundantly storing the `EMP_CATEGORY` and WAGES details. All the attributes should depend only on the primary key according to the third normal form. Therefore, you will split the table, as shown in tables 4.19 and 4.20.

EMPLOYEE table:

E_Id	E_Name	Emp_Category
41	Charles	I
50	Peter	II
46	Jack	III
51	Mike	I
70	George	III

Table 4.19: EMPLOYEE Table in Third Normal Form

RATE table:

Emp_Category	Wages
I	100
II	60
III	40

Table 4.20: EMPLOYEE Table Split in Third Normal Form

- **Boyce-Codd Normal Form or BCNF**

  An entity is said to be in the Boyce-Codd normal form, if it is already in the third normal form and contains only one unique identifier.

  The tables in the earlier example are already in the Boyce-Codd normal form.

4.6 Database Concepts

Referential integrity is a feature that prevents you from storing inconsistent data. You implement referential integrity when you define a relationship between two tables in a database. Consider an example, where Table Y has a foreign key pointing to a field in Table X, referential integrity prevents addition of a new record in Table Y that cannot be linked to Table X. In addition, whenever you delete a record from Table X, any record linked to that record in Table Y is also deleted. Also, when you alter or update a linked field in Table X, all the records linked to it in Table Y are updated accordingly.

You can use indexing on databases to make search operations faster. Normally, you index the fields in a table, on which the search is to be performed. Also, you can index several fields in a table at a time.

4.6.1 Using Indexes

Indexes are lists that specify the location of data in a table within a database. Indexes support faster data

storage and retrieval capabilities. Indexes in a database are similar to the indexes in books. Consider the example where you use the index to find some information on a topic in a book instead of reading the entire book. Indexes in a database use the same concept.

You can define indexes on a field in a table within a database. The only disadvantage of indexing is that it uses more storage space. In addition, data manipulation commands, such as insert, update, and delete require longer duration to execute.

You can create an index in a table by indexing the fields. You must define the index on a field that will sort the records in the table. The primary key serves as the index key when the index key is not specified explicitly.

You must remember the following points while indexing fields in a table:

- Fields should have unique values
- Fields on which the table is to be sorted
- Fields should have data types such as text, number, date or time

4.6.2 Referential Integrity

You can use referential integrity to maintain consistency of records in different tables. It means that when a record in a table refers to a corresponding record in another table, that corresponding record should exist. Referential integrity can be defined by linking a foreign key of a table to the primary key of another. This means that a foreign key of one table must have a corresponding row and primary key in the related table.

Consider an example, where you have two tables, `EMP_DETAILS` and `EMP_SALARY`. Employees can be uniquely identified with the `E_ID`. Hence this is the primary key in the `EMP_DETAILS` table. The salary details stored in the `EMP_SALARY` table will have `E_ID` as the foreign key.

In this example, referential integrity of the database will not allow you to make inconsistent changes in either of these two tables.

Consider the following records stored in the `EMP_DETAILS` and `EMP_DEPARTMENT` table as shown in tables 4.21 and 4.22 respectively.

EMP_DETAILS

E_ID	E_FNAME	E_LNAME	E_ADDRESS	E_PHONE_NO
101	Jack	Williams		765432
102	Peter	Adams		712832

Table 4.21: EMP_DETAILS Table

Concepts

EMP_DEPARTMENT

E_ID	D_NAME	DATE_OF_JOIN	DESIGNATION
101	Sales	2001-01-01	Supervisor
102	Sales	2000-02-05	Worker
103	Research	2003-04-06	Worker

Table 4.22: EMP_DEPARTMENT Table

You can identify inconsistency in the data from the tables 4.21 and 4.22. The employee with the `E_ID` as 103 does not have a corresponding entry in the `EMP_DETAILS`. You will use referential integrity to remove inconsistency from the tables.

To implement referential integrity, you will have to use a foreign key.

The syntax for defining a foreign key while creating a table is:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table (create_clause,...) [table_
options] [IGNORE [REPLACE] select],FOREIGN KEY(col_name) REFERENCES <table_
name>(referenced_column);
```

where,

`FOREIGN KEY` – defines a foreign key

`col_name` – specifies the name of the column to be used for the foreign key

`REFERENCES` – defines a relationship with other tables

`table_name` – specifies the name of the table to define a relationship

`referenced_column` – specifies the column of the table to define a relationship

Consider the following example where the `E_ID` is defined as a primary key in the `EMP_DETAILS` table.

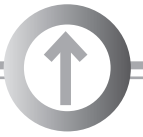
```
CREATE TABLE EMP_DETAILS (E_ID INT,E_FNAME CHAR (10),E_LNAME CHAR(15),ADDRESS
CHAR(15),E_PHONE_NO INT(6),PRIMARY KEY (E_ID));
```

Consider the following example, where the `E_ID` is defined as a Foreign key referencing `E_ID` column of `EMP_DETAILS` table.

```
CREATE TABLE EMP_DEPARTMENT (E_ID INT, D_NAME CHAR(),DATE_OF_JOIN
DATE,DESIGNATION CHAR(6),FOREIGN KEY (E_ID) REFERENCES EMP_DETAILS (E_ID));
```

In the example, `EMP_DETAILS` is the master table and `EMP_DEPARTMENT` is the child table. This relationship enables to store consistent records in the tables. Referential integrity monitors addition and deletion of records from these tables. MySQL will generate an error message when you attempt to add a new record in the child table that does not have a corresponding record in the parent table. Similarly, MySQL will generate an error message when you delete a record from the child table that has a corresponding entry in the master table.

Concepts



Summary

- Databases are used to store information and can contain more than one table.
- A table consists of rows and columns. A column in a table is known as a field and row is known as a record.
- MySQL supports data types, such as numeric, string, date, and complex.
- Numeric data type column consists of different types of numbers. A number can be classified as signed or unsigned. A signed number stores a negative or positive value.
- String data type columns consist of data in the form of characters.
- Date data type column have date values entered as string values in the form of '`YYYY-MM-DD`' format.
- MySQL provides the `CREATE` command to generate new databases and tables.
- Normalization removes redundancies from the database.
- A Primary key is a field in the table which is used as the unique identifier. A primary key having more than one attribute is called Composite key.
- A Foreign key in a table is a Primary Key in another table.
- An entity is said to be in the first normal form if all the attributes are single valued.
- An entity is said to be in the second normal form, if it is in the first normal and all non key attributes entirely depend on the unique identifier of the entity.
- An entity is said to be in the third normal form, if it is in the second normal form and no non-identifying attributes are dependent on any other non-identifying attributes.
- An entity is said to be in the Boyce-Codd normal form if it is already in the third normal form and only one unique identifier exists.
- Referential integrity means that when you have a record in a table that refers to corresponding record in another table then that particular record must exist.
- Indexing enables faster search and retrieval operations in a database.

Check Your Progress

1. The Boolean data type is similar to _____.

   a. TINYTEXT

   b. MEDIUMTEXT

   c. TINYINT

   d. MEDIUMINT

2. A foreign key in a table can also be a _____.

   a. Primary key in another table

   b. Composite key in another table

   c. Primary key and composite key in another table

   d. Foreign key in another table

3. An entity is said to be in the first normal form if _____.

   a. all the attributes are single valued

   b. all the attributes are multiple valued

   c. all the attributes are not null valued

   d. there is no top to bottom ordering of rows

4. Which type of key is used by the child table for referential integrity?

   a. Primary key

   b. Foreign key

   c. Composite key

   d. Indexes

Check Your Progress

5. Which of the following options helps to remove redundancies from a database?

a. Referential integrity

b. Indexing

c. Normalization

d. Boyce-Codd Rule

Session 5

Using MySQL (Lab)

Objectives

At the end of this session, the student will be able to:

- *Create a database.*
- *Create a table using different data types.*

The steps given in this session are detailed, comprehensive, and carefully thought through in order to meet the learning objectives and understand the tool completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Creating a Database

A relational database consists of several tables that store a particular set of data. Data in all these tables are related to each other. You will create a `LIBRARY` database to store details about the books, user's category, and date of issue.

1. **Open the Linux terminal and start MySQL.**
2. **To create the LIBRARY database to store details about the books, user's category, and date of issue, enter the following command at the command prompt:**

```
CREATE DATABASE LIBRARY;
```

Figure 5.1 displays the output of the command.

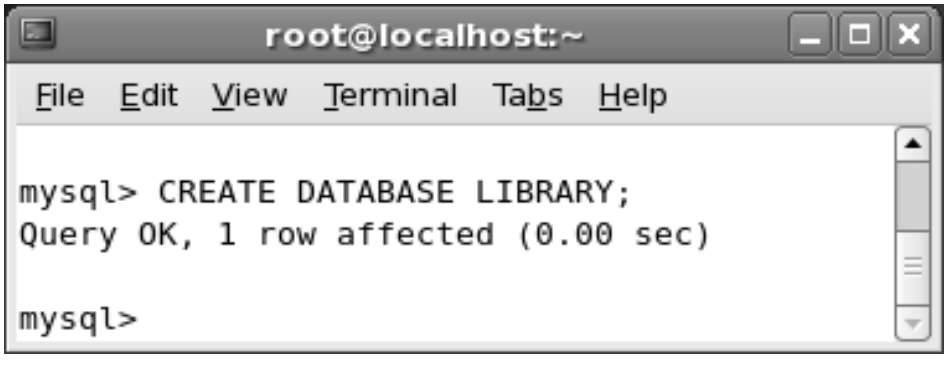


Figure 5.1: Creation of a Database

Creating a Table Using Different Data Types

Tables store information in a row and column format. A row is known as a record and a column is called a field. You can use different data types for fields depending on the data stored in the field. Create the following tables in the `LIBRARY` database:

- `BOOK_DETAILS`
- `USER_DETAILS`
- `CATEGORY`
- `ISSUE_DETAILS`

3. **In order to create tables in the LIBRARY database, you must activate the database. To activate the LIBRARY database, enter the following command at the command prompt:**

```
USE LIBRARY;
```

Figure 5.2 displays the output of the command.

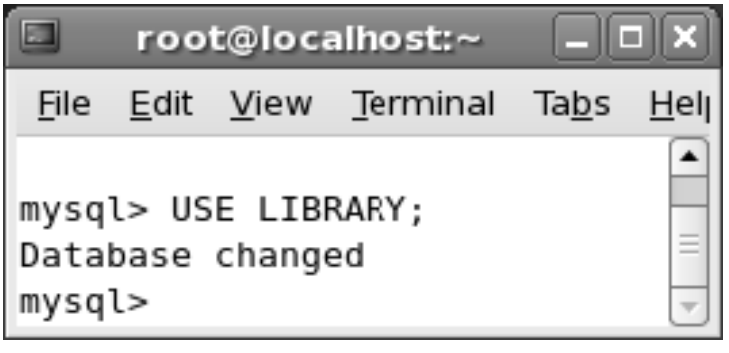


Figure 5.2: Activation of the LIBRARY Database

First, you will create a table to store the book details, such as book identification number, book name, author, publication, edition, and description about the book. The `BOOK_DETAILS` table has the structure as listed in table 5.1.

Field Name	Data Type
BOOK_ID	INT
BOOKNAME	VARCHAR(35)
AUTHOR	CHAR(25)
EDITION	CHAR(15)

Field Name	Data Type
DESCRIPTION	TEXT

Table 5.1: BOOK_DETAILS Table Structure

4. **To create the BOOK_DETAILS table to store the book details, such as book identification number, book name, author, edition, and description, enter the following command at the command prompt:**

```
CREATE TABLE BOOK_DETAILS(BOOK_ID INT(3),BOOK_NAME VARCHAR(35)
, AUTHOR CHAR(25), EDITION CHAR(15), DESCRIPTION TEXT);
```

Figure 5.3 displays the output of the command.

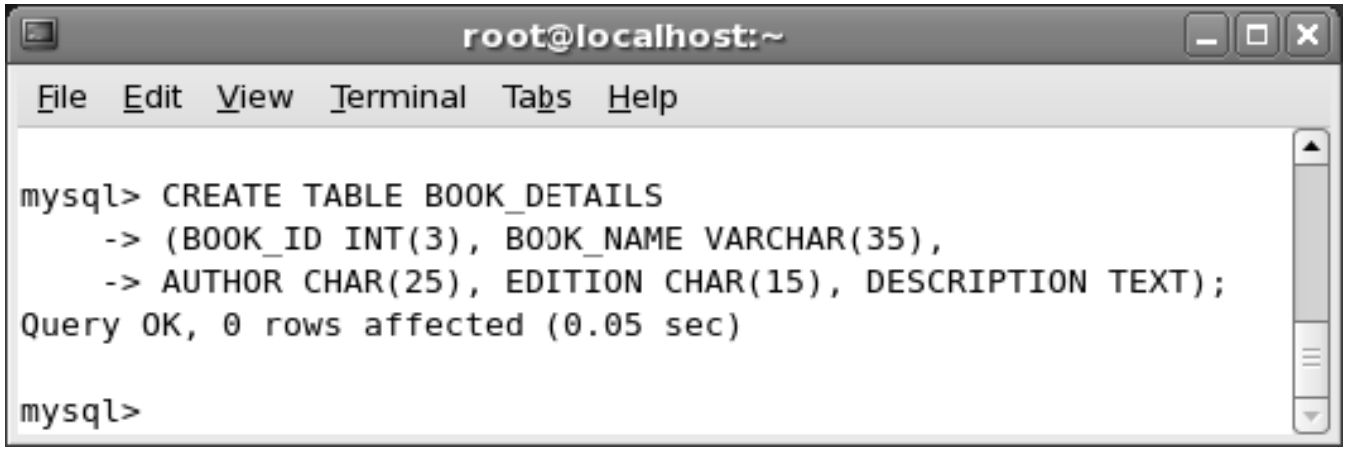


Figure 5.3: Creation of BOOKDETAILS Table

5. **To view the structure of the BOOK_DETAILS table, enter the following command at the command prompt:**

```
DESC BOOK_DETAILS;
```

Figure 5.4 displays the output of the command.

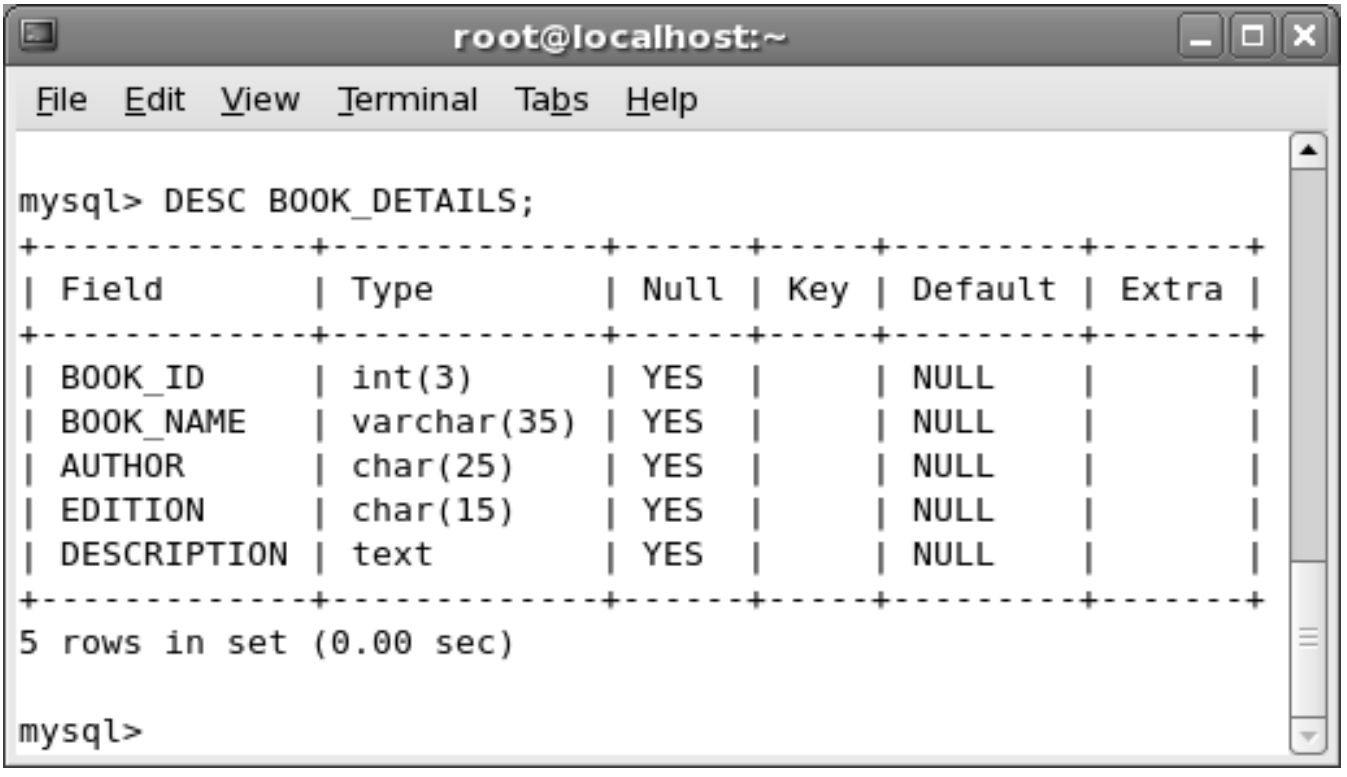


Figure 5.4: BOOK_DETAILS Table Structure

Create a table to store the user details, such as the user identification number, first name, last name, address, and phone number. The `USER_DETAILS` table has the structure as listed in table 5.2.

Field Name	Data Type
USER_ID	INT(3)
NAME	VARCHAR(25)
LOCATION	CHAR(15)
PHONE_NO	INT(6)

Table 5.2: USER_DETAILS Table Structure

6. **To create a USER_DETAILS table to store the user details, such as the user identification number, first name, last name, address, and phone number, enter the following command at the command prompt:**

```
CREATE TABLE USER_DETAILS(USER_ID INT(3),NAME VARCHAR(35),

LOCATION CHAR(25), PHONE_NO INT(6));
```

Figure 5.5 displays the output of the command.

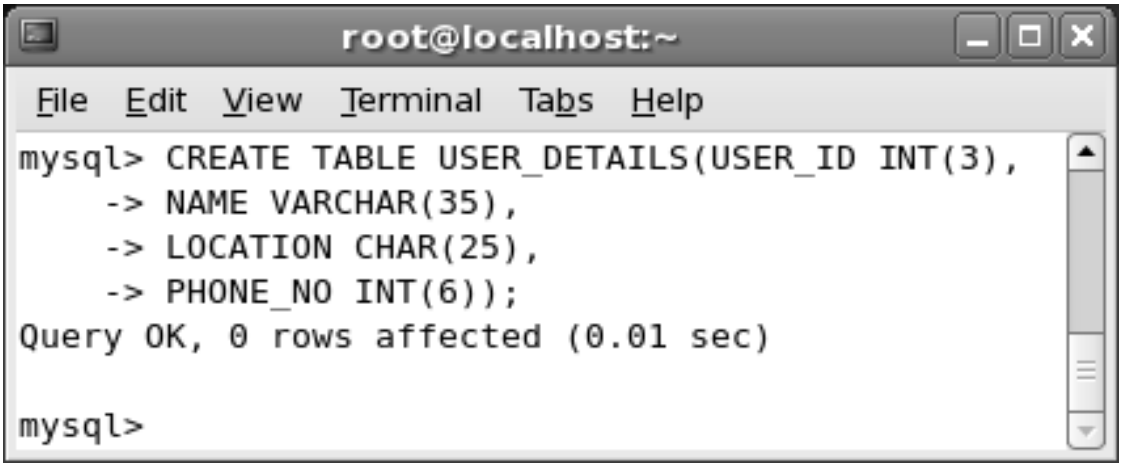


Figure 5.5: Creation of USER_DETAILS Table

Now, create a category table to store information, such as user identification number, membership type of the user, facility provided to the user, and the issue status (whether any book is issued to the user or not). The `CATEGORY` table has the structure as listed in table 5.3.

Field Name	Data Type
USER_ID	INT(3)
TYPE	VARCHAR(35)
FACILITY	CHAR(25)
ISSUE_STATUS	BOOLEAN

Table 5.3: CATEGORY Table Structure

7. **To create a CATEGORY table to store information, such as user identification number, membership type of the user, facility provided to the user, and the issue status, enter the following command at the command prompt:**

```
CREATE TABLE CATEGORY (USER_ID INT(3),TYPE VARCHAR(25),

FACILITY CHAR(25), ISSUE_STATUS BOOLEAN);
```

Figure 5.6 displays the output of the command.

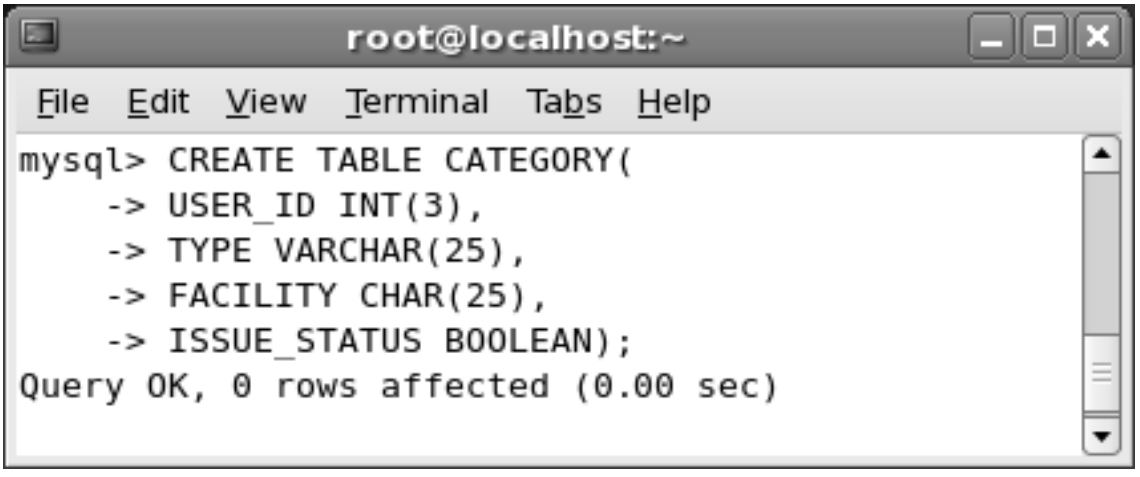


Figure 5.6: Creation of CATEGORY Table

Finally, create an `ISSUE_DETAILS` table to store the user ID, book ID, book name, date of issue, date of return, and fine. The `ISSUE_DETAILS` table has the structure as listed in table 5.4.

Field Name	Data Type
USER_ID	INT(3)
BOOK_ID	INT(3)
BOOK_NAME	CHAR(35)
DATE_OF_ISSUE	DATE
DATE_OF_RETURN	DATE
FINE	DECIMAL(5,2)

Table 5.4: ISSUE_DETAILS Table Structure

8. **To create a ISSUE_DETAILS table to store the user ID, book ID, book name, date of issue, date of return, and fine, enter the following command at the command prompt:**

```
CREATE TABLE ISSUE_DETAILS (USER_ID INT(3),BOOK_ID INT(25)
, BOOK_NAME CHAR (35), DATE_OF_ISSUE DATE, DATE_OF_RETURN DATE, FINE
DECIMAL (5,2));
```

Figure 5.7 displays the output of the command.

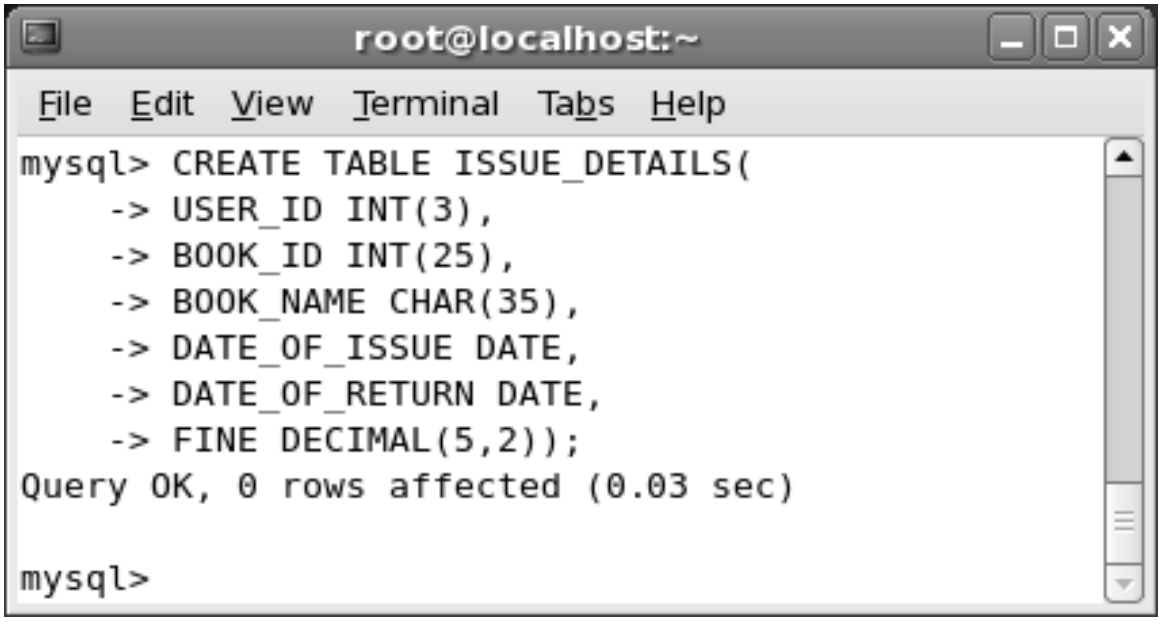


Figure 5.7: Creation of ISSUE_DETAILS Table

Do It Yourself

Create a database for storing the product sales detail information. The `Product_Sales` database contains two tables, `Stock_Details` and `Sales_Details`. The `Stock_Details` table contains fields, such as code, name, description, cost price, and date of manufacture of the product.

1. Create a `PRODUCT_SALES` database.

2. Create a `STOCK_DETAILS` table having the structure as listed in table 5.5.

Field Name	Data Type
Product_Code	INT(4)
Product_Name	CHAR(15)
Description	TEXT
Cost_Price	DECIMAL(8,2)
Date_of_Manufacture	DATE

Table 5.5: STOCK_DETAILS Table Structure

3. Create a `SALES_DETAILS` table having the structure as listed in table 5.6.

Field Name	Data Type
Product _Code	INT(4)
Selling Price	DECIMAL(8,2)
Qty	INT(4)
Units Sold	INT(4)
Month	SET('JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN', 'JUL', 'AUG', 'SEP', 'OCT', 'NOV', 'DEC')
Location	TEXT

Table 5.6: SALES_DETAILS Table Structure

Session 6

Implementing SQL Queries Using MySQL - I

Concepts

Objectives

At the end of this session, the student will be able to:

- *Describe the commands to view and alter a database.*
- *Explain the commands to retrieve data from a table.*
- *Describe the commands to modify the table definitions.*
- *Describe the commands to delete the table definitions.*

6.1 Introduction

MySQL supports Structured Query Language (SQL) queries to handle data. You can use queries in MySQL to create databases and tables, insert data into tables, and retrieve data from tables. The SQL queries are short and single line statements. There may be multiple statements in a single query.

In this session, you will learn to write queries to view tables in a database and modify the database. You will also learn to use queries to retrieve data from a table and modify the table structure. In addition, you will learn to delete a table from the database.

6.2 Viewing and Altering Databases on the File System

While working with databases, you may need information about the number of databases present in the server, the names and other details of these databases, or you may need details about the structure of any database. You may also have to modify the structure of the database. MySQL enables you to view and alter an existing database on the file system through queries.

6.2.1 Viewing Information about Databases on the File System

Before executing any query in MySQL, you can list the databases that are managed by the server. You will use the `SHOW` command to display the list of databases and index keys along with the privileges. The `SHOW` command provides information about databases, tables, columns, and server status.

To view a list of databases present on the server, use the following syntax:

```
SHOW DATABASES;
```

Concepts

where,

`SHOW` – displays the object specified in the clause

`DATABASES` – displays the databases present in the instance of MySQL

Figure 6.1 displays the output of the `SHOW DATABASES` command.

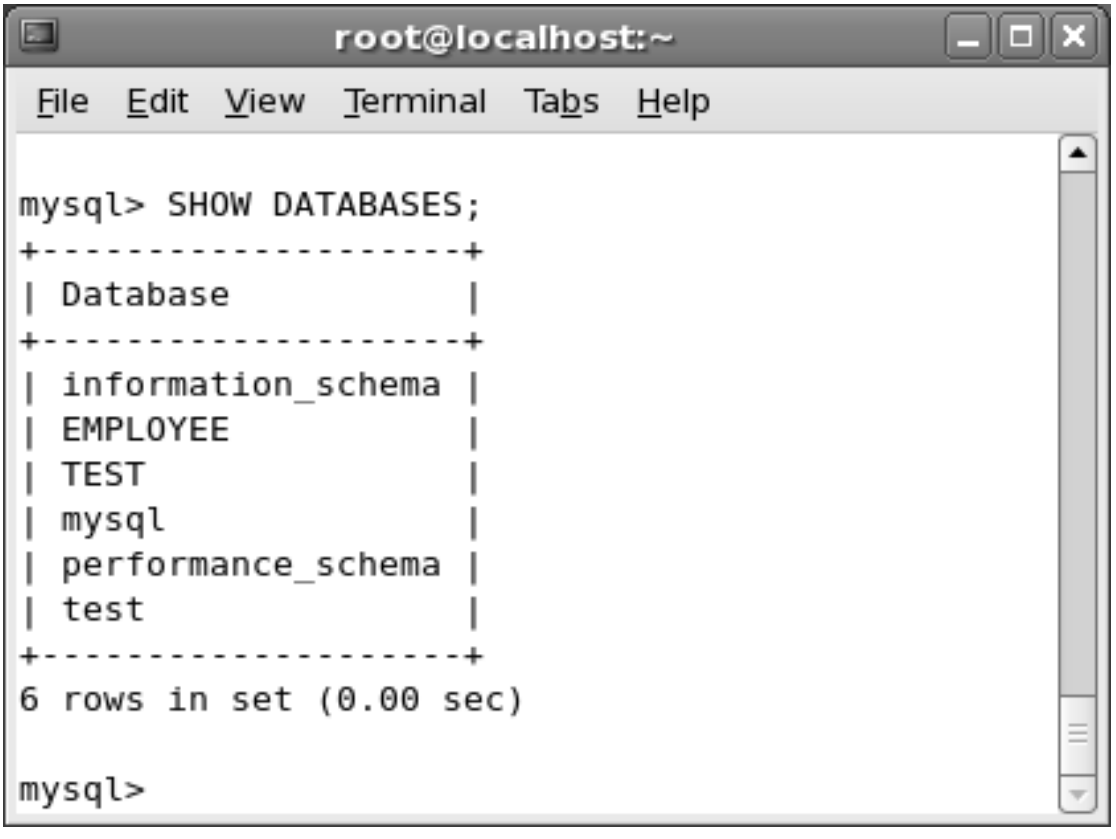


Figure 6.1: SHOW DATABASES Command

Figure 6.1 displays the output of the `SHOW DATABASES` command. The output lists all the databases present on the server. It displays the total number of databases in the form of number of rows. In addition, it also displays the time taken to execute the command in seconds.

To view a list of databases that begin or contain specified characters in the database name, use the following syntax:

```
SHOW DATABASES [LIKE <condition>];
```

where,

`SHOW DATABASES` – displays the list of databases present in the instance of MySQL

`LIKE <condition>` – contains conditions that must be satisfied before displaying the list of databases

The `LIKE` clause can use conditions with wildcard characters such as '`%`' and '`_`'. This clause is optional. The '`%`' represents any string of zero or more characters. The '`_`' represents any single character.

For example, to view all databases that begin with the letter '`E`', enter the following command at the command prompt:

```
SHOW DATABASES LIKE 'E%';
```

Figure 6.2 displays the output of the command. The output lists all the databases starting with the letter '`E`', present on the server.

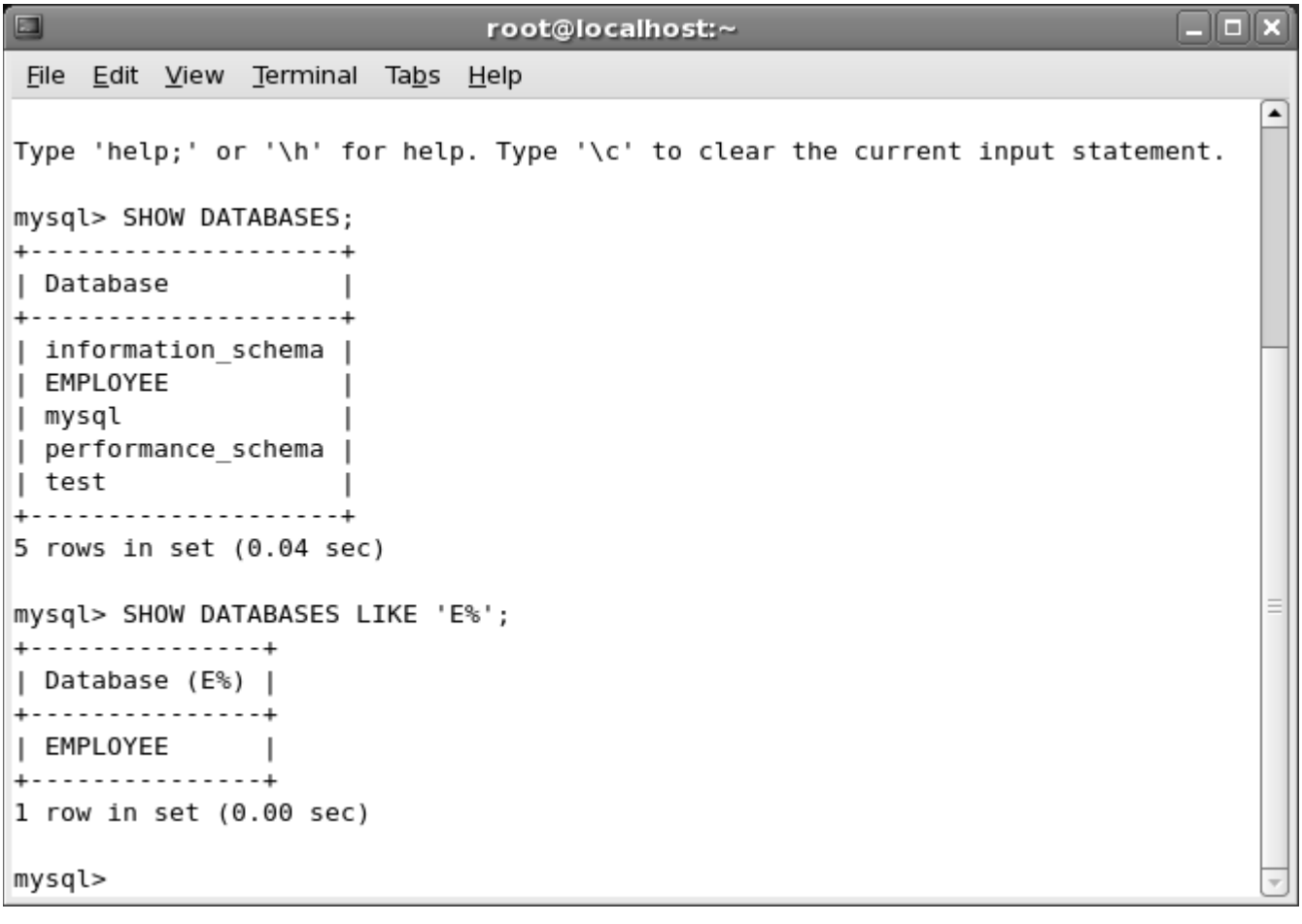


Figure 6.2: Using LIKE clause with SHOW DATABASES

To view a list of tables in a database, use the following syntax:

```
SHOW TABLES [FROM database_name] [LIKE <condition>];
```

where,

`SHOW TABLES` – displays the tables from the selected database

`FROM database_name` – displays the tables from the database specified in the clause

The `FROM` clause enables you to specify the database name for which tables are to be listed. You can use this keyword when you have not activated a database from the server with the `USE` command.

`LIKE <condition>` – displays the tables from the database based on the condition specified in the clause.

For example, to view the tables of the `EMPLOYEE` database that has been set as current database through the `USE` command, enter the following command at the command prompt:

```
SHOW TABLES;
```

Figure 6.3 displays the output of the command.

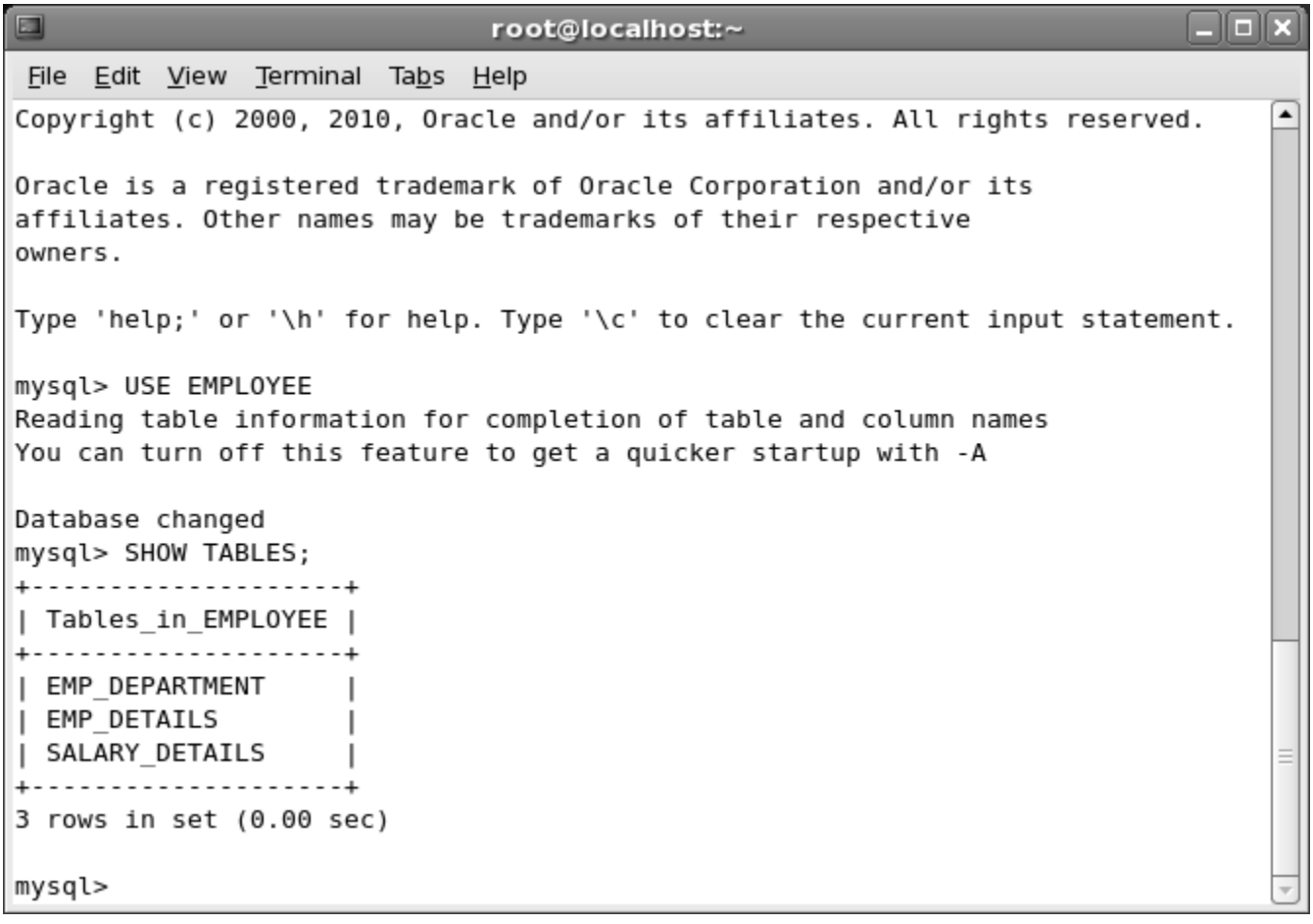


Figure 6.3: Tables in the EMPLOYEE Database

Figure 6.3 displays the output of the command. The output lists all the tables present in the `EMPLOYEE` database. You must select the database with `USE` command before displaying the tables using the command shown in figure 6.3.

Note: You can also use '`mysqlshow database_name`' command for displaying all the tables in a database.

To view a list of tables in a database when the database has not been selected earlier through the `USE` command, enter the following command at the command prompt:

```
SHOW TABLES FROM EMPLOYEE;
```

Figure 6.4 displays the output of the command.

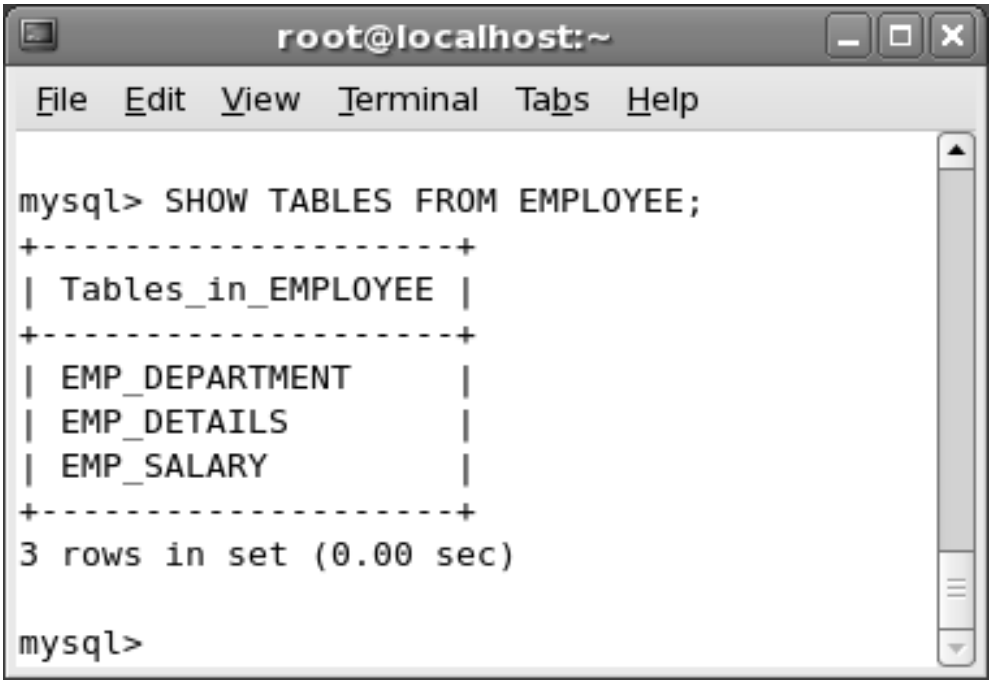


Figure 6.4: Listing the Tables in EMPLOYEE Database

You will use the `FROM` keyword here because the `EMPLOYEE` database has not been activated with the `USE` command before displaying the tables.

To view tables that begin with the letter 'E' from the `EMPLOYEE` database, enter the following command at the command prompt:

```
SHOW TABLES FROM EMPLOYEE LIKE 'E%';
```

Figure 6.5 displays the output of the command.

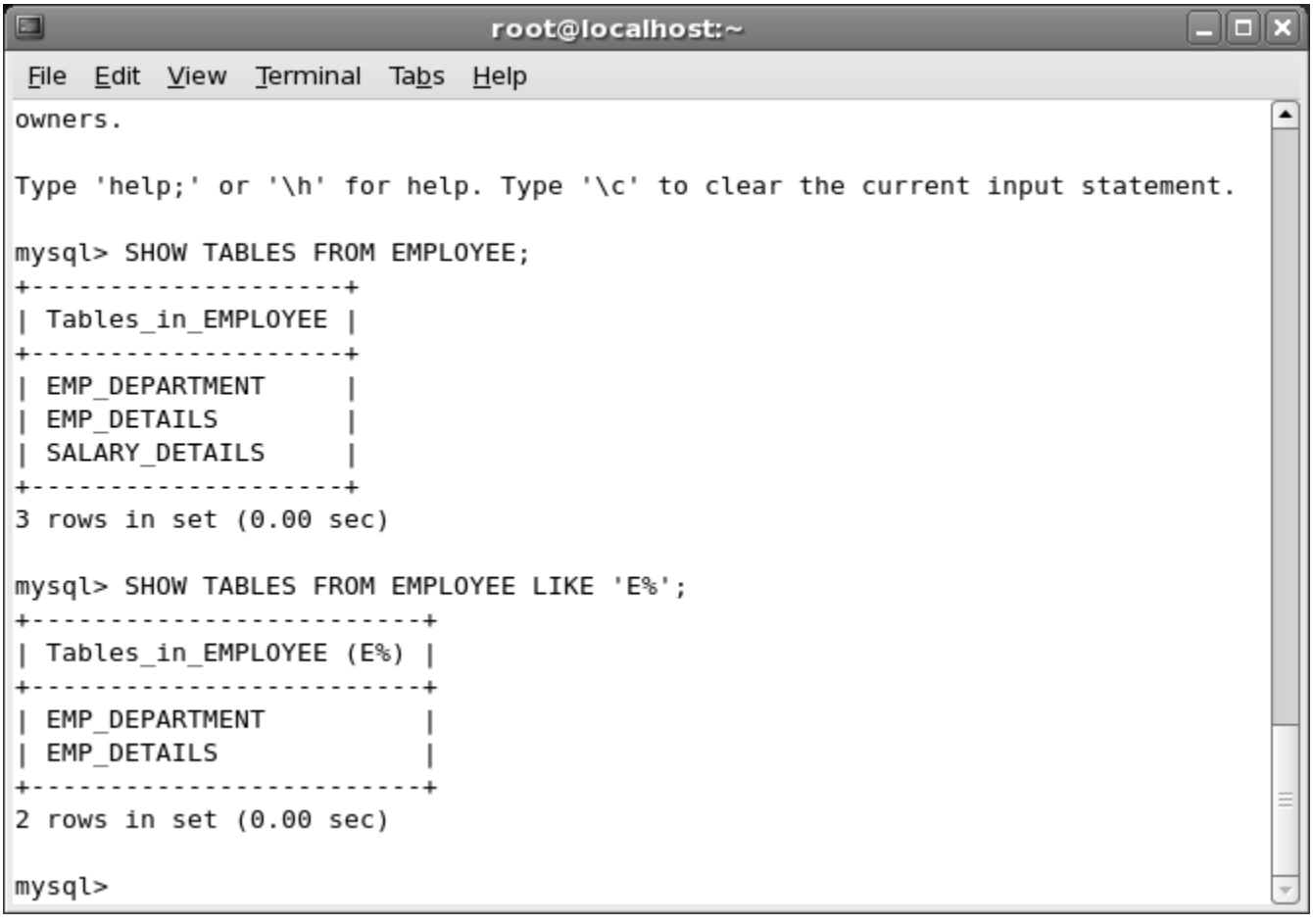


Figure 6.5: Listing the Tables Beginning with 'E'

Figure 6.5 displays all the tables present in the `EMPLOYEE` database starting with letter 'E' and any number of characters after the letter E. The `LIKE` keyword is used to display only those tables, which matches with the specified letter.

To view the column structure of a table from the database, use the following syntax:

```
SHOW COLUMNS FROM table_name [FROM database_name] [LIKE clauses];
```

where,

`table_name` – specifies the name of the table

`COLUMNS` – displays the columns from the specified table

`table_name` – specifies the name of the table that contains the columns

Concepts

`database_name` – specifies the name of the database

`LIKE` – specifies conditions, if any

For example, to view the column structure of the `EMP_DETAILS` table, enter the following command at the command prompt:

```
SHOW COLUMNS FROM EMP_DETAILS;
```

Figure 6.6 displays the output of the command. Here, the database name is omitted because the `EMPLOYEE` database is the current database.

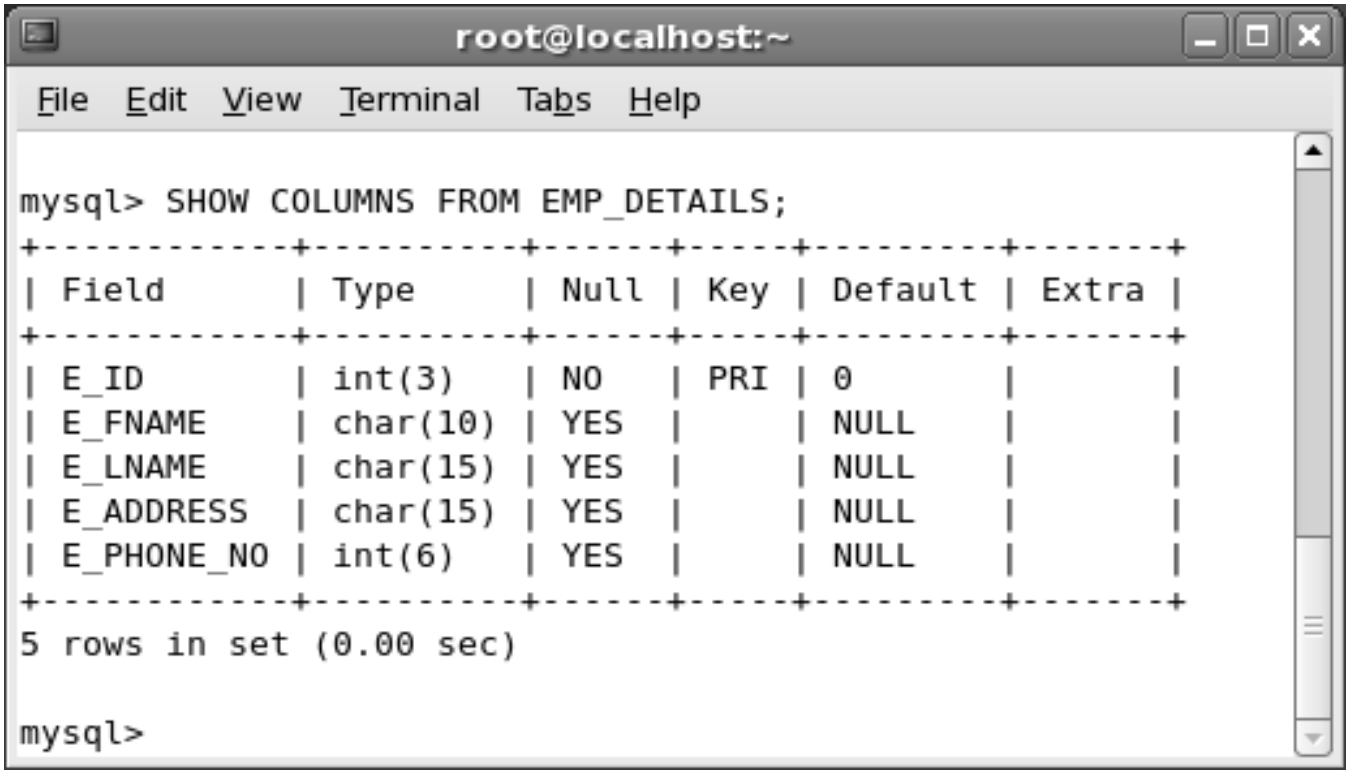


Figure 6.6: Columns of EMP_DETAILS

Figure 6.6 displays the columns present in the `EMP_DETAILS` table.

Table 6.1 lists the description of each of columns displayed in figure 6.6.

Column	Description
`FIELD`	Indicates column name
`TYPE`	Indicates the data type for a column

Column	Description
NULL	Specifies that the column can contain empty values
KEY	Specifies if the column is indexed
DEFAULT	Specifies the default value of the column
EXTRA	Specifies additional characteristics for the columns

Table 6.1: Detailed Information of Columns from EMP_DETAILS

To display the same output with the EMPLOYEE database specified explicitly, enter the following command at the command prompt:

```
SHOW COLUMNS FROM SALARY_DETAILS FROM EMPLOYEE;
```

Figure 6.7 displays the output of the command.

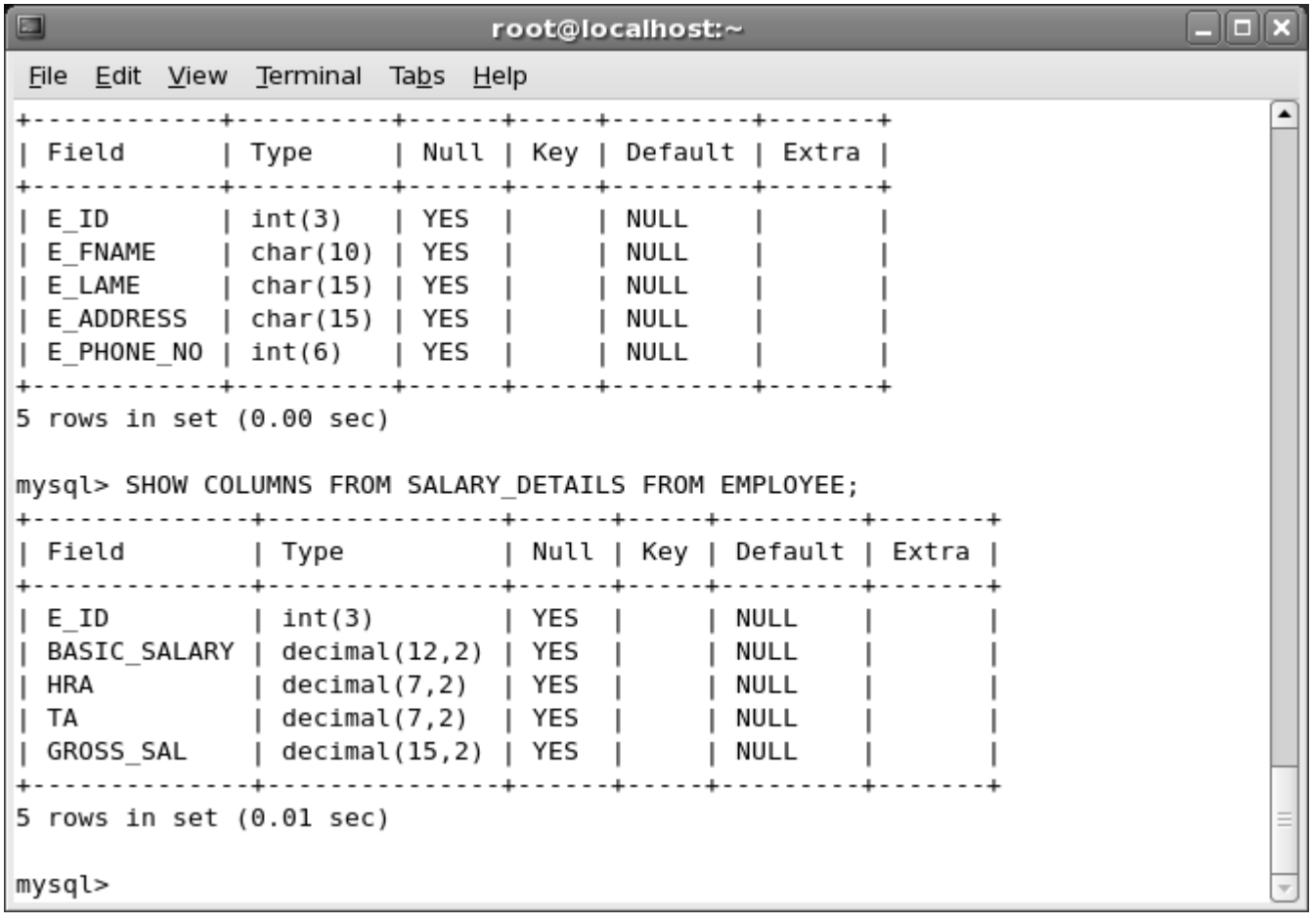


Figure 6.7: Columns of SALARY_DETAILS with database specified

Figure 6.7 displays the columns present in the SALARY_DETAILS table along with the information about each column. The first FROM keyword specifies the table to use for retrieving data. The second FROM keyword is used because you have not selected the database earlier with the USE command. Here, it specifies EMPLOYEE as the database in which the table SALARY_DETAILS is present.

Similarly, to view columns that start with the letter 'H' of the SALARY_DETAILS table from the EMPLOYEE database, enter the following command at the command prompt:

```
SHOW COLUMNS FROM SALARY_DETAILS LIKE 'H%';
```

Figure 6.8 displays the output of the command.

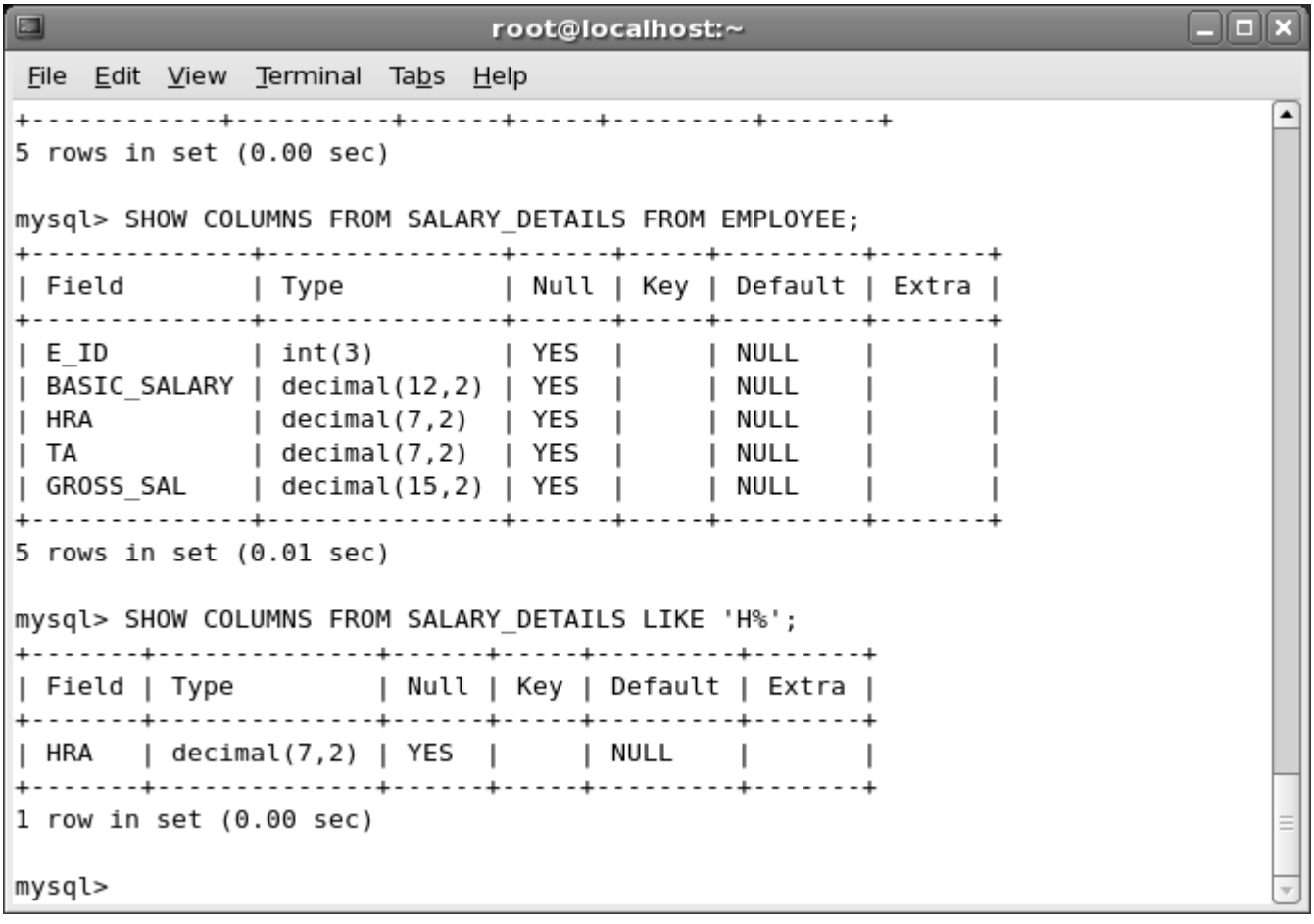


Figure 6.8: Columns of SALARY_DETAILS starting with letter 'H'

Figure 6.8 displays the columns present in the SALARY_DETAILS table starting with letter 'H'. You can use the LIKE keyword to display only those columns that start with letter 'H'.

A database index is a data structure that speeds up retrieval operations on a table. Indexes can be created

using one or more columns of a database table. They enable faster searches and efficient organization of data.

To view the index in a table from a database, use the following syntax:

```
SHOW INDEX FROM table_name [FROM database_name];
```

where,

`SHOW INDEX` - displays the index information of the table specified in the `table_name` clause of the command

`FROM table_name` – specifies the name of the table to retrieve the index

`FROM database_name` - specifies the name of the database where the table exists

The `SHOW INDEX` command displays index information as shown in table 6.2.

Column	Description
`Table`	Displays name of the table
`Non_unique`	Displays 0 if the index cannot contain duplicate value
`Key_name`	Displays index name
`Seq_in_index`	Displays sequence number of columns in index, starting with 1
`Column_name`	Displays column name
`Collation`	Displays the sorting order of columns in the index
`Cardinality`	Displays number of unique values in the index
`Sub_part`	Displays number of indexed characters when the column is partly indexed

Table 6.2: Indexes on a Table

For example, to view the index in the `EMP_DETAILS` table of the `EMPLOYEE` database, enter the following command at the command prompt:

```
SHOW INDEX FROM EMP_DETAILS FROM EMPLOYEE;
```

Figure 6.9 displays the output of the command.

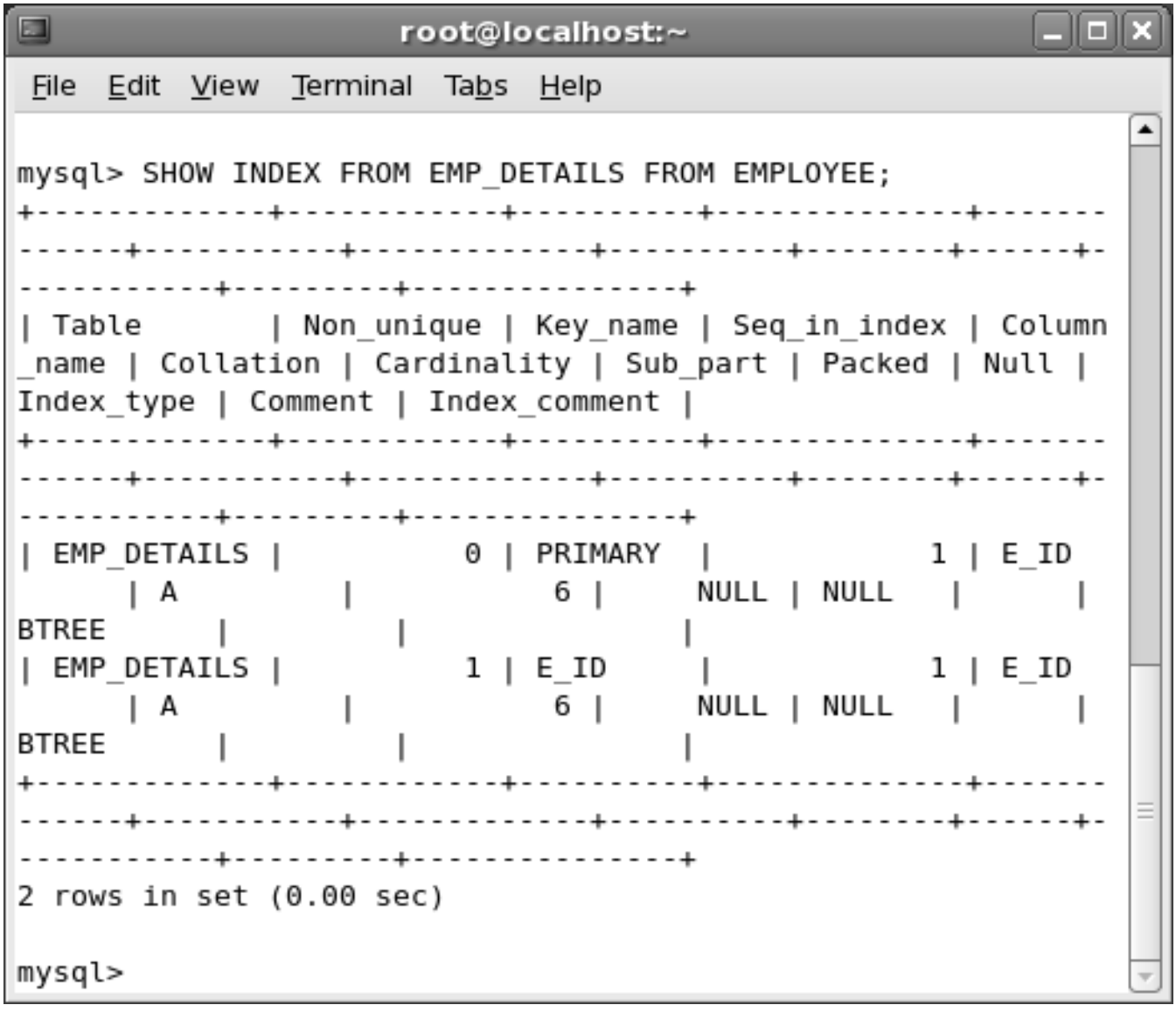


Figure 6.9: Index Columns of EMP_DETAILS

Alternatively, you can use the following syntax:

```
SHOW INDEX FROM database_name.table_name;
```

where,

`SHOW INDEX` – displays the table index

`database_name` – specifies the name of the database that contains the table

`table_name` – specifies the name of the table for which you want to display the index

For example, to display index keys of `EMP_DETAILS` from `EMPLOYEE` database, enter the following command at the command prompt:

```
SHOW INDEX FROM EMPLOYEE.EMP_DETAILS;
```

This command is more compact and concise.

To view the server status, use the following syntax:

```
SHOW STATUS;
```

where,

`SHOW` – displays the object specified in the clause

`STATUS` – displays information about the server

Figure 6.10 displays the output of the command.

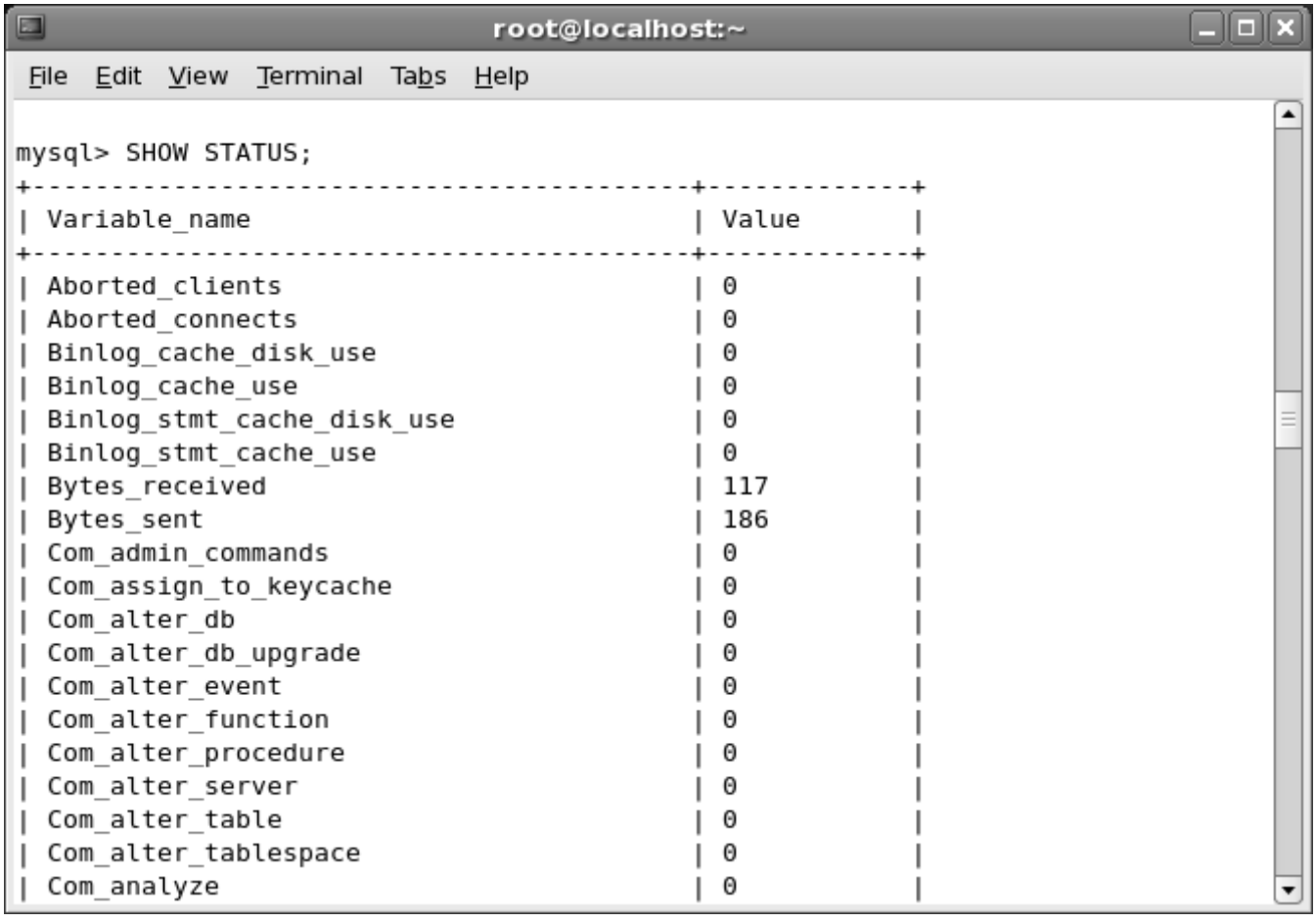


Figure 6.10: Server Status

Note: The numbers and format may be different in various servers.

To view the values of the system variables, use the following syntax:

```
SHOW VARIABLES;
```

Figure 6.11 displays the output of the command.

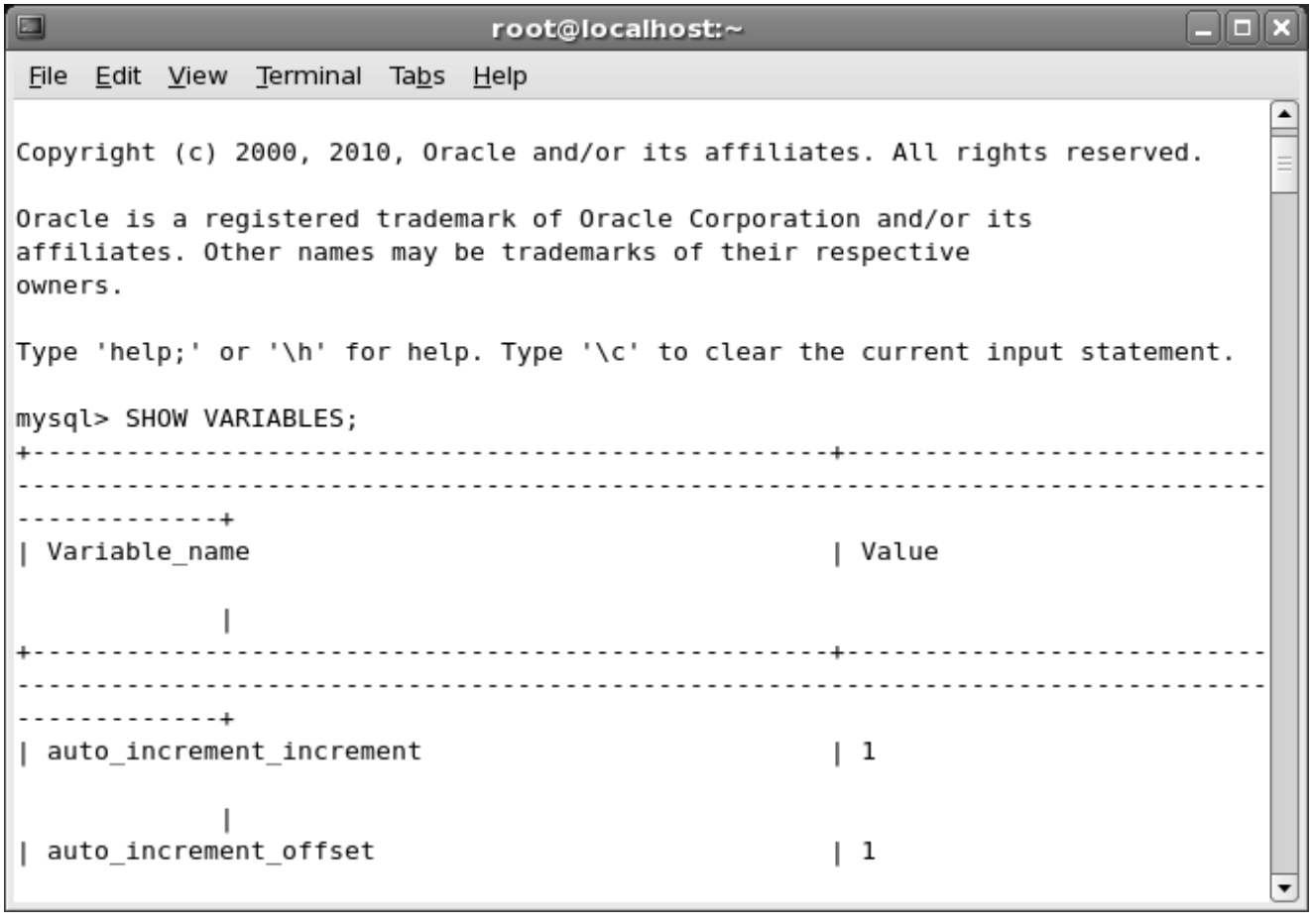


Figure 6.11: System Variables

You can change the default values of the variables using the command-line options when MySQL starts. The `SET` statement can also be used to edit the default values at runtime.

Note: The list of names and values may be different for various servers.

You can also view the system variable information using the `mysqladmin` command.

Similarly, to view only the variables that match a specified condition use the following syntax:

```
SHOW VARIABLES [LIKE <condition>];
```

For example, to display the variables that start with `'HAVE'`, enter the following command at the command prompt:

```
SHOW VARIABLES LIKE 'HAVE%';
```

Figure 6.12 displays the output of the command.

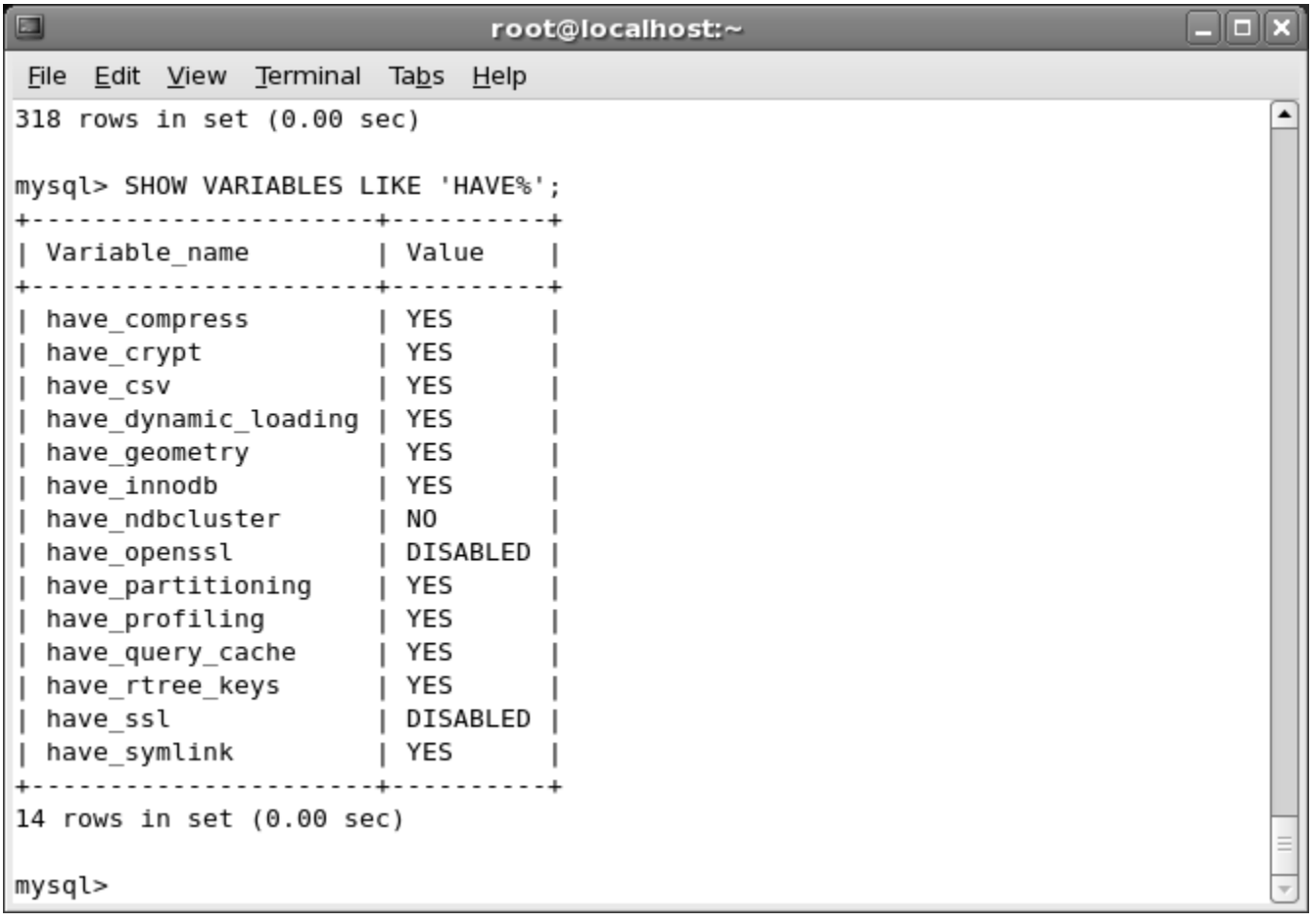


Figure 6.12: Server Variables starting with 'HAVE'

To view the running threads or processes, use the following syntax:

```
SHOW [FULL] PROCESSLIST;
```

You must have `SUPER` privileges to view all the threads. The user accounts that do not have `SUPER` privileges can view only their threads.

Note: If the `[FULL]` option is not specified, MySQL will only display the first `100` characters of each process.

The `SHOW TABLE STATUS` command is similar to the `SHOW TABLE` command except that it displays more information about each table.

The syntax for SHOW TABLE STATUS command is as follows:

```
SHOW TABLE STATUS [FROM database_name] [LIKE <condition>];
```

where,

SHOW TABLE STATUS – displays information about tables in the database

FROM database_name – enables you to specify the database to select tables. You can use the FROM clause if you have not activated the database with the USE command.

LIKE <condition> – specifies one or more conditions to be satisfied before displaying the table information

Table 6.3 lists the information returned by the SHOW TABLE STATUS command.

Column	Description
Name	Displays the name of the table
Type	Displays type of table
Row_format	Displays storage format of the row (such as fixed, dynamic, or compressed)
Rows	Displays number of rows
Avg_row_length	Displays average row length
Data_length	Displays length of the data file
Max_data_length	Displays max length of the data file
Index_length	Displays length of the index file
Data_free	Displays number of unused allocated bytes
Auto_increment	Displays next auto increment value
Create_time	Displays when the table was created
Update_time	Displays the date of last updation of data file
Check_time	Displays when the table was last checked
Comment	Displays the comments for a column. The comment can be 255 characters long.

Table 6.3: Information Returned by SHOW TABLE command

For example, to view the status of all the tables of the EMPLOYEE database, enter the following command at the command prompt:

```
SHOW TABLE STATUS FROM EMPLOYEE;
```

Figure 6.13 displays the output of the command.

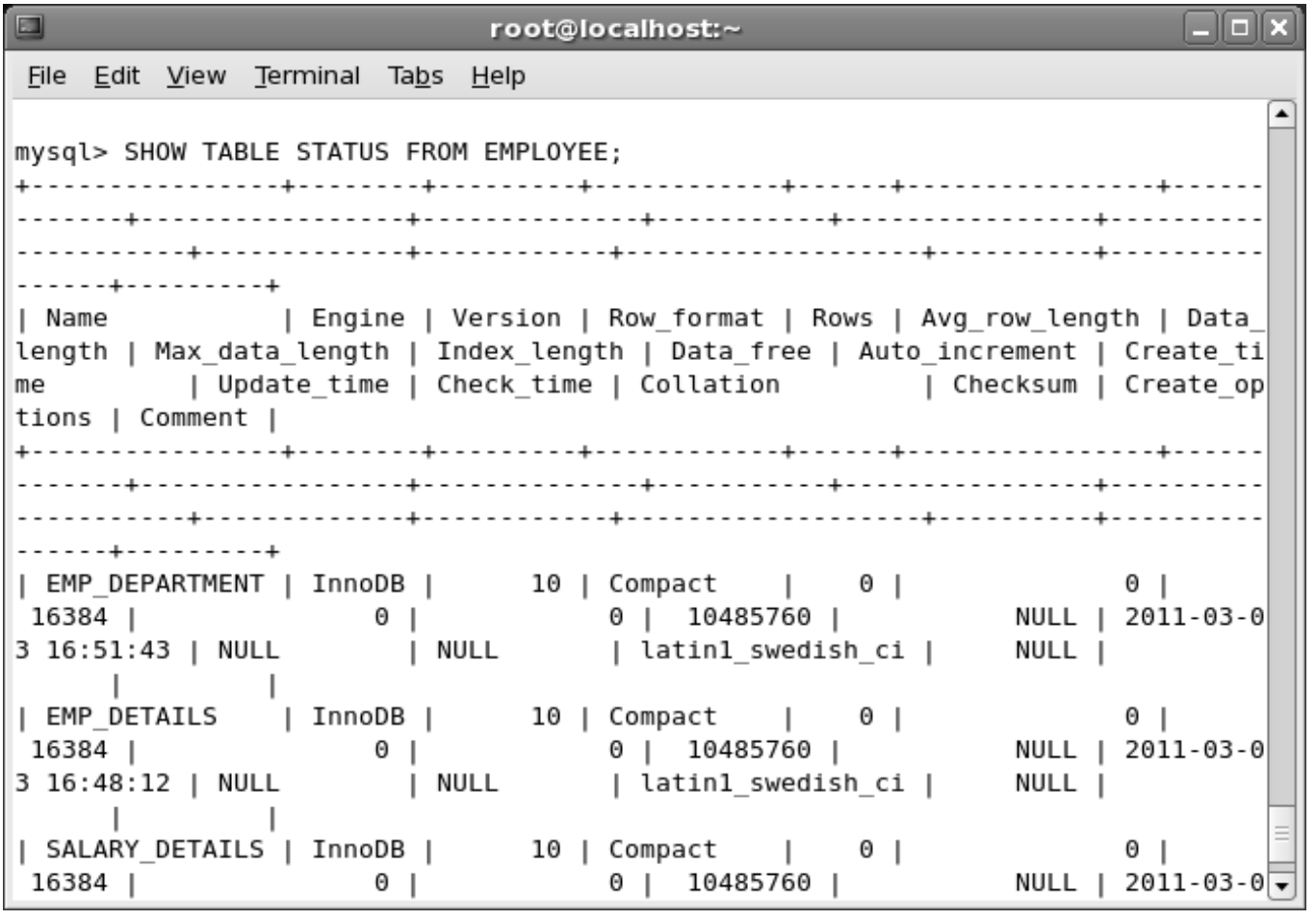


Figure 6.13: Status of Tables in EMPLOYEE Database

To view a list of grants that are assigned for a user, use the following syntax:

```
SHOW GRANTS FOR user;
```

where,

`SHOW` – displays information specified in the clause

`GRANTS` – displays privileges or account rights

`FOR` – specifies the object to display the privileges

`user` – specifies the type of object for which privileges are to be displayed

For example, to display the rights granted to the root user, enter the following command at the command prompt:

```
SHOW GRANTS FOR root@localhost;
```

Figure 6.14 displays the output of the command. The output shows the list of grants for the root user.

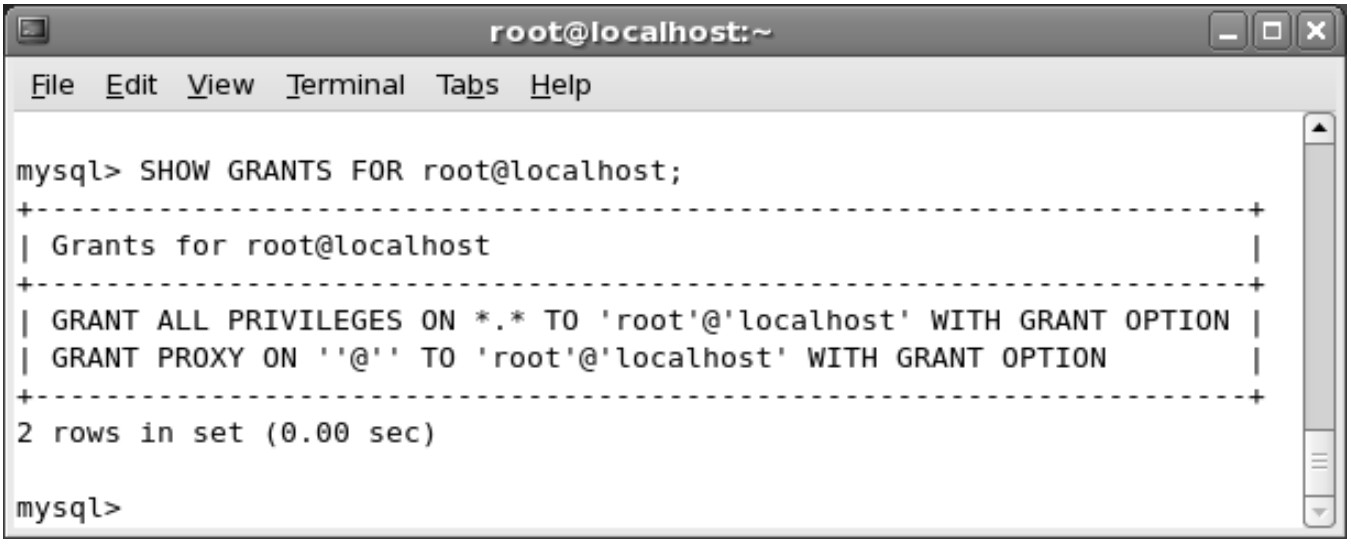


Figure 6.14: Rights Granted to Root User

6.2.2 Altering Database on the File System

Alteration of a database involves making changes to the database. You can make changes to the data in the database, as well as the database structure. MySQL provides the `ALTER DATABASE` command to modify the global characteristics or attributes of a database stored in the `db.opt` file of the database directory. You will use the `CHARACTER SET` clause to modify the default database character set. In addition, you will use the `COLLATE` clause to modify the default database collation. A collation in MySQL database is a set of rules used in data comparisons.

The syntax for modifying the character set of a database is:

```
ALTER DATABASE database_name DEFAULT CHARACTER SET charset_name;
```

where,

`ALTER DATABASE` – edits the database

`database_name` – specifies the name of the database on which you need to make changes

Concepts

`DEFAULT CHARACTER SET` – specifies the default character set for the database

`charset_name` – specifies the type of character set for the database

For example, to modify the character set of the database, create a temporary database named `TEST` and then alter the database by entering the following command at the command prompt:

```
ALTER DATABASE TEST DEFAULT CHARACTER SET swe7;
```

Figure 6.15 displays the output of the command.

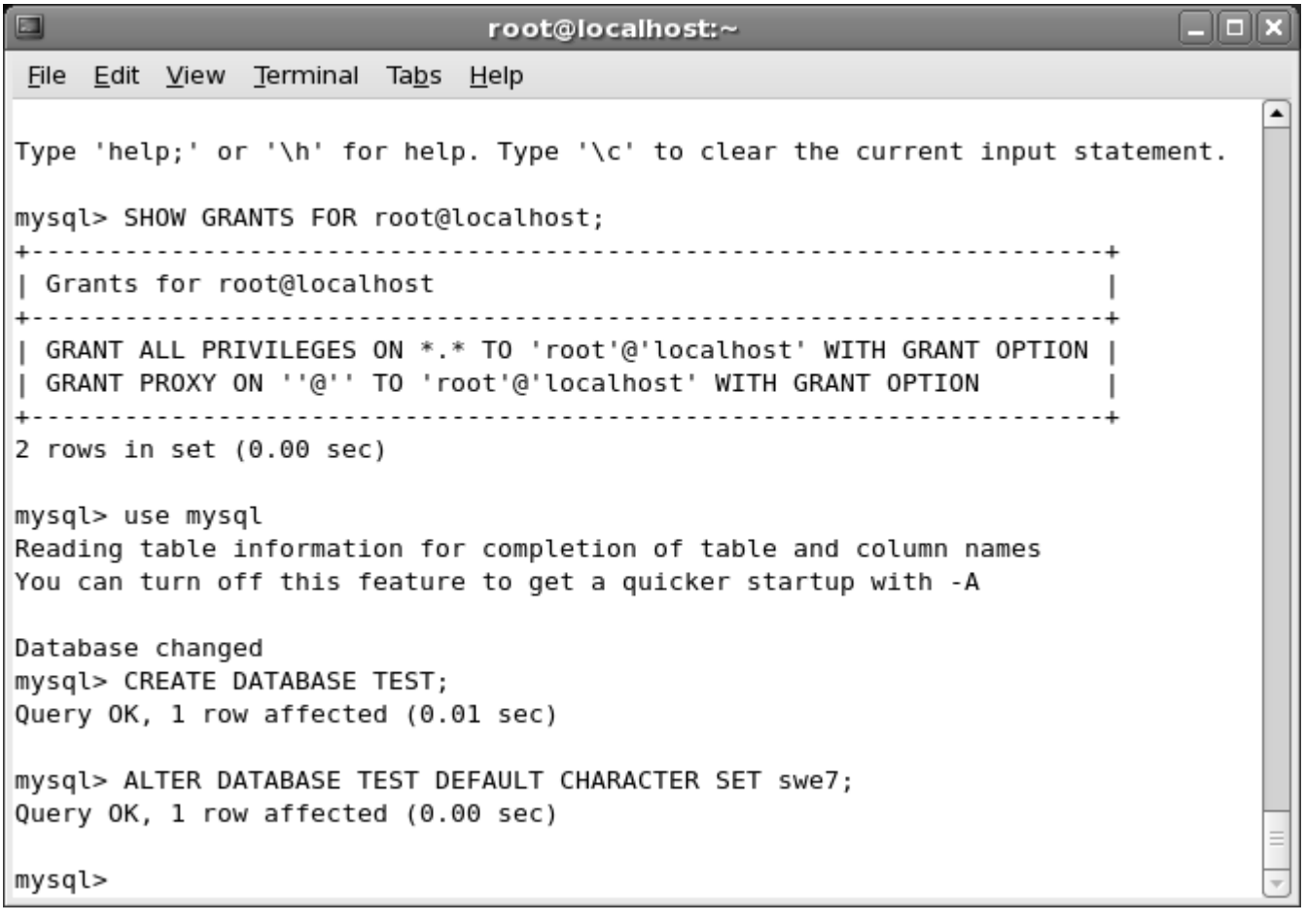


Figure 6.15: ALTER Database

The syntax to modify the collation of a database is:

```
ALTER DATABASE database_name DEFAULT COLLATE collation_name;
```

Note: You must have `ALTER` privileges to use the `ALTER DATABASE` command.

6.3 Retrieving Data Using SELECT, FROM, DISTINCT, and WHERE Clauses

A table contains structured data in the form of rows and columns. The table must contain data to display. You cannot view the contents of a table if it does not contain data. Therefore, you must insert data before retrieving data from tables. MySQL provides the `INSERT` command to add data to the table. The syntax for `INSERT` command is:

```
INSERT INTO table_name {VALUES | VALUE} (value1, value2…);
```

where,

`INSERT INTO` – adds a new record to the table

`table_name` – specifies the name of the table to add the record

`{VALUES | VALUE}` – any one of these two may be used to specify the values and either may be used for a single values list or multiple lists

`value1` – specifies the data that will be added to the column

Figure 6.16 displays the insertion of records in the table.

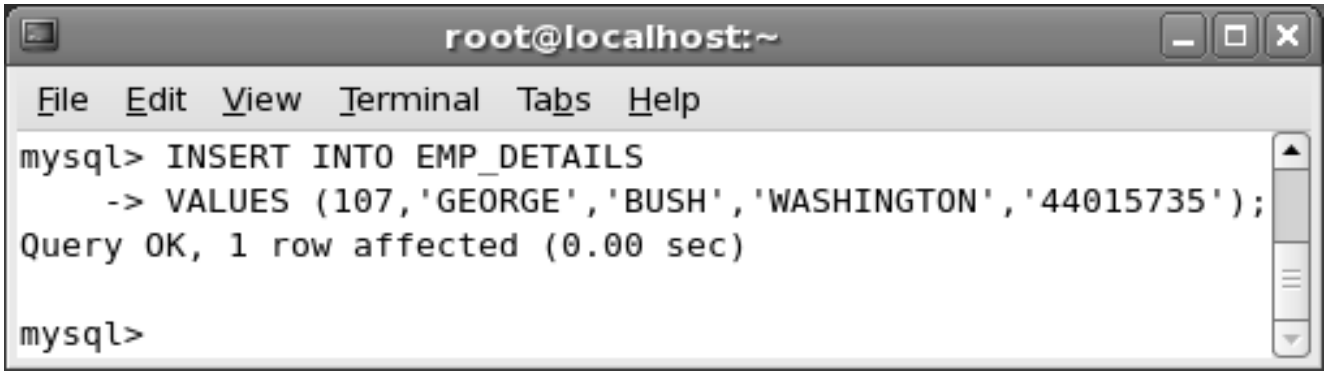


Figure 6.16: INSERT Values

Figure 6.16 displays the method of inserting values in the table. The character value is specified with single quotation mark before and after the string.

You can use the `SELECT` command to retrieve data from one or more tables.

Concepts

The SELECT command has two clauses:

- FROM - specifies the table name whose records are to be retrieved.
- WHERE - specifies the condition, based on which the records are retrieved. This clause is optional.

The syntax for retrieving all the records of a table is:

```
SELECT [*] FROM table_name;
```

where, '*' displays all the columns of the specified table.

For example, to view all records of the EMP_DETAILS table, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS;
```

Figure 6.17 displays the output of the command.

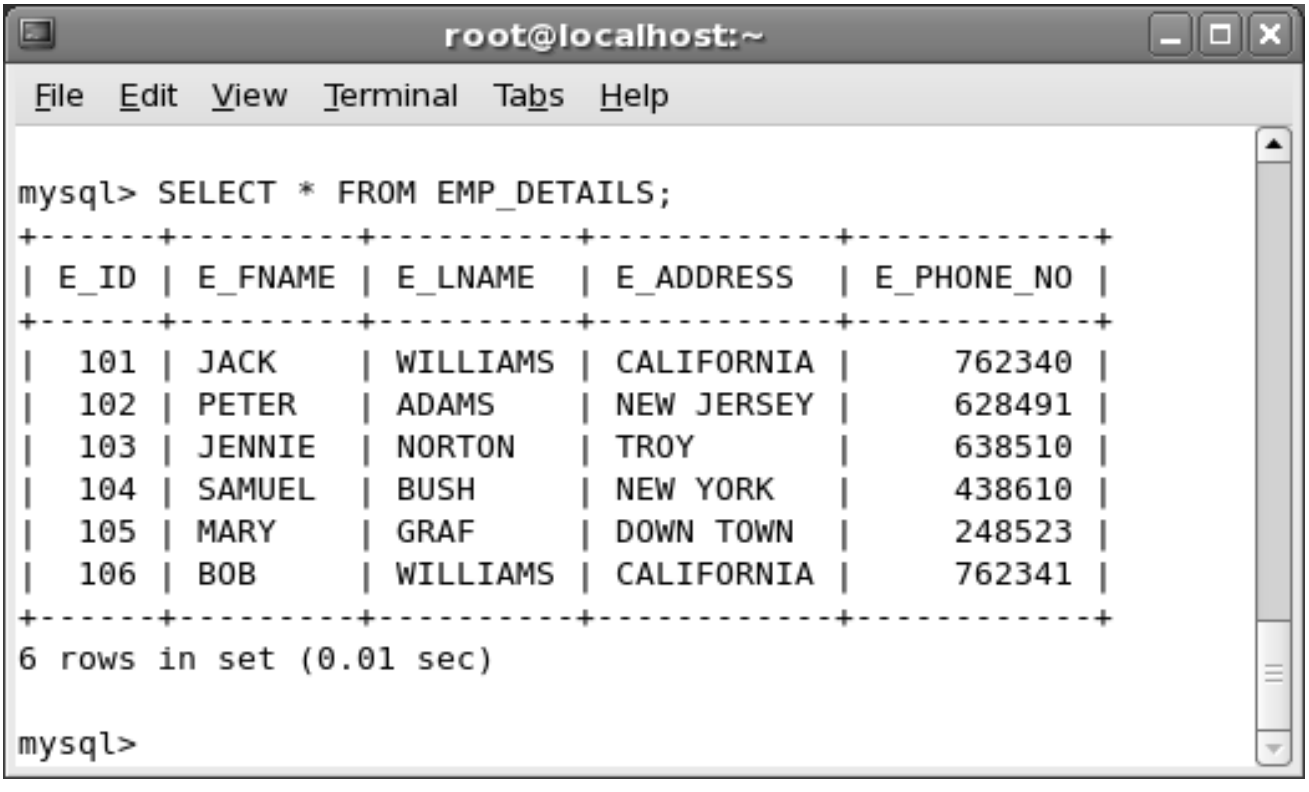


Figure 6.17: Displaying all Records of the EMP_DETAILS

You can use the DISTINCT keyword in the SELECT command to display columns that do not contain duplicate data. The DISTINCT keyword displays only those rows containing unique values in the specified column.

To display unique information from columns using DISTINCT keyword with SELECT command, use the syntax:

```
SELECT DISTINCT column_name FROM table_name;
```

where,

SELECT – displays data from the table

DISTINCT – is the keyword that displays unique information from columns. MySQL does not display duplicate records when you use this clause

column_name – specifies the name of the column

table_name – specifies the name of the table where the column exists

This command will display the unique records from the columns specified in the column_name keyword.

For example, to display only the unique department names from the EMP_DEPARTMENT table, enter the following command at the command prompt:

```
SELECT DISTINCT D_NAME FROM EMP_DEPARTMENT;
```

Figure 6.18 displays the output of the command.

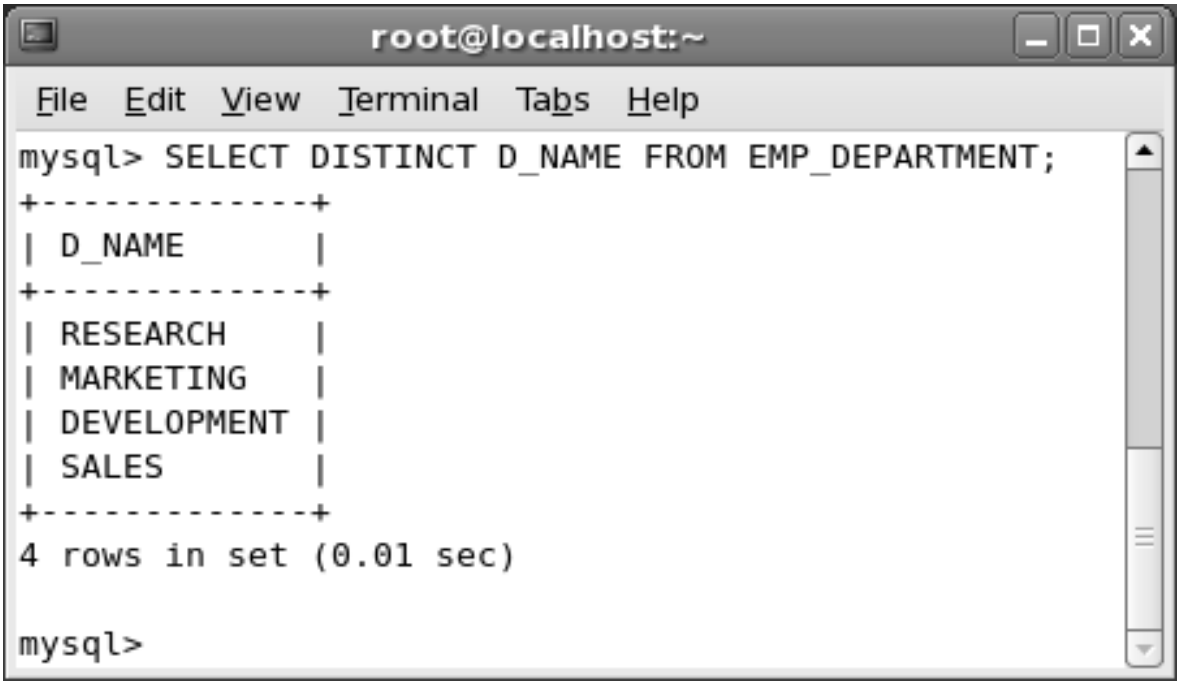


Figure 6.18: Displaying Unique Department Names

The `SELECT` command also enables you to view specific columns of a table.

The syntax for viewing only specific columns of a table is:

```
SELECT column_name1, column_name2 FROM table_name;
```

where,

`SELECT` – displays the information

`column_name1` – specifies the name of the column to be retrieved

`column_name2` – specifies the name of the column to be retrieved

`table_name` – specifies the name of the table that contains the columns

This command will display only those columns that are specified in the `column_name` clause of the command.

For example, to view the E_FNAME, E_LNAME, and E_ADDRESS columns of the EMP_DETAILS table, enter the following command at the command prompt:

```
SELECT E_FNAME, E_LNAME, E_ADDRESS FROM EMP_DETAILS;
```

Figure 6.19 displays the output of the command.

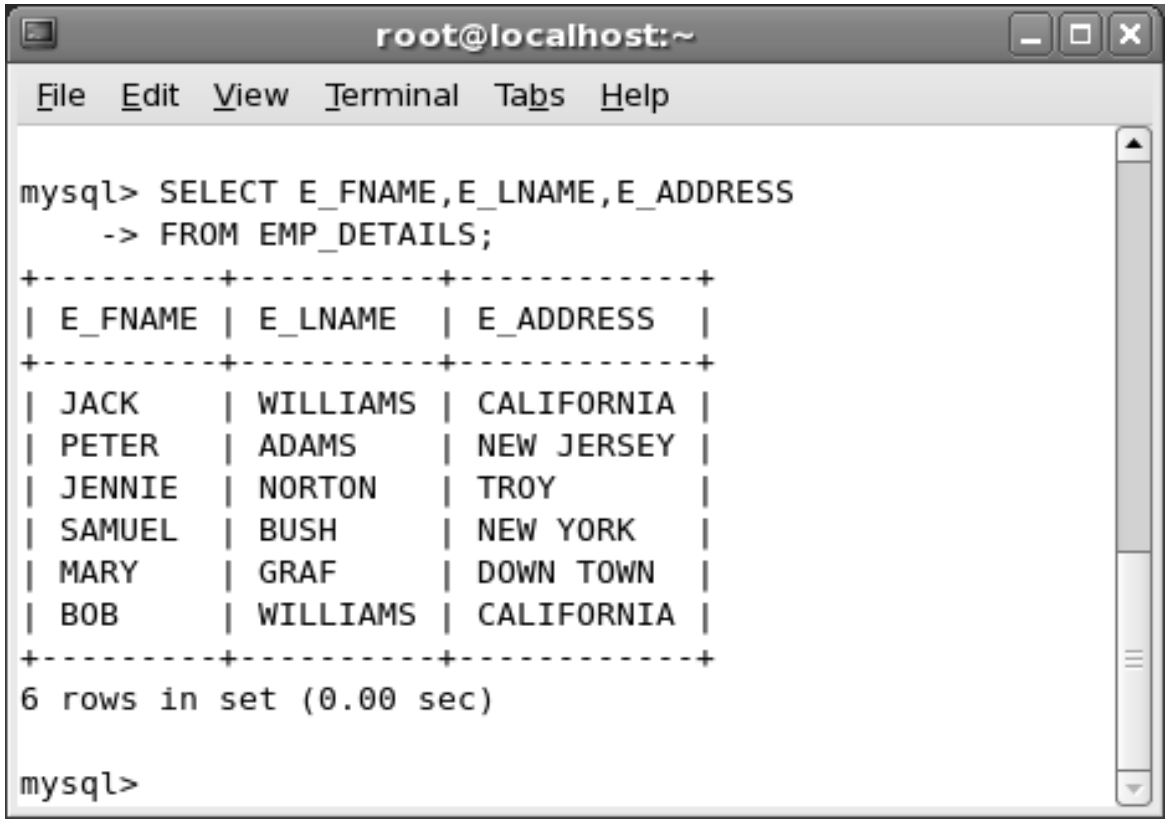


Figure 6.19: Displaying Specific Columns

Figure 6.19 displays the first name, last name, and address of all employees from the EMP_DETAILS table.

You can also use the SELECT command to make calculations. These calculations do not require a reference to any table.

```
SELECT 1+1;
```

Figure 6.20 displays the output of the command.

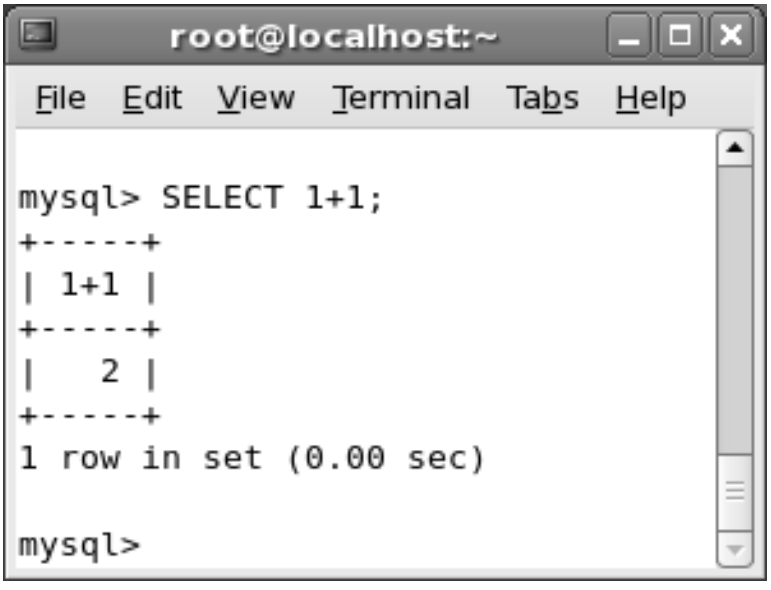


Figure 6.20: Computed Result

To perform arithmetic operations using the `SELECT` command, consider the following example:

```
SELECT GROSS_SAL + 1000 FROM SALARY_DETAILS;
```

Figure 6.21 displays the output of the command.

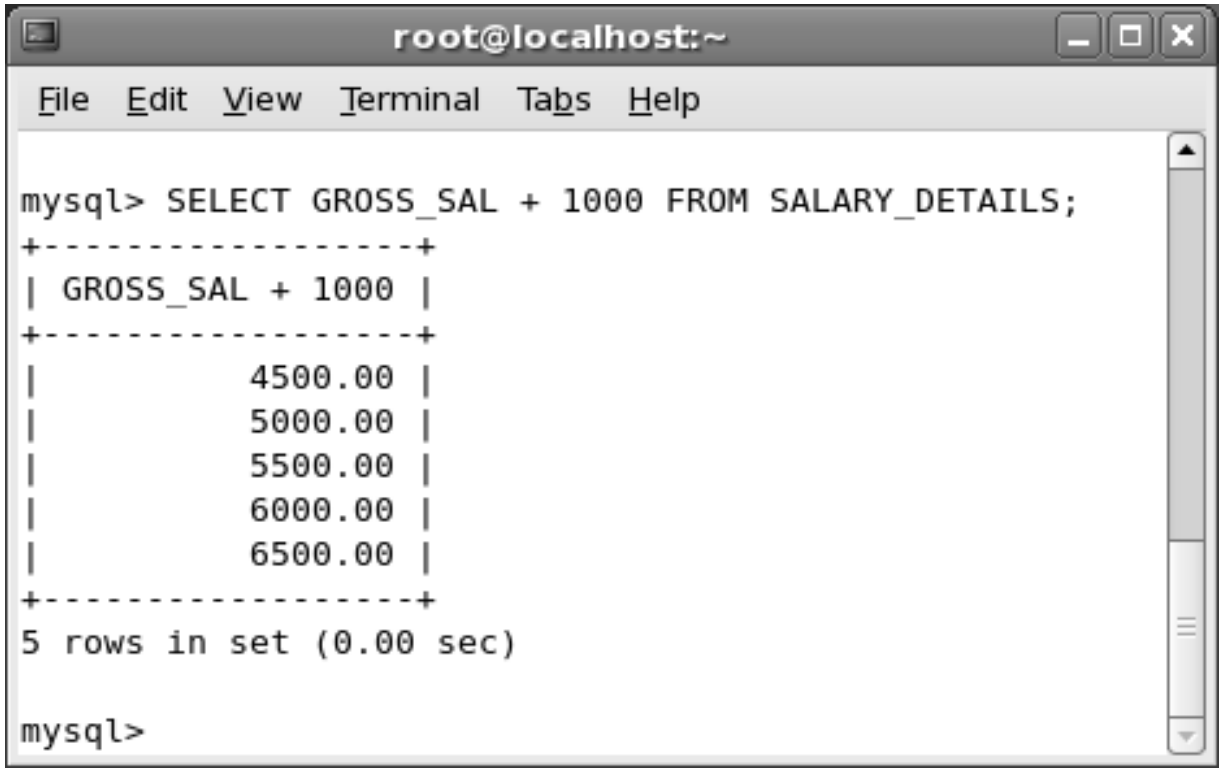


Figure 6.21: Computed Gross Salary of Employees

Figure 6.21 displays the calculated sum of gross salary and 1000 in the `GROSS_SAL` column for each of the rows of the table.

6.3.1 SELECT command Using WHERE Clause

The `WHERE` clause contains one or more conditions that must be satisfied before the query retrieves a row. It searches for data according to the condition specified in the query and narrows the selection of data. The `WHERE` clause uses logical and conditional operators in the query.

The syntax to specify a condition using the `WHERE` clause is:

```
SELECT * FROM table_name WHERE <condition to satisfy>;
```

where,

`SELECT` – retrieves the data

`*` – specifies to retrieve all the columns of the table

`table_name` – specifies the name of the table

`WHERE` – specifies the clause to filter data

`<condition to satisfy>` – contains conditions to satisfy before retrieving and displaying data

For example, to view the records of `EMP_SALARY` table, where the basic salary is greater than `2500`, enter the following command at the command prompt:

```
SELECT * FROM EMP_SALARY WHERE BASIC_SALARY > 2500;
```

Figure 6.22 displays the output of the command.

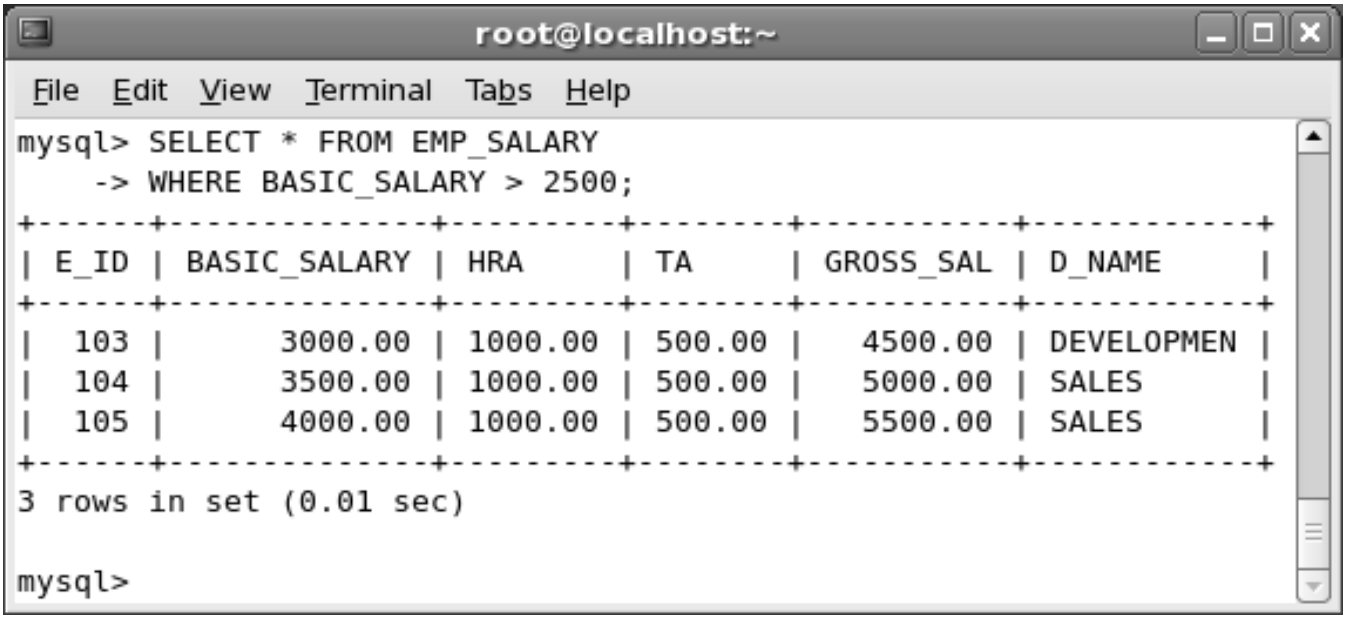


Figure 6.22: Displaying Records Using WHERE Clause

Similarly, to view the records of the `EMP_DETAILS` table where the last name is `NORTON`, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS WHERE E_LNAME = 'NORTON';
```

Figure 6.23 displays the output of the command.

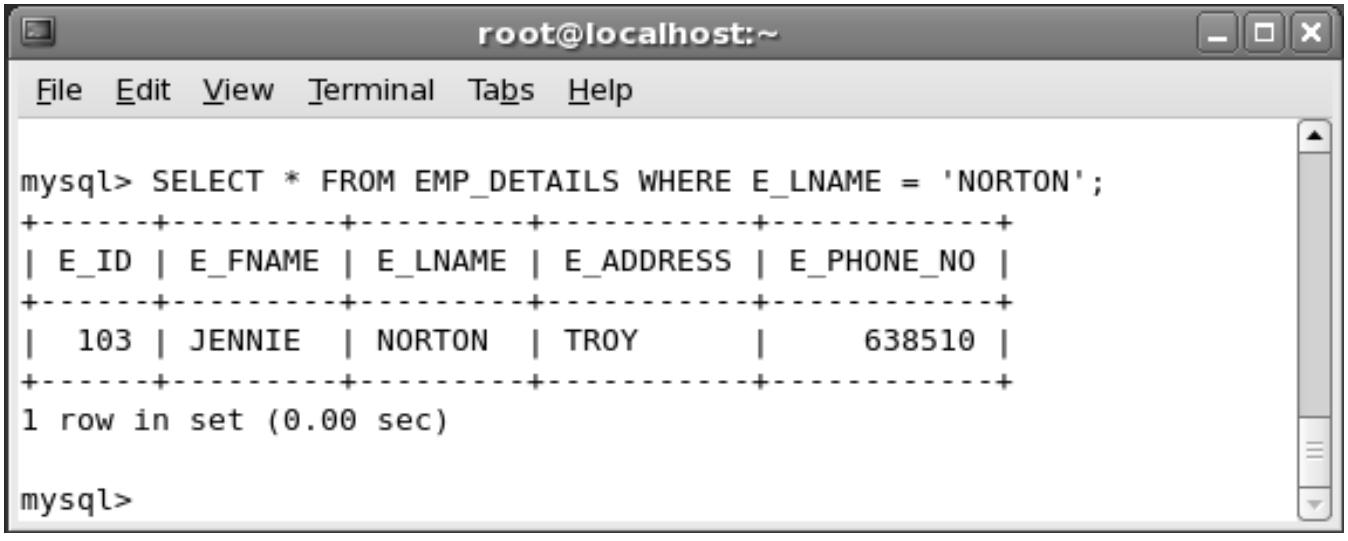


Figure 6.23: Displaying Records Using WHERE Clause With Text

Figure 6.23 displays all records from `EMP_DETAILS` table that have the last name as '`NORTON`'.

To view all the fields of the `EMP_DETAILS` table, where the address of the employee is `TROY` or the address of employee is `NEW JERSEY`, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS WHERE E_ADDRESS = 'TROY' OR E_ADDRESS = 'NEW
JERSEY';
```

Figure 6.24 displays the output of the command.

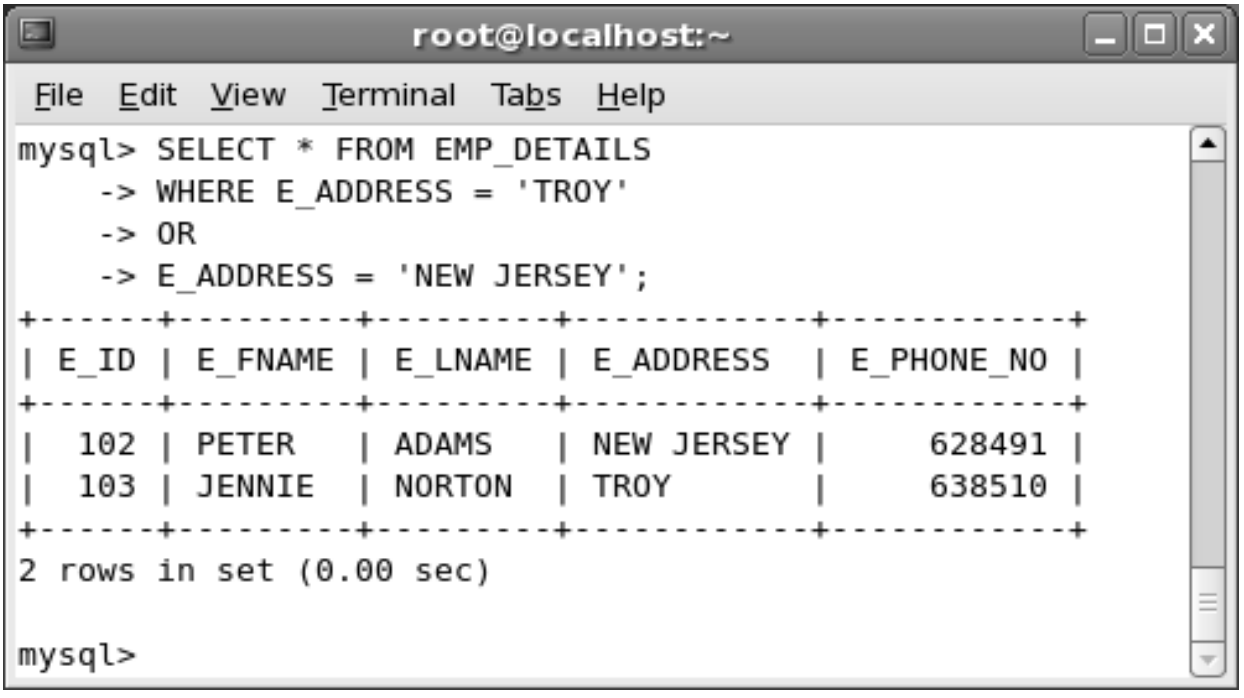


Figure 6.24: Displaying Records Using WHERE and OR Clause

6.4 Modifying Table Definitions Using ALTER Command

MySQL provides the `ALTER TABLE` command to modify the structure of a table. You can add or delete columns, rename columns or the table, create or destroy indexes, and modify the column type.

The `ALTER TABLE` command creates a temporary copy of the original table, on which the alteration is performed. MySQL alters the temporary copy of the table, deletes the original copy, and renames the temporary copy of the table. MySQL uses this feature as a security measure to prevent data loss in case the table modification fails.

The syntax for altering a table is:

```
ALTER [IGNORE] TABLE table_name alter_spec [, alter_spec...];
```

where,

`ALTER` – specifies to edit the object

`IGNORE` – checks for duplicate records on the index keys in the new table. If MySQL identifies a duplicate record on the unique keys, it returns only the first row and deletes the duplicates

`TABLE` – specifies the type of object to edit

`table_name` – specifies the name of the table

`alter_spec` – contains modification information

Note: If the `[IGNORE]` clause is not specified, MySQL aborts the copy operation if the index column contains duplicate records.

The syntax to add a column in the existing table is:

```
ALTER TABLE table_name ADD [COLUMN] create_definition [FIRST | AFTER column_
name];
```

where,

`table_name` – specifies the name of the table to modify

`ADD [COLUMN]` – appends a new column to the table

`create_definition` – defines the column type in a table creation

`FIRST | AFTER column_name` – specifies the location of the new column in the table

For example, to add a column `CITY` to the `EMP_DETAILS` table for entering the name of the city of employees, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS ADD (CITY CHAR (10));
```

Figure 6.25 displays the output of the command.

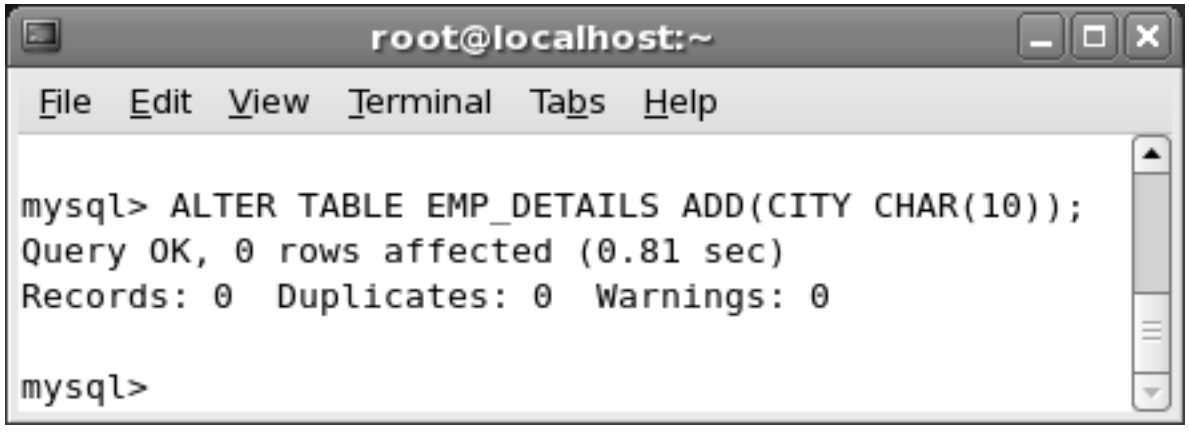


Figure 6.25: ALTER TABLE Command

Concepts

The syntax to add an index key to a column of a table is:

```
ALTER TABLE table_name ADD INDEX [index_name] (index_column_name...);
```

where,

`ALTER TABLE` – specifies to modify the table

`table_name` – specifies the name of the table

`ADD INDEX` – appends an index to the table

`index_name` – specifies a name for the index

`index_column_name` – specifies the column in the table to index

This command adds an index to the table and indexes the specified column.

For example, to add an index on column `E_ID` on `EMP_DETAILS` table, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS ADD (INDEX (E_ID));
```

Figure 6.26 displays the output of the command.

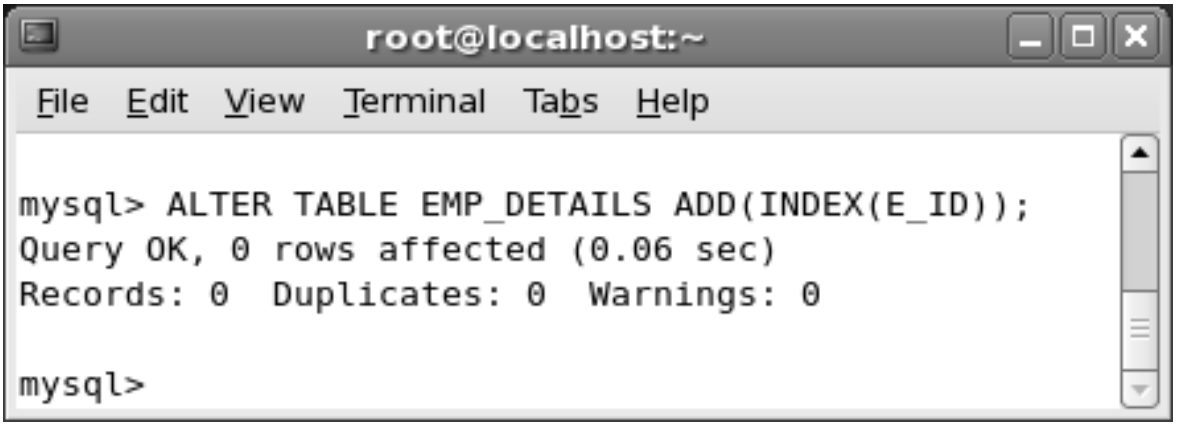


Figure 6.26: Adding an Index Key

The syntax to add a primary key to the column of a table is:

```
ALTER TABLE table_name ADD PRIMARY KEY (index_column_name...);
```

where,

`ALTER TABLE` – modifies the table structure

`table_name` – specifies the name of the table

`ADD PRIMARY KEY` – appends a primary key to the table

`index_column_name` – specifies the name of the column to use as an index for the primary key

This command adds a primary key to the specified indexed column in the table.

For example, to add a primary key on column `E_ID` on `EMP_DETAILS` table, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS ADD (PRIMARY KEY (E_ID));
```

Figure 6.27 displays the output of the command.

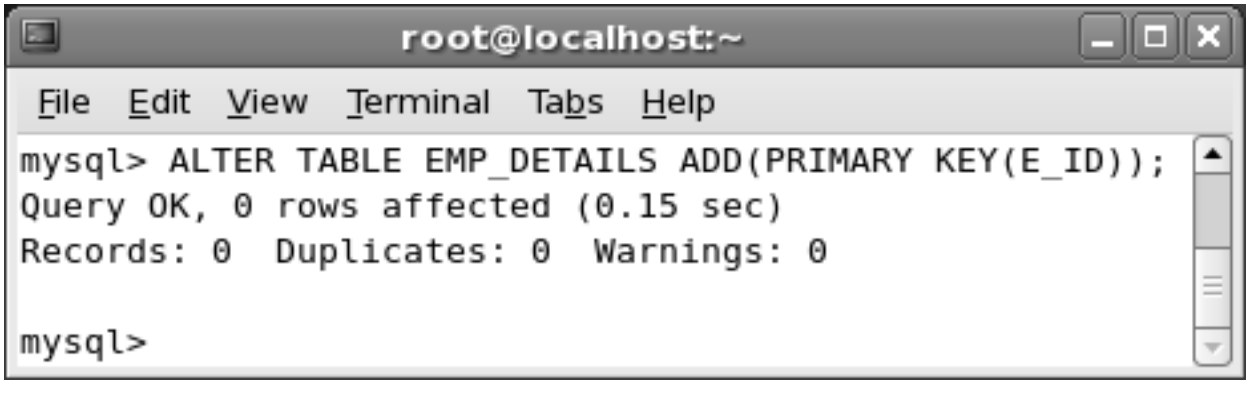


Figure 6.27: Adding Primary Key

To modify a column definition, use the following syntax:

```
ALTER TABLE table_name MODIFY [COLUMN] create_definition;
```

where,

`ALTER TABLE` – specifies to edit the table

`table_name` – specifies the name of the table

`MODIFY [COLUMN]` – specifies to change the column structure

`create_definition` – specifies the new rules for the column

For example, to modify `DESIGNATION` column from `CHAR(50)` to `CHAR(20)` data type of `EMP_DEPARTMENT` table, enter the following command at the command prompt:

```
ALTER TABLE EMP_DEPARTMENT MODIFY DESIGNATION CHAR(20);
```

Figure 6.28 displays the output of the command.

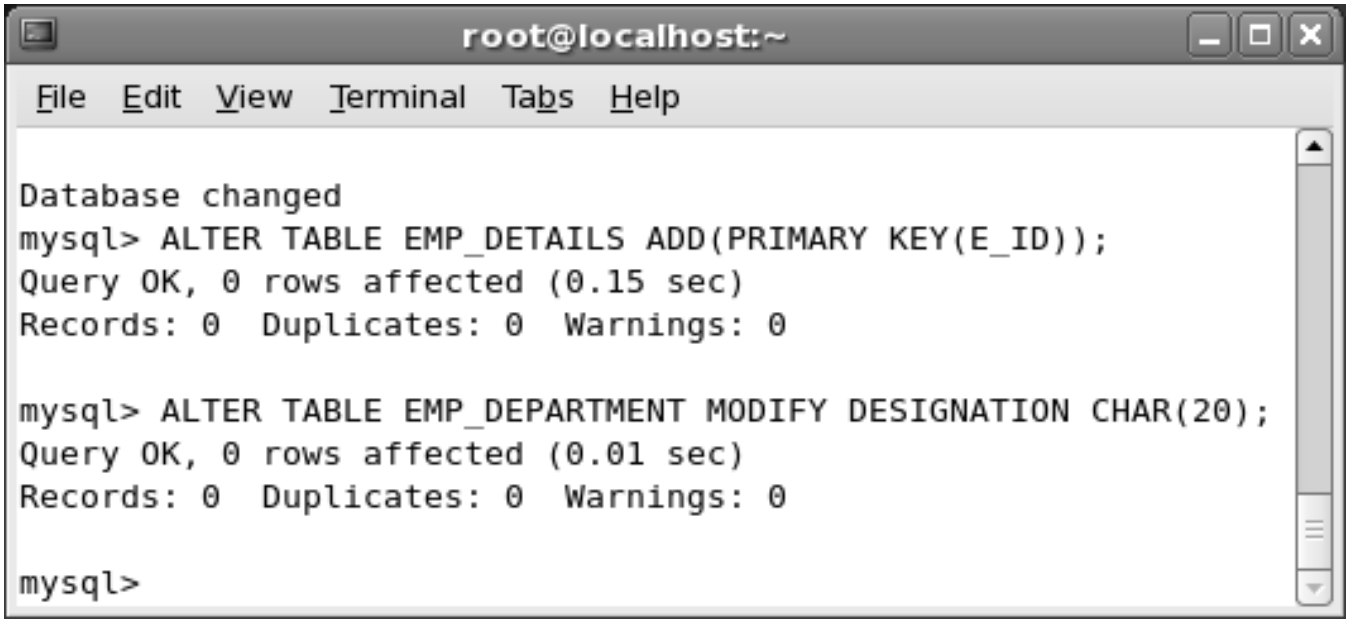


Figure 6.28: Modifying a Designation Column

To drop a column from a table, use the following syntax:

```
ALTER TABLE table_name DROP [COLUMN] column_name;
```

where,

`ALTER TABLE` – modifies the table structure

`table_name` – specifies the name of the table to modify

`DROP [COLUMN]` – removes the column from the table

`column_name` – specifies the name of the column to remove from the table

This command removes a column from the table.

For example, to remove the column `CITY` from `EMP_DETAILS` table, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS DROP COLUMN CITY;
```

Figure 6.29 displays the output of the command.

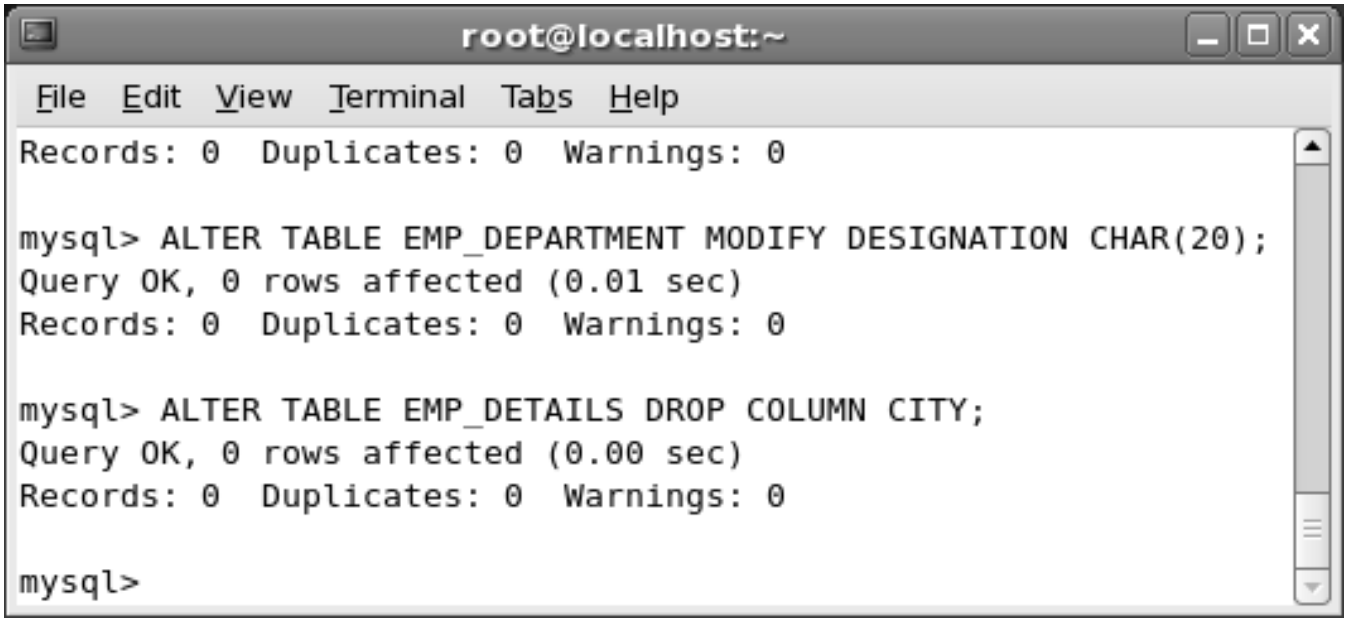


Figure 6.29: Deleting a Column

To drop a primary key of a column from the table, use the following syntax:

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

where,

`ALTER TABLE` – edits the table structure

`table_name` – specifies the name of the table to modify

`DROP PRIMARY KEY` – removes the primary key from the table

Concepts

Note: The `DROP PRIMARY KEY` option deletes the primary index. MySQL will delete the first indexed column, if a primary index is not defined in the table.

For example, to remove primary key constraint from `EMP_DETAILS` table, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS DROP PRIMARY KEY;
```

To change the name of a table, use the following syntax:

```
ALTER TABLE table_name RENAME new_table_name;
```

where,

`ALTER TABLE` – edits the table structure

`table_name` – specifies the name of the table to modify

`RENAME` – changes the name of the table

`new_table_name` – specifies the new name for the table

For example, to rename the table from `SALARY_DETAILS` to `EMP_SALARY`, enter the following command at the command prompt:

```
ALTER TABLE SALARY_DETAILS RENAME EMP_SALARY;
```

Figure 6.30 displays the output of the command.

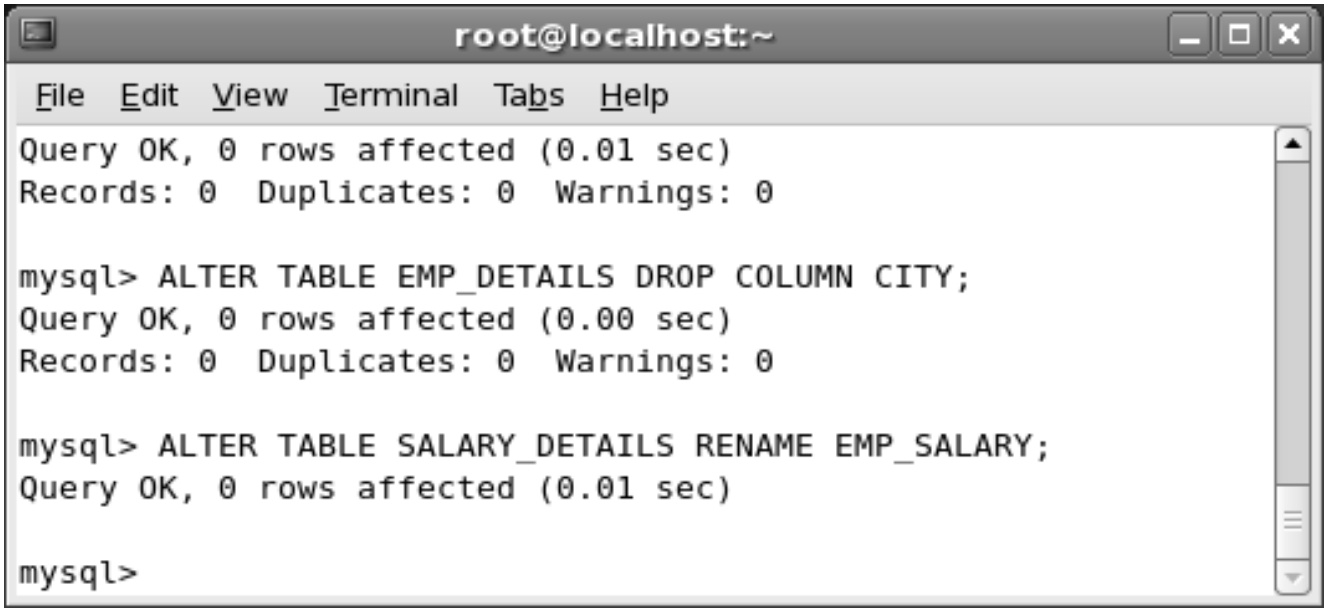


Figure 6.30: Renaming a Table

Following are some of the points to be remembered while working with `ALTER TABLE` command:

- MySQL deletes the index values of columns when you drop columns from a table. MySQL also removes the index when you drop or delete all indexed columns.
- If `DESCRIBE table_name` displays that the column specification, modified using the `ALTER TABLE` command, has not changed, then it is possible that the modification is ignored because the size of the column to be changed may be greater than or less than the required size length. For example, you try to change `VARCHAR` column to `CHAR`, MySQL may not allow the modification to `CHAR` if the respective column length is not less than four.
- A single `ALTER TABLE` command can contain several `ADD`, `ALTER`, `DROP`, and `CHANGE` clauses.

Note: To use `ALTER TABLE`, you will require `SELECT`, `INSERT`, `DELETE`, `UPDATE`, `CREATE`, and `DROP` privileges on the table.

6.5 Deleting Table Definitions Using DROP Command

MySQL provides the `DROP TABLE` command to remove or delete tables from a database. This command removes table definition, data, indexes, triggers, constraints, and permissions for that table.

Note: The `DROP` command should be used carefully because it removes all the data from a table along with table definitions.

The syntax for the DROP command is:

```
DROP TABLE [IF EXIST] table_name[table_name1,…][RESTRICT | CASCADE];
```

where,

table_name – specifies the name of the table to delete

IF EXIST - prevents error occurrence while executing the command if the table does not exist

RESTRICT - specifies that if any dependencies exist, MySQL will not delete the table, if dependencies exist. If MySQL, encounters a dependency, it returns an error but does not delete the table.

CASCADE - specifies to remove dependencies before deleting the table

For example, to remove a table named SAMPLE from the EMPLOYEE database, enter the following command at the command prompt:

```
DROP TABLE SAMPLE;
```

Figure 6.31 displays the output of the command.

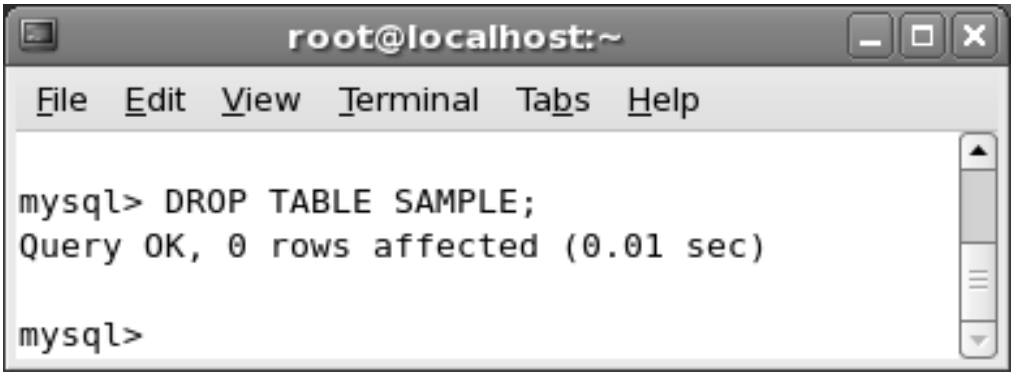


Figure 6.31: Displays DROP TABLE SAMPLE

You can use the IF EXIST clause with DROP command if you are unsure about the existence of a table in the database.

Note: The DROP TABLE command cannot be used to delete tables that are referencing FOREIGN KEY. You will have to create a sample table to execute this command. Do not drop the existing tables from the database.

Summary

- MySQL provides the SHOW command to view the list of databases and tables on the server, displays the server status, and user account privileges.
- The `ALTER` command enables modification of the characteristics or attributes of a table or a database.
- The `CHARACTER SET` clause defines the default character set of a database.
- The default collation of a database can be defined using the `COLLATE` clause.
- The `SELECT` command is used to retrieve data from a table.
- The FROM clause in the `SELECT` command specifies the table name whose records are to be retrieved.
- The `WHERE` clause in the `SELECT` command specifies conditions for retrieving data from the table. MySQL displays only those records that satisfy the conditions in the `WHERE` clause.
- Conditional and logical operators can be used with `SELECT` command to retrieve data after satisfying data retrieval conditions.
- The `DROP` command removes or deletes a table from a database. This command removes all the data and the table definition from the database.
- MySQL provides the IF `EXIST` option in the `DROP` command to check the existence of the table before you delete it.

Check Your Progress

1. You can view the columns of a table using the _____ command.

   a. SELECT

   b. VIEW

   c. SHOW

   d. USE

2. MySQL displays only the first 100 characters of each process in the SHOW PROCESSLIST command, if you do not specify the _____ clause.

   a. STATUS

   b. VARIABLE

   c. FULL

   d. INDEX

3. Which of the following is similar to the SHOW TABLE command except that it displays detailed information about each table?

   a. SHOW STATUS

   b. SHOW TABLE STATUS

   c. SHOW TABLES

   d. SHOW VARIABLES

4. Identify the command that performs actions on temporary copy of original database.

   a. MODIFY

   b. ALTER

Check Your Progress

c. `CHANGE`

d. `UPDATE`

5. In which file of the database directory does MySQL store the global characteristics attributes of a database?

a. `tables_priv`

b. `db`

c. `user`

d. `db.opt`

6. Which of the following commands will display all the records of EMPLOYEE table?

a. `SELECT ALL FROM EMPLOYEE;`

b. `SELECT COLUMNS FROM EMPLOYEE;`

c. `SELECT * FROM EMPLOYEE;`

d. `SELECT RECORDS FROM EMPLOYEE;`

7. You will use the _____ option of the DROP command to display the dependencies of a table.

a. `IF EXIST`

b. `RESTRICT`

c. `CASCADE`

d. `DROP TABLE`

Onlinevarsity

INDUSTRY BEST PRACTICES

Session 7

Implementing SQL Queries Using MySQL - I (Lab)

Objectives

At the end of this session, the student will be able to:

- *Modify the database.*
- *Use the SELECT command.*
- *Use the WHERE clause with the SELECT command.*
- *Modify the table structure.*
- *Delete a table from the database.*

The steps given in this session are detailed, comprehensive, and carefully thought through in order to meet the learning objectives and understand the command completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Modifying the Database

The `ALTER DATABASE` command modifies the global characteristics or attributes of a database. These attributes are stored in the `db.opt` file of the database directory. For example, to modify the `CHARACTER SET` clause using the `ALTER DATABASE` command, execute the following steps.

1. **To activate the PRODUCT_SALES database, enter the following command at the command prompt:**

```
USE PRODUCT_SALES;
```

Figure 7.1 displays the output of the command.

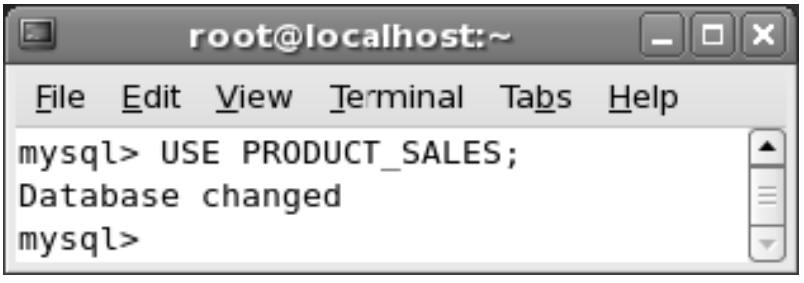


Figure 7.1: Activating PRODUCT_SALES Database

2. **To view the default character set for the PRODCUT_SALES database, enter the following command at the command prompt:**

```
SHOW CHARACTER SET;
```

Figure 7.2 displays the output of the command.

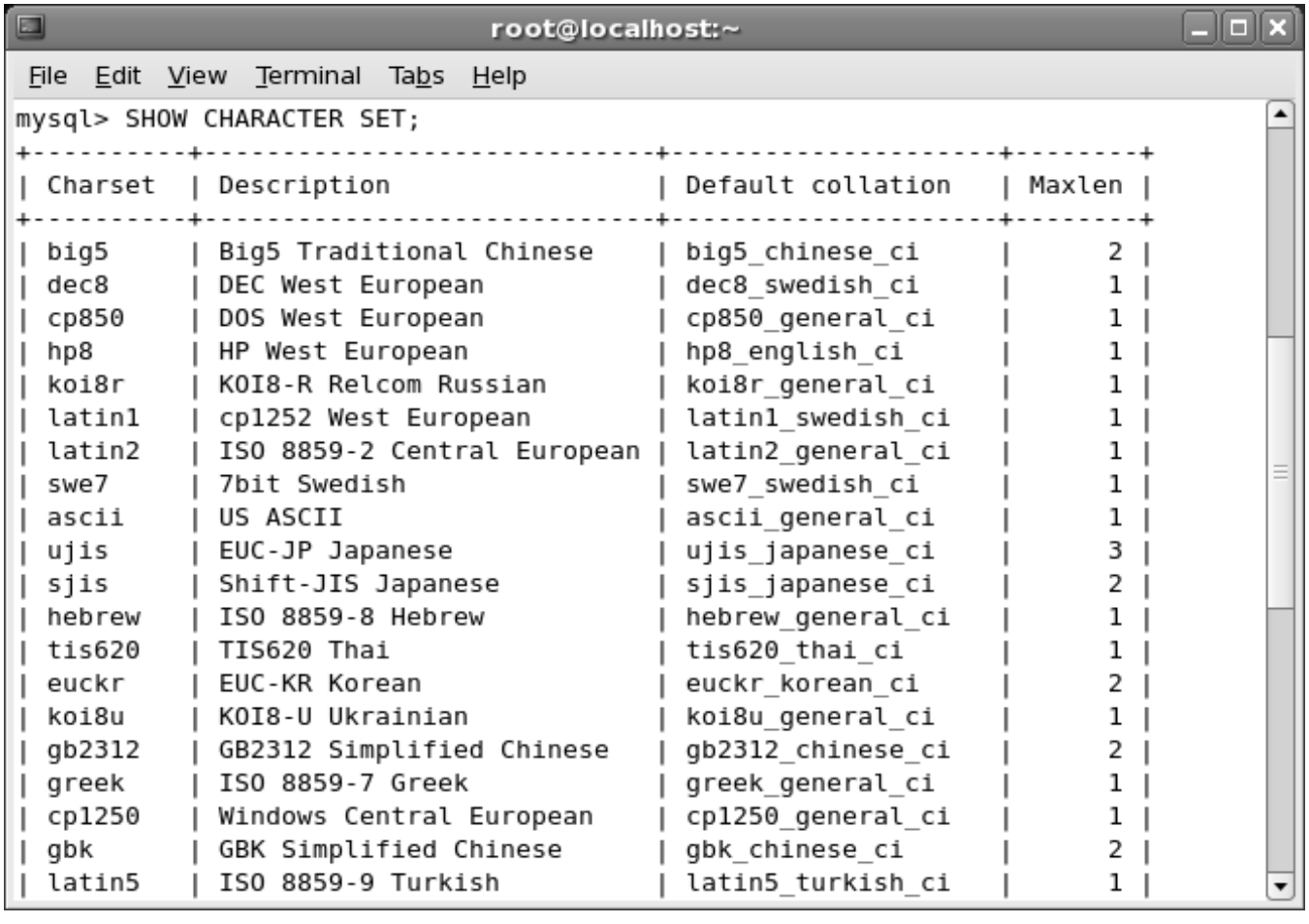


Figure 7.2: Displaying Default Character Set

3. **To modify the character set of the PRODUCT_SALES database, enter the following command at the command prompt:**

```
ALTER DATABASE PRODUCT_SALES DEFAULT CHARACTER SET macroman;
```

Figure 7.3 displays the output of the command.

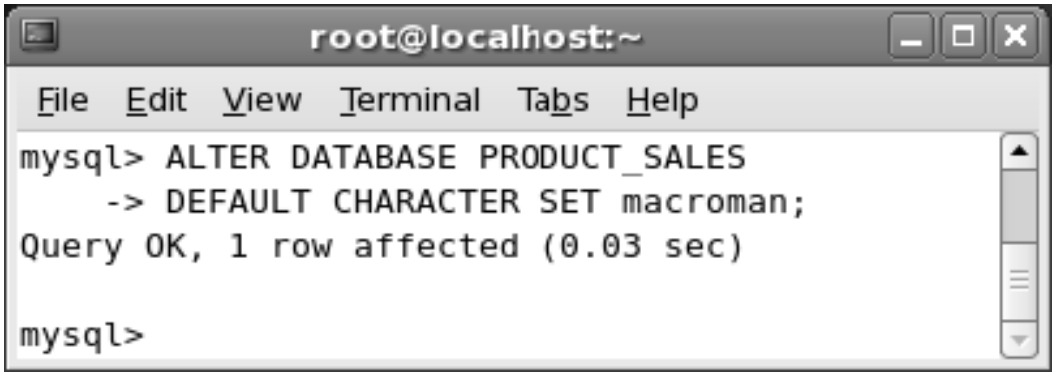


Figure 7.3: Altering the Database

Figure 7.3 indicates that the character set for the `PRODUCT_SALES` database has been modified.

Using SELECT Command

The `SELECT` command retrieves data from the tables in the database. The `WHERE` clause is used for retrieving records from the tables that satisfy the specified conditions. For example, to retrieve data from the `BOOK_DETAILS` table, execute the following steps.

1. **To activate the LIBRARY database, enter the following command at the command prompt:**

```
USE LIBRARY;
```

Figure 7.4 displays the output of the command.

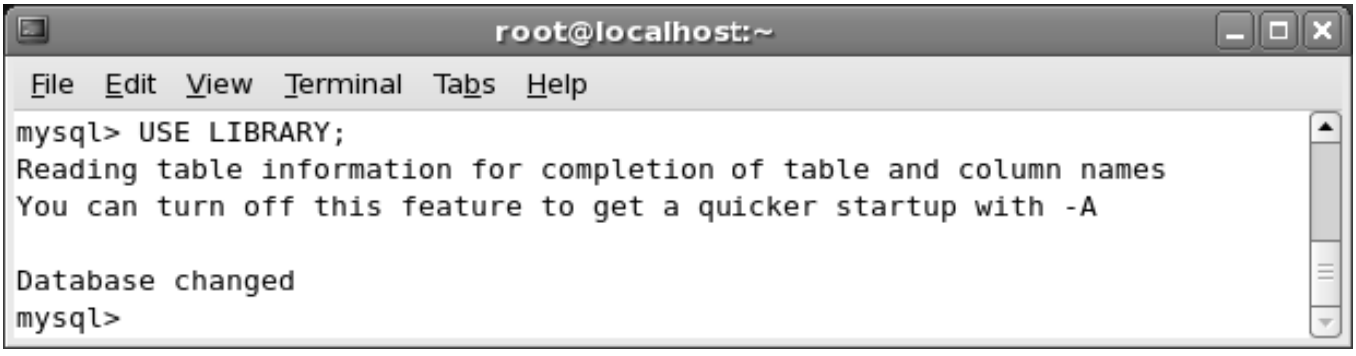


Figure 7.4: Activating Library Database

2. **In order to view, records from a table, you must enter data into the tables. To enter data in the BOOK_DETAILS table, enter the following command at the command prompt:**

```
INSERT INTO BOOK_DETAILS

VALUES(101, 'PROGRAMMING WITH C','GODFRIED','THIRD','C PROGRAMMING');
```

Figure 7.5 displays the output of the command.

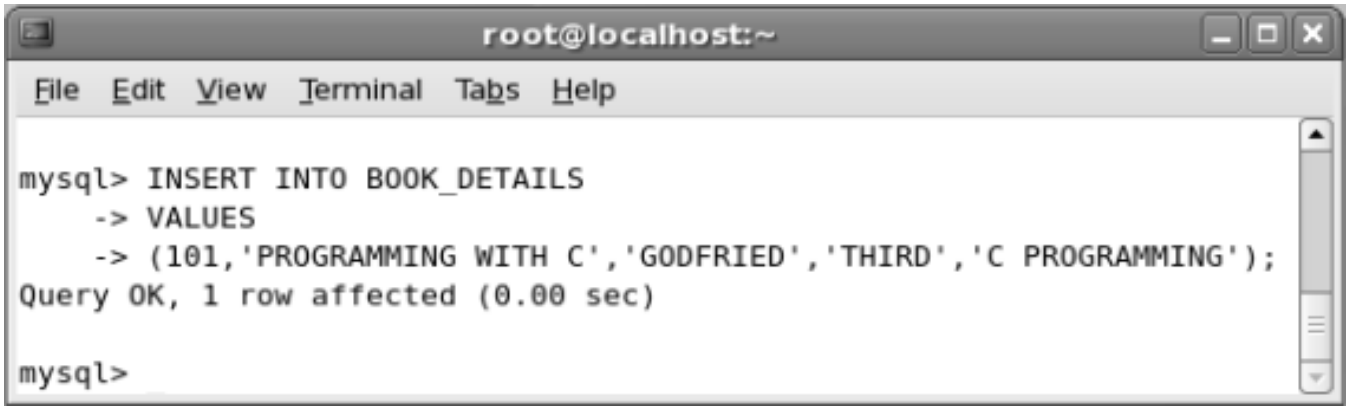


Figure 7.5: Adding Records to a Table

Similarly, add the remaining records to the BOOK_DETAILS table as listed in table 7.1

BOOK_ID	BOOK_NAME	AUTHOR	EDITION	DESCRIPTION
102	C++ PROGRAMMING	PETER NORTON	SECOND	PROGRAMMING WITH C++
103	UNIX	JOHN MACBILL	SECOND	UNIX OPERATING SYSTEM
104	LINUX	SCOTT MANN	FIRST	LINUX OPERATING SYSTEM
105	VISUAL BASIC	Black Book	FIRST	DESIGN USING VB

Table 7.1: Records for BOOK_DETAILS Table

Add the records to the CATEGORY table as listed in table 7.2.

USER_ID	TYPE	FACILITY	ISSUE_STATUS
100	REGULAR	ISSUE 3 BOOKS AT A TIME	0
101	ONE DAY MEMBER	ISSUE 1 BOOK AT A TIME	0
102	REGULAR	ISSUE 3 BOOKS AT A TIME	0
103	REGULAR	ISSUE 3 BOOKS	0

Table 7.2: Records for CATEGORY Table

Add the records to the ISSUE_DETAILS table as listed in table 7.3.

USER_ID	BOOK_ID	BOOK_NAME	DATE_OF_ISSUE	DATE_OF_RETURN	FINE
100	101	PROGRAMMING WITH C	2010-06-05	2010-11-05	60.00
101	102	C++ PROGRAMMING	2010-04-04	2009-04-20	0.00
102	103	UNIX	2010-09-05	2010-10-09	40.00
103	104	LINUX	2010-01-01	2010-01-11	0.00

Table 7.3: Records for ISSUE_DETAILS Table

Add the records to the USER_DETAILS table as listed in table 7.4.

USER_ID	NAME	LOCATION	PHONE_NO
100	TOBY	MEXICO	45637
101	BINCY	MEXICO	45663
102	DENVER	MEXICO	45123
103	HELEN	MEXICO	45867

Table 7.4: Records for USER_DETAILS Table

3. **The librarian wants to view the list of all books available in the Book_Details table. To view all the records from the BOOK_DETAILS table, enter the following command at the command prompt:**

```
SELECT * FROM BOOK_DETAILS;
```

Figure 7.6 displays the output of the command.

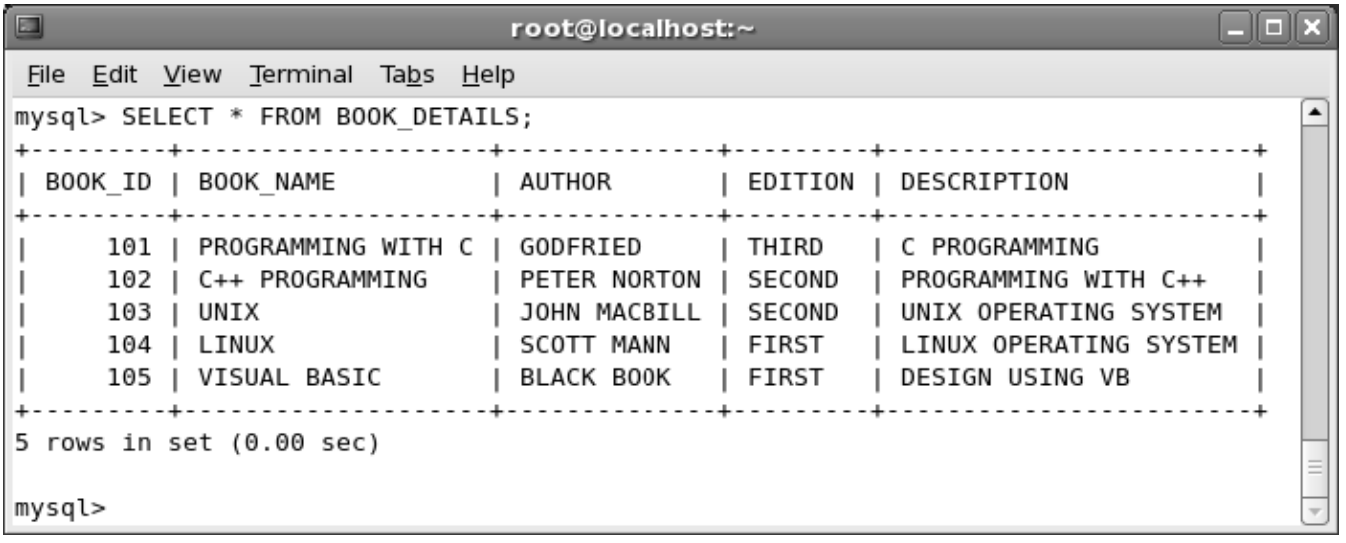


Figure 7.6: Displaying all Records from BOOK_DETAILS Table

4. **The librarian wants to view only the book name, author and the edition for books available in the Book_Details table. To view the BOOK_NAME, AUTHOR, and the EDITION fields from the BOOK_DETAILS table, enter the following command at the command prompt:**

```
SELECT BOOK_NAME, AUTHOR, EDITION FROM BOOK_DETAILS;
```

Figure 7.7 displays the output of the command.

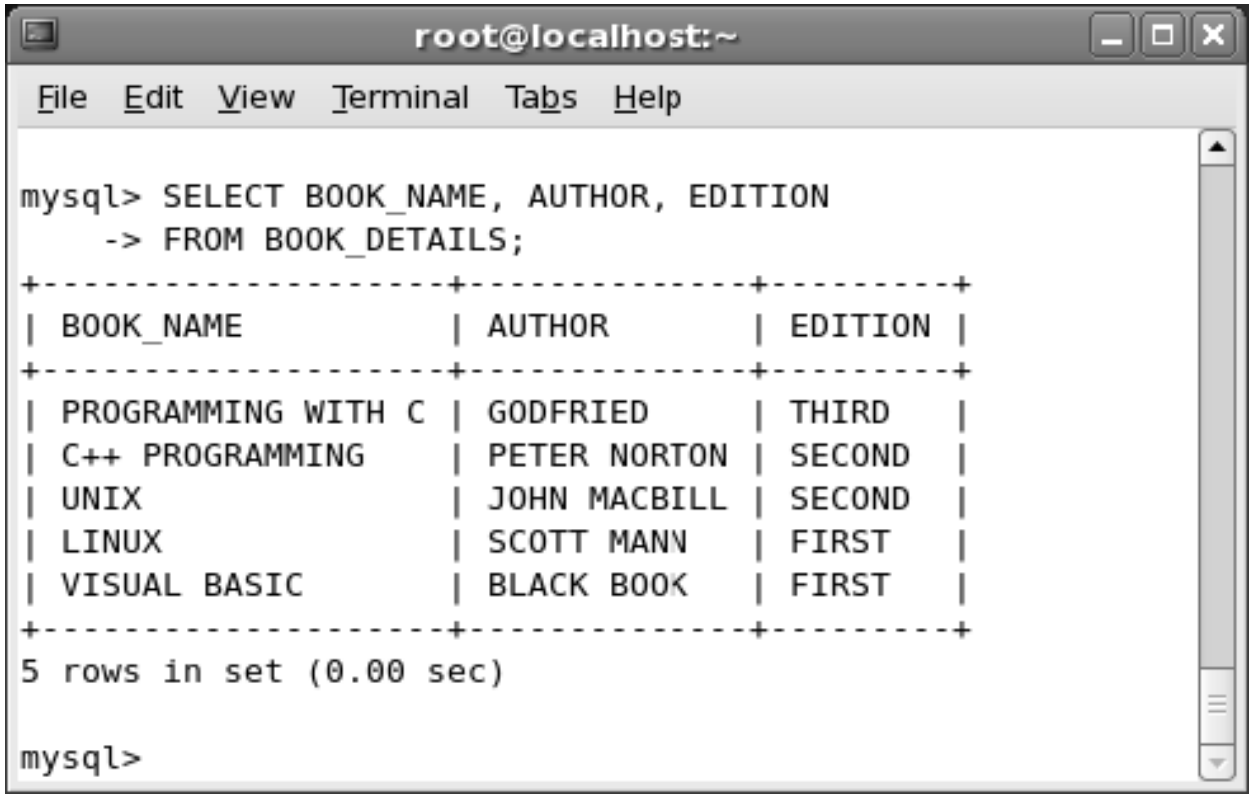


Figure 7.7: Displaying Specific Columns of the BOOK_DETAILS Table

5. **A student wants to be a member in the library. The student wants to know the different types of membership available in the library management system. The librarian wants to view the types of membership from the CATEGORY table. To view the distinct records from the CATEGORY table, depending on the column TYPE, enter the following command at the command prompt:**

```
SELECT DISTINCT TYPE FROM CATEGORY;
```

Figure 7.8 displays the output of the command.

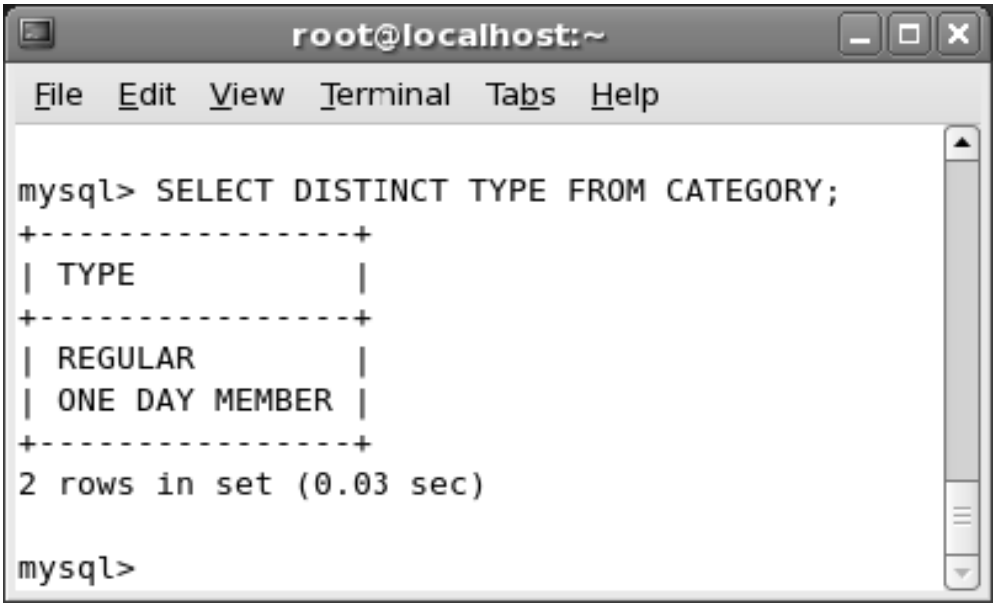


Figure 7.8: Viewing Unique Membership types from CATEGORY table

6. **The librarian has received a request from different students for Linux books. The librarian wants to know whether Linux books exist in the library. To view those records where the book name is LINUX, enter the following command at the command prompt:**

```
SELECT * FROM BOOK_DETAILS WHERE BOOK_NAME = 'LINUX';
```

Figure 7.9 displays the output of the command.

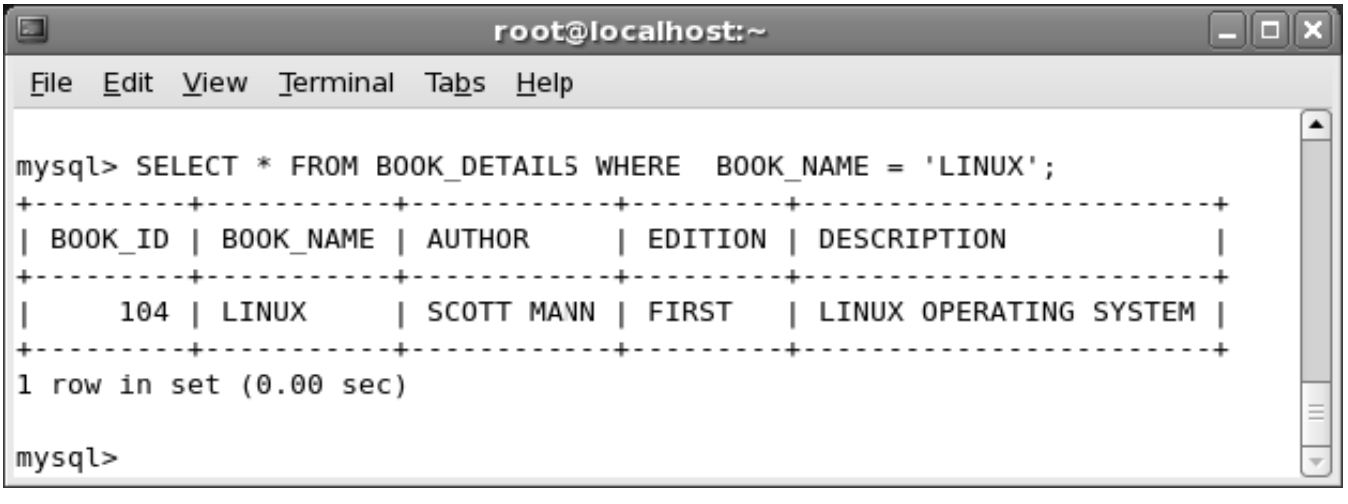


Figure 7.9: Displaying Records Where the Book_Name is LINUX

7. **To view the BOOK_ID, BOOK_NAME, and the EDITION fields of those books whose BOOK_ID is greater than 102, enter the following command at the command prompt:**

```
SELECT BOOK_ID, BOOK_NAME, EDITION FROM BOOK_DETAILS WHERE
BOOK_ID > 102;
```

Figure 7.10 displays the output of the command.

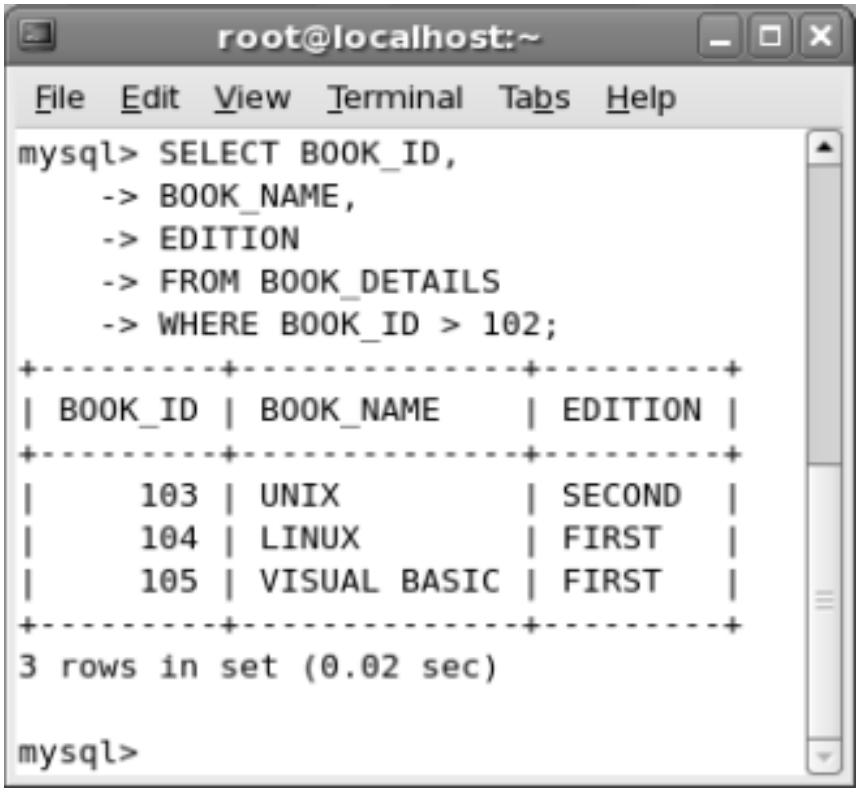


Figure 7.10: Displaying Records Where BOOK_ID > 102

8. **A student is preparing for examinations using different books. The student requires books that are marked SECOND or THIRD edition. The librarian wants to know if these books exist in the library. To view the second or third edition books from the BOOK_DETAILS table, enter the following command at the command prompt:**

```
SELECT * FROM BOOK_DETAILS

WHERE EDITION = 'THIRD' OR EDITION = 'SECOND';
```

Figure 7.11 displays the output of the command.

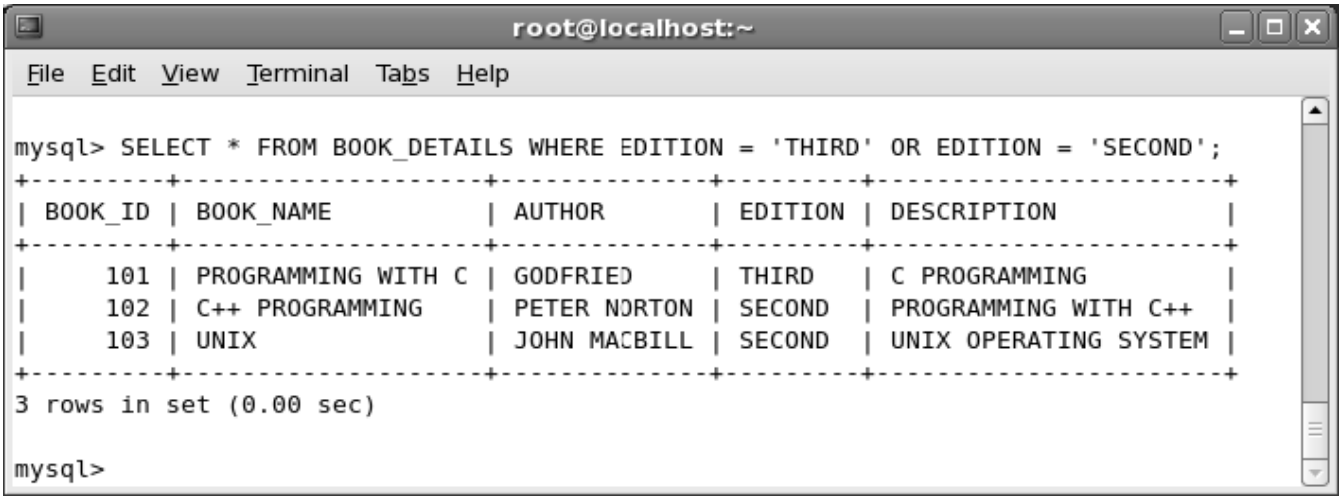


Figure 7.11: Displaying Records Where the Book Edition is Second or Third

In figure 7.11, two conditions have been specified in the `WHERE` clause using the OR operator.

Modifying the Table

You can change the structure of a table with the `ALTER` command. This command enables you to add or delete columns or constraints, create or destroy indexes, change or modify the type of existing columns, rename columns or the table, and change the type of the table.

1. **The librarian wants to sort the books for any subject based on publisher names. This column will be added to the BOOK_DETAILS table. To add a new column, PUBLISHER_NAME, to the BOOK_DETAILS table, enter the following command at the command prompt:**

```
ALTER TABLE BOOK_DETAILS ADD PUBLISHER_NAME CHAR(15) NULL;
```

Figure 7.12 displays the output of the command.

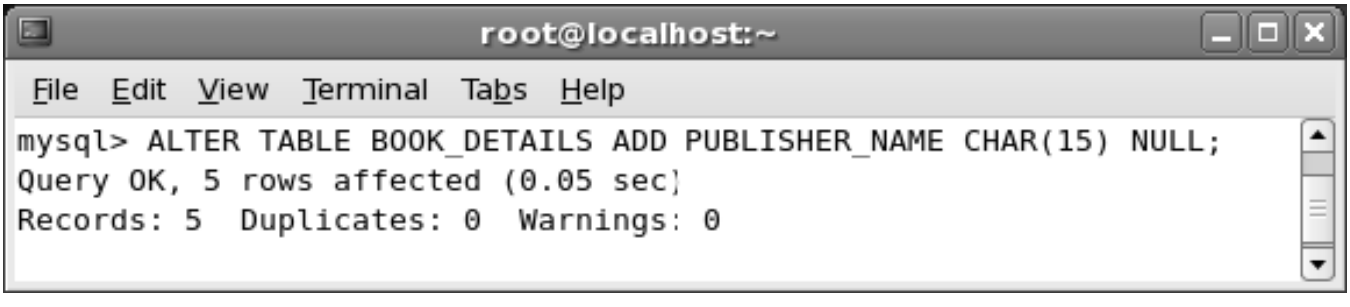


Figure 7.12: Adding a New Column Using ALTER command

2. **The librarian wants to ensure that the new column, PUBLISHER_NAME has been added to the BOOK_DETAILS table. To view the table structure after adding the PUBLISHER_NAME column, enter the following command at the command prompt:**

```
DESC BOOK_DETAILS;
```

Figure 7.13 displays the output of the command.

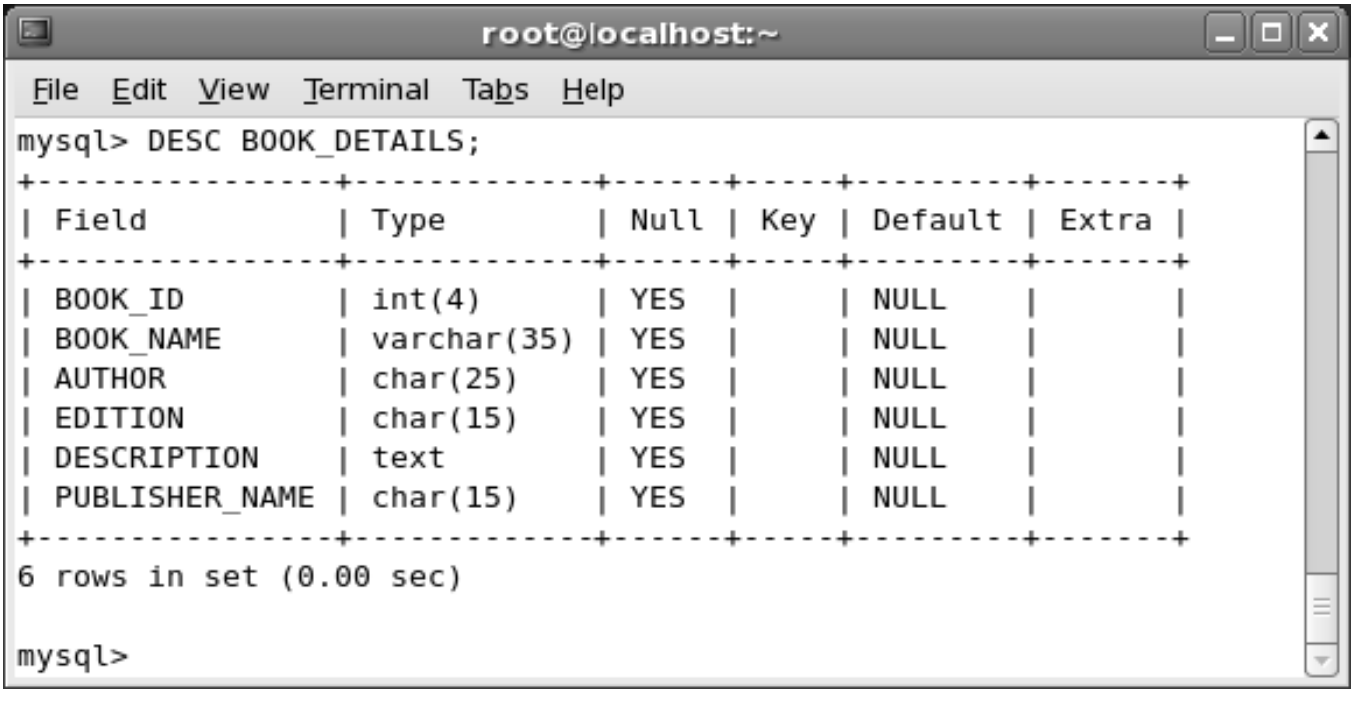


Figure 7.13: Table Description after Adding a Column

3. **The IT department wants to change the name of the BOOK_DETAILS table to match their records. To rename the table from BOOK_DETAILS to BOOK_TABLE, enter the following command at the command prompt:**

```
ALTER TABLE BOOK_DETAILS RENAME BOOK_TABLE;
```

Figure 7.14 displays the output of the command.

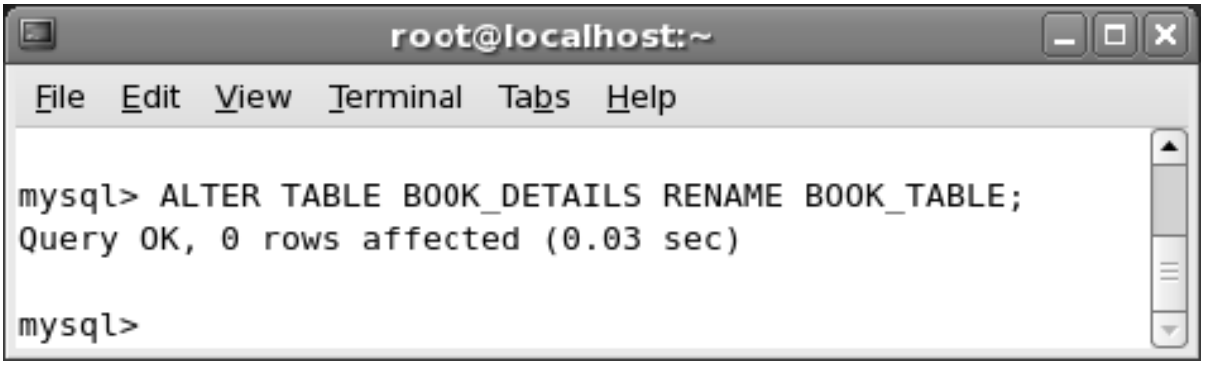


Figure 7.14: Renaming the Table

4. **A primary key can be defined to accept unique values. The librarian wants to use the primary key constraints in the table containing book details so that the name of the books does not duplicate in the table. To define a primary key on the BOOK_ID field in the BOOK_TABLE table, enter the following command at the command prompt:**

```
ALTER TABLE BOOK_TABLE MODIFY BOOK_ID INT(3) PRIMARY KEY;
```

Figure 7.15 displays the output of the command.

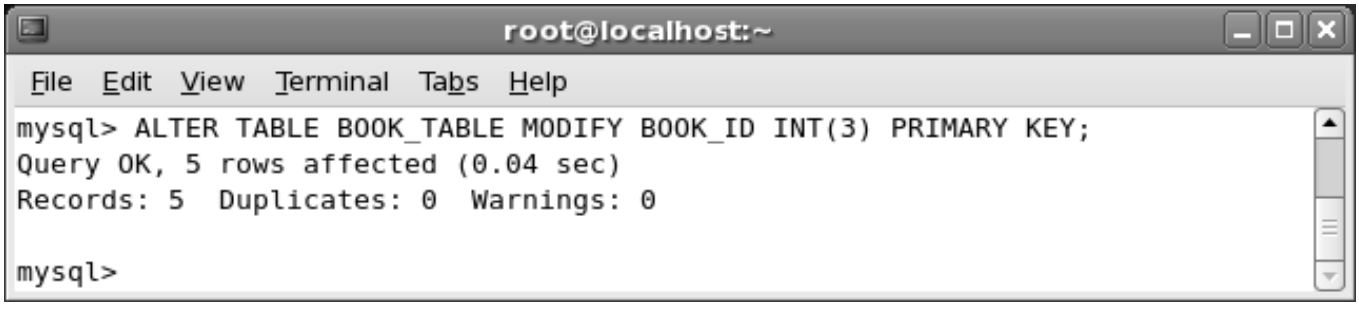


Figure 7.15: Defining a Primary Key

5. **After defining the primary key constraint the librarian wants to view the table structure, enter the following command at the command prompt:**

```
DESC BOOK_TABLE;
```

Figure 7.16 displays the output of the command.

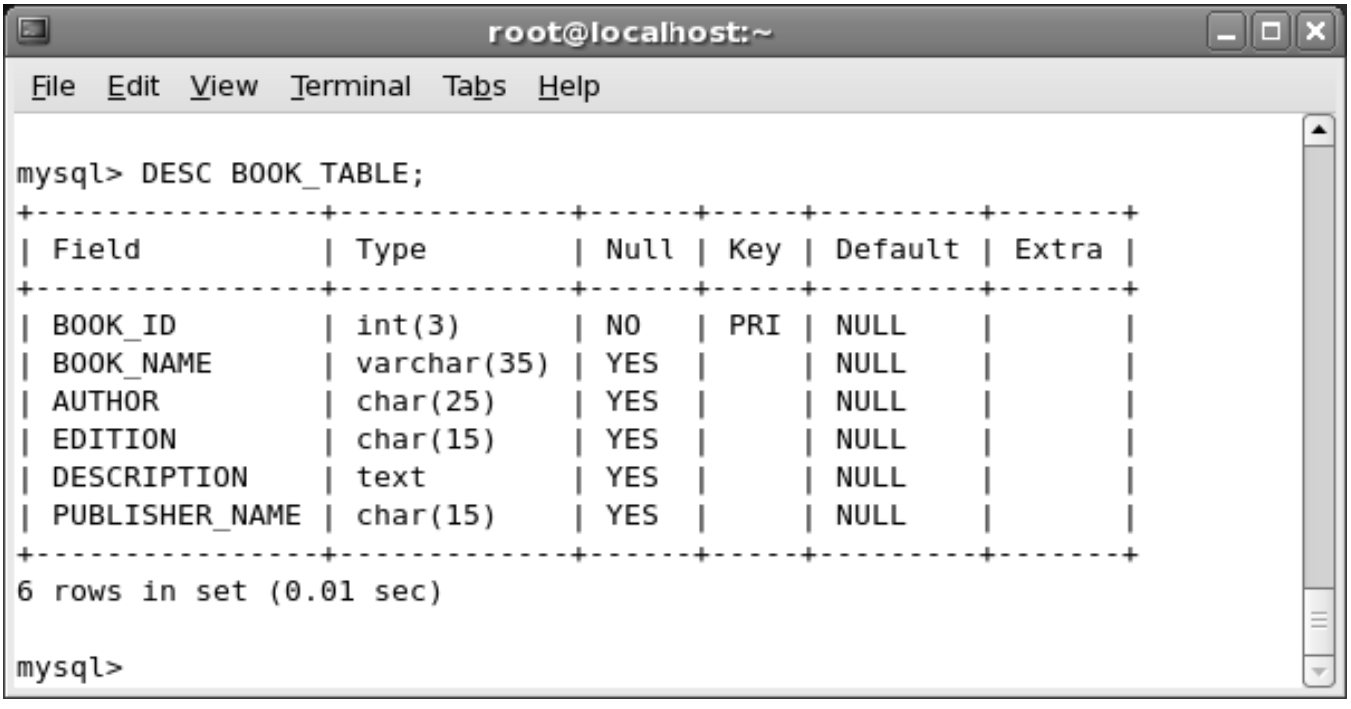


Figure 7.16: Table Structure after Defining Primary Key

6. **The librarian has identified problems to the BOOK_DETAILS table because of the addition of the new column PUBLISHER_NAME and wants to remove that column from the table. To delete the PUBLISHER_NAME column from the BOOK_TABLE, enter the following command at the command prompt:**

```
ALTER TABLE BOOK_TABLE DROP PUBLISHER_NAME;
```

Figure 7.17 displays the output of the command.

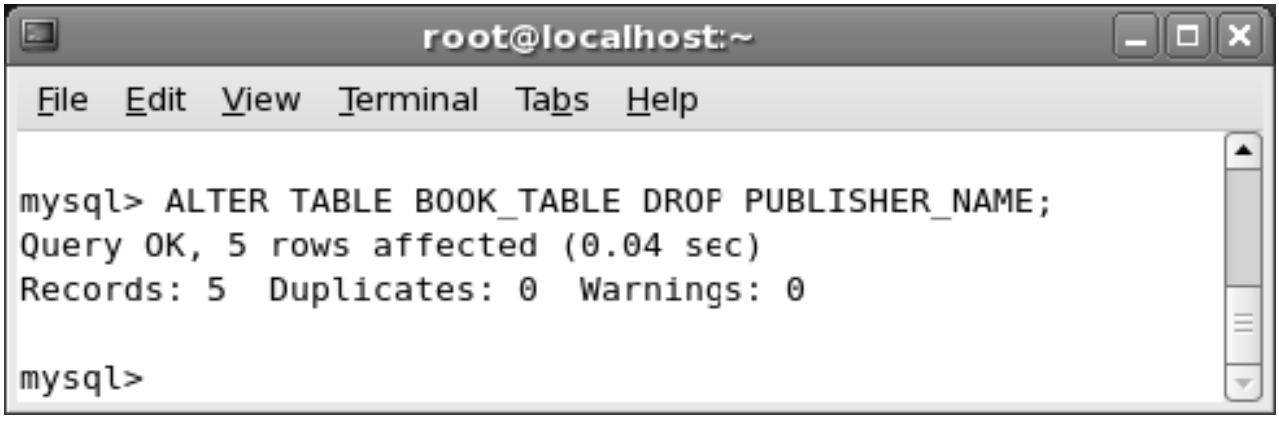


Figure 7.17: Deleting a Column from the Table

7. **After deleting the column, the librarian wants to ensure that the column no longer exists in the table. To view the structure of BOOK_TABLE after deleting the PUBLISHER_NAME column, enter the following command at the command prompt:**

```
DESC BOOK_TABLE;
```

Figure 7.18 displays the output of the command.

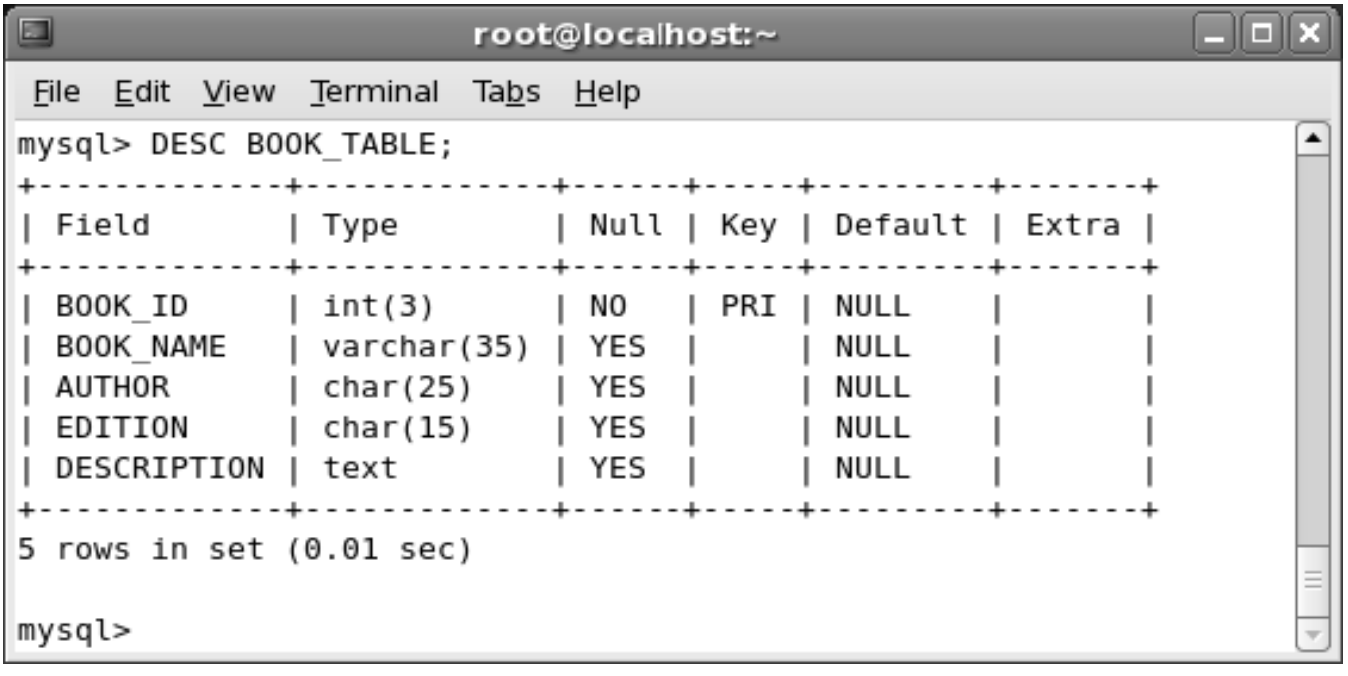


Figure 7.18: BOOK_TABLE Description after Modification

8. **The librarian has procured new books on various subjects. The ID for these new books are not decided. An error is generated when the librarian attempts to add book details without the BOOK_ID in the BOOK_TABLE. To resolve this, the librarian has to remove the primary key of the table. To drop the primary key on the BOOK_ID column from BOOK_TABLE, enter the following command at the command prompt:**

```
ALTER TABLE BOOK_TABLE DROP PRIMARY KEY;
```

Figure 7.19 displays the output of the command.

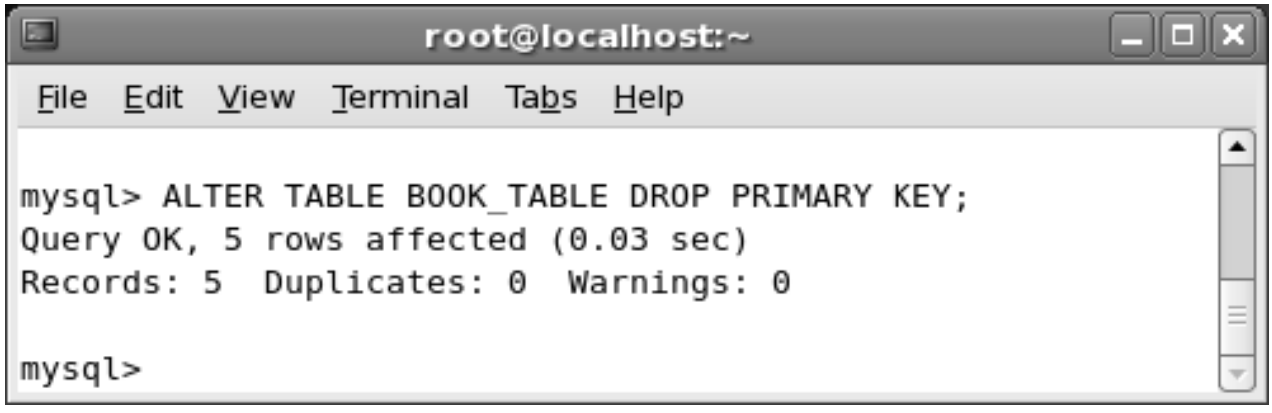


Figure 7.19: Removing Primary key from the table

9. **To add the new books to the database without any errors, the librarian wants to ensure that the primary key has been removed from the BOOK_ID column of the BOOK_TABLE. To view the structure of BOOK_TABLE after deleting the primary key, enter the following command at the command prompt:**

```
DESC BOOK_TABLE;
```

Figure 7.20 displays the output of the command.

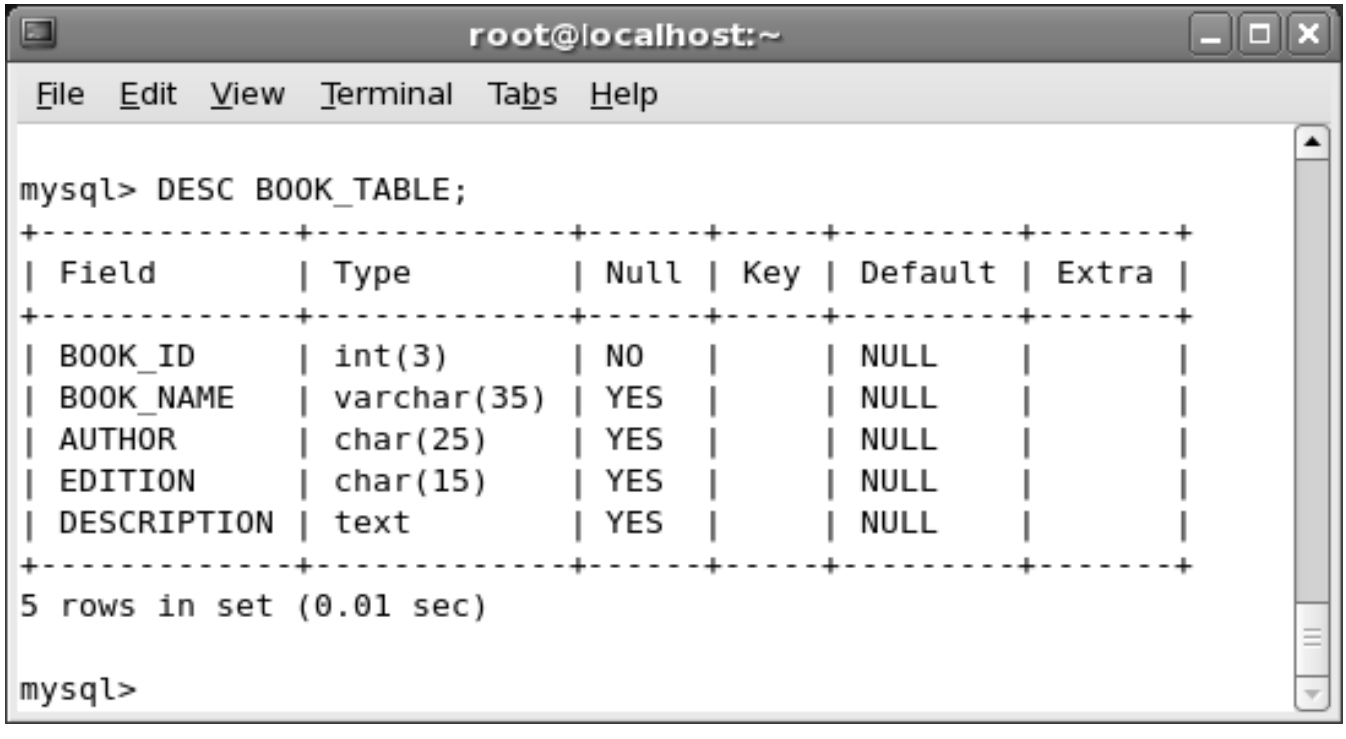


Figure 7.20: BOOK_TABLE Structure after Deleting Primary Key

Figure 7.20 displays that the primary key has been deleted from the table structure.

Using DROP Command

You can use the `DROP` command not only to remove a table from the database but also to remove keys, fields, and database from the server.

1. **To reactivate the LIBRARY database, enter the following command at the command prompt:**

```
USE LIBRARY;
```

Figure 7.21 displays the output of the command.

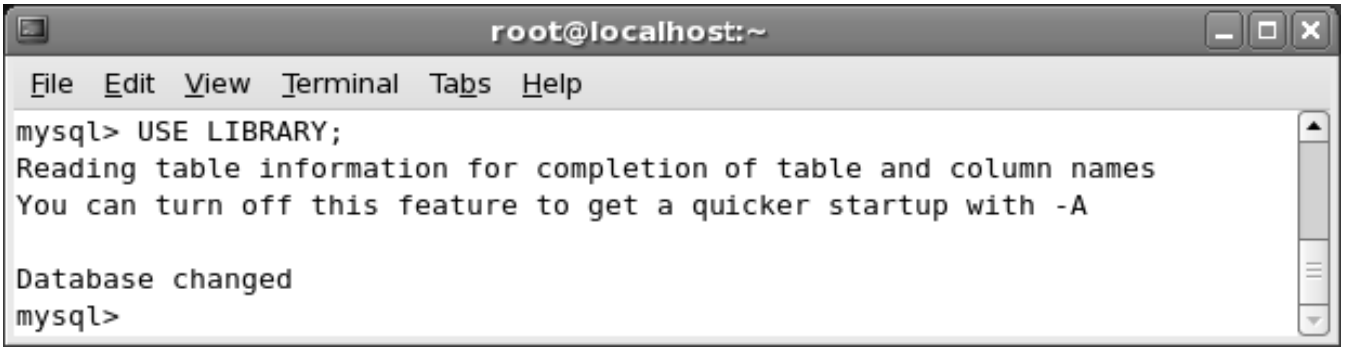


Figure 7.21: Reactivating LIBRARY Database

2. **To view the list of tables in the LIBRARY database, enter the following command at the command prompt:**

```
SHOW TABLES;
```

Figure 7.22 displays the output of the command.

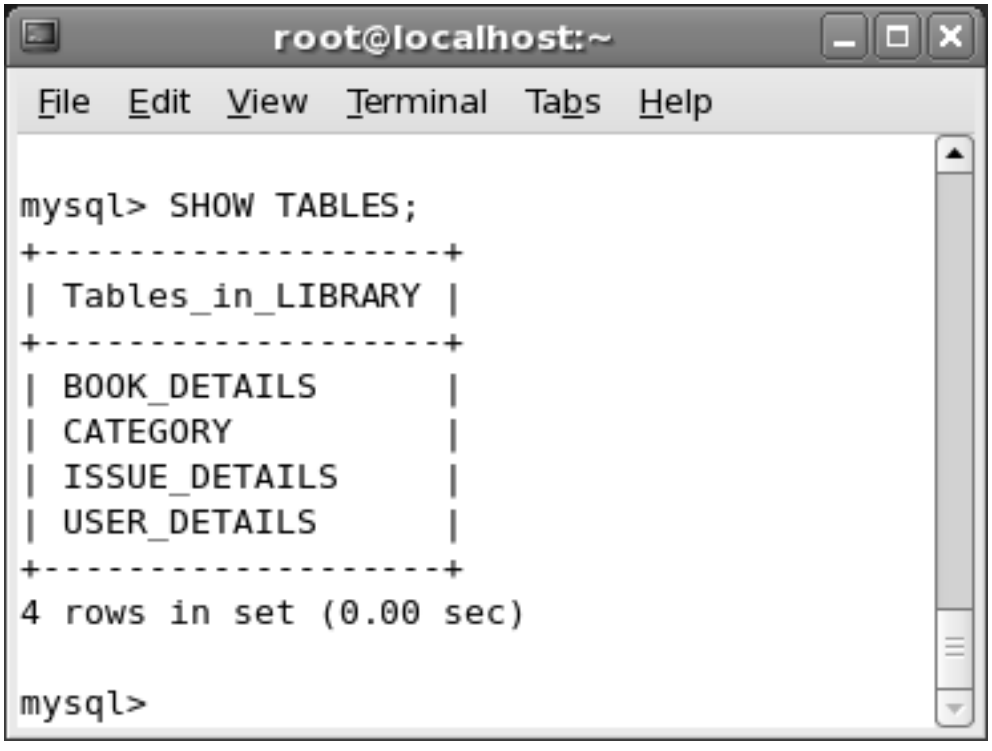


Figure 7.22: Displaying List of Tables Under LIBRARY Database

3. **The librarian is learning to delete unused tables from the database. Unused tables cannot be deleted from the database at random. In order to learn, how to delete a table, the librarian will first create a sample table TRIAL and then use the DROP TABLE command to delete it. To create a table TRIAL with the structure listed in table 7.5, enter the following command at the command prompt:**

```
CREATE TABLE TRIAL (Sample_ID INT(3), Sample_Name Char(5));
```

Figure 7.23 displays the output of the command.

Field	Data Type
Sample_ID	INT(3)
Sample_Name	Char(5)

Table 7.5: Trial Table Structure

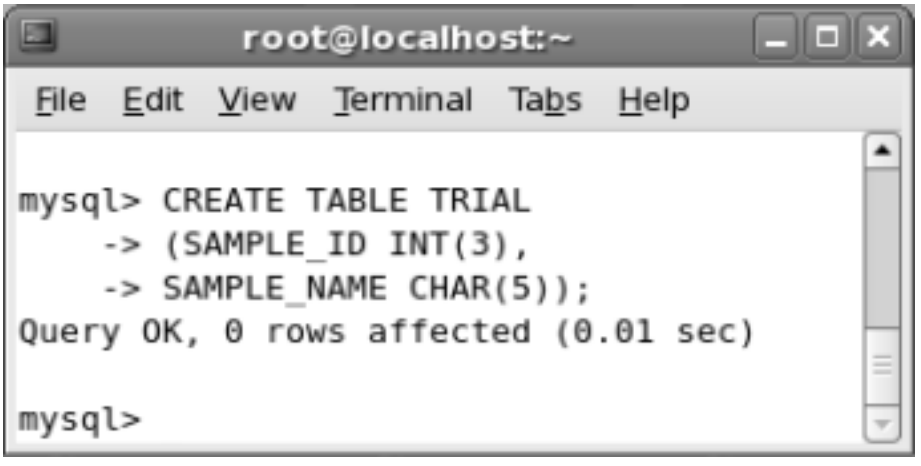


Figure 7.23: TRIAL Table Creation

4. **The librarian wants to ensure that the table TRIAL has been created in the LIBRARY database. To view the list of tables in the LIBRARY database, enter the following command at the command prompt:**

```
SHOW TABLES;
```

Figure 7.24 displays the output of the command.

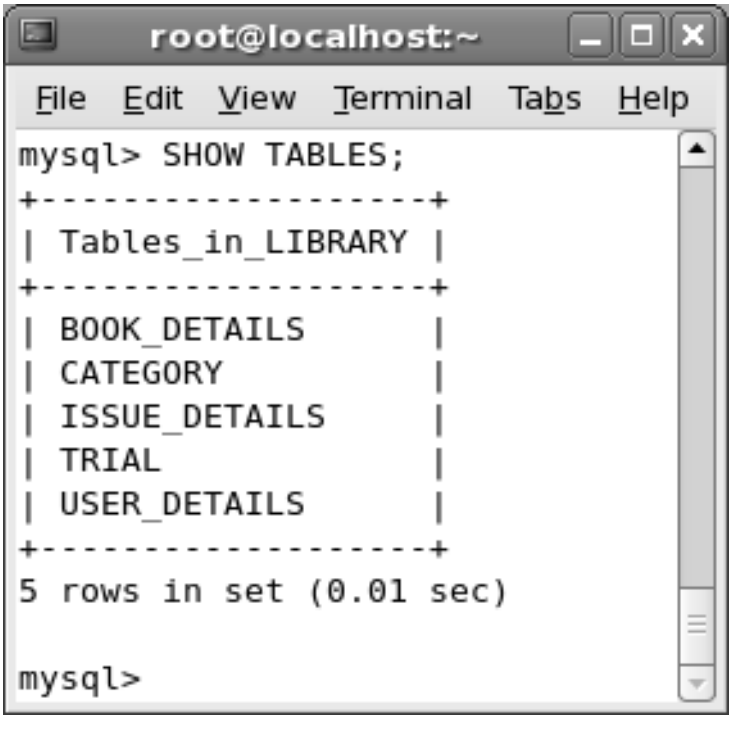


Figure 7.24: Displaying Tables In LIBRARY Database

5. **To drop the TRIAL table, enter the following command at the command prompt:**

```
DROP TABLE TRIAL;
```

Figure 7.25 displays the output of the command.

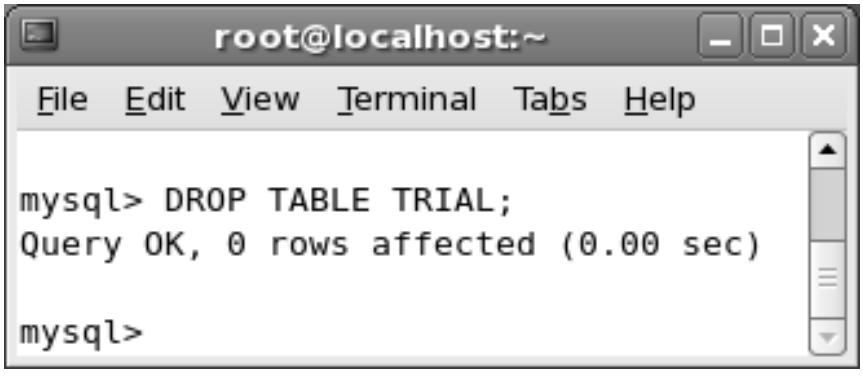


Figure 7.25: Deletion of TRIAL Table from the LIBRARY Database

6. **The librarian wants to ensure that the table TRIAL has been removed from the LIBRARY database To view the list of tables in the LIBRARY database after removing the TRIAL table, enter the following command at the command prompt:**

```
SHOW TABLES;
```

Figure 7.26 displays the output of the command.

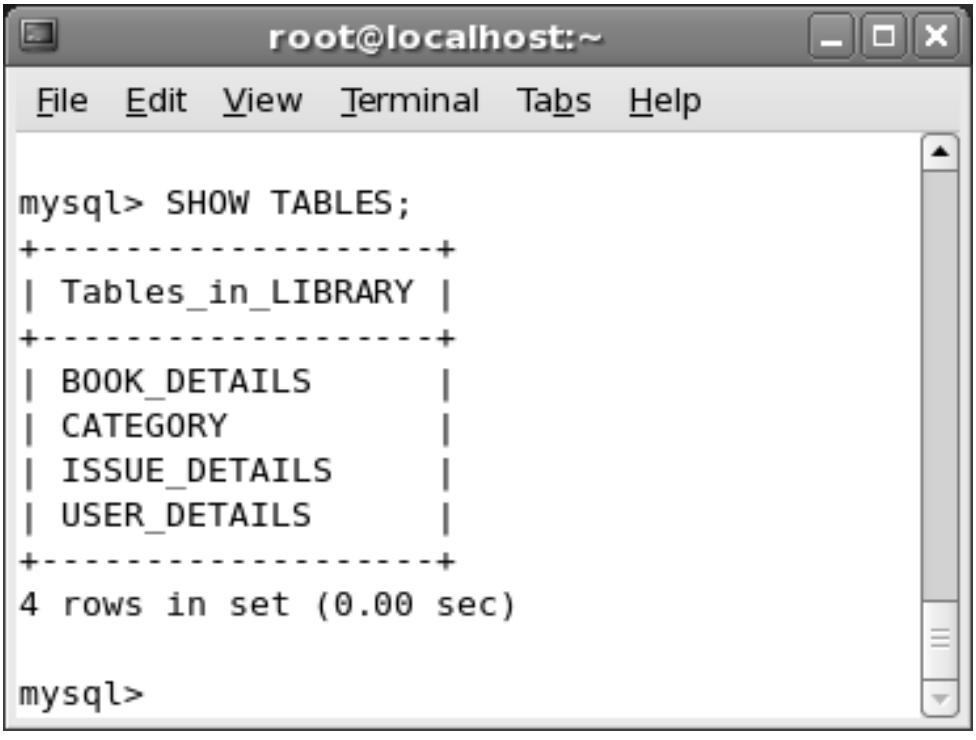


Figure 7.26: Tables in the LIBRARY Database after Removing TRIAL Table

Figure 7.26 indicates that the `TRIAL` table does not appear in the list of tables.

Do It Yourself

1. Create a `Computer_Course` database and then create a `Course_Table` in the `Computer_Course` database with the following structure:

Field Name	Data Type
COURSE_NAME	VARCHAR(20)
DURATION	INT(5)
START_DATE	DATE
FEES	DECIMAL(5,2)

2. View the list of character sets.

3. Modify the character set of `Computer_Course` database.

4. Create a `PRODUCT_SALES` database and a `STOCK` table having the following table structure:

Field Name	Data Type
Product _Code	INT(4)
Product_Name	CHAR(15)
Description	TEXT
Cost Price	DECIMAL(8,2)

5. Enter the following records in the STOCK table:

Product_Code	Product_Name	Description	Cost Price
101	Bags	Leather Bags	150
102	Books	Notepads	10
103	Pens	Fine Tip Pens	20

6. Display all the records of the `PRODUCT_SALES` table.

7. Display `Product_Code`, `Product_Name`, and `Description` fields of the table.

8. Display records where the cost price is less than `25`.

9. Display records where the `Product_Name = Books`.

10. Add a new field `Quantity` in the table.

Do It Yourself

11. Add the primary key constraint to the `Product_Code` field.

12. Rename the table to `PRODUCT_DETAILS`.

13. Remove the `DESCRIPTION` field from the table.

Session 8

Implementing SQL Queries Using MySQL - II

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain the use of keys in a table*
- *Explain the use of indexes in a table*
- *Explain modification of tables*
- *Explain the use of the ORDER BY command*
- *Explain the use of the GROUP BY command*

8.1 Introduction

MySQL stores records in tables. You not only add data to the table after you create it but you must also regularly update the data stored in the database. The update operation may require replacing certain rows in a table. In addition, you will also have to delete data from the table that you no longer require. Besides performing the additions, modifications, and deletions of records, you can also define an index on a column to enhance the speed of data search and sort operations in a database. To organize the data efficiently in tables and for performing update and delete operations on tables, you need to have certain columns designated as keys. A primary key in the table uniquely identifies a record. A primary key does not allow duplication of records. An index specifies the location of the record in a database.

In this session, you will learn to create an index. You will also identify the different types of keys. In addition, you will learn to modify, group, and sort the data in a table.

8.2 Working with Keys and Indexes

Columns with key definitions have a unique value for each row. Indexing a column will help to enhance the speed of data search and sort operations on the database. MySQL creates an index file and searches for data according to the index file when you index a column.

8.2.1 Defining Keys

Keys are columns that uniquely identify information present in a table. Some of the different types of keys include primary key, composite key, and foreign key.

Concepts

➢ **Primary key**

A primary key is used to uniquely identify each row in a table. It can consist of one or more fields on a table. A primary key specifies that there cannot be a column in a table that contains two similar values. The primary key does not allow blank or `NULL` values for the column. All the key columns are declared as `NOT NULL`.

You must remember the following rules while defining a primary key:

- Contains a value and cannot contain a null value
- Is unique for each record

You can define a primary key when you create the table. The syntax for defining a primary key while creating a table is:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name col_name column
definition PRIMARY KEY (fieldname);
```

where,

`CREATE` – adds an object to the database

`TABLE` – adds a table to the database

`tbl_name` – specifies the name for the table to add to the database

`PRIMARY KEY` – creates a primary key on the specified column

`fieldname` – specifies the name of the column to define the primary key

MySQL provides different ways to define a primary key for a table. You can define the primary key on a table while creating it. You can also modify an existing table structure and define a primary key using the `ALTER TABLE` command. In addition, you can modify the existing primary key on a table and specify conditions for the primary key. However, you will have to remember the rules for a primary key while defining or modifying it.

For example, you have already created a primary key for the `EMP_DETAILS` table. You will now modify this key as `NOT NULL` and `AUTO_INCREMENT` so that this primary key does not accept or contain a `NULL` value.

To modify the primary key as `NOT NULL` and `AUTO_INCREMENT`, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS MODIFY E_ID INT(3) NOT NULL PRIMARY KEY AUTO_
INCREMENT;
```

Figure 8.1 displays the output of the command.

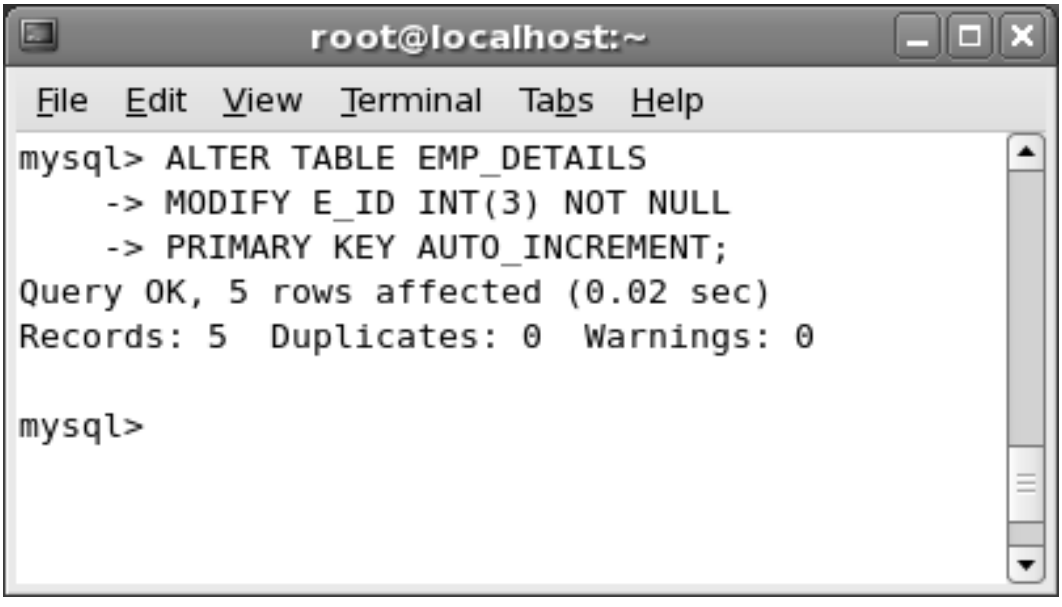


Figure 8.1: Defining a Primary Key

- **Composite Key**

When you use more than one column to define a primary key, then this type of a primary key is known as a composite key. The rules for defining a composite key are the same as the primary key.

- **Foreign Key**

A foreign key is used to establish relationship between two tables. A foreign key is a column in a table that has a corresponding primary key column in a different table. In other words, a foreign key in a table is used for referencing a primary key present in another table. The main goal of foreign key is to maintain referential integrity of the data.

In other words, it can be said that it is a database constraint that refers to a column in another table for reference. You can define a column as a foreign key only if both the tables have `InnoDB` as their storage engine. By default, MySQL assigns the `MyISAM` storage engine when you create a table. However, you can change the storage engine of the table using the `ALTER TABLE` command.

The advantages of using InnoDB as the storage engine are:

- Cancels INSERT and UPDATE commands that attempt to add records to the child table without having a corresponding record in the parent table
- Allows the foreign key to reference a group of columns in the parent table

To modify the storage engine of the EMP_DETAILS to InnoDB, enter the following command at the command prompt:

```
ALTER TABLE EMP_DETAILS ENGINE=INNODB;
```

Similarly, to modify the EMP_SALARY storage engine to InnoDB, enter the following command at the command prompt:

```
ALTER TABLE EMP_SALARY ENGINE=INNODB;
```

After modifying the storage engine, you can now define the foreign key.

The syntax to define a foreign key is:

```
ALTER TABLE tbl_name ADD [Constraint symbol] FOREIGN KEY [index_
name](index_col_name,...) REFERENCES table_name (index_col_name,...) [ON
DELETE {CASCADE|SET NULL|NO ACTION|RESTRICT}] [ON UPDATE {CASCADE|SET
NULL|NO ACTION|RESTRICT}];
```

where,

ALTER TABLE – modifies the table structure

tbl_name – specifies the name of the table to edit

ADD – appends an object to the table structure

FOREIGN KEY – defines the foreign key to the column

index_name – specifies the name of the index

index_col_name – specifies the name of the column defined as index

REFERENCES – defines the relationship with a primary key of another table

The options for defining a foreign key are listed in table 8.1.

Option	Description
ON DELETE	Deletes the child row if the related parent row is deleted
ON UPDATE	Modifies the child row if the related parent row is modified
CASCADE	Delete or update the row in the parent table and automatically delete or update corresponding rows in the child table
SET NULL	Deletes or modifies the row in the parent table and sets the corresponding row in the child table to NULL provided NOT NULL is not specified in the foreign key column
NO ACTION	Prevents a delete or update to the primary key if a foreign key value exists
RESTRICT	Prevents the deletion or updation of row in the parent table if a dependent child row exits

Table 8.1: Foreign Key Options

To define a foreign key, enter the following command at the command prompt:

```
ALTER TABLE EMP_SALARY
ADD FOREIGN KEY(E_ID)
REFERENCES EMP_DETAILS(E_ID);
```

Figure 8.2 displays the output of the command.

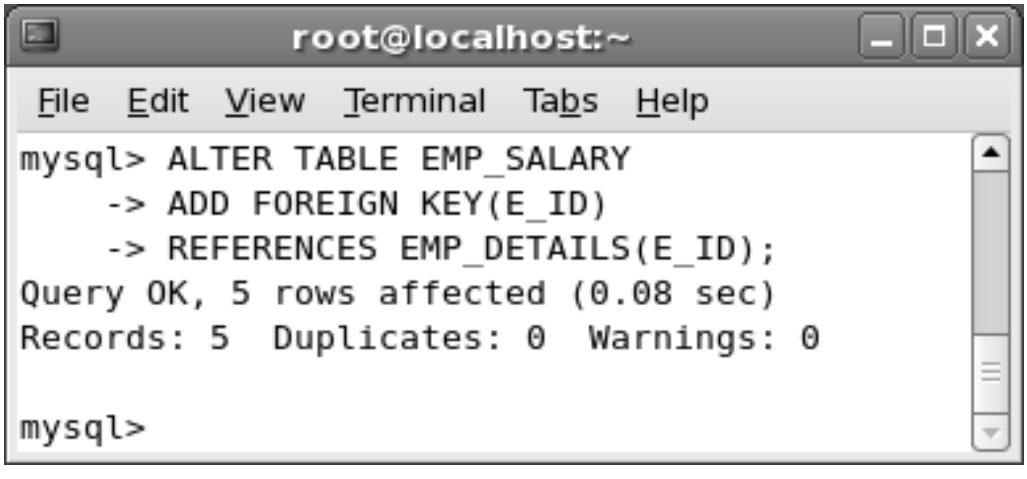


Figure 8.2: Defining a Foreign Key

Concepts

8.2.2 Creating Indexes

You can use indexes to improve the search characteristics of the database. For example, if you have a table with thousands of rows of data then searching for a particular row would take time. Indexing enables the search operation on a database to be faster. MySQL creates separate files to store and track indexes when you define an index for a table.

MySQL allows upto 64 indexes for each table. Each index allows upto 16 columns to be included. You can define a multicolumn index when you have to search on the same set of multiple columns. Indexing improves the speed of data retrieval. Indexing also has certain disadvantages. The process of altering data in a database becomes slow. The process such as update, insert, and delete require more time to execute. MySQL requires this additional time to record the changes in the index file also.

In MySQL, indexes can be created either during the initial Data Definition Language (DDL) to create the table or during later operations. You can index the field that:

- Is used in the `WHERE` clause of query
- Is used in `ORDER BY` clause of the query
- Is having a unique value

If you do not specify an index key, MySQL automatically indexes the primary key.

The syntax to create an index for a table is:

```
CREATE INDEX index_name ON tablename (column1,column2,..., columnN);
```

where,

`CREATE INDEX` – appends an index to the table

`index_name` – specifies the name for the index

`tablename` – specifies the name of the table to create the index

`column1` – specifies the names of the columns to be indexed in the table

Note: MySQL enables you to create a unique index and eliminate the duplicate values. You can use the `UNIQUE` clause to define an unique index.

To create an index named INDEX1 on the columns ID and NAME while creating a SAMPLE table, enter the following command at the command prompt:

```
CREATE TABLE SAMPLE (ID INT(2)NOT NULL,NAME CHAR(10),INDEX INDEX1(ID,NAME),
UNIQUE INDEX(ID));
```

You have already created a table. You will now create an index on the DATE_OF_JOIN field in the EMP_DEPARTMENT table to search data.

To create an index, INDEX1 on EMP_DEPARTMENT, enter the following command at the command prompt:

```
CREATE INDEX INDEX1 ON EMP_DEPARTMENT(DATE_OF_JOIN);
```

Figure 8.3 displays the output of the command.

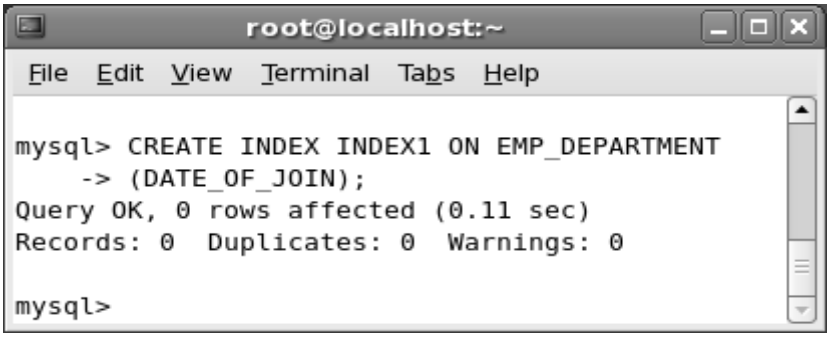


Figure 8.3: Creating an Index

8.3 Manipulating Tables

MySQL only defines the structure of the table when you create a table. This table structure does not contain any data. You will have to add data into the table after creating it. You can also replace or modify the data after adding it to the table. In addition, you can delete data from a table if you do not require the data.

8.3.1 Inserting Data into a Table

You are required to add data to a table after creating it. MySQL provides the INSERT command to add new records into a table. MySQL appends the newly inserted record after the last record in the table if the table already contains records.

Concepts

The syntax to insert data into a table is:

```
INSERT [LOW_PRIORITY | DELAYED] [IGNORE] [INTO] tbl_name [(col_name,...)]
VALUES ({expression | DEFAULT},...),(...),...
[ON DUPLICATE KEY UPDATE col_name=expression,...];
```

where,

`INSERT` – adds new row to an existing table

`tbl_name` – specifies the name of the table where MySQL will append the row

`col_name` – specifies the name of the column where MySQL will add the data

`VALUES` – defines the data that will be added to the table

The options for the `INSERT` command are listed in table 8.2.

Option	Description
`LOW_PRIORITY`	Postpones addition of records till all operations (reading) from the client have completed
`DELAYED`	Delays the insertion against all incoming `SELECT` statements
`IGNORE`	Overrides the error conditions for columns and adds the record to the table
`INTO`	Specifies the name of the table into which the columns are to be inserted
`col_name`	Specifies the name of the columns to insert values
`expression`	Includes a mathematical expression for a column
`DEFAULT`	Stores the default value into the specified columns
`ON DUPLICATE KEY UPDATE col_name=expression`	Updates the existing record if the addition of a new record creates a duplicate value in the `PRIMARY KEY` column

Table 8.2: INSERT Command Options

You can specify an expression for a column in the `INSERT` command. The condition to be satisfied is that the value for the column on which you build the expression should already be present.

MySQL allows you to insert values for specific column names. All other columns, if not specified, will contain a `NULL` value. If you have created columns to contain default data, then you can use the `DEFAULT` clause of the `INSERT` command to specify a particular column name and the value for that column. An error occurs if the column does not have a default value.

Note: If you have not set a default value for a column and the column is `NOT NULL`, then that column must have a value in the `INSERT` statement.

To insert rows in `EMP_DEPARTMENT`, enter the following command at the command prompt:

```
INSERT INTO EMP_DEPARTMENT (E_ID, D_NAME, DATE_OF_JOIN, DESIGNATION) VALUES
(108, 'MARKETING', '2000-05-15', 'MANAGER');
```

Note: The number of columns and the number of values specified inside the parentheses with or without quotes must match when you insert data into a table. MySQL will return an error if these numbers do not match.

You can also enter the following command at the command prompt:

```
INSERT INTO EMP_DEPARTMENT VALUES (108, 'MARKETING', '2000-05-15 ',
'MANAGER');
```

This command will work only if values for all the columns are specified following the `VALUES` keyword. Figure 8.4 displays the output of the command.

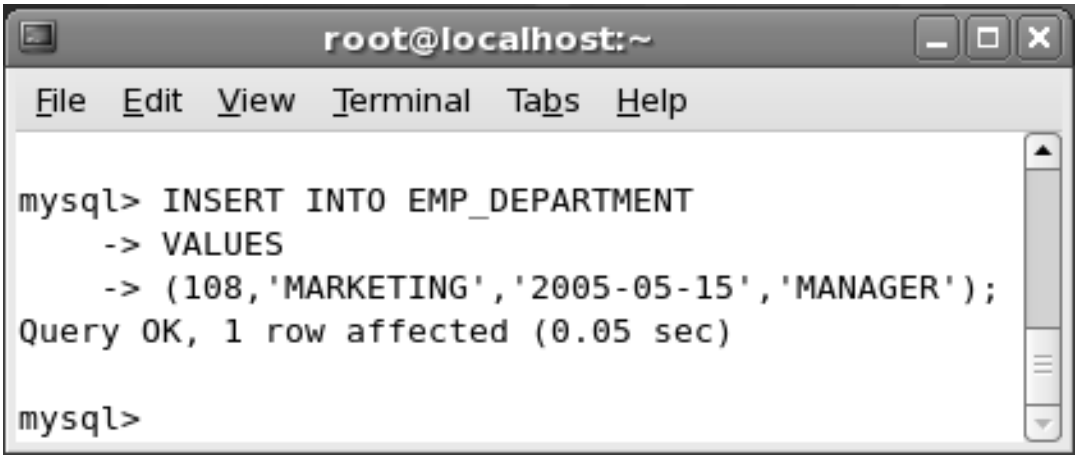


Figure 8.4: Inserting Data in DEPARTMENT Table

Figure 8.4 displays a message that specifies the authenticity or correctness of the query and the number of rows affected by the `INSERT` query.

You had earlier defined the primary key of `EMP_DEPARTEMENT` table as `NOT NULL` and `AUTO_INCREMENT`. Now you can try to insert a row having the `E_ID` that already exists, by entering the following command at the command prompt:

```
INSERT INTO EMP_DETAILS VALUES (101,'MARK','BLAIR','CALIFORNIA', 56574);
```

Figure 8.5 displays the output of the command.

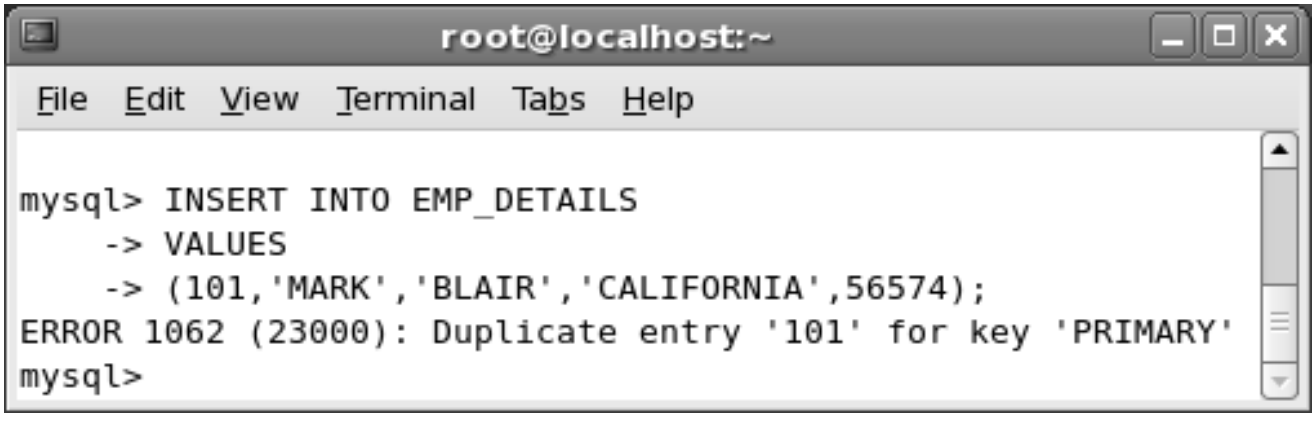


Figure 8.5: Inserting Data in EMP_DETAILS Table

Figure 8.5 displays an error message stating that MySQL does not allow a duplicate entry for the primary key.

A different form of the `INSERT` command enables you to set the column to a specific value. You can specify the column name and the values together. The syntax is as follows:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE] [INTO] tbl_name
SET col_name = {expression | DEFAULT},... [ON DUPLICATE KEY UPDATE col_
name=expression,...];
```

where,

`INSERT` – adds data to the table

`tbl_name` – specifies the name of the table

`SET` – specifies the column names explicitly to which the values will be added

To insert rows in `EMP_DEPARTMENT`, enter the following command at the command prompt:

```
INSERT INTO EMP_DEPARTMENT SET D_NAME = 'SALES', DATE_OF_JOIN = '2001-04-
16', DESIGNATION= 'TRAINEE';
```

Figure 8.6 displays the output of the command.

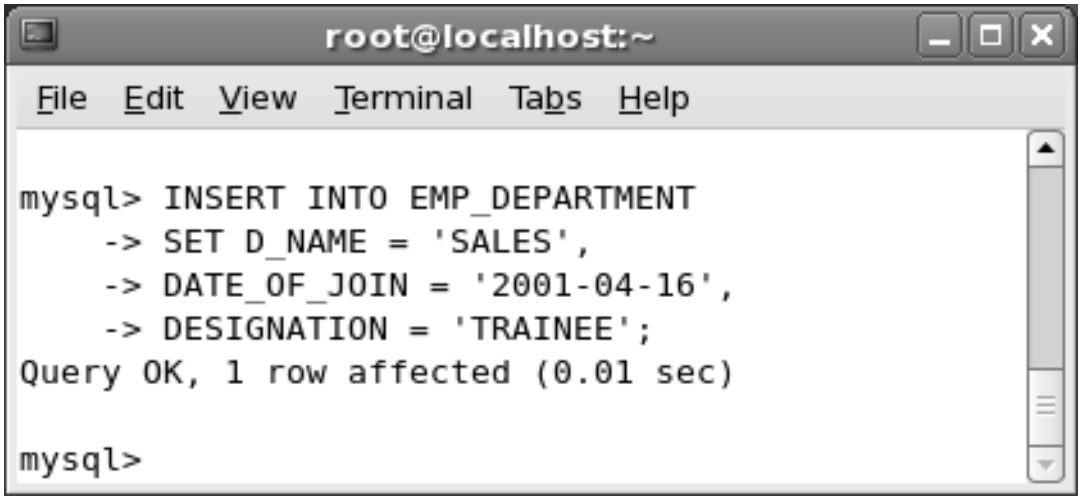


Figure 8.6: Inserting Data in EMP_DEPARTMENT Table

MySQL also provides options to copy data across tables. The `INSERT` command enables you to add data to a table returned from a `SELECT` query.

```
INSERT [LOW_PRIORITY | HIGH_PRIORITY] [IGNORE] [INTO] tbl_name [(col_
name,...)] SELECT ...;
```

where,

`INSERT` – adds data into the table

`tbl_name` – specifies the name of the table to add data

`col_name` – specifies the name of the column

`SELECT` – defines the data to be added in the table specified in the `tbl_name` clause. The data to be added will be retrieved using the conditions specified in the `SELECT` clause

To insert `E_ID` in the `EMP_DETAILS` table where the `E_ID` is retrieved from the `EMP_DEPARTMENT` table using the `SELECT` command, enter the following command at the command prompt:

```
INSERT INTO EMP_DETAILS (E_ID)
SELECT E_ID FROM EMP_DEPARTMENT
WHERE E_ID > 105;
```

Figure 8.7 displays the output of the command.

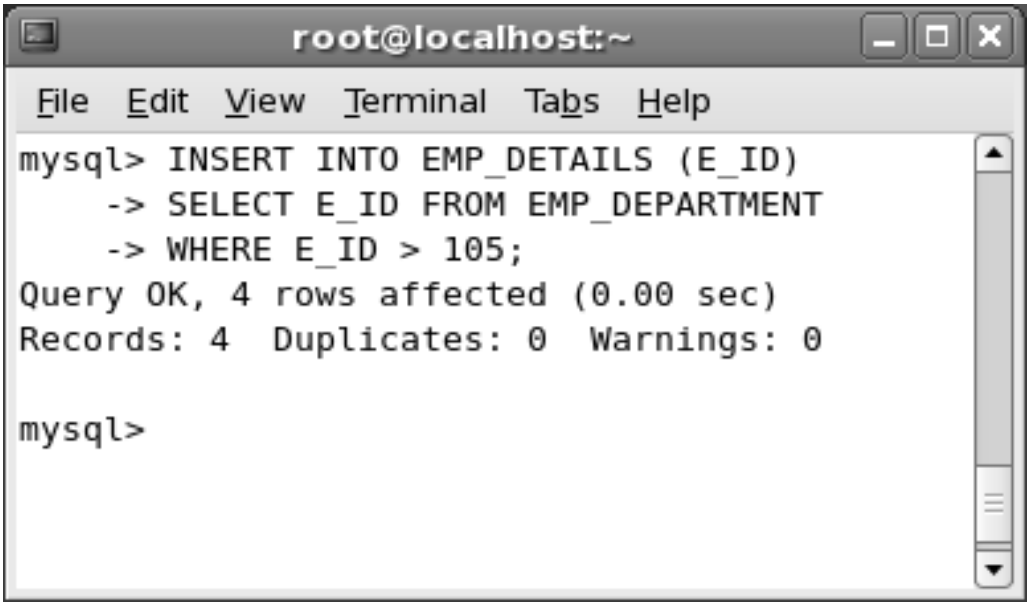


Figure 8.7: Inserting Data into a Table Using INSERT INTO…SELECT Command

Figure 8.7 displays the number of rows that have been copied into the `EMP_DETAILS` table from the `EMP_DEPARTMENT` table. In the output, Records specify the number of rows processed by the query, Duplicates indicates the number of rows that could not be inserted and Warnings indicates the number of attempts that was undertaken to insert column values.

8.3.2 Updating Data in a Table

You may require changing or modifying data in a table. MySQL provides the `UPDATE` command to change or modify data within a table.

The syntax to update a table is:

```
UPDATE [LOW_PRIORITY] [IGNORE] tbl_name SET column=value,... [WHERE clause][LIMIT n];
```

where,

`UPDATE` – modifies the column data

`tbl_name` – specifies the name of the table to modify

`SET` – defines the value to change

`column` – specifies the name of the column

`value` – defines the new value to be entered for the column

The options for the `UPDATE` command are listed in table 8.3.

Option	Description
`LOW_PRIORITY`	Assigns a low priority to the update against all other commands
`IGNORE`	Updates a row even if there is an error
`WHERE`	Specifies the rows to be updated based on the specific condition
`LIMIT`	Specifies a limit on the number of rows that can be updated

Table 8.3: UPDATE Command Options

To update a row in `EMP_DETAILS` table, enter the following command at the command prompt:

```
UPDATE EMP_DETAILS SET E_LNAME = 'KAY' WHERE E_ID = 105;
```

Figure 8.8 displays the output of the command.

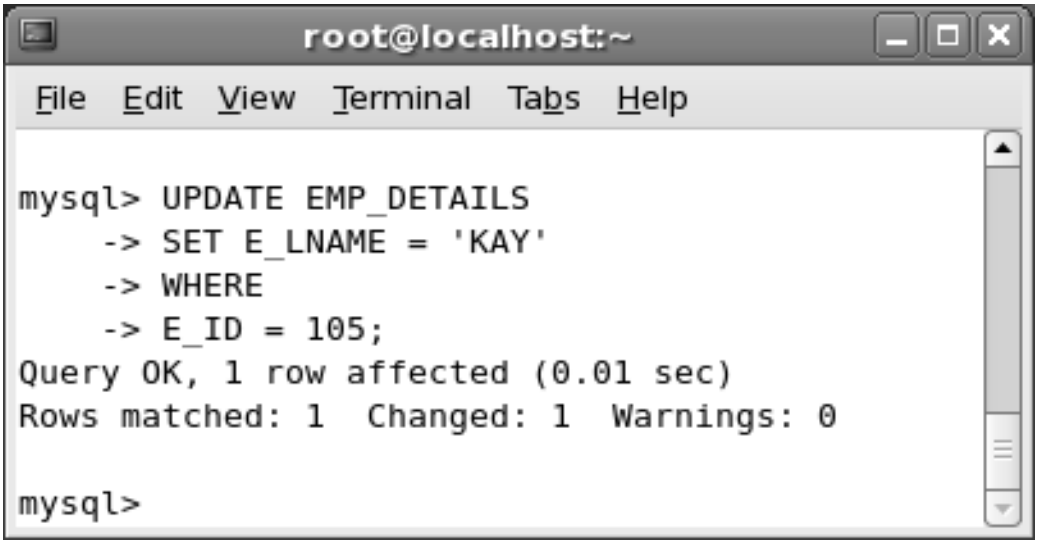


Figure 8.8: Updating a Table

Figure 8.8 displays the result of the `UPDATE` command. The `UPDATE` command returns the number of row that was changed. When you include the `ORDER BY` clause in an `UPDATE` command, MySQL will update the rows in the order specified in the `ORDER BY` clause. You can include the `ORDER BY` clause in an `UPDATE` command to modify records in a table sorted on specific conditions.

You can now use the `SELECT` command to view the updated data in the table.

Concepts

To view the updated results, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS;
```

Figure 8.9 displays the output of the command.

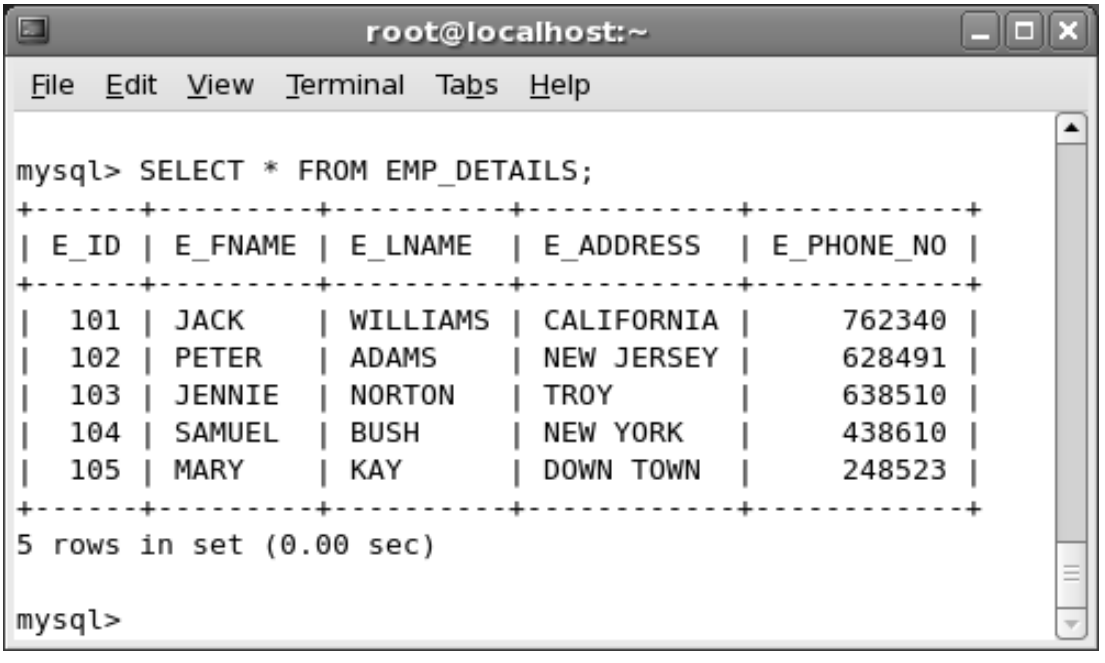


Figure 8.9: Viewing Data after Updating

In figure 8.9, you can observe the change in data in the row as result of the `UPDATE` command.

8.3.3 Replacing Data in a Table

MySQL provides the `REPLACE` command to substitute data in the table. The `REPLACE` command is similar to the `INSERT` command, except that it replaces the old value of a column with a new value. The `REPLACE` command will delete the old row only if the new row satisfies the conditions of the column definitions. If the value in the `REPLACE` command does not match the existing record then MySQL executes it as an `INSERT` statement.

The syntax for replacing data in a table is:

```
REPLACE [DELAYED|LOW_PRIORITY] INTO tbl_name [(column,…)] VALUES
(value,...);
```

where,

`REPLACE` – substitutes the column data

`tbl_name` – specifies the name of the table to modify

`value` – defines the new value for the column

The options of the `REPLACE` command are listed in table 8.4.

Option	Description
`DELAYED`	Postpones the execution of the command until no client is accessing the table
`LOW_PRIORITY`	Specifies the rows to be inserted into a buffer
`column`	Specifies the column names for which the values are to be replaced
`value`	Defines the new values for the columns

Table 8.4: REPLACE Command Options

To replace a row in `EMP_DETAILS`, enter the following command at the command prompt:

```
REPLACE INTO EMP_DETAILS
VALUES (105,'GEORGE','BLAIR', 'CALIFORNIA',56474);
```

Figure 8.10 displays the output of the command.

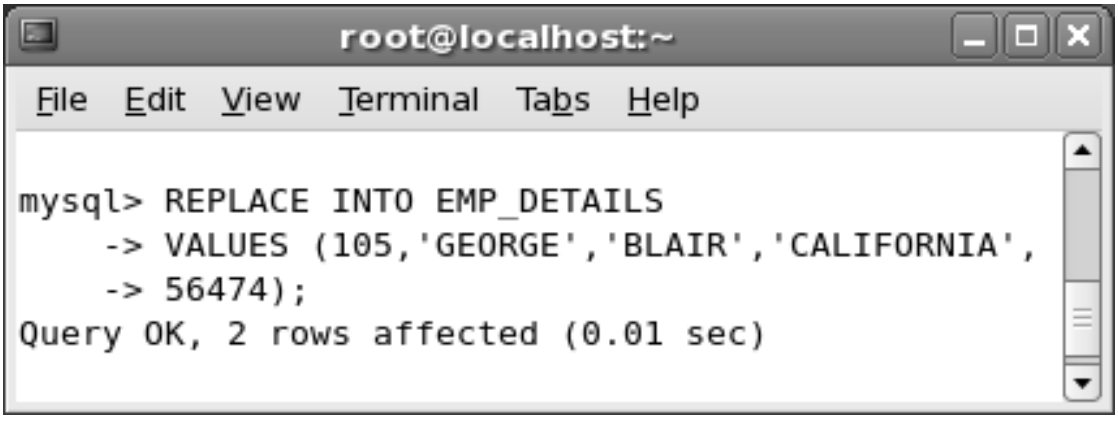


Figure 8.10: Replacing data into EMP_DETAILS Table

MySQL displays the number of rows affected by the `REPLACE` query. This includes the number of old rows that have been deleted and the number of new rows that have been added to the table. If MySQL returns the number of rows affected as one for a single row modification, it means that a row has been added and existing rows have not been deleted. When the number of rows affected is more than one, it means that an existing record has been deleted and a new record has been added to the table.

Concepts

To view the data in the EMP_DETAILS table, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS;
```

Figure 8.11 displays the output of the command.

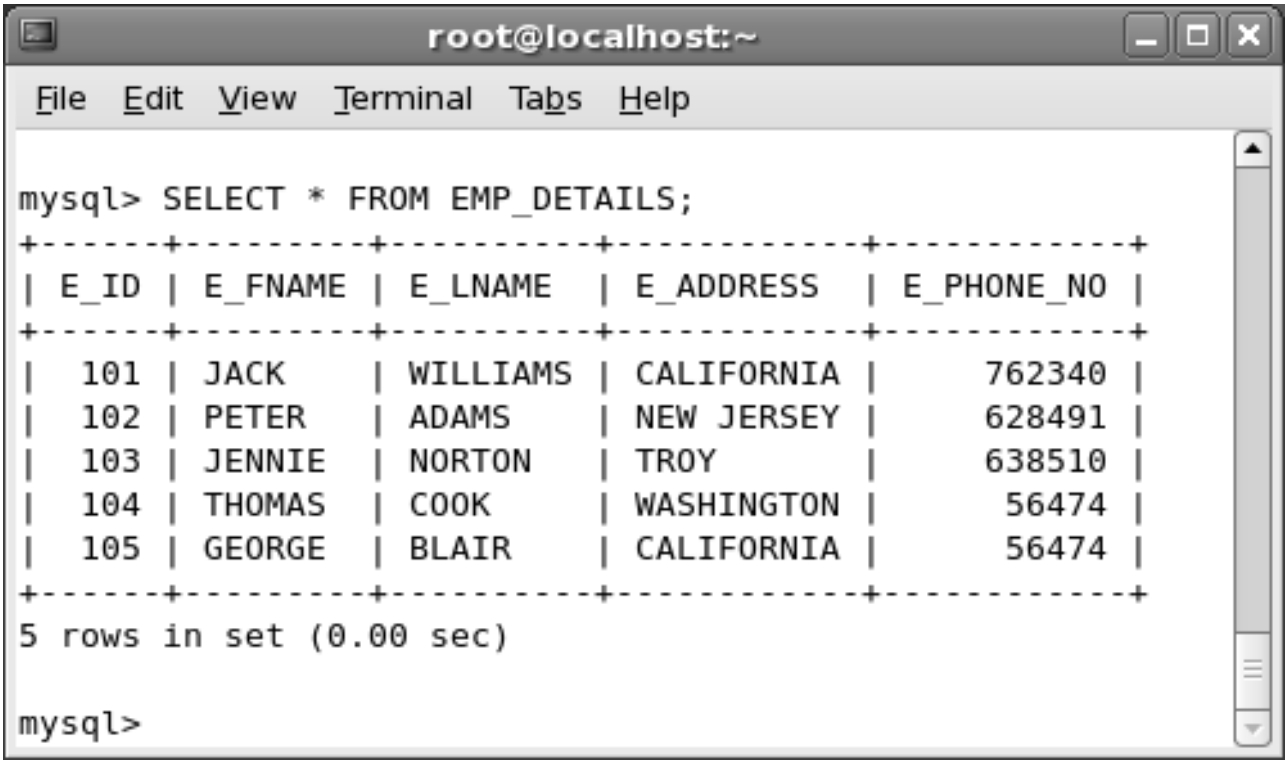


Figure 8.11: Viewing Data After REPLACE command

Figure 8.11 displays the change in row data as a result of the REPLACE command.

You can also use the SELECT query within a REPLACE command. The syntax to include a SELECT query within a REPLACE command is:

```
REPLACE [DELAYED|LOW_PRIORITY] INTO tbl_name [(column,...)] SELECT select_
clause;
```

where,

REPLACE – substitutes the column data

tbl_name – specifies the name of the table to modify

`SELECT` – executes the select query specified in the clause

The `REPLACE` command also enables you to specify the column names and the values together. MySQL assigns a `NULL` value to the columns if you fail to specify a value in the `REPLACE` command.

```
REPLACE [DELAYED|LOW_PRIORITY] INTO tbl_name SET column=value,
column=value,...;
```

where,

`REPLACE` – substitutes the column data

`tbl_name` – specifies the name of the table to modify

`SET` – defines the new value to be entered in the column

`column` – specifies the name of the column

`value` – defines the new value for the specified column

For example, to add a new record in the `EMP_DETAILS` table, where you only specify the `E_ID`, enter the following command at the command prompt:

```
REPLACE INTO EMP_DETAILS
SET E_ID = 106;
```

Figure 8.12 displays the output of the command.

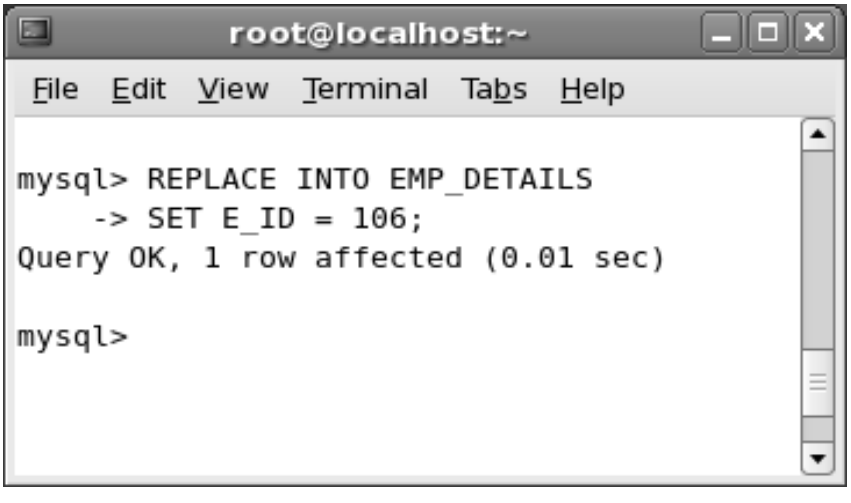


Figure 8.12: Adding Data into Table using REPLACE Command

Figure 8.12 displays the results of the REPLACE query. In this command, you have specified a value only for the E_ID field. MySQL will assign NULL values to the other columns in the table for this record. To view the NULL values for this record, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS;
```

Figure 8.13 displays the output of the command.

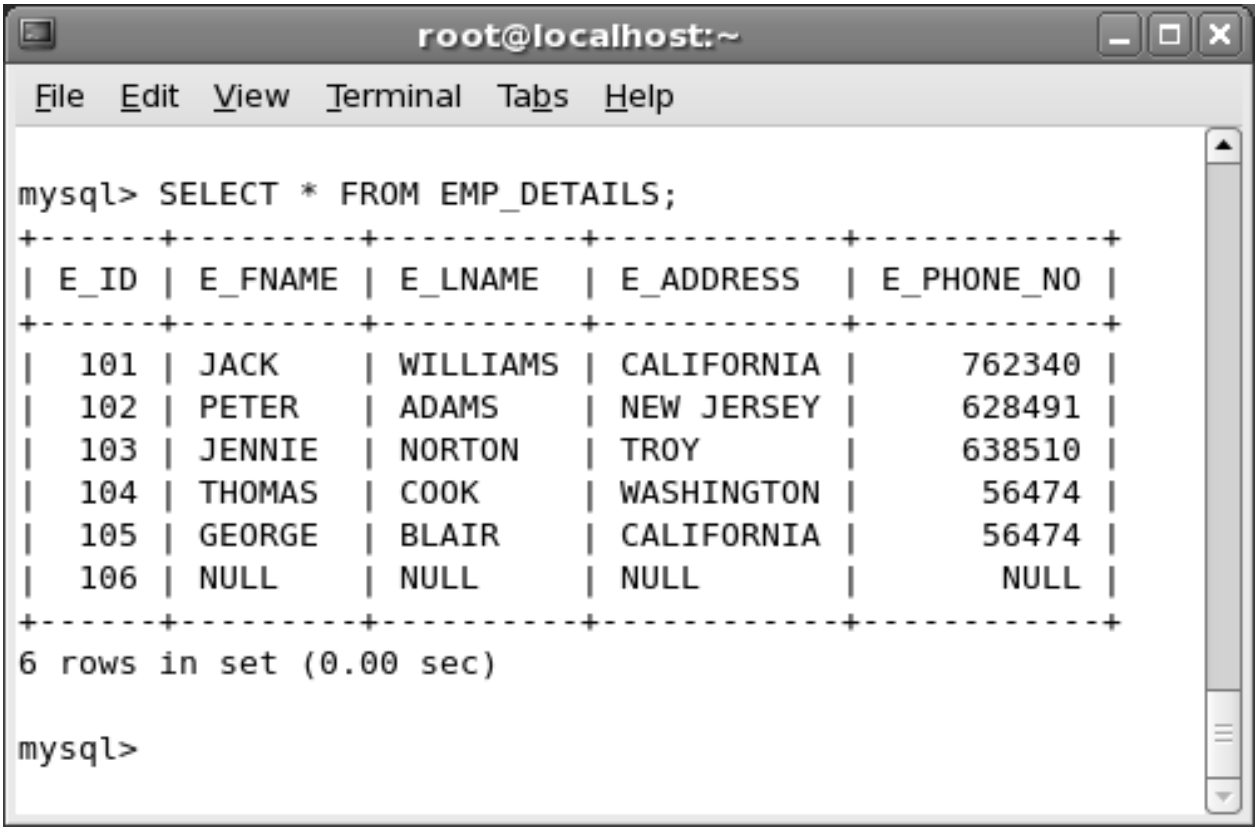


Figure 8.13: Viewing NULL Data after REPLACE

8.3.4 Deleting Data from a Table

MySQL stores data in a table in the form of rows. You may require deleting old and obsolete data from the table. MySQL provides the DELETE command to remove rows from a table. You can choose to remove selected rows of a table or the entire table.

The syntax to delete rows from a table is:

```
DELETE [LOW_PRIORITY|QUICK] [IGNORE} FROM tbl_name [WHERE clause] [ORDER BY
column,...] [LIMIT n];
```

where,

DELETE – removes data from the table

tbl_name – specifies the name of the table from which rows are to be deleted

The options for the DELETE command are provided in table 8.5.

Option	Description
LOW_PRIORITY	Postpones deletion until all other operations on the table have been completed by the client such as reading
QUICK	Suspends the merging of indexes during execution
IGNORE	Ignores error while deleting a row
WHERE	Deletes rows that satisfy the specified conditions
ORDER BY	Deletes data in the order specified
LIMIT n	Specifies the maximum number of rows to be deleted in a single attempt

Table 8.5: DELETE Command Options

If you use the DELETE command without the WHERE clause, the entire table will be deleted. If you specify the WHERE clause in the DELETE command, the system prompts a message displaying the number of rows that will be deleted. If no WHERE clause is used, all the rows are deleted. MySQL returns a '0' value, when the entire table is deleted because MySQL cannot identify the number of rows to be deleted.

Note: The DELETE command will only remove all the data from the table. It does not remove the table structure. The DELETE command without the WHERE clause will delete all the files associated with the table except for the file that contains the table structure.

To delete a row from the EMP_DETAILS table, enter the following command at the command prompt:

```
DELETE FROM EMP_DETAILS WHERE E_ID=106;
```

Figure 8.14 displays the output of the command.

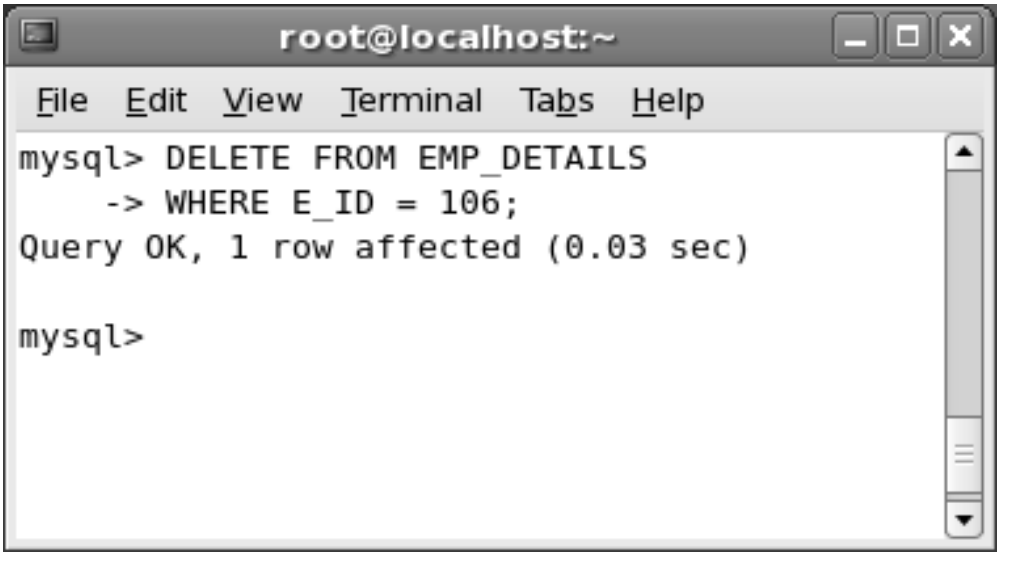


Figure 8.14: Deleting Data from a Table

MySQL displays a message containing the number of rows affected by the execution of the `DELETE` query.

8.4 Using the ORDER BY Clause

You use the `SELECT` command to retrieve data from a database. MySQL displays the `SELECT` command result in the order in which the data is inserted in the table.

You can also sort the output on more than one column in ascending or descending order. You will use the `ORDER BY` clause with `SELECT` command to sort the data. The use of `DISTINCT` keyword causes only one row of data to be displayed for every group of rows that is identical.

The syntax for `ORDER BY` is:

```
SELECT column_name FROM tbl_name ORDER BY column [ASC|DESC] [,column2[ASC|D
ESC],...];
```

where,

`SELECT` – retrieves data from the table

`column_name` – specifies the name for the column to retrieve data from

`tbl_name` – specifies the name of the table that contains the column and data

`ORDER BY` – sorts the column data in the specified order

`column` – specifies the name of the column

`ASC|DESC` – specifies the sort direction for data. MySQL supports two types of sorting. Ascending order sorts the data from lowest to highest whereas Descending order sorts the data from highest to lowest

To organize the output of `EMP_DEPARMENT` table on the `DATE_OF_JOIN` field in an ascending order, enter the following command at the command prompt:

```
SELECT * FROM EMP_DEPARTMENT ORDER BY DATE_OF_JOIN ASC;
```

Figure 8.15 displays the output of the command.

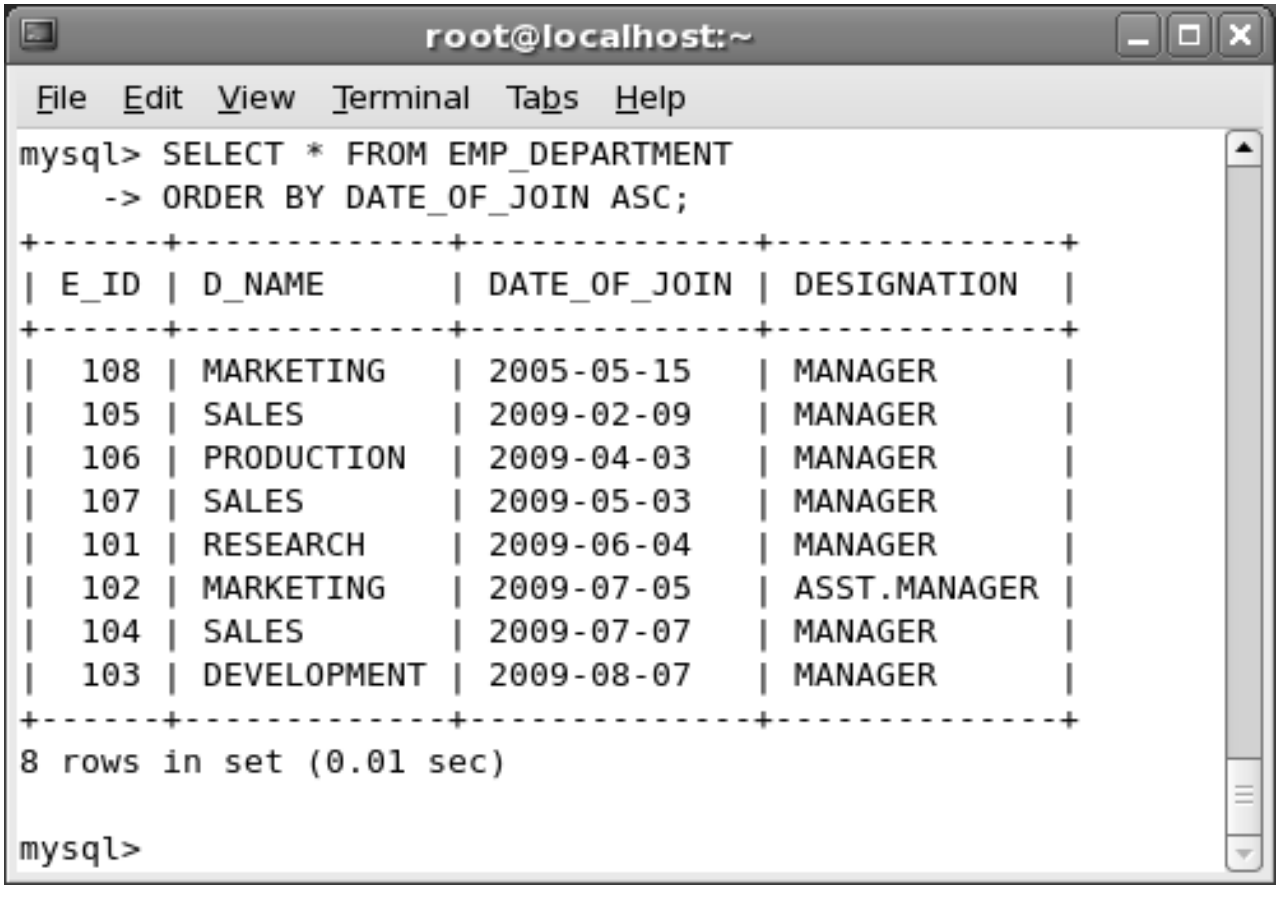


Figure 8.15: Organizing Data

You will now sort the data in the `EMP_DETAILS` table on the `E_FNAME` and `E_LNAME` columns. Also, you will sort data from the `E_FNAME` column in the descending order.

To sort the output of EMP_DETAILS table on the E_FNAME field in the descending order and then by E_LNAME field, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS ORDER BY E_FNAME DESC, E_LNAME;
```

Figure 8.16 displays the output of the command.

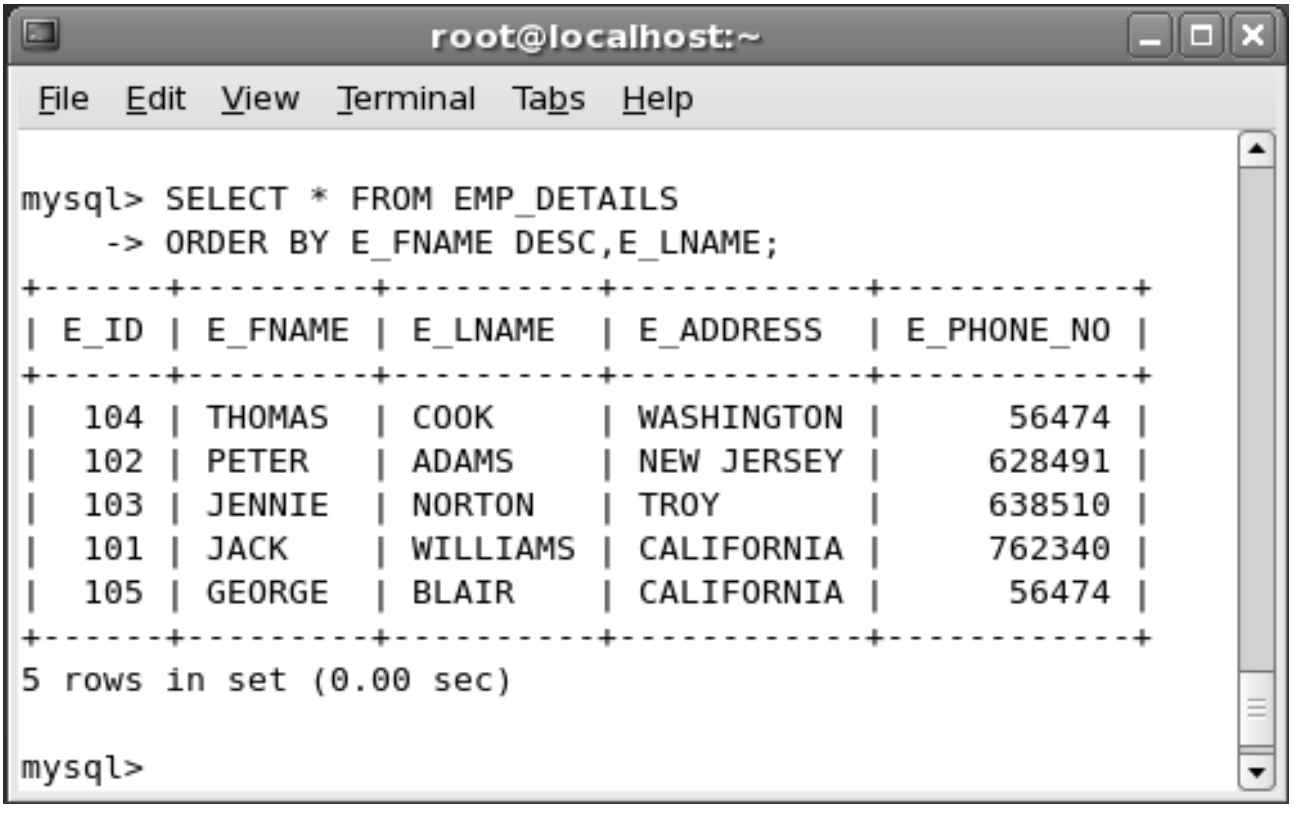


Figure 8.16: Ordering the data by E_FNAME and E_LNAME

8.5 Using the GROUP BY Clause

You may require to group data from columns. MySQL provides the GROUP BY command to group rows with similar values for a specific column into a single row. This will enable you to execute tasks on these grouped rows.

The syntax for grouping data is:

```
SELECT column_name FROM tbl_name GROUP BY column [,column2...];
```

where,

`SELECT` – retrieves data from the table

`column_name` – specifies the column name to retrieve data from

`tbl_name` – specifies the name of the table that contains the column and data

`GROUP BY` – categorizes the data

`column` – specifies the name of the column in the table

You can group the employees based on their designations from the `EMP_DEPARTMENT` table on the `DESIGNATION` column.

To group the output of `EMP_DEPARTMENT` table on the `DESIGNATION` field, enter the following command at the command prompt:

```
SELECT * FROM EMP_DEPARTMENT GROUP BY DESIGNATION;
```

Figure 8.17 displays the output of the command.

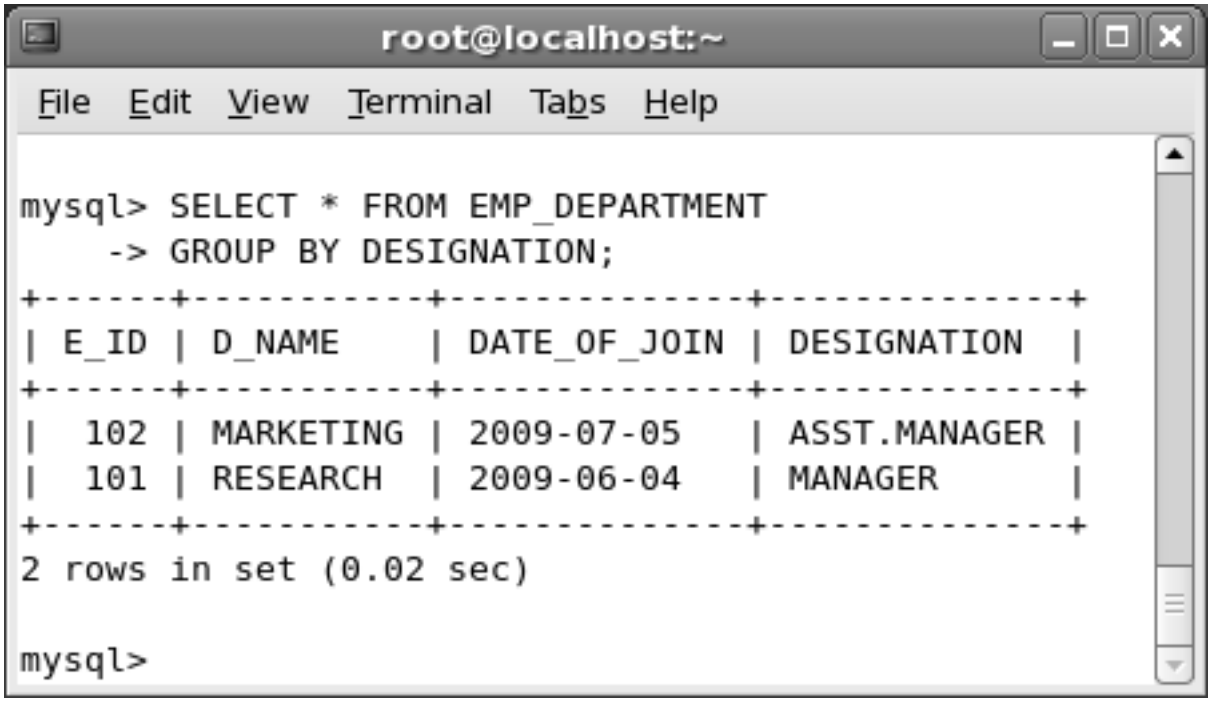


Figure 8.17: GROUP BY Clause

Concepts

Summary

- Keys help to identify a record in the table.
- A primary key uniquely identifies the records in a table.
- A foreign key in a table must have a corresponding primary key in a different table.
- A primary key and a foreign key can be defined while creating a table.
- The `ALTER TABLE` command enables to define keys in a table.
- The `INSERT` command allows you to add a row in a table.
- The `UPDATE` command allows you to modify the values present in the columns of a table.
- MySQL enables you to substitute values in a column using the `REPLACE` command.
- MySQL provides the `DELETE` command to remove unwanted or obsolete rows in a table.
- The `ORBER BY` clause can be used with the `SELECT` command to sort the data.
- Grouping categorizes rows with similar values for a specific column into a single row to execute combined data processing operations.

Check Your Progress

1. While creating a foreign key _____ option prevents the deletion of parent row if a dependent child row exists.

   a. `RESTRICT`

   b. `NO ACTION`

   c. `ON DELETE`

   d. `ON UPDATE`

2. The _____ option of `INSERT` command postpones the addition of records unless all the clients finish reading from the table.

   a. `LOW_PRIORITY`

   b. `IGNORE`

   c. `DELAYED`

   d. `DEFAULT`

3. If you use the _____ option with the `UPDATE` command then the conflicting rows will not be updated.

   a. `IGNORE`

   b. `WHERE`

   c. `LIMIT`

   d. `LOW_PRIORITY`

4. The _____ option of the `DELETE` command suspends the merging of indexes.

   a. `LOW_PRIORITY`

   b. `QUICK`

Check Your Progress

c. `LIMIT`

d. `ORDER BY`

5. Grouping combines rows with _____ in order to operate on them together.

a. values for a specific column into multiple rows

b. values for a specific row

c. values for a specific column into a single row

d. values for specific columns into a single column

Session 9

Implementing SQL Queries Using MySQL-II (Lab)

Objectives

At the end of this session, the student will be able to:

- *Define keys and indexes.*
- *Use the INSERT, ALTER, UPDATE, and DELETE statements.*
- *Use the GROUP BY command.*
- *Use the ORDER BY command.*

The steps given in this session are detailed, comprehensive, and carefully thought-through in order to meet the learning objectives and understand the tool completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Working with Keys and Indexes

A key is the unique identifier in a table. A column that uniquely identifies a record is defined as the primary key. You index a column on which you wish to search the data frequently.

1. **To activate the LIBRARY database, enter the following command at the command prompt:**

```
USE LIBRARY;
```

Figure 9.1 displays the output of the command.

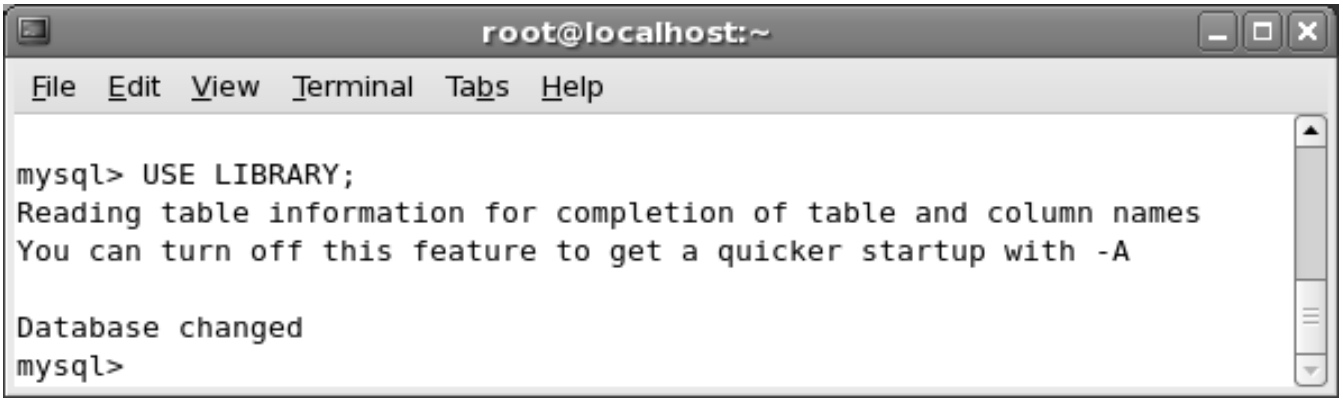


Figure 9.1: Activating the Library Database

2. **The librarian has identified two new members who have the same User IDs. This is causing problems while issuing books. To resolve this problem, the librarian will define a primary key on the USER_ID field in the USER_DETAILS table. To modify the USER_ID field as the primary key in the USER_DETAILS table, enter the following command at the command prompt.**

```
ALTER TABLE USER_DETAILS MODIFY USER_ID INT(3)

NOT NULL PRIMARY KEY AUTO_INCREMENT;
```

Figure 9.2 displays the output of the command.

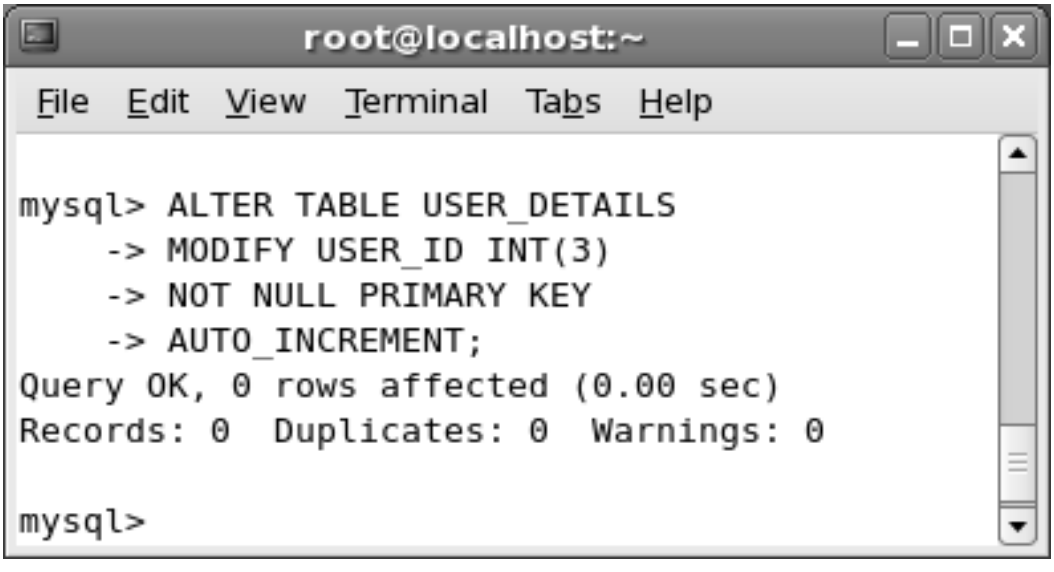


Figure 9.2: Defining the Primary key of USER_DETAILS Table

Now, create an index for the CATEGORY table to enhance sorting and searching operations.

3. **The time taken to search and retrieve data from the CATEGORY table is taking longer than required. An index can be defined on the USER_ID column of the CATEGORY table to resolve the problem. To create an index key named INDEX1 on the USER_ID field of the CATEGORY table, enter the following command at the command prompt:**

```
CREATE INDEX INDEX1 ON CATEGORY(USER_ID);
```

Figure 9.3 displays the output of the command.

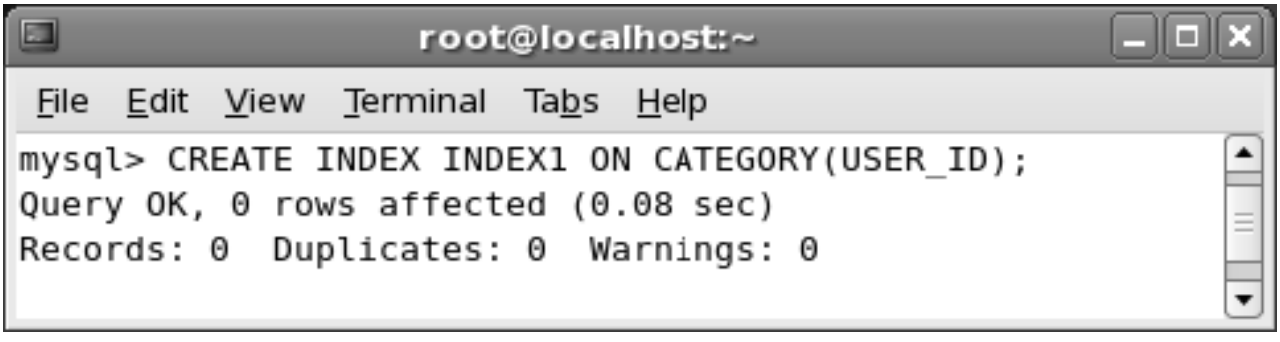


Figure 9.3: Creating Index on the USER_ID Field

4. **The librarian wants to add information about a new member in the USER_DETAILS table. To insert a record into the USER_DETAILS table, enter the following command at the command prompt:**

```
INSERT INTO USER_DETAILS VALUES(100, 'TOBY', 'MEXICO',45637);
```

Figure 9.4 displays the output of the command.

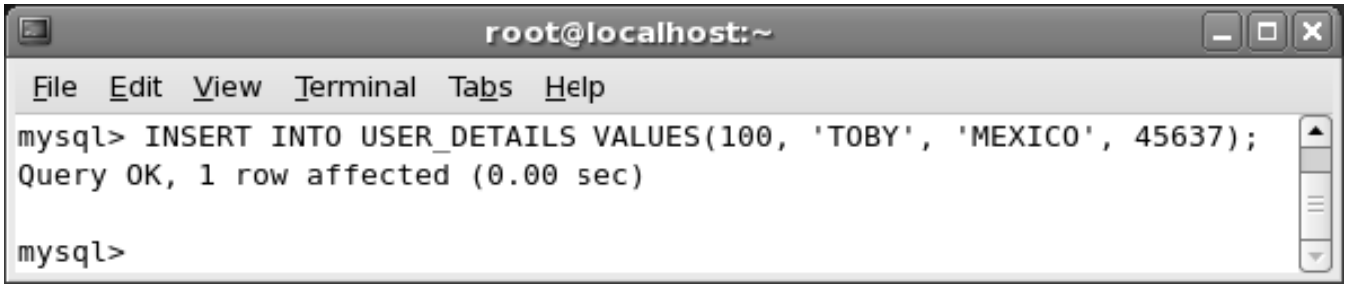


Figure 9.4: Inserting Data

A primary key can be defined as AUTO_INCREMENT. This feature automatically generates a new unique value for the record when NULL value is specified in the input. Now, insert a NULL value for the primary key and view the result.

5. **The librarian is adding a user to the database. The librarian requires the system to generate the USER_ID for the user. To insert a record into the USER_DETAILS table, enter the following command at the command prompt:**

```
INSERT INTO USER_DETAILS VALUES(NULL, 'BINCY', 'MEXICO', 45663);
```

Figure 9.5 displays the output of the command.

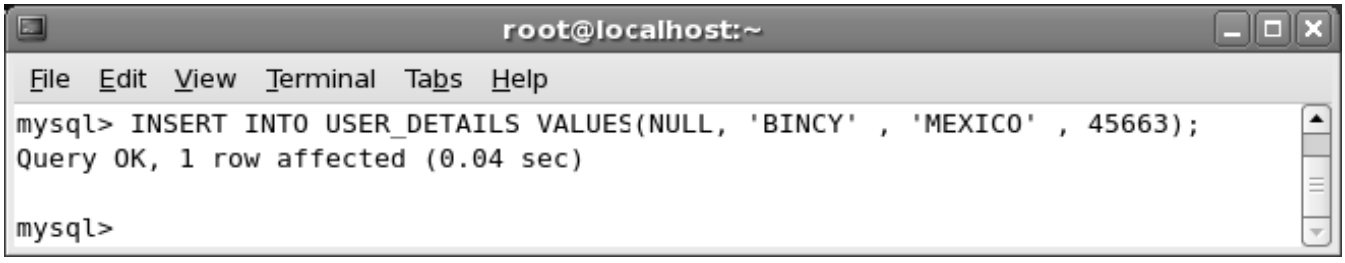


Figure 9.5: Inserting Data into USER_DETAILS Table

6. **After adding the new user, the librarian wants to check if the system has automatically generated the correct USER_ID. To view the changes after inserting a new record into the USER_DETAILS table, enter the following command at the command prompt:**

```
SELECT * FROM USER_DETAILS;
```

Figure 9.6 displays the output of the command.

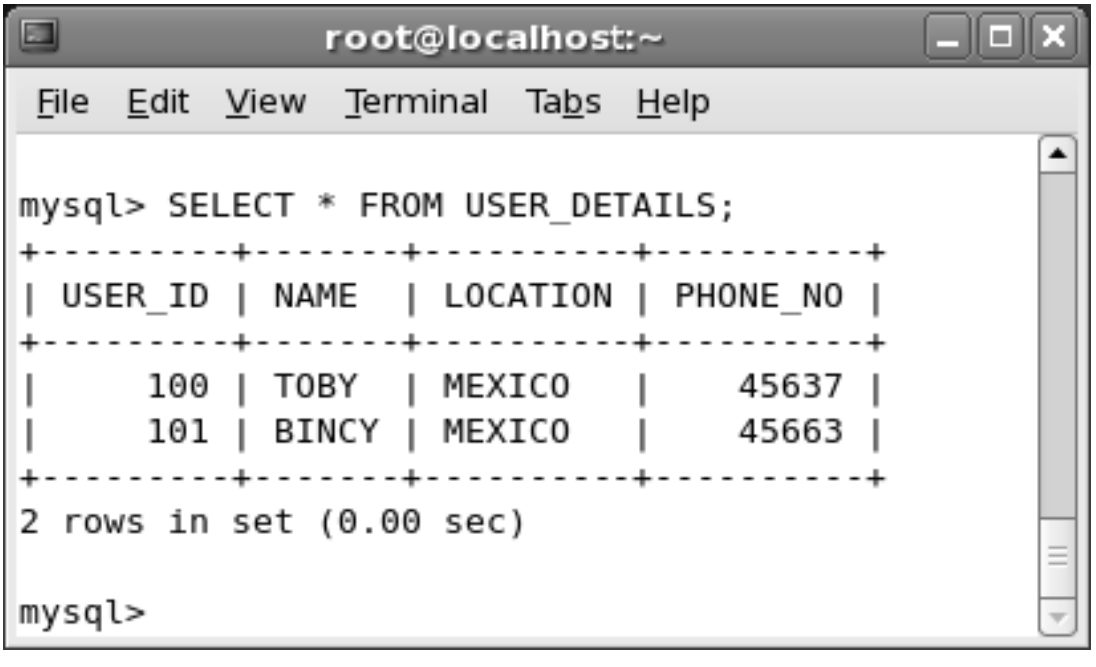


Figure 9.6: Viewing Data After Adding Rows

7. **The librarian has entered incorrect edition information for a book and wants to update this information in the BOOK_TABLE table. To update a record from the BOOK_TABLE table, enter the following command at the command prompt:**

```
UPDATE BOOK_TABLE SET EDITION = 'FOURTH' WHERE BOOK_ID=103;
```

Figure 9.7 displays the output of the command.

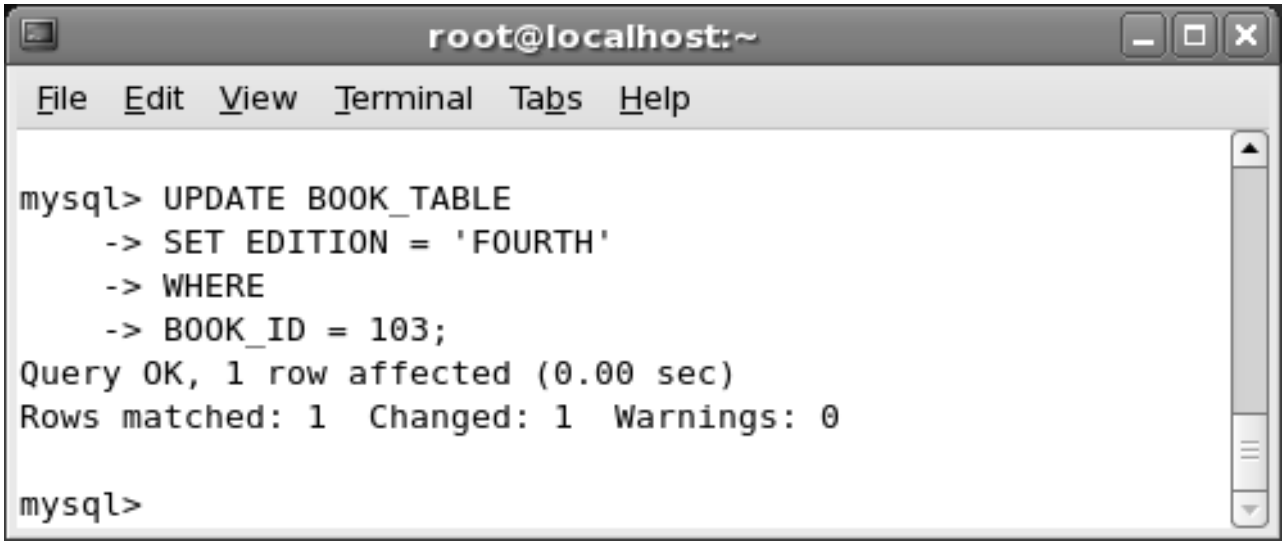


Figure 9.7: Updating the BOOK_TABLE Table

8. **The librarian wants to ensure that the information about the edition has been updated. To view the changes after updating the record, enter the following command at the command prompt:**

```
SELECT * FROM BOOK_TABLE;
```

Figure 9.8 displays the output of the command.

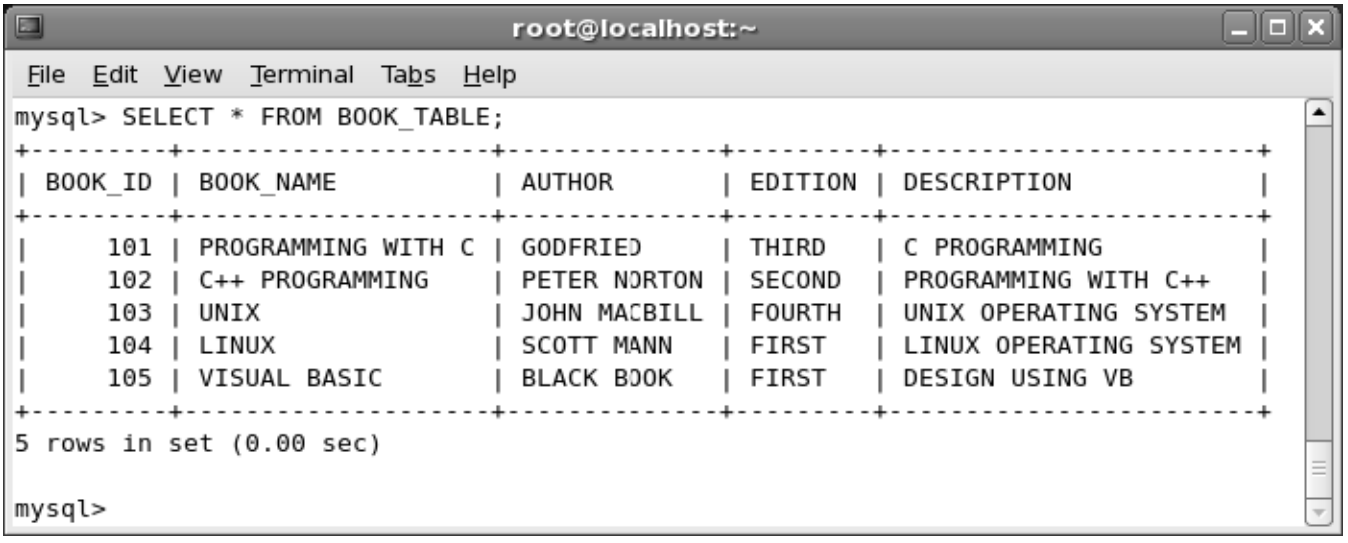


Figure 9.8: Viewing the BOOK_TABLE Table after Updating

9. **The librarian wants to provide additional facilities for a member. The member requires three books at a time. The librarian wants to replace this information in the facility column of the CATEGORY table. To replace the information in the CATEGORY table, enter the following command at the command prompt:**

```
REPLACE INTO CATEGORY(USER_ID,TYPE, FACILITY,ISSUE_STATUS)

VALUES(103, 'REGULAR', 'ISSUE 3 BOOKS', 'YES');
```

Figure 9.9 displays the output of the command.

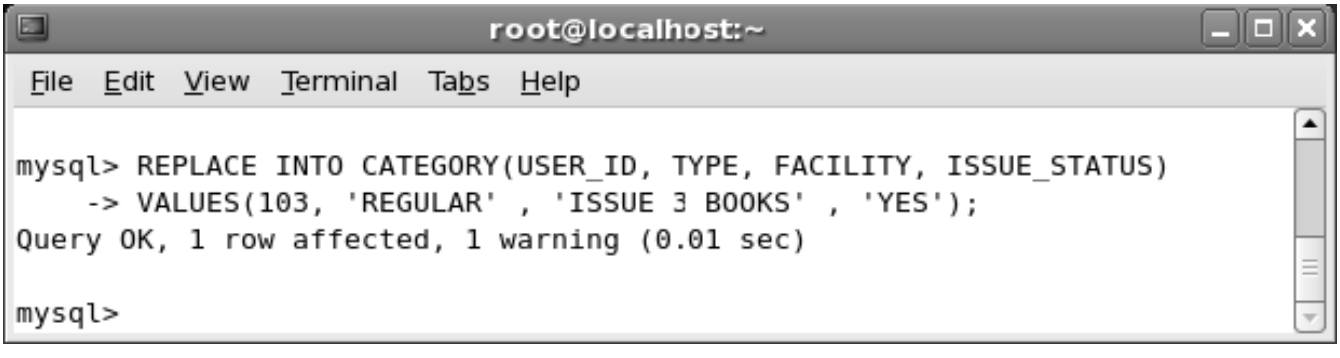


Figure 9.9: Replacing Data in CATEGORY Table

10. **After replacing the information, the librarian wants to view the updated information. To view the changes after replacing a record in the CATEGORY table, enter the following command at the command prompt:**

```
SELECT * FROM CATEGORY;
```

Figure 9.10 displays the output of the command.

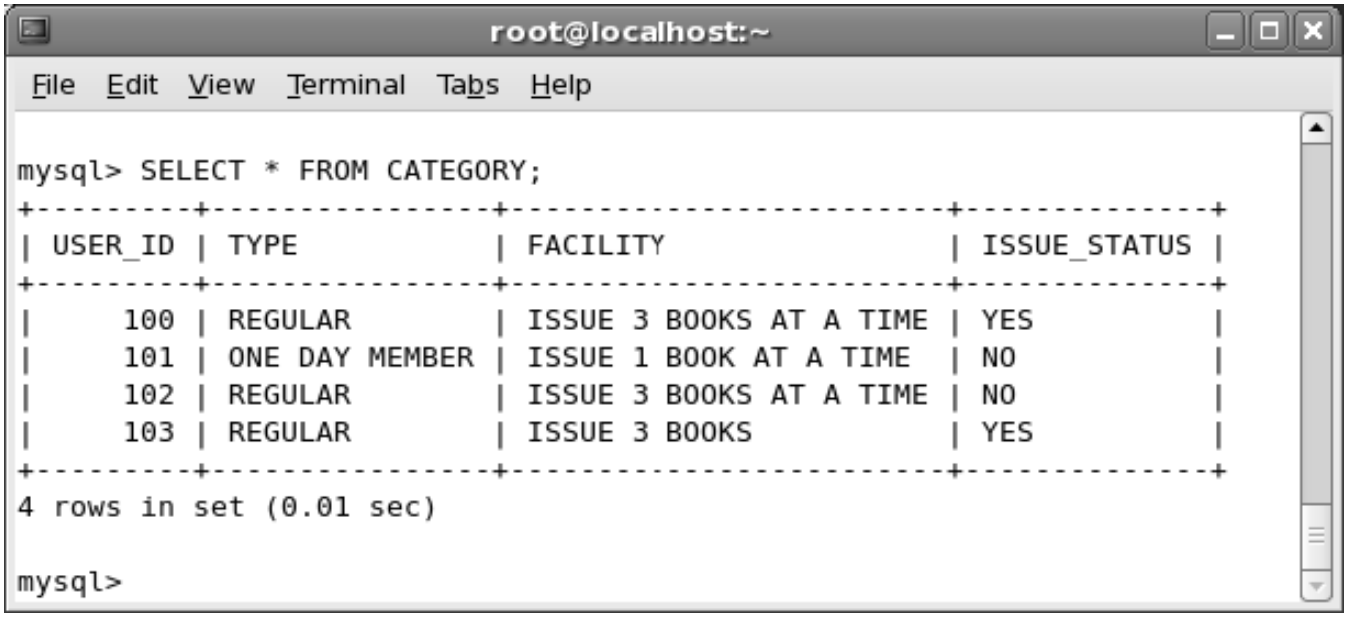


Figure 9.10: Viewing Data After Replace

11. **A book has been lost and the librarian is unable to trace it. Therefore, the librarian wants to delete that book from the table. To delete a record from the BOOK_TABLE table, enter the following command at the command prompt:**

```
DELETE FROM BOOK_TABLE WHERE BOOK_ID=105;
```

Figure 9.11 displays the output of the command.

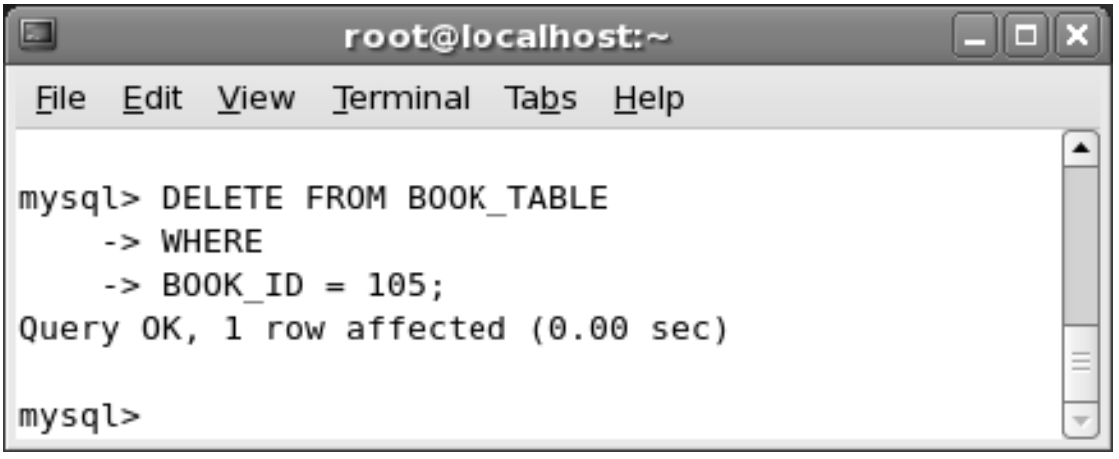


Figure 9.11: Deleting Data from the BOOK_TABLE Table

12. **The librarian wants to ensure that the book has been removed from the database. To view the changes after deleting a record from the BOOK_TABLE table, enter the following command at the command prompt:**

```
SELECT * FROM BOOK_TABLE;
```

Figure 9.12 displays the output of the command.

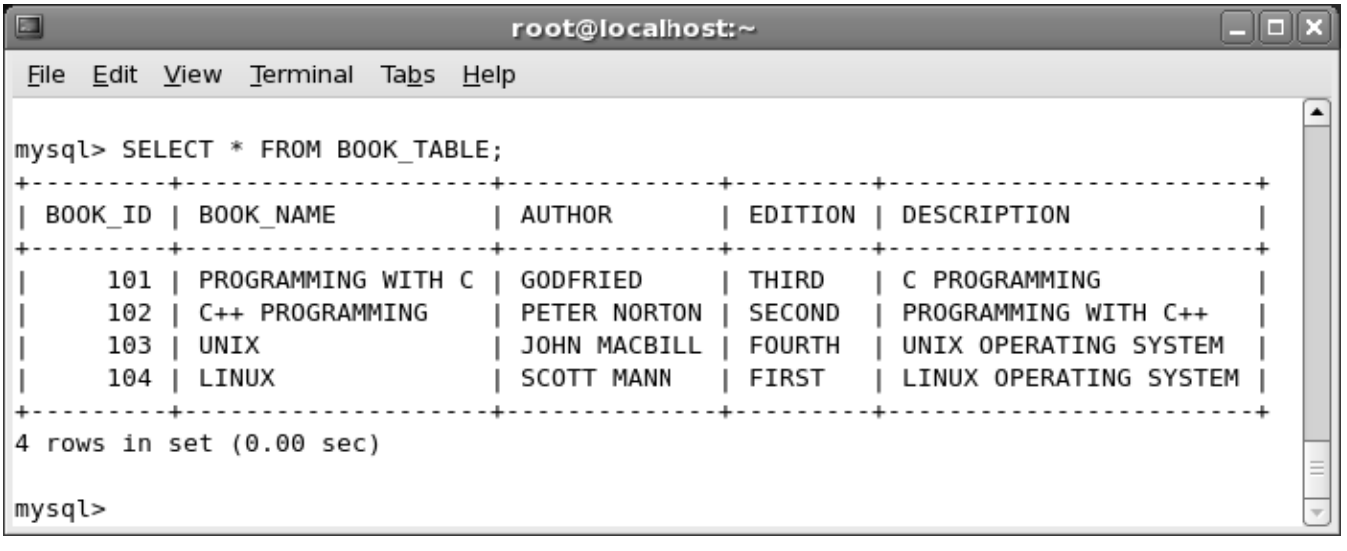


Figure 9.12: Viewing Data After Deletion

Using the ORDER BY Command

You can use the `ORDER BY` option with the `SELECT` command to arrange the output in an ascending or descending order.

1. **The librarian wants to view all the users in the database. This information requires the data to be sorted on the USER_ID field. To arrange the records on the USER_ID field in the CATEGORY table, enter the following command at the command prompt:**

```
SELECT * FROM CATEGORY ORDER BY USER_ID;
```

Figure 9.13 displays the output of the command.

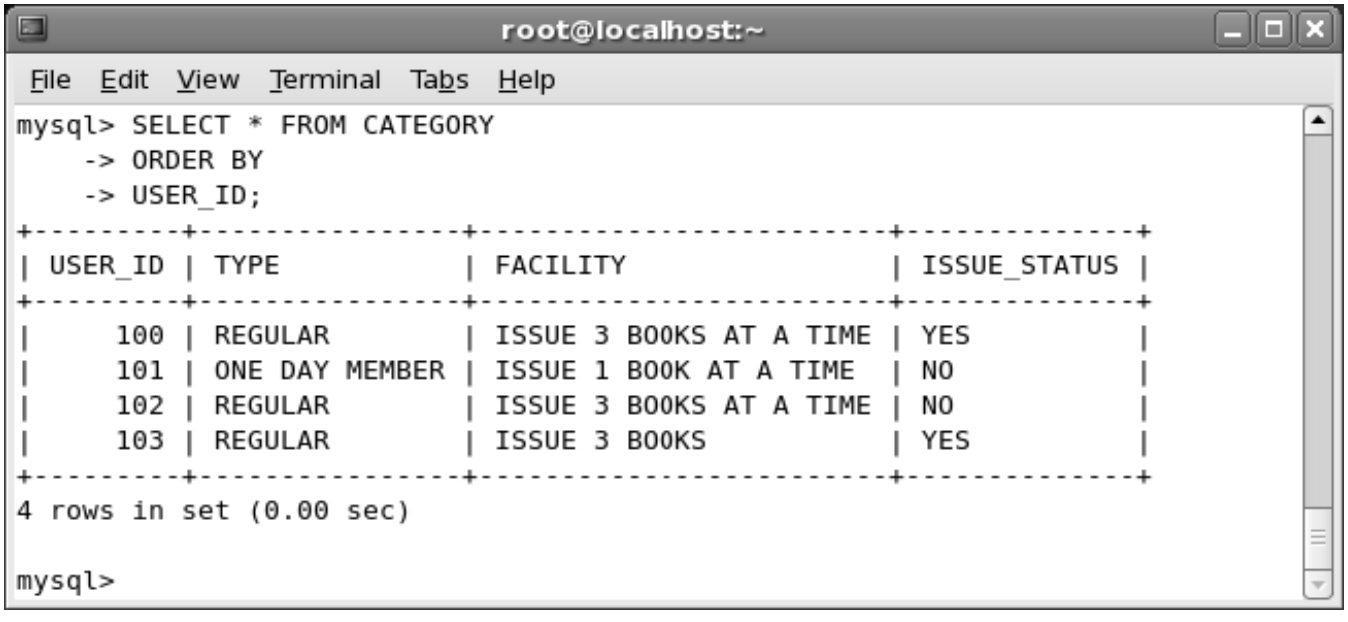


Figure 9.13: Sorting Data in the CATEGORY Table

Organizing the Output Using the GROUP BY Command

Grouping enables you to group rows with matching values for a specific column into a single row, allowing you to operate on them together.

1. **The librarian wants to view records from the CATEGORY table by grouping them on the column TYPE. To view records by grouping them on the column TYPE from the CATEGORY table, enter the following command at the command prompt:**

```
SELECT * FROM CATEGORY GROUP BY TYPE;
```

Figure 9.14 displays the output of the command.

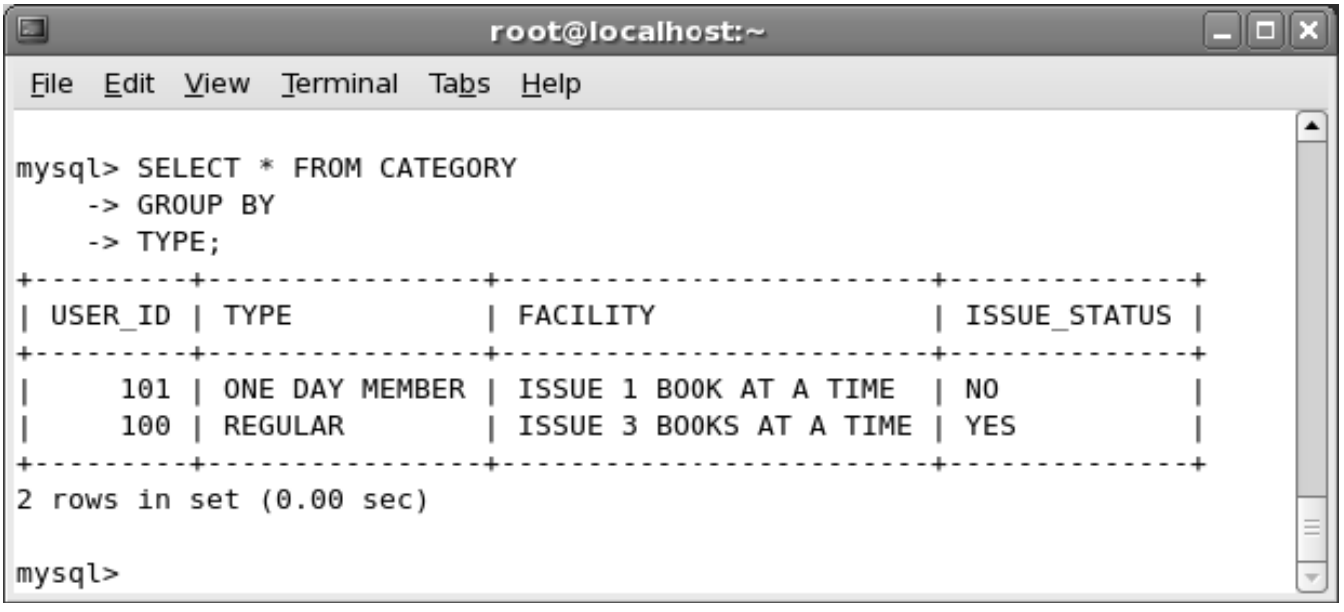


Figure 9.14: Grouping Data on a Field

Do It Yourself

1. Create a primary key on the `Product_Code` column for the `Sales_Details` table.

2. Create an index named `IND1` on the `Date_Of_Manufacture` column for the `Stock_Details` table.

3. Insert the following records in the `Sales_Details` table:

Product_Code	Selling Price	Qty	Units Sold	Month	Location
101	200	1500	1450	Jan	California
102	100	5000	3000	Jan	New York
103	20	2900	2000	Jan	Los Angeles
104	56	120	9	Jan	Sydney
101	200	1000	500	Jan	New York

4. Insert the following records in the `Stock_Details` table:

Product_Code	Product_Name	Description	Cost Price	Date_Of_Manufacture
101	Bags	Leather Bags	150	2003-10-05
102	Books	Notepads	10	2003-12-05
103	Pens	Fine Tip Pens	20	2003-01-09
104	Staplers	Staplers	40	2003-10-05
105	Clips	Paper Clips	100	2003-10-05

5. Replace the following records in the `Sales_Details` table:

Product_Code	Selling Price	Qty	Units Sold	Month	Location
103	20	2900	2000	Jan	Los Angeles

6. Update the following record in the Product table:

Product_Code	Selling Price	Qty	Units Sold	Month	Location
103	20	2900	2500	Jan	Los Angeles

7. Delete the following record from the table:

Product_Code	Selling Price	Qty	Units Sold	Month	Location
104	56	120	9	Jan	Sydney

Do It Yourself

8. Group the `Stock_Details` table on the `Date_Of_Manufacture` column.

9. Sort the data in the `Stock_Details` table on the `Date_Of_Manufacture` column.

Session
10

Using Joins

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain the different types of table joins in MySQL*
- *Explain the use of Equi-Join*
- *Explain the use of Inner Join*
- *Explain the use of Outer-Join*
- *Explain the use of Self-Join*
- *Explain the use of multiple SELECT queries in a single SELECT query*
- *Explain the use of UNION with the query*

10.1 Introduction

Data is often spread across multiple tables. Sometimes, it becomes necessary to retrieve data from one table based on the information present in another table. For this, you need to join the tables. MySQL provides commands such as `JOIN`, `UNION`, and subqueries to join multiple tables.

In this session, you will learn about the various types of joins and how to use them. In addition, you will also learn to execute multiple queries and the `UNION` clause.

10.2 Joining Tables in MySQL

Combining two or more records of different tables from the same database, into one comprehensive structure, is known as joining of tables. The purpose of joining tables is to retrieve data from multiple tables into a single result without repeating the data in the tables. Joining tables enables ease of manipulation, increases the speed of access, and reduces data redundancy.

The concept of joining tables is to display a single table that will be derived from the base of all related tables linked to each other with a common attribute. You can join the tables either using the `WHERE` clause with the `SELECT` command or by using the `JOIN` keyword.

Concepts

10.3 Using Equi-Join

In Equi-Join, MySQL combines records from two or more tables based on a common column. An Equi-Join displays only those records where a match is found in both the tables. The syntax for Equi-Join is:

```
SELECT column_name,...[*] FROM table1 [as alias],table2 [as alias] WHERE
(join_condition);
```

where,

`SELECT` – retrieves data from the specified column and the table

`column_name` – specifies the name of the column to retrieve from the table

`table1` – specifies the name of the table that contains the columns

`as alias` – specifies the reference name for the table

`WHERE` – specifies the conditions that need to be satisfied before retrieving data

`join_condition` – compares data from the tables as specified in this clause

You can specify the `join_condition` using any one of the following clauses:

- `ON` clause - specifies the conditional expression that can be used in a `WHERE` clause. If there is no matching record present in the specified table according to the query, then the row with all columns set to `NULL` will be used.
- `USING` clause - specifies the common attribute of the joining tables.

For example, to display the first name, department name, and designation of all employees from the `EMP_DETAILS` and the `EMP_DEPARTMENT` tables of `EMPLOYEE` database enter the following command at the command prompt:

```
SELECT EMP_DETAILS.E_FNAME, EMP_DEPARTMENT.D_NAME, EMP_DEPARTMENT.DESIGNATION
FROM EMP_DETAILS, EMP_DEPARTMENT WHERE (EMP_DETAILS.E_ID = EMP_DEPARTMENT.
E_ID);
```

Figure 10.1 displays the output of the command.

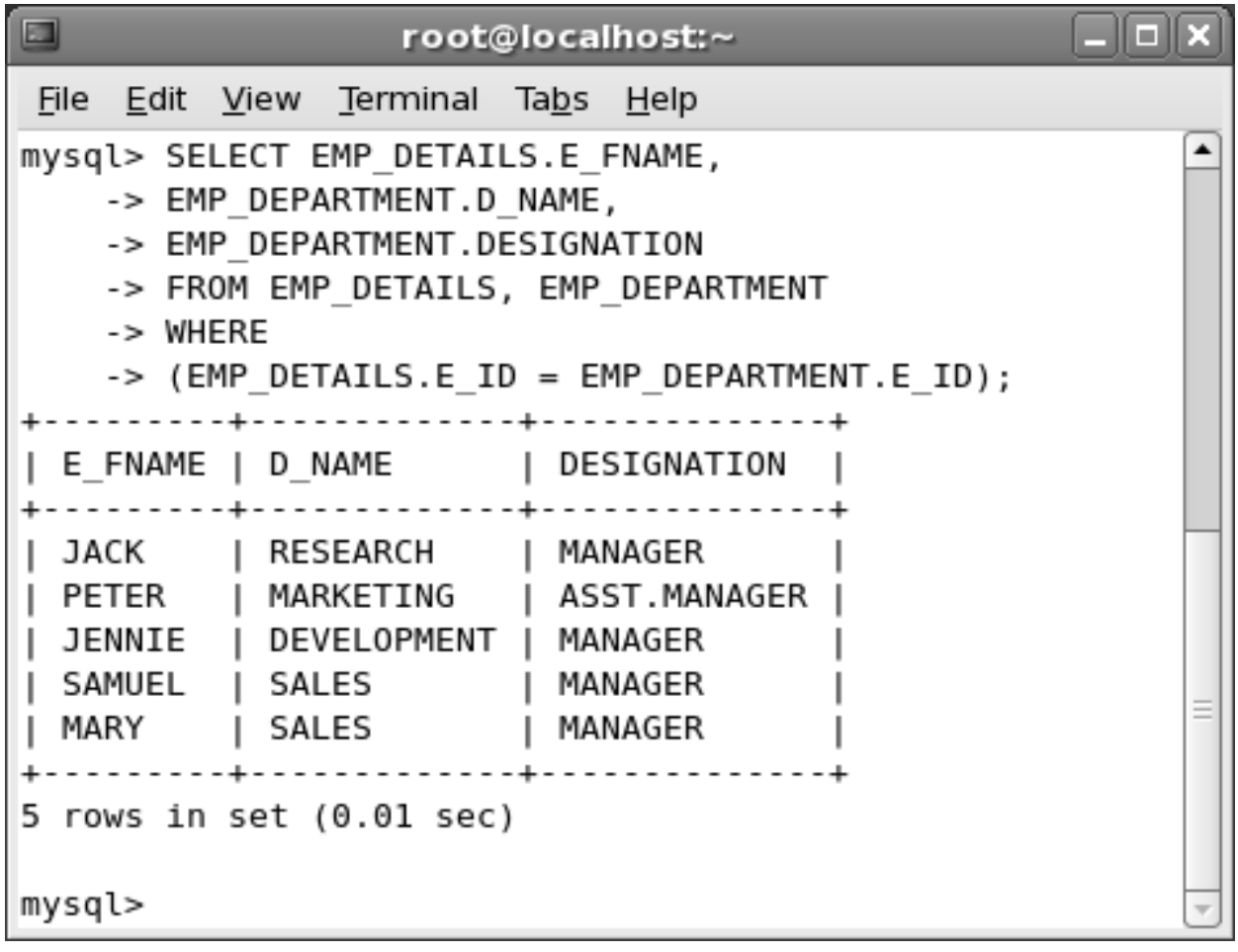


Figure 10.1: Joining tables with Equi-Join

Figure 10.1 displays all the employee names with their corresponding department names and designations. Here `E_ID` is a common attribute or column for both the tables.

10.4 Using INNER JOIN

The most common join operation used in applications is an `INNER JOIN`. An `INNER JOIN` creates a virtual table by combining the fields of both tables that satisfy the query for both the tables. The output of the `INNER JOIN` is same as that of the `Equi-join`. The only difference is in the syntax.

The syntax for inner join is:

```
SELECT column_name1, column_name2, column_name3 FROM table_name1 INNER JOIN
table_name2 ON table_name1.primary_keyfield = table_name2.foreign_keyfield;
```

Concepts

where,

`SELECT` – retrieves data from the table

`column_name` – specifies the name of the column

`table_name` – specifies the name of the table that contain data

`INNER JOIN` – creates a virtual table by combining the fields from both tables that satisfy the query

`table_name1.primary_keyfield = table_name2.foreign_keyfield` – specifies the relationship between the tables

For example, to display the first names, departments, and designations of all employees from `EMP_DETAILS` and `EMP_DEPARTMENT` tables from `EMPLOYEE` database, enter the following command at the command prompt:

```
SELECT EMP_DETAILS.E_FNAME, EMP_DEPARTMENT.D_NAME, EMP_DEPARTMENT.
DESIGNATION FROM EMP_DETAILS INNER JOIN EMP_DEPARTMENT ON EMP_DETAILS.E_ID
= EMP_DEPARTMENT.E_ID;
```

Figure 10.2 displays the output of the command.

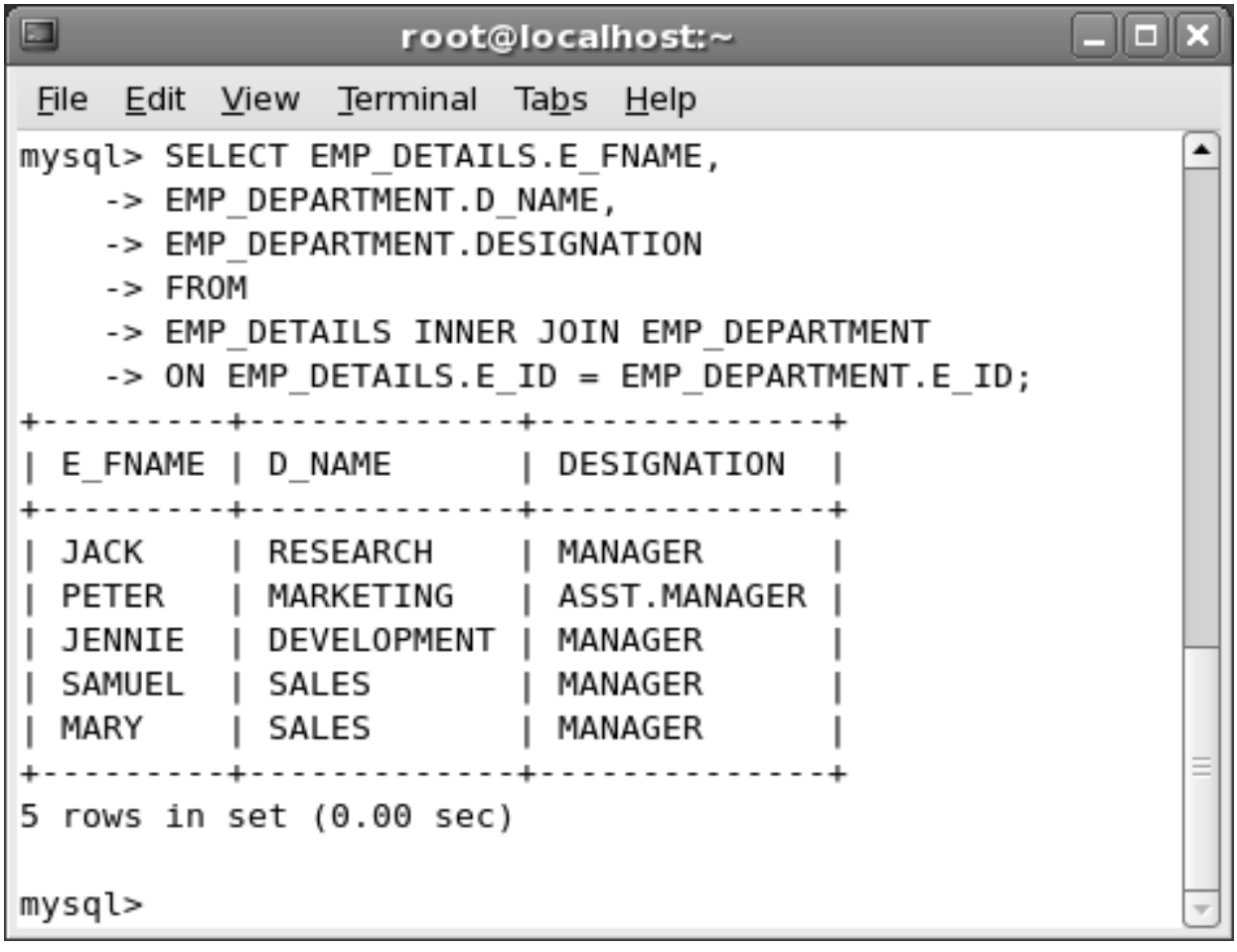


Figure 10.2: Performing Inner Join

Figure 10.2 displays all the employee names with their corresponding department names and designations using `INNER JOIN` between both the tables. Here, `INNER JOIN` is used for `Equi-Join`.

You can join numeric fields together, as long as they are of same data type such as `AutoNumber` or `Long`. In case of non-numeric data, the fields must be of same type and length and should contain same kind of data.

In the `ON` clause, you can use relational operators such as =, <, >, <=, >=, or <>. In the example, MySQL displays the first names and the designations of those employees whose `E_ID` of the `EMP_DETAILS` table matches with `E_ID` of the `EMP_DEPARTMENT` table. The relational operator, =, checks for equality of `E_ID` of both the tables.

To display the designations of those employees whose `E_ID` of `EMP_DETAILS` table does not match with the `E_ID` of `EMP_DEPARTMENT` table, enter the following command at the command prompt:

```
SELECT DISTINCT EMP_DETAILS.E_ID,EMP_DETAILS.E_FNAME, EMP_DEPARTMENT.
```

```
DESIGNATION FROM EMP_DETAILS INNER JOIN EMP_DEPARTMENT ON EMP_DETAILS.E_ID
<> EMP_DEPARTMENT.E_ID;
```

Figure 10.3 displays the output of the command.

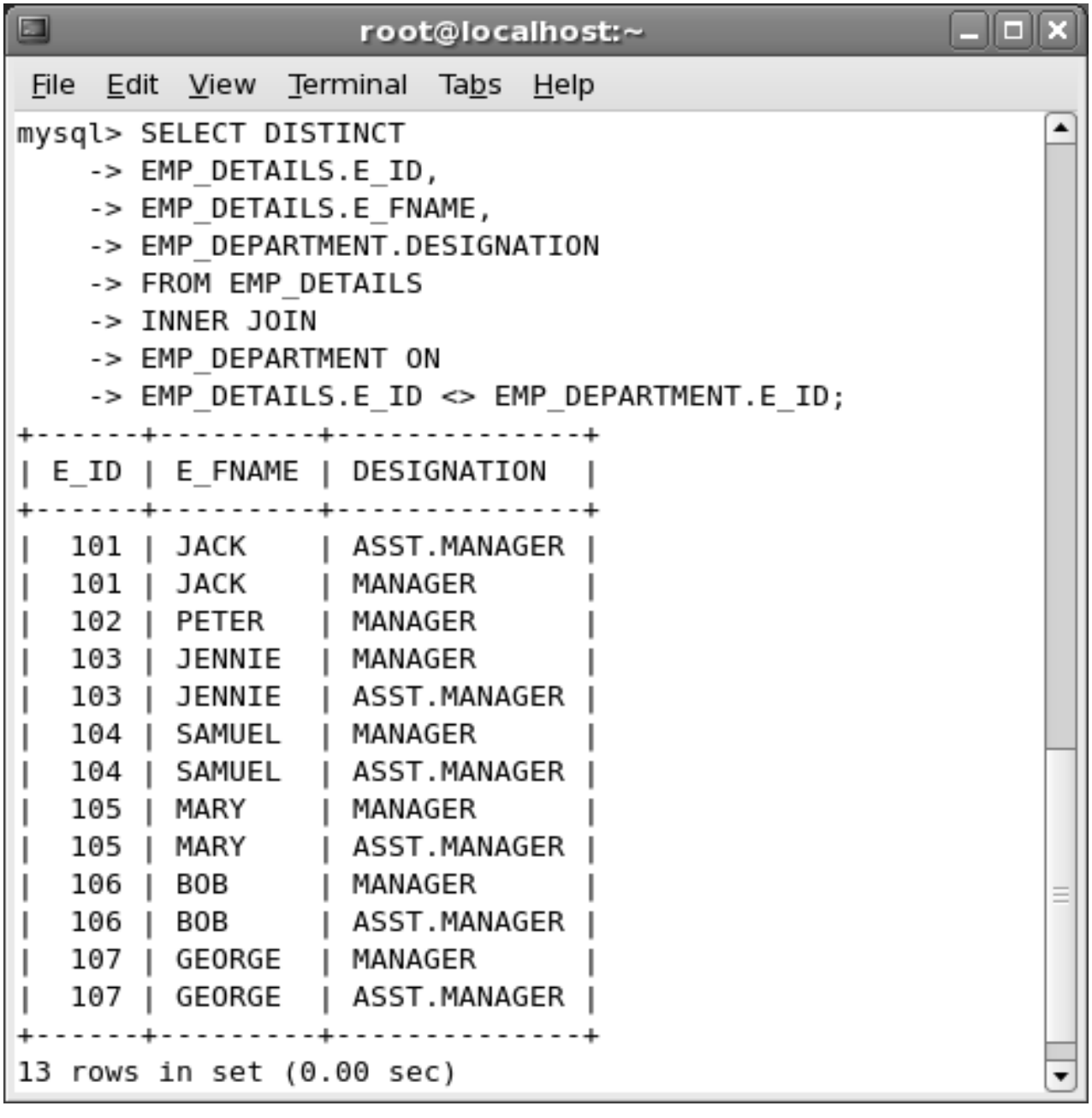


Figure 10.3: INNER JOIN

You can also use the `AND` or the `OR` operator with `INNER JOIN` command. For example, to view all employees whose `E_ID` values are same and who live in California city, enter the following command at

the command prompt:

```
SELECT EMP_DETAILS.E_FNAME, EMP_DEPARTMENT.D_NAME, EMP_DETAILS.E_ADDRESS FROM
EMP_DETAILS INNER JOIN EMP_DEPARTMENT ON (EMP_DETAILS.E_ID = EMP_DEPARTMENT.
E_ID) AND (EMP_DETAILS.E_ADDRESS = 'CALIFORNIA ');
```

Figure 10.4 displays the output of the command.

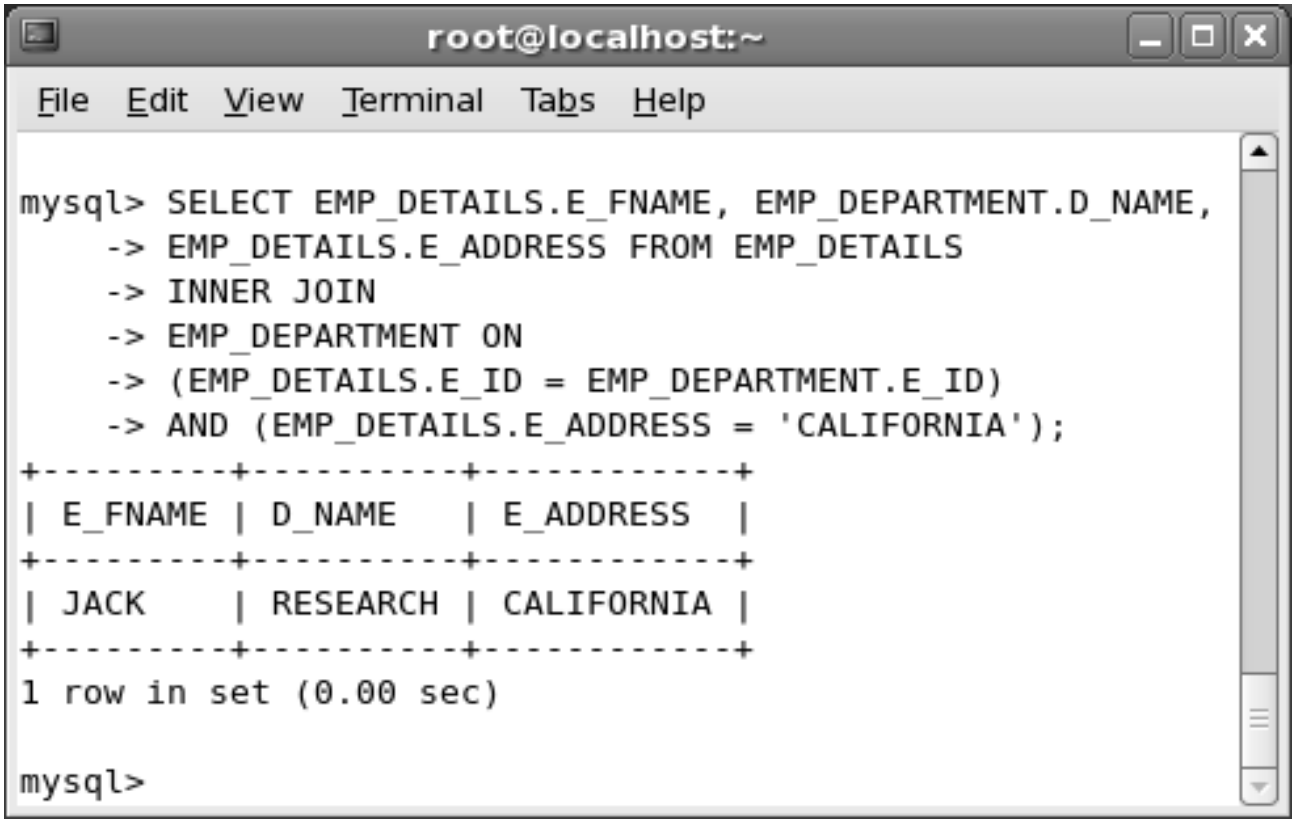


Figure 10.4: Joining tables Using INNER JOIN

Similarly, you can combine records from three or more tables and display the output in a single result set. For example, to view all employees who have a basic salary greater than $2,000, you will retrieve data from three tables (`EMP_DETAILS`, `EMP_DEPARTMENT`, and `EMP_SALARY`) and combine them into a single result set.

To join data from three tables, enter the following command at the command prompt:

```
SELECT EMP_DETAILS.E_FNAME, EMP_DEPARTMENT.D_NAME, EMP_SALARY.BASIC_SALARY
FROM EMP_DETAILS INNER JOIN (EMP_DEPARTMENT INNER JOIN EMP_SALARY ON (EMP_
DEPARTMENT.E_ID = EMP_SALARY.E_ID) AND (EMP_SALARY.BASIC_SALARY > 2000)) ON
EMP_DETAILS.E_ID = EMP_DEPARTMENT.E_ID;
```

Figure 10.5 displays the output of the command.

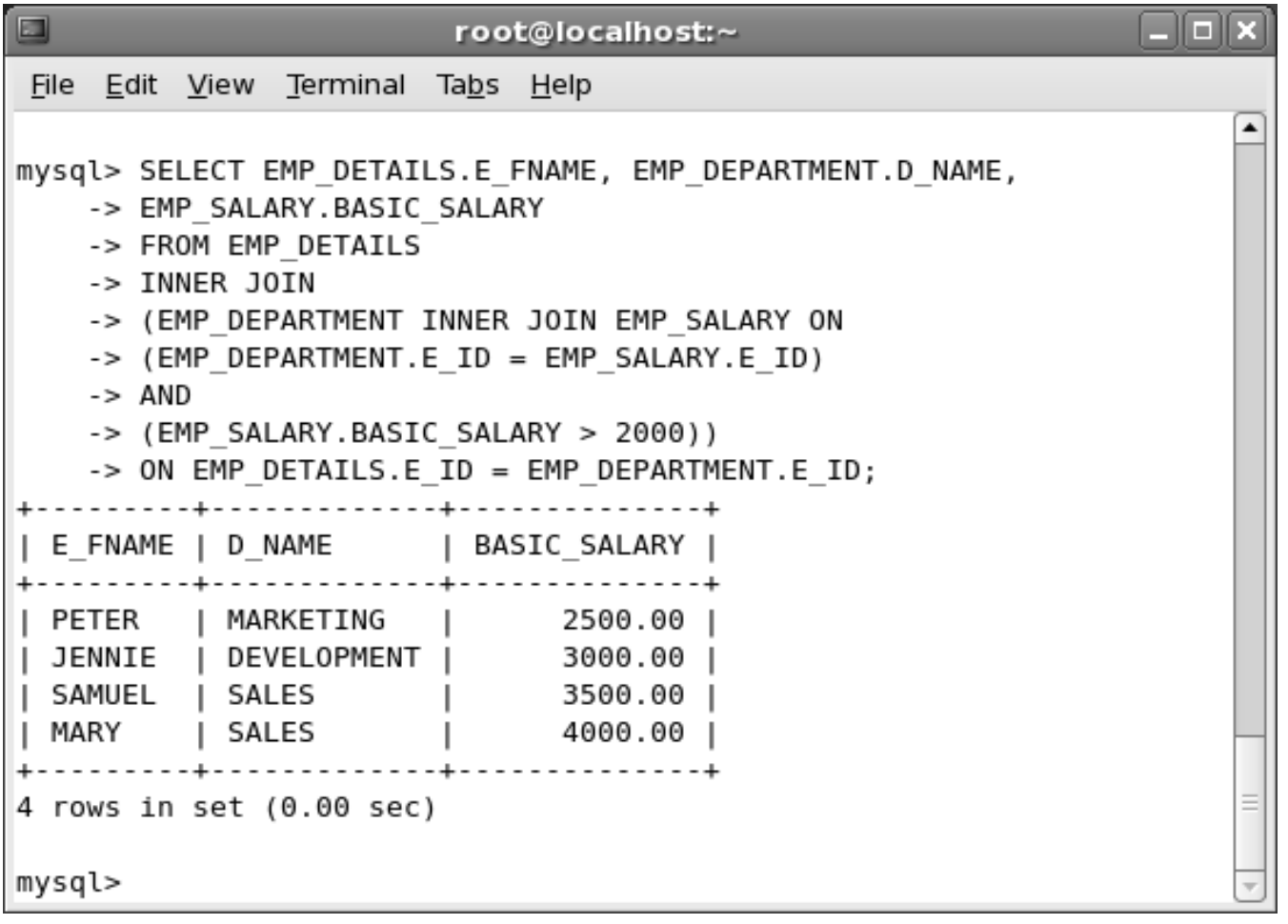


Figure 10.5: Joining three tables Using INNER JOIN

Figure 10.5 displays the employee details of only those employees whose `E_ID` from the `EMP_DETAILS` table matches with the `E_ID` of the `EMP_DEPARTMENT` and `EMP_SALARY` tables and whose basic salary is greater than $2000.

To view employee details using table aliases, enter the following command at the command prompt:

```
SELECT E.E_FNAME, E.E_LNAME, S.BASIC_SALARY FROM EMP_DETAILS AS E, EMP_SALARY
AS S WHERE E.E_ID = S.E_ID;
```

Figure 10.6 displays the output of the command.

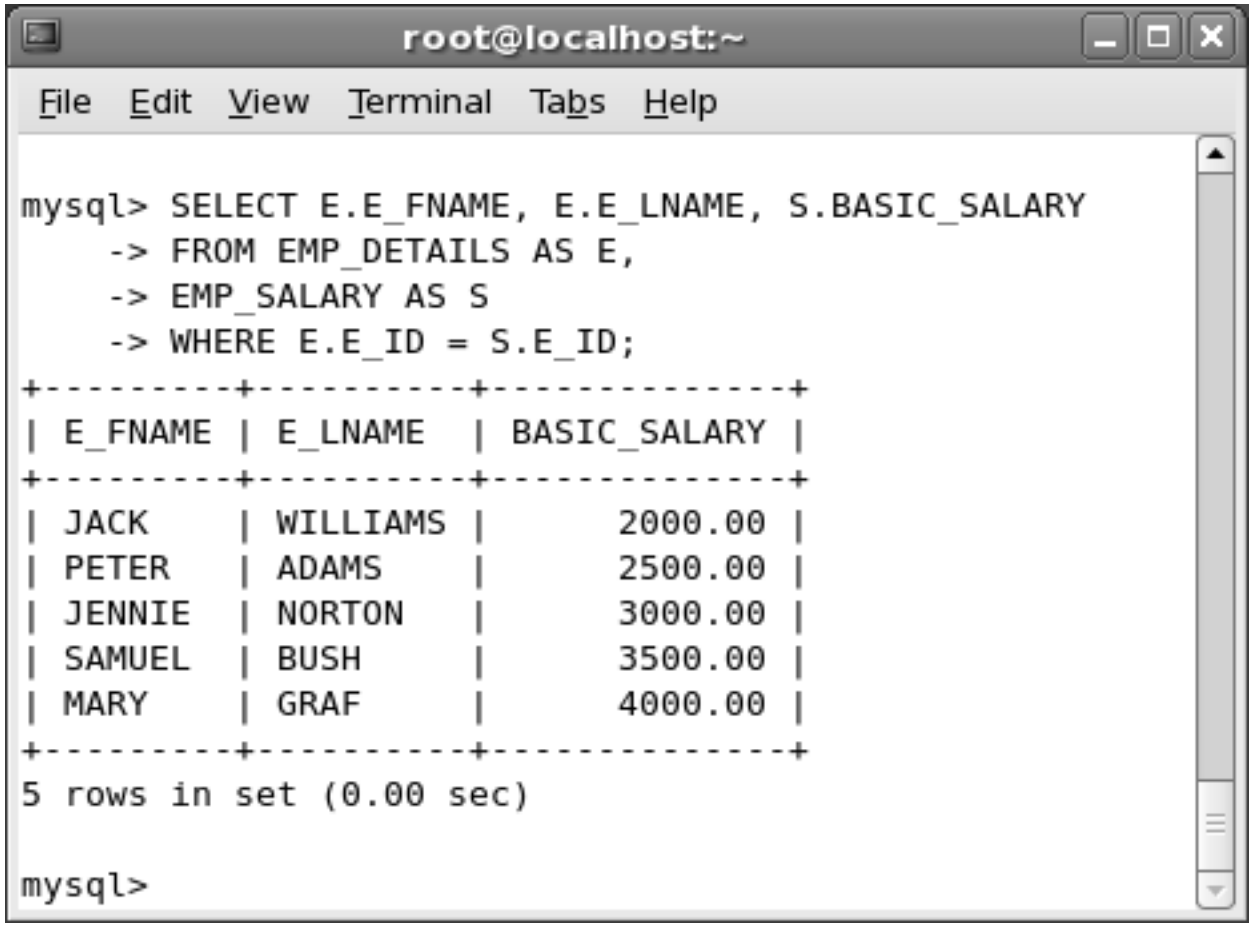


Figure 10.6: Joining Two Tables Using Aliases

In figure 10.6, the `AS` clause is used to refer to aliases for `EMP_DETAILS` and `EMP_SALARY` tables. The alias used for `EMP_DETAILS` table is E and for `EMP_SALARY` table is S.

MySQL also supports multi-table delete statements using the `DELETE` command. You can specify to remove data from multiple tables in the `DELETE` command. The syntax to delete matching rows from two tables is:

```
DELETE [LOW PRIORITY|QUICK] [IGNORE] tbl_name[.*][,tbl_name[.*]...] FROM
table_references [WHERE where_definition];
```

where,

`DELETE` – removes data from the table

`tbl_name[.*]` – specifies the names of the table that contain data to delete

table_references – specifies the tables that are linked with a key

WHERE – defines the condition to satisfy before deleting records from the table

This command deletes matching rows from the tables that are listed before the FROM clause.

You can also include the FROM and USING clause with the DELETE command to remove matching rows from tables. The syntax to delete matching rows with DELETE command having FROM and USING clause is:

```
DELETE [LOW PRIORITY|QUICK][IGNORE] FROM tbl_name[.*][,tbl_name[.*]...]USING
table_references [WHERE where_definition];
```

where,

DELETE – removes data from the table

tbl_name[.*] – specifies the names of the table that contain data to delete

USING – compares data between tables and deletes the matching records that satisfy the conditions specified in the WHERE clause

table_references – specifies the tables that are linked with a key

WHERE – defines the condition to satisfy before deleting records from the table

This command deletes matching rows from the tables specified in the FROM clause and before the USING clause.

For example, to delete records from the EMP_DEPARTMENT table that have matching values with EMP_DETAILS table, enter the following command at the command prompt:

```
DELETE FROM EMP_DEPARTMENT
USING EMP_DETAILS
INNER JOIN EMP_DEPARTMENT
ON EMP_DETAILS.E_ID = EMP_DEPARTMENT.E_ID;
```

Figure 10.7 displays the output of the command.

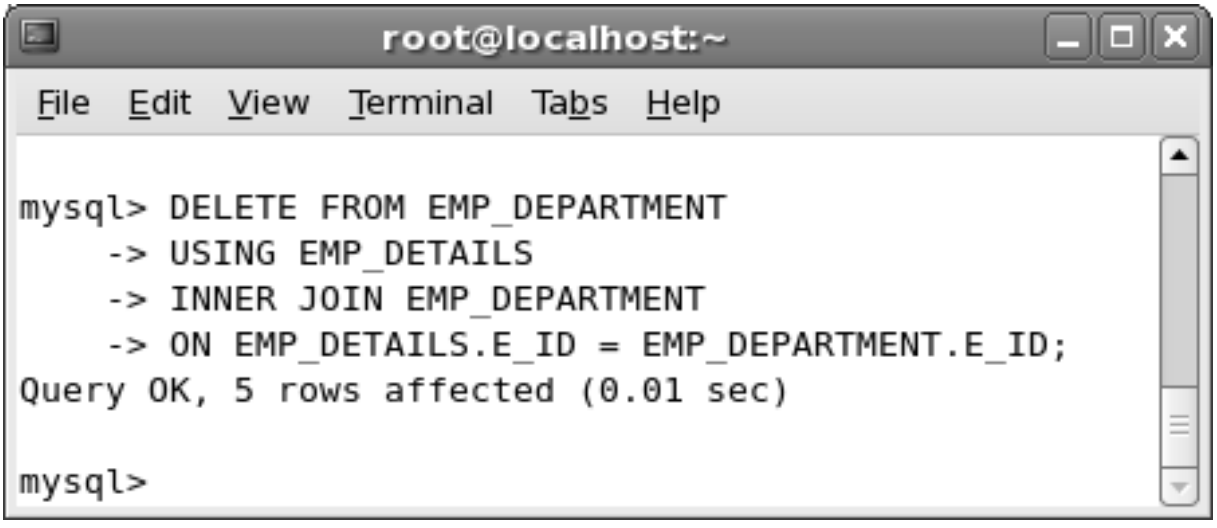


Figure 10.7: Deleting Matching Records after Comparison

Figure 10.7 displays the deletion of records from the `EMP_DEPARTMENT` table.

Note: You will have to add data to the `EMP_DEPARTMENT` table using the `INSERT` command after executing this command because the `DELETE` command removes all rows from `EMP_DEPARTMENT` table that have matching `E_ID` with `EMP_DETAILS` table.

10.5 Using OUTER JOIN

In the previous section, you used the `INNER JOIN` to view records that were common between two or more tables. An `OUTER JOIN` displays rows of the tables that may not have any matching value in the other tables to appear in the result set. To use an `OUTER JOIN`, the tables must have one or more columns in common.

The two types of **OUTER JOIN** are as follows:

- **LEFT JOIN** – is also known as **LEFT OUTER JOIN**. It displays all the records from the table specified on the left of the `OUTER JOIN` operator in the result set, with or without any matching records from the table specified on the right. If no matching record is found in any of the rows from the table placed on the right, then the corresponding columns of that row will contain `NULL` values.

  The syntax for `LEFT OUTER JOIN` is:

```
SELECT COLUMN1, COLUMN2 FROM TABLE1
LEFT OUTER JOIN TABLE2 ON (JOIN CONDITION);
```

where,

SELECT – retrieves data from columns in the table

COLUMN1,… – specifies the name of the column to retrieve data from

LEFT OUTER JOIN - displays all rows from the table specified on the left of the OUTER JOIN operator in the result set, with or without any matching record from the table specified on the right

TABLE1 – specifies the name of the table

JOIN CONDITION – specifies the conditions to retrieve data from tables

Table 10.1 lists the JOIN conditions that can be specified in the ON clause.

Join Condition	Description
BETWEEN	Compares values that lie in between a specific range
COMPARISON	Compares two values using operators such as =, >, <, >=, <=, <>
EXIST	Determines whether a value exist in the given table
IN	Determines whether a value exist in the list of values or a table
IS NULL	Compares a value with an empty or NULL value
LIKE	Compares one value with another
SOME/ANY/ALL	Performs quantified comparisons

Table 10.1: JOIN Conditions in the ON Clause

You can also specify the join conditions using the WHERE and HAVING clauses.

To view all the records from EMP_DETAILS and EMP_DEPARTMENT tables by joining them using LEFT OUTER JOIN, enter the following command at the command prompt:

```
SELECT EMP_DETAILS.E_ID, EMP_DEPARTMENT.D_NAME, EMP_DEPARTMENT.DATE_OF_
JOIN FROM EMP_DETAILS LEFT OUTER JOIN EMP_DEPARTMENT ON EMP_DETAILS.E_ID
= EMP_DEPARTMENT.E_ID;
```

Note: The term OUTER in the LEFT OUTER JOIN clause is optional.

Figure 10.8 displays the output of the command.

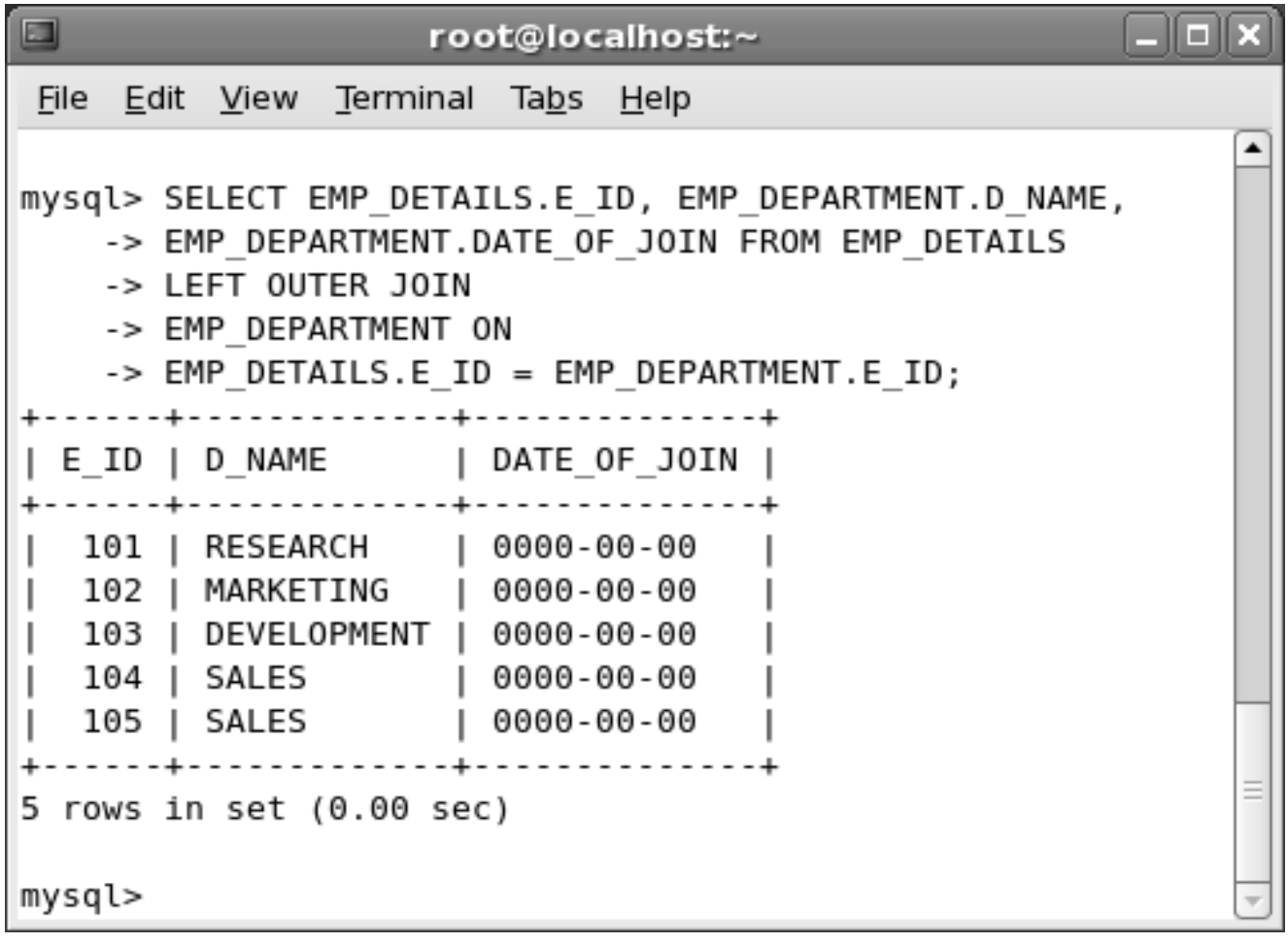


Figure 10.8: Joining Tables Using LEFT OUTER JOIN

In figure 10.8, `EMP_DETAILS` table is joined with `EMP_DEPARTMENT` table with the help of `E_ID` as the common attribute.

You can specify conditions in a join with the `ON` and `USING` clauses in the `SELECT` command. The `ON` clause specifies the conditions to join tables. The `USING` clause specifies the requirement for matching columns to be present in both the tables.

To view all the records from `EMP_DETAILS` and `EMP_DEPARTMENT` tables by joining them with the `USING` clause, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS LEFT JOIN EMP_DEPARTMENT USING(E_ID);
```

Figure 10.9 displays the output of the command.

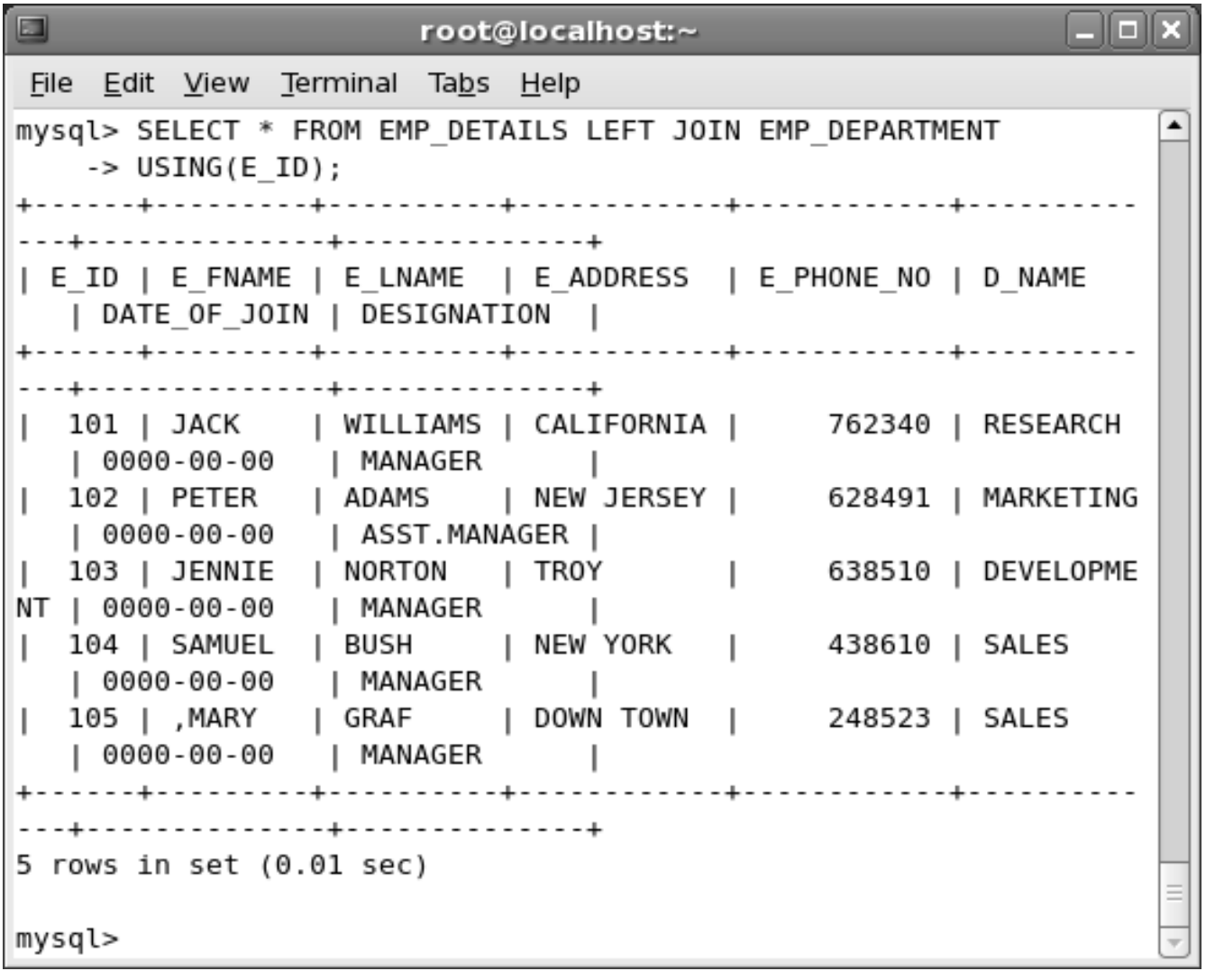


Figure 10.9: Joining Tables Using LEFT JOIN with USING clause

Figure 10.9 displays the records of `EMP_DETAILS` table that corresponds with `E_ID` of `EMP_DEPARTMENT` table. Both the tables are joined with `E_ID` as a common attribute and the `USING` clause is used for specifying a condition.

The syntax for joining three tables using the `LEFT OUTER JOIN` is:

```
SELECT COLUMN1, COLUMN2 FROM TABLE1
LEFT OUTER JOIN TABLE2 ON (JOIN CONDITION)
LEFT OUTER JOIN TABLE3 ON (JOIN CONDITION);
```

where,

`SELECT` – retrieves data from the table

`COLUMN1,…` – specifies the name of the column to be retrieved

`TABLE1` – specifies the name of the table that contain the columns

`LEFT OUTER JOIN` – compares the records from tables using the conditions specified

`JOIN CONDITON` – contains the clauses to satisfy, before retrieving data from the tables

For example, to view all the records from `EMP_DETAILS`, `EMP_SALARY`, and `EMP_DEPARTMENT` tables by joining them with the common attribute `E_ID`, enter the following command at the command prompt:

```
SELECT E_FNAME,E_LNAME,E_ADDRESS,
BASIC_SALARY FROM EMP_DETAILS
LEFT OUTER JOIN
EMP_SALARY ON
EMP_DETAILS.E_ID = EMP_SALARY.E_ID
LEFT OUTER JOIN
EMP_DEPARTMENT ON
EMP_SALARY.E_ID = EMP_DEPARTMENT.E_ID;
```

Figure 10.10 displays the output of the command.

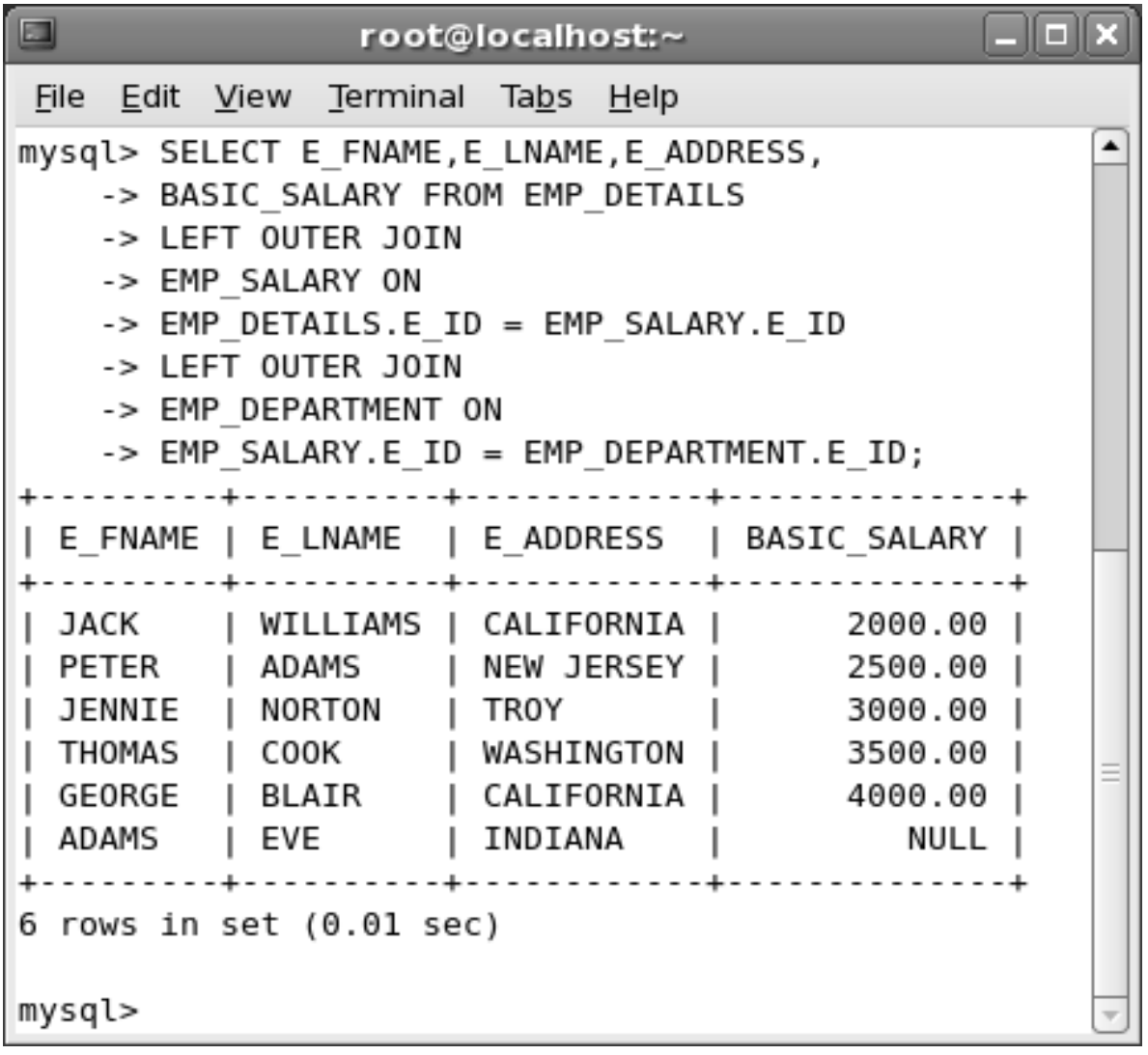


Figure 10.10: Joining three tables Using LEFT OUTER JOIN

In figure 10.10, the `EMP_DETAILS` table is joined with the `EMP_SALARY` table and the `EMP_SALARY` table is joined with the `EMP_DEPARTMENT` table. All the three tables have `E_ID` as a common attribute for joining the tables.

- **RIGHT OUTER JOIN:** displays all records from the table specified on the right of the `OUTER JOIN` operator in the result set, with or without any matching records from the table specified on the left. If there is no matching record found in any of the row from the table placed on the left, then the corresponding columns of that row will contain `NULL` values.

The syntax for `RIGHT OUTER JOIN` is:

```
SELECT COLUMN1, COLUMN2 FROM TABLE1
RIGHT OUTER JOIN TABLE2 ON (JOIN CONDITION);
```

where,

`SELECT` – retrieves data from the table

`COLUMN1,…` – specifies the name for the column to retrieve data from

`TABLE1` – specifies the name of the table that contains the columns

`RIGHT OUTER JOIN` – specifies the comparison type to fetch data from the tables

For example, to display `E_LNAME`, `E_ADDRESS`, and `E_PHONE_NO` from `EMP_DETAILS` table where the employee ids range from `102` to `105`, you will compare the values of `EMP_DEPARTMENT` table with the `EMP_DETAILS` table and display the output.

```
SELECT DISTINCT E_LNAME,E_ADDRESS,E_PHONE_NO,D_NAME
FROM EMP_DETAILS E
RIGHT OUTER JOIN
EMP_DEPARTMENT D ON
(E.E_ID BETWEEN 102 AND 105);
```

Figure 10.11 displays the output of the command.

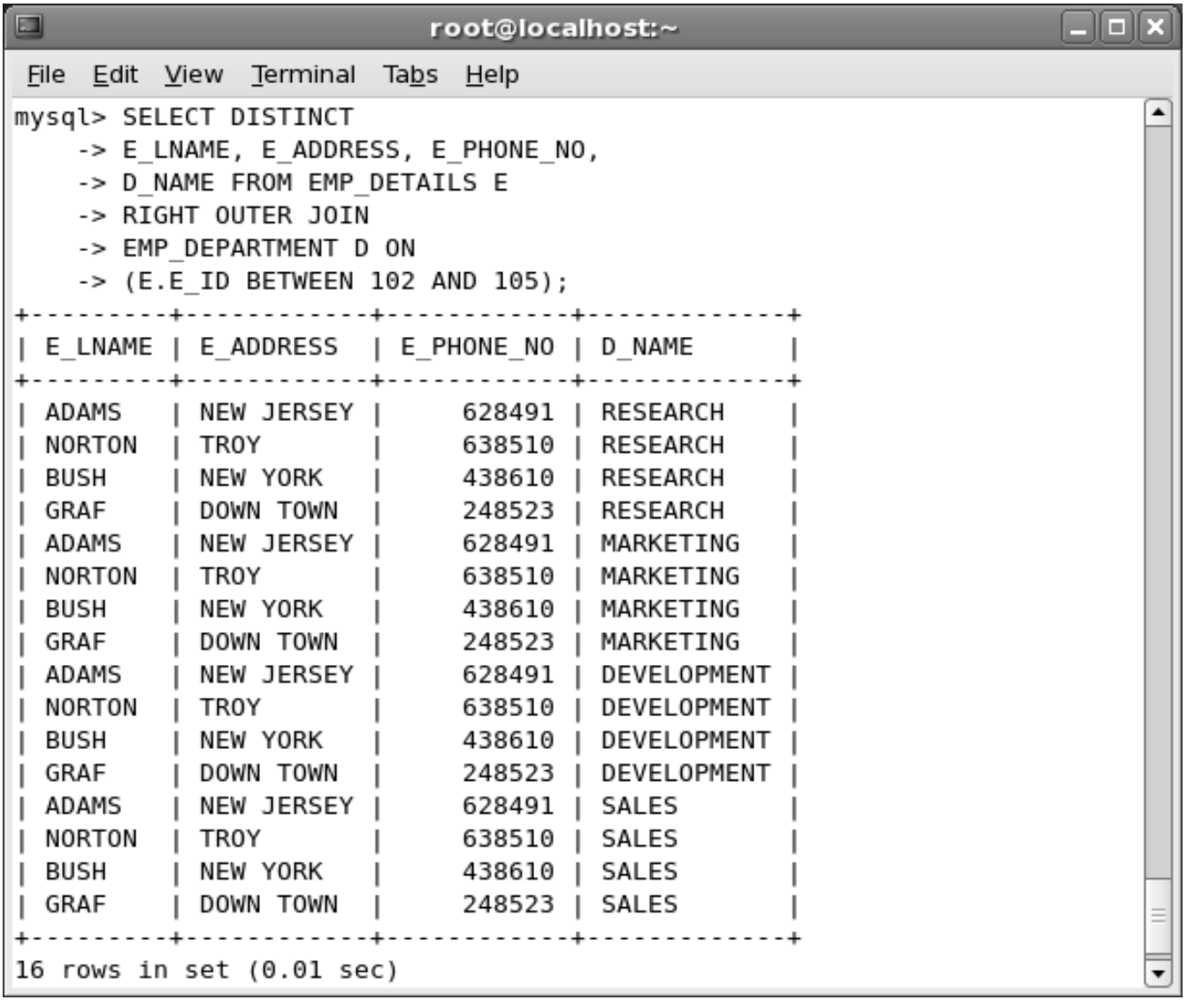


Figure 10.11: Displaying Details Using Right Outer Join

For example, to display `E_LNAME`, `E_ADDRESS`, and `E_PHONE_NO` from `EMP_DETAILS` table where the employee ids are greater than `102`, perform a join operation of the `EMP_DEPARTMENT` table with the `EMP_DETAILS` table.

```
SELECT DISTINCT E_LNAME,E_ADDRESS,E_PHONE_NO FROM
EMP_DETAILS E
RIGHT OUTER JOIN EMP_DEPARTMENT D ON (E.E_ID > 102);
```

Figure 10.12 displays the output of the command.

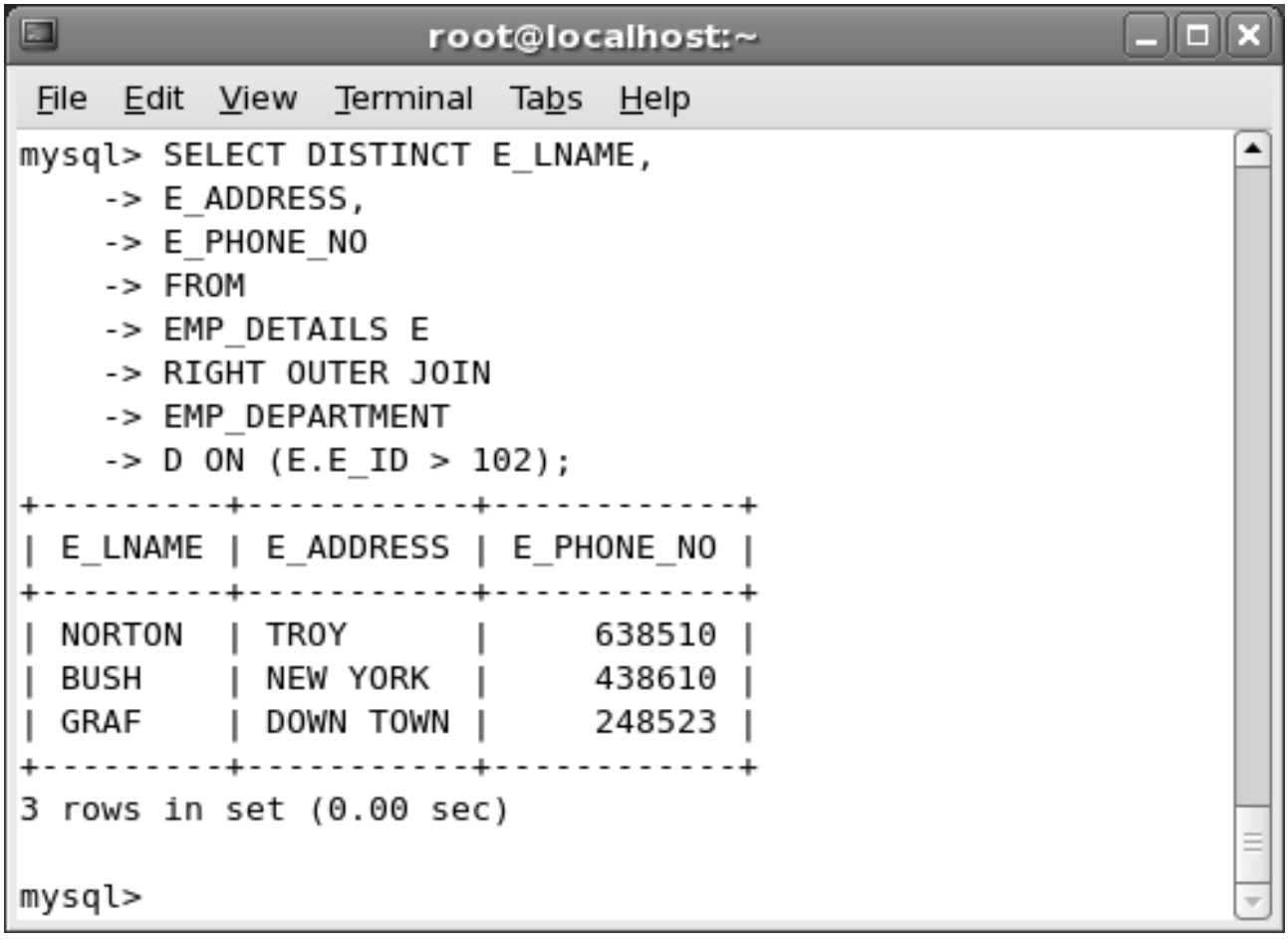


Figure 10.12: Displaying Unique Values Using RIGHT OUTER JOIN

You can make a comparison using the `SUBSTRING` function. A `SUBSTRING` function extracts string or character literal from a column of a table, specified in the column reference. The `FROM` clause of `SUBSTRING` function specifies the character position from where the extracted string should start. The `FOR` clause of the function specifies the length of the extracted string.

Similarly, to use the `SUBSTRING` function with the `RIGHT OUTER JOIN`, enter the following command at the command prompt:

```
SELECT D.D_NAME,
D.DATE_OF_JOIN,
D.DESIGNATION
FROM EMP_DEPARTMENT D
RIGHT OUTER JOIN
EMP_DETAILS E
ON(SUBSTRING(E.E_ADDRESS,1,3) = 'ÇAL');
```

Figure 10.13 displays the output of the command.

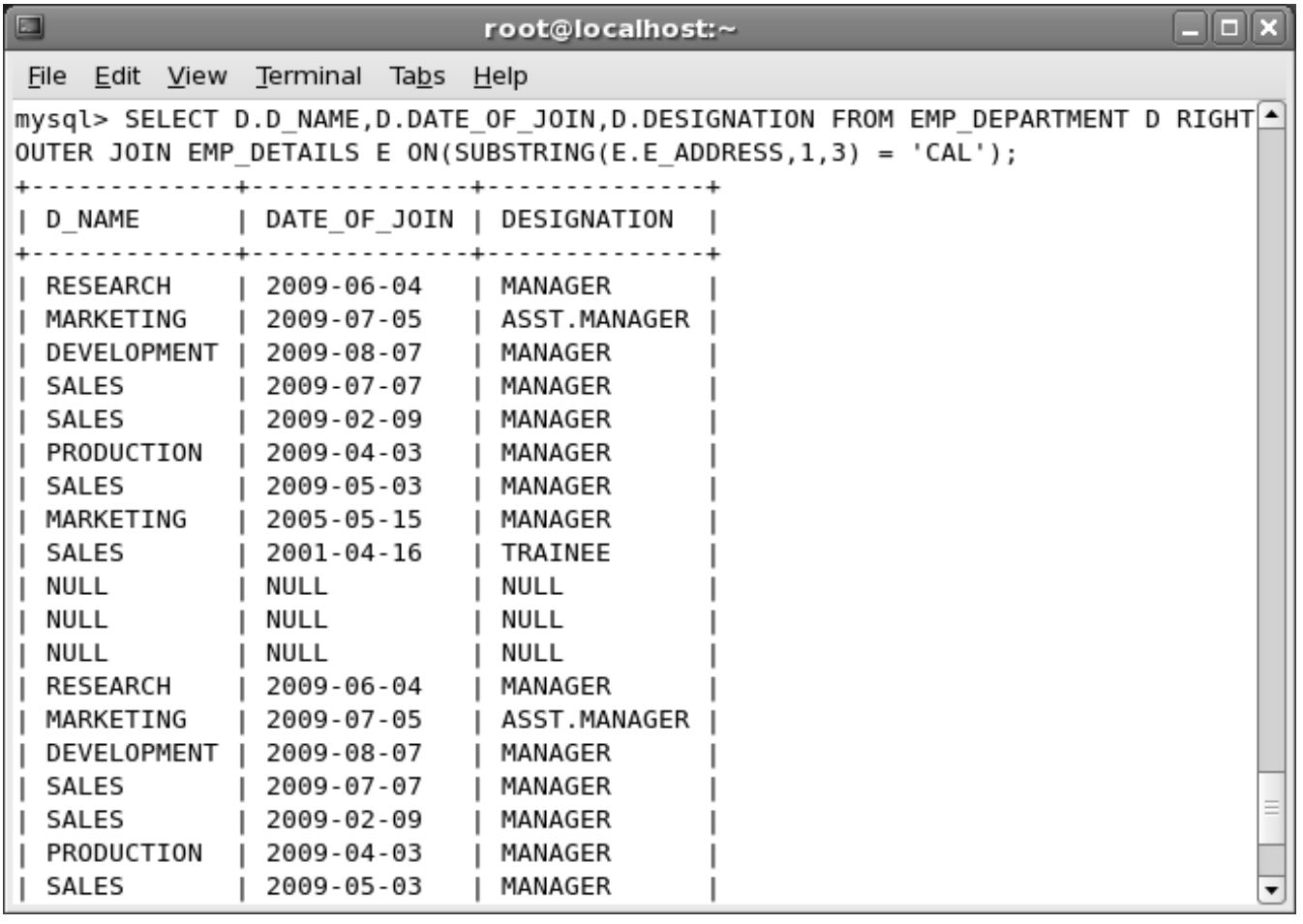


Figure 10.13: Joining tables Using RIGHT OUTER JOIN

10.6 Using Self-Join

A Self-Join statement joins or compares a table with itself. It means that self-join compares records of a column with the other.

The syntax for Self Join is:

```
SELECT DISTINCT COLUMN1, COLUMN2 FROM TABLE WHERE (JOIN CONDITION);
```

where,

`SELECT` – retrieves data from the table

`DISTINCT` – displays unique values

`COLUMN1,…` – specifies the name for the column to retrieve data from

`TABLE` – specifies the name of the table that contains the columns

Note: The `DISTINCT` clause ignores duplicates and lists unique records.

For example, to view all the employees, who are located in the same city, enter the following command at the command prompt:

```
SELECT DISTINCT E1.E_ID, E1.E_FNAME, E1.E_LNAME, E1.E_ADDRESS
FROM EMP_DETAILS AS E1, EMP_DETAILS AS E2
WHERE E1.E_ADDRESS = E2.E_ADDRESS
AND E1.E_ADDRESS IS NOT NULL
AND E2.E_ADDRESS IS NOT NULL
AND E1.E_ID <> E2.E_ID
ORDER BY E1.E_ADDRESS, E1.E_ID;
```

Figure 10.14 displays the output of the command.

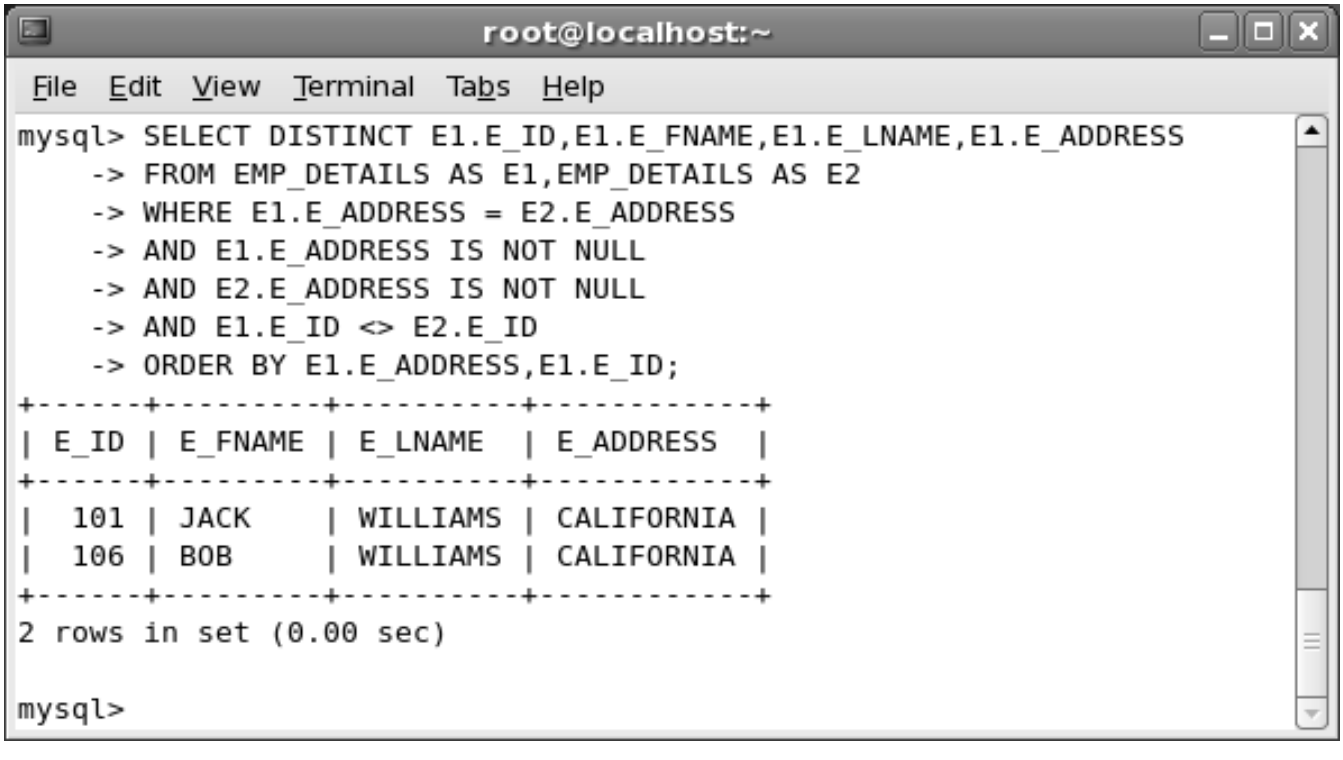


Figure 10.14: Performing Self-Join

In figure 10.14, two different aliases for one table are used to compare the values of the E_ADDRESS field with itself.

10.7 Using Subqueries and UNIONS

A query used inside another select query is known as subquery. You will use the UNION command when you want to combine tables having same fields into a single result using the SELECT command.

10.7.1 Subqueries

When a SELECT query is placed inside another SELECT query, then the inner SELECT query is said to be a subquery. The syntax for the subquery is:

```
SELECT [*] column_name… FROM table_name1 WHERE column_name IN
(SELECT column_name FROM table_name2);
```

where,

SELECT – retrieves data from the table

* - retrieves all the records from the table

column_name – specifies the name of the column

IN – compares data from both the tables

table_name1 – specifies the name of the table

Consider an example, where you want to view the details of all the employees from all the departments. In this case, you will compare the employee ID in the EMP_DEPARTMENT and the EMP_DETAILS table. You will display the results only if the E_ID from EMP_DEPARTMENT matches with E_ID of EMP_DETAILS. To view all records from EMP_DETAILS and EMP_DEPARTMENT tables using subqueries, enter the following command at the command prompt:

```
SELECT * FROM EMP_DETAILS WHERE E_ID IN (SELECT E_ID FROM EMP_DEPARTMENT);
```

Figure 10.15 displays the output of the command.

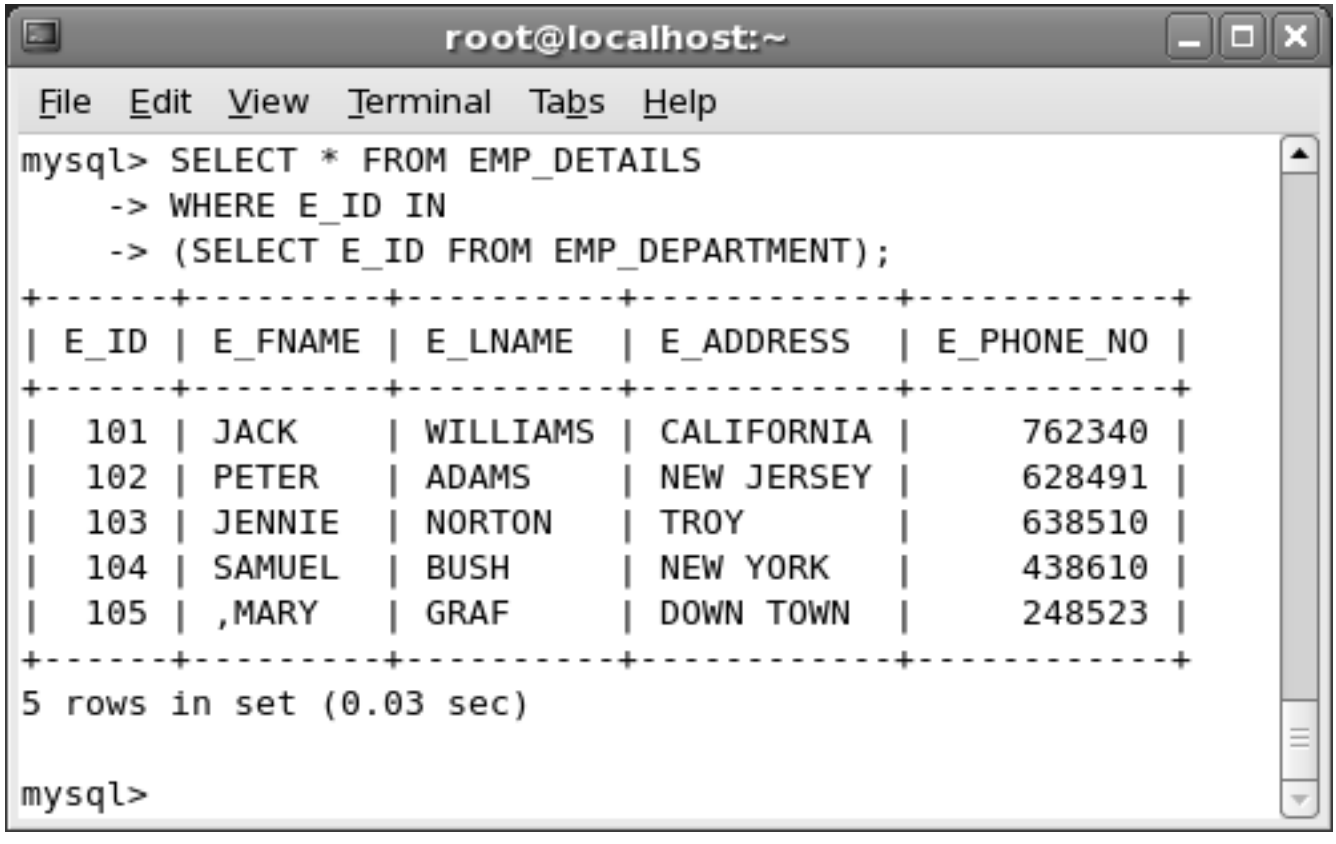


Figure 10.15: Retrieve Data Using Subqueries

In figure 10.15, the `IN` operator is used for comparing the employee IDs from both the tables.

You can use the `NOT IN` operator to display the unmatched values of tables. For example, to view `E_ID` of `EMP_DEPARMENT` table that do not match with the `E_ID` of `EMP_DETAILS` table, enter the following command at the command prompt:

```
SELECT * FROM EMP_DEPARTMENT
WHERE E_ID NOT IN
(SELECT E_ID FROM EMP_DETAILS);
```

Figure 10.16 displays the output of the command.

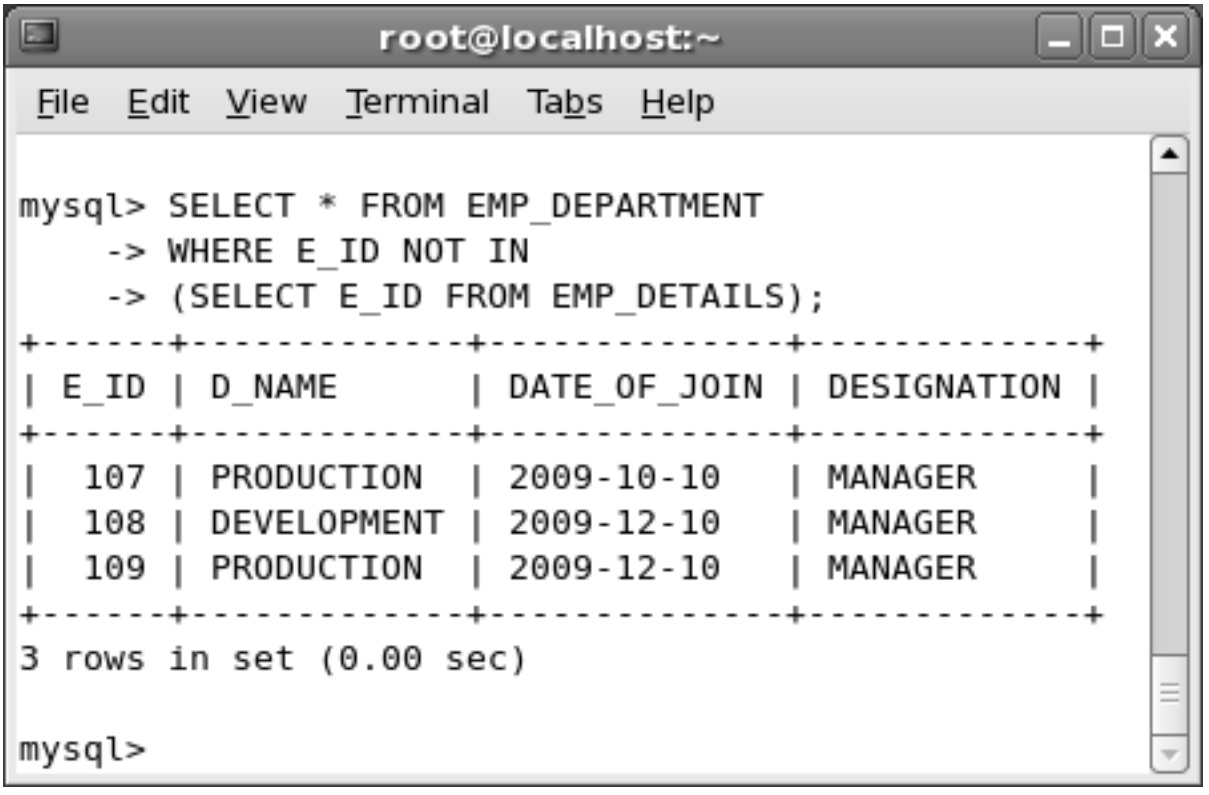


Figure 10.16: Retrieving Unmatched Data Using Subqueries

Figure 10.16 displays all the `E_ID` from the `EMP_DEPARTMENT` table that does not match with the `E_ID` of `EMP_DETAILS` table.

MySQL also supports the use of `SELECT` statement within the `REPLACE` and `INSERT` queries. It means that when a `SELECT` query is used within an `INSERT` or `REPLACE` query, then that `SELECT` query is said to be subquery of `INSERT` or `REPLACE` query.

For example, to insert department names of all employees from `EMP_DEPARTMENT` table to `EMP_SALARY` table, enter the following command at the command prompt:

```
INSERT INTO EMP_SALARY(D_NAME) SELECT D_NAME FROM EMP_DEPARTMENT;
```

Figure 10.17 displays the output of the command.

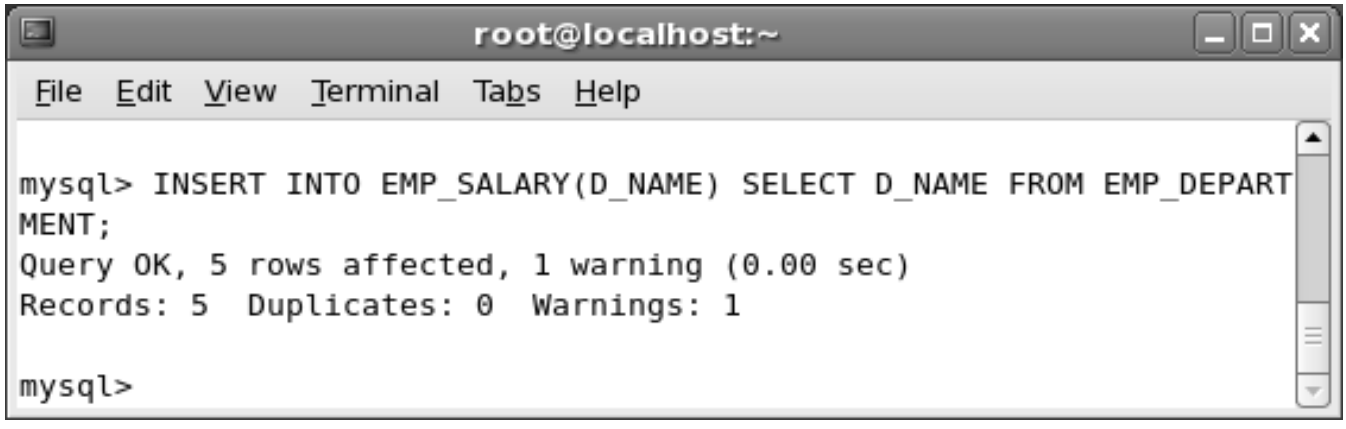


Figure 10.17: Using Subquery in INSERT Statement

10.7.2 Unions

A Union is used to combine many result sets used with `SELECT` command into a single result set. The syntax for the `UNION` command is:

```
SELECT * FROM TABLE1
UNION [ALL | DISTINCT]
SELECT * FROM TABLE2
UNION [ALL | DISTINCT];
```

where,

`SELECT` – retrieves data from the table

`*` - retrieves all data from the table

`TABLE1` – specifies the name of the table that contains data

`UNION` – combines query results into a single result

The columns specified in the `SELECT` command should be of the same type. MySQL will use the column names from the first `SELECT` query as the column names for the results returned.

MySQL provides the `ALL` keyword to display matching rows from all the `SELECT` statements. If this keyword is not used, only unique rows are displayed.

In other words, if you do not use the keyword `ALL` in the `UNION` clause, all returned rows would be unique. If you specify `ALL`, then you will get all matching rows from all the `SELECT` statements.

Concepts

Note: The DISTINCT keyword is an optional keyword.

For example, to view E_ID and E_FNAME from EMP_DETAILS table and E_ID and D_NAME from EMP_DEPARTMENT table, enter the following command at the command prompt:

```
SELECT E_ID, E_FNAME FROM EMP_DETAILS
UNION
SELECT E_ID, D_NAME FROM EMP_DEPARTMENT;
```

Figure 10.18 displays the output of the command.

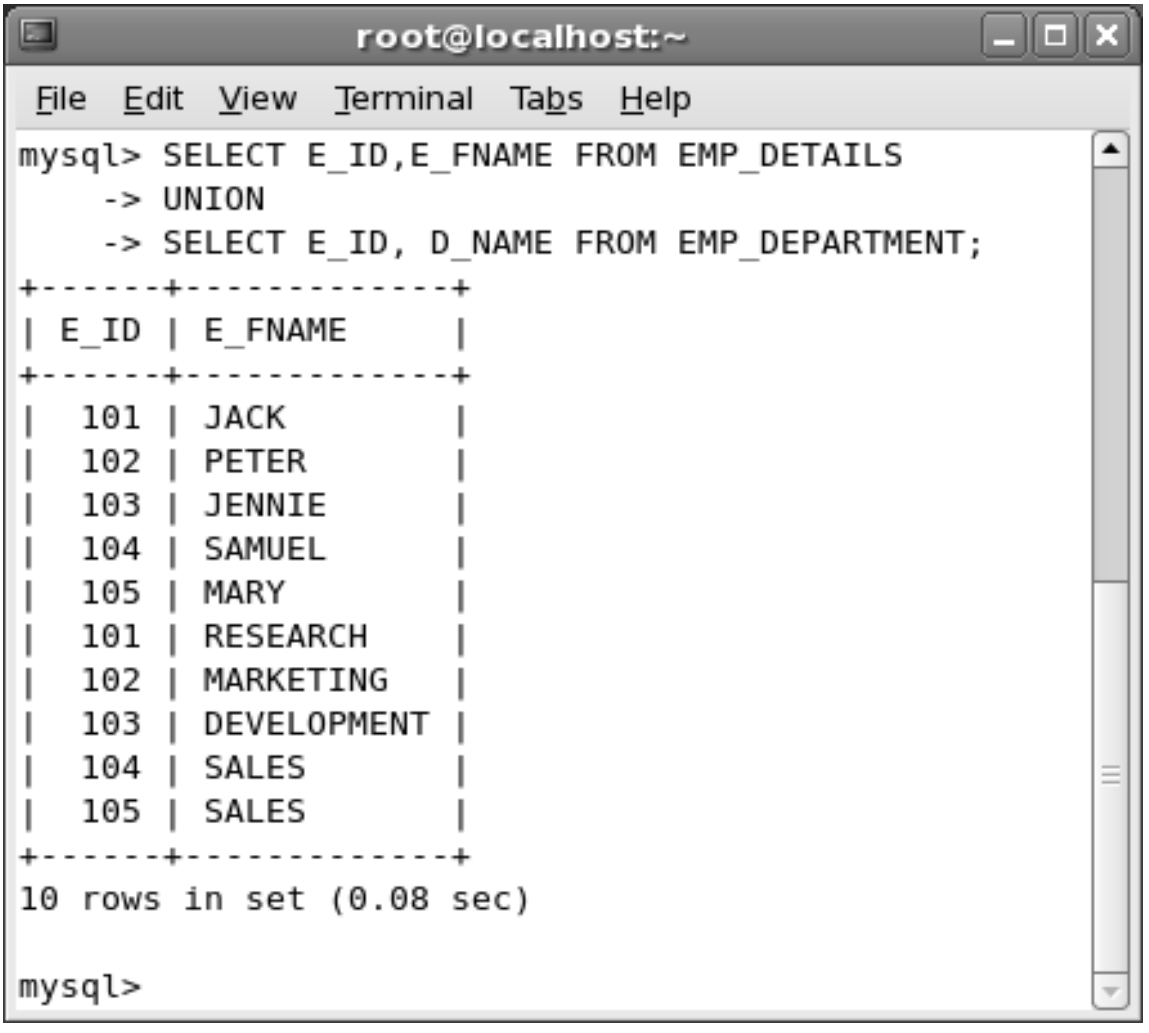


Figure 10.18: Using UNION for Joining Tables

The ORDER BY clause can also be used to sort the entire UNION results. The ORDER BY clause used to sort the results of the UNION must be included in the last SQL statement. The SELECT statements must

be included within parenthesis.

MySQL supports the LIMIT clause to restrict the number of records returned by the query. The restriction for the number of rows must be specified in one or two numeric values. These values must be in the form of non-negative integer constants. The first value in the argument defines the first row to return and the second value defines the maximum number of rows to return.

For example, to sort the records retrieved from the EMP_DETAILS table where the employee addresses are California and Troy, enter the following command at the command prompt:

```
(SELECT * FROM EMP_DETAILS WHERE E_ADDRESS = 'TROY' ORDER BY
E_ID LIMIT 2)
UNION
(SELECT * FROM EMP_DETAILS WHERE E_ADDRESS = 'CALIFORNIA' ORDER BY
E_ID LIMIT 1)ORDER BY E_ID;
```

Figure 10.19 displays the output of the command.

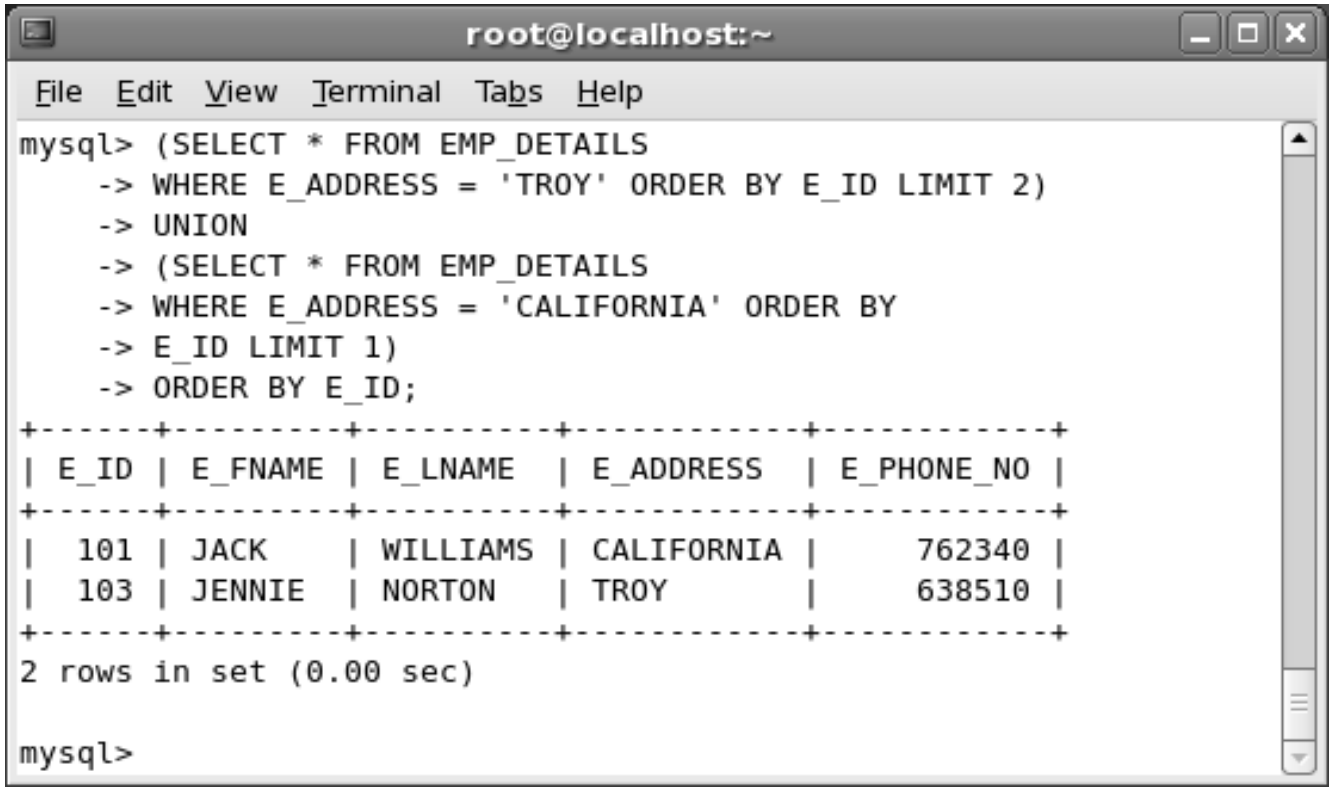


Figure 10.19: Using ORDER BY with UNION clause

Figure 10.19 displays the output that has been restricted using the LIMIT clause.

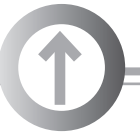
Concepts

Summary

- Joining of tables means combining two or more records from different tables of the same database into one comprehensive structure.
- Joining of tables is performed either using `WHERE` clause with the `SELECT` command or using the `JOIN` keyword. This will enhance manipulation, increase access speed, and reduce data redundancy.
- In an Equi-Join, the comparison is made between two columns that contain similar values.
- The `ON` clause specifies the conditional expression to be used in a `WHERE` clause. It also supports the use of relational operators such as =, <, >, <=, >=, or <>.
- The `USING` clause can be used in a join only when the tables have matching columns.
- Numeric fields can be joined together, as long as they are of same data type such as, `AutoNumber` or `Long`. Incase of non-numeric data, the fields must be of same type and length, and should contain same kind of data.
- An `INNER JOIN` creates a virtual table by combining the fields of both tables that satisfy the query for both the tables. The `AND` or the `OR` operators can also be used with `INNER JOIN` command.
- An `OUTER JOIN` is used to join two tables, a source and joining table, which have one or more columns in common. The rows of the tables that may not have any matching value in the other tables are also displayed in the result set.
- MySQL supports two types of `OUTER JOINS: LEFT OUTER JOIN` and `RIGHT OUTER JOIN`.
- The `LEFT OUTER JOIN` displays all rows from the table specified on the left of the `OUTER JOIN` operator in the result set, with or without any matching record with the table specified on the right.
- The `RIGHT OUTER JOIN` displays all rows from the table specified on the right of the `OUTER JOIN` operator in the result set, with or without any matching record with the table specified on the left.
- When using the `OUTER JOIN`, if there is no match found in any of the rows from the table placed on the right, then the corresponding columns of that row will contain `NULL` values.

Summary

- A Self-Join is a query that is used to join or compare a table to itself.
- The `DISTINCT` clause lists unique records from tables.
- A query used inside another select query is known as subquery.
- The `UNION` clause can be used to combine results from different `SELECT` commands.
- The `ORDER BY` clause must be used with a parenthesis to sort results displayed by the `UNION` option.
- The `LIMIT` clause restricts the number of results displayed by the query



Check Your Progress

1. A comparison is made between two columns that carry same values are termed as _____.

   a. Self-Join

   b. Equi-Join

   c. Left Join

   d. Sub-Select

2. When there is no match in the columns of the joining table with the source table, then those columns contain _____ values.

   a. Equal

   b. Different

   c. Null

   d. Single

3. When a select query is used inside another select query then the second select query is known as _____.

   a. Multiple query

   b. Subquery

   c. Join query

   d. Single query

4. A _____ function is used to extract string or _____ from a column of a table, specified in the column reference.

   a. IN, Numeric

   b. SUB-STRING, Character literal

Check Your Progress

c. EXIST, Decimal

d. LIKE, Boolean

5. Which of the following specifies the conditional expression that can be used in a `WHERE` clause?

a. USING

b. FROM

c. ON

d. HAVING

6. Which of the following clause determine the existence of values in the list or the table?

a. BETWEEN

b. LIKE

c. EXIST

d. IN

7. Which of the following options displays all the rows from all the tables specified in the `FROM` clause in the result set, with or without any match with the specified table?

a. LEFT OUTER JOIN

b. RIGHT OUTER JOIN

c. SELF JOIN

d. INNER JOIN

Onlinevarsity

Session
11 Using Joins (Lab)

Objectives

At the end of this session, the student will be able to:

- *Use different types of JOINS.*
- *Use a subquery.*
- *Use the UNION clause.*

The steps given in this session are detailed, comprehensive, and carefully thought-through in order to meet the learning objectives and understand the tool completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Using various types of JOINS

Tables are joined either using the `WHERE` clause with the `SELECT` command or by using the `JOIN` keyword.

1. **In order to display all the details from BOOK_TABLE and ISSUE_DETAILS table based on the common field BOOK_ID, enter the following command at the command prompt:**

```
SELECT BOOK_TABLE.BOOK_NAME, BOOK_TABLE.AUTHOR,

ISSUE_DETAILS.DATE_OF_ISSUE

FROM BOOK_TABLE, ISSUE_DETAILS

WHERE BOOK_TABLE.BOOK_ID = ISSUE_DETAILS.BOOK_ID;
```

Figure 11.1 displays the output of the command.

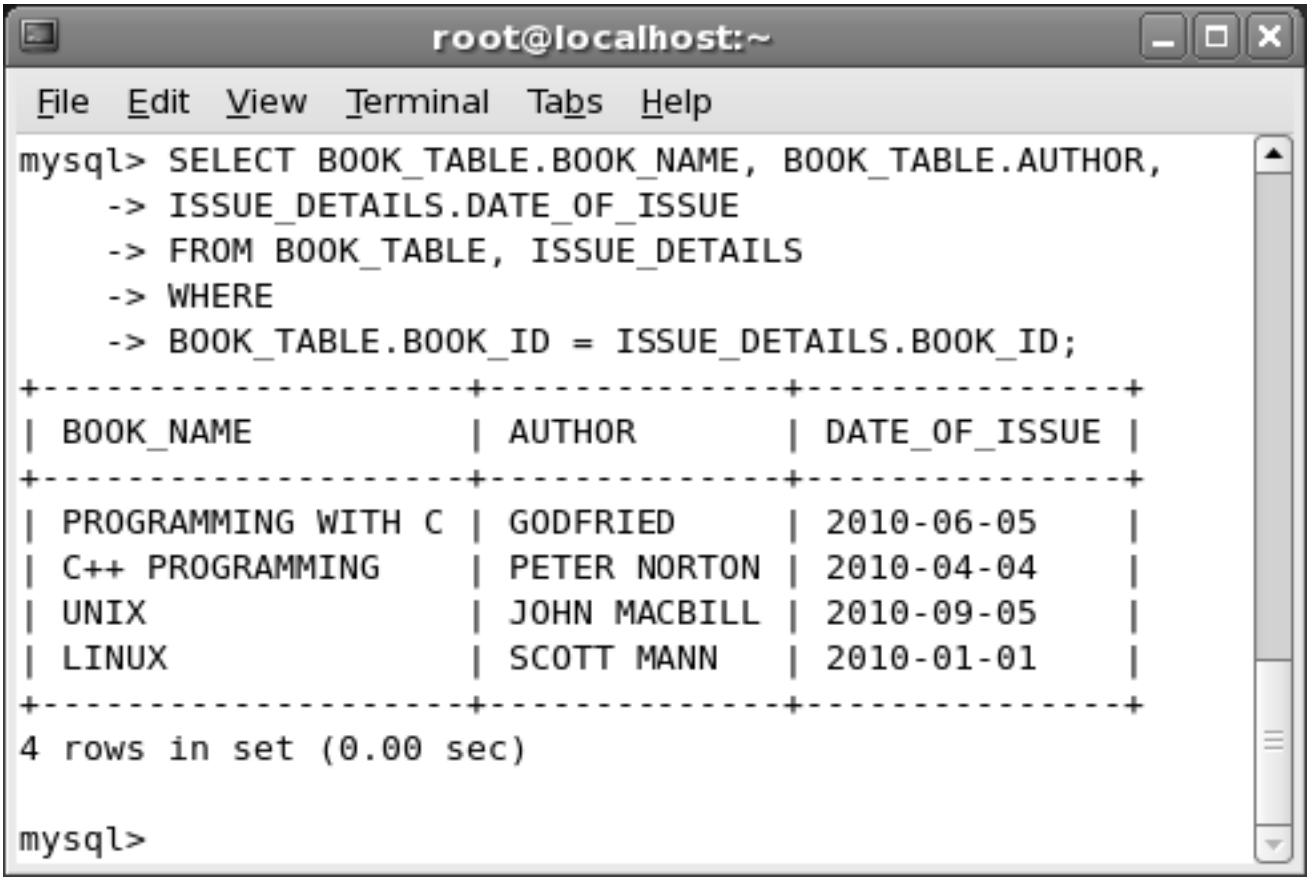


Figure 11.1: Joining Two Tables

2. **In order to display the BOOK_NAME, AUTHOR, and the DATE_OF_ISSUE from the BOOK_TABLE and ISSUE_DETAILS table, using alias names for the tables and BOOK_ID as the common field, enter the following command at the command prompt:**

```
SELECT B.BOOK_NAME, B.AUTHOR, I.DATE_OF_ISSUE

FROM BOOK_TABLE AS B, ISSUE_DETAILS AS I

WHERE B.BOOK_ID = I.BOOK_ID;
```

Figure 11.2 displays the output of the command.

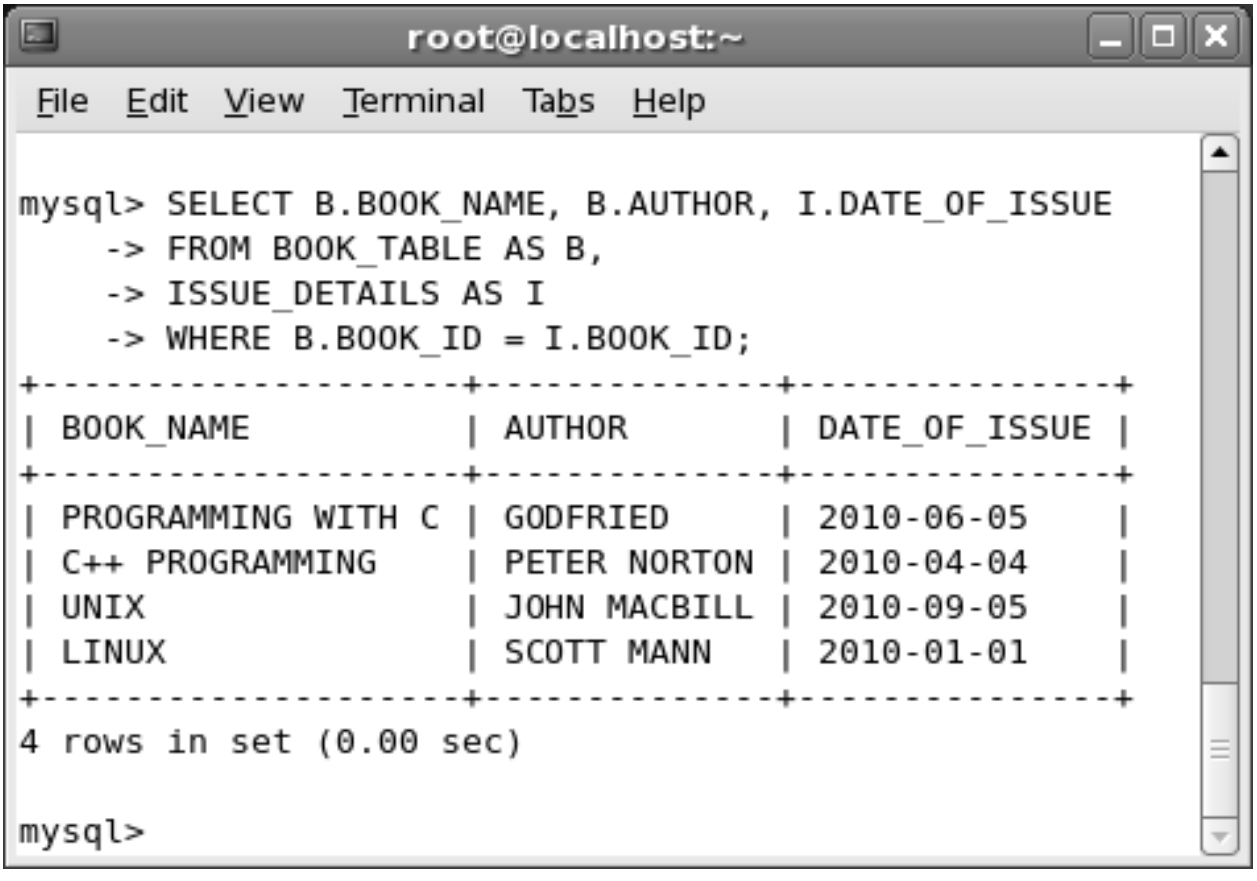


Figure 11.2: Joining Two Tables Using Alias

3. **In order to compare and view the name, location, and phone numbers of books issued to members, you need to use LEFT JOIN and ON clause with USER ID as the common field. To do so, enter the following command at the command prompt:**

```
SELECT NAME, LOCATION, PHONE_NO FROM USER_DETAILS

LEFT JOIN ISSUE_DETAILS ON

USER_DETAILS.USER_ID = ISSUE_DETAILS.USER_ID;
```

Figure 11.3 displays the output of the command.

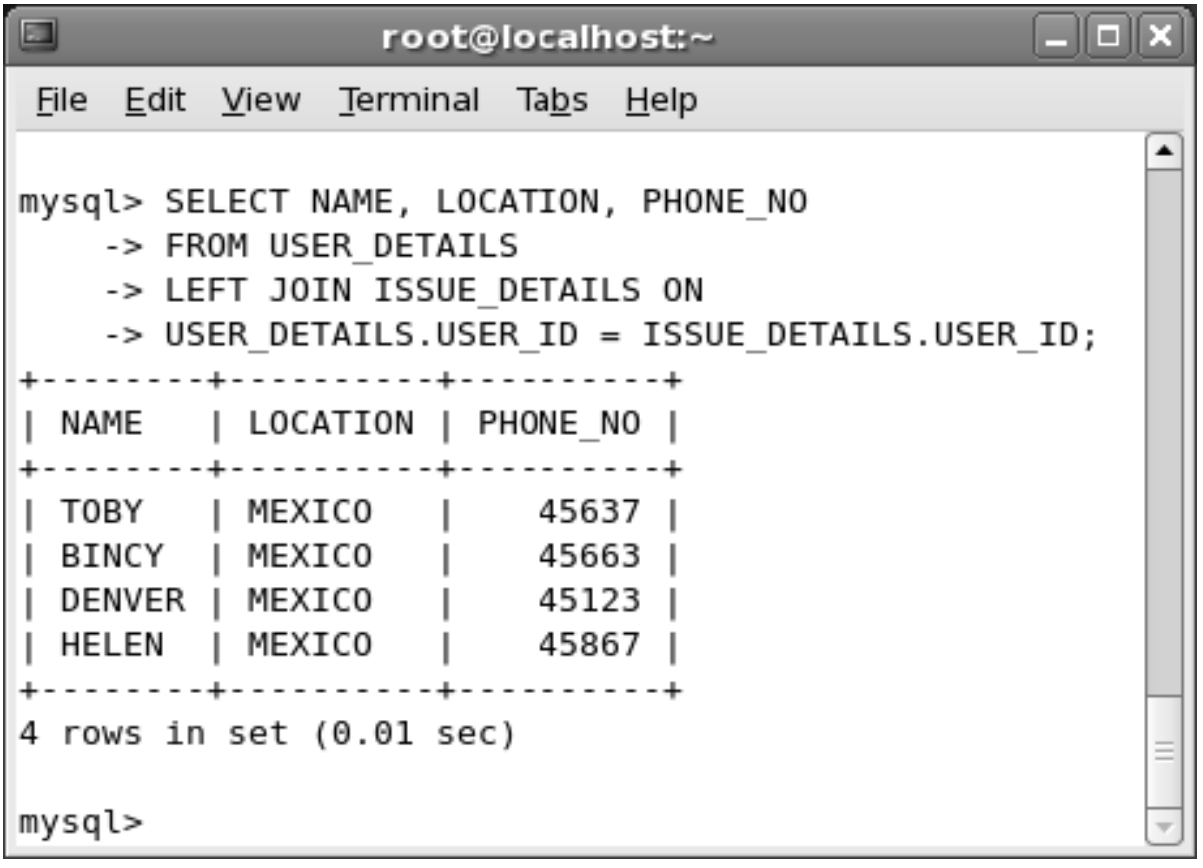


Figure 11.3: Joining Two Tables Using LEFT JOIN with ON Clause

4. **In order to compare and view the name, location, and phone numbers of members who have been issued books, you need to use LEFT JOIN and USING clause with USER_ID as the common field. To do so, enter the following command at the command prompt:**

```
SELECT NAME, LOCATION, PHONE_NO

FROM USER_DETAILS

LEFT JOIN

ISSUE_DETAILS USING(USER_ID);
```

Figure 11.4 displays the output of the command.

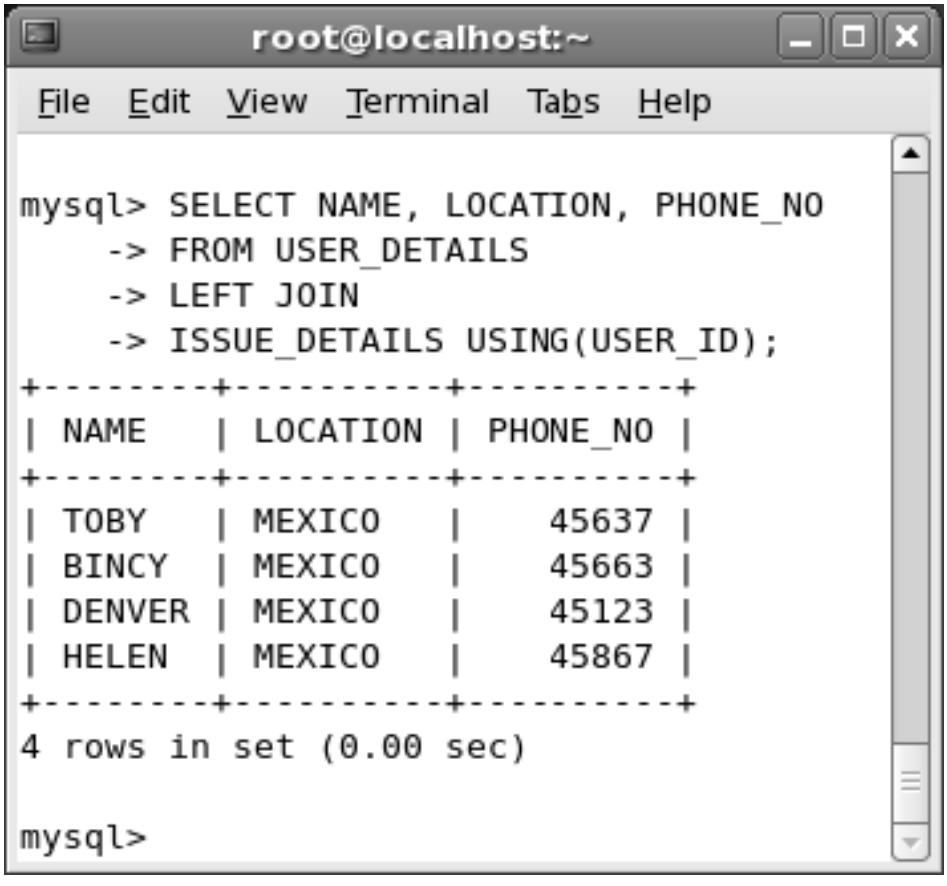


Figure 11.4: Joining Two Tables Using LEFT JOIN with USING Clause

5. **In order to display the name, phone number, and location from USER_DETAILS, ISSUE_DETAILS, and the CATEGORY tables you need to use LEFT JOIN with ON clause. To do so, enter the following command at the command prompt:**

```
SELECT NAME, LOCATION, PHONE_NO FROM USER_DETAILS

LEFT JOIN ISSUE_DETAILS ON

USER_DETAILS.USER_ID = ISSUE_DETAILS.USER_ID

LEFT JOIN CATEGORY ON

ISSUE_DETAILS.USER_ID = CATEGORY.USER_ID;
```

Figure 11.5 displays the output of the command.

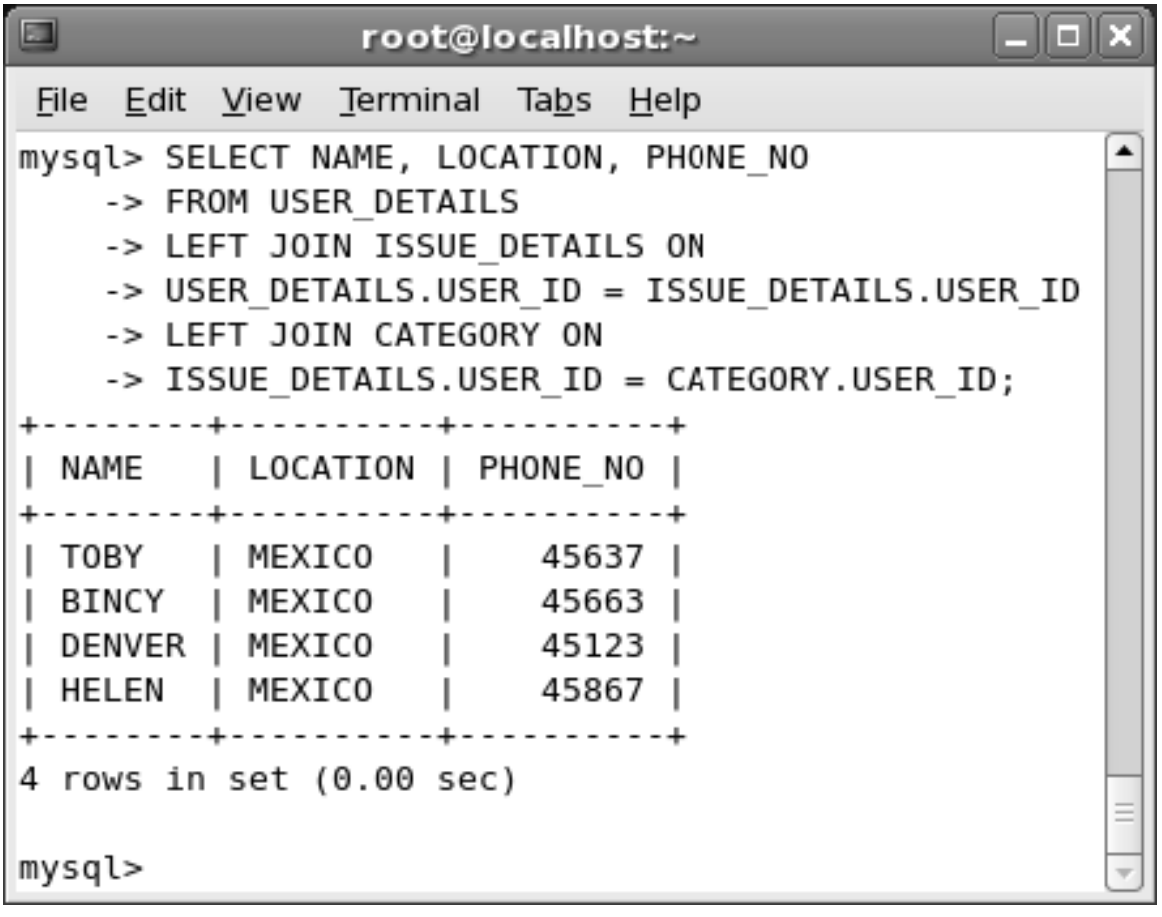


Figure 11.5: Joining Three Tables Using LEFT JOIN with ON Clause

6. **In order to display the book name, author, and return date from BOOK_TABLE and ISSUE_TABLE using equi-join with BOOK_ID as the common field, enter the following command at the command prompt:**

```
SELECT BOOK_TABLE.BOOK_NAME, BOOK_TABLE.AUTHOR,

ISSUE_DETAILS.DATE_OF_RETURN

FROM BOOK_TABLE, ISSUE_DETAILS

WHERE(BOOK_TABLE.BOOK_ID = ISSUE_DETAILS.BOOK_ID);
```

Figure 11.6 displays the output of the command.

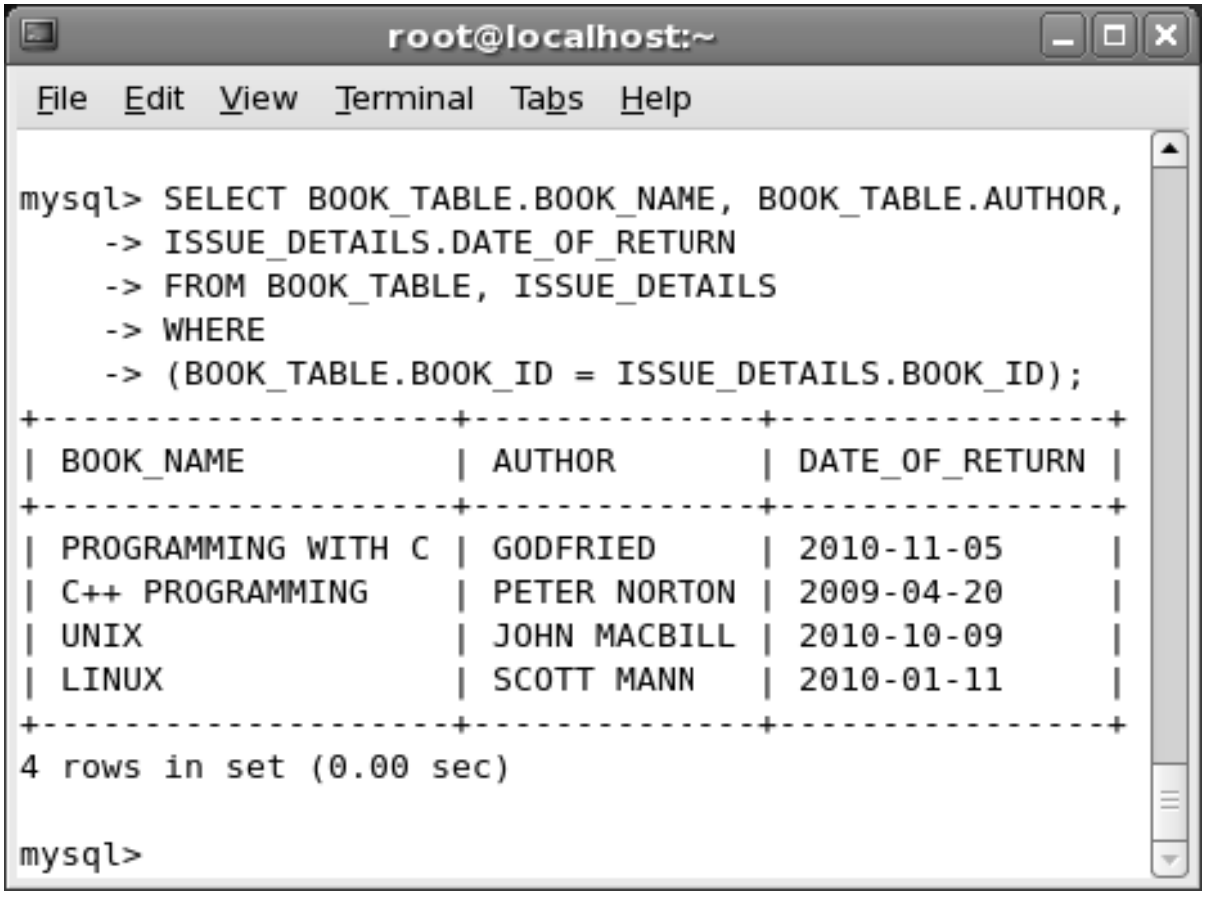


Figure 11.6: Joining Two Tables Using Equi-Join

7. **In order to view the return date for a book, book name, and the author from BOOK_TABLE and ISSUE DETAILS table using INNER JOIN, enter the following command at the command prompt:**

```
SELECT BOOK_TABLE.BOOK_NAME, BOOK_TABLE.AUTHOR,

ISSUE_DETAILS.DATE_OF_RETURN

FROM BOOK_TABLE

INNER JOIN ISSUE_DETAILS ON

BOOK_TABLE.BOOK_ID = ISSUE_DETAILS.BOOK_ID;
```

Figure 11.7 displays the output of the command.

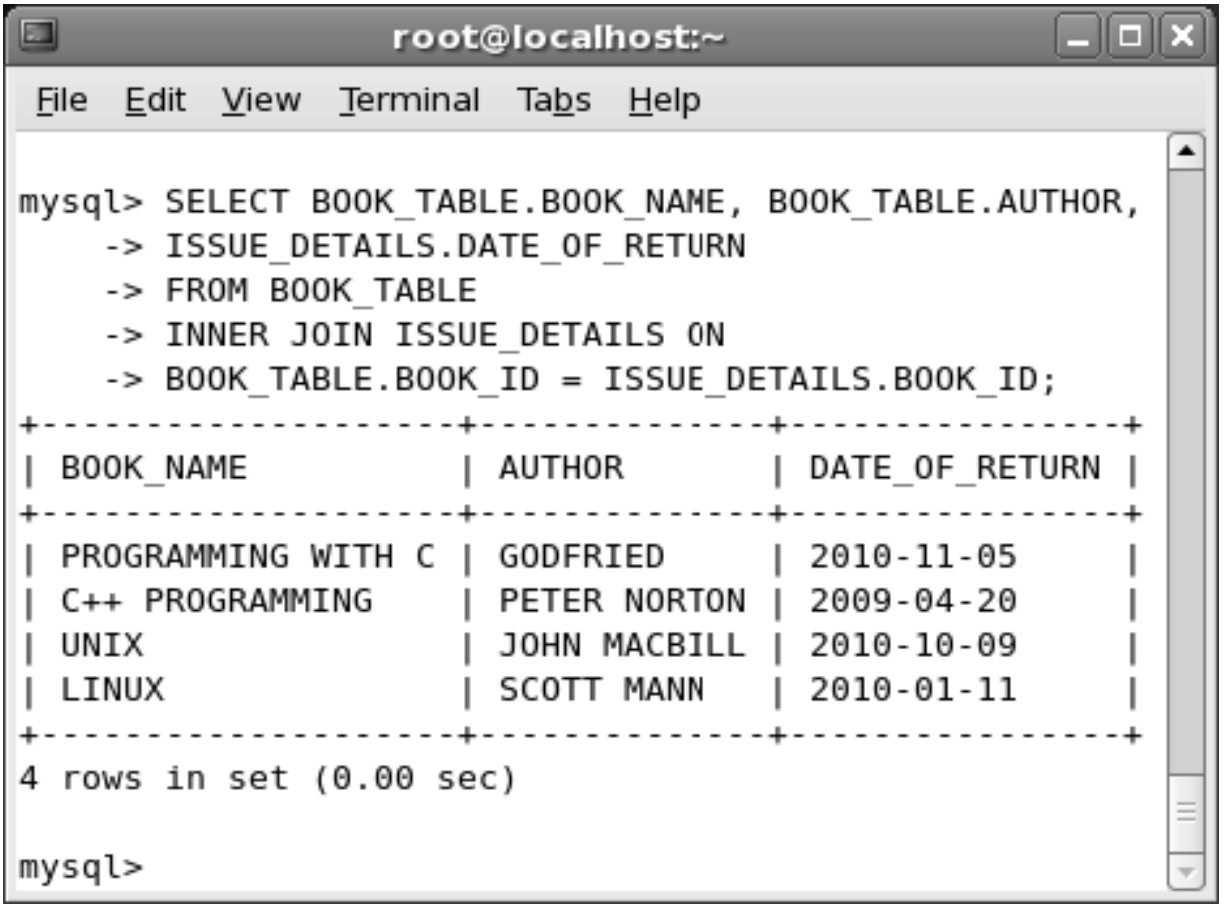


Figure 11.7: Joining Two Tables Using INNER JOIN with ON Clause

8. **In order to display the book name, author, and date of return from the BOOK_TABLE and ISSUE_DETAILS table for Linux books. To display these details using INNER JOIN with the ON clause with BOOK_ID as the common field, enter the following command at the command prompt:**

```
SELECT BOOK_TABLE.BOOK_NAME, BOOK_TABLE.AUTHOR,

ISSUE_DETAILS.DATE_OF_RETURN

FROM BOOK_TABLE INNER JOIN

ISSUE_DETAILS ON

(BOOK_TABLE.BOOK_ID = ISSUE_DETAILS.BOOK_ID)

AND(BOOK_TABLE.BOOK_NAME = 'LINUX');
```

Figure 11.8 displays the output of the command.

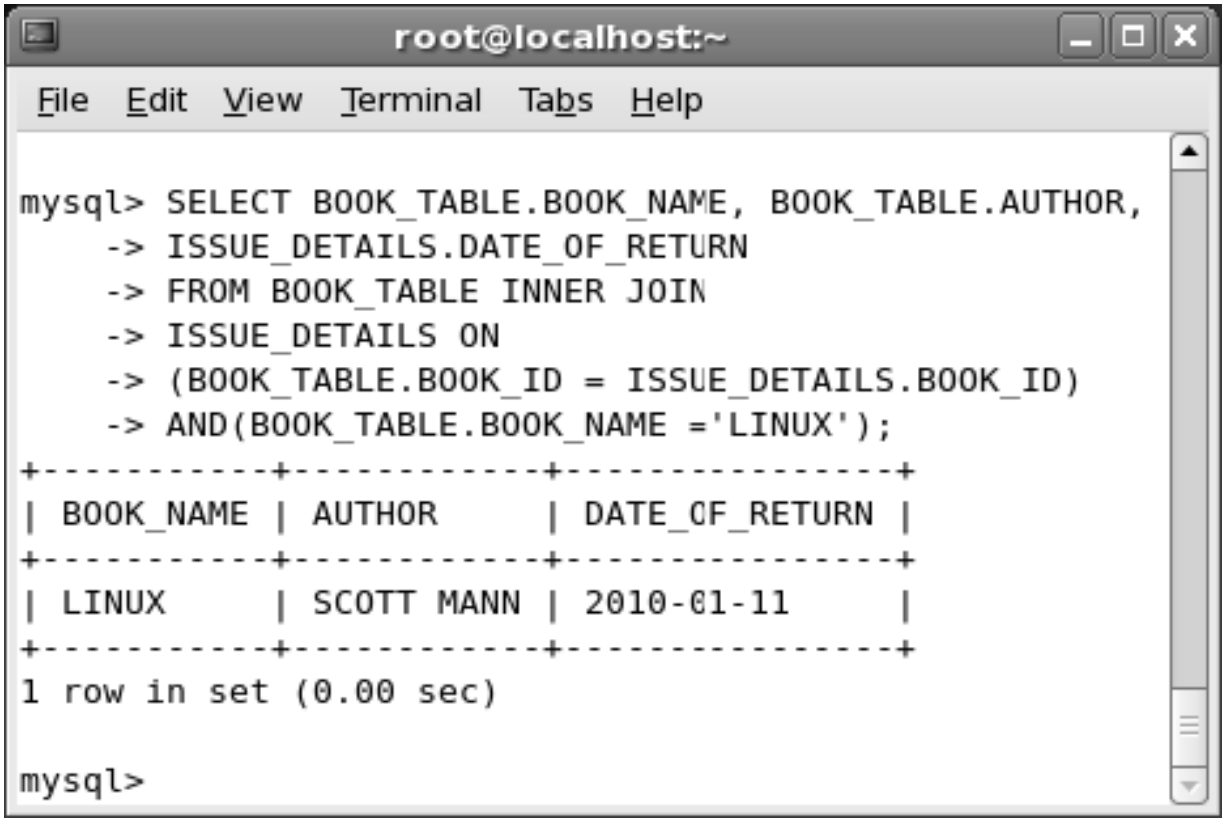


Figure 11.8: Joining Two Tables Using INNER JOIN

9. **In order to view the user details, such as name, location, and phone number of registered members, using LEFT OUTER JOIN and USER_ID as the common field, enter the following command at the command prompt:**

```
SELECT NAME, LOCATION, PHONE_NO FROM USER_DETAILS U

LEFT OUTER JOIN CATEGORY C ON

(U.USER_ID = C.USER_ID);
```

Figure 11.9 displays the output of the command.

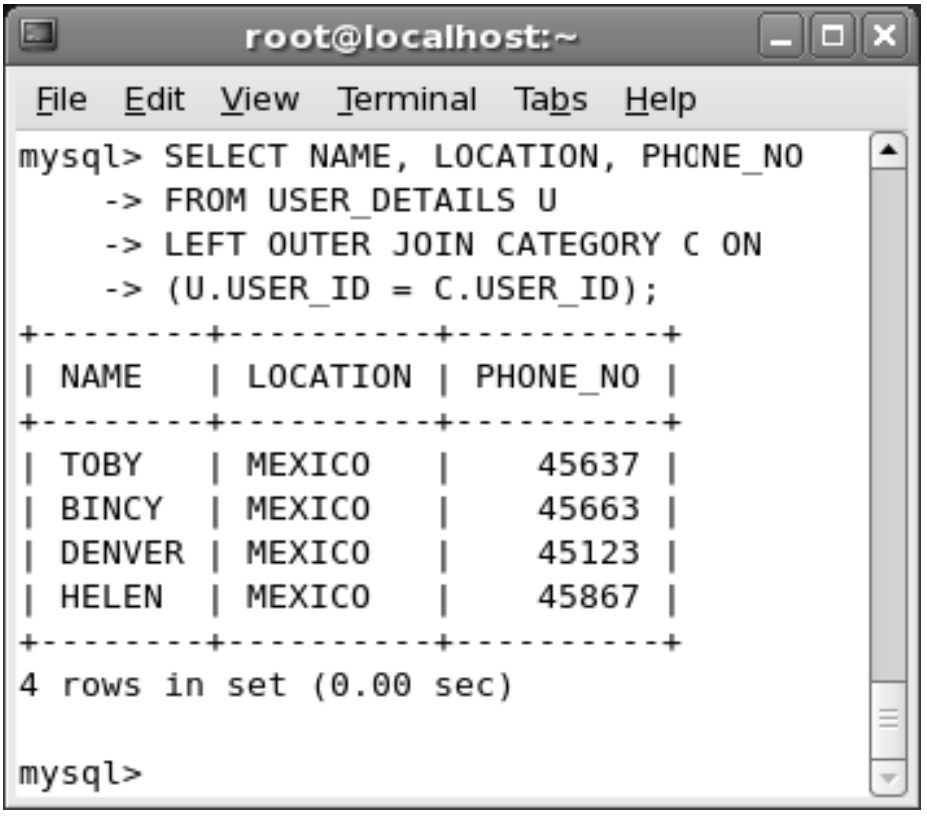


Figure 11.9: Joining Two Tables Using the LEFT OUTER JOIN

Using Subqueries and UNION

A query used within a `SELECT` query is called a subquery. You can use the `UNION` clause to combine the results of multiple `SELECT` statements into a single result set.

1. **To display the book name, author, and edition for all the books that have been issued to the users, you need to create a subquery between the BOOK_TABLE and ISSUE_DETAILS table. To view the records, enter the following command at the command prompt:**

```
SELECT BOOK_NAME, AUTHOR, EDITION

FROM BOOK_TABLE

WHERE BOOK_ID IN(SELECT BOOK_ID FROM ISSUE_DETAILS);
```

Figure 11.10 displays the output of the command.

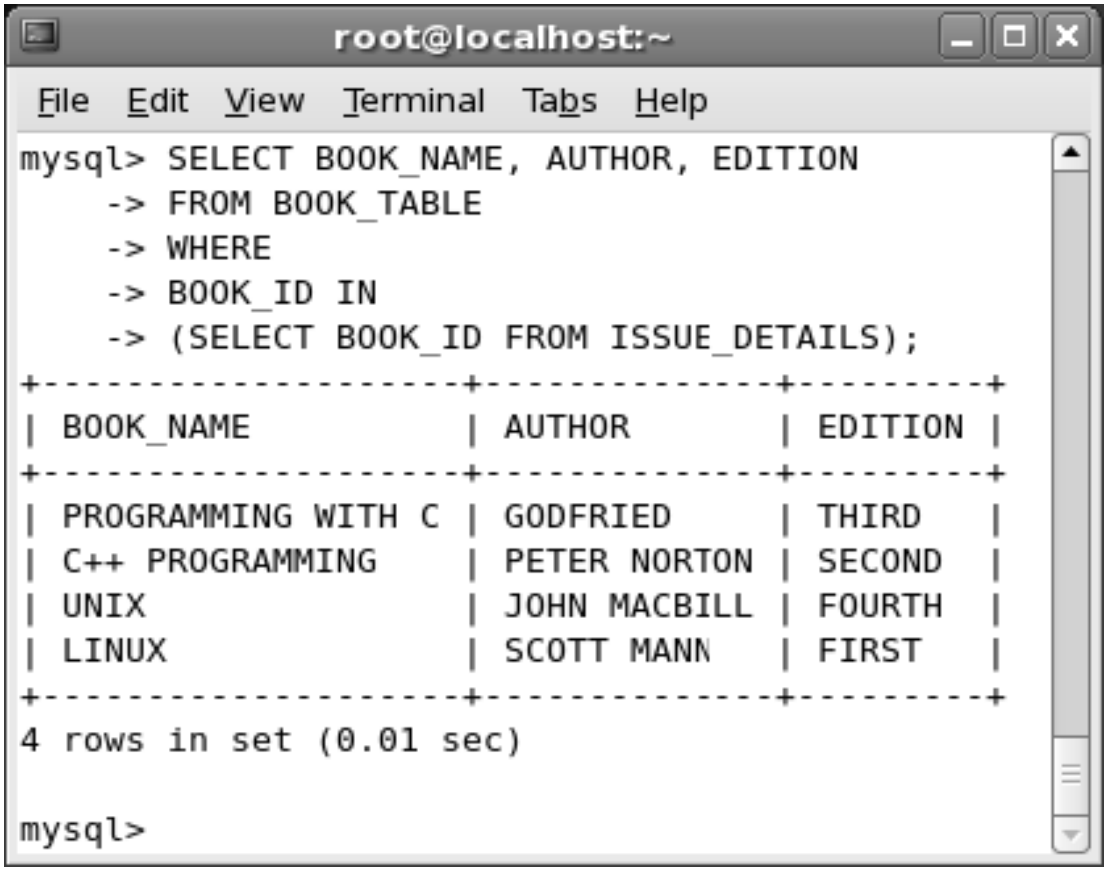


Figure 11.10: Joining Two Tables Using Subquery

2. **In order to combine the result of two SELECT queries and display books from the BOOK_TABLE table using UNION clause and a condition specified using the BOOK_ID, enter the following command at the command prompt:**

```
(SELECT * FROM BOOK_TABLE WHERE BOOK_ID = 101

ORDER BY BOOK_ID LIMIT 3)

UNION

(SELECT * FROM BOOK_TABLE WHERE BOOK_ID = 103

ORDER BY BOOK_ID LIMIT 3)

ORDER BY BOOK_ID;
```

Figure 11.11 displays the output of the command.

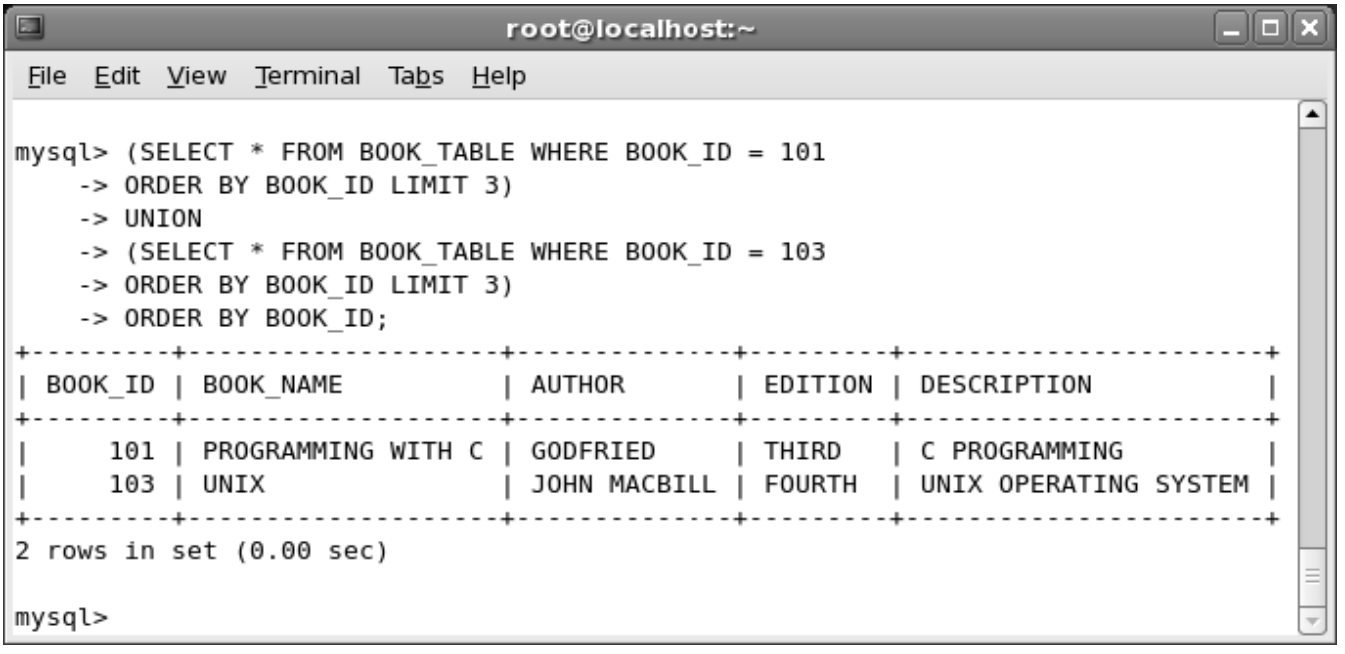


Figure 11.11: Joining Two Tables Using UNION Clause

Do It Yourself

1. Select the `PRODUCT_SALES` database.

2. Join the `Stock_Details` table and `Sales_Details` tables with the help of the common field, `Product_Code`.

   `Stock_Details` table:

Product_Code	Product_Name	Description	Cost Price	Date_Of_Manufacture
101	Bags	Leather Bags	150	2003-10-05
102	Books	Notepads	10	2003-12-05
103	Pens	Fine Tip Pens	20	2003-01-09
104	56	120	9	2003-10-05

   `Sales_Details` table:

Product_Code	Selling Price	Qty	Units Sold	Month	Location
101	200	1500	1450	Jan	California
102	100	5000	3000	Jan	New York
103	20	2900	2000	Jan	Los Angeles
104	56	120	9	Jan	Sydney
101	200	1000	500	Jan	New York

3. Display `Product_Name`, `Description`, `Cost Price`, `Selling Price`, `Qty`, and `Location` field values from `Stock_Details` and `Sales_Details` table using table reference.

4. Display `Product_Name`, `Description`, `Cost Price`, `Selling Price`, and `Qty` field values from `Stock_Details` and `Sales_Details` table using table alias names.

5. Display `Product_Name`, `Description`, `Cost Price`, and `Date_Of_Manufacture` field values from `Stock_Details` and `Sales_Details` table. Display all the records from the table on the left with or without matching records on the right using the `ON` clause.

6. Display `Product_Name`, `Description`, `Cost Price`, and `Date_Of_Manufacture` field values from `Stock_Details` and `Sales_Details` table. Display all the records from the table on the left with or without matching records on the right with `USING` clause.

7. Display `Product_Name`, `Description`, `Cost Price`, `Selling Price`, and `Qty` field values from `Stock_Details` and `Sales_Details` table using Inner Join.

Do It Yourself

8. Display `Product_Name`, `Description`, `Cost Price`, and `Date_Of_Manufacture` field values from `Stock_Details` and `Sales_Details` table using the Left Outer Join.

9. Display `Product_Name`, `Description`, `Cost Price`, and `Date_Of_Manufacture` field values from `Stock_Details` and `Sales_Details` table. Display the records by using a `SELECT` query within another `SELECT` query.

10. Display `Product_Name`, `Description`, `Cost Price`, and `Date_Of_Manufacture` field values from `Stock_Details` and `Sales_Details` table using the `UNION` clause.

Session
12

Using Basic Functions in MySQL - I

Concepts

Objectives

At the end of this session, the student will be able to:

- *Use the Aggregate functions in MySQL.*
- *Use the Mathematical functions in MySQL.*

12.1 Introduction

Functions are independent programs that work on the parameter passed and returns a value depending on the parameter. Parameters are also known as arguments. You can use MySQL functions within a query to operate on single or multiple columns of a table.

In this session, you will learn about the basics of using functions. You will use different types of functions, such as aggregate and mathematical functions. In addition, you will also learn the use of functions in the `GROUP BY` and `HAVING` clause.

12.2 Basics of Using Functions

The functions used in MySQL are similar to any other programming language. In MySQL, you can use functions along with the `SELECT` command in two different ways.

- **Value to be retrieved** - In this form, the function is used with column names. Consider the following query:

```
SELECT E_ID, E_FNAME, LENGTH(E_FNAME) FROM EMP_DETAILS WHERE E_ID=101;
```

  This query returns the `E_ID` and `E_FNAME` columns from the `EMP_DETAILS` table with the length of `E_FNAME`.

- **Part of a WHERE clause** - In this form, the function is used in the `WHERE` clause. The value specified is compared for each row in the table. Consider the following query:

```
SELECT E_ID, D_NAME FROM EMP_SALARY HAVING AVG(BASIC_SAL)<8000;
```

12.3 Using Aggregate Functions in MySQL

Aggregate functions operate on a group of values and return a single value as the final result. Some

Concepts

aggregate functions work with the GROUP BY and HAVING clause. The GROUP BY clause is used to group rows having similar values for a specific column into a single row. Group functions do not accept NULL values. The HAVING clause defines the result set based on a set of calculations. You can use the GROUP BY and HAVING clause with the SELECT statement to retrieve data that satisfies the specified conditions defined in the HAVING clause. The functions discussed in this section illustrate the use of functions in a GROUP BY and HAVING clause.

12.3.1 AVG Function

The AVG function returns the mean value of the argument. The syntax for using this function is:

```
SELECT COLUMN_NAME FROM TABLE_NAME HAVING AVG(expression);
```

For example, to obtain the E_ID and D_NAME of the employee whose average basic salary is less than 8000, enter the following command at the command prompt:

```
SELECT E_ID, D_NAME FROM EMP_SALARY HAVING AVG(BASIC_SAL)<8000;
```

Figure 12.1 displays the output of the command.

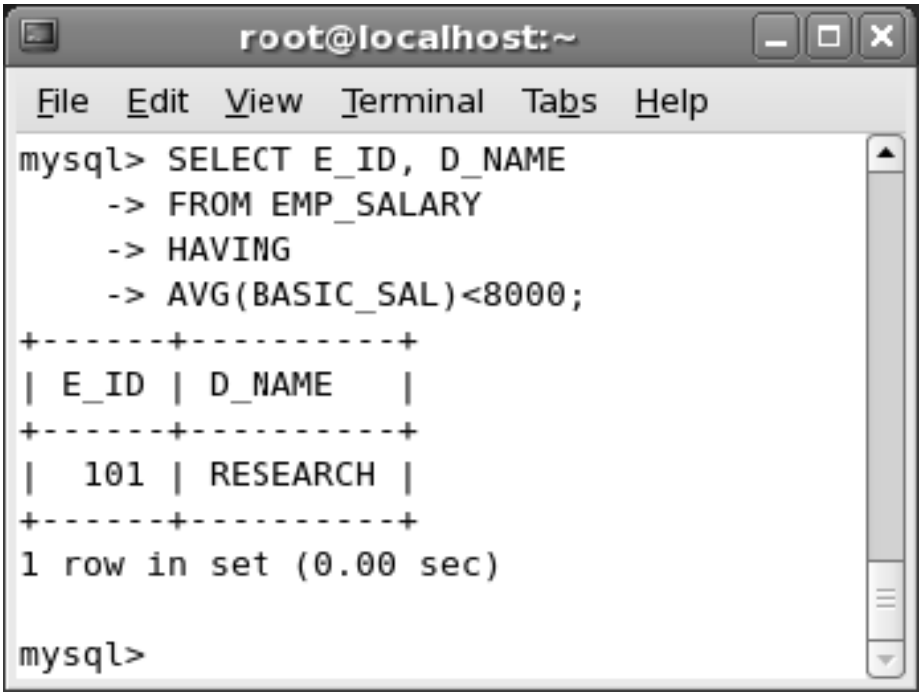


Figure 12.1: AVG Function

12.3.2 BITWISE AND Function

The `BITWISE AND` function works with two arguments that have equal length. `BITWISE AND` converts the arguments into binary format and compares every bit of the two arguments.

The syntax for this function is:

```
SELECT BIT1 & BIT2;
```

where,

`BIT1`, `BIT2` – defines the operands

To find out the `BITWISE AND` for `29` and `15`, enter the following command at the command prompt:

```
SELECT 29 & 15;
```

Figure 12.2 displays the output of the command.

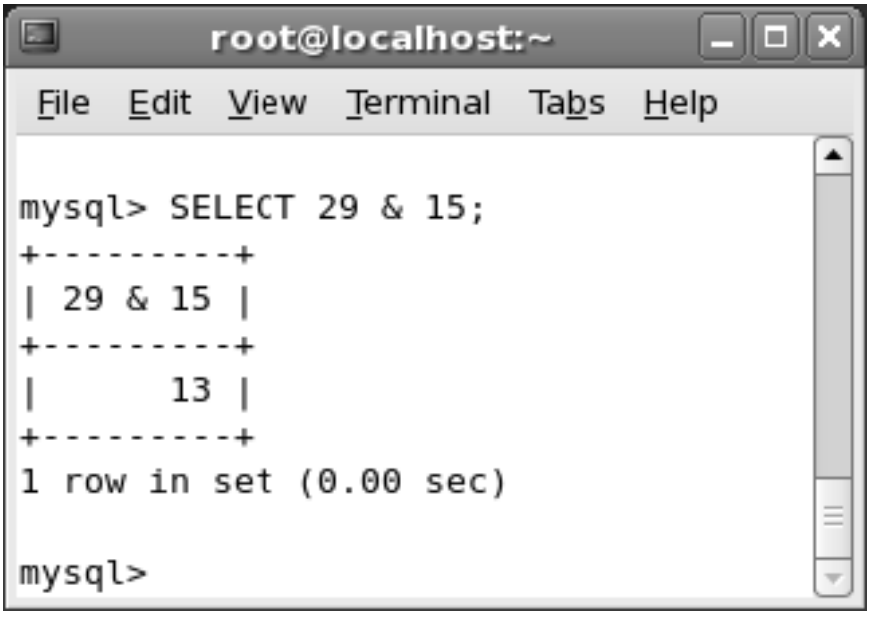


Figure 12.2: BIT AND Function

12.3.3 COUNT Function

The `COUNT` function returns the total number of `non-NULL` values of the expression specified as the argument.

Concepts

The syntax for obtaining the count of any expression is:

```
SELECT COUNT(expression) FROM TABLE_NAME;
```

For example, to obtain the count of the `E_ID` column from the `EMP_SALARY` table, enter the following command at the command prompt:

```
SELECT COUNT(E_ID) FROM EMP_SALARY;
```

Figure 12.3 displays the output of the command.

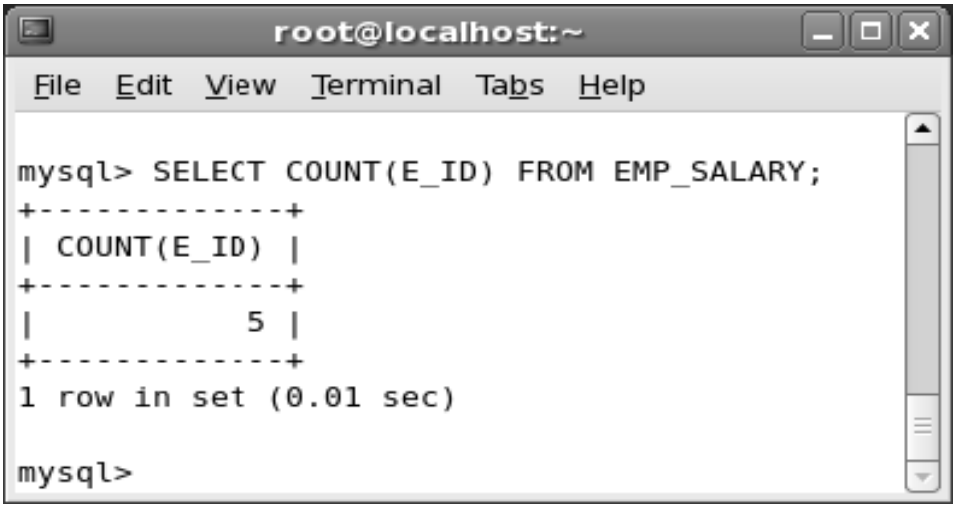


Figure 12.3: COUNT Function

12.3.4 COUNT(DISTINCT) Function

The `COUNT(DISTINCT)` function returns the number of unique values in a column. You can retrieve the number of different combinations of unique values in two or more columns of the same table. If there are no matching values in any of the columns, then the `COUNT(DISTINCT)` function returns 0. The syntax for using this function is:

```
SELECT COUNT(DISTINCT expression) FROM TABLE_NAME;
```

For example, to obtain the number of employees who have different basic salary, enter the following command at the command prompt:

```
SELECT COUNT(DISTINCT BASIC_SAL) FROM EMP_SALARY;
```

Figure 12.4 displays the output of the command.

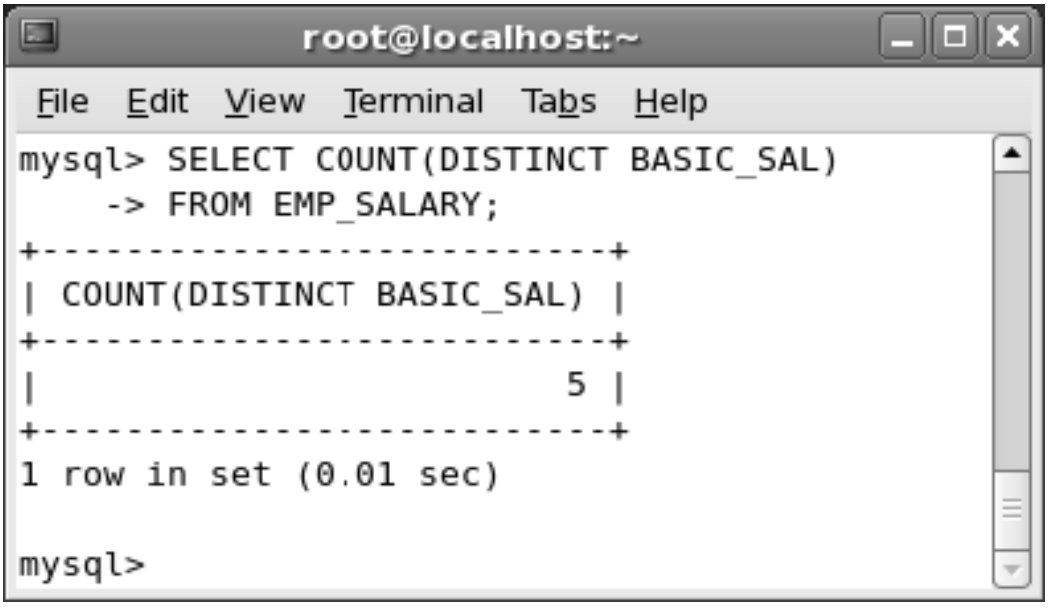


Figure 12.4: COUNT (DISTINCT) Function

12.3.5 GROUP_CONCAT Function

The `GROUP_CONCAT` function joins the unique values of the argument and returns them as a string. This function returns `NULL` if the argument has no `non-NULL` values. To eliminate duplicate values, you can use the `DISTINCT` clause. The syntax for using this function is:

```
SELECT GROUP_CONCAT(expression) FROM TABLE_NAME;
```

For example, to display all the `E_ID` from the `EMP_SALARY` table that have `RESEARCH` as the `D_NAME`, enter the following command at the command prompt:

```
SELECT D_NAME, GROUP_CONCAT(E_ID) FROM EMP_SALARY;
```

Figure 12.5 displays the output of the command.

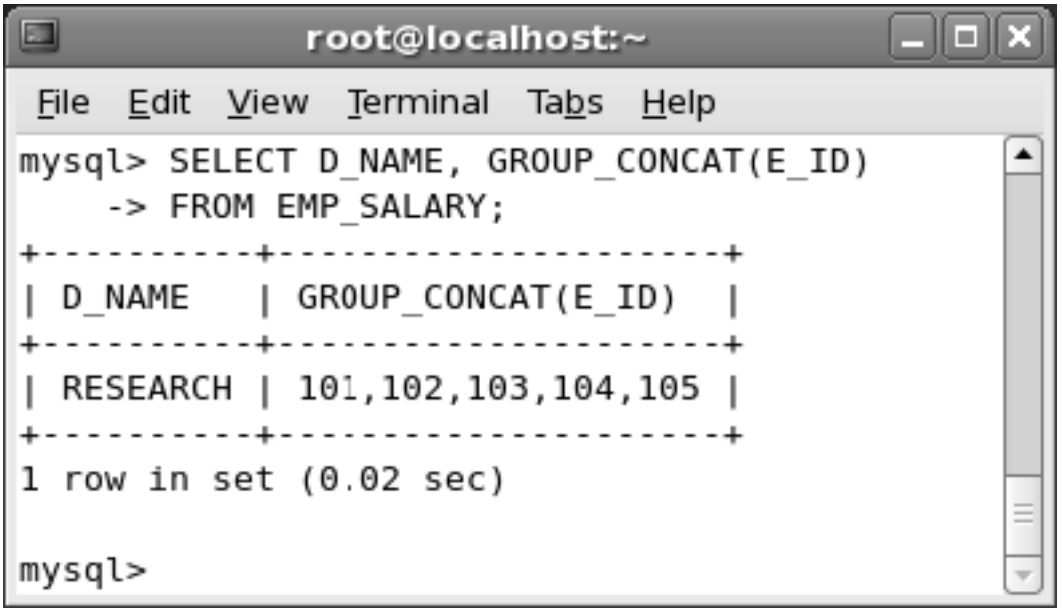


Figure 12.5: GROUP_CONCAT Function

12.3.6 MAX Function

The MAX function returns the greatest value of the expression. The syntax for obtaining the greatest value is:

```
SELECT COLUMN_NAME FROM TABLE_NAME HAVING MAX(expression);
```

For example, display the maximum gross salary earned by any of the employee.

```
SELECT MAX(GROSS_SAL) FROM EMP_SALARY;
```

Figure 12.6 displays the output of the command.

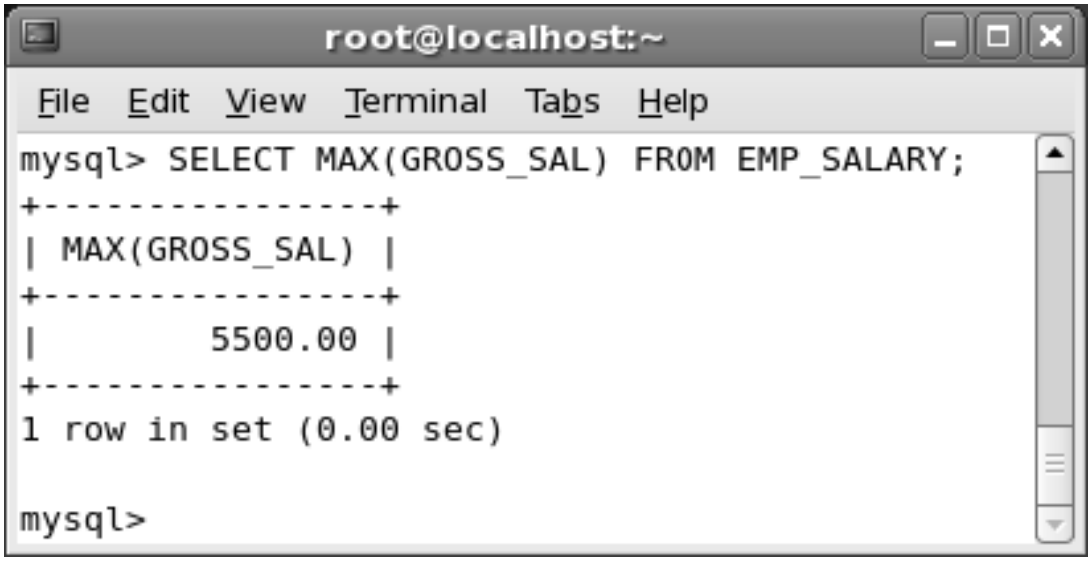


Figure 12.6: MAX Function

12.3.7 STD Function

The `STD` function returns the standard deviation of the argument. Standard deviation specifies the variation of a population from its mean value. The output of this function is `NULL` if there are no rows satisfying the given query. The syntax to use this function is:

```
SELECT STD(expression)FROM TABLE_NAME;
```

For example, to obtain the standard deviation of the basic salary of all of the employees, enter the following command at the command prompt:

```
SELECT STD(BASIC_SAL) FROM EMP_SALARY;
```

Figure 12.7 displays the output of the command.

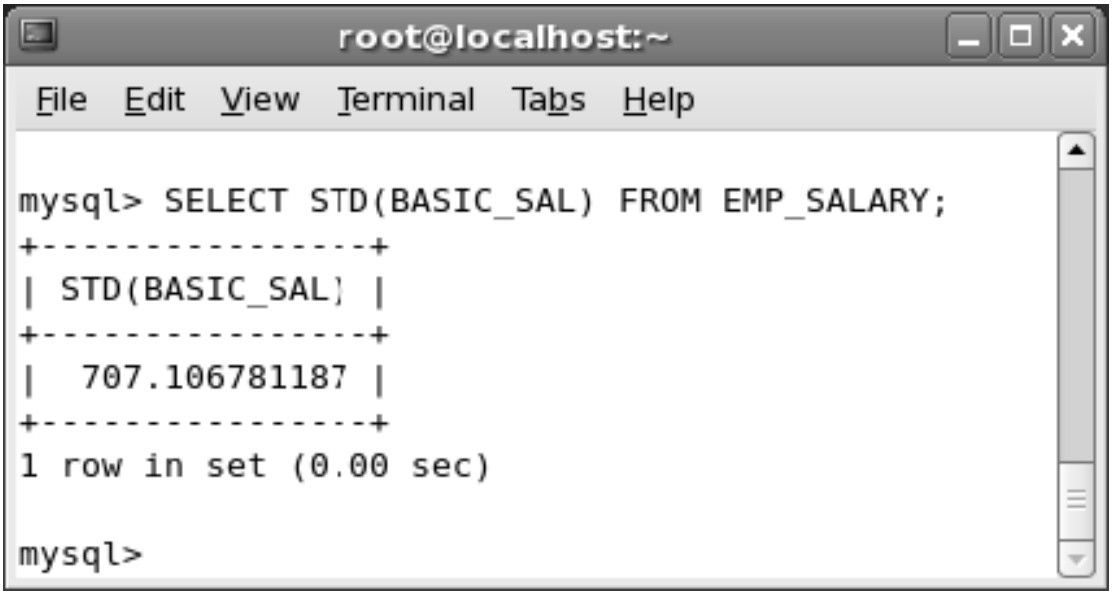


Figure 12.7: STD Function

12.3.8 SUM Function

The `SUM` function adds the values specified in the expression. The syntax for obtaining the sum is:

```
SELECT SUM(expression) FROM TABLE_NAME;
```

For example, to obtain the sum of `HRA` of all the employees, enter the following command at the command prompt:

```
SELECT SUM(HRA) FROM EMP_SALARY;
```

Figure 12.8 displays the output of the command.

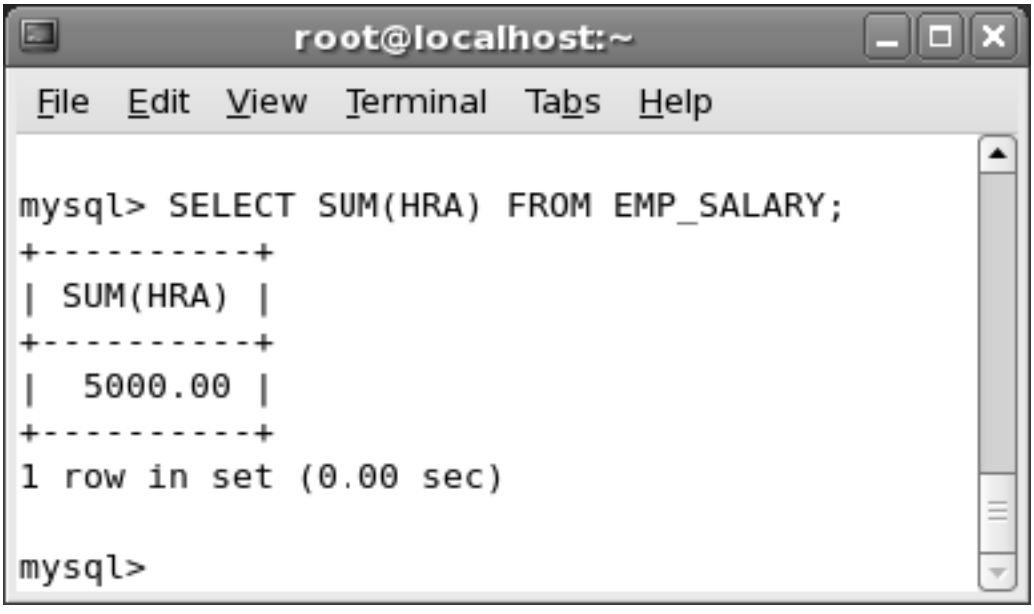


Figure 12.8: SUM Function

12.3.9 VARIANCE Function

The `VARIANCE` function returns the variance of the values of the argument. The syntax for obtaining the variance is:

```
SELECT VARIANCE(expression) FROM TABLE_NAME;
```

To obtain the variance of gross salary of all the employees, enter the following command at the command prompt:

```
SELECT VARIANCE(GROSS_SAL) FROM EMP_SALARY;
```

Figure 12.9 displays the output of the command.

Concepts

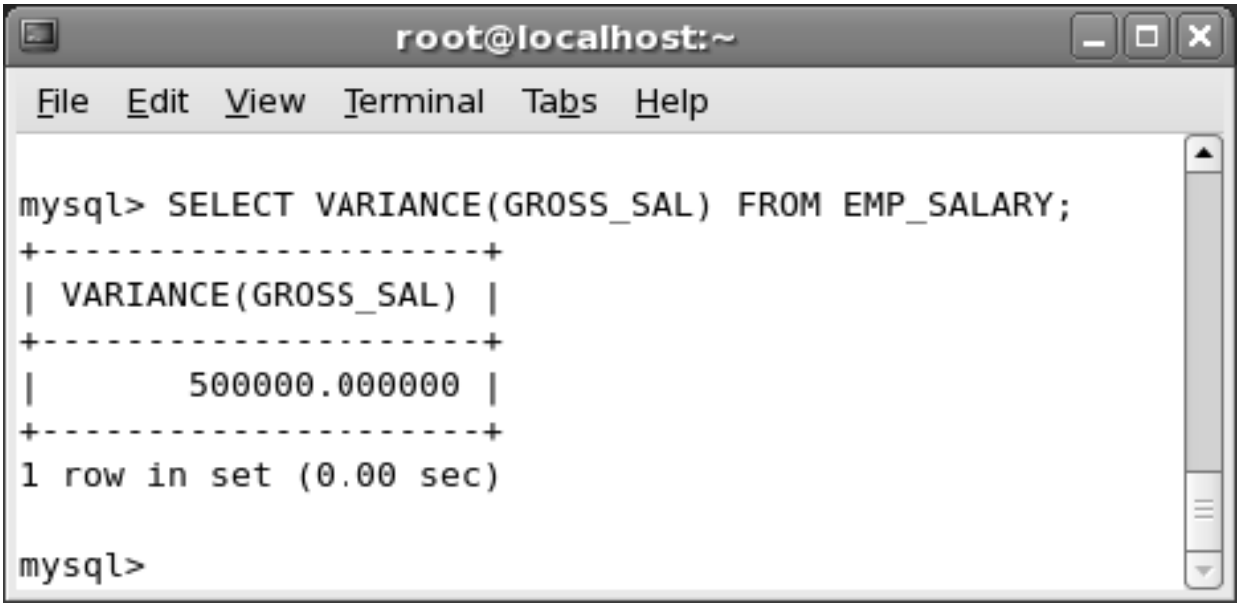


Figure 12.9: VARIANCE Function

Table 12.1 describes additional AGGREGATE functions supported by MySQL:

Name	Description	Example
BITWISE OR	The BITWISE OR function works with two arguments of equal length. BITWISE OR compares every bit of the two arguments. This function returns 0 only if both the bits being compared equal to 0, else it returns 1. The syntax for this function is: SELECT BIT1 | BIT2; where, BIT1, BIT2 – specify the values for the operands	To calculate the BITWISE OR for 30 and 12, enter the following command at the command prompt: SELECT 30 | 12; The output of the command is: 30
BITWISE XOR	The BITWISE XOR function works with two arguments of equal length. BITWISE XOR compares every bit of the two arguments. This function returns 0 in the output if there are no matching rows in the comparison. The syntax for this function is:	To find out the BITWISE XOR for 5 and 2, enter the following command at the command prompt: SELECT 5 ^ 2;

Name	Description	Example
BITWISE XOR	SELECT BIT1 ^ BIT2; where, BIT1, BIT2 – specifies a value for the operands	The output of the command is: 7
MIN	The MIN function returns the smallest value of the expression. The syntax for using this function is: SELECT COLUMN_NAME FROM TABLE_NAME HAVING MIN(expression);	For example, display the minimum gross salary earned by any of the employee. SELECT MIN(GROSS_SAL) FROM EMP_SALARY; The output of this function is: 3500.00
STDDEV	The STDDEV function also returns the standard deviation of the values of the argument. The output of this function is NULL if there are no rows satisfying the given query. The syntax for obtaining the standard deviation is: SELECT STDDEV(expression) FROM TABLE_NAME;	For example, to obtain the standard deviation of the basic salary of all the employees, enter the following command at the command prompt: SELECT STDDEV(BASIC_SAL) FROM EMP_SALARY; The output of the function is: 707.106781
STDDEV_POP	The STDDEV_POP function calculates the population standard deviation and returns the square root of the variance. The syntax for this function is: SELECT STDDEV_POP(expression) FROM TABLE_NAME;	For example, to obtain the standard deviation of the gross salary of all the employees, enter the following command at the command prompt: SELECT STDDEV_POP(GROSS_SAL) FROM EMP_SALARY; The output of the function is: 707.106781
STDDEV_SAMP	The STDDEV_SAMP function returns the sample standard deviation of the argument. Sample standard deviation is	For example, to obtain the sample standard deviation of the gross salary of all the employees, enter the following

Name	Description	Example

STDDEV_SAMP	applicable only to a sample, that is, a part of an entire population. The syntax for using this function is:
	
	`SELECT STDDEV_SAMP(expression) FROM TABLE_NAME;`
		command at the command prompt:
		
		`SELECT STDDEV_SAMP(GROSS_SAL) FROM EMP_SALARY;`
		
		The output of the function is:
		
		`790.569415`

VAR_POP	The VAR_POP function returns the standard variance of the argument. This function accepts the rows as the argument and defines the number of rows as the denominator. The syntax for using this function is:
	
	`SELECT VAR_POP(expression) FROM TABLE_NAME;`
		For example, to obtain the variance of gross salary of all the employees, enter the following command at the command prompt:
		
		`SELECT VAR_POP(GROSS_SAL) FROM EMP_SALARY;`
		
		The output of this function is:
		
		`500000.000000`

VAR_SAMP	The VAR_SAMP function returns the sample variance of the argument. The denominator returned by this function is the number of rows minus one. The syntax for using this function is:
	
	`SELECT VAR_SAMP(expression) FROM TABLE_NAME;`
		For example, to obtain the variance of gross salary of all the employees, enter the following command at the command prompt:
		
		`SELECT VAR_SAMP(GROSS_SAL) FROM EMP_SALARY;`
		
		The output of this function is:
		
		`625000.000000`

Table 12.1: Additional AGGREGATE Functions in MySQL

12.4 Using Mathematical Functions in MySQL

MySQL provides mathematical functions that process the data provided as an argument and return a numerical value. All the mathematical functions will return a NULL value if an error occurs.

12.4.1 ABS Function

The ABS function returns the absolute value of the argument. In mathematics, the absolute value of a

number is its distance from the origin and is therefore, never negative. The syntax for using this function is:

```
SELECT ABS(expression);
```

For example, to calculate the absolute value of `-32`, enter the following command at the command prompt:

```
SELECT ABS(-32);
```

Figure 12.10 displays the output of the command.

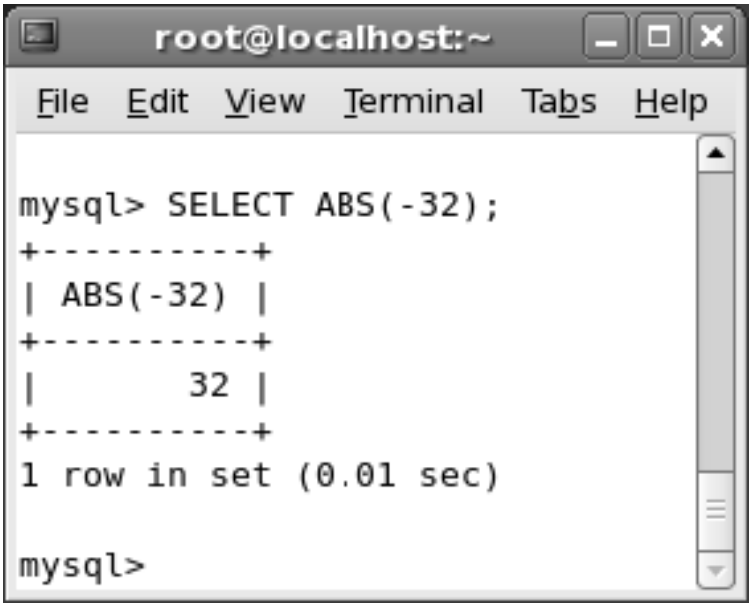


Figure 12.10: ABS Function

12.4.2 ACOS Function

The `ACOS` function returns the arc cosine of the specified argument. The arc cosine function returns the angle, expressed in radians, of the argument whose cosine is specified. The syntax to obtain the arc cosine of a function is:

```
SELECT ACOS(expression);
```

For example, to calculate the arc cosine value of `1`, enter the following command at the command prompt:

```
SELECT ACOS(-1);
```

Figure 12.11 displays the output of the command.

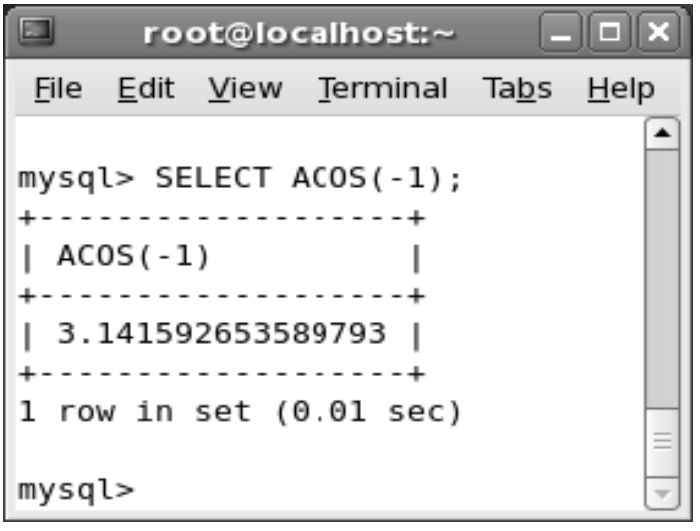


Figure 12.11: ACOS Function

12.4.3 CEILING Function

The `CEILING` function returns the smallest integer value greater than the argument. The syntax for using this function is:

```
SELECT CEILING(X);
```

To use the `CEILING` function on `5.56`, enter the following command at the command prompt:

```
SELECT CEILING(5.56);
```

Figure 12.12 displays the output of the command.

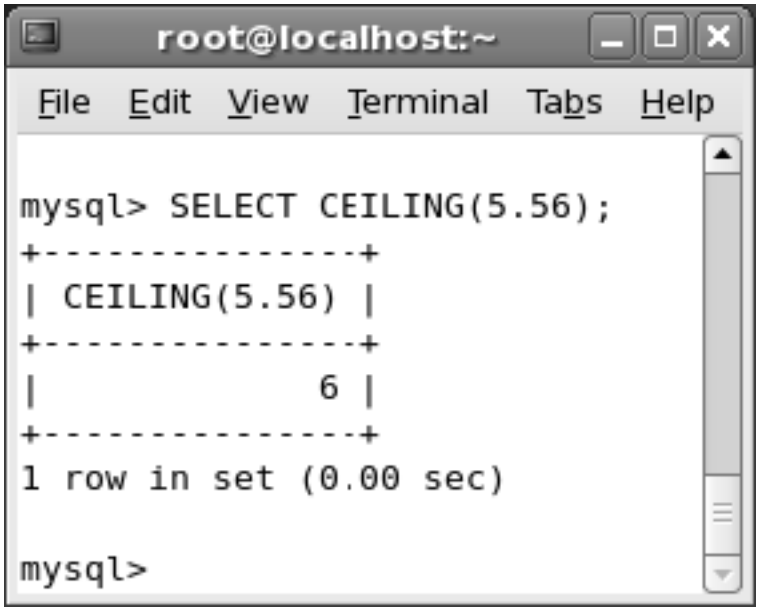


Figure 12.12: CEILING Function

12.4.4 CONV Function

The `CONV` function changes a number from one base to another. The output is either generated as a string or as `NULL` if any argument is `NULL`. The minimum and maximum base values are `2` and `36` respectively. You can perform case-insensitive comparisons with this function. The syntax to use the `CONV` function is:

```
SELECT CONV(N, from_base, to_base);
```

For example, to convert `-7` from base `10` to base `-2`, enter the following command at the command prompt:

```
SELECT CONV(-7, 10, -2);
```

Figure 12.13 displays the output of the command.

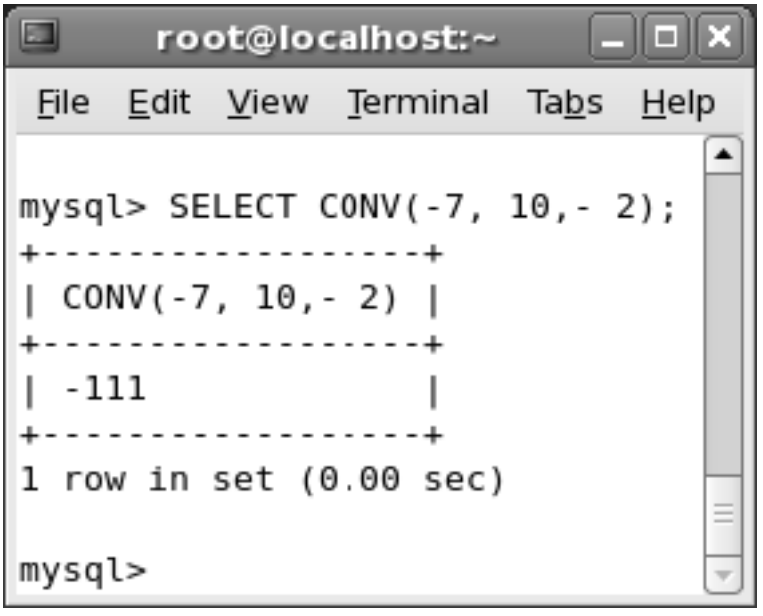


Figure 12.13: CONV Function

12.4.5 COS Function

The `COS` function returns the cosine of an argument. The argument specified in the function should be in radians. The syntax for obtaining the cosine of an argument X is:

```
SELECT COS(X);
```

For example, to obtain the cosine value of `pi/6`, enter the following command at the command prompt:

```
SELECT COS(PI()/6);
```

Figure 12.14 displays the output of the command.

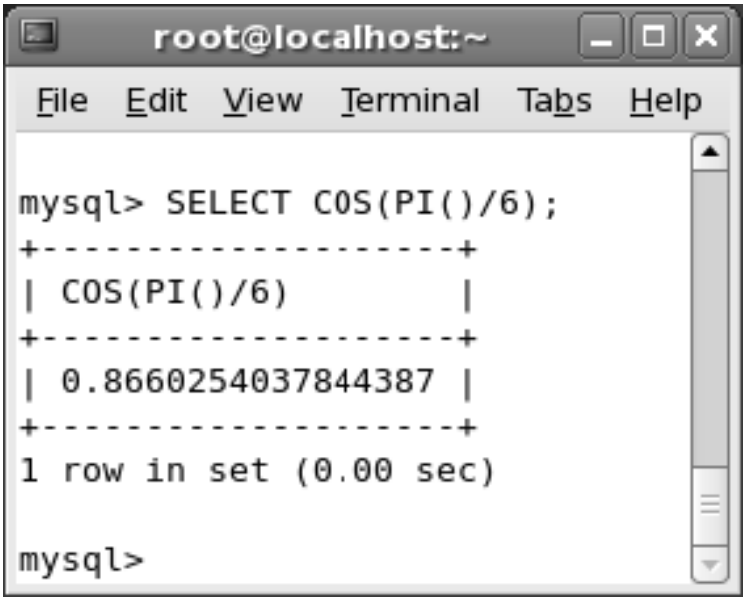


Figure 12.14: COS Function

12.4.6 CRC Function

The `CRC` function calculates a cyclic redundancy check value. It returns a 32-bit unsigned value. It returns a `NULL` value if a `NULL` argument is specified. The argument must be a string. The function converts the input to a string and computes the cyclic redundancy check value. The syntax to calculate the cyclic redundancy value of an argument X is:

```
SELECT CRC32(X);
```

For example, to calculate the cyclic redundancy check value for the string `JOHN`, enter the following command at the command prompt:

```
SELECT CRC32('JOHN');
```

Figure 12.15 displays the output of the command.

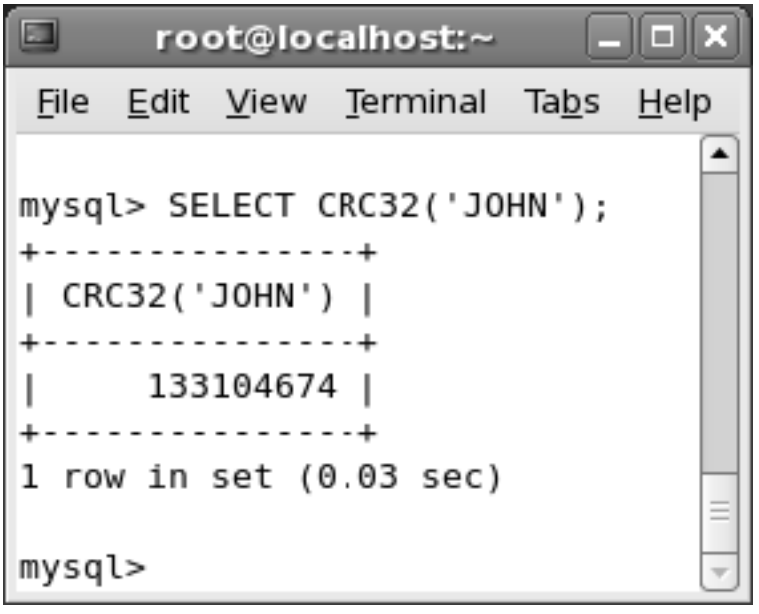


Figure 12.15: CRC Function

12.4.7 COT Function

The `COT` function returns the cotangent of the argument. Cotangent of an argument is the ratio of the adjacent side to the opposite side. The syntax to use this function is:

```
SELECT COT(expression);
```

To calculate the cotangent of `45`, enter the following command at the command prompt:

```
SELECT COT(45);
```

Figure 12.16 displays the output of the command.

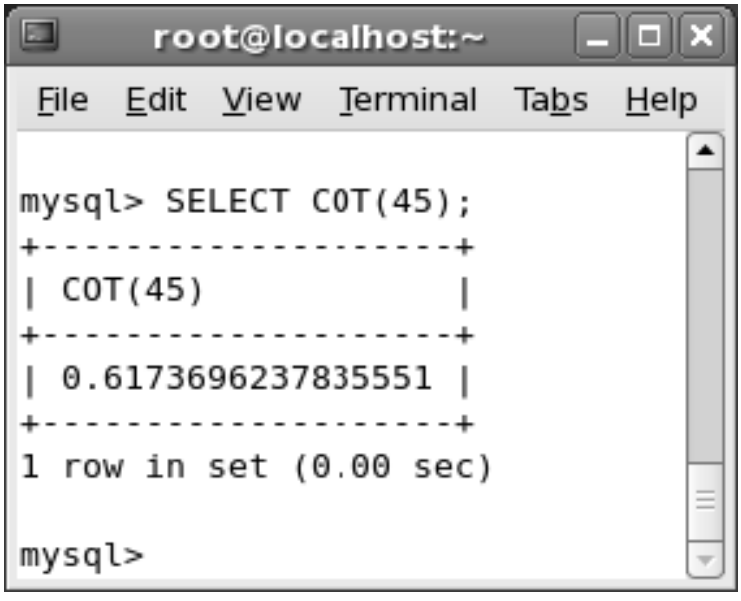


Figure 12.16: COT Function

12.4.8 DEGREES Function

The `DEGREES` function converts the specified argument from radians to degrees. The syntax to use this function is:

```
SELECT DEGREES(expression);
```

For example, to obtain the value of `pi` in degrees, enter the following command at the command prompt:

```
SELECT DEGREES(PI());
```

Figure 12.17 displays the output of the command.

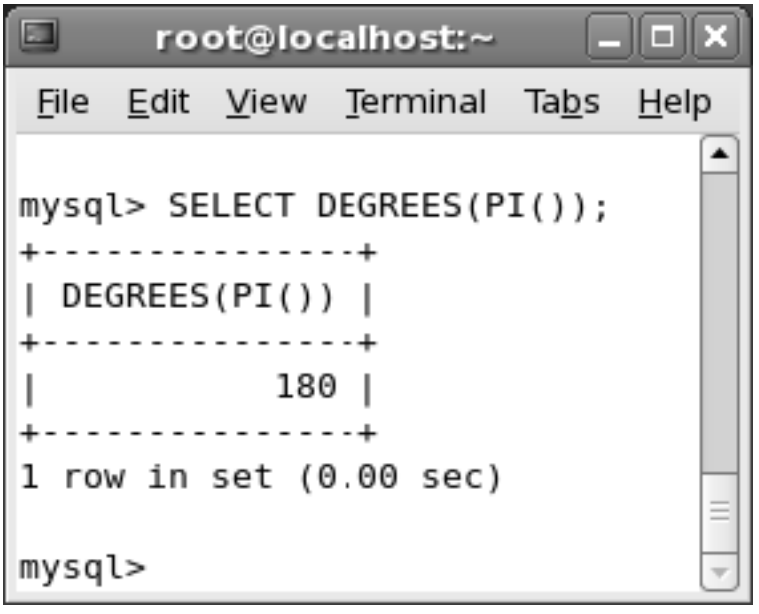


Figure 12.17: DEGREES Function

12.4.9 EXP Function

The `EXP` function returns the exponential value of the given argument. The syntax for obtaining the exponential value of an argument is:

```
SELECT EXP(X);
```

To calculate the result of `e` raised to `5`, enter the following command at the command prompt:

```
SELECT EXP(5);
```

Figure 12.18 displays the output of the command.

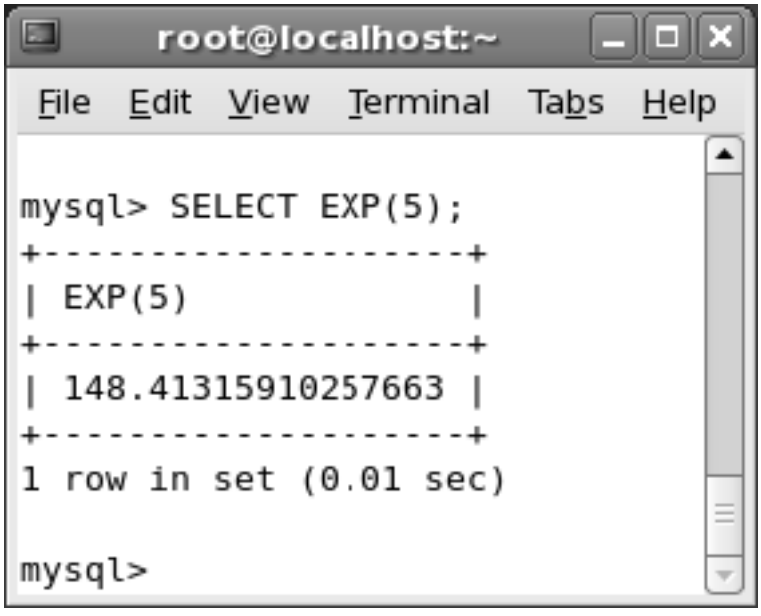


Figure 12.18: EXP Function

12.4.10 FLOOR Function

The `FLOOR` function returns the largest value less than or equal to the given numeric value. The syntax for obtaining the largest value not greater than the argument specified is:

```
SELECT FLOOR(X);
```

To apply the `FLOOR` function on `4.55`, enter the following command at the command prompt:

```
SELECT FLOOR(4.55);
```

Figure 12.19 displays the output of the command.

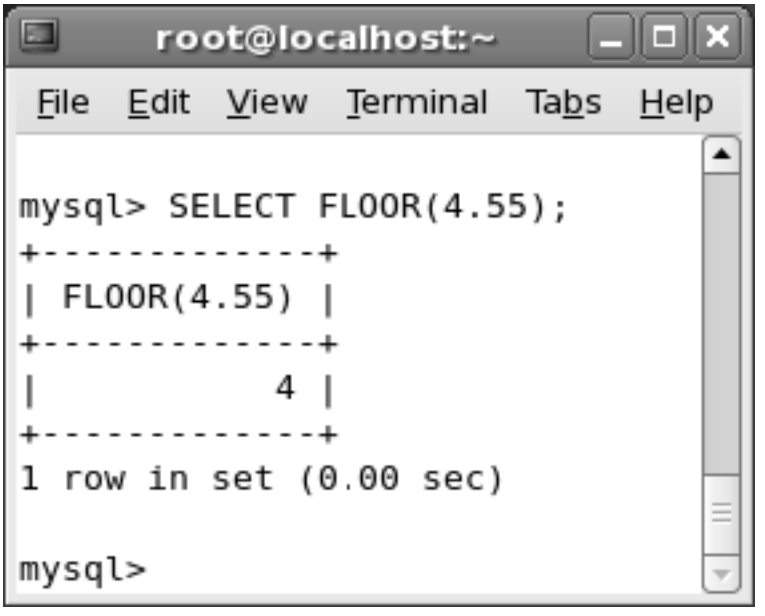


Figure 12.19: FLOOR Function

12.4.11 LOG Function

The `LOG` function returns the natural logarithm of the argument. The syntax to calculate the logarithm value for a single argument is:

```
SELECT LOG(X);
```

For example, to calculate the logarithm of `100`, enter the following command at the command prompt:

```
SELECT LOG(100);
```

Figure 12.20 displays the output of the command.

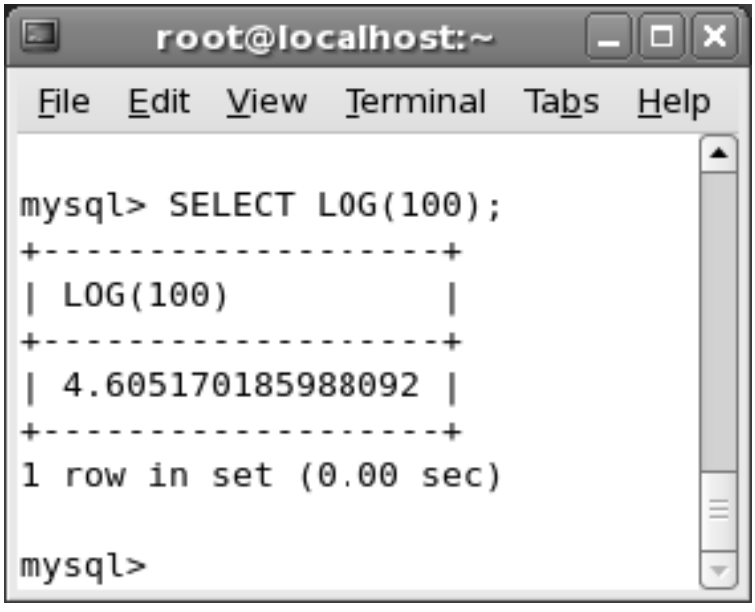


Figure 12.20: LOG Function

The `LOG` function allows you to specify two values as arguments. Consider an example, where two arguments `X` and `Y` are specified in the `LOG` function. This function returns the logarithm of `X` for an arbitrary base `Y`. The syntax for specifying two arguments in `LOG` function is:

```
LOG(X,Y);
```

Note that `LOG(X,Y)` is equivalent to `Log(Y)/Log(X)`.

12.4.12 MOD Function

The `MOD` function operates on two numeric arguments. The output of the function is the remainder after dividing the number by the divisor. The syntax for obtaining the remainder is:

```
SELECT MOD(X,Y);
```

This function returns the remainder of `X` divided by `Y`.

For example, to obtain the remainder of `59` divided by `3`, enter the following command at the command prompt:

```
SELECT MOD(59,3);
```

Figure 12.21 displays the output of the command.

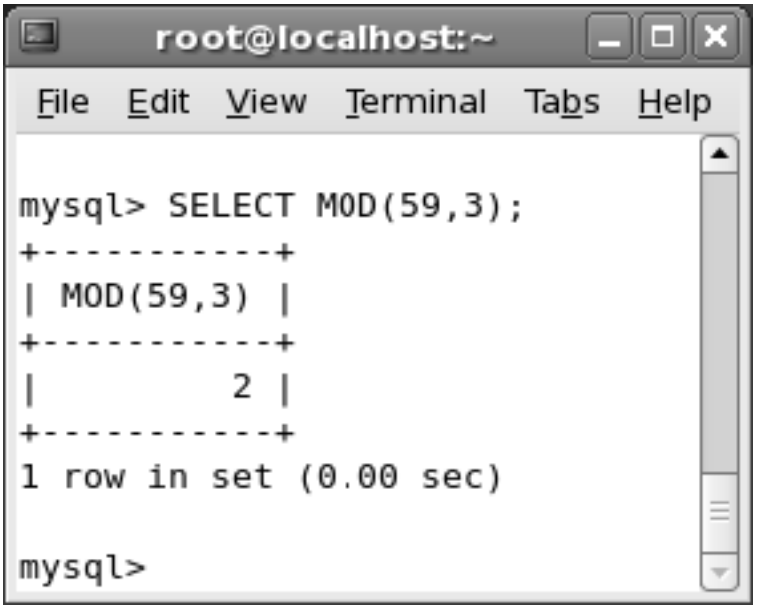


Figure 12.21: MOD Function

12.4.13 OCT Function

The `OCT` function returns the octal value of a number. Octal is a numbering system that has a base of 8 and each digit in the number is represented using only the numerals 0-7. The syntax for using this function is:

```
SELECT OCT(expression);
```

For example, to obtain the octal value of `12`, enter the following command at the command prompt:

```
SELECT OCT(12);
```

Figure 12.22 displays the output of the command.

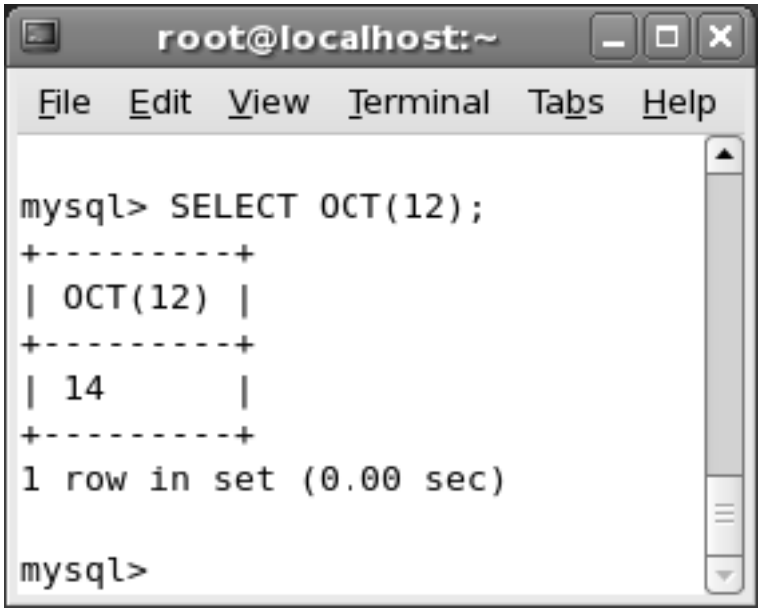


Figure 12.22: OCT Function

12.4.14 PI Function

The `PI` function returns the value of the mathematical constant, `pi`. The syntax for using this function is:

```
SELECT PI();
```

To view the output of this function, enter the following command at the command prompt:

```
SELECT PI();
```

Figure 12.23 displays the output of the command.

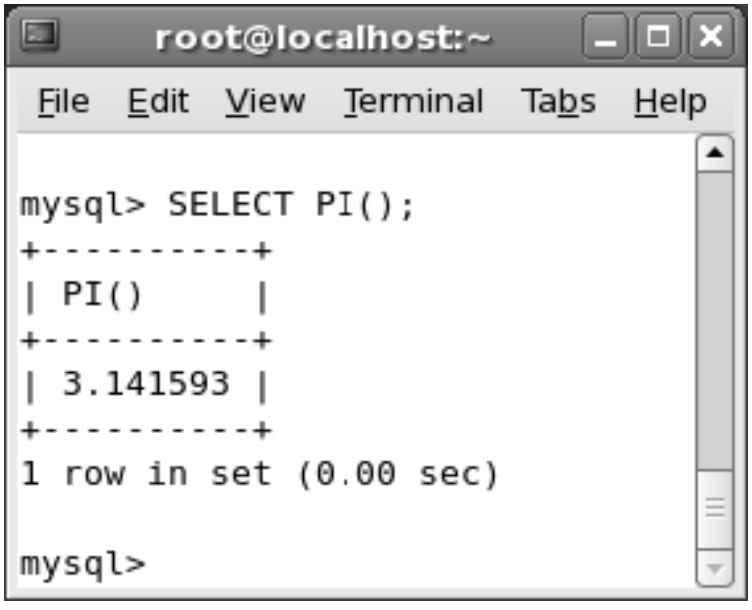


Figure 12.23: PI Function

12.4.15 POWER Function

The `POWER` function allows you to enter two arguments. This function raises the first argument to the power of the second one. The syntax for using this function is:

```
SELECT POWER(X,Y);
```

This command returns the value of `X` raised to the power of `Y`.

For example, to calculate the result of `24` raised to `6`, enter the following command at the command prompt:

```
SELECT POWER(24,6);
```

Figure 12.24 displays the output of the command.

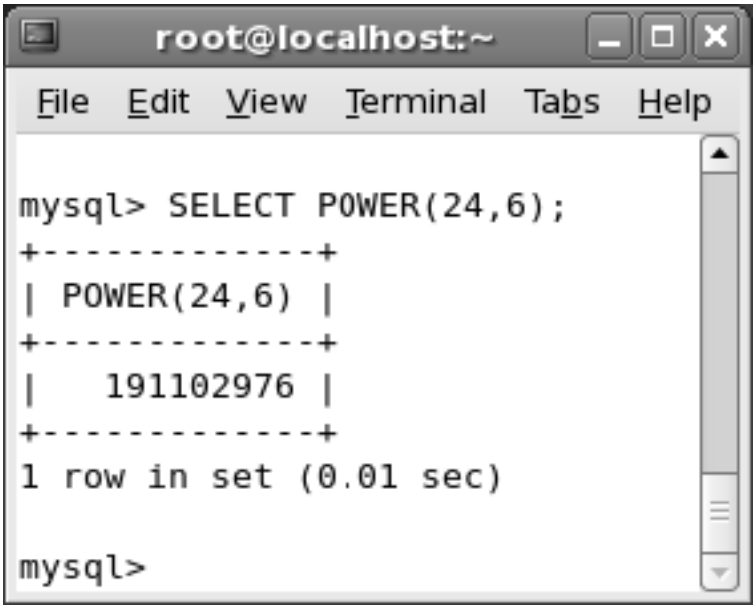


Figure 12.24: POWER Function

The alternative syntax for the `POWER` function is:

```
SELECT POW(X,Y);
```

12.4.16 RADIANS Function

The `RADIANS` function converts the specified argument from degrees to radians. The syntax to use this function is:

```
SELECT RADIANS(expression);
```

For example, to calculate the value of `45` in radians, enter the following command at the command prompt:

```
SELECT RADIANS(45);
```

Figure 12.25 displays the output of the command.

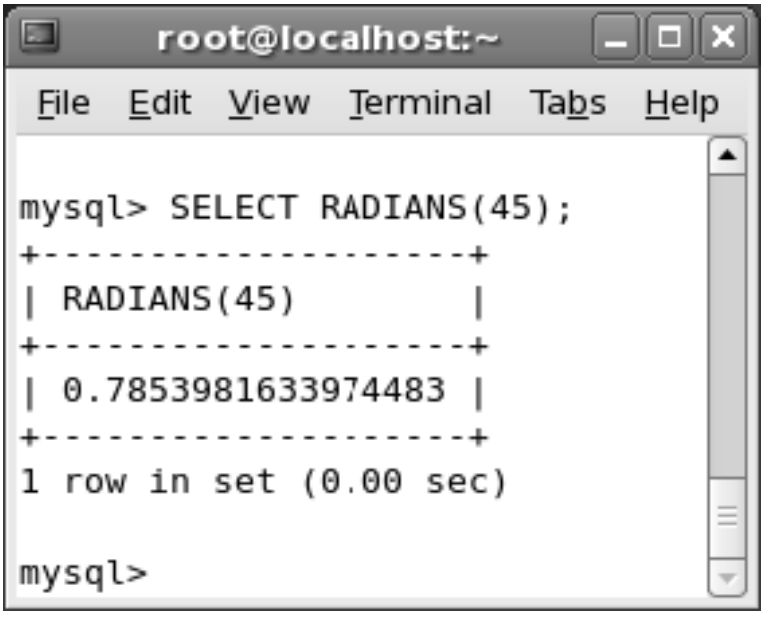


Figure 12.25: RADIANS Function

12.4.17 SIGN Function

The `SIGN` function specifies if the specified argument is negative or non-negative. It returns a `-1`, `0`, or `1` depending upon the argument, whether it is positive, zero, or negative respectively. The syntax for obtaining the sign of an argument is:

```
SELECT SIGN(X);
```

For example, to calculate the sign of `-900`, enter the following command at the command prompt:

```
SELECT SIGN(-900);
```

Figure 12.26 displays the output of the command.

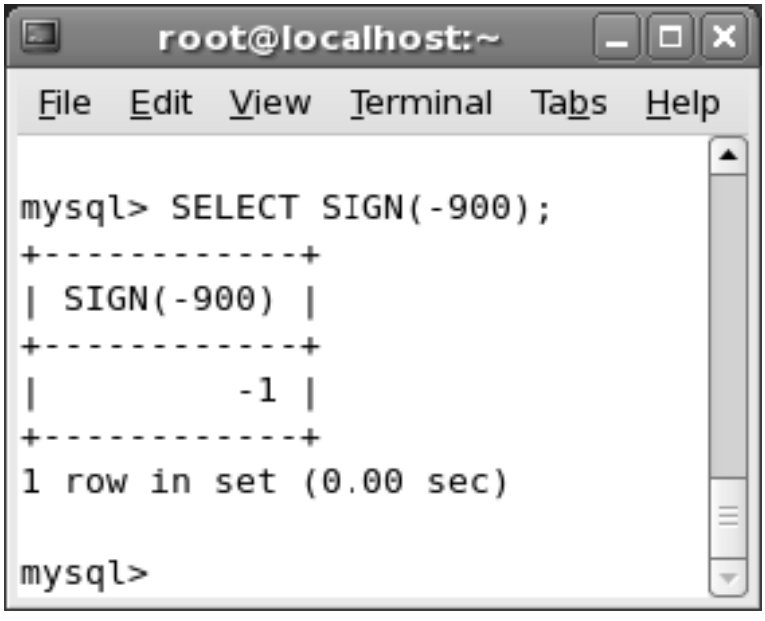


Figure 12.26: SIGN Function

12.4.18 SQRT Function

The `SQRT` function returns the square root of the argument. The function returns the square root as a non-negative integer. The syntax to calculate the square root of an argument is:

```
SELECT SQRT(X);
```

To calculate the square root of `2500`, enter the following command at the command prompt:

```
SELECT SQRT(2500);
```

Figure 12.27 displays the output of the command.

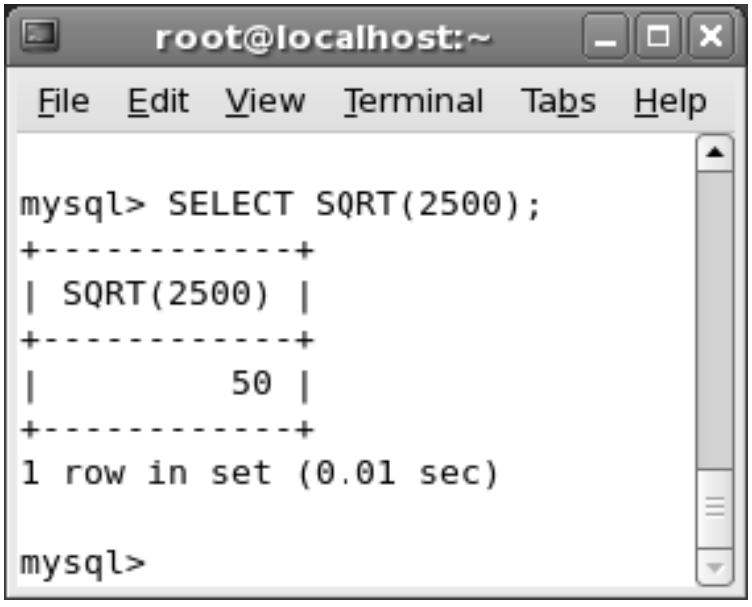


Figure 12.27: SQRT Function

12.4.19 TRUNCATE Function

The `TRUNCATE` function trims the given argument to the specified number of decimal places. The syntax to use this function is:

```
SELECT TRUNCATE(x, d);
```

where,

`x` – specifies the value for the argument

`d` – specifies the number of decimal places

If `d = 0`, then the number has no decimal part. If d is negative, then d digits to the left of the decimal point are made zero.

To truncate `2.4567` to two decimal places, enter the following command at the command prompt:

```
SELECT TRUNCATE(2.4567, 2);
```

Figure 12.28 displays the output of the command.

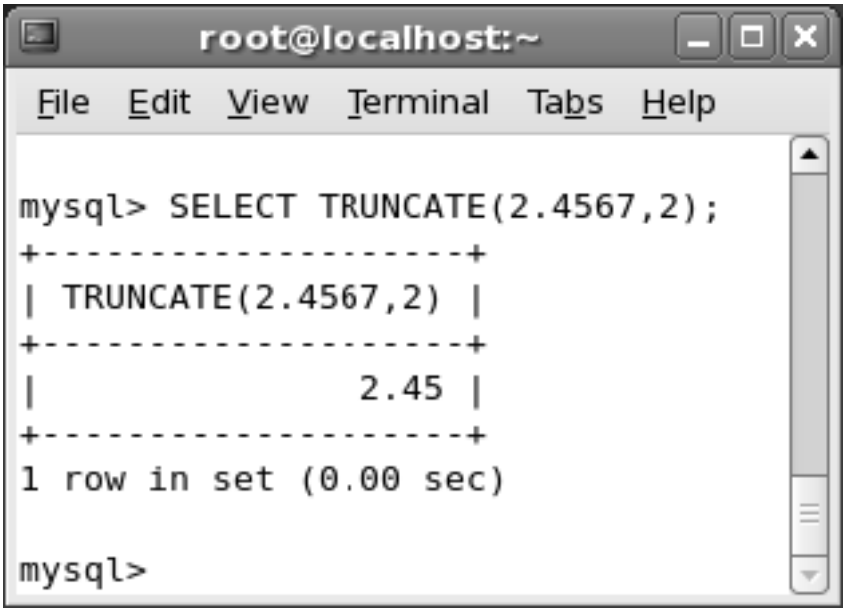


Figure 12.28: TRUNCATE Function

Table 12.2 describes additional Mathematical functions supported by MySQL:

Name	Description	Example
ASIN	The ASIN function returns the arc sine of the specified argument. The arc sine function returns the angle, expressed in radians, of the argument whose sine is specified. The syntax to obtain the arc sine of a function is: SELECT ASIN(expression);	For example, to obtain the arcsine value of 1, enter the following command at the command prompt: SELECT ASIN(-1); The output of this function is: -1.5707963267948966
LOG10	The LOG10 function returns the base-10 logarithm of the argument. The syntax to use this function is: SELECT LOG10(expression);	For example, to obtain the logarithm of 100 to the base 10, enter the following command at the command prompt: SELECT LOG10(100); The output of this function is: 2

Name	Description	Example

LOG2	The LOG2 function returns the base-2 logarithm of the argument. The syntax to use this function is:
	
	`SELECT LOG2(expression);`
		For example, to obtain the logarithm of 16 to the base 2, enter the following command at the command prompt:
		
		`SELECT LOG2(16);`
		
		The output of this function is:
		
		`4`

SIN	The SIN function returns the sine value of the argument. You should specify the argument in radians. The syntax to use this function is:
	
	`SELECT SIN(expression);`
		For example, to calculate the sin value of pi/2, enter the following command at the command prompt:
		
		`SELECT SIN(PI()/2);`
		
		The output of this function is:
		
		`1`

TAN	The TAN function returns the tangent value of the argument passed. You must specify the argument in radians. The syntax to use this function is:
	
	`SELECT TAN(expression);`
		For example, to obtain the tan value of pi/4, enter the following command at the command prompt:
		
		`SELECT TAN(PI()/4);`
		
		The output of this function is:
		
		`0.9999999999999999`

Table 12.2: Additional Mathematical Functions in MySQL

Summary

- In SQL queries, functions can be used in place of the column name and in the `WHERE` clause.
- The `GROUP BY` clause can be used with functions to group the data on a specific constraint.
- `GROUP BY` functions are used to obtain the average value of the argument, count the number of values in a given column, calculate the variance and standard deviation of a given set of values, and calculate the maximum or minimum value from a given set of numbers.
- Examples of functions that can be used with the `GROUP BY` clause are `AVG`, `COUNT`, `COUNT(DISTINCT)`, and `SUM`.
- Mathematical functions are used to operate on numbers. They obtain the trigonometric values such as `SIN`, `COS`, `TAN`, `ATAN`, and `ACOS`. In addition, you can use these functions to obtain the square root, logarithmic value, exponential value of a number, and the modulus value.
- Mathematical functions return a `NULL` value if an error occurs while processing. Some examples of mathematical functions are `ABS`, `ACOS`, `ASIN`, `EXP`, `LOG`, and `SQRT`.

Check Your Progress

1. The output of `CEIL (7.86)` would be _____.

   a. `7`

   b. `8`

   c. `0.86`

   d. `0.06`

2. Which function returns the number of distinct `non-NULL` values in a given column?

   a. `COUNT`

   b. `COUNT(DISTINCT)`

   c. `MAX`

   d. `LOCATE`

3. The output of `FLOOR(9.85)` would be _____.

   a. `9`

   b. `10`

   c. `8`

   d. `0.85`

4. Which function returns the remainder of the first argument divided by the second?

   a. `LOG`

   b. `EXP`

Check Your Progress

c. `POWER`

d. `MOD`

5. Which function returns the non-negative square root of the value specified in the argument?

a. `SIN`

b. `CRC`

c. `PI`

d. `SQRT`

GET SKILLED TO BECOME A PROFESSIONAL

MAKE ME JOB READY

Session
13

Using Basic Functions in MySQL - I (Lab)

Objectives

At the end of this session, the student will be able to:

- *Use the Aggregate functions.*
- *Use the Mathematical functions.*

The steps given in the session are detailed, comprehensive, and carefully thought through in order to meet the learning objectives and understand the tool completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Using Aggregate Functions in MySQL

Aggregate functions operate on a group of values and return a single value as the final result. Aggregate functions can be used with the `GROUP BY` and `HAVING` clause. The `GROUP BY` clause is used to group rows having similar values for a specific column into a single row and the `HAVING` clause specifies the condition to be applied on the selected columns. You can use the `GROUP BY` and `HAVING` clause with the `SELECT` statement to retrieve data that satisfies conditions defined in the `HAVING` clause. The following steps illustrate the use of `GROUP BY` and `HAVING` clause:

1. **Activate the LIBRARY database.**

2. **The librarian wants to view the USER_ID and the BOOK_ID to get details about the fines imposed on the members. However, the average fine amount must not be greater than 10. To display the USER_ID and BOOK_ID columns of the ISSUE_DETAILS table having average fine less than 10, enter the following command at the command prompt:**

```
SELECT USER_ID, BOOK_ID FROM ISSUE_DETAILS

GROUP BY USER_ID

HAVING AVG(FINE)<10;
```

Figure 13.1 displays the output of the command.

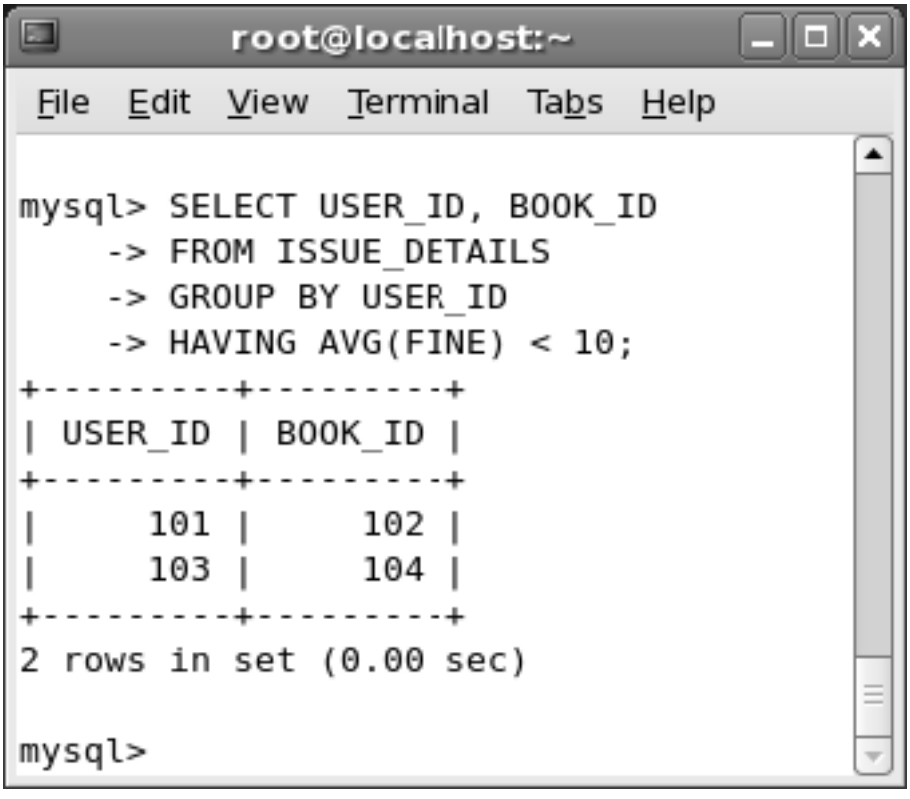


Figure 13.1: Displays Output of AVG Function

3. **The librarian wants to view the total number of REGULAR members. To calculate the number of users whose membership type is REGULAR, enter the following command at the command prompt:**

```
SELECT COUNT(USER_ID) FROM CATEGORY

WHERE TYPE = 'REGULAR';
```

Figure 13.2 displays the output of the command.

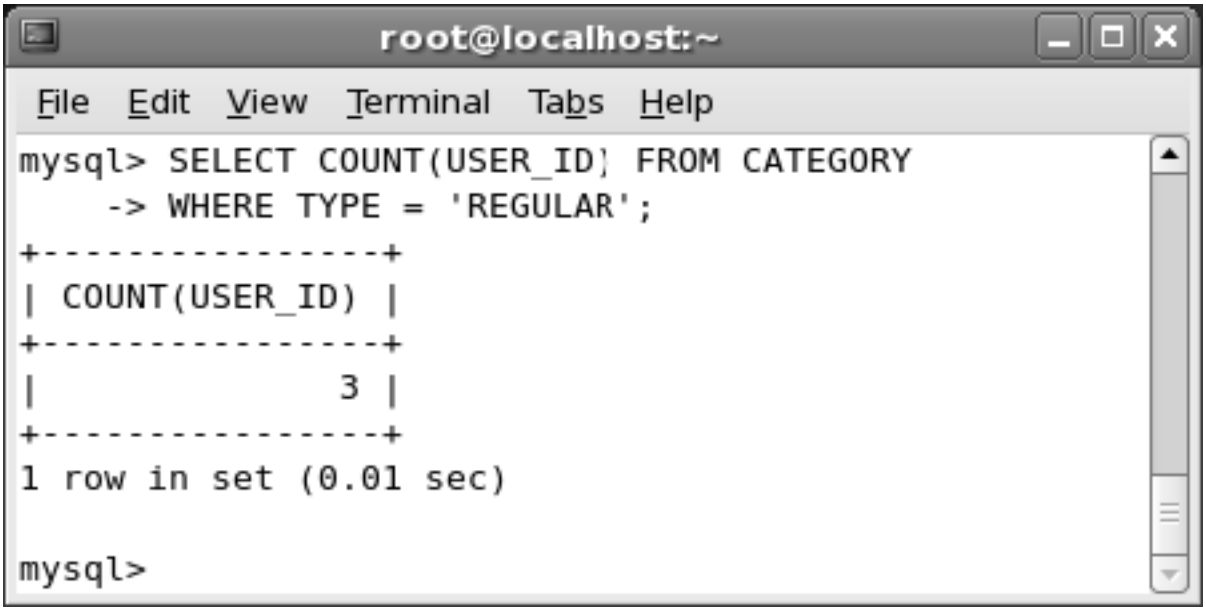


Figure 13.2: Displays Output of COUNT Function

4. **The librarian wants to view the total number of unique memberships available. To calculate the total number of unique membership type, enter the following command at the command prompt:**

```
SELECT COUNT(DISTINCT TYPE) FROM CATEGORY;
```

Figure 13.3 displays the output of the command.

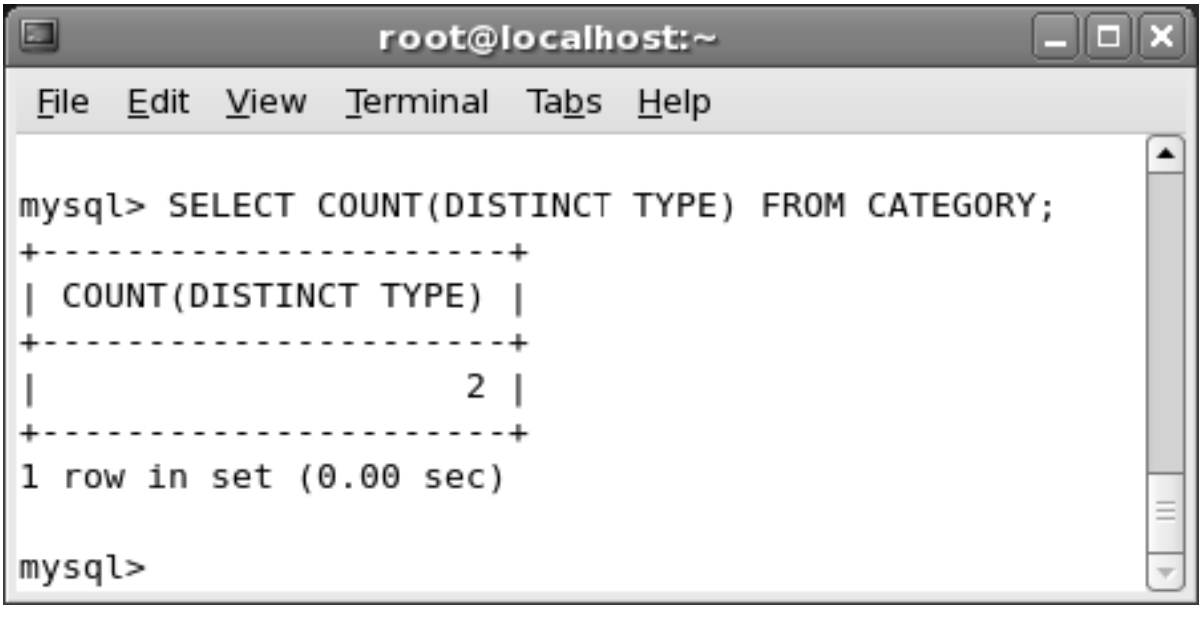


Figure 13.3: Displays Output of COUNT(DISTINCT TYPE) Function

5. **To display the concatenated values of name and user ID of the users belonging to the location, 'MEXICO', enter the following command at the command prompt:**

```
SELECT GROUP_CONCAT(USER_ID), GROUP_CONCAT(USER_NAME), LOCATION

FROM USER_DETAILS WHERE LOCATION = 'MEXICO';
```

Figure 13.4 displays the output of the command.

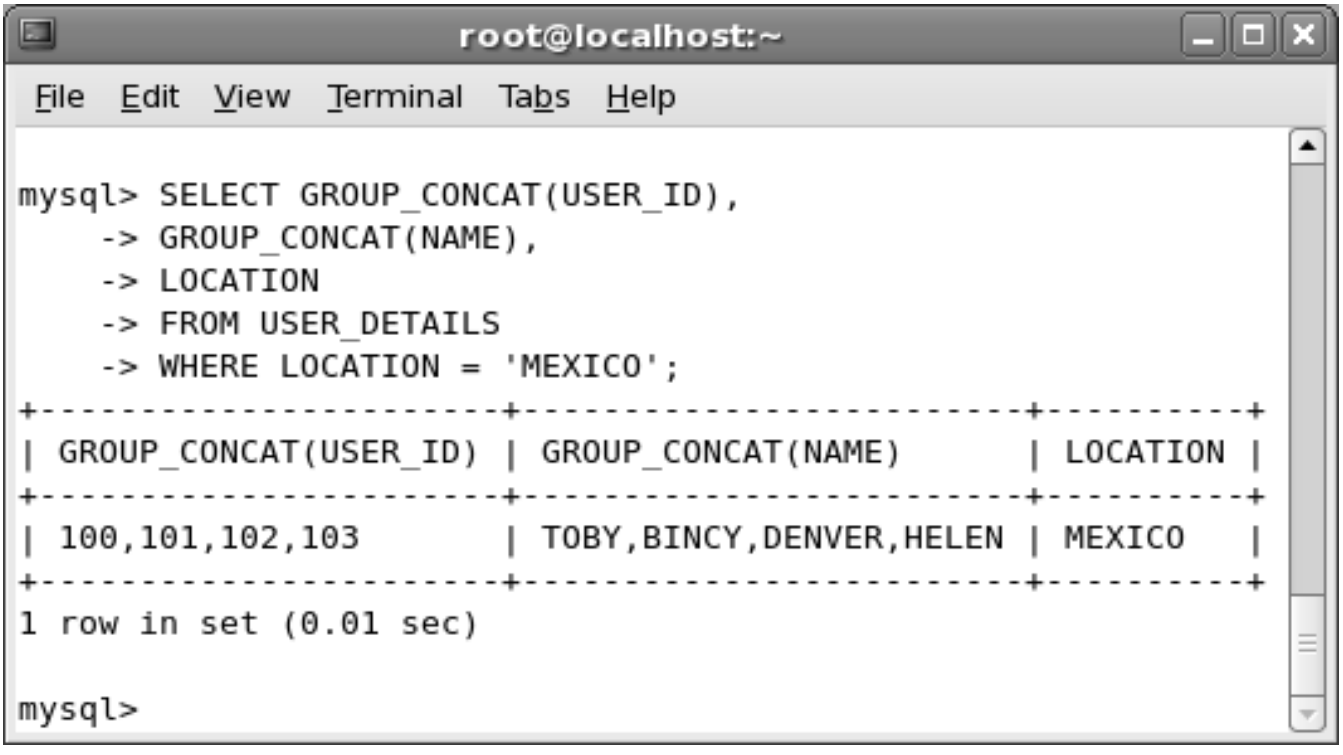


Figure 13.4: GROUP_CONCAT Function

6. **The librarian wants to view the users who have been imposed a fine less than 50 and also have been issued a book. To display the users, enter the following command at the command prompt:**

```
SELECT USER_ID, BOOK_ID

FROM ISSUE_DETAILS

GROUP BY USER_ID

HAVING MAX(FINE)<50;
```

Figure 13.5 displays the output of the command.

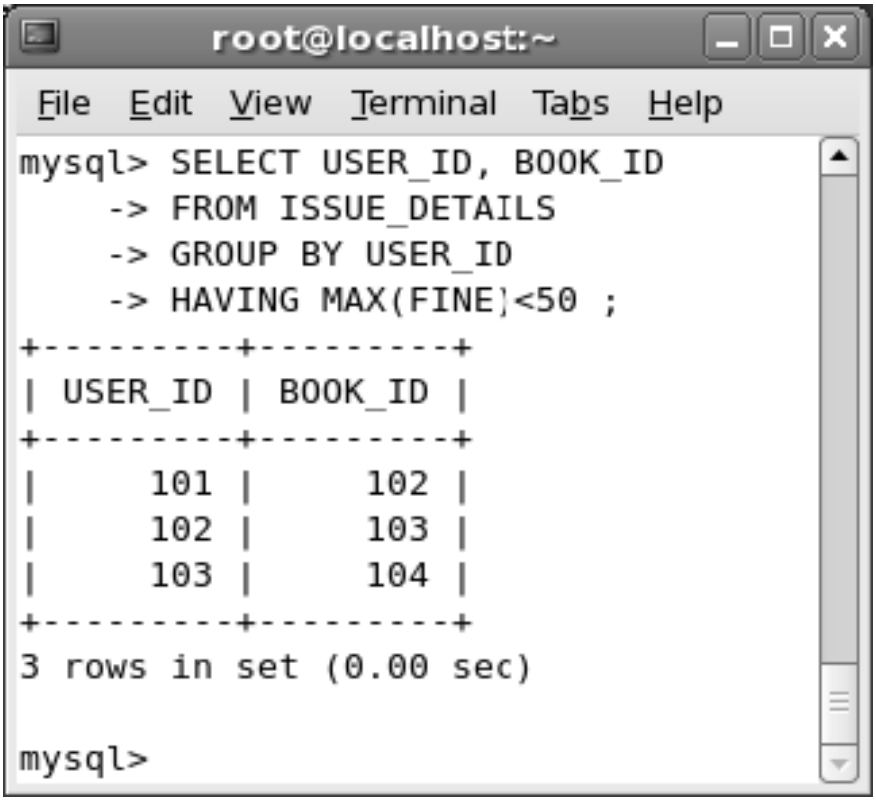


Figure 13.5: MAX Function

For steps 8 to 10, you will be working with the `EMPLOYEE` database that consists of the `EMP_DETAILS`, `EMP_DEPARTMENT`, and `EMP_SALARY` tables. You must activate the `EMPLOYEE` database before executing steps from 8 to 10.

7. **To activate the EMPLOYEE database, enter the following command at the command prompt:**

```
USE EMPLOYEE;
```

8. **To calculate the standard deviation of the gross salary of all the employees, enter the following command at the command prompt:**

```
SELECT STD(GROSS_SAL) FROM EMP_SALARY;
```

Figure 13.6 displays the output of the command.

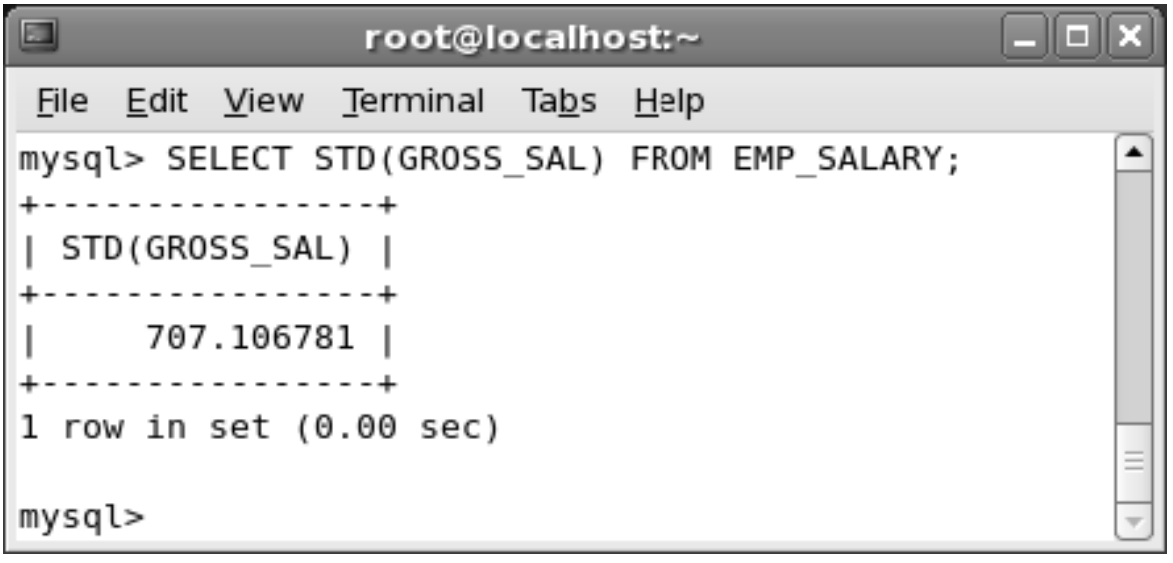


Figure 13.6: STD Function

9. **To calculate the sum of HRA of all the employees belonging to the same department, enter the following command at the command prompt:**

```
SELECT SUM(HRA), D_NAME

FROM EMP_SALARY

GROUP BY D_NAME;
```

Figure 13.7 displays the output of the command.

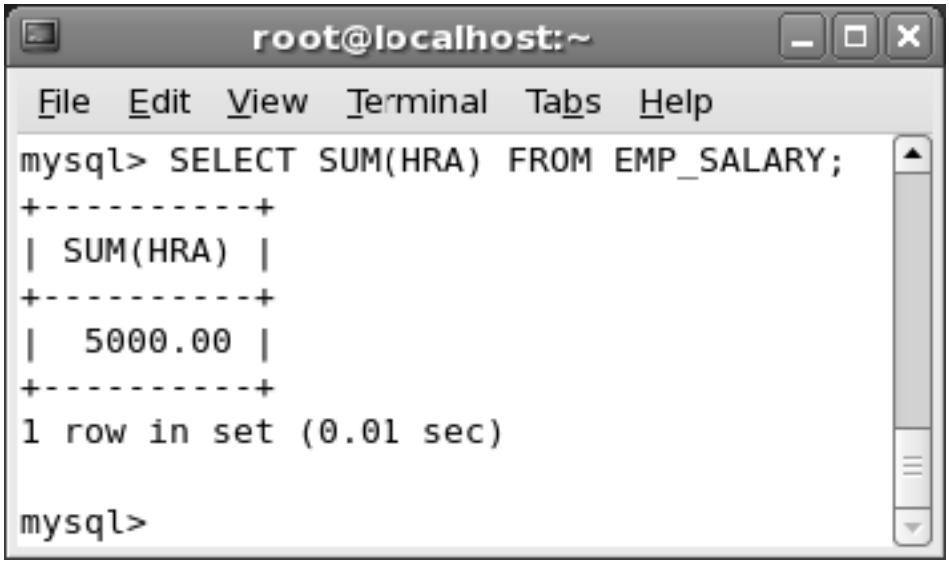


Figure 13.7: SUM Function

10. **To calculate the variance of basic salary for all the employees, enter the following command at the command prompt:**

```
SELECT VARIANCE(BASIC_SAL) FROM EMP_SALARY;
```

Figure 13.8 displays the output of the command.

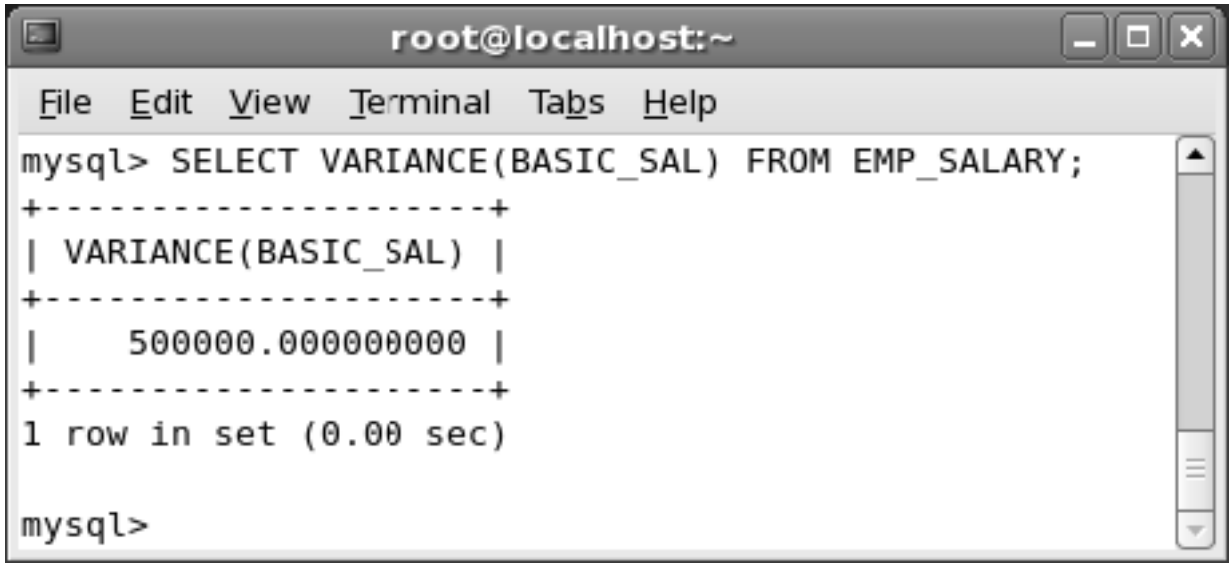


Figure 13.8: VARIANCE Function

Lab Guide

Using Mathematical Functions in MySQL

Mathematical functions operate on numerical data. The following steps illustrate the use of mathematical functions in MySQL.

1. **The basic salary of an employee is in negative. This negative salary has to be converted to a positive value in order to compute results. To calculate the absolute value of the basic salary where the value is in negative, enter the following command at the command prompt:**

```
SELECT E_ID, BASIC_SAL, ABS(BASIC_SAL)

FROM EMP_SALARY

WHERE E_ID = 106;
```

Figure 13.9 displays the output of the command.

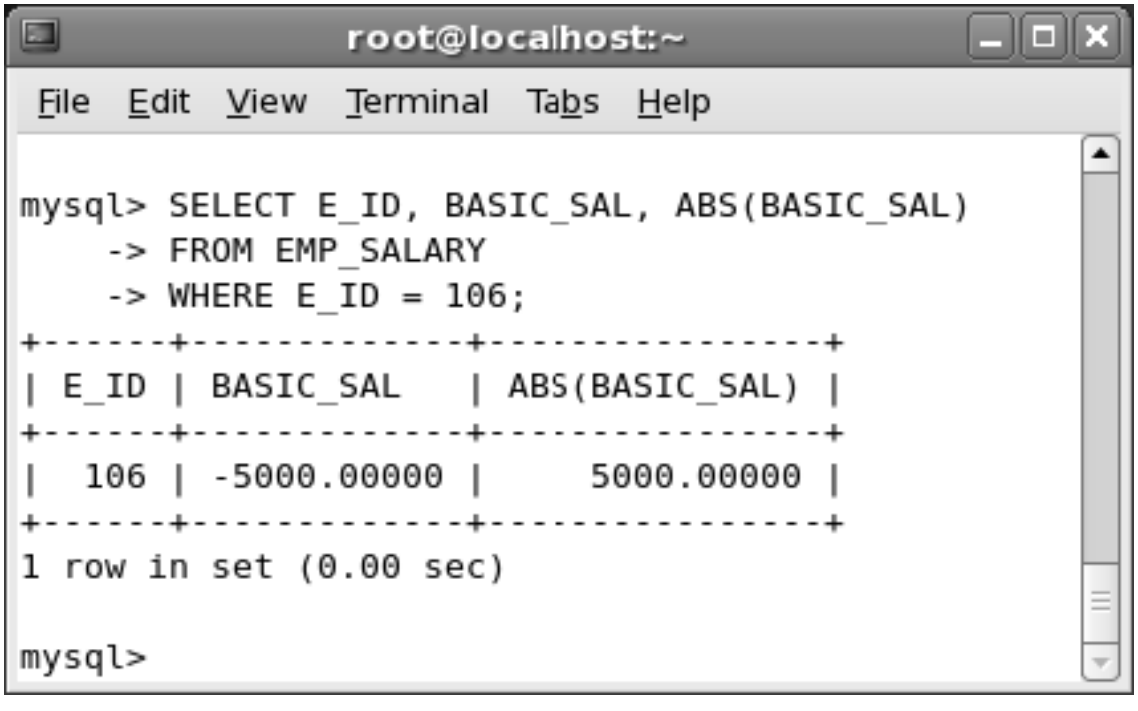


Figure 13.9: ABS Function

2. **To calculate the arc cosine value of 0, enter the following command at the command prompt:**

```
SELECT ACOS(0);
```

Figure 13.10 displays the output of the command.

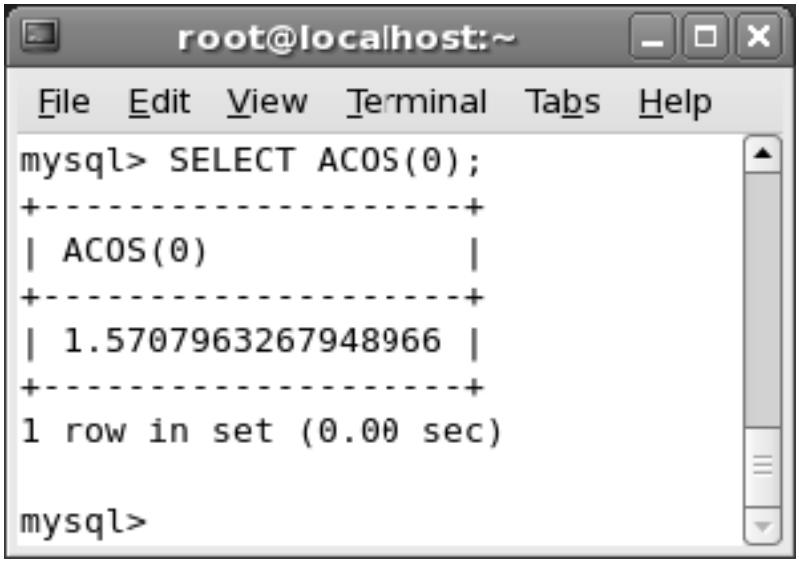


Figure 13.10: ACOS Function

3. **To use the CEILING function on 7.45, enter the following command at the command prompt:**

```
SELECT CEILING(7.45);
```

Figure 13.11 displays the output of the command.

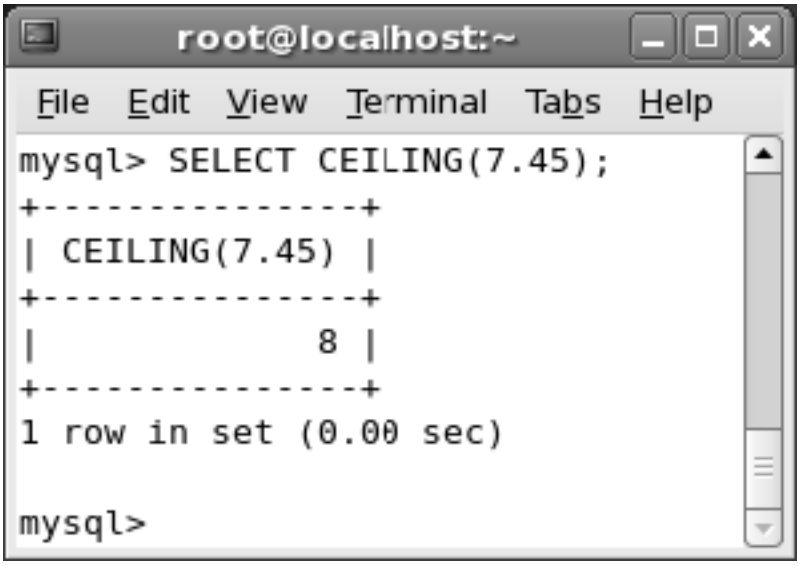


Figure 13.11: CEILING Function

4. **To convert 14 from base 16 to base 10, enter the following command at the command prompt:**

```
SELECT CONV(14, 16, 10);
```

Figure 13.12 displays the output of the command.

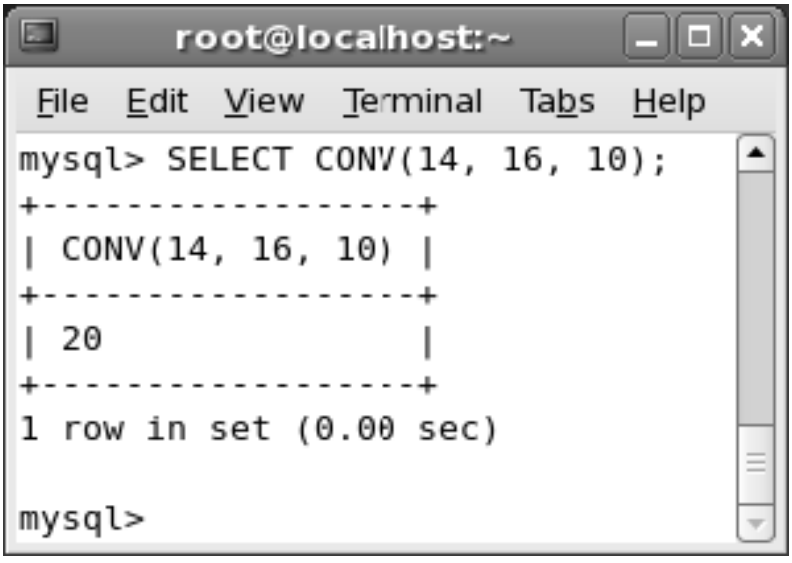


Figure 13.12: CONVERT Function

5. **To calculate the cosine value of pi()/4, enter the following command at the command prompt:**

```
SELECT COS(PI()/4);
```

Figure 13.13 displays the output of the command.

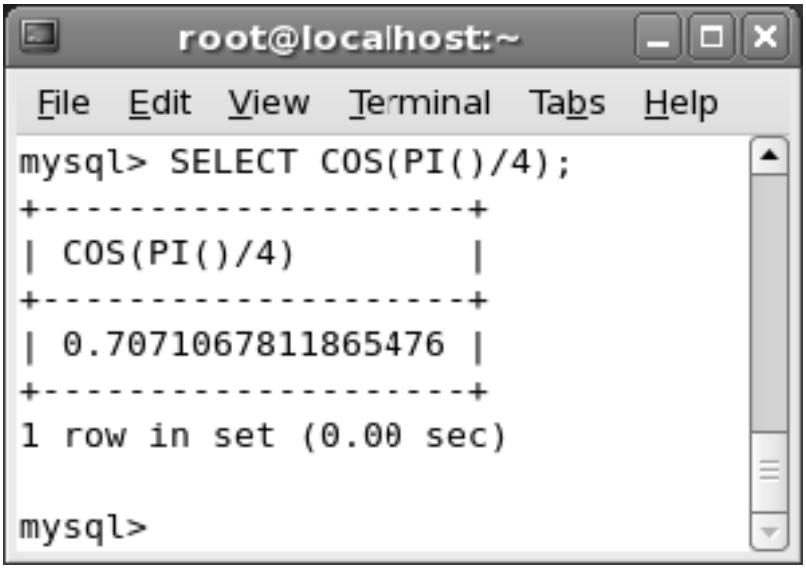


Figure 13.13: COS Function

6. **To calculate the cyclic redundancy check value of the string MYSQL, enter the following command at the command prompt:**

```
SELECT CRC32('MYSQL');
```

Figure 13.14 displays the output of the command.

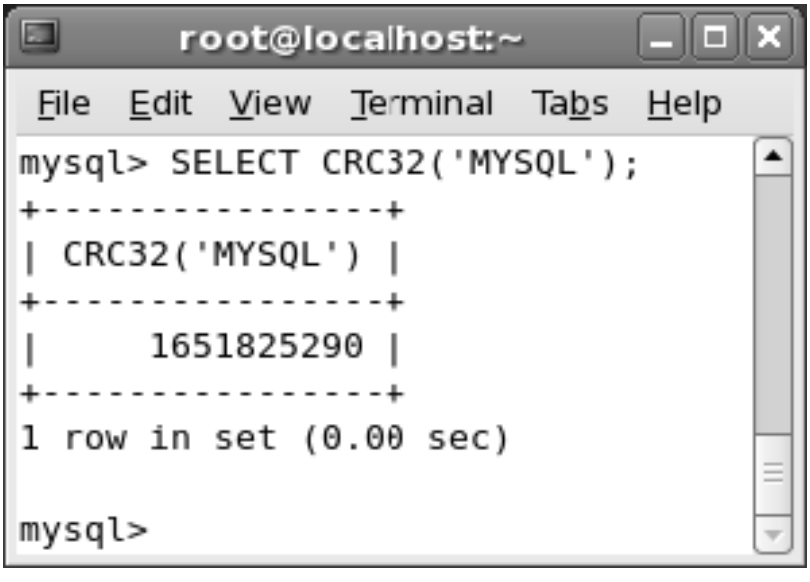


Figure 13.14: CRC32 Function

7. **To calculate the value of pi()/2 in degrees, enter the following command at the command prompt:**

```
SELECT DEGREES(PI()/2);
```

Figure 13.15 displays the output of the command.

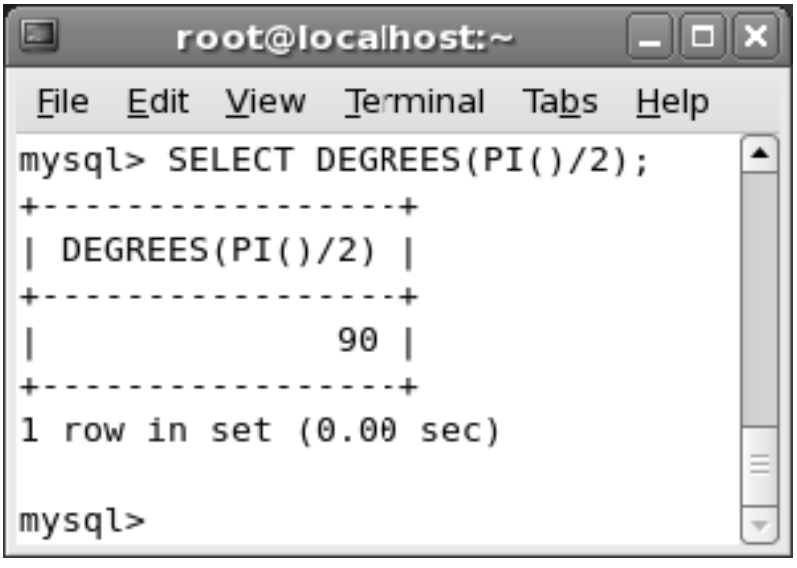


Figure 13.15: DEGREES Function

8. **To calculate the exponential value of -1, enter the following command at the command prompt:**

```
SELECT EXP(-1);
```

Figure 13.16 displays the output of the command.

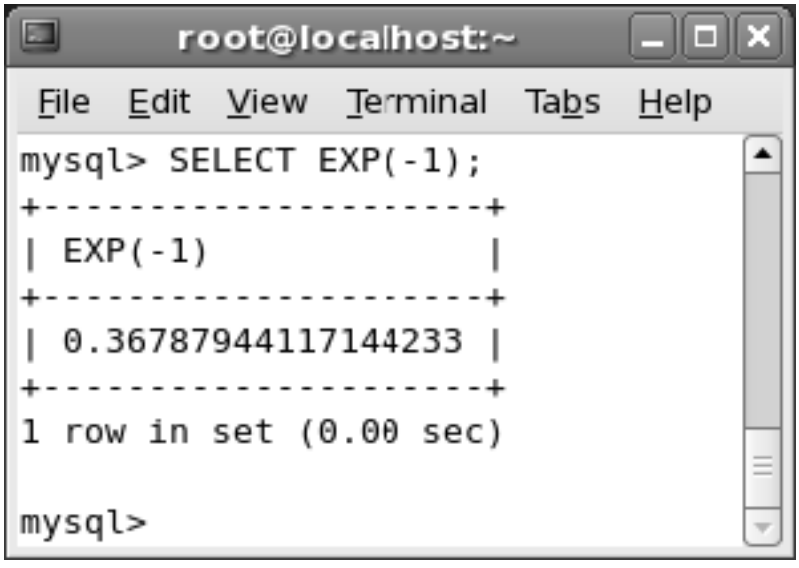


Figure 13.16: EXP Function

9. **John has a bank balance of 45,668.87. He wants to view the account balance without the decimals. To display the bank balance without the decimal, enter the following command at the command prompt:**

```
SELECT FLOOR(45,668.87);
```

Figure 13.17 displays the output of the command.

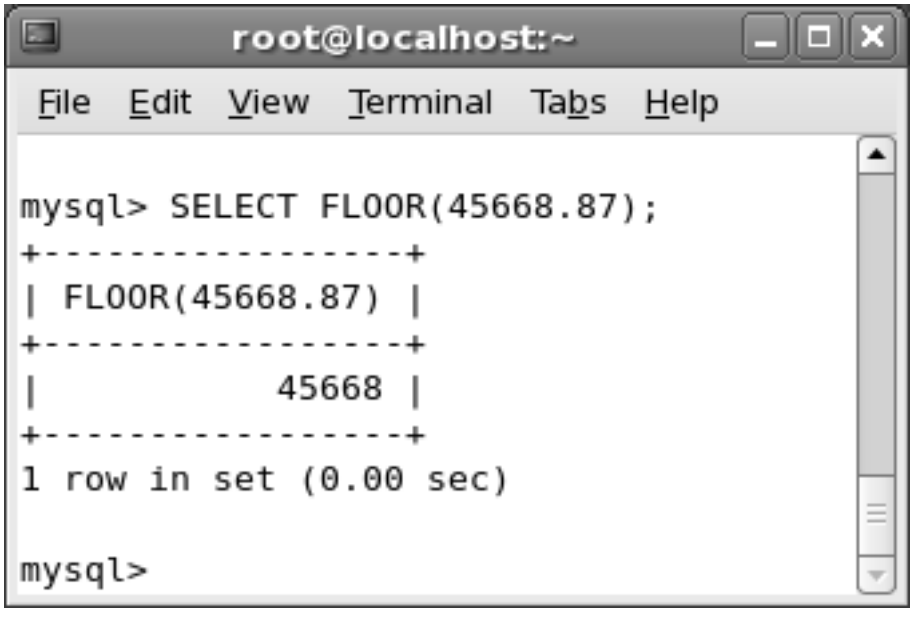


Figure 13.17: FLOOR Function

10. **To calculate the natural logarithm of 10, enter the following command at the command prompt:**

```
SELECT LOG(10);
```

Figure 13.18 displays the output of the command.

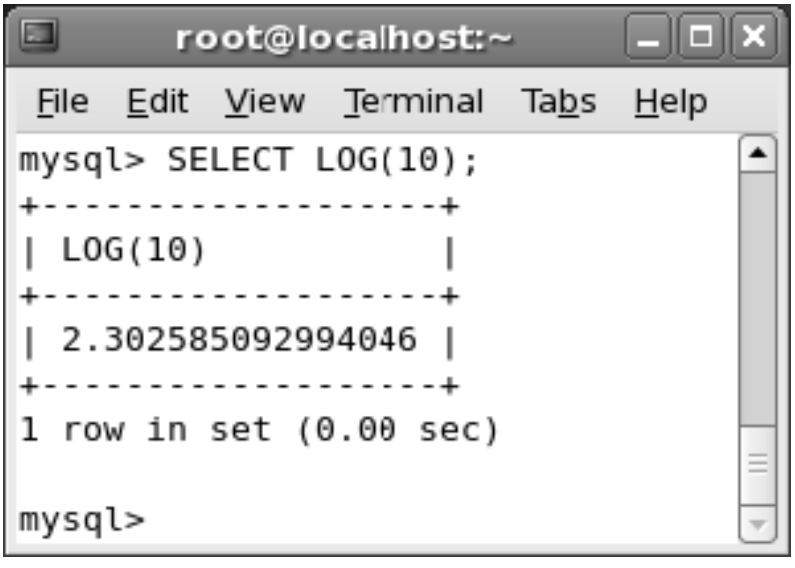


Figure: 13.18: LOG Function

11. **A restaurant bill of 10060 is to be split among 22 people. They will pay only round figure. Now, if 22 people pay round figure, how much from the total bill amount will be pending to be paid? You need to calculate the pending amount from 10060. To calculate the remainder of 10060 divided by 22, enter the following command at the command prompt:**

```
SELECT MOD(100060,22);
```

Figure 13.19 displays the output of the command.

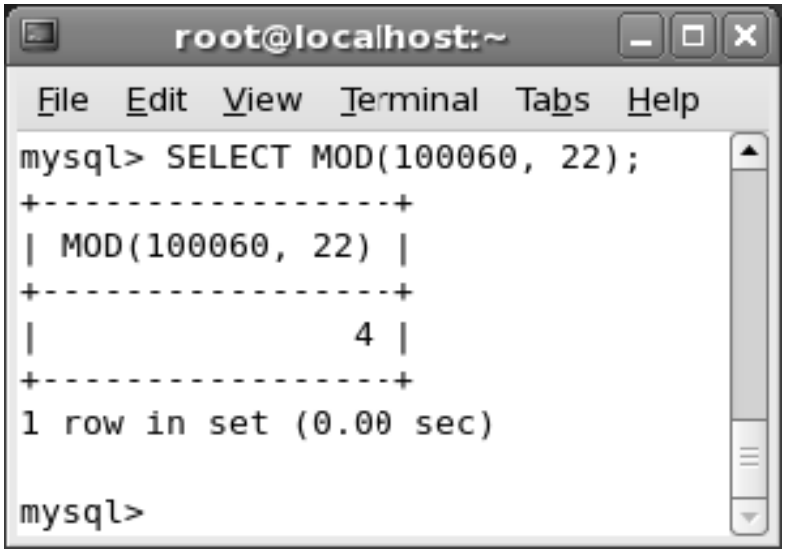


Figure 13.19: MOD Function

12. **To calculate the octal value of 15, enter the following command at the command prompt:**

```
SELECT OCT(15);
```

Figure 13.20 displays the output of the command.

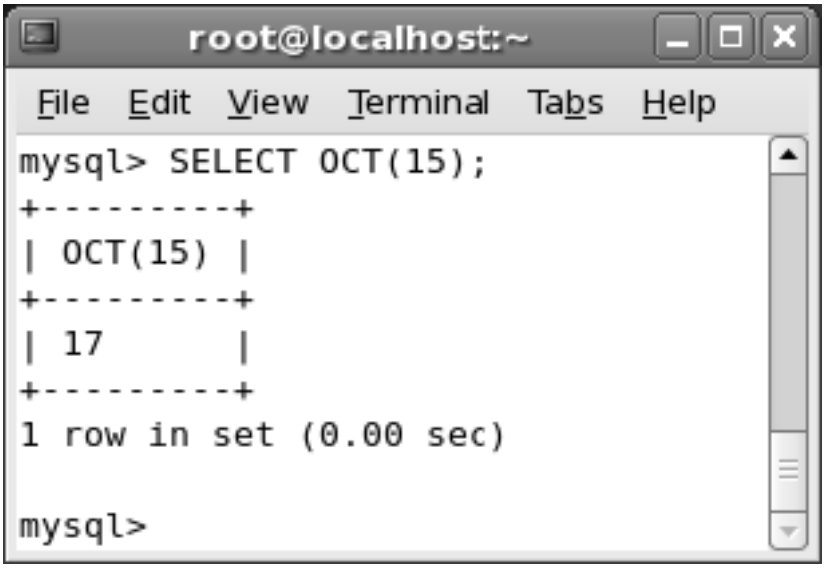


Figure 13.20: OCT Function

13. **John wants to calculate the area of the circle whose radius is 5. The formula to calculate the area of a circle is pi * r2. To calculate the area of the circle, enter the following command at the command prompt:**

```
SELECT PI()* POW(5,2);
```

Figure 13.21 displays the output of the command.

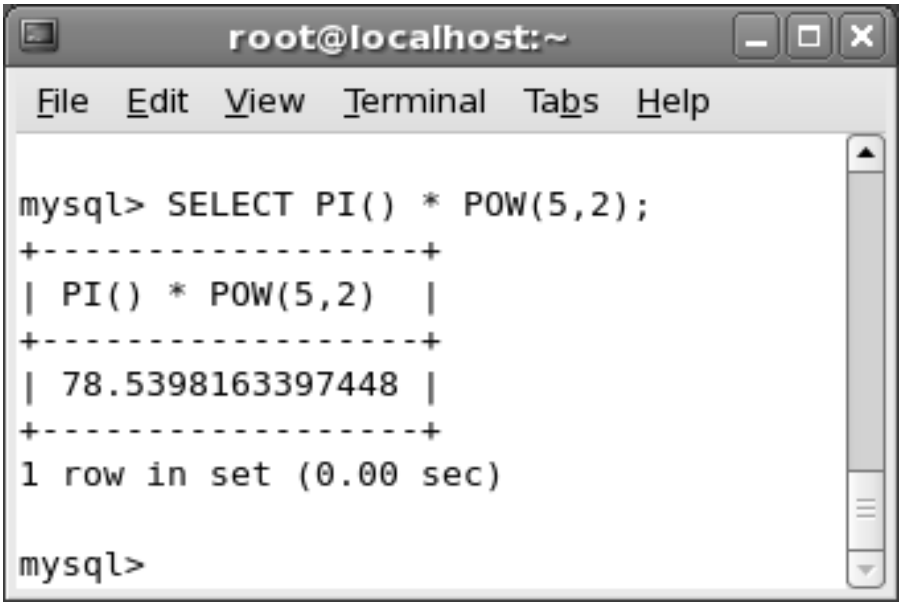


Figure 13.21: PI and POW Function

14. **To calculate the value of 60 in radians, enter the following command at the command prompt:**

```
SELECT RADIANS(60);
```

Figure 13.22 displays the output of the command.

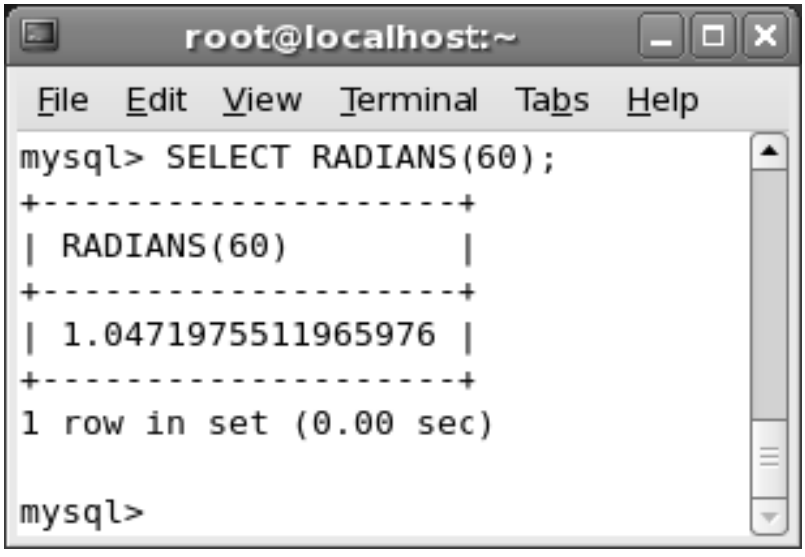


Figure 13.22: RADIANS Function

15. **To calculate the sign of -1250, enter the following command at the command prompt:**

```
SELECT SIGN(-1250);
```

Figure 13.23 displays the output of the command.

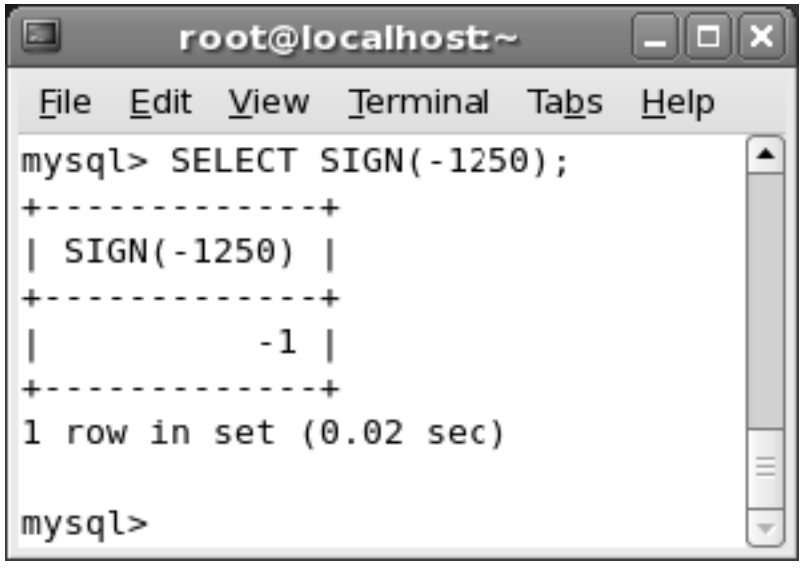


Figure 13.23: SIGN Function

16. **John wants to calculate the measurement of one side of a right angled triangle using the Pythagoras theorem. The equation for Pythagoras theorem is c2 = a2 + b2 where c is the hypotenuse and a and b are the other two sides. Therefore the formula to calculate the length of the hypotenuse will be c = √a2 + b2. To calculate the length of the hypotenuse of a right angled triangle having length of sides as 3 cm and 4 cm, enter the following command at the command prompt:**

```
SELECT SQRT(POWER(4,2)+POWER(3,2));
```

Figure 13.24 displays the output of the command.

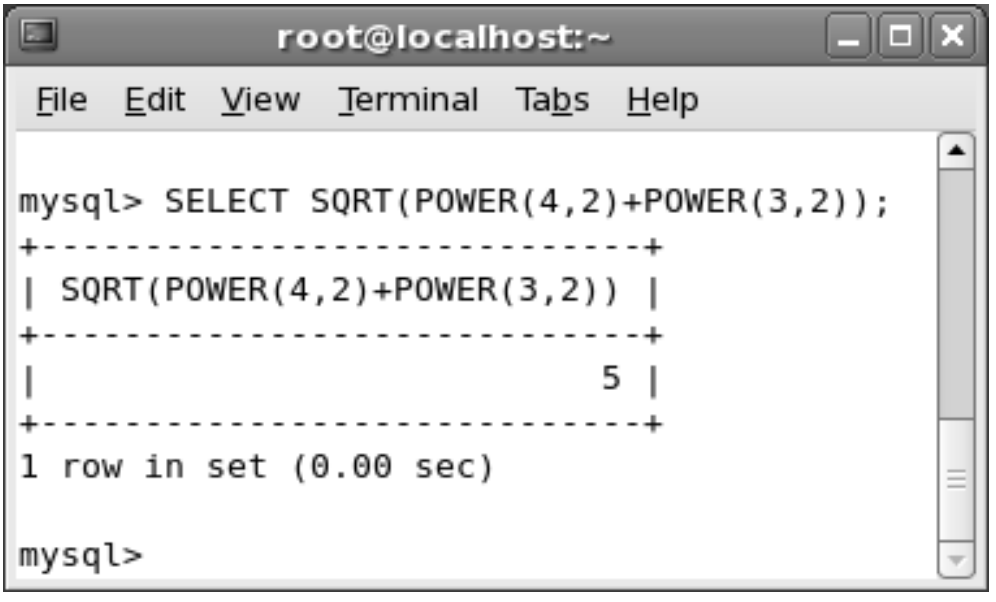


Figure 13.24: SQRT Function

17. **To truncate the BASIC_SAL from 5 decimal places to 2 decimal places, enter the following command at the command prompt:**

```
SELECT BASIC_SAL, TRUNCATE(BASIC_SAL,2)

FROM EMP_SALARY;
```

Figure 13.25 displays the output of the command.

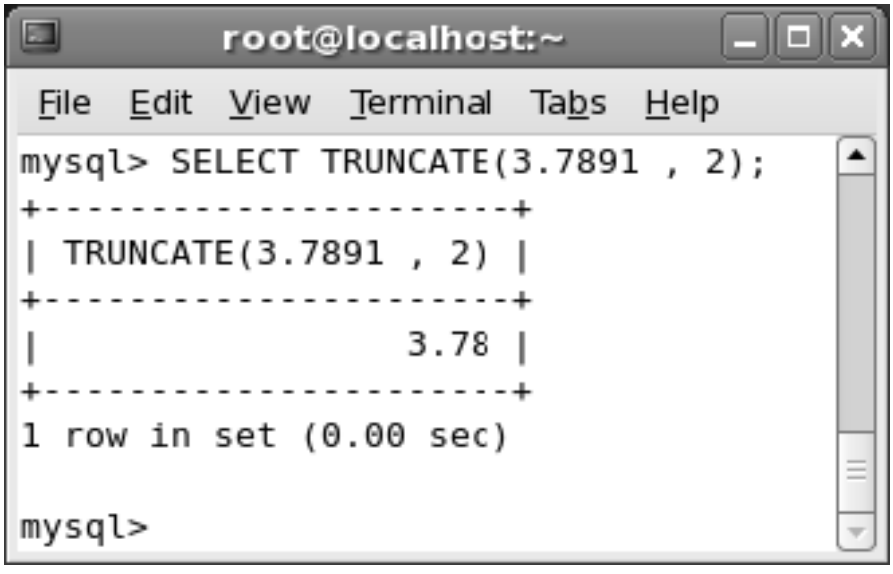


Figure 13.25: TRUNCATE Function

Do It Yourself

1. Calculate the `BITWISE AND` for `15` and `27`.
2. Calculate the `BITWISE OR` for `21` and `25`.
3. Calculate the `BITWISE XOR` for `16` and `20`.
4. From the `LIBRARY` database, display all the columns of the `ISSUE_DETAILS` table having minimum fine greater than `10`.
5. From the `EMPLOYEE` database, calculate the standard deviation of the gross salary of the employees.
6. Calculate the sample standard deviation of the gross salary of all the employees.
7. Calculate the sample variance of the basic salary of all the employees.
8. Calculate the arc sine value of `1`.
9. Calculate the cotangent value of `30`.
10. Calculate the logarithm of `1000` to the base `10`.
11. Calculate the logarithm of `64` to the base `2`.
12. Calculate the sin value of `pi/2`.
13. Calculate the tangent value of `60`.
14. Calculate the length of the hypotenuse of a right-angled triangle having two sides with length as 6 cm and 7 cm.

Session
14

Using Basic Functions in MySQL - II

Concepts

Objectives

At the end of this session, the student will be able to:

- *Describe the use of Date functions in MySQL.*
- *Describe the use of String functions in MySQL.*
- *Describe the use of System Information functions in MySQL.*

14.1 Introduction

Functions are independent programs that execute on the parameter passed and return a value depending on the parameter. Parameters are also known as arguments. You can use MySQL functions within a query to operate on single or multiple columns of a table.

In this session, you will learn about the basics of using functions. You will use different types of functions, such as date, string, and system information functions in MySQL.

14.2 Date Functions in MySQL

MySQL provides date functions that use the date and time data type. The date functions enable you to perform different functions on a date.

14.2.1 ADDDATE Function

The `ADDDATE` function adds two date expressions. The syntax for adding two date expressions is:

```
SELECT ADDDATE(expr1,expr2);
```

where,

`ADDDATE` – calculates the date

`expr1` – specifies a date

`expr2` – defines the date to be added

To add 20 days to the date 2000-12-09, enter the following command at the command prompt:

```
SELECT ADDDATE('2000-12-09', 20);
```

Figure 14.1 displays the output of the command.

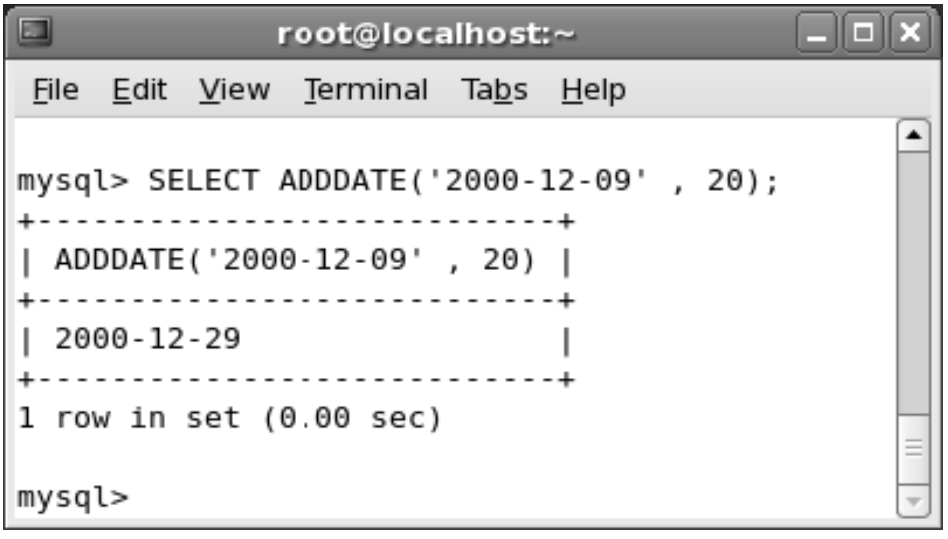


Figure 14.1: ADDDATE Function

14.2.2 ADDTIME Function

The ADDTIME function is used to add two time expressions. The syntax for using this function is:

```
SELECT ADDTIME(expr1, expr2);
```

where,

ADDTIME – calculates the time

expr1 – defines a time or datetime expression

expr2 – specifies a time expression

To calculate the addition of 20:10:25 and 01:02:35, enter the following command at the command prompt:

```
SELECT ADDTIME('20:10:25','01:02:35');
```

Figure 14.2 displays the output of the command.

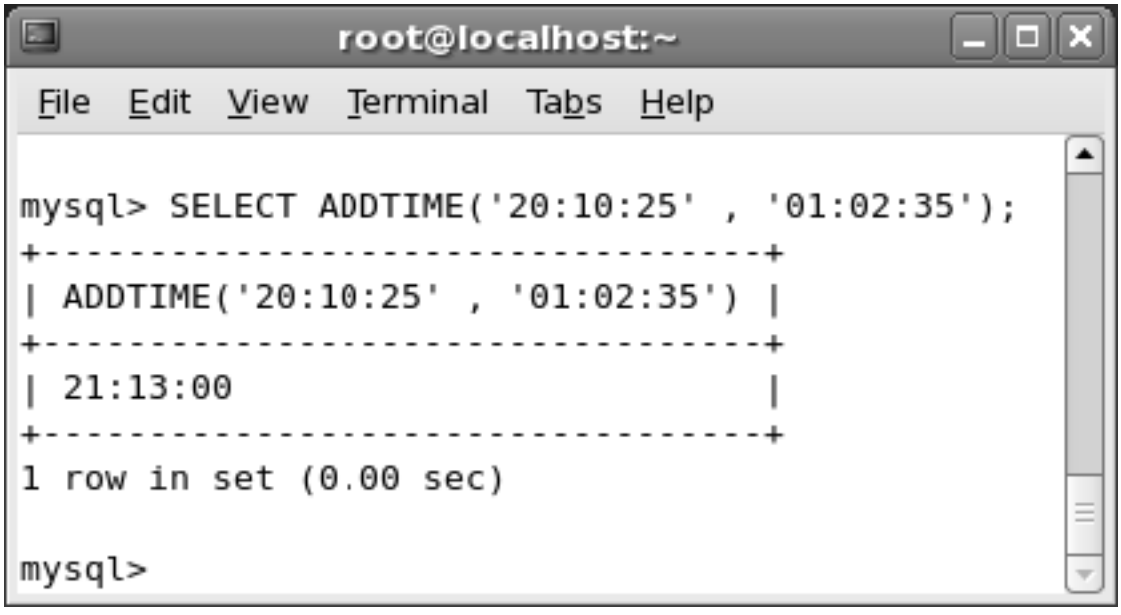


Figure 14.2: ADDTIME Function

14.2.3 CURDATE Function

The `CURDATE` function returns the current date in the `YYYY-MM-DD` or `YYYYMMDD` format. The syntax to retrieve the current date is:

```
SELECT CURDATE();
```

For example, to view the current date, enter the following command at the command prompt:

```
SELECT CURDATE();
```

Figure 14.3 displays the output of the command.

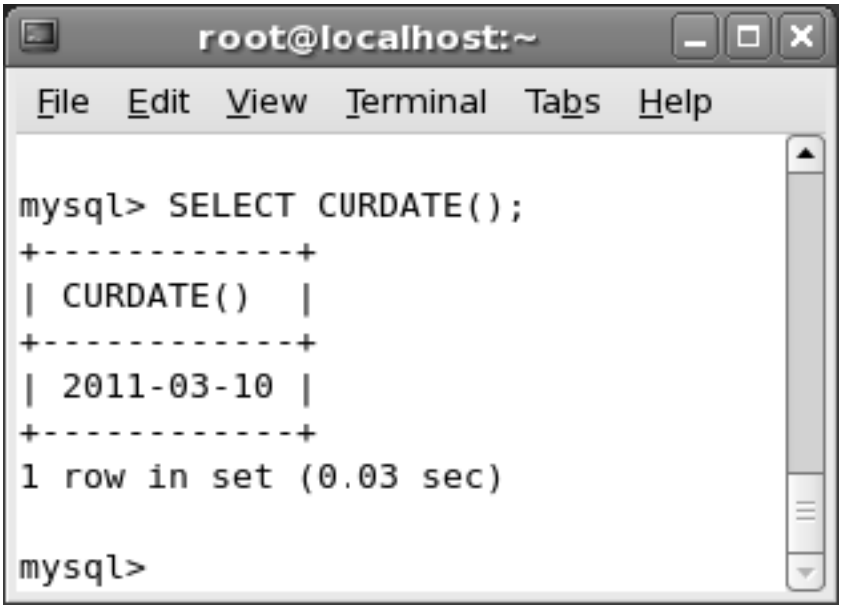


Figure 14.3: CURDATE Function

The alternative syntax for obtaining the current date is:

```
SELECT CURRENT_DATE();
```

14.2.4 CURTIME Function

The `CURTIME` function displays the current time. The syntax to view the current time is:

```
SELECT CURTIME();
```

For example, to obtain the current time, enter the following command at the command prompt:

```
SELECT CURTIME();
```

The alternative syntax for viewing the current time is:

```
SELECT CURRENT_TIME();
```

Figure 14.4 displays the output of the command.

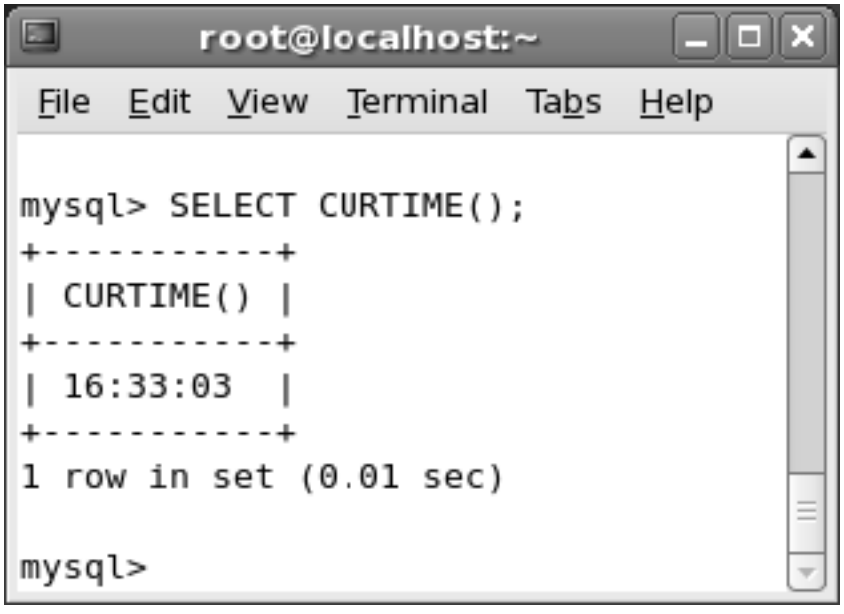


Figure 14.4: CURTIME Function

14.2.5 DATE_ADD Function

The `DATE_ADD` function appends a specified time interval to the given date. The syntax for this function is:

```
SELECT DATE_ADD(date, INTERVAL expr unit);
```

where,

`DATE_ADD` – alters the date with the specified interval

`date` – specifies the starting date or datetime value

`INTERVAL expr unit` – specifies the interval to be added to the given date

For example, to add `1` day to the date `2011-01-31`, enter the following command at the command prompt:

```
SELECT DATE_ADD('2011-01-31', INTERVAL 1 DAY);
```

Figure 14.5 displays the output of the command.

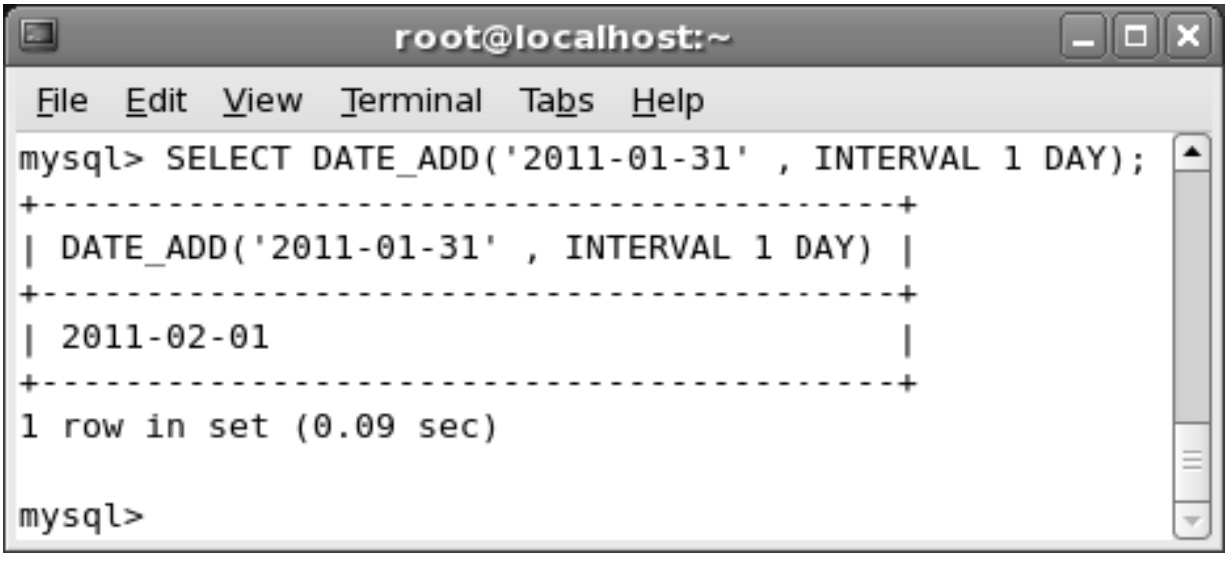


Figure 14.5: DATE_ADD Function

Note: The value returned can be a datetime or a string depending upon the arguments.

14.2.6 DATE function

The `DATE` function displays the date part of the specified date or timestamp. The syntax for this function is:

```
SELECT DATE(expression);
```

For example, to view only the date part from `2007-11-12 21:20:00`, enter the following command at the command prompt:

```
SELECT DATE('2007-11-12 21:20:00');
```

Figure 14.6 displays the output of the command.

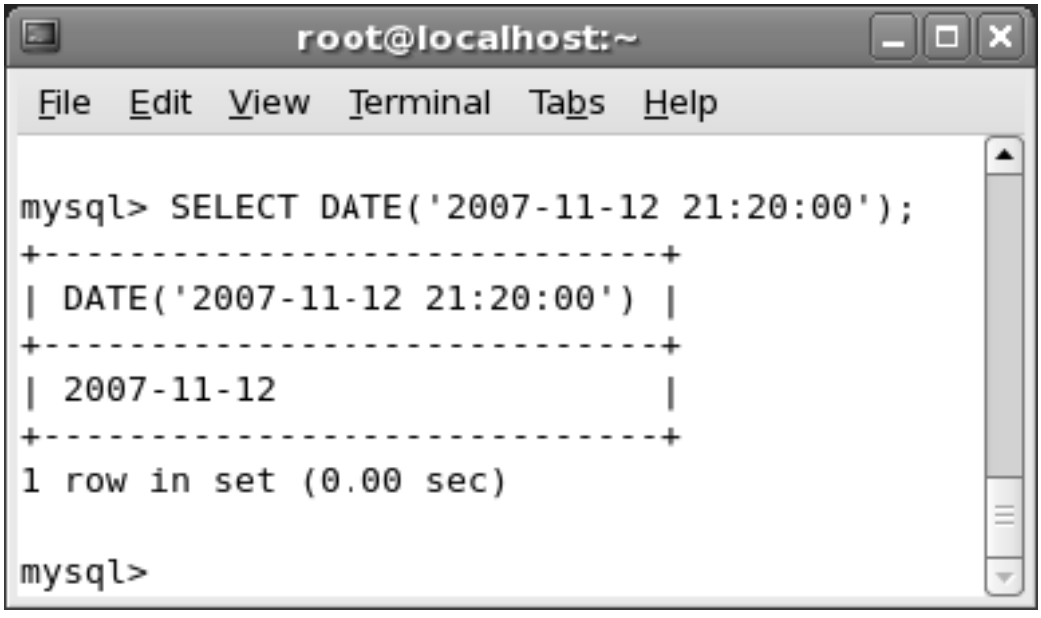


Figure 14.6: DATE Function

14.2.7 DATEDIFF Function

The `DATEDIFF` function returns the number of days between a start date and an end date. These dates are entered as arguments. The syntax to calculate the date difference is:

```
SELECT DATEDIFF(expr1,expr2);
```

where,

`DATEDIFF` – calculates the number of days

`expr1` – defines the start date

`expr2` – defines the end date

For example, to calculate the difference between the dates `2003-10-12` and `2003-09-03`, enter the following command at the command prompt:

```
SELECT DATEDIFF('2003-10-12','2003-09-03');
```

Figure 14.7 displays the output of the command.

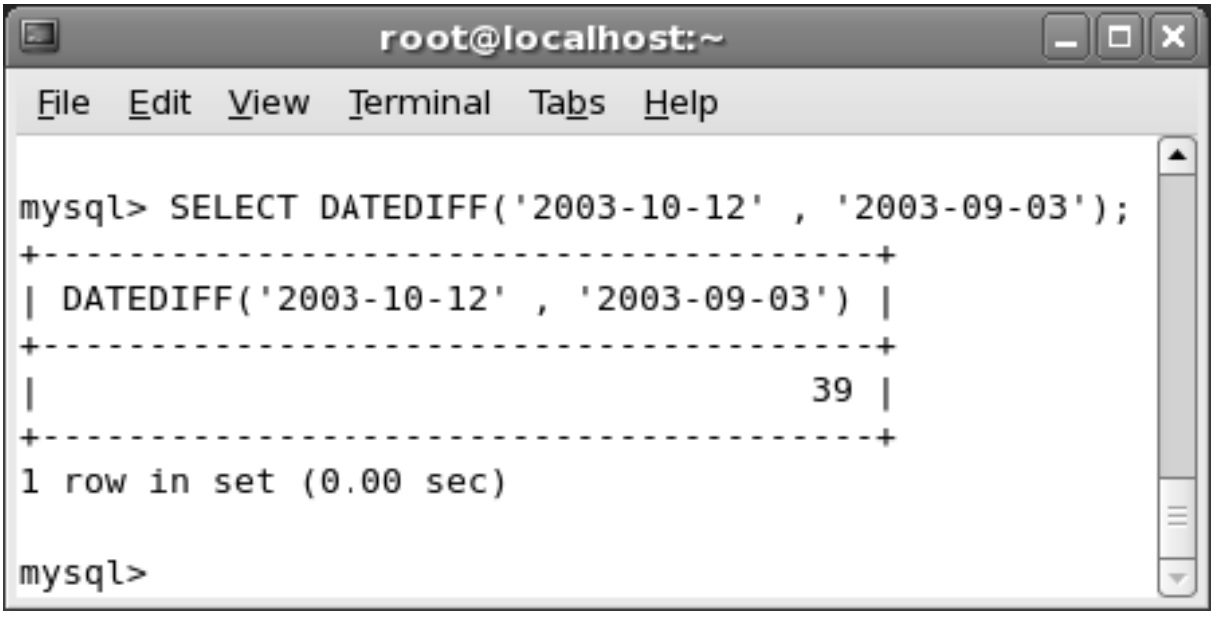


Figure 14.7: DATEDIFF Function

14.2.8 DATE_FORMAT Function

The `DATE_FORMAT` function displays the specified date in a particular format. The syntax to use this function is:

```
SELECT DATE_FORMAT(date, format);
```

where,

`date` – defines a valid date

`format` – defines the output format to display the date

To display a particular date in the `MM-DD-YYYY` format, enter the following command at the command prompt:

```
SELECT DATE_FORMAT('1999-11-04','%M-%D-%Y');
```

Figure 14.8 displays the output of the command.

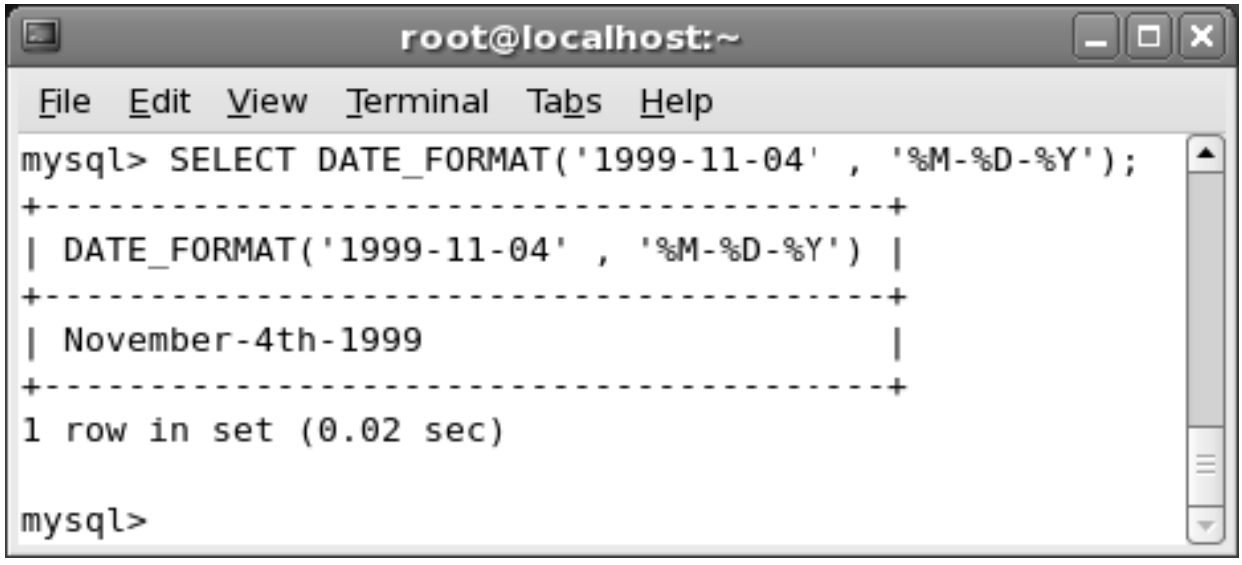


Figure 14.8: DATE_FORMAT Function

Table 14.1 lists some of the format specifiers.

Specifier	Description
%a	Abbreviated weekday name
%f	Microseconds
%H	Hour (00 – 23)
%h	Hour (00 -12)
%M	Month name (January – December)
%m	Month, numeric (01 – 12)
%p	A.M. or P.M.
%s	Seconds
%D	Date
%Y	Year, numeric, four digits

Table 14.1: Format Specifiers

14.2.9 DAY Function

The `DAY` function returns the day of the month for the specified date. The range is from 1 to 31. The syntax to display the day of the month is:

```
SELECT DAY(date);
```

Concepts

For example, to obtain the day for the date `2004-08-15`, enter the following command at the command prompt:

```
SELECT DAY('2004-08-15');
```

Figure 14.9 displays the output of the command.

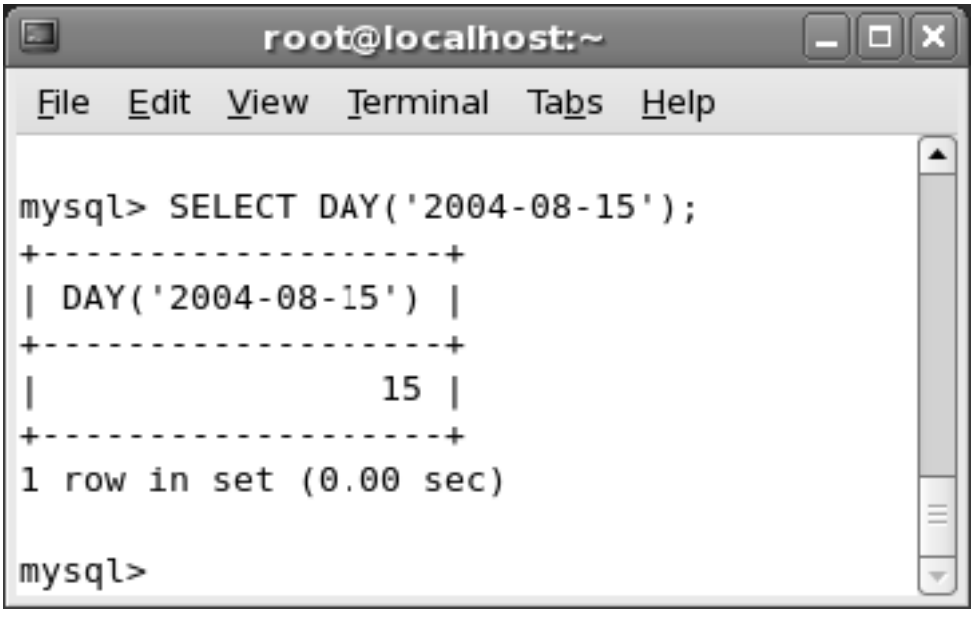


Figure 14.9: DAY Function

14.2.10 DAYNAME Function

The `DAYNAME` function returns the name of the weekday for a date entered as an argument. The syntax to display the name of the weekday is:

```
SELECT DAYNAME(date);
```

For example, to obtain the name of the day for the date `2002-05-09`, enter the following command at the command prompt:

```
SELECT DAYNAME('2002-05-09');
```

Figure 14.10 displays the output of the command.

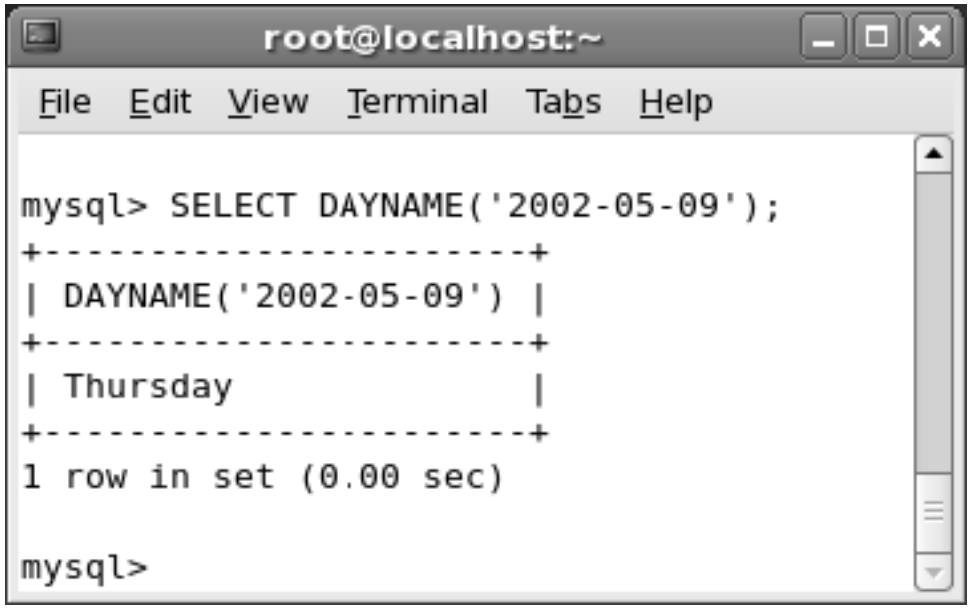


Figure 14.10: DAYNAME Function

14.2.11 HOUR Function

The `HOUR` function returns the hour of the time specified as an argument. The syntax for this function is:

```
SELECT HOUR(expression);
```

For example, to retrieve the hour from the expression `21:05:57`, enter the following command at the command prompt:

```
SELECT HOUR('21:05:57');
```

Figure 14.11 displays the output of the command.

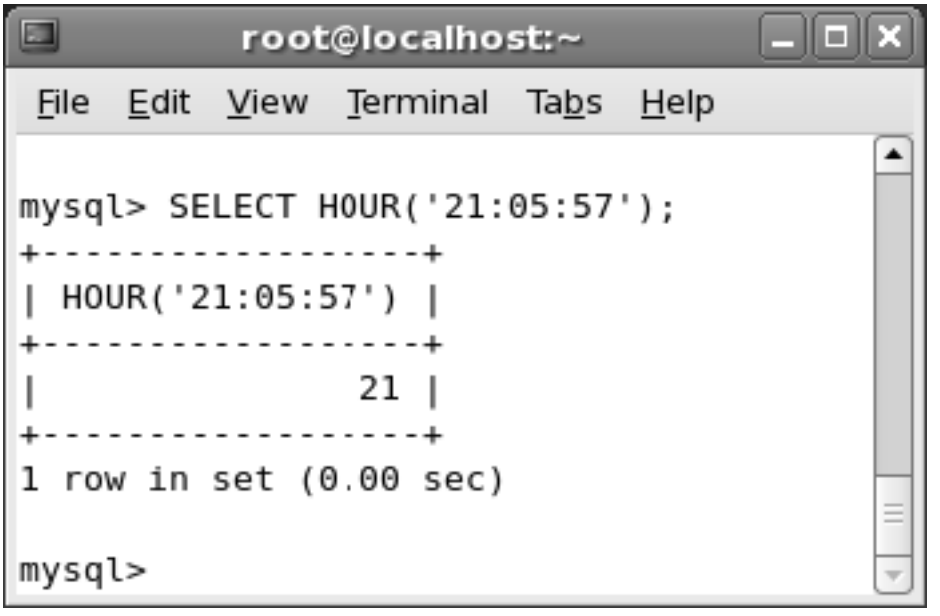


Figure 14.11: HOUR Function

14.2.12 MINUTE Function

The `MINUTE` function extracts the minutes from the specified time argument. The output of this function is a numeric value ranging from `0` to `59`. The syntax for this function is:

```
SELECT MINUTE(expression);
```

For example, to display the minutes from a datetime expression, enter the following command at the command prompt:

```
SELECT MINUTE('09:30:55');
```

Figure 14.12 displays the output of the command.

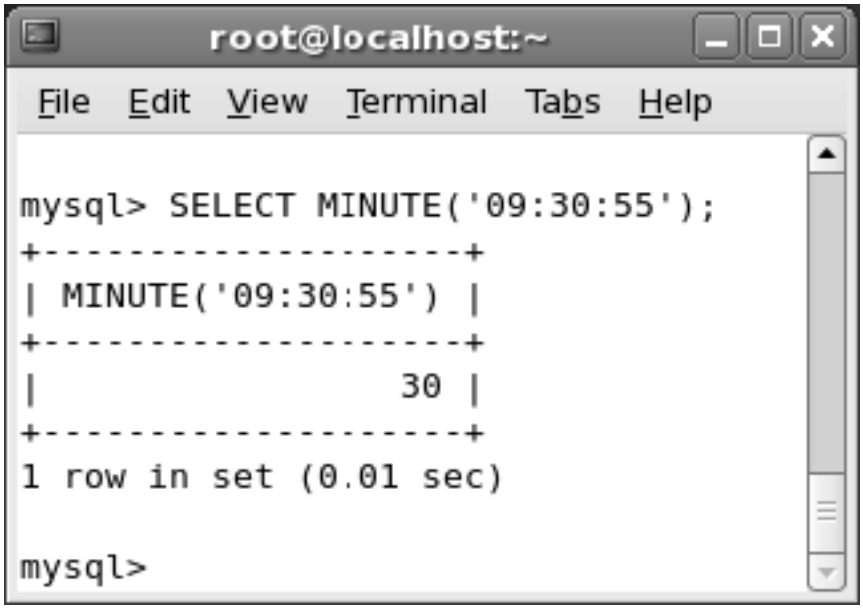


Figure 14.12: MINUTE Function

14.2.13 MONTH Function

The `MONTH` function extracts the month from the specified argument. The output of this function is a numeric value ranging from `1` to `12`. The syntax for this function is:

```
SELECT MONTH(expression);
```

For example, to display the month from the datetime expression `1993-11-22`, enter the following command at the command prompt:

```
SELECT MONTH('1993-11-22');
```

Figure 14.13 displays the output of the command.

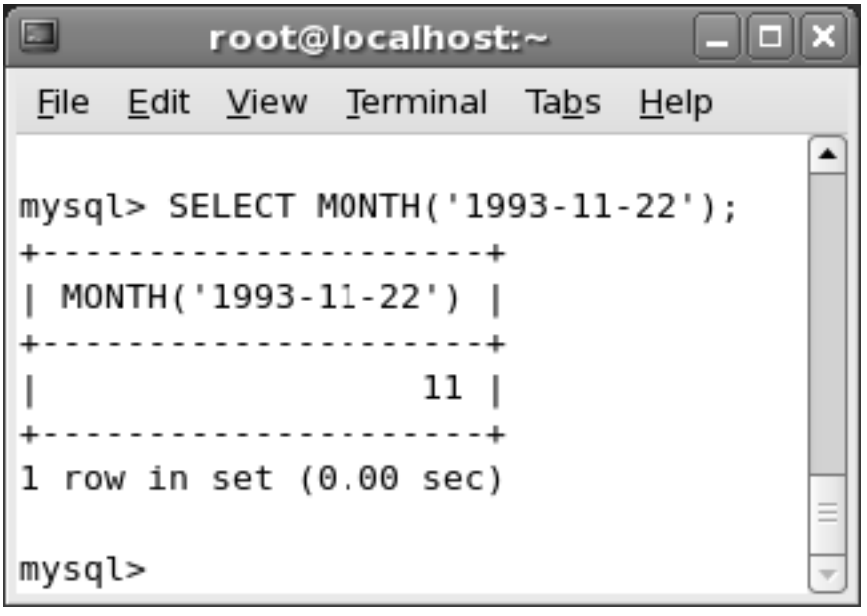


Figure 14.13: MONTH Function

14.2.14 MONTHNAME Function

The `MONTHNAME` function returns the name of the month for the date passed as an argument. The syntax for displaying the name of the month is:

```
SELECT MONTHNAME(date);
```

For example, to display the name of the month for the date `2006-07-03`, enter the following command at the command prompt:

```
SELECT MONTHNAME('2006-07-03');
```

Figure 14.14 displays the output of the command.

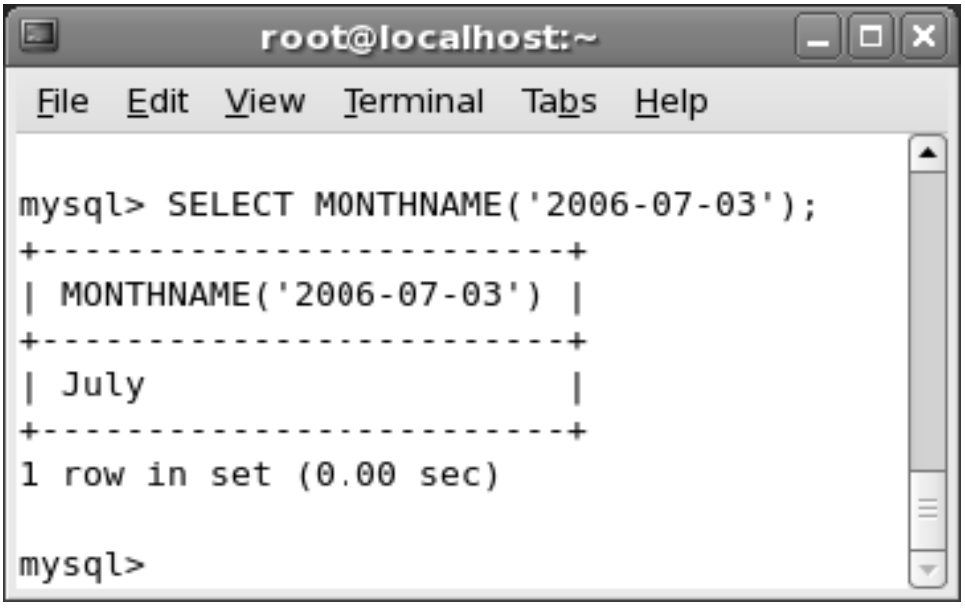


Figure 14.14: MONTHNAME Function

14.2.15 NOW Function

The `NOW` function returns the current date and time. The output of this function is in the `YYYY-MM-DD HH:MM:SS` or `YYYYMMDDHHMMSS.uuuuuu` format, and the value is expressed in the current time zone. The syntax to view the current date and time value is:

```
SELECT NOW();
```

For example, to view the current timestamp, enter the following command at the command prompt:

```
SELECT NOW();
```

The alternative commands to display the current time and date are:

```
SELECT CURRENT_TIMESTAMP();
SELECT LOCALTIME();
SELECT LOCALTIMESTAMP();
```

Figure 14.15 displays the output of the NOW function.

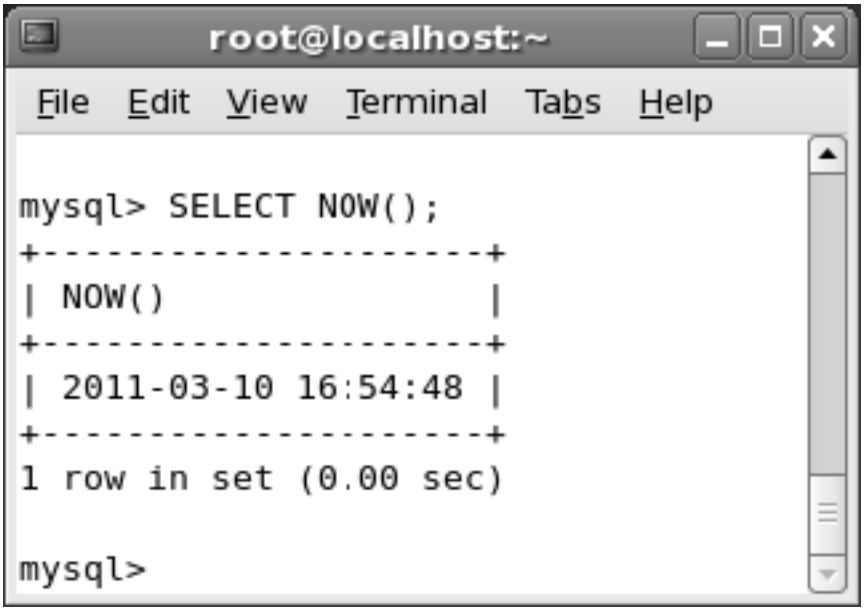


Figure 14.15: NOW Function

14.2.16 SECOND Function

The `SECOND` function returns the number of seconds specified in the argument. The output of this function is a numeric value ranging from 1 to 59. The syntax for this function is:

```
SELECT SECOND(expression);
```

For example, to calculate the number of seconds for `21:10:25`, enter the following command at the command prompt:

```
SELECT SECOND('21:10:25');
```

Figure 14.16 displays the output of the command.

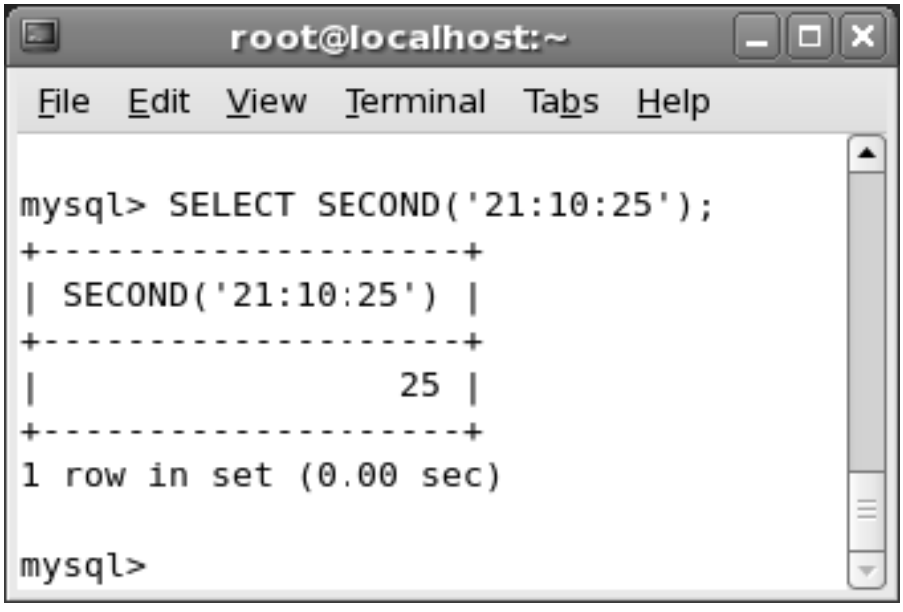


Figure 14.16: SECOND Function

14.2.17 SEC_TO_TIME Function

The `SEC_TO_TIME` function converts the number of seconds specified as an argument to `HH:MM:SS` format. The syntax for this function is:

```
SELECT SEC_TO_TIME(expression);
```

where,

`SEC_TO_TIME` – converts seconds to `HH:MM:SS` format

`expression` – defines a numeric value specifying the number of seconds

To convert `3600` to the `HH:MM:SS` format, enter the following command at the command prompt:

```
SELECT SEC_TO_TIME(3600);
```

Figure 14.17 displays the output of the command.

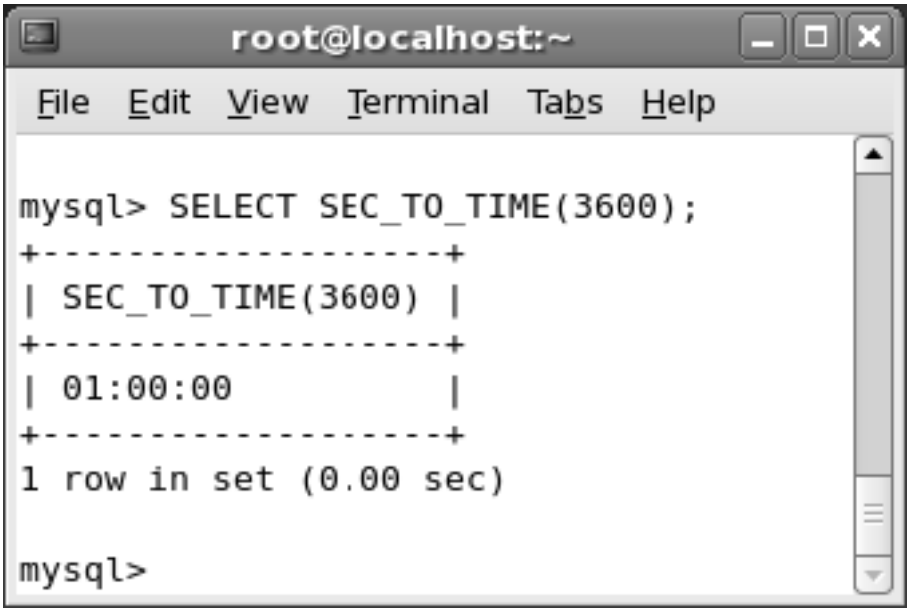


Figure 14.17: SEC_TO_TIME Function

14.2.18 SYSDATE Function

The `SYSDATE` function returns the current date and time value after execution. The `SYSDATE` function is different from the `NOW` function. Consider an example, where you have to add 50,000 records to a table in the database. These rows will have a different timestamp because all the rows will not be added at the same time to the table in the database. The `SYSDATE` function returns this unique time required to execute a function. On the other hand, the `NOW` function will display the same execution time for adding all the 50,000 records to the table. The syntax for this function is:

```
SELECT SYSDATE(expression);
```

For example, to display the current date and time value, enter the following command at the command prompt:

```
SELECT SYSDATE();
```

Figure 14.18 displays the output of the command.

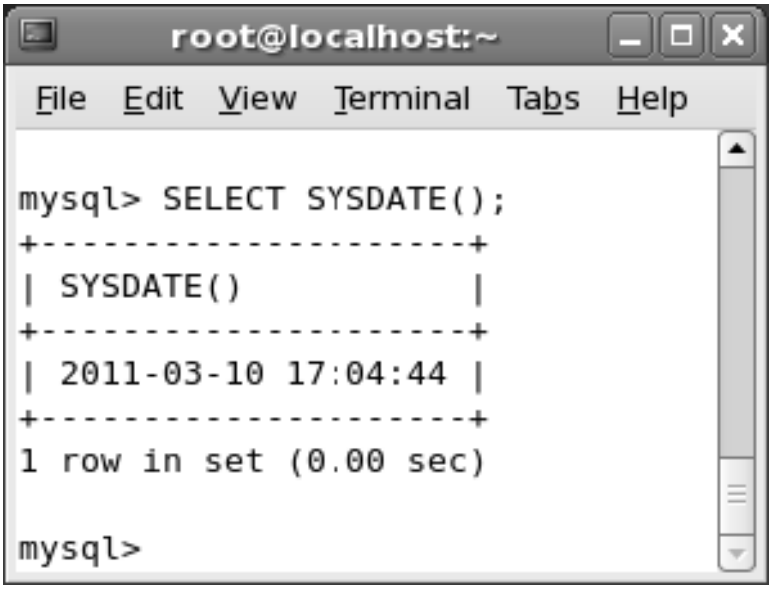


Figure 14.18: SYSDATE Function

14.2.19 TIME Function

The `TIME` function displays only the time part of the specified date or timestamp. The syntax for this function is:

```
SELECT TIME(expression);
```

For example, to display only the time from `2007-09-12 20:20:00`, enter the following command at the command prompt:

```
SELECT TIME(NOW());
```

Figure 14.19 displays the output of the command.

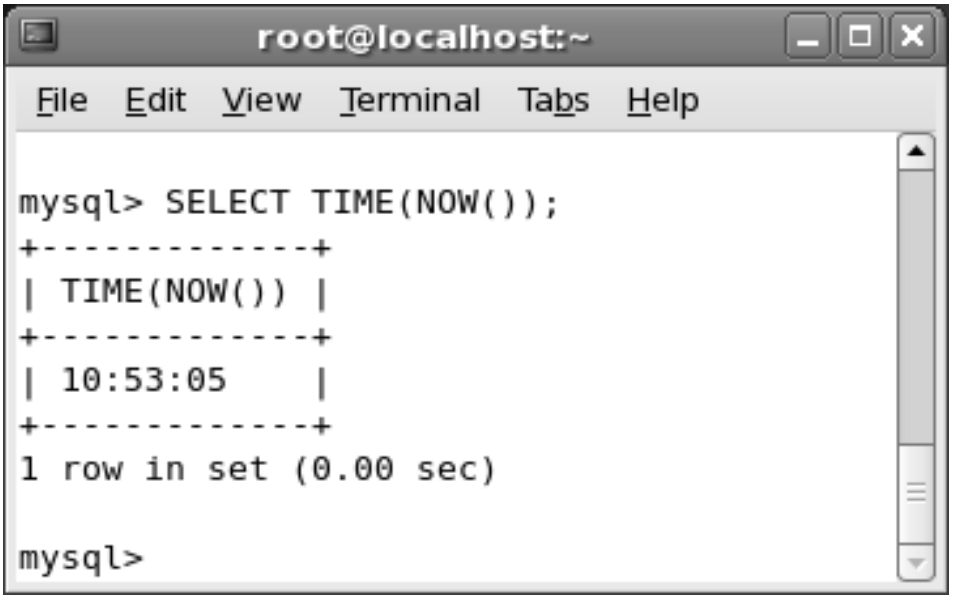


Figure 14.19: TIME Function

14.2.20 TIMEDIFF Function

The `TIMEDIFF` function returns time interval between two time arguments. The syntax to calculate the time difference is:

```
SELECT TIMEDIFF(time1, time2);
```

where,

`TIMEDIFF` – calculates the time difference

`time1`, `time2` – specifies the time expressions

Note: In the syntax, the values for `time1` and `time2` must be of the Date data type.

For example, to calculate the difference between `10:15:20` and `12:45:20`, enter the following command at the command prompt:

```
SELECT TIMEDIFF('10:15:20','12:45:20');
```

Figure 14.20 displays the output of the command.

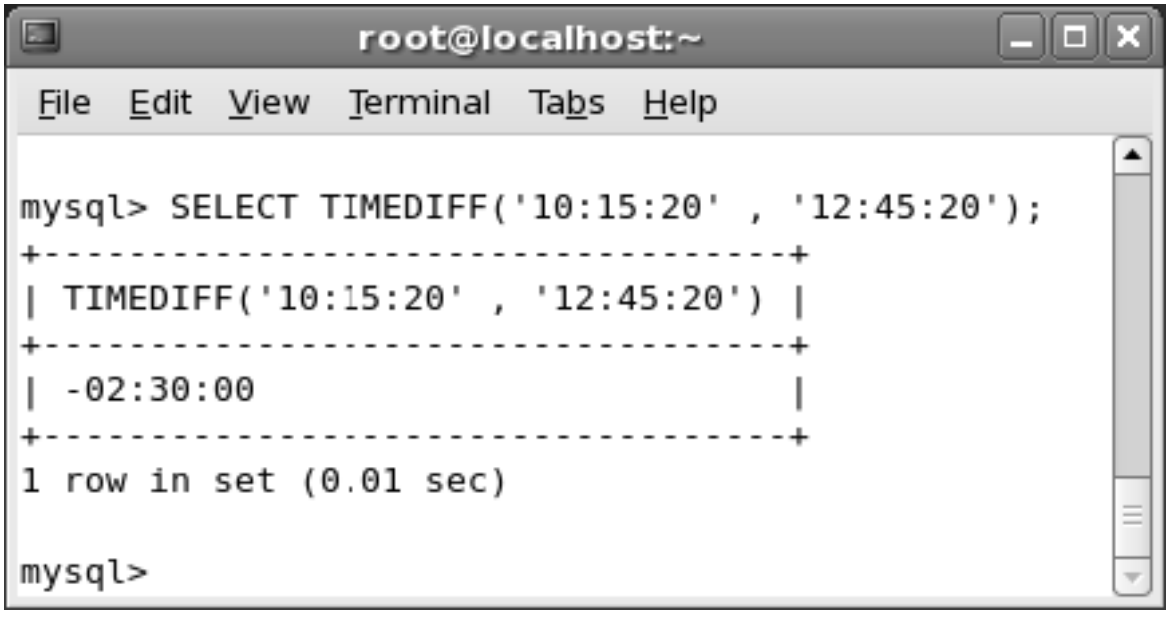


Figure 14.20: TIMEDIFF Function

14.2.21 WEEK Function

The `WEEK` function returns the week number for the value specified in the argument. The syntax for this function is:

```
SELECT WEEK(date, mode);
```

where,

`WEEK` – displays the week number

`date` – specifies the date

`mode` – defines the return value. The return value can be within a range of `1 to 53` or `0 to 53` depending on whether the week starts on Monday or Sunday

Note: When you do not specify the mode, value for the default week format is used.

For example, to retrieve the week number for `1985-07-19`, enter the following command at the command prompt:

```
SELECT WEEK('1985-07-19', 0);
```

Figure 14.21 displays the output of the command.

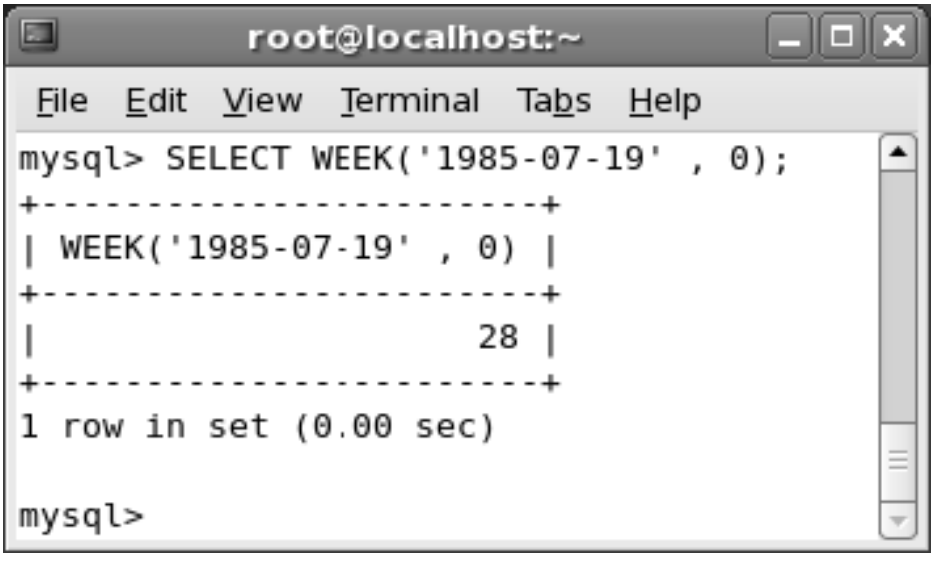


Figure 14.21: WEEK Function

14.2.22 WEEKOFYEAR Function

The `WEEKOFYEAR` function returns the week number for the specified date. The output of this function is a numeric value ranging between 1 to 53. The syntax for this function is:

```
SELECT WEEKOFYEAR(expression);
```

For example, to calculate the week number for the date `2005-04-25`, enter the following command at the command prompt:

```
SELECT WEEKOFYEAR('2005-04-25');
```

Figure 14.22 displays the output of the command.

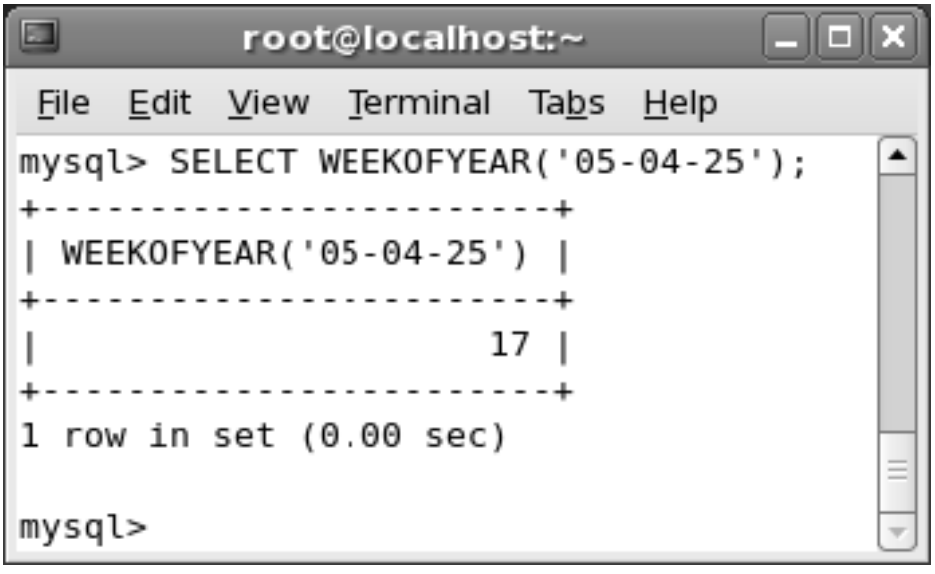


Figure 14.22: WEEKOFYEAR Function

14.2.23 YEAR Function

The `YEAR` function returns the year for the date specified in the argument. The syntax for obtaining the year from a date argument is:

```
SELECT YEAR(date);
```

For example, to display the year from the date `2000-09-09`, enter the following command at the command prompt:

```
SELECT YEAR('2000-09-09');
```

Figure 14.23 displays the output of the command.

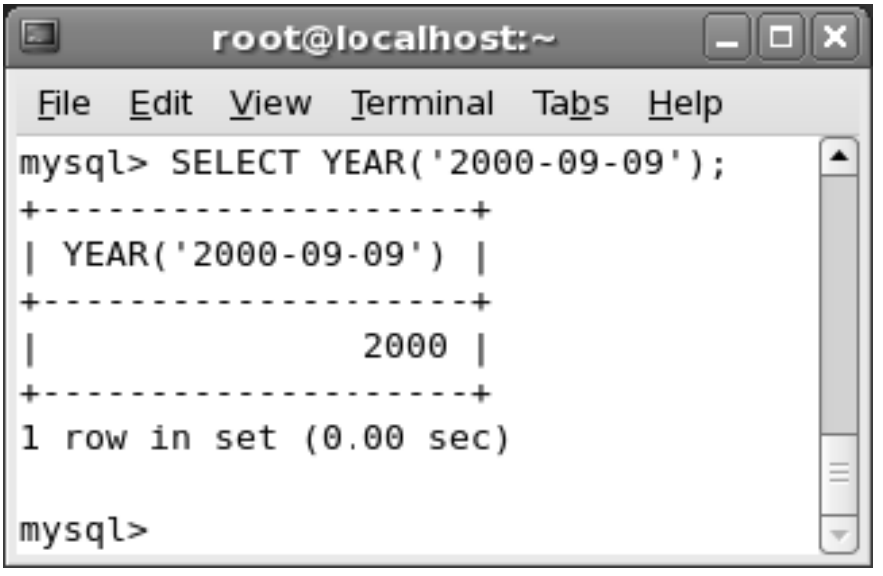


Figure 14.23: YEAR Function

Table 14.2 describes the additional Date functions in MySQL.

Name	Description	Example
CONVERT_TZ	The CONVERT_TZ function changes the time zone of an argument. The syntax for using this function is: SELECT CONVERT_TZ(dt, from_tz, to_tz); where, CONVERT – edits the time zone dt – specifies the date in the current time zone from_tz – specifies the current time zone of the argument to_tz – specifies the new time zone	To convert the time zone for the date 2004-01-01, enter the following command at the command prompt: SELECT CONVERT_TZ('2004-01-01 12:00:00', '+00:00', '+10:00'); The output of this function is: 2004-01-01 22:00:00
CURRENT_TIMESTAMP	The CURRENT_TIMESTAMP function displays the current date and time as a value in YYYY-MM-DD HH:MM:SS or YYYYMMDDHHMMSS.uuuuuu format. The value is expressed in	For example, to view the current timestamp, enter the following command at the command prompt:

Name	Description	Example
CURRENT_TIMESTAMP	the current time zone. The syntax to view the current timestamp is: `SELECT CURRENT_TIMESTAMP();`	`SELECT CURRENT_TIMESTAMP();` The output of this function will be the current date and time.
DATE_SUB	The `DATE_SUB` function subtracts a specified time interval from the given date. The syntax for this function is: `SELECT DATE_SUB(date, INTERVAL expr unit);` where, `DATE_SUB` – subtracts the time `date` – defines the starting date or datetime value `INTERVAL expr` – specifies the interval to be subtracted from the given date	To subtract `1` day from the current date, enter the following command at the command prompt: `SELECT DATE_SUB(CURDATE(), INTERVAL 1 DAY);` The output of this function will be yesterday's date.

DAYOFMONTH	The `DAYOFMONTH` function returns the day of the month for a date. The output of this function is a numeric value between 1 and 31. The syntax for obtaining the day of the month is:
	
	`SELECT DAYOFMONTH(date);`
		For example, to extract the day of the month for the date `2005-03-18`, enter the following command at the command prompt:
		
		`SELECT DAYOFMONTH('2005-03-18');`
		
		The output of this function is:
		
		`18`

DAYOFWEEK	The `DAYOFWEEK` function returns the day of the week for a date. The output for this function is a numeric value between 1 and 7, where 1 is Sunday and 7 is Saturday. The syntax to display the day of the week is:
	
	`SELECT DAYOFWEEK(date);`
		For example, to display the day of the week for a particular date, enter the following command at the command prompt:
		
		`SELECT DAYOFWEEK('1995-06-07');`
		
		The output of this function is:
		
		`4`

Name	Description	Example

DAYOFYEAR	The DAYOFYEAR function returns the day of the year for a date. The output of this function is a numeric value between 1 and 366. The syntax for obtaining the day of the month is:
	
	`SELECT DAYOFYEAR(date);`
		For example, to display the day of the year for a particular date, enter the following command at the command prompt:
		
		`SELECT DAYOFYEAR('2001-10-21');`
		
		The output of this function is:
		
		`294`

EXTRACT	The EXTRACT function extracts a particular part from a given date. The syntax for this function is:
	
	`SELECT EXTRACT(unit from date);`
	
	where,
	
	`date` – defines a valid date expression
	
	`unit` – specifies any valid unit, such as year, month, second, minute, or day
		For example, to display only the year from a particular date, enter the following command at the command prompt:
		
		`SELECT EXTRACT(YEAR FROM '2008-04-06');`
		
		The output of this function is:
		
		`2008`

FROM_DAYS	The FROM_DAYS function returns the date of the number specified as an argument. This function works only for dates after the advent of the Gregorian calendar (1582). The syntax for this function is:
	
	`SELECT FROM_DAYS(argument);`
		For example, to display the date for `5,00,000`, enter the following command at the command prompt:
		
		`SELECT FROM_DAYS(500000);`
		
		The output of this function is:
		
		`1368-12-14`

LAST_DAY	The LAST_DAY function returns the last date of the month for the argument. The output is a numeric value ranging from 1 to 31, if the argument is valid. However, if the argument is invalid, the output is NULL. The syntax for using this function is:
	
	`SELECT LAST_DAY(expression);`
		For example, enter the following command at the command prompt:
		
		`SELECT LAST_DAY('2008-02-05');`
		
		The output of this function is:
		
		`2008-02-29`

Name	Description	Example
MICROSECOND	The MICROSECOND function returns the number of microseconds in the argument specified as a datetime value. The output of this function is a numeric value ranging from 0 to 999999. The syntax for this function is: SELECT MICROSECOND(expression);	For example, to display the microseconds from the datetime expression 11:45:45.000123, enter the following command at the command prompt: SELECT MICROSECOND ('11:45:45.000123'); The output of this function is: 123
PERIOD_ADD	The PERIOD_ADD function adds a specified number of months to a given period expressed either as YYMM or YYYYMM. The final output is displayed in the form of YYYYMM. The syntax for this function is: SELECT PERIOD_ADD(P, N); where, PERIOD_ADD – adds the specified number of months P – defines the specified period N – defines the number of months to be added	For example, to add two months to the period 200705, enter the following command at the command prompt: SELECT PERIOD_ADD('200705', 02); The output of this function is: 200707
PERIOD_DIFF	The PERIOD_DIFF function calculates the difference in number of months between two periods. The final output is a numeric value. The syntax for this function is: SELECT PERIOD_DIFF(P1, P2); where, PERIOD_DIFF – calculates the number of months between two periods	For example, to calculate the number of months between 199608 and 199503, enter the following command at the command prompt: SELECT PERIOD_DIFF('199608', '199503'); The output of this function is: 17

Name	Description	Example
PERIOD_DIFF	P1 - stands for Period1 P2 - stands for Period2	
QUARTER	The QUARTER function returns the quarter number for the argument specified as a date value. The output of this function is a numeric value ranging from 1 - 4. The syntax for this function is: `SELECT QUARTER(expression);`	For example, to calculate the quarter for the date 1987-12-24, enter the following command at the command prompt: `SELECT QUARTER('1987-12-24');` The output of this function is: 4
STR_TO_DATE	The STR_TO_DATE function converts a string into a specified format. The output of this function can be a date, time, or a datetime value depending upon the format. The syntax of this function is: `SELECT STR_TO_DATE(String, format);`	To convert the string 01,05,2013 into the YYYY-MM-DD format, enter the following command at the command prompt: `SELECT STR_TO_DATE('01,05,2013','%m, %d, %Y');` The output of this function is: 2013-01-05
SUBTIME	The SUBTIME function subtracts a specified time interval from the given datetime or time value. The syntax for this function is: `SELECT SUBTIME(expr1, expr2);` where, SUBTIME – subtracts a specified time interval expr1 – defines the starting datetime or time value expr2 – specifies the time value to be subtracted	To subtract 45 minutes from the current date, enter the following command at the command prompt: `SELECT SUBTIME(NOW(),'00:45:00');` The output of this function will be a timestamp of 45 minutes prior to the current time.

Name	Description	Example
TIME_FORMAT	The TIME_FORMAT function displays the time in a specific format. The syntax for this function is: `SELECT TIME_FORMAT(time, format);` where, `time` – defines a valid time `format` – defines the output to display the time	To display the current time in the HH-MM-SS format, enter the following command at the command prompt: `SELECT TIME_FORMAT(NOW(), '%H-%I-%S');` The output of this function will be the current time in HH-MM-SS format.

TIME_TO_SEC	The TIME_TO_SEC function converts a time argument into seconds. The output of this function is a numeric value and the syntax for this function is:
	
	`SELECT TIME_TO_SEC(expression);`
		To convert the time expression 23:45:02 into seconds, enter the following command at the command prompt:
		
		`SELECT TIME_TO_SEC('23:45:02');`
		
		The output of this function is:
		
		`85502`

TIMESTAMP	The TIMESTAMP function returns the datetime expression of the specified argument. When you specify only a single argument, the output is a datetime value of only that argument. If you specify two arguments, the output would be the sum of the two expressions. The syntax for this function is: `SELECT TIMESTAMP(expression);`	For example, to display the current timestamp value, enter the following command at the command prompt: `SELECT TIMESTAMP(NOW());` The output of this function will be the current date and time.
TO_DAYS	The TO_DAYS function converts a datetime value into a numeric value, which is the day number. It starts counting the number of days after the advent of the Gregorian calendar (1582). The syntax for this function is: `SELECT TO_DAYS(expression);`	For example, to calculate the day number for the date 1982-06-24, enter the following command at the command prompt: `SELECT TO_DAYS('1982-06-24');`

Name	Description	Example
TO_DAYS		The output of this function will be: 724085
TO_SECONDS	The TO_SECONDS function converts the argument into seconds. The argument is a datetime value and the output is a numeric value. The syntax for this function is: SELECT TO_SECONDS(expression);	To convert 1975-02-25 into seconds, enter the following command at the command prompt: SELECT TO_SECONDS('1975-02-25'); The output of this function is: 62329737600
UTC_DATE	The UTC_DATE function returns the current UTC date as a value in YYYY-MM-DD or YYYYMMDD format, depending on whether the function is used in a string or numeric context. The syntax for this function is: SELECT UTC_DATE(expression);	For example, to retrieve the current UTC date, enter the following command at the command prompt: SELECT UTC_DATE(); The output of this function will be the current UTC date.
UTC_TIME	The UTC_TIME function returns the current UTC time as a value in HH:MM:SS or HHMMSS.uuuuuu format, depending on whether the function is used in a string or numeric context. The syntax for this function is: SELECT UTC_TIME(expression);	For example, to retrieve the current UTC time, enter the following command at the command prompt: SELECT UTC_TIME(); The output of this function will be the current UTC time.
UTC_TIMESTAMP	The UTC_TIMESTAMP function returns the current UTC date and time as a value in YYYY-MM-DD HH:MM:SS or YYYYMMDDHHMMSS.uuuuuu format, depending on whether the function is used in a string or numeric context. The syntax for this function is: SELECT UTC_TIMESTAMP(expression);	For example, to retrieve the current UTC date and time, enter the following command at the command prompt: SELECT UTC_TIMESTAMP(); The output of this function will be the current date and time.
WEEKDAY	The WEEKDAY function returns the weekday index for the argument. The output of this function is a numeric value between 0 to 6	The output of this function will be a numerical value corresponding to the weekday.

Name	Description	Example
WEEKDAY	where, 0 represents Monday and 6 represents Sunday. The syntax for this function is: `SELECT WEEKDAY(expression);`	
YEARWEEK	The YEARWEEK function returns the year and the week number of the date specified in the argument. The syntax for this function is: `SELECT YEARWEEK(expression);`	For example, to retrieve the year and week for the date 1982-12-29, enter the following command at the command prompt: `SELECT YEARWEEK('1982-12-29');` The output of this function is: 198252

Table 14.2: Additional Date Functions in MySQL

14.3 String Functions in MySQL

MySQL provides string functions that work with the string data type. The string functions consider the specified arguments as strings even if you enter numeric data.

14.3.1 CHAR Function

The CHAR function interprets the values specified in the argument as integers. This function returns a string that contain characters returned by the code value of the integers. If the arguments contain NULL values, then they are skipped. The syntax to retrieve the string of characters is:

```
SELECT CHAR(N,...);
```

For example, to retrieve the code value for the integers 150, 120, 83, 81, '76', enter the following command at the command prompt:

```
SELECT CHAR(150, 120, 83, 81, '76');
```

Figure 14.24 displays the output of the command.

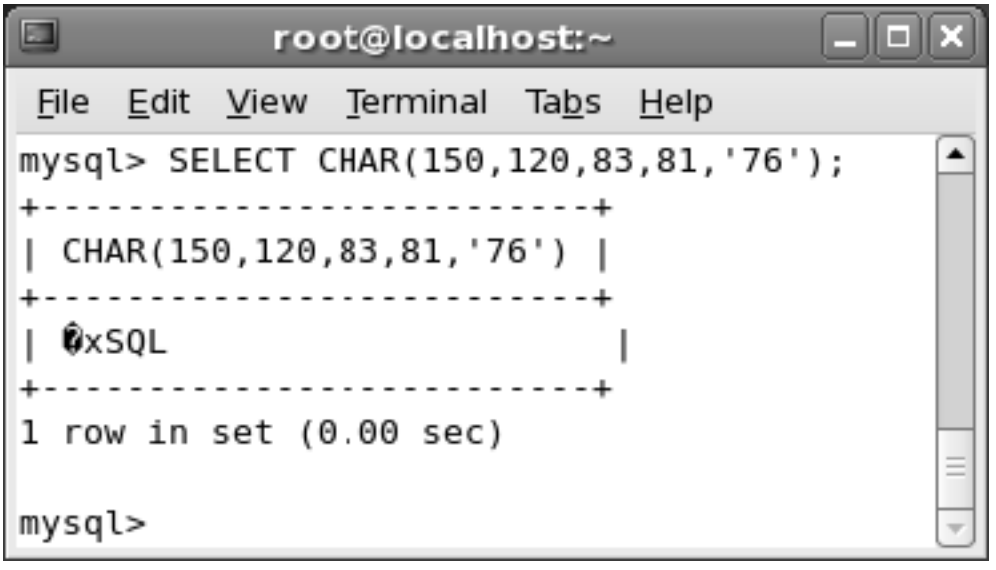


Figure 14.24: CHAR Function

14.3.2 CHARACTER_LENGTH Function

The `CHARACTER_LENGTH` function also known as `CHAR_LENGTH` function returns the number of characters present in the string. This function counts a multibyte character as a single character. The syntax for this function is:

```
SELECT CHARACTER_LENGTH(expression);
```

For example, to calculate the number of characters present in the string `abc`, enter the following command at the command prompt:

```
SELECT CHARACTER_LENGTH('abc');
```

Figure 14.25 displays the output of the command.

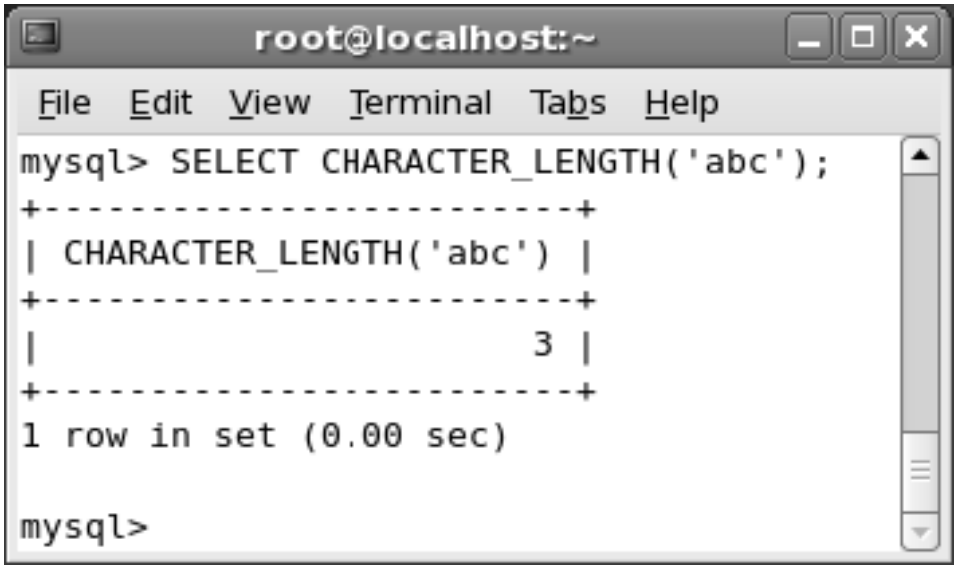


Figure 14.25: CHARACTER_LENGTH Function

14.3.3 CONCAT Function

The `CONCAT` function returns a string after joining the specified arguments. This function returns a `NULL` value if `NULL` arguments are specified. The syntax for this function is:

```
SELECT CONCAT(STR1,STR2,...);
```

To concatenate the strings `My`, `SQL`, `DBMS`, enter the following command at the command prompt:

```
SELECT CONCAT('My','SQL',' ','DBMS');
```

Figure 14.26 displays the output of the command.

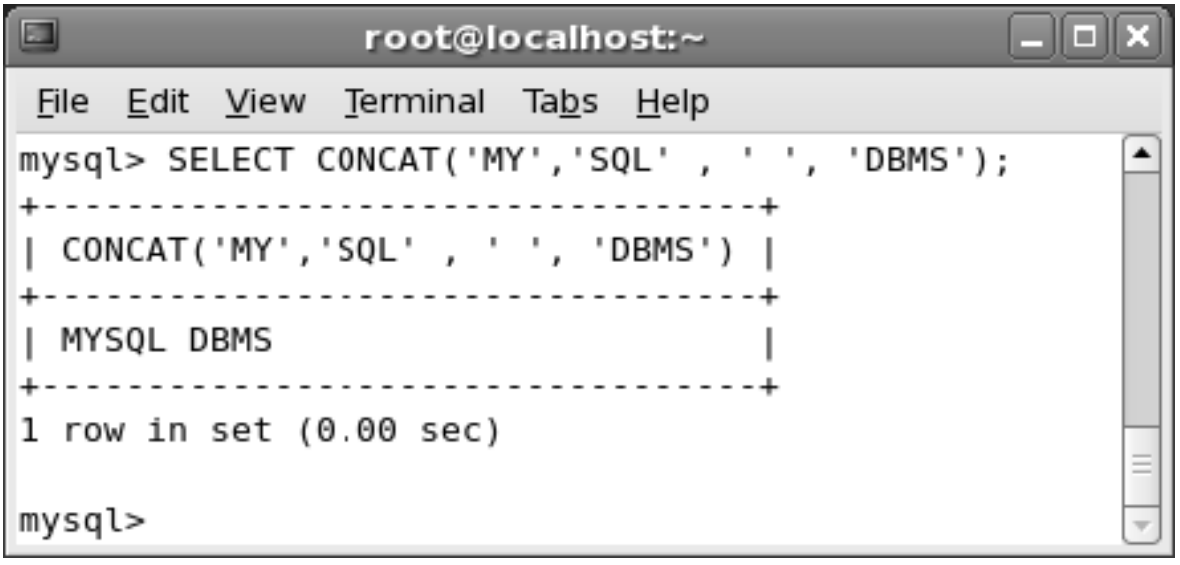


Figure 14.26: CONCAT Function

14.3.4 INSERT Function

The `INSERT` function adds a new string to the existing string at a specified position. The output of the function is also a string. The syntax for the `INSERT` function is:

```
INSERT (str, pos, len, newstr);
```

where,

`INSERT` – adds a new string

`str` – specifies the existing string

`pos` – defines the starting position of the new string

`len` – specifies the length of the new string

`newstr` – specifies the new string to be inserted

This function returns a string where the substring starting at position, `pos`, and of length, `len`, is replaced by the string `newstr`.

To insert MATHEMATICAL at position 6 of the string MATHE, enter the following command at the command prompt:

```
SELECT INSERT('MATHE', 6, 12, 'MATICAL');
```

Figure 14.27 displays the output of the command.

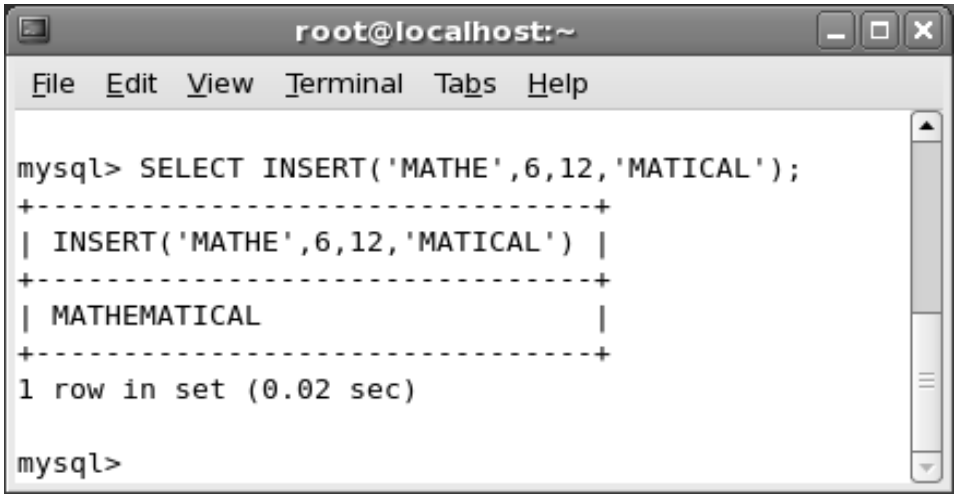


Figure 14.27: INSERT Function

14.3.5 INSTR Function

The INSTR function works with two arguments, such as string and substring. It returns the position of the first instance of a substring in a string. The output of this function is a numeric value ranging from 0 to the length of the string. The syntax for this function is:

```
SELECT INSTR(string, substring);
```

For example, to retrieve the position of the string cd from the string abcd, enter the following command at the command prompt:

```
SELECT INSTR('abcd', 'cd');
```

Figure 14.28 displays the output of the command.

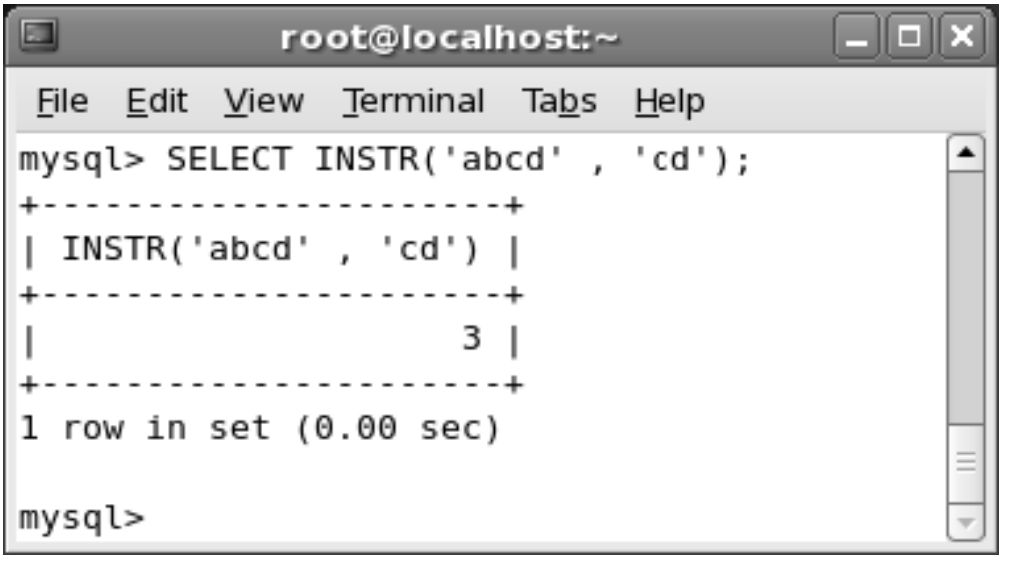


Figure 14.28: INSTR Function

14.3.6 LCASE Function

The `LCASE` function changes all the characters of the values specified in the argument to lowercase. This function does not work with binary strings. The syntax for this function is:

```
SELECT LCASE(expression);
```

To convert `MYSQL` to lowercase, enter the following command at the command prompt:

```
SELECT LCASE('MYSQL');
```

Figure 14.29 displays the output of the command.

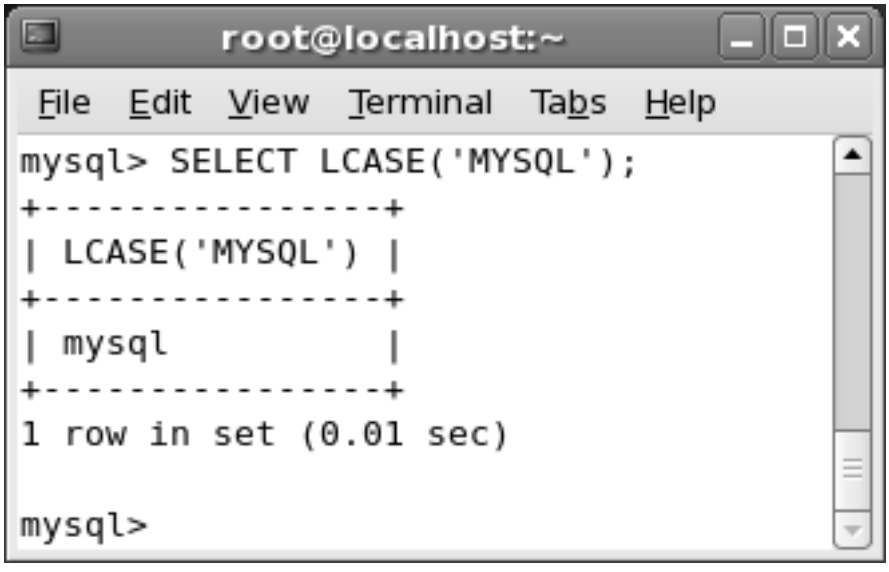


Figure 14.29: LCASE Function

14.3.7 LEFT Function

The `LEFT` function returns the leftmost characters of the values specified in the argument. The syntax for this function is:

```
SELECT LEFT('str', len);
```

where,

`LEFT` – returns the leftmost character

`str` – contains the original string

`len` – specifies the number of characters to be extracted from the leftmost end

For example, to obtain the first four character of the string `DATABASE`, enter the following command at the command prompt:

```
SELECT LEFT('DATABASE' , 4);
```

Figure 14.30 displays the output of the command.

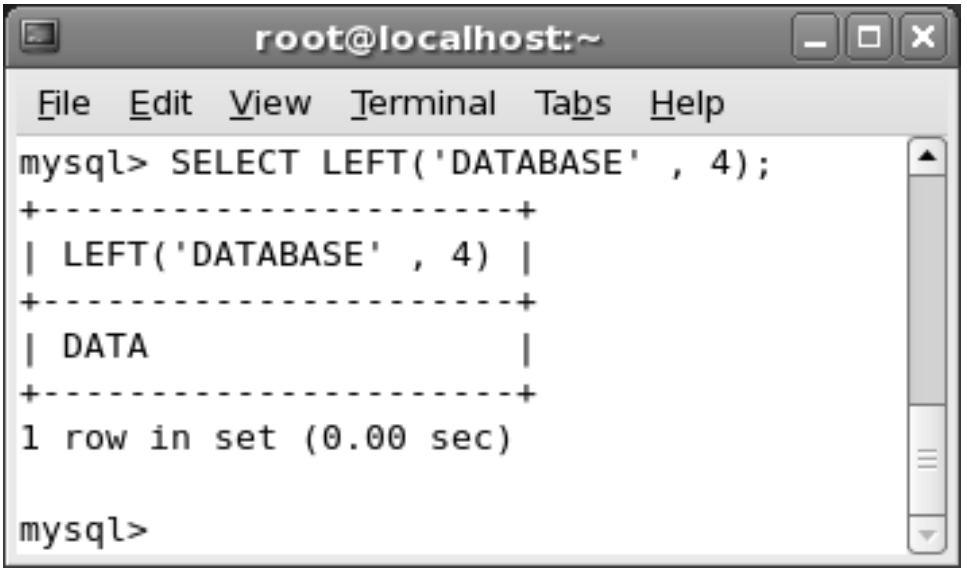


Figure 14.30: LEFT Function

14.3.8 LENGTH Function

The `LENGTH` function returns the length of a string. The syntax to calculate the length of a string is:

```
SELECT LENGTH(str);
```

For example, to calculate the length of the string `MYSQL`, enter the following command at the command prompt:

```
SELECT LENGTH('MYSQL');
```

Figure 14.31 displays the output of the command.

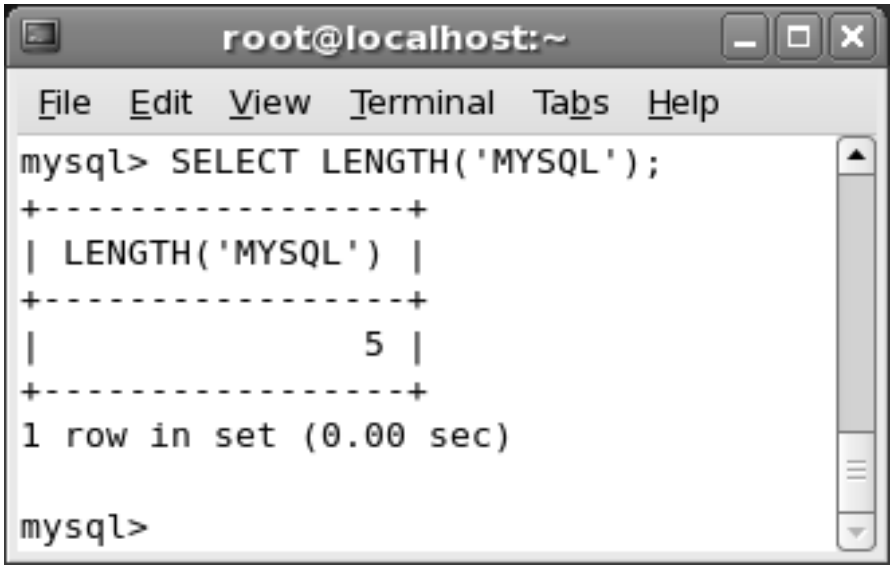


Figure 14.31: LENGTH Function

14.3.9 LOCATE Function

The `LOCATE` function returns the position of the first instance of a substring in a string. The syntax to retrieve the position of the first instance of a substring in a string is:

```
SELECT LOCATE(substr, str);
```

For example, to identify the position of the substring math in the string `mathematics`, enter the following command at the command prompt:

```
SELECT LOCATE('math', 'mathematics');
```

Figure 14.32 displays the output of the command.

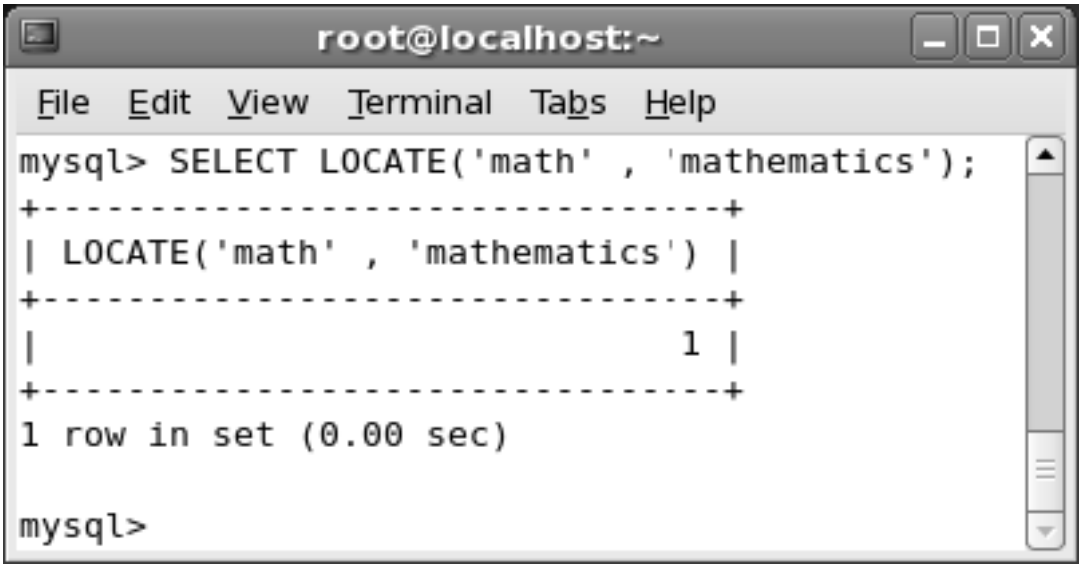


Figure 14.32: LOCATE Function

14.3.10 REPEAT Function

The `REPEAT` function copies the given string N number of times. If N is less than 1, the output is an empty string. The syntax for this function is:

```
SELECT REPEAT(String, count);
```

where,

`REPEAT` – copies the string

`String` – specifies the string to copy

`count` – specifies the number of times to copy the string

To replicate the string `DATA` four times, enter the following command at the command prompt:

```
SELECT REPEAT('DATA', 4);
```

Figure 14.33 displays the output of the command.

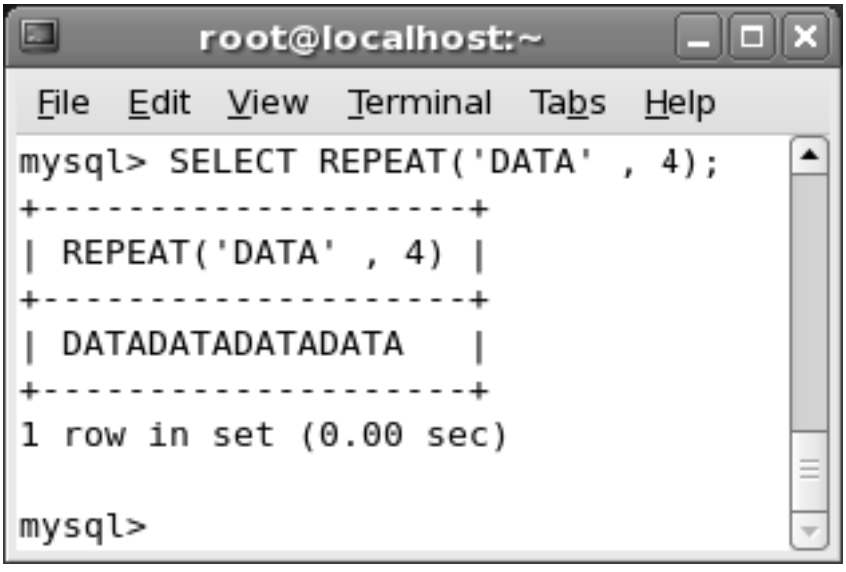


Figure 14.33: REPEAT Function

14.3.11 REPLACE Function

The `REPLACE` function modifies a part or whole of the string with the new string. The syntax for this function is:

```
SELECT REPLACE(str, from_str, to_str);
```

where,

`REPLACE` – modifies the string

`str` – specifies the original string

`from_str` – defines the string to be replaced

`to_str` – defines the new string to be inserted

Note: The REPLACE function is case-sensitive.

To change the string `DATA` to `BETA`, enter the following command at the command prompt:

```
SELECT REPLACE('DATA', 'DA', 'BE');
```

Figure 14.34 displays the output of the command.

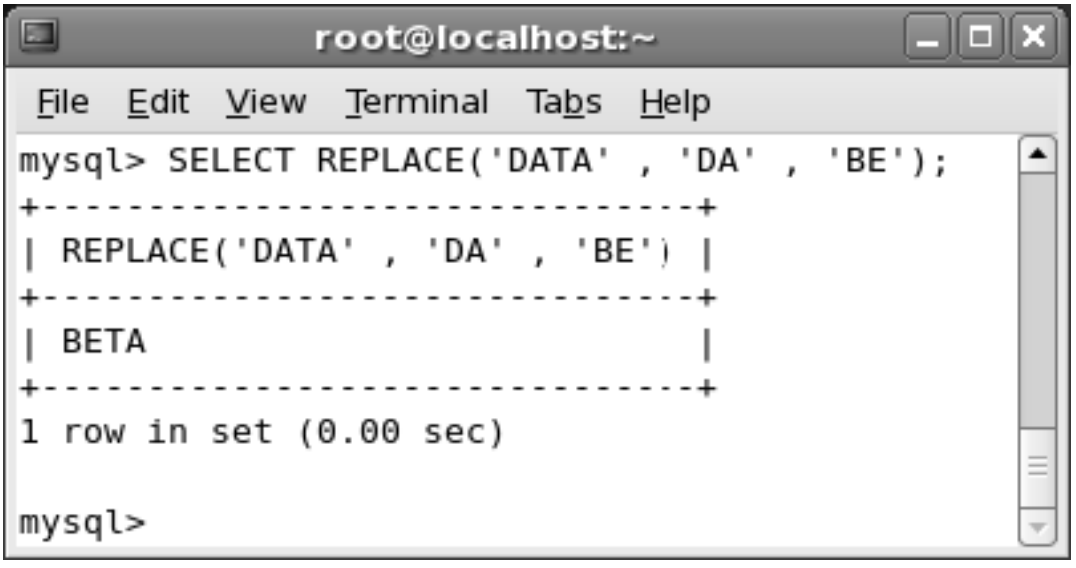


Figure 14.34: REPLACE Function

14.3.12 REVERSE Function

The `REVERSE` function inverts the sequence of characters in an argument. The syntax for this function is:

```
SELECT REVERSE(expression);
```

To change the order of characters of the string `XYZ`, enter the following command at the command prompt:

```
SELECT REVERSE('XYZ');
```

Figure 14.35 displays the output of the command.

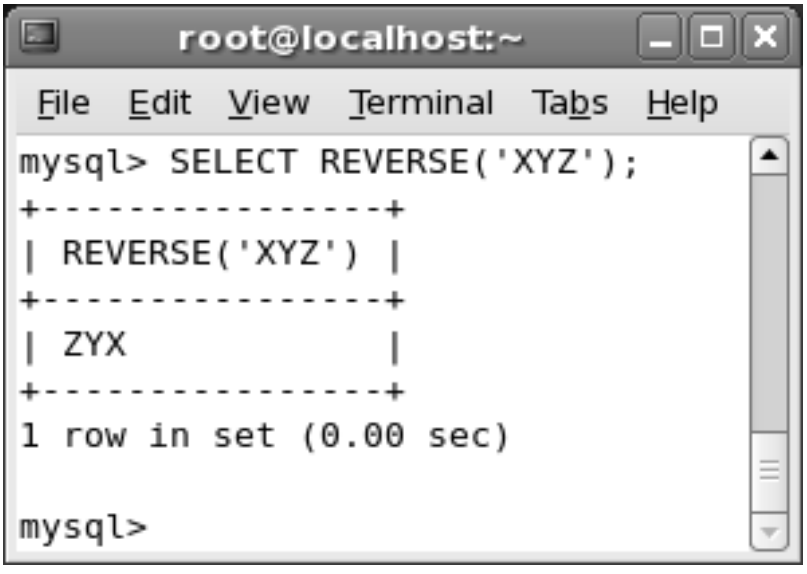


Figure 14.35: REVERSE Function

14.3.13 RIGHT Function

The `RIGHT` function accepts two arguments, such as string and length. The function returns an output that contains string values starting from the right end of the input string. The syntax for this function is:

```
SELECT RIGHT(string, length);
```

where,

`string` – defines the given string

`length` – specifies the number of rightmost characters

For example, to obtain the last four characters of the string `database`, enter the following command at the command prompt:

```
SELECT RIGHT('database', 4);
```

Figure 14.36 displays the output of the command.

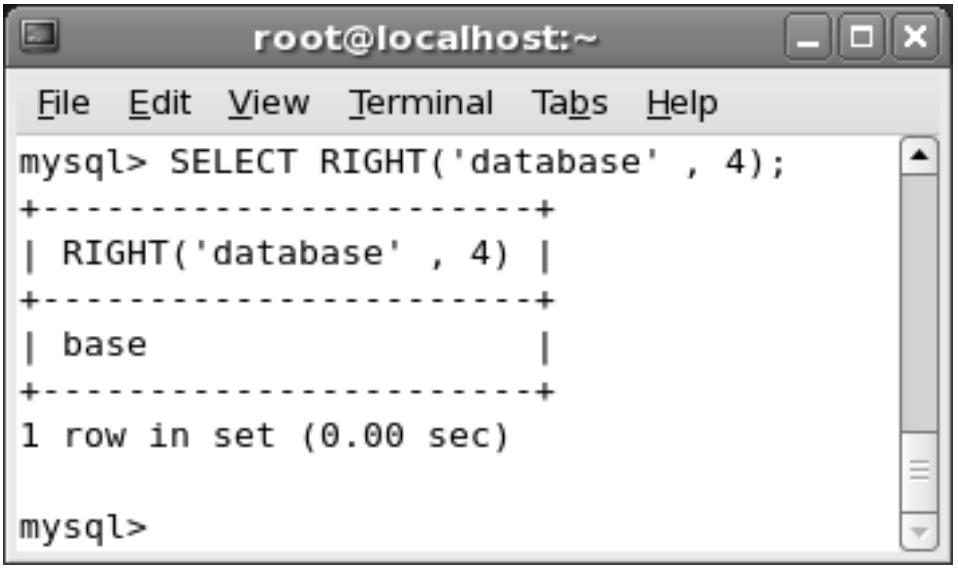


Figure 14.36: RIGHT Function

14.3.14 STRCMP Function

The `STRCMP` function accepts two strings as arguments. It compares the strings and returns a numeric value ranging from -1 to 1 in the output. The function returns 0 if the strings have same values. The output of the function is -1 when the first argument is smaller than the second based on the conditions specified in the sort order. This function returns 1 when the first argument is greater than the second based on the conditions specified in the sort order. The syntax for this function is:

```
SELECT STRCMP(str1, str2);
```

where,

`str1`, `str2` – specify the strings to compare

If `str1 = str2`, the output is 0.

If `str1 < str2`, the output is -1.

If `str1 > str2`, the output is 1.

To compare two strings `data` and `database`, enter the following command at the command prompt:

```
SELECT STRCMP('data', 'database');
```

Figure 14.37 displays the output of the command.

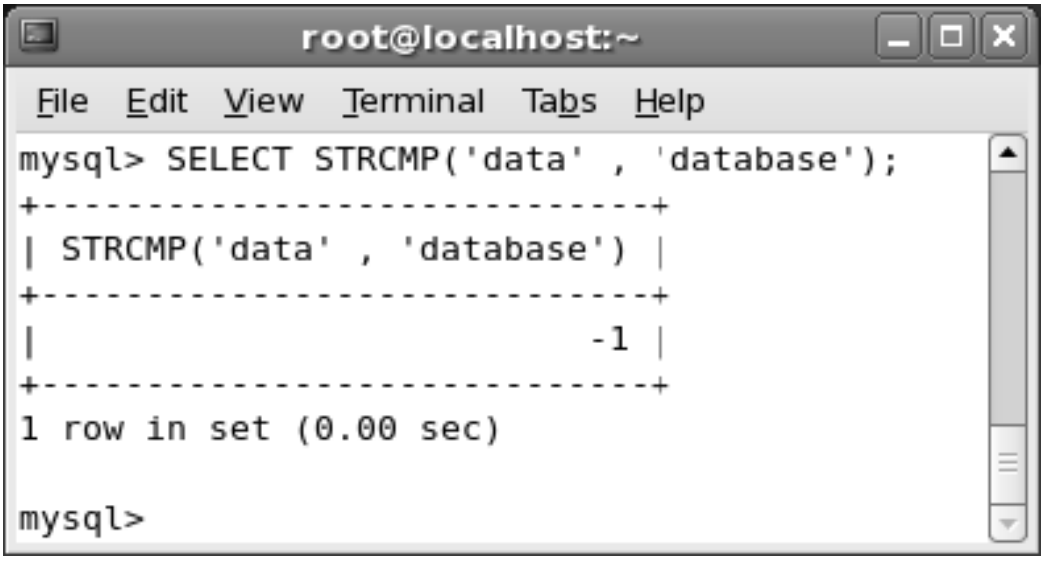


Figure 14.37: STRCMP Function

14.3.15 TRIM Function

The `TRIM` function removes a specified prefix or suffix from a given string. The syntax for this function is:

```
SELECT TRIM(location 'remstring' from 'string');
```

where,

`location` – specifies the starting position for deletion

`remstring` – specifies the string to be removed

`string` – specifies the original string

To remove the string `quant` from the string `quantitative`, enter the following command at the command prompt:

```
SELECT TRIM(LEADING 'quant' from 'quantitative');
```

Figure 14.38 displays the output of the command.

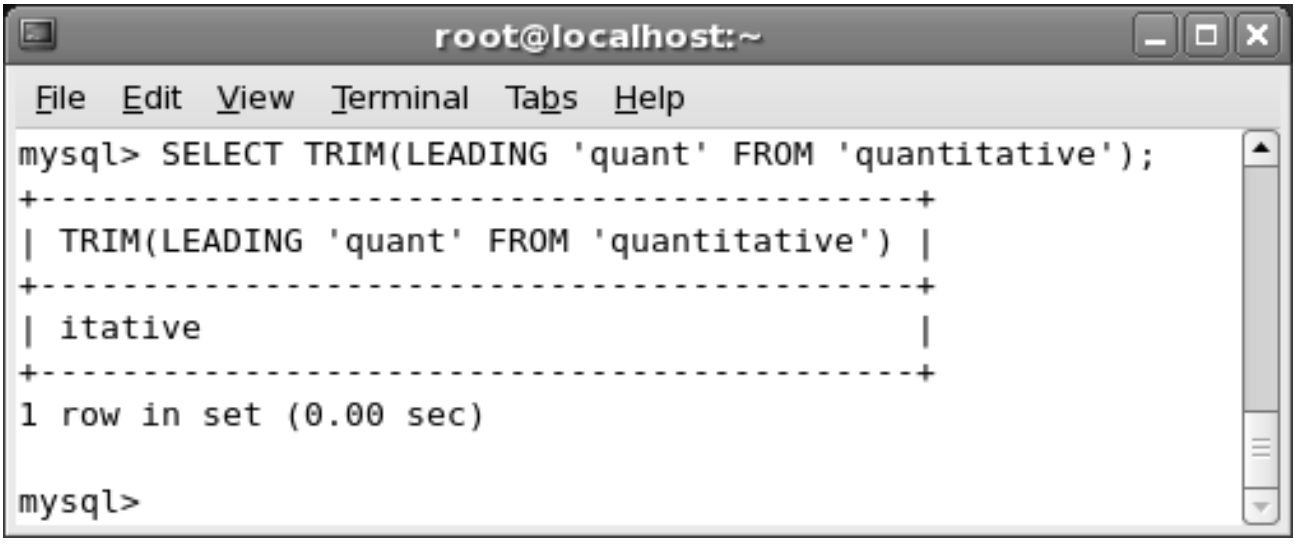


Figure 14.38: TRIM Function

Note: In the syntax, if the location is not specified, then the string to be removed is deleted from both ends. If no remstring is specified, then whitespaces are removed.

14.3.16 UCASE Function

The `UCASE` function changes all the characters of the argument into uppercase. This function does not work with binary strings. The syntax for this function is:

```
SELECT UCASE(expression);
```

To convert `database` to uppercase, enter the following command at the command prompt:

```
SELECT UCASE('database');
```

Figure 14.39 displays the output of the command.

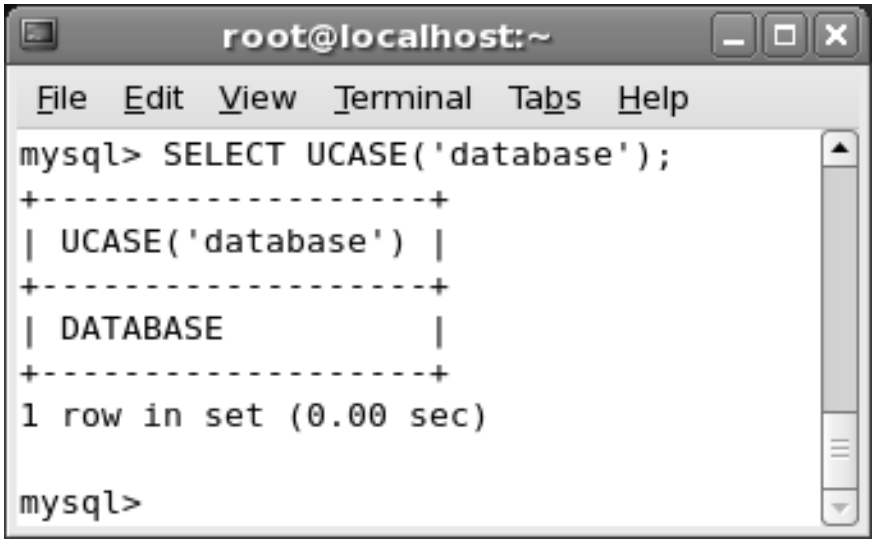


Figure 14.39: UCASE Function

Table 14.3 describes the additional String functions in MySQL.

Name	Description	Example
ASCII	The ASCII function returns the numeric value of the leftmost character of the string, specified as the argument. This function returns a value of 0 if you enter an empty string. This function is applicable for characters whose numeric value ranges from 0 to 255. The syntax to retrieve the ASCII value is: SELECT ASCII(string);	For example, to retrieve the ASCII value of the string JOHN, enter the following command at the command prompt: SELECT ASCII('JOHN'); The output of this function is: 74
BIN	The BIN function returns a string representation of the binary value of N, where N contains values of BIGINT data type. The syntax to retrieve the string representation is: SELECT BIN(expression);	For example, to retrieve the binary value of 55, enter the following command at the command prompt: SELECT BIN(55); The output of this function is: 110111
BIT_LENGTH	The BIT_LENGTH function returns the length of the string specified in the argument. The syntax to calculate the length of the string in bits is:	For example, to calculate the length of the string MYSQL in bits, enter the following command at the command prompt:

Name	Description	Example
BIT_LENGTH	SELECT BIT_LENGTH(string);	SELECT BIT_LENGTH('MYSQL'); The output of this function is: 40
COMPRESS	The COMPRESS function compresses a string. The syntax to compress a string is: SELECT COMPRESS(string); The compressed string outputs are stored as follows: ➢ Empty strings are stored as it is. ➢ Non-empty strings are stored as a four-byte length of the uncompressed string (low byte first), followed by the compressed string. ➢ If the string contains a space at the end, the space is replaced by the '.' character to resolve trim errors while storing data in a CHAR or VARCHAR column.	To compress the string XYZ and obtain the length of the compressed string, enter the following command at the command prompt: SELECT LENGTH(COMPRESS('XYZ')); The output of this function is: 15
CONCAT_WS	The CONCAT_WS function is similar to the CONCAT() function. However, it allows you to add a separator between the arguments. The syntax for this function is: CONCAT_WS(separator, str1, str2, ...);	To join the strings My and SQL with a comma as the separator, enter the following command at the command prompt: SELECT CONCAT_WS(',',My','SQL'); The output of this function is: MY,SQL
ELT	The ELT function extracts the value of the string whose position is specified in the argument. The position is a numeric value. The syntax for using this function is:	Enter the following command at the command prompt: SELECT ELT(1, 'a', 'b', 'c');

Name	Description	Example
ELT	SELECT ELT(N, str1, str2, ...); where, N - specifies the numeric position of the value to be searched str1, str2 - defines the strings passed as arguments	The output of this function is: a **Note:** For the ELT function, the numeric value specified must be between 1 and the number of strings present in the given argument, else the output is NULL.
FIELD	The FIELD function returns the position of a particular string in a given set of strings specified in the argument. If the string is not found, this function returns a value of zero. The syntax for this function is: SELECT FIELD(string1,string2,string3....); where, string1 – represents the string to search for string2, string3 – list of strings to be searched from	To find the index of string C in the strings A, B, C, D, and E, enter the following command at the command prompt: SELECT FIELD('C', 'A', 'B', 'C', 'D', 'E'); The output of this function is: 3
FIND_IN_SET	The FIND_IN_SET function returns the position of the character present in the string list passed as an argument. A string list contains strings that are separated by a comma. The output of this function is a numeric value between 0 and N, where N is the number of substrings in the string list. If either of the argument is NULL, the final result is also NULL. The syntax for this function is: SELECT FIND_IN_SET(expression);	For example, to retrieve the index value of B in the string B,Y,E, enter the following command at the command prompt: SELECT FIND_IN_SET('B', 'B,Y,E'); The output of this function is: 1
HEX	The HEX function displays the values specified in the argument in its hexadecimal format. The syntax for this function is:	For example, to retrieve the hexadecimal value of MYSQL, enter the following command at the command prompt:

Name	Description	Example
HEX	SELECT HEX(expression);	SELECT HEX('MYSQL'); The output of this function is: 4D5953514C
LOWER	The LOWER function changes the given string into lowercase. The syntax to change the case for a string is: SELECT LOWER(str);	For example, to convert the string PETER to lowercase, enter the following command at the command prompt: SELECT LOWER('PETER'); The output of this function is: peter
LTRIM	The LTRIM function removes the whitespaces before the values specified in the argument. The syntax for this function is: SELECT LTRIM(expression);	To delete the whitespaces before ' xyz', enter the following command at the command prompt: SELECT LTRIM(' xyz'); The output of this function is: xyz
ORD	The ORD function returns the ASCII value of the first character for the string specified in the argument. The arguments should be a string. The syntax for this function is: SELECT ORD(expression);	To display the ASCII value of the first character from the string BAR, enter the following command at the command prompt: SELECT ORD('BAR'); The output of this function is: 66
RTRIM	The RTRIM function deletes the spaces after the values specified in the argument. The syntax for this function is: SELECT RTRIM(expression);	To remove the trailing space from the string 'SPACE ', enter the following command at the command prompt: SELECT RTRIM('SPACE '); The output of this function is: SPACE

Name	Description	Example
SPACE	The SPACE function returns a string that contains only space characters. You can specify the number of space characters as a numeric value in the argument. The syntax for this function is: `SELECT SPACE(N);` where, N – defines the number of spaces	For example, to obtain a string with seven spaces, enter the following command at the command prompt: `SELECT SPACE(7);` The output of this function will be a string of 7 spaces.
UNHEX	The UNHEX function interprets each character of the string in the hexadecimal format and returns the character as the output. The syntax for this function is: `SELECT UNHEX(expression);`	For example, to obtain the characters for the hexadecimal string 'AB', enter the following command at the command prompt: `SELECT UNHEX('AB');` The output of this function is: ? **Note:** For the UNHEX function, if the argument is not in the hexadecimal format (0 to 9, A to F, a to f), then the final output is NULL.

Table 14.3: Additional String Functions in MySQL

14.4 System Information Functions in MySQL

System functions provide information about the system. MySQL provides certain system functions that return system-related information such as, connection ID, current users and so on.

14.4.1 CHARSET Function

The CHARSET function returns the character set of the argument. The argument entered must be a string argument. The syntax for obtaining the character set of a string is:

```
SELECT CHARSET(str);
```

For example, to obtain the character set of 'John', enter the following command at the command prompt:

```
SELECT CHARSET('JOHN');
```

Figure 14.40 displays the output of the command.

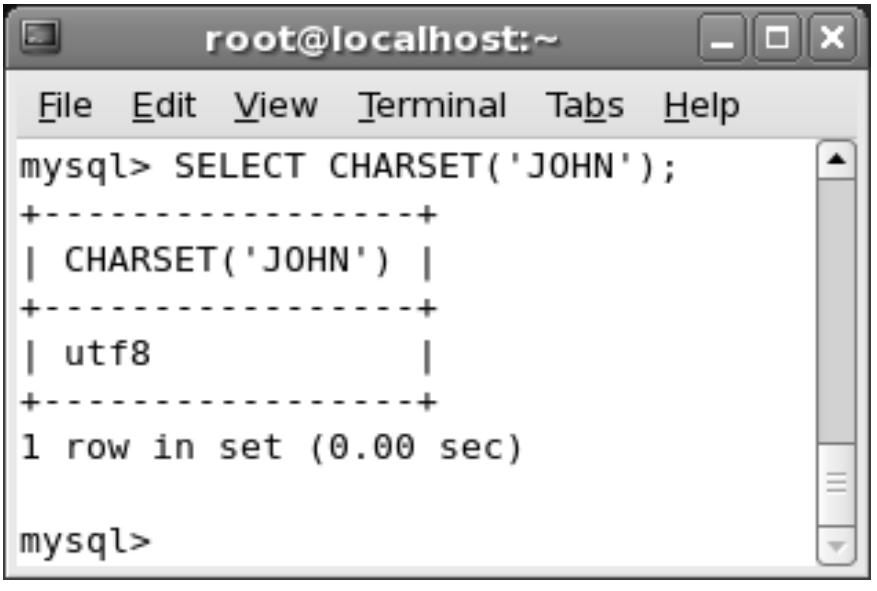


Figure 14.40: CHARSET Function

14.4.2 CURRENT_USER Function

The CURRENT_USER function returns the username and the hostname of the current session. The syntax to view the hostname and the username is:

```
SELECT CURRENT_USER();
```

For example, to obtain the username of the current account, enter the following command at the command prompt:

```
SELECT CURRENT_USER();
```

Figure 14.41 displays the output of the command.

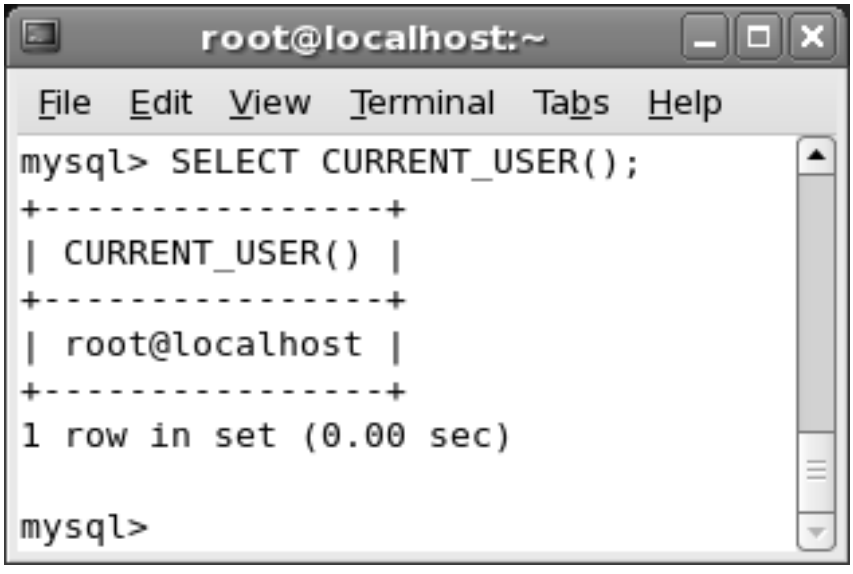


Figure 14.41: CURRENT_USER Function

14.4.3 DATABASE Function

The `DATABASE` function returns the name of the current database. If no database is activated, then this function returns a `NULL` value. The syntax to view the name of the current database is:

```
SELECT DATABASE();
```

For example, to retrieve the name of the current database, enter the following command at the command prompt:

```
SELECT DATABASE();
```

Figure 14.42 displays the output of the command.

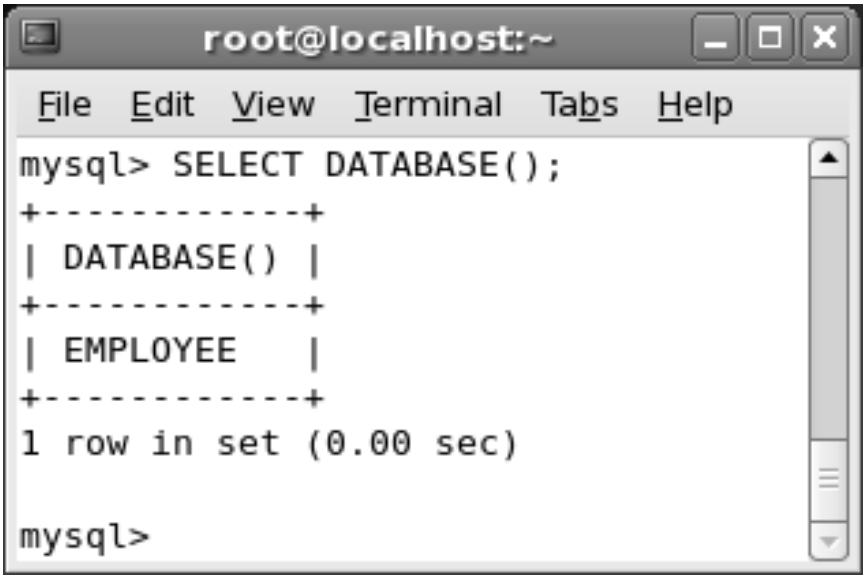


Figure 14.42: DATABASE Function

14.4.4 ROW_COUNT Function

The `ROW_COUNT` function returns the number of rows inserted, updated, or deleted after the `INSERT`, `UPDATE`, or `DELETE` statements are executed. The number of affected-rows returned in an `UPDATE` statement is the number of rows actually changed. In `REPLACE` statement, the number of affected rows is two if the new row replaces an old row, because, the `REPLACE` statement inserts one row after deleting the duplicate row. In case of an `INSERT` statement, the number of rows affected is one if a new row is inserted and the number of rows affected is two if an existing row is updated. By default, MySQL displays the row count stored in the `mysql_affected_rows()` function when the `ROW_COUNT()` function is executed.

The syntax for this function is:

```
SELECT ROW_COUNT();
```

Figure 14.43 displays the output of the command.

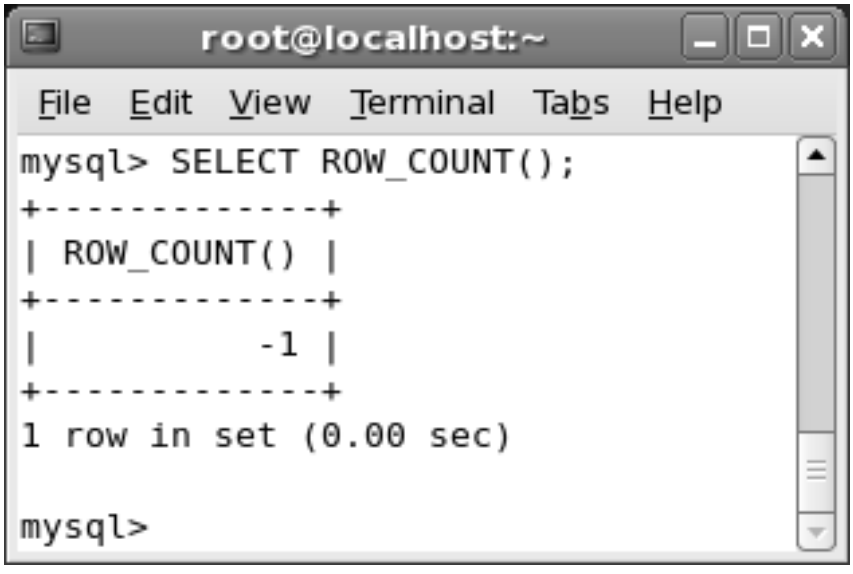


Figure 14.43: ROW_COUNT Function

14.4.5 VERSION Function

The `VERSION` function returns the version of MySQL. The syntax to view the version of MySQL is:

```
SELECT VERSION();
```

For example, to view the version of MySQL, enter the following command at the command prompt:

```
SELECT VERSION();
```

Figure 14.44 displays the output of the command.

Figure 14.44: VERSION Function

Table 14.4 displays the description and use of other system information functions.

Name	Description	Example
CONNECTION_ID	The CONNECTION_ID function returns the thread ID for the connection. MySQL assigns a unique ID for every thread or connection. The syntax to retrieve the thread ID is: SELECT CONNECTION_ID();	For example, to obtain the thread ID for the current connection, enter the following command at the command prompt: SELECT CONNECTION_ID(); The output of this function is: 2
ROW_COUNT	The ROW_COUNT function returns the number of rows inserted, updated, or deleted after the INSERT, UPDATE, or DELETE statements are executed. The syntax for this function is: SELECT ROW_COUNT();	Execute the following command and view the output. SELECT ROW_COUNT; The output of this function is: -1

Name	Description	Example
USER	The USER function returns the current username and hostname. The syntax to view the username is: `SELECT USER();`	For example, to obtain the current username and hostname, enter the following command at the command prompt: `SELECT USER();`
USER		The output of this function is: `root@localhost.com`

Table 14.4: Additional System Information Functions in MySQL

Concepts

Summary

- In SQL queries, functions can be used in place of the column name and in the `WHERE` clause.
- Date functions can execute computations on date, time, and datetime data types. These functions can add and subtract the date, extract a part of the date, and convert a date from one format to another. Some examples of date functions are `ADDDATE`, `YEAR`, `DATEDIFF`, and `CONVERT _ TZ`.
- String functions operate on string data type. They can be used to retrieve the ASCII value of a string, calculate the number of bits in a string, join two or more strings with or without the separator, convert the given string to lowercase or uppercase, and calculate the hexadecimal value of string. Some examples of `STRING` functions are `ASCII`, `CHARACTER _ LENGTH`, `BIT _ LENGTH`, `LCASE`, and `UCASE`.
- System information functions display system-related information. The version number of the database, name of the current user and host, and number of rows affected due to the preceding `INSERT`, `UPDATE`, or `DELETE` statement can be retrieved using system information functions. Some examples of system information functions are `BENCHMARK`, `CHARSET`, `USER`, and `VERSION`.

Check Your Progress

1. Which of the following function returns the numeric value of the leftmost character of the string specified in the argument?

   a. `CHAR`

   b. `ASCII`

   c. `MOD`

   d. `BIN`

2. Which of the following function returns the index of the string in the other strings specified in the argument?

   a. `FIELD`

   b. `CHAR`

   c. `BIN`

   d. `BIT_LENGTH`

3. Which of the following function returns the length of an argument in bits?

   a. `LENGTH`

   b. `COMPRESS`

   c. `BIT_LENGTH`

   d. `COUNT`

4. Diana wants to extract the time from a given expression. Which of the following functions will help Diana to achieve this?

   a. `TIME_FORMAT`

   b. `TIME`

Check Your Progress

c. TIMEDIFF

d. TIMESTAMP

5. John wants to delete leading and trailing spaces before displaying the records from a table. Which of the following functions will help John to achieve this?

a. LTRIM

b. TRIM

c. RTRIM

d. TRUNCATE

6. Which of the following function returns the value of the string whose position in the argument is passed as a numeric value?

a. ELT

b. FIELD

c. ORD

d. FIND_IN_SET

7. John wants to view the username and the hostname. Which of the following functions will help John to display the current username and hostname?

a. SCHEMA

b. DATABASE

c. USER

d. VERSION

Check Your Progress

8. Which of the following function returns the name of the month for the date entered as the argument?

   a. `MONTH`

   b. `MONTHNAME`

   c. `DAYOFMONTH`

   d. `LAST_DAY`

9. Which of the following function changes all the characters of the argument into uppercase?

   a. `LOWER`

   b. `LCASE`

   c. `UPPER`

   d. `REPLACE`

10. Which of the following function returns the day number for a datetime value?

   a. `TO_DAYS`

   b. `DAY`

   c. `FROM_DAYS`

   d. `LAST_DAY`

Onlinevarsity

Session
15

Using Basic Functions in MySQL - II (Lab)

Objectives

At the end of this session, the student will be able to:

- *Use Date functions in MySQL.*
- *Use String functions in MySQL.*
- *Use System Information functions in MySQL.*

The steps given in the session are detailed, comprehensive, and carefully thought through in order to meet the learning objectives and understand the tool completely. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Date Functions in MySQL

Date functions operate on date and time type of data. The date data type allows you to manipulate date values, allowing a user to add or subtract dates. In addition, several functions extract the required part from the date argument.

1. **John's hire date is '2009-09-06'. He starts actual work one day after the hire date. You have to display the date when he actually started working. To achieve this, you will add 1 day to the hire date. To display the working date, enter the following command at the command prompt:**

```
SELECT ADDDATE('2009-09-06', 1);
```

Figure 15.1 displays the output of the command.

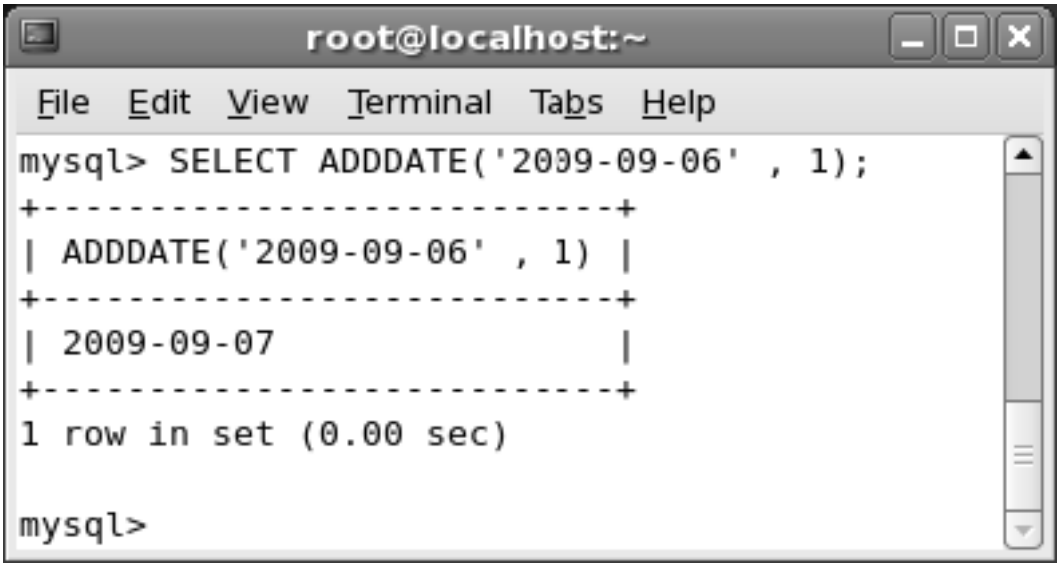


Figure 15.1: ADDATE Function

2. **John convened a meeting with his team at '20:10:00', which continued for an interval of '01:02:45'. To calculate the time when the meeting ended, you will add the two time expressions. To calculate the meeting end time, enter the following command at the command prompt:**

```
SELECT ADDTIME('20:10:00' , '01:02:45');
```

Figure 15.2 displays the output of the command.

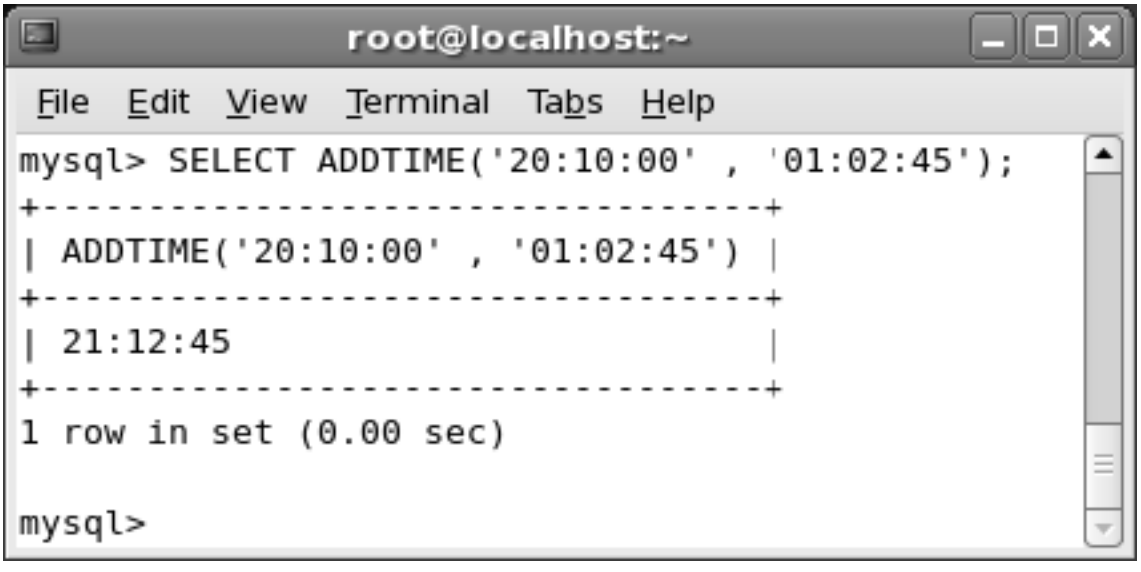


Figure 15.2: ADDTIME Function

3. **John wants to have a Web conference with his client at '2004-01-01 12:00:00' GMT. However, the client resides in a city which is 10 hours ahead of the specified time zone. To adjust his schedule, he needs to adapt to his client's time zone accordingly. To calculate the new time zone, enter the following command at the command prompt:**

```
SELECT CONVERT_TZ('2004-01-01 12:00:00' , '+00:00', '+10:00');
```

Figure 15.3 displays the output of the command.

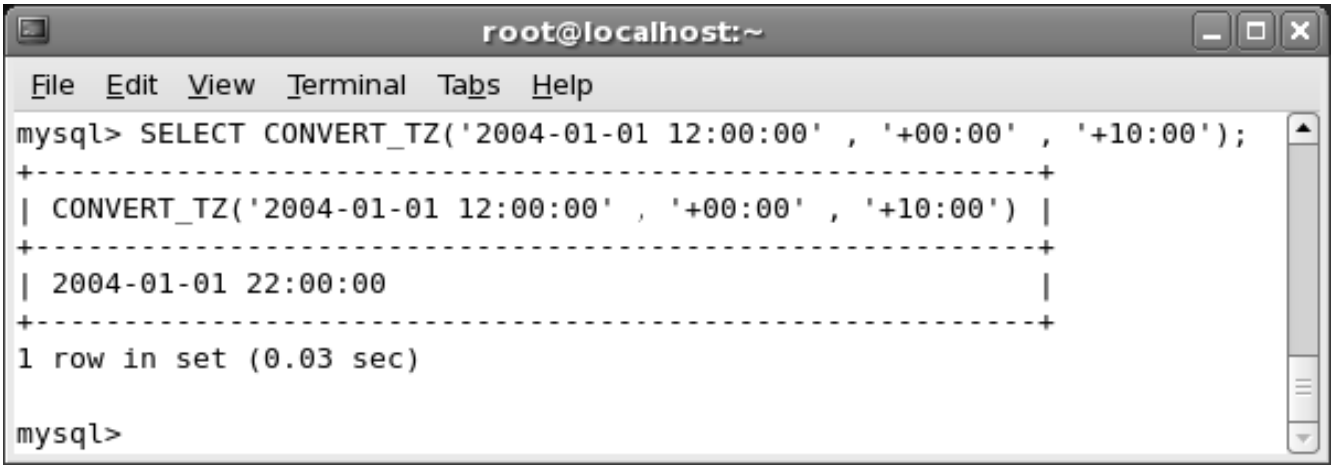


Figure 15.3: CONVERT_TZ Function

4. **John wants to know the current date and time because he lost his watch and has to proceed for a meeting. To view the current timestamp, enter the following command at the command prompt:**

```
SELECT CURRENT_TIMESTAMP();
```

Figure 15.4 displays the output of the command.

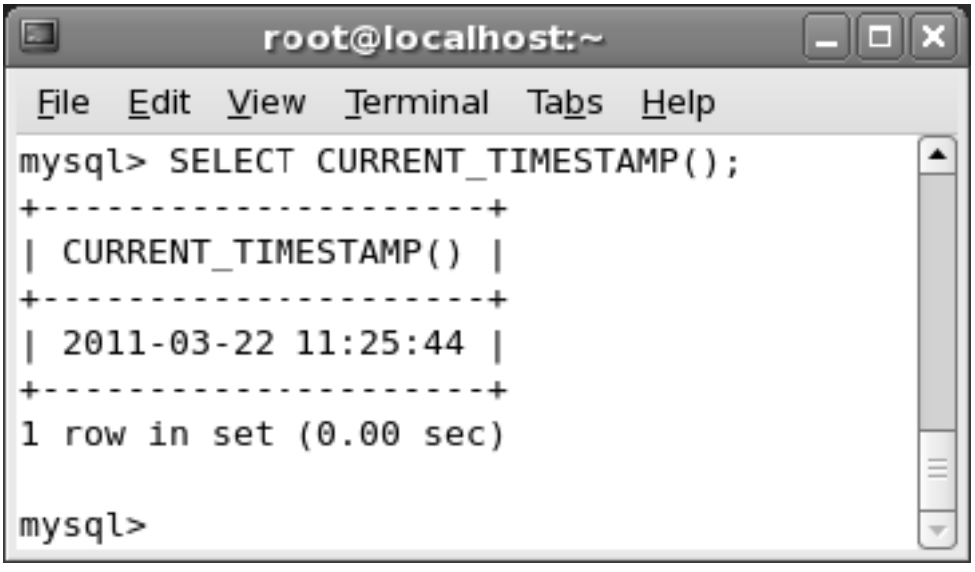


Figure 15.4: CURRENT_TIMESTAMP Function

5. **Peter joined the library on '2011-01-20' and he will have to renew the subscription exactly one year from the joining date. To calculate the subscription renewal date, enter the following command at the command prompt:**

```
SELECT DATE_ADD('2011-01-20' , INTERVAL 1 YEAR);
```

Figure 15.5 displays the output of the command.

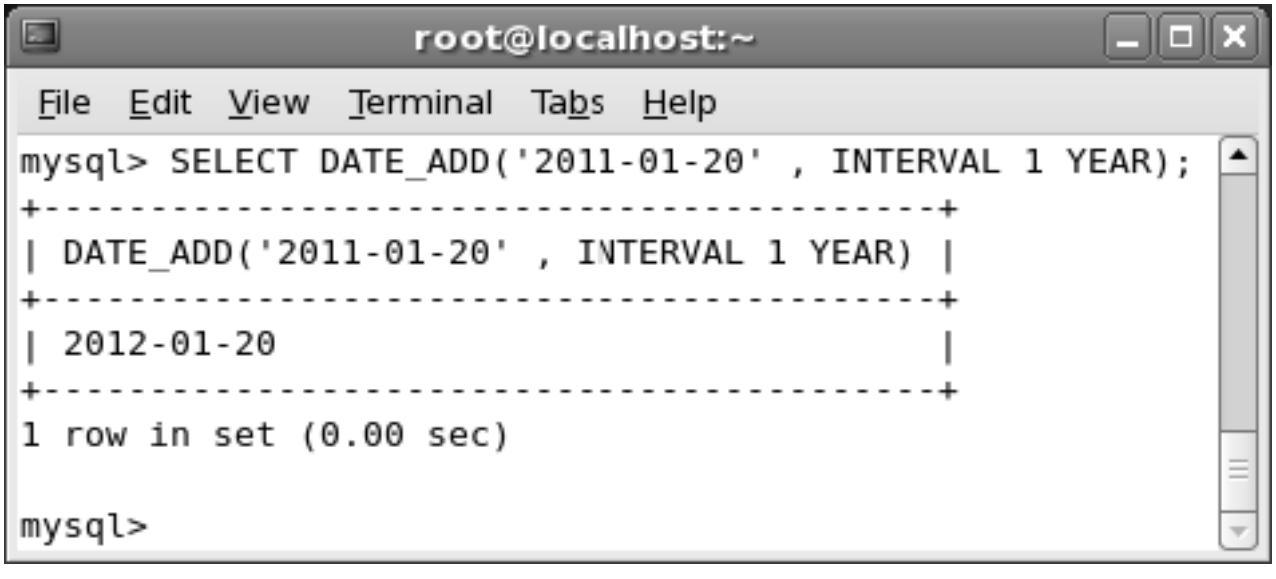


Figure 15.5: DATE_ADD Function

6. **Tim wants to know the date from the expression '2010-02-20 09:15:00'. To display the date from the expression, enter the following command at the command prompt:**

```
SELECT DATE('2010-02-20 09:15:00');
```

Figure 15.6 displays the output of the command.

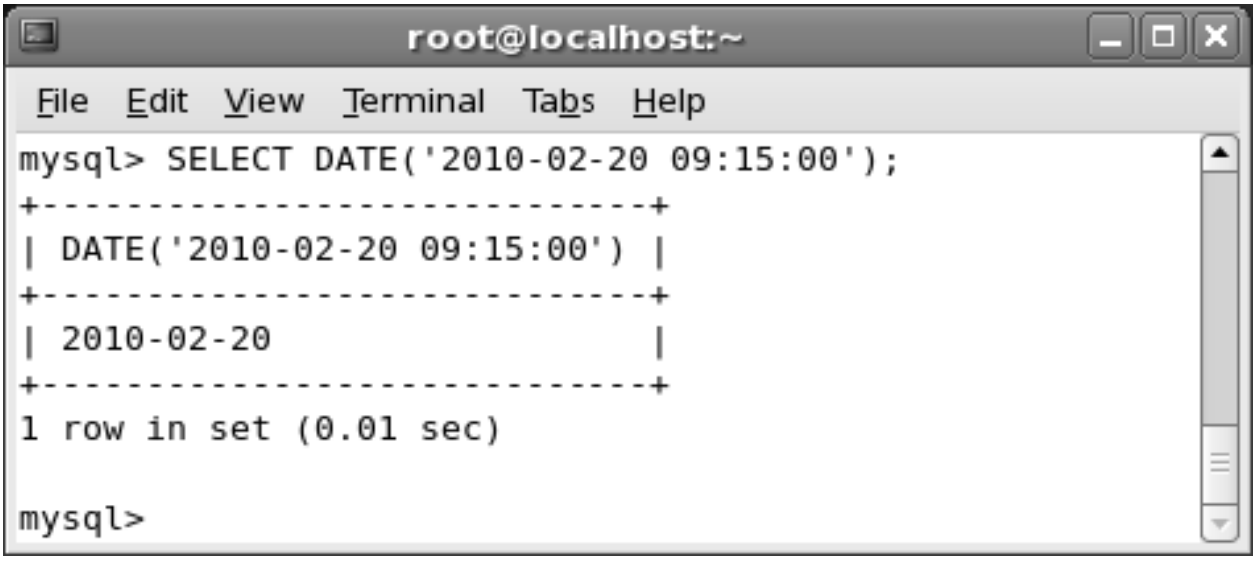


Figure 15.6: DATE Function

7. **Diana joined the firm on '2009-12-06' and sent her resignation letter on '2010-05-09'. To calculate her tenure in number of days in the firm you must calculate the difference between the join date and the end date. To calculate the tenure, enter the following command at the command prompt:**

```
SELECT DATEDIFF('2010-05-09' , '2009-12-06');
```

Figure 15.7 displays the output of the command.

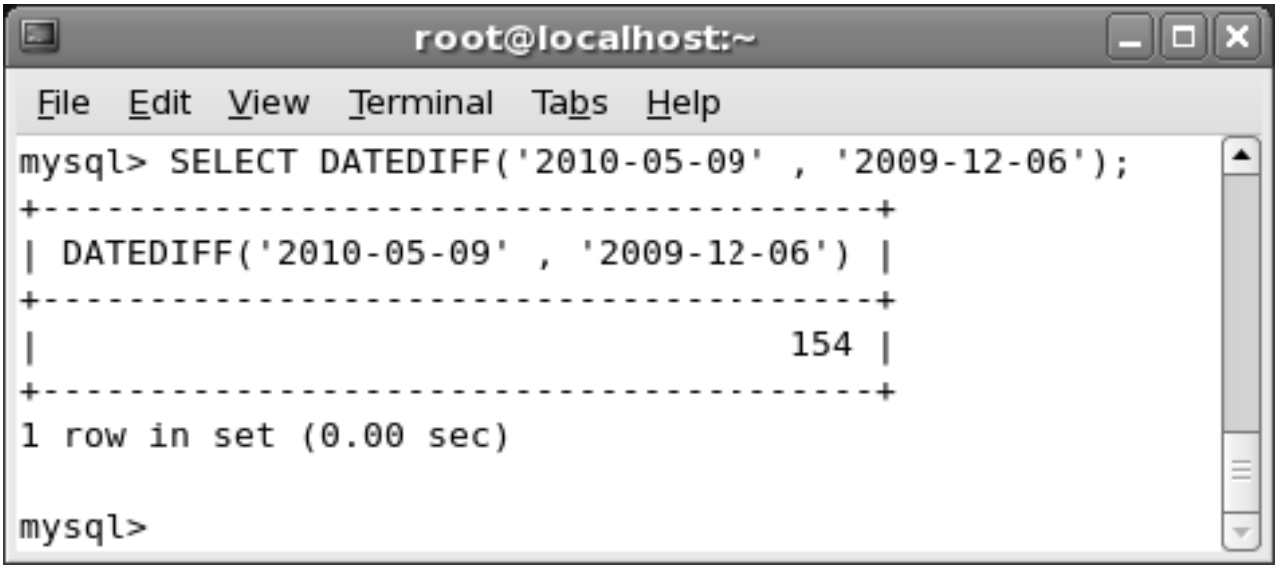


Figure 15.7: DATEDIFF Function

8. **Allen does not know the format in which the date is stored in the system. He wants to view the date '2005-07-21' in the 'MM-DD-YYYY' format. To view the date in the specified format, enter the following command at the command prompt:**

```
SELECT DATE_FORMAT('2005-07-21' , '%M-%D-%Y');
```

Figure 15.8 displays the output of the command.

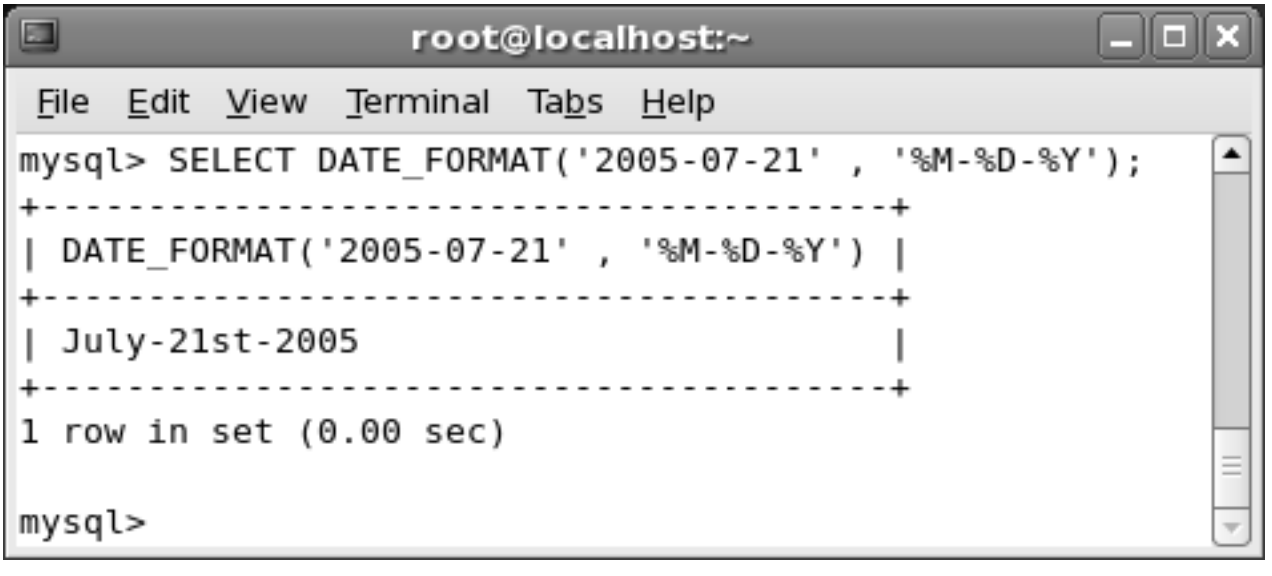


Figure 15.8: DATE_FORMAT Function

9. **Tim started a project two days earlier and will be finishing today. The manager wants to know the date on which the project work started. To calculate the start date, you will subtract two days from the current date. To subtract two days from the current date, enter the following command at the command prompt:**

```
SELECT DATE_SUB(CURDATE() , INTERVAL 2 DAY);
```

Figure 15.9 displays the output of the command.

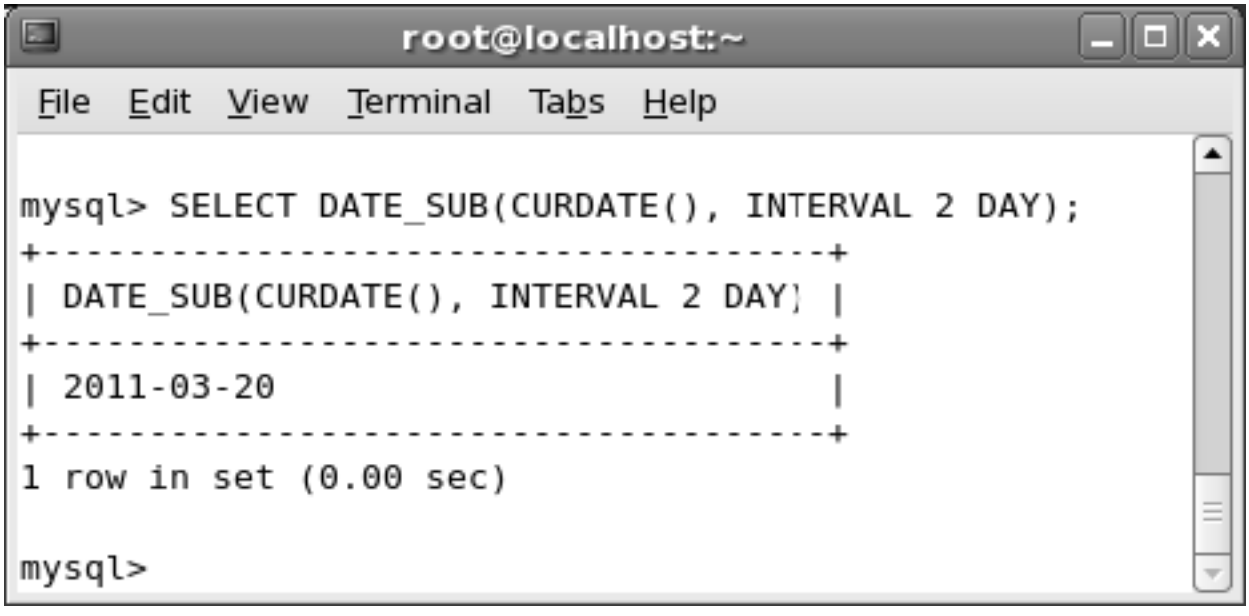


Figure 15.9: DATE_SUB Function

10. **Adam wants to know the day for his date of joining '2008-11-19', To view the day for a given date, enter the following command at the command prompt:**

```
SELECT DAY('2008-11-19');
```

Figure 15.10 displays the output of the command.

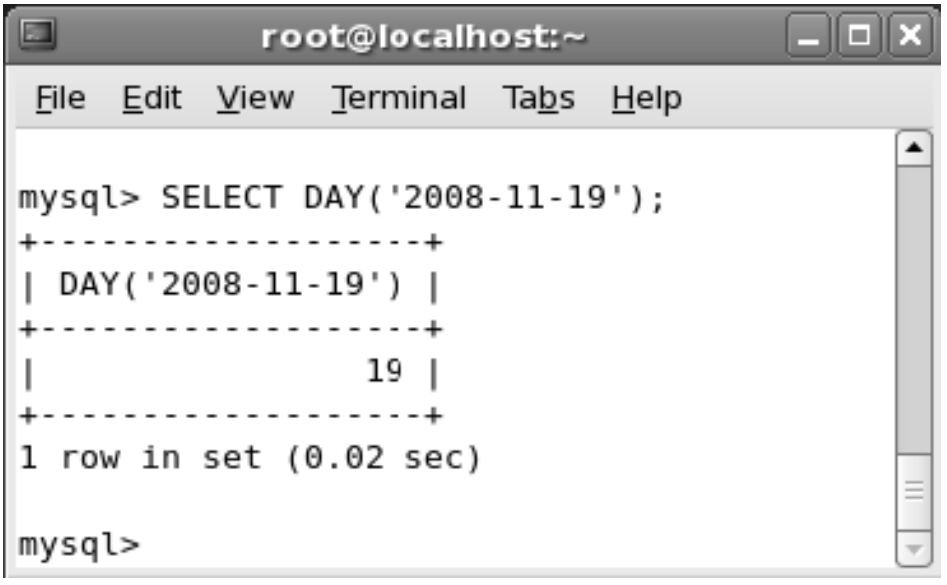


Figure 15.10: DAY Function

11. **Peter wanted to know the day of the week when he started working on '2009-04-21'. To display the name of day for the date '2009-04-21', enter the following command at the command prompt:**

```
SELECT DAYNAME('2009-04-21');
```

Figure 15.11 displays the output of the command.

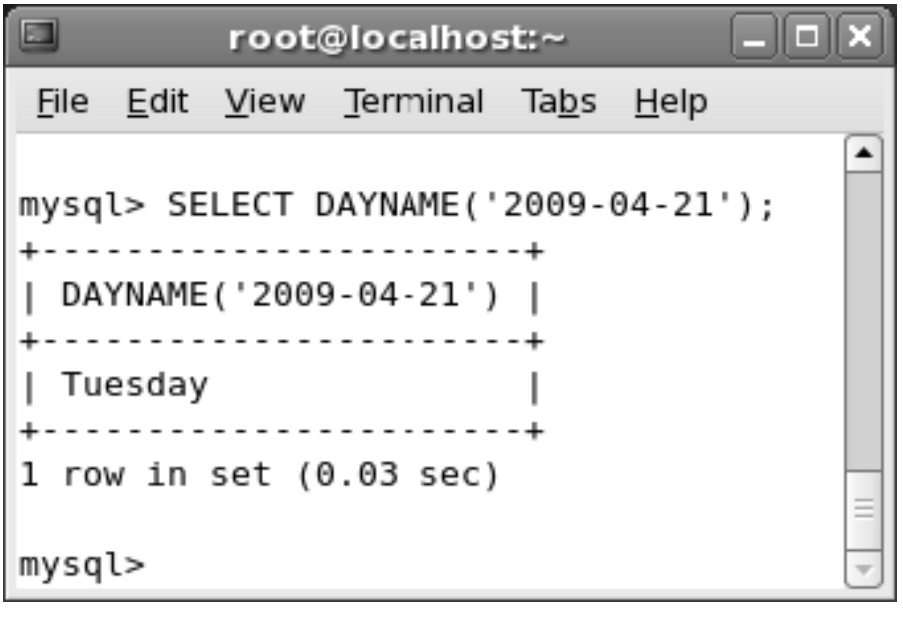


Figure 15.11: DAYNAME Function

12. **John wants to know the year in the date '2010-08-17'. To view the year from a date, enter the following command at the command prompt:**

```
SELECT EXTRACT(YEAR FROM '2010-08-17');
```

Figure 15.12 displays the output of the command.

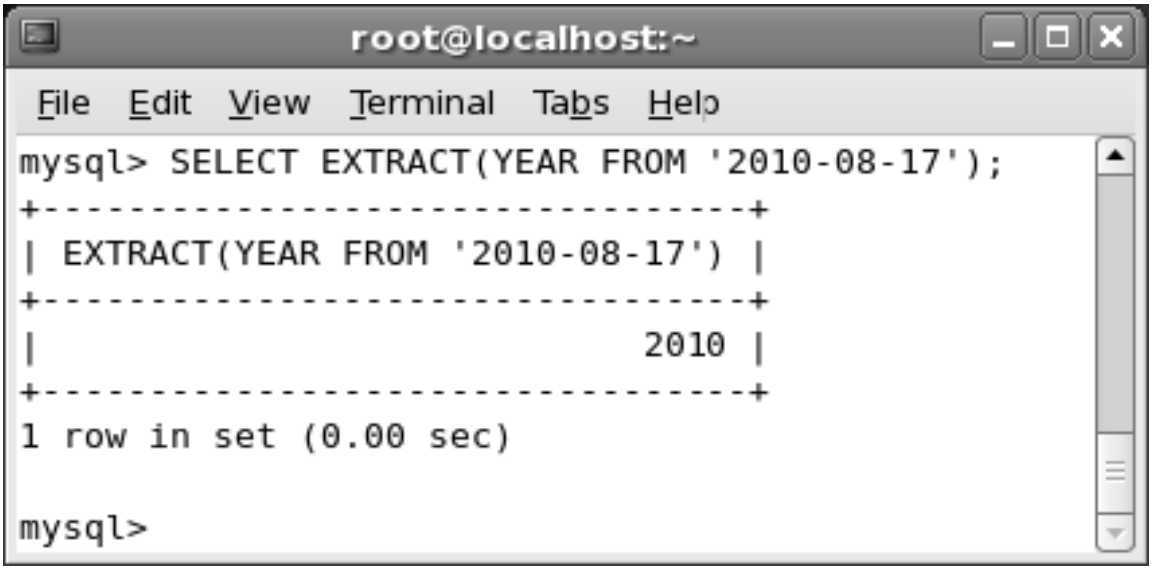


Figure 15.12: EXTRACT Function

13. **As a part of an assignment, John requires to calculate the date for the number '250000'. To calculate the date, enter the following command at the command prompt:**

```
SELECT FROM_DAYS(250000);
```

Figure 15.13 displays the output of the command.

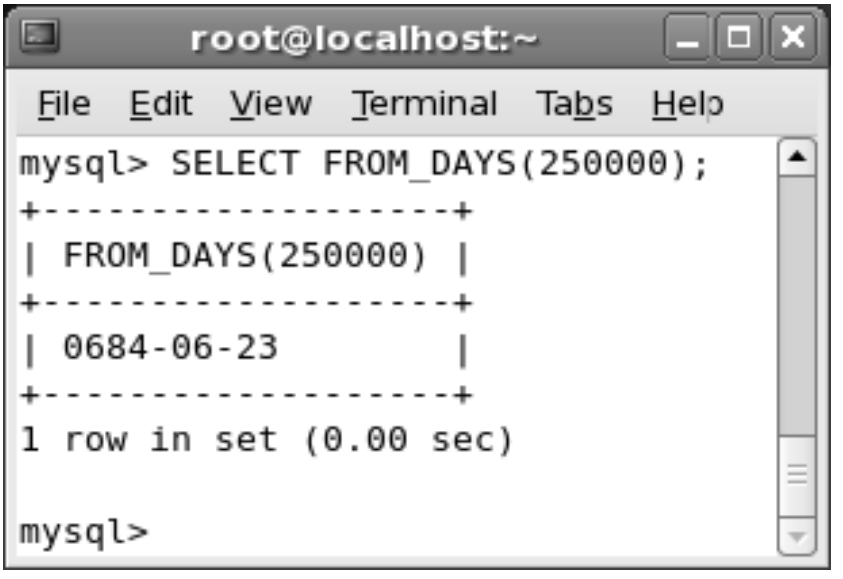


Figure 15.13: FROM_DAYS Function

14. **Jeanne wants to view the hours for the time '05:30:45'. To display the hours from the time expression, enter the following command at the command prompt:**

```
SELECT HOUR('05:30:45');
```

Figure 15.14 displays the output of the command.

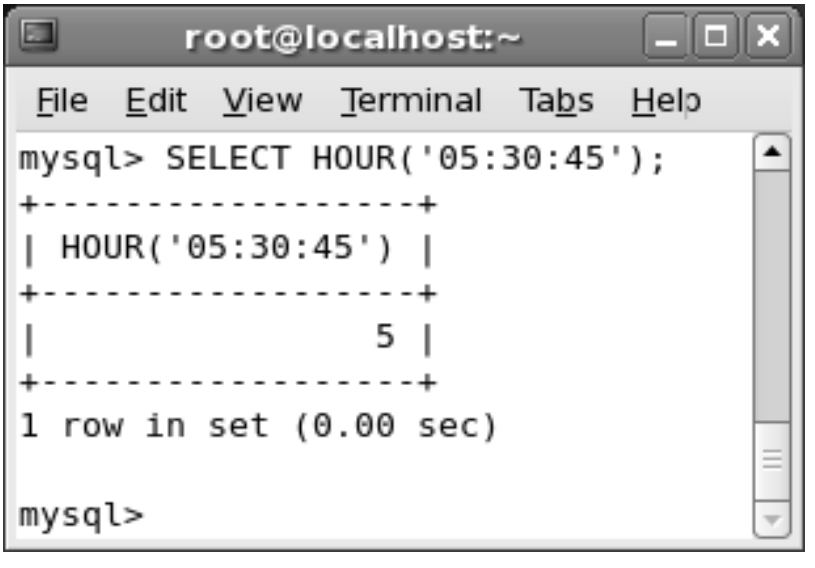


Figure 15.14: HOUR Function

15. **Sarah wants to view the last day of February 2004 to know whether it was a leap year. To view the last day, enter the following command at the command prompt:**

```
SELECT LAST_DAY('2004-02-05');
```

Figure 15.15 displays the output of the command.

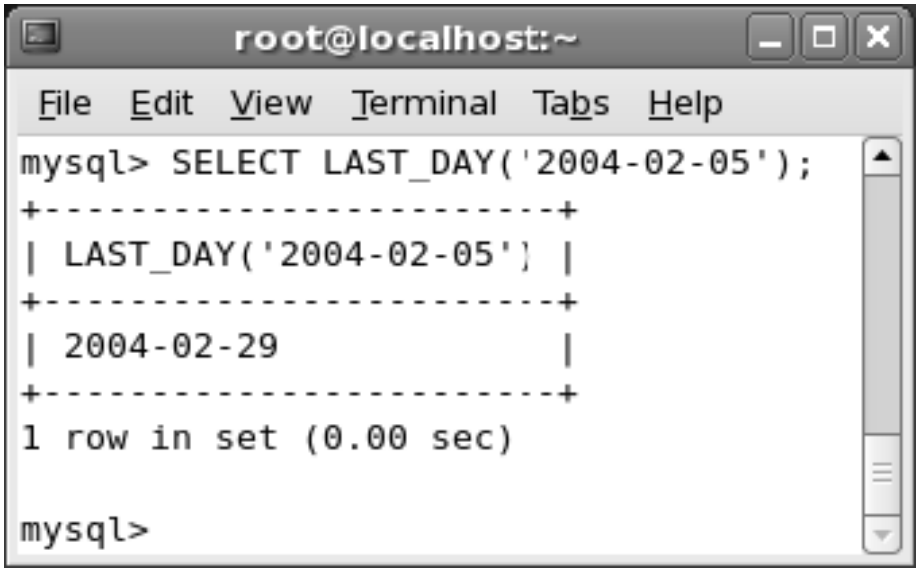


Figure 15.15: LAST_DAY Function

16. **Lavina wants to view the exact minutes when the space shuttle was launched from the time expression '05:30:45'. To display the minutes from a time expression, enter the following command at the command prompt:**

```
SELECT MINUTE('05:30:45');
```

Figure 15.16 displays the output of the command.

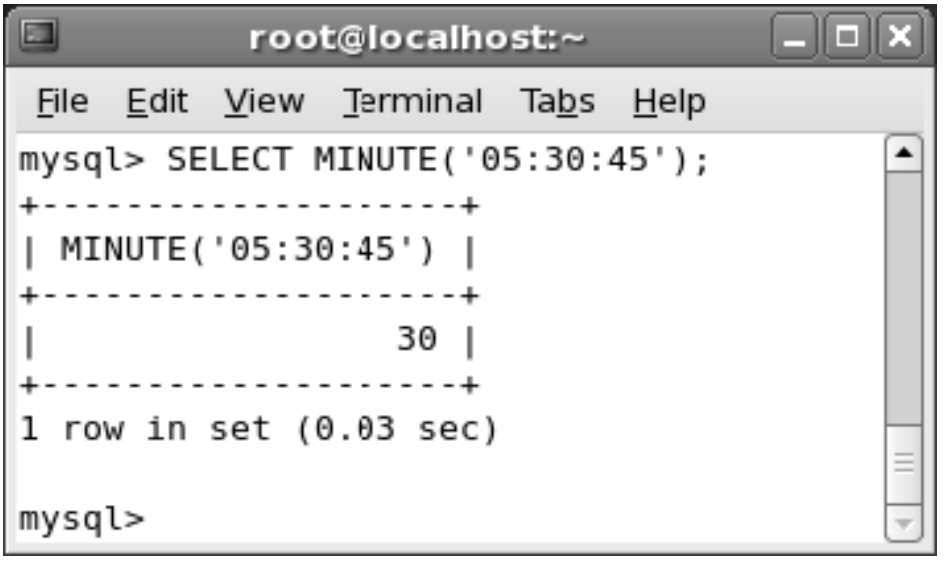


Figure 15.16: MINUTE Function

17. **Barbara wants to view the month and the month name for the books that have been issued. To display the month and the month name for books that have been issued, enter the following command at the command prompt:**

```
SELECT MONTH(DATE_OF_ISSUE), MONTHNAME(DATE_OF_ISSUE)

FROM ISSUE_DETAILS;
```

Figure 15.17 displays the output of the command.

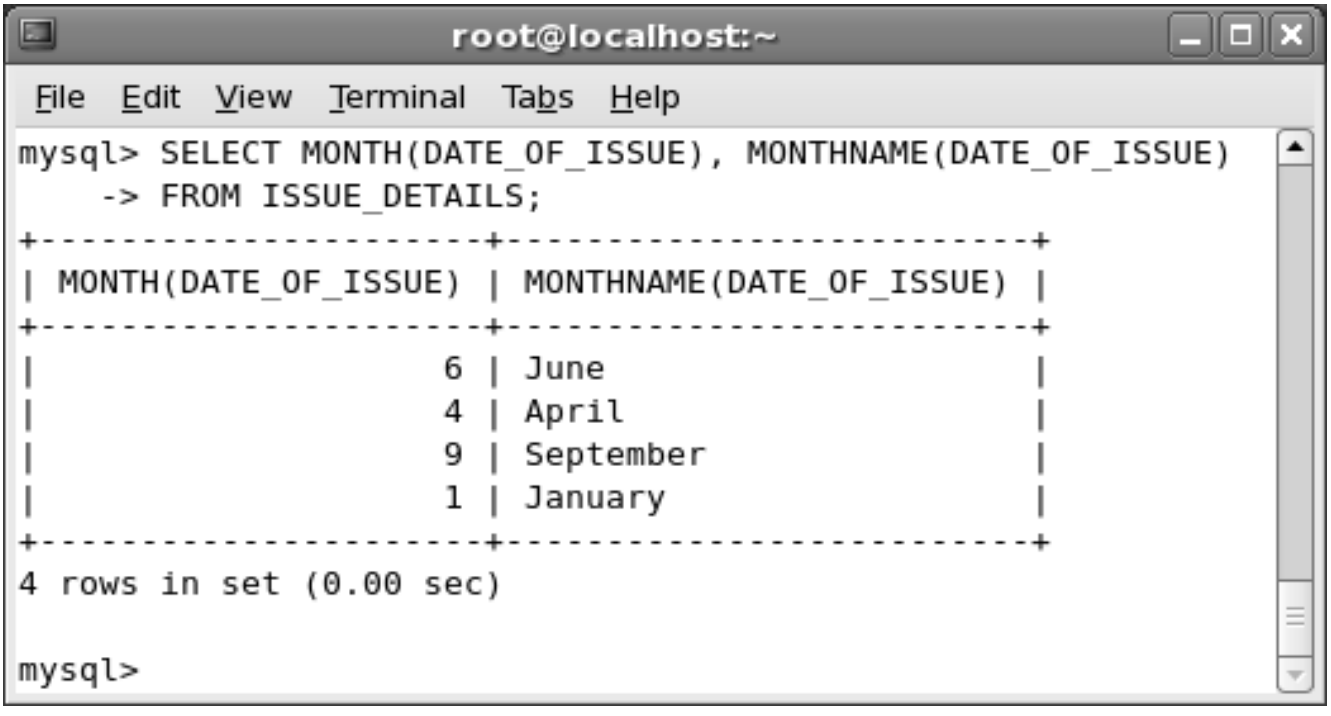


Figure 15.17: MONTH and MONTHNAME Function

18. **To display the current timestamp, enter the following command at the command prompt:**

```
SELECT NOW();
```

Figure 15.18 displays the output of the command.

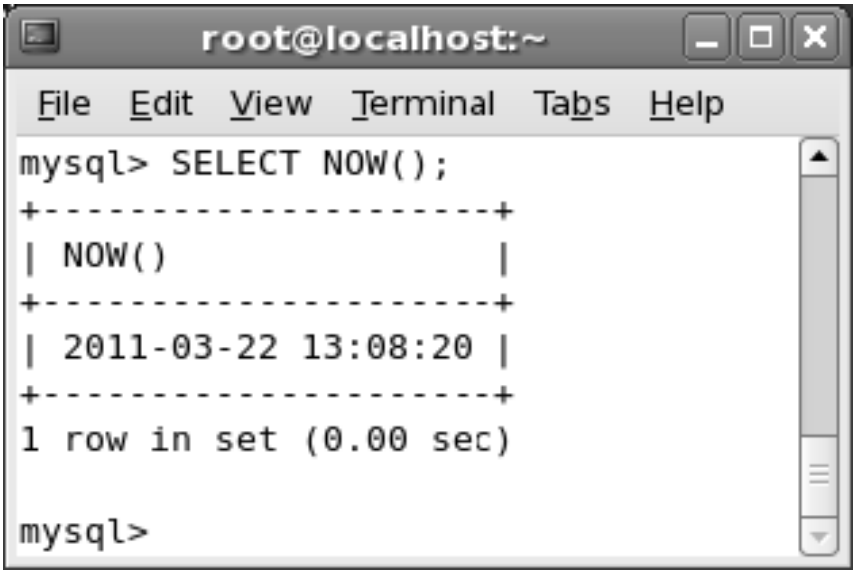


Figure 15.18: NOW Function

19. **Celina joined the firm in '200709' and will be on probation for a period of 12 months. To display the probation end date, enter the following command at the command prompt:**

```
SELECT PERIOD_ADD('200709' , 12);
```

Figure 15.19 displays the output of the command.

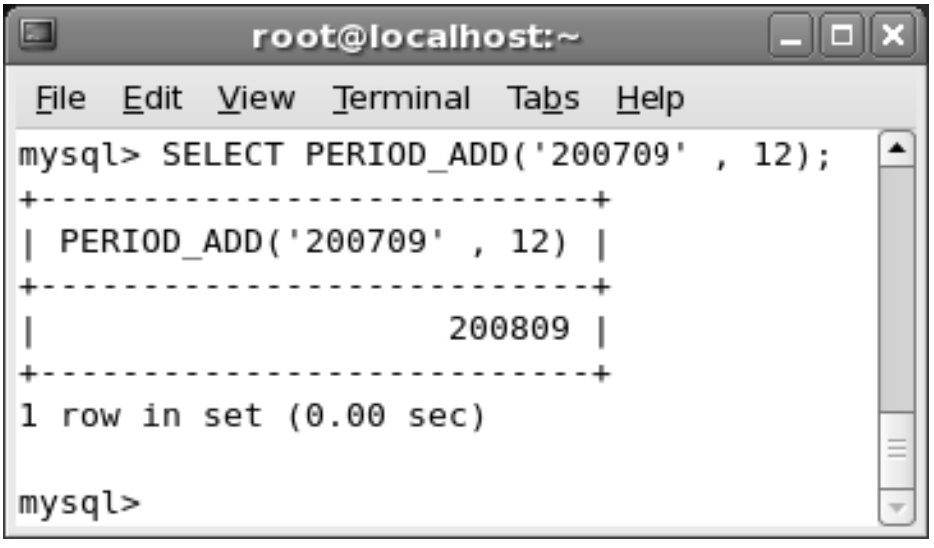


Figure 15.19: PERIOD_ADD Function

20. **To convert 4200 into the HH:MM:SS format, enter the following command at the command prompt:**

```
SELECT SEC_TO_TIME(4200);
```

Figure 15.20 displays the output of the command.

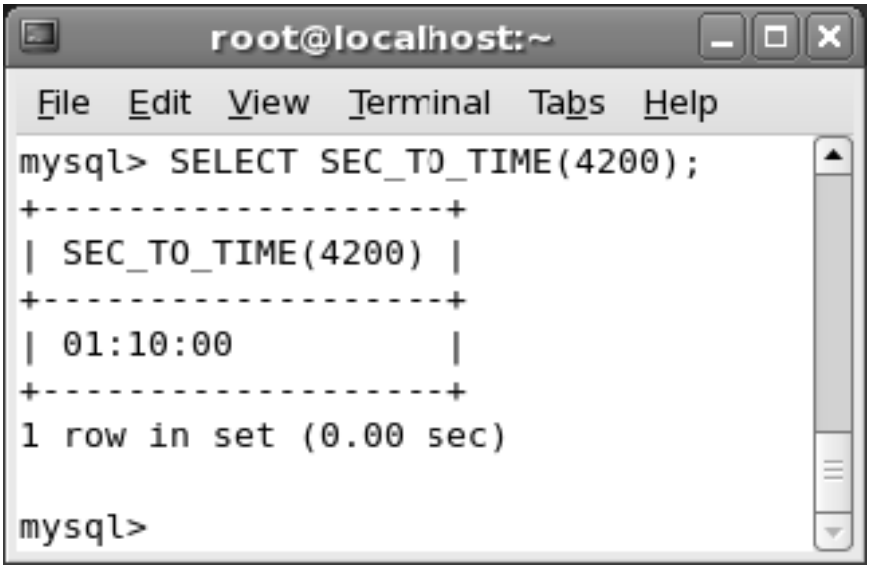


Figure 15.20: SEC_TO_TIME Function

21. **Michael wants to convert the sting '02, 03, 2014' into the 'YYYY-MM-DD' format. To convert the string to 'YYYY-MM-DD' format, enter the following command at the command prompt:**

```
SELECT STR_TO_DATE('02,03,2014' , '%m,%d,%Y');
```

Figure 15.21 displays the output of the command.

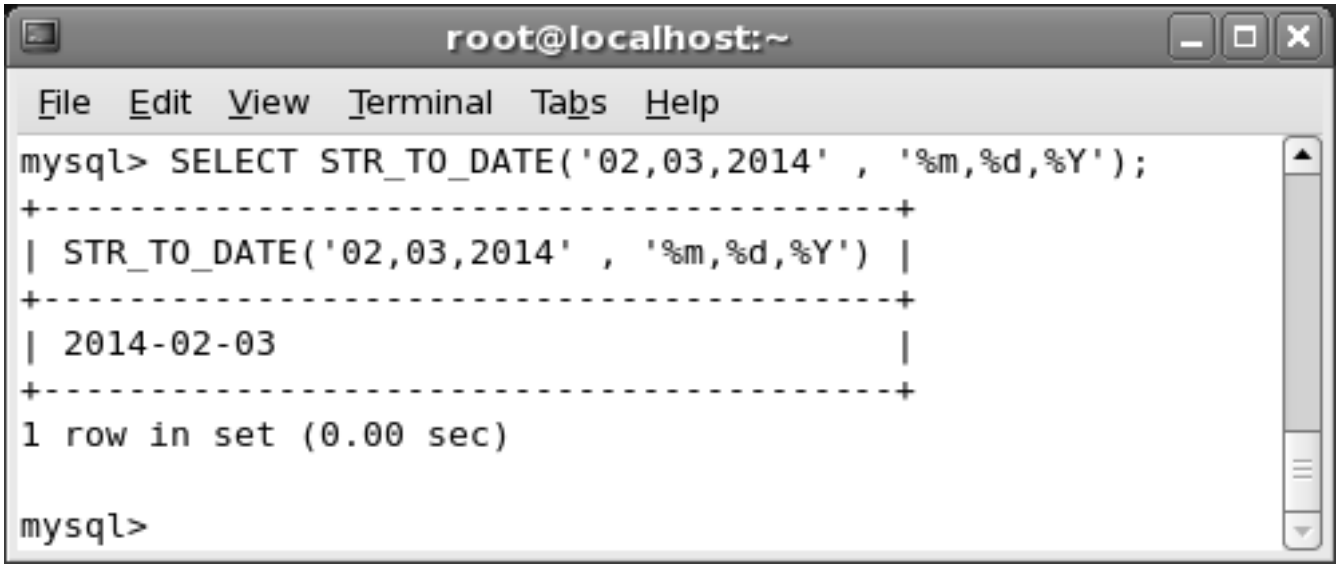


Figure 15.21: STR_TO_DATE Function

22. **Peter reached office at '12:45:00' and left at '17:55:00'. To calculate the total working hours, enter the following command at the command prompt:**

```
SELECT SUBTIME('12:45:00' , '17:55:00');
```

Figure 15.22 displays the output of the command.

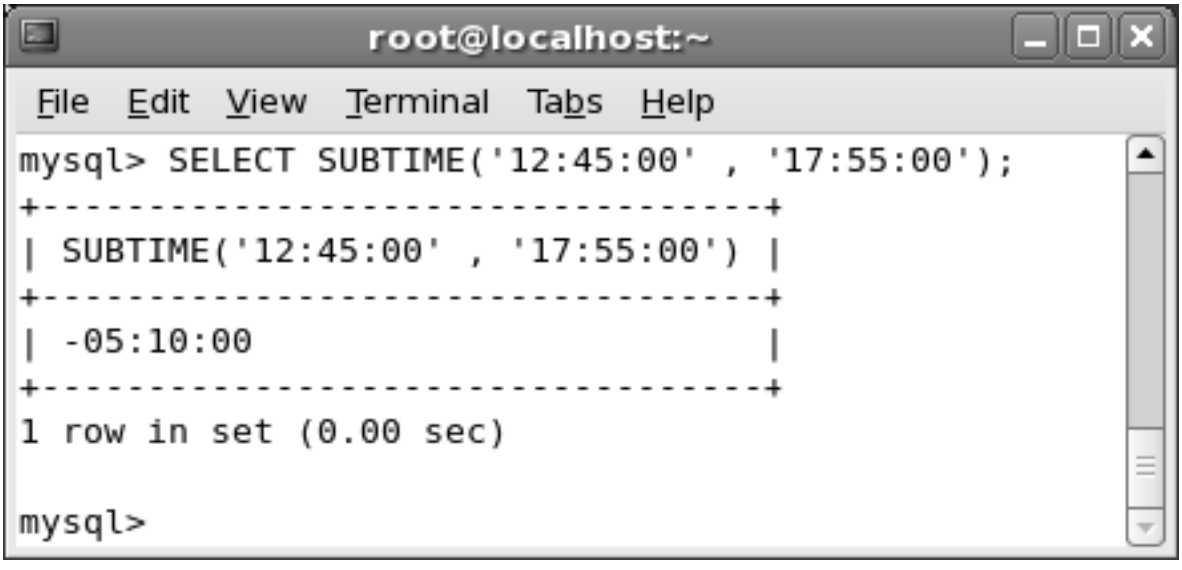


Figure 15.22: SUBTIME Function

23. **Annie wants to view the current time in HH:MM:SS format. To display the current time in the 'HH:MM:SS' format, enter the following command at the command prompt:**

```
SELECT TIME_FORMAT(NOW() , '%H:%I:%S');
```

Figure 15.23 displays the output of the command.

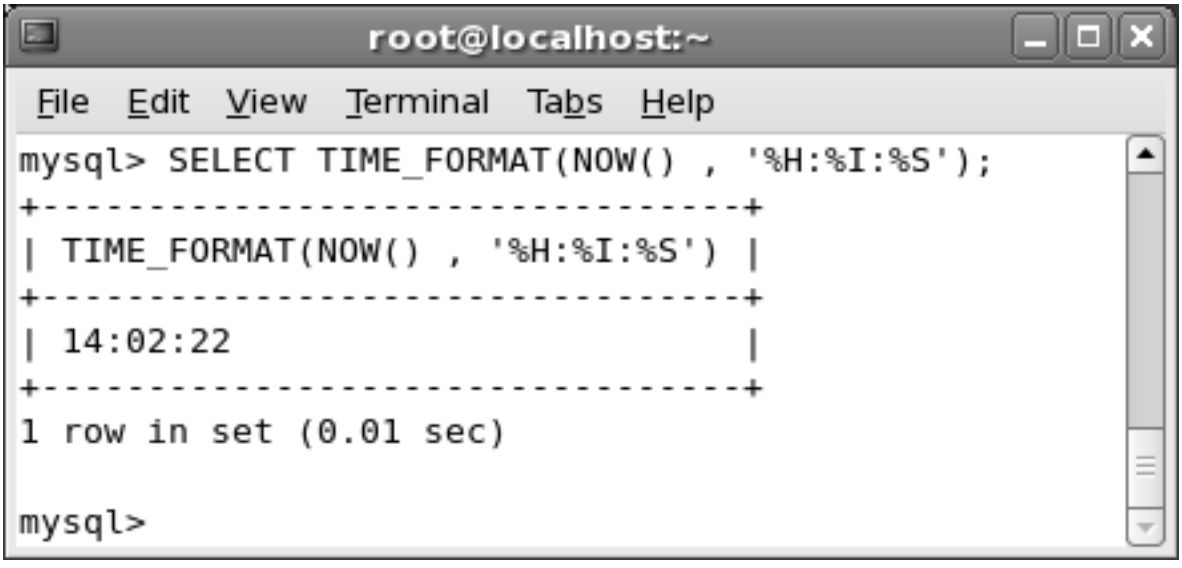


Figure 15.23: TIME_FORMAT Function

24. **Dave wants to calculate the day number for the date '2000-01-01'. To calculate the number of days that have elapsed, enter the following command at the command prompt:**

```
SELECT TO_DAYS('2000-01-01');
```

Figure 15.24 displays the output of the command.

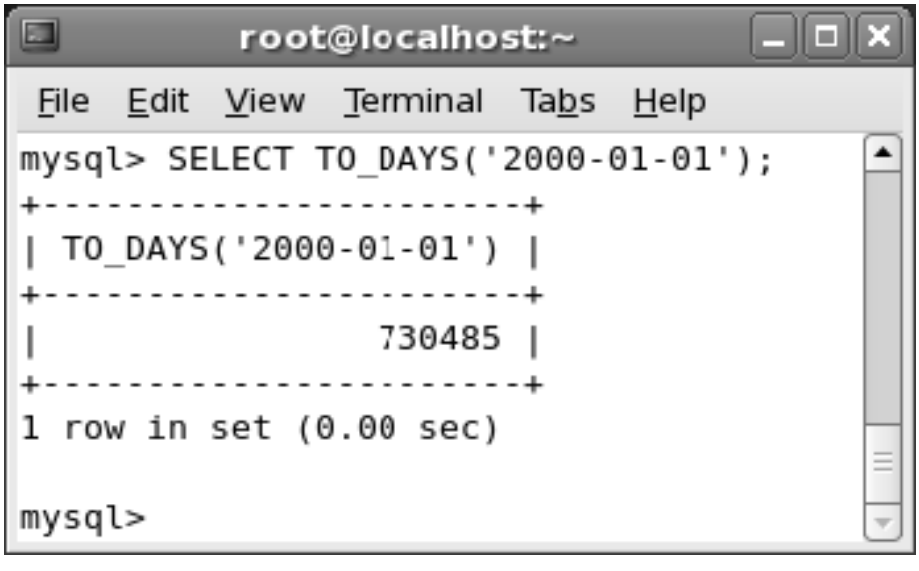


Figure 15.24: TO_DAYS Function

25. **To display the current UTC date and time, enter the following command at the command prompt:**

```
SELECT UTC_TIMESTAMP;
```

Figure 15.25 displays the output of the command.

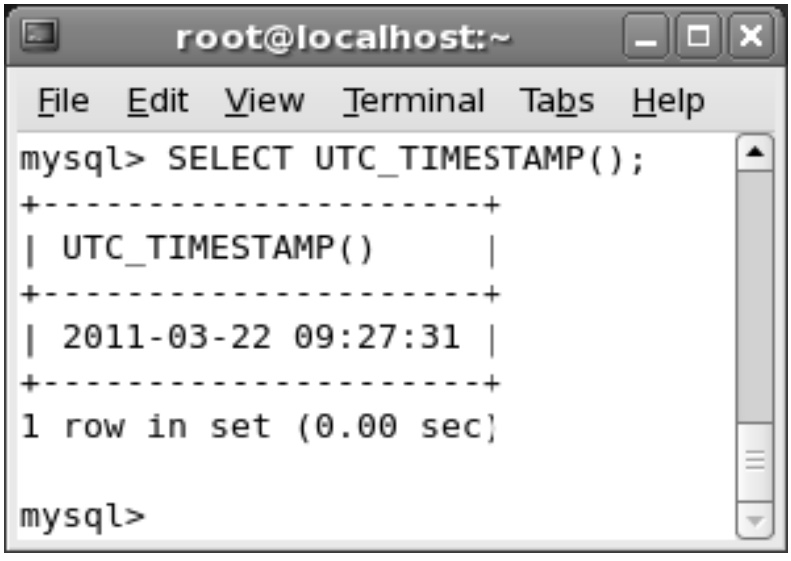


Figure 15.25: UTC_TIMESTAMP Function

26. **Gia is recording all important events and dates. She completed one year in the organization on '2005-04-20'. To display the week number for the date '2005-04-20', enter the following command at the command prompt:**

```
SELECT WEEK('2005-04-20');
```

Figure 15.26 displays the output of the command.

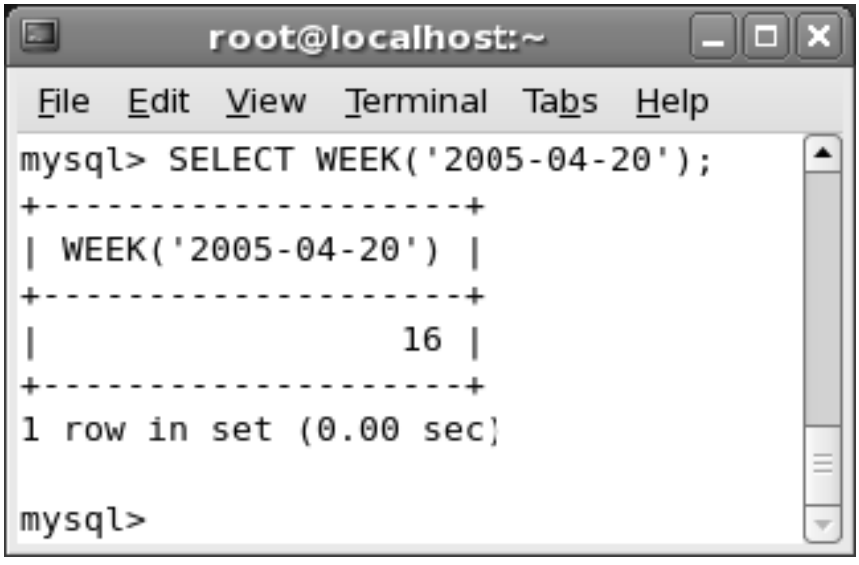


Figure 15.26: WEEK Function

27. **Gia also wants to view the weekday index for the date when she completed one year in the organization. To display the weekday index for '2005-04-20', enter the following command at the command prompt:**

```
SELECT WEEKDAY('2005-04-20');
```

Figure 15.27 displays the output of the command.

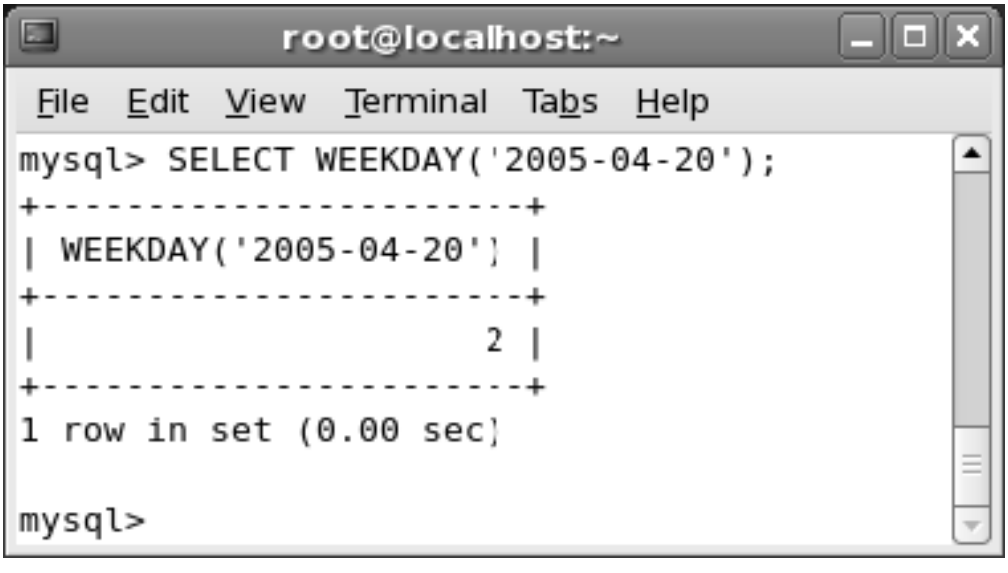


Figure 15.27: WEEKDAY Function

28. **George wants to view the week number for the date '2005-04-20'. To display the week number for '2005-04-20', enter the following command at the command prompt:**

```
SELECT WEEKOFYEAR('2005-04-20');
```

Figure 15.28 displays the output of the command.

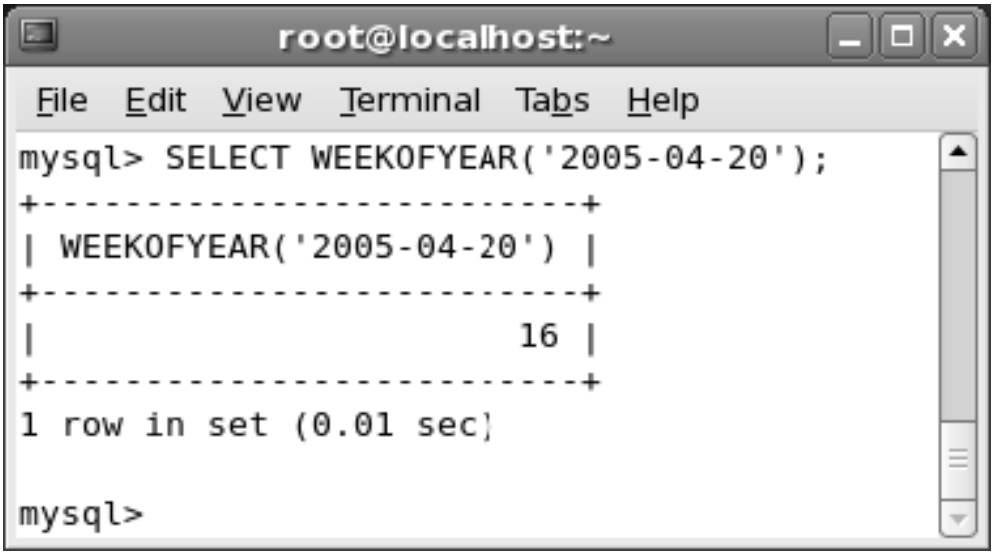


Figure 15.28: WEEKOFYEAR Function

29. **To display the year from '2010-12-05', enter the following command at the command prompt:**

```
SELECT YEAR('2010-12-05');
```

Figure 15.29 displays the output of the command.

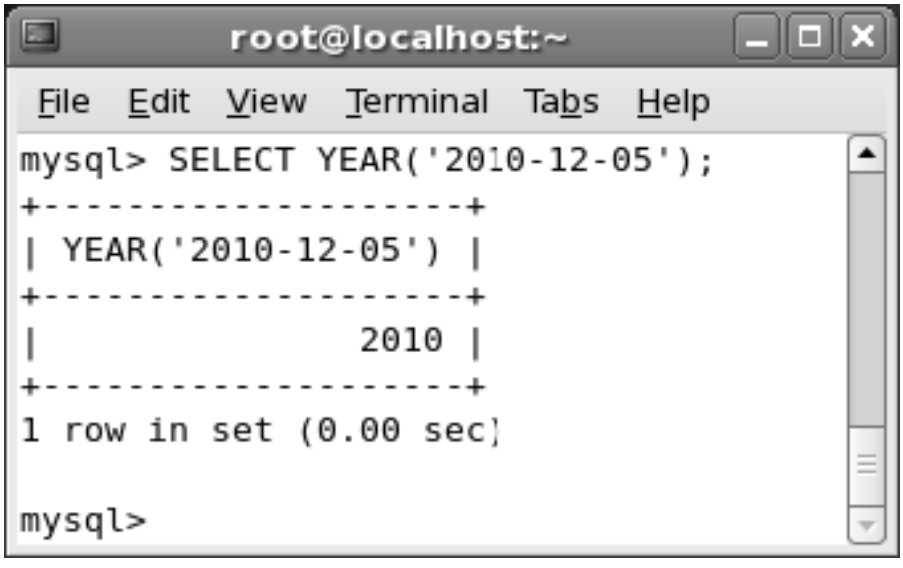


Figure 15.29: YEAR Function

30. **To display the year and week for '2011-11-16', enter the following command at the command prompt:**

```
SELECT YEARWEEK('2011-11-16');
```

Figure 15.30 displays the output of the command.

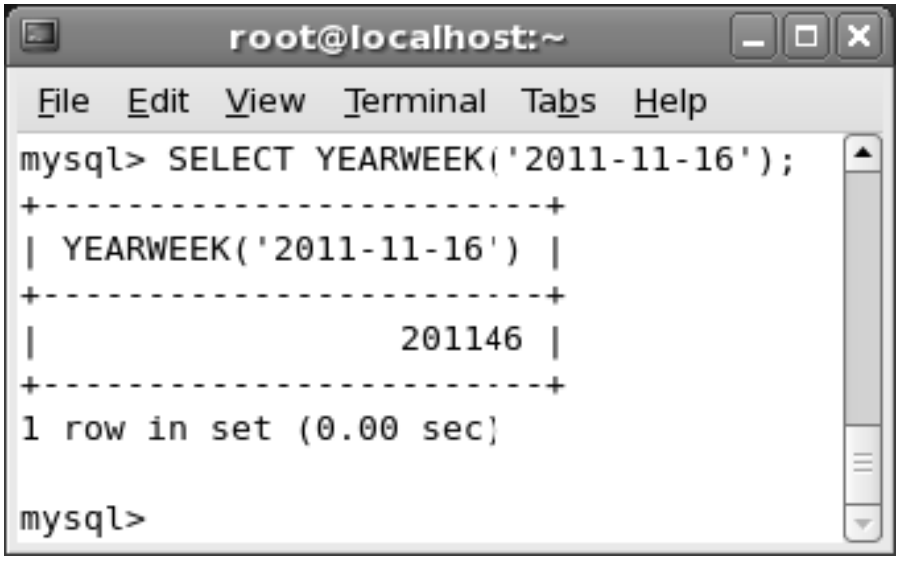


Figure 15.30: YEARWEEK Function

Lab Guide

String functions in MySQL

MySQL provides string functions that operate on string data type. The string functions accept arguments as strings even if you input numeric data. You will work with the **LIBRARY** database to use string functions in MySQL.

1. **Activate the LIBRARY database.**

2. **To display the name of the users and the ASCII value of the names, enter the following command at the command prompt:**

```
SELECT NAME, ASCII(NAME) FROM USER_DETAILS;
```

Figure 15.31 displays the output of the command.

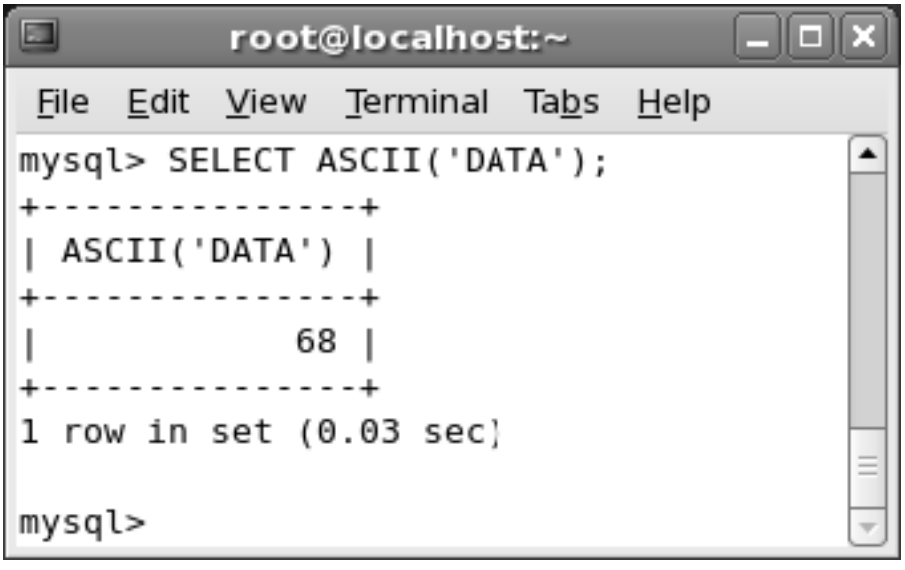


Figure 15.31: ASCII Function

3. **John wants to view the phone numbers of users as encrypted values. To display the phone numbers as encrypted values, enter the following command at the command prompt:**

```
SELECT PHONE_NO, BIN(PHONE_NO)FROM USER_DETAILS;
```

Figure 15.32 displays the output of the command.

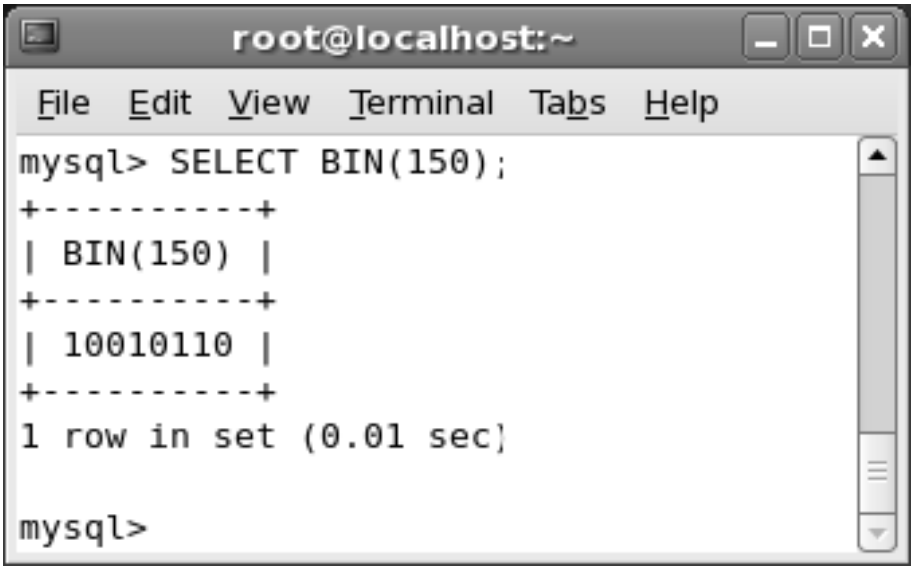


Figure 7.32: BIN Function

4. **To display the BOOK_ID, char value of BOOK_ID, enter the following command at the command prompt:**

```
SELECT BOOK_ID, CHAR(BOOK_ID)FROM BOOK_TABLE;
```

Figure 15.33 displays the output of the command.

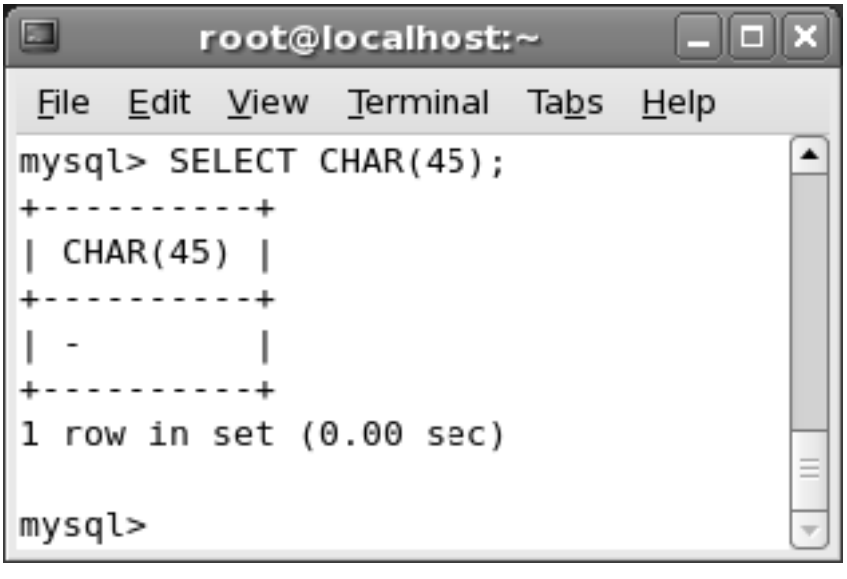


Figure 15.33: CHAR Function

5. **To display the BOOK_NAME and the maximum number of characters in the BOOK_NAME column, enter the following command at the command prompt:**

```
SELECT BOOK_NAME, CHARACTER_LENGTH(BOOK_NAME)

FROM BOOK_TABLE;
```

Figure 15.34 displays the output of the command.

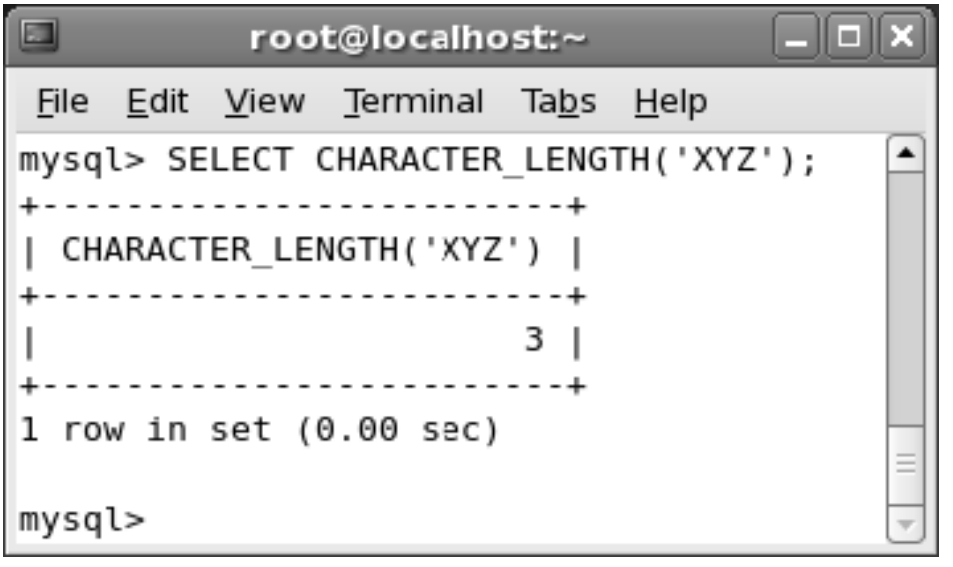


Figure 15.34: CHARACTER_LENGTH Function

6. **To display the names of the author and the length of the compressed string of the names of the author, enter the following command at the command prompt:**

```
SELECT AUTHOR, LENGTH(COMPRESS(AUTHOR))

FROM BOOK_TABLE;
```

Figure 15.35 displays the output of the command.

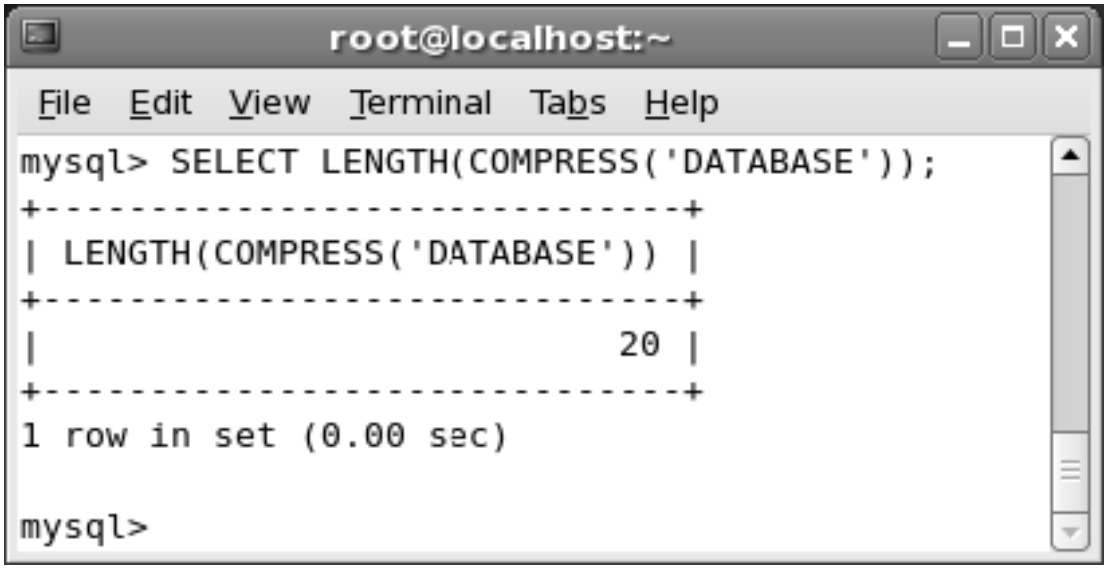


Figure 15.35: COMPRESS Function

7. **To display the USER_ID and the NAME as a single string, enter the following command at the command prompt:**

```
SELECT CONCAT(USER_ID, NAME) FROM USER_DETAILS;
```

Figure 15.36 displays the output of the command.

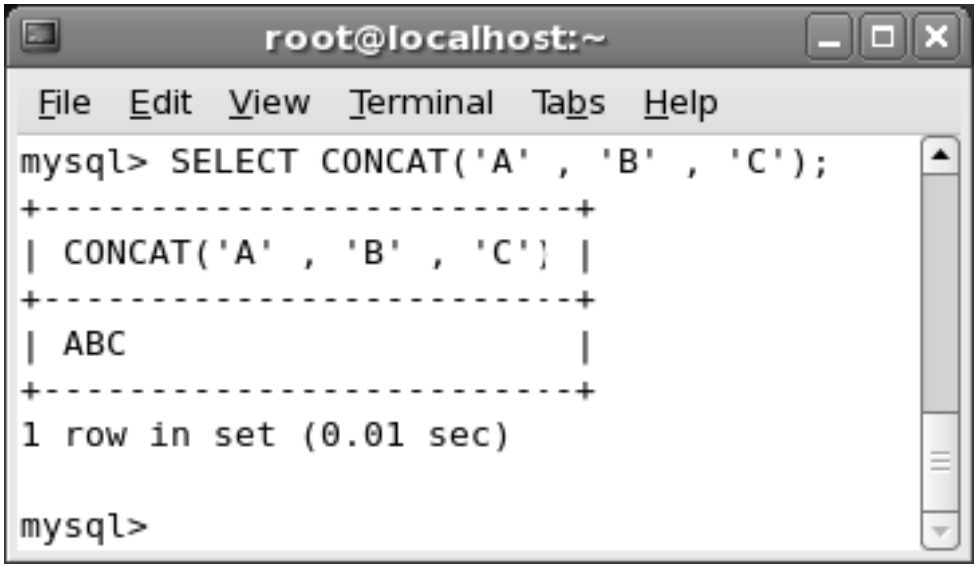


Figure 15.36: CONCAT Function

8. **To display the character at position two of the string 'X', 'Y', 'Z', enter the following command at the command prompt:**

```
SELECT ELT(2, 'X', 'Y', 'Z');
```

Figure 15.37 displays the output of the command.

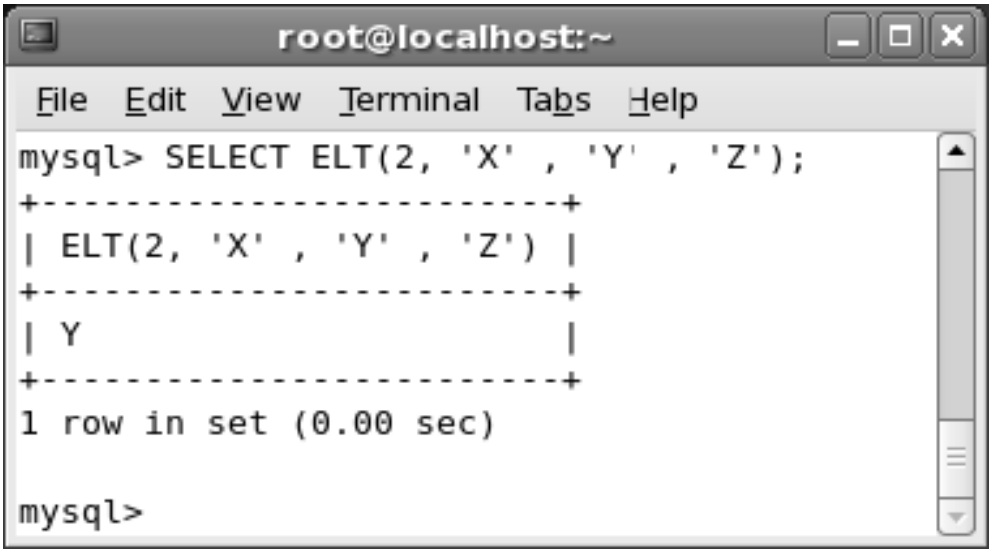


Figure 15.37: ELT Function

9. **To display the index of the field 'T' in the strings 'CONE', 'ONE', and 'T', enter the following command at the command prompt:**

```
SELECT FIELD('T', 'CONE', 'ONE', 'T');
```

Figure 15.38 displays the output of the command.

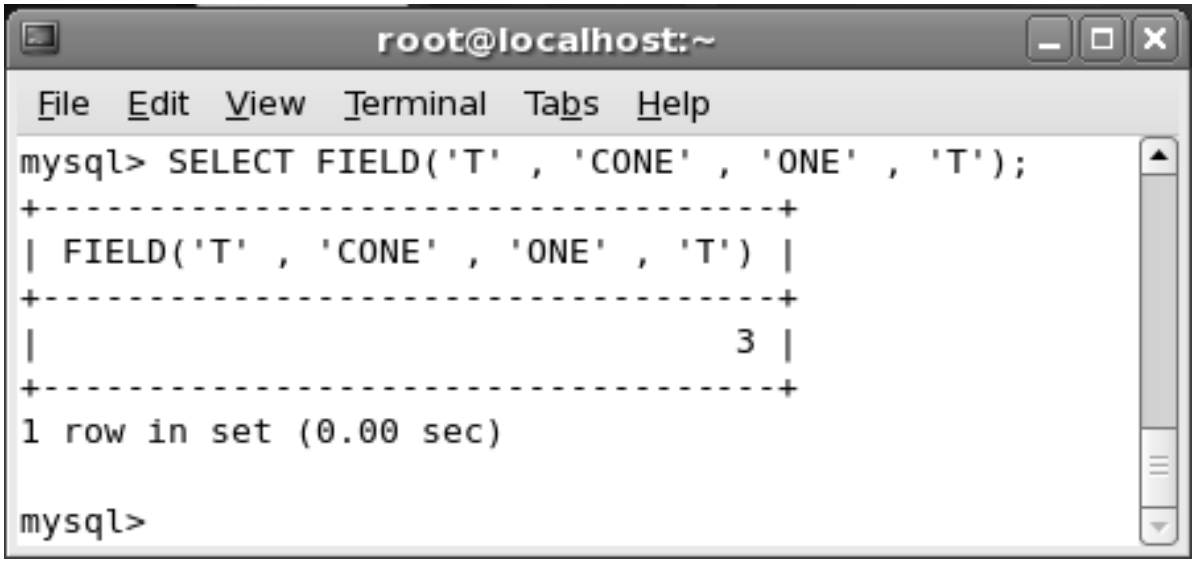


Figure 15.38: FIELD Function

10. **To display the position of the field 'C' in the string 'C, O, N, E', enter the following command at the command prompt:**

```
SELECT FIND_IN_SET('C', 'C,O,N,E');
```

Figure 15.39 displays the output of the command.

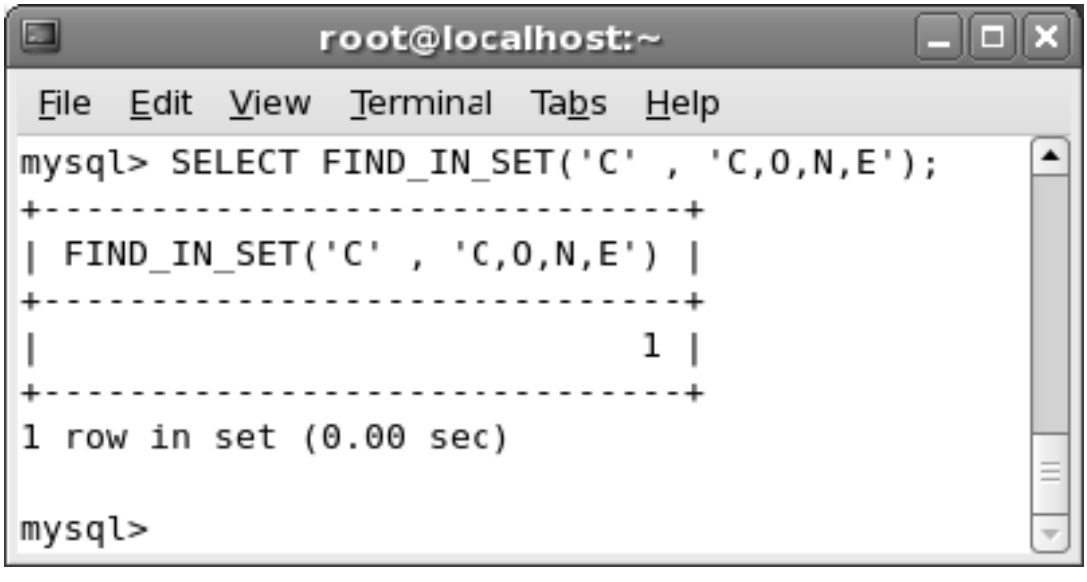


Figure 15.39: FIND_IN_SET Function

11. **To display the hexadecimal value of book names, enter the following command at the command prompt:**

```
SELECT BOOK_NAME , HEX(BOOK_NAME)

FROM BOOK_TABLE;
```

Figure 15.40 displays the output of the command.

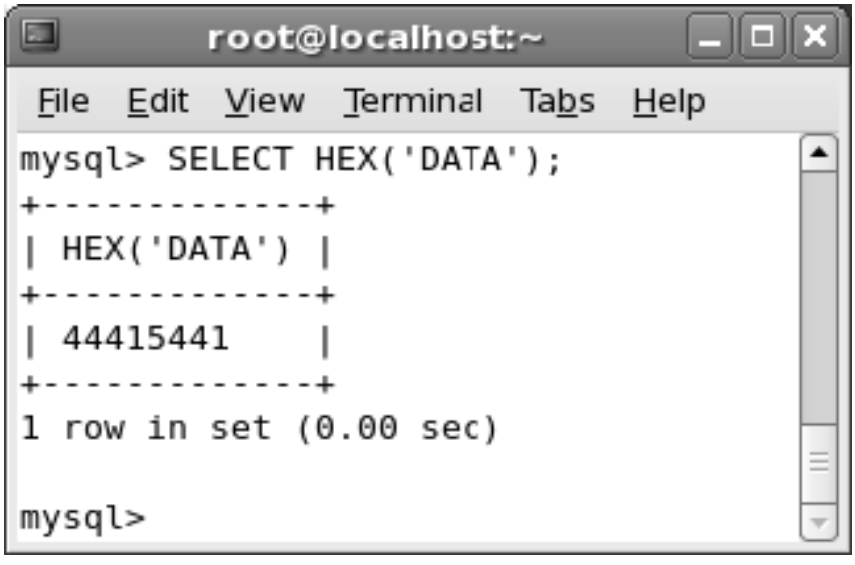


Figure 15.40: HEX Function

12. **The librarian wants to concatenate word 'MEMBER' where the type of membership has the value as REGULAR. To insert the string MEMBER where the TYPE is REGULAR, enter the following command at the command prompt:**

```
SELECT INSERT(TYPE, 8,6, 'MEMBER') FROM CATEGORY WHERE TYPE =
'REGULAR';
```

Figure 15.41 displays the output of the command.

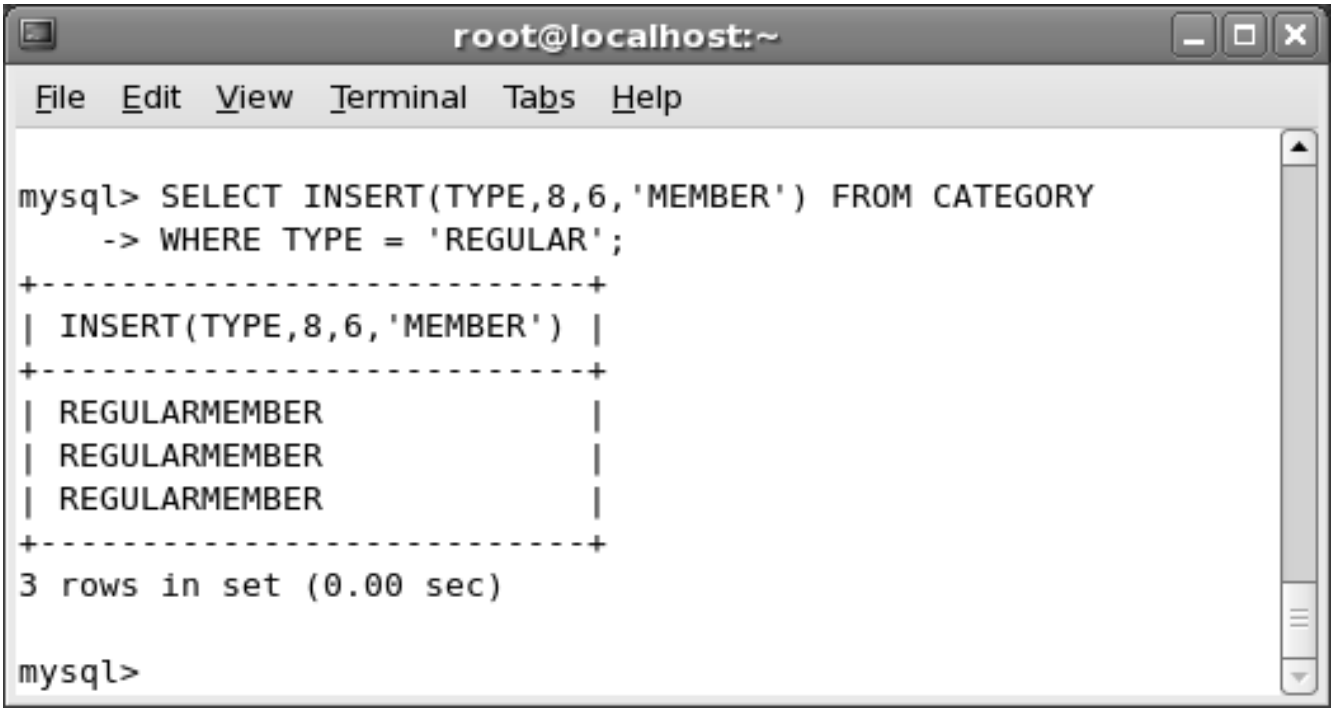


Figure 15.41: INSERT Function

13. **To display the position of the substring 'BASE' in the string 'DATABASE', enter the following command at the command prompt:**

```
SELECT INSTR('DATABASE', 'BASE');
```

Figure 15.42 displays the output of the command.

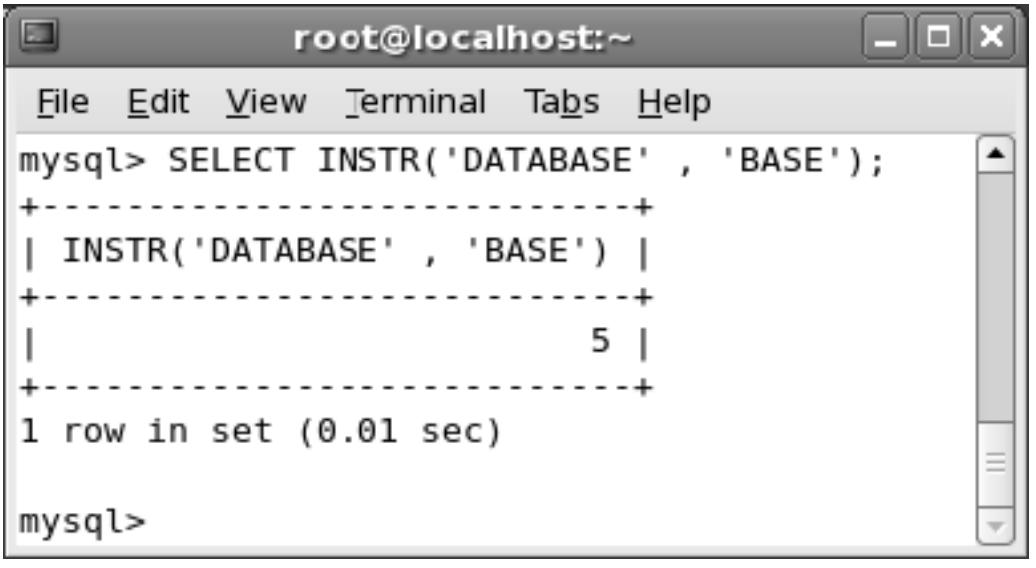


Figure 15.42: INSTR Function

14. **To convert the book names to lowercase, enter the following command at the command prompt:**

```
SELECT BOOK_NAME, LCASE(BOOK_NAME) FROM BOOK_TABLE;
```

Figure 15.43 displays the output of the command.

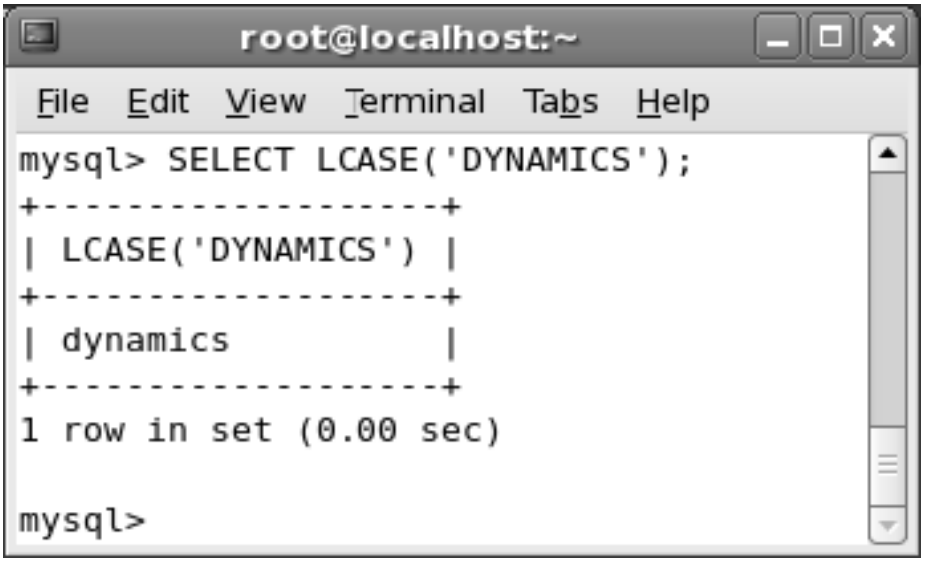


Figure 15.43: LCASE Function

15. **The librarian wants to view the first three characters of the names of the books in the library. To display the first three characters of the book name, enter the following command at the command prompt:**

```
SELECT LEFT(BOOK_NAME, 3) FROM BOOK_TABLE;
```

Figure 15.44 displays the output of the command.

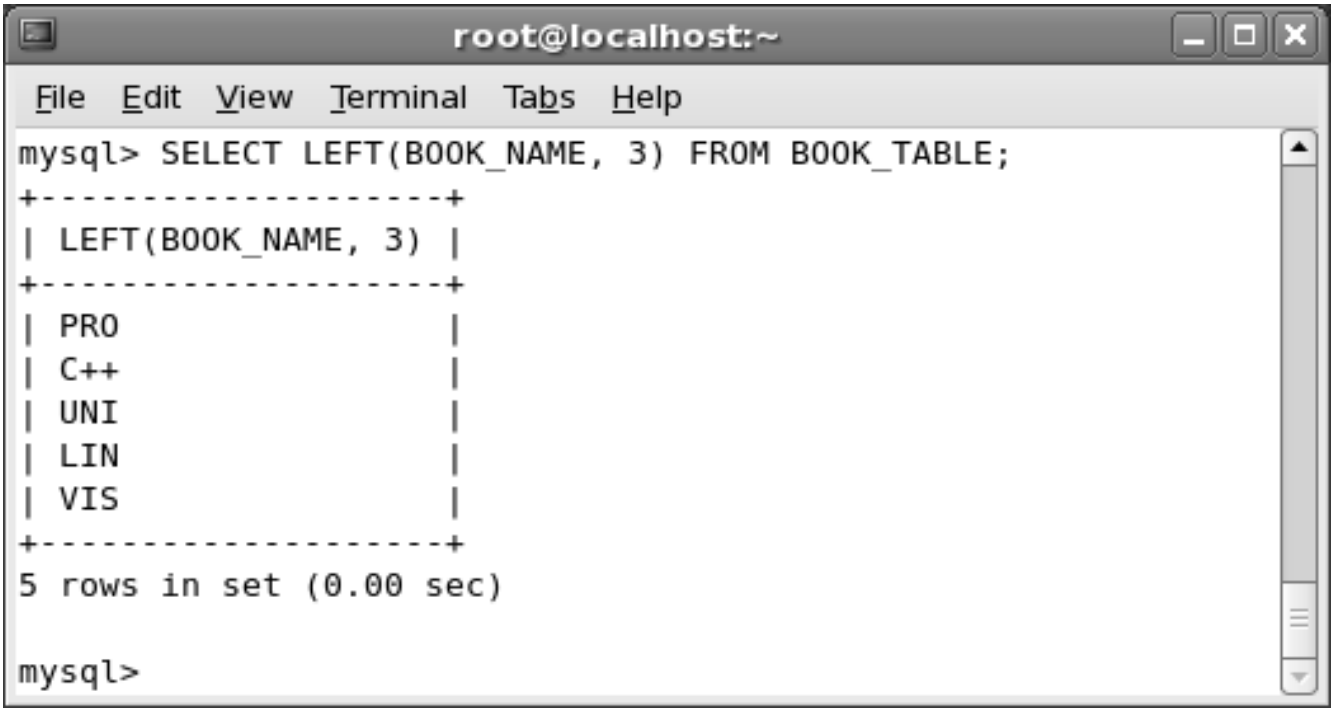


Figure 15.44: LEFT Function

16. **To display the name of the author and the number of characters present in the author field, enter the following command at the command prompt:**

```
SELECT AUTHOR, LENGTH(AUTHOR)FROM BOOK_TABLE;
```

Figure 15.45 displays the output of the command.

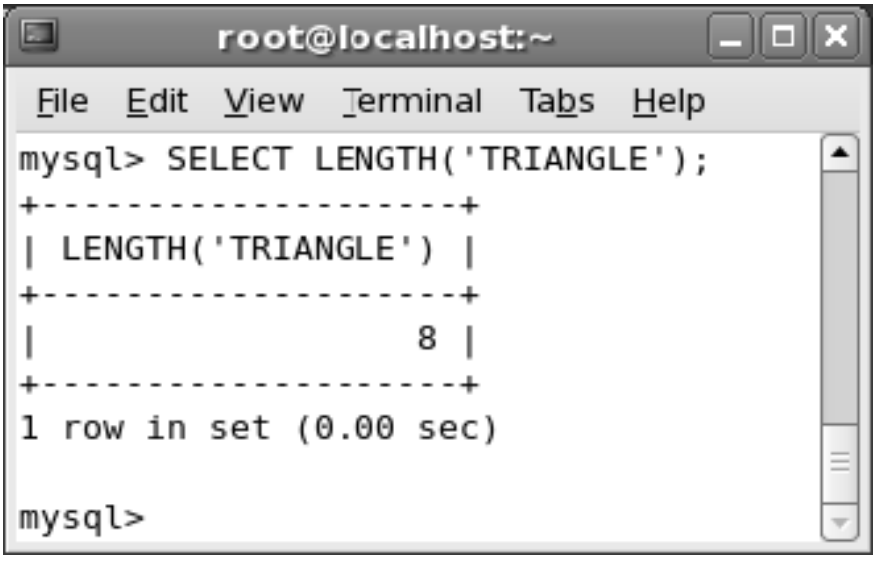


Figure 15.45: LENGTH Function

17. **The librarian wants to find the index position of the substring 'C' in the book name column. To find the index position of C in the BOOK_NAME column, enter the following command at the command prompt:**

```
SELECT LOCATE('C', BOOK_NAME) FROM BOOK_TABLE;
```

Figure 15.46 displays the output of the command.

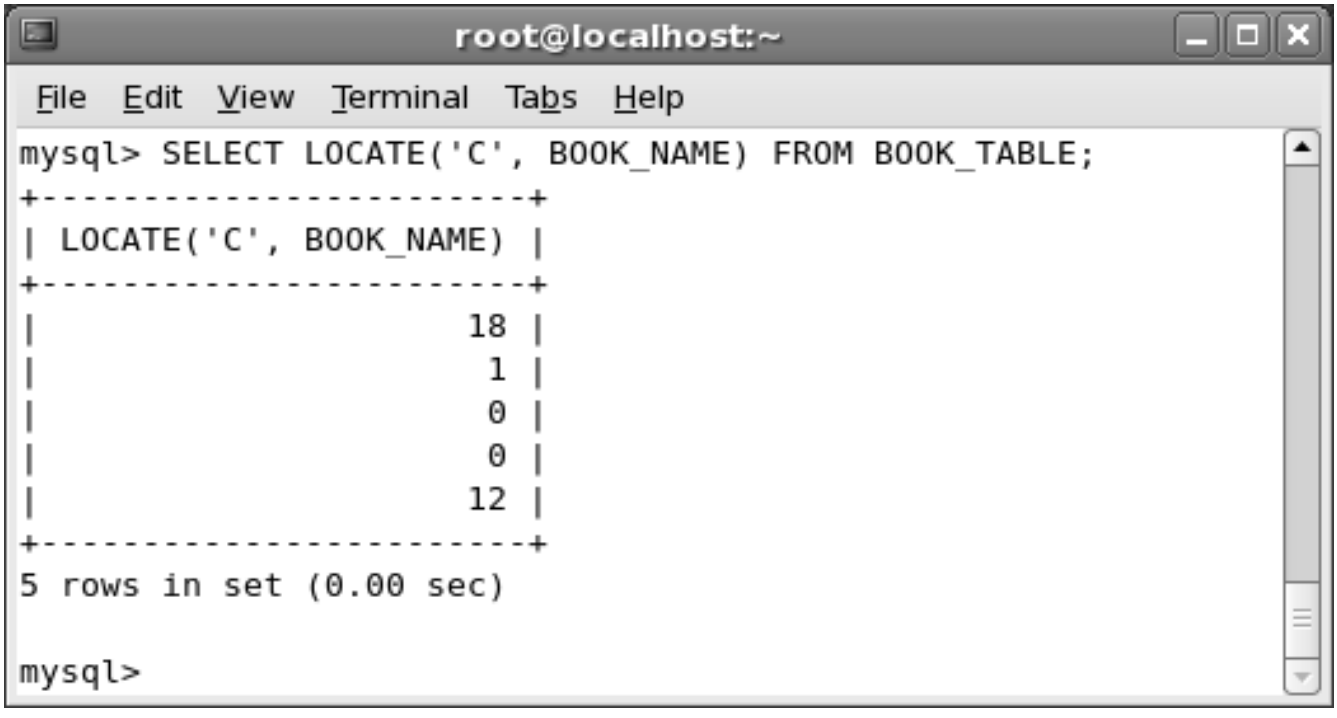


Figure 15.46: LOCATE Function

18. **To replicate the USER_ID thrice, enter the following command at the command prompt:**

```
SELECT USER_ID, REPEAT(USER_ID, 3)
FROM USER_DETAILS;
```

Figure 15.47 displays the output of the command.

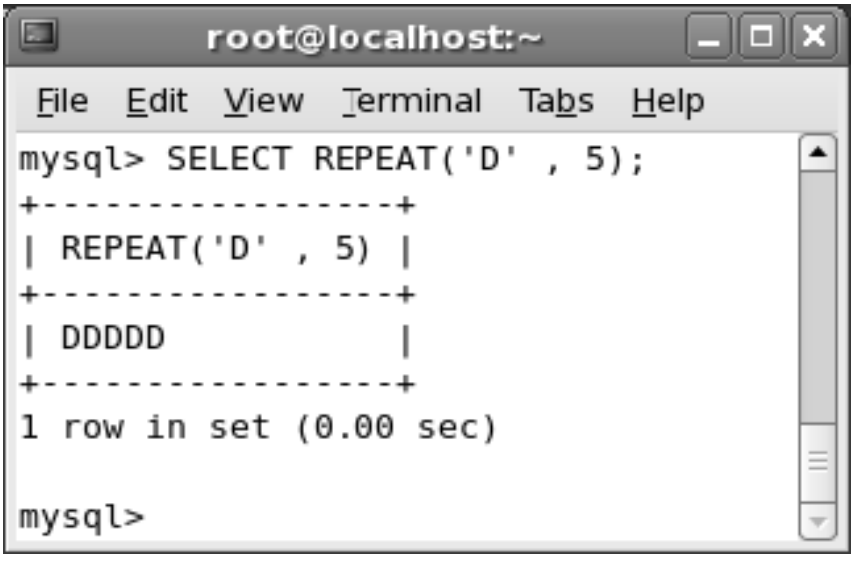


Figure 15.47: REPEAT Function

19. **The librarian wants to replace the word REGULAR with REG for the membership type. To replace the letter 'REGULAR' with 'REG' in the field TYPE, enter the following command at the command prompt:**

```
SELECT REPLACE(TYPE, 'REGULAR', 'REG') FROM CATEGORY;
```

Figure 15.48 displays the output of the command.

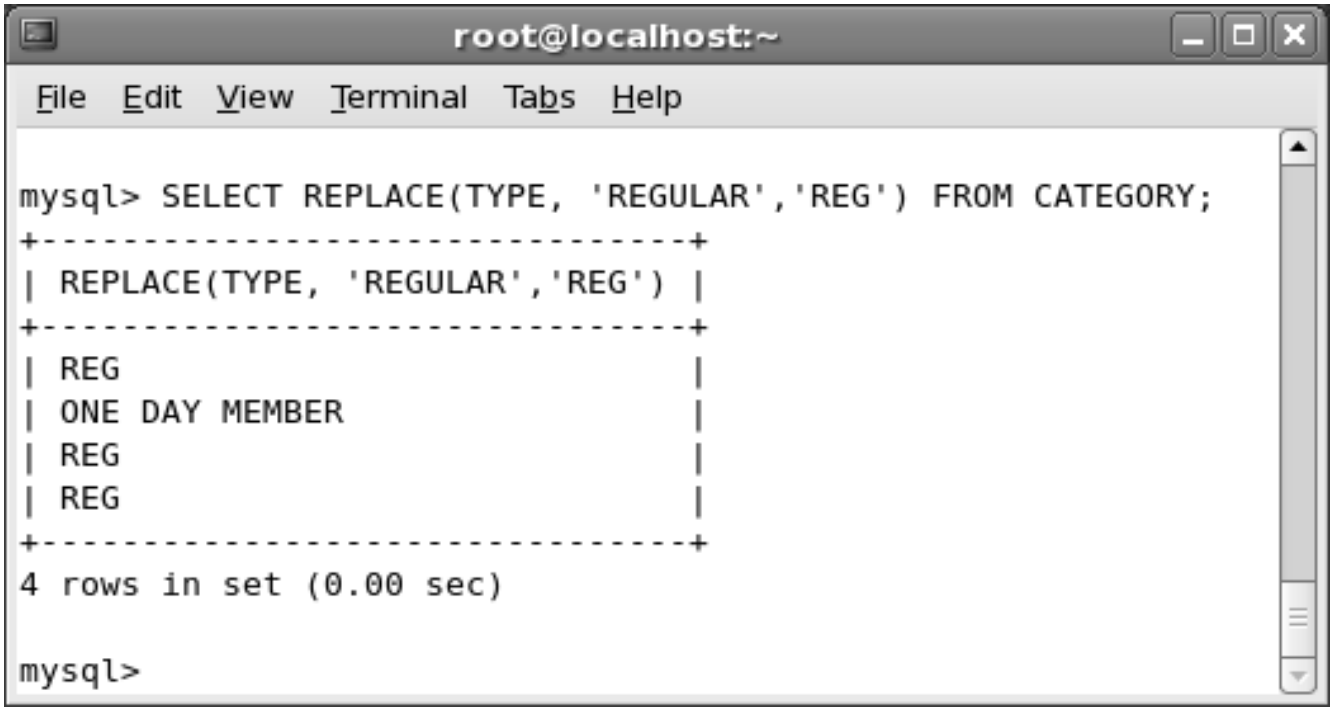


Figure 15.48: REPLACE Function

20. **To invert the sequence of the characters of EDITION, enter the following command at the command prompt:**

```
SELECT EDITION, REVERSE(EDITION)
FROM BOOK_TABLE;
```

Figure 15.49 displays the output of the command.

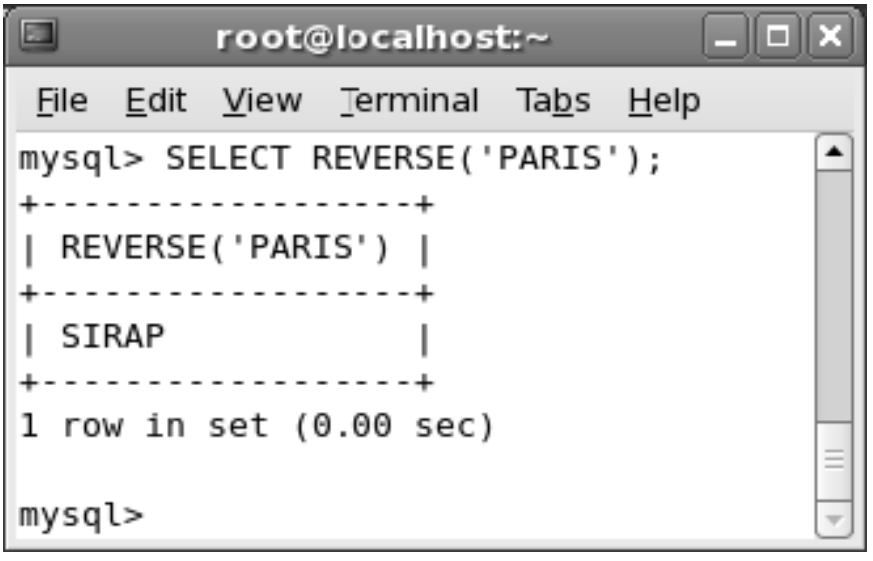


Figure 15.49: REVERSE Function

21. **To compare the strings 'ANGLE' and 'RECTANGLE', enter the following command at the command prompt:**

```
SELECT STRCMP('ANGLE', 'RECTANGLE');
```

Figure 15.50 displays the output of the command.

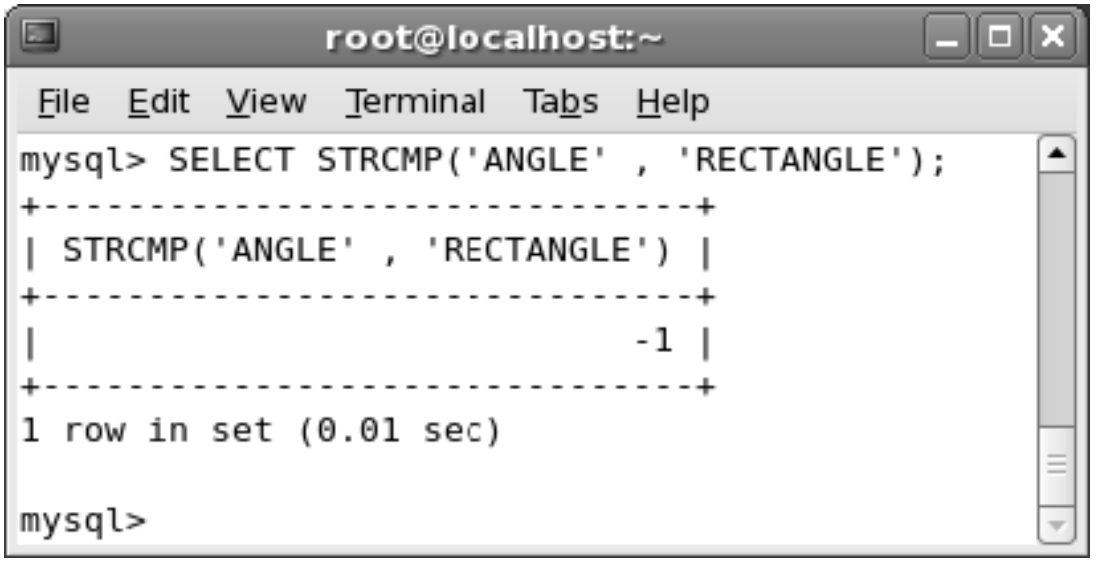


Figure 15.50: STRCMP Function

22. **The librarian wants to remove all the extra spaces existing before the book names from the BOOK_TABLE table. To remove the leading spaces in BOOK_NAME from the BOOK_TABLE table, enter the following command at the command prompt:**

```
SELECT TRIM(LEADING FROM BOOK_NAME) FROM BOOK_TABLE;
```

Figure 15.51 displays the output of the command.

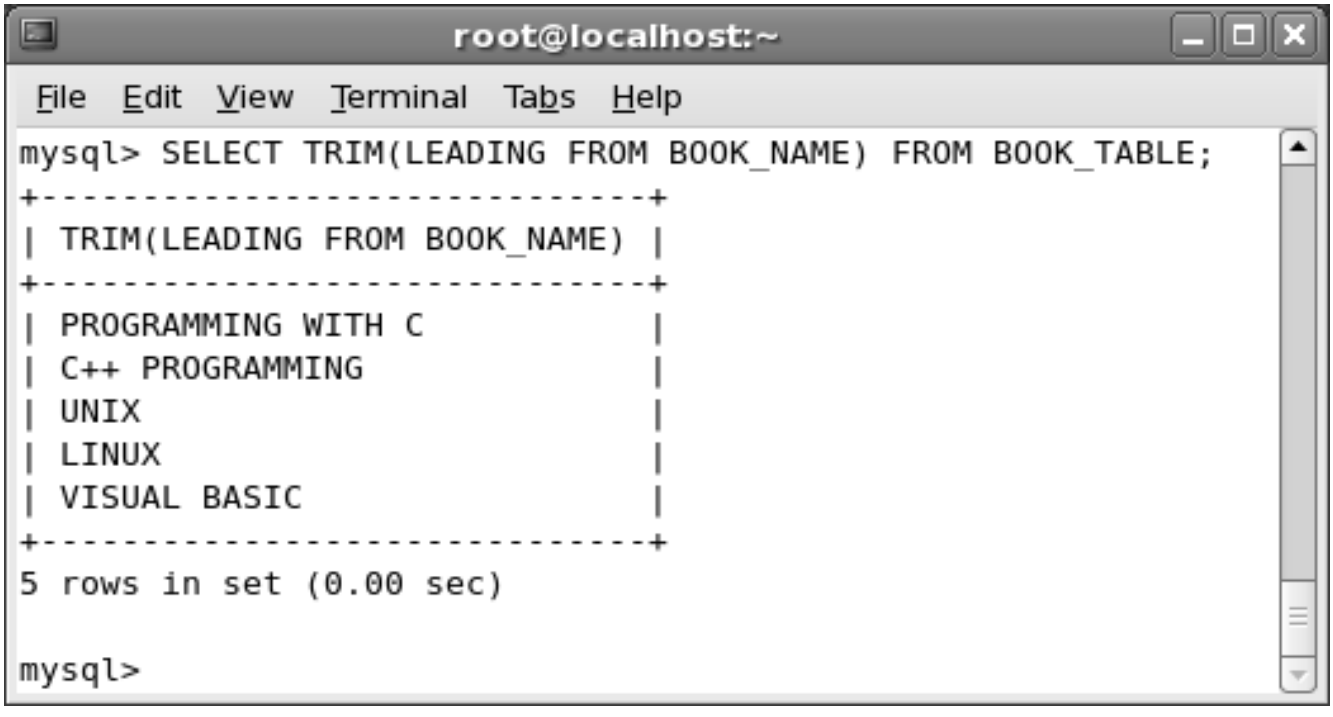


Figure 15.51: TRIM Function

System Information Functions in MySQL

System information functions are used to retrieve system-related information. The following steps illustrate the use of system information functions in MySQL:

1. **Will wants to know the charset being used in the active database, To display the character set being used for the string 'ABC', enter the following command at the command prompt:**

```
SELECT CHARSET('ABC');
```

Figure 15.52 displays the output of the command.

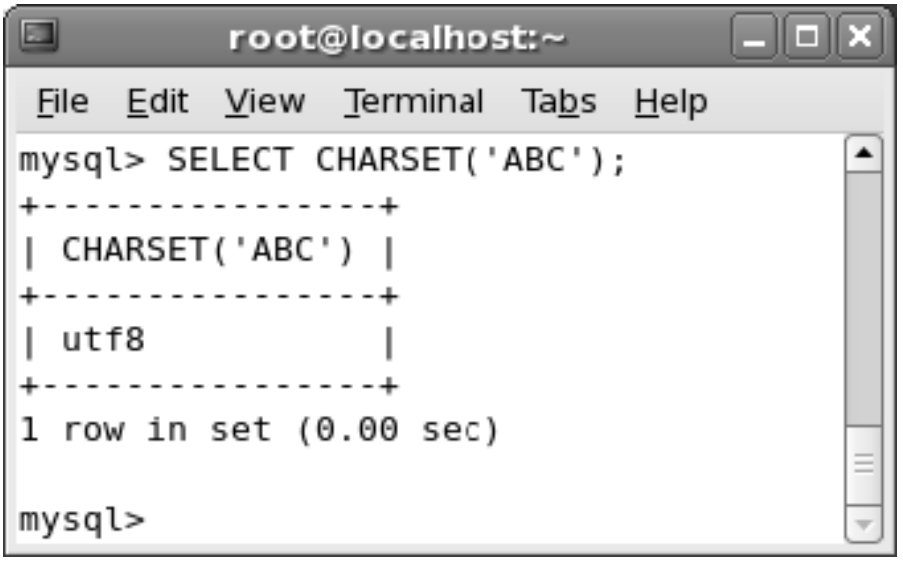


Figure 15.52: CHARSET Function

2. **Will wants to know the thread ID of the current connection. To display the thread id, enter the following command at the command prompt:**

```
SELECT CONNECTION_ID();
```

Figure 15.53 displays the output of the command.

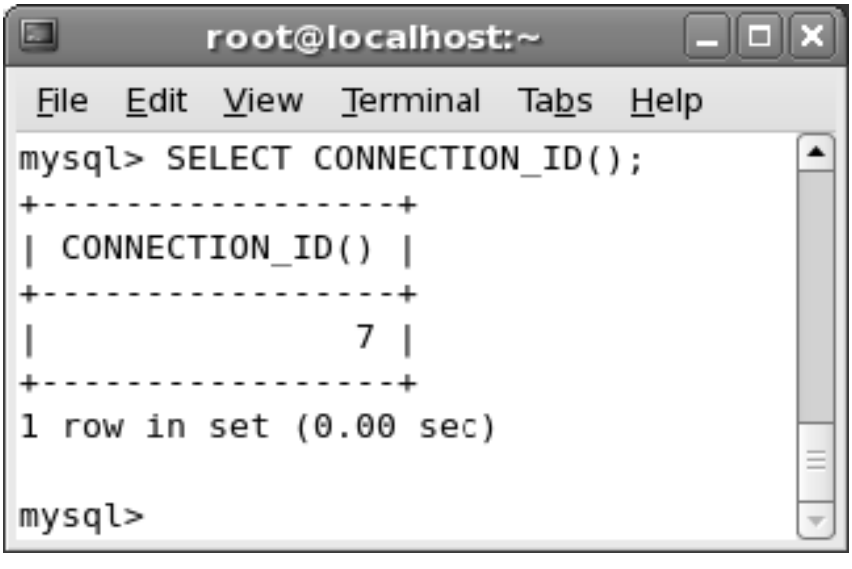


Figure 15.53: CONNECTION_ID Function

3. **Will wants to know the current user name. To display the current user name, enter the following command at the command prompt:**

```
SELECT CURRENT_USER();
```

Figure 15.54 displays the output of the command.

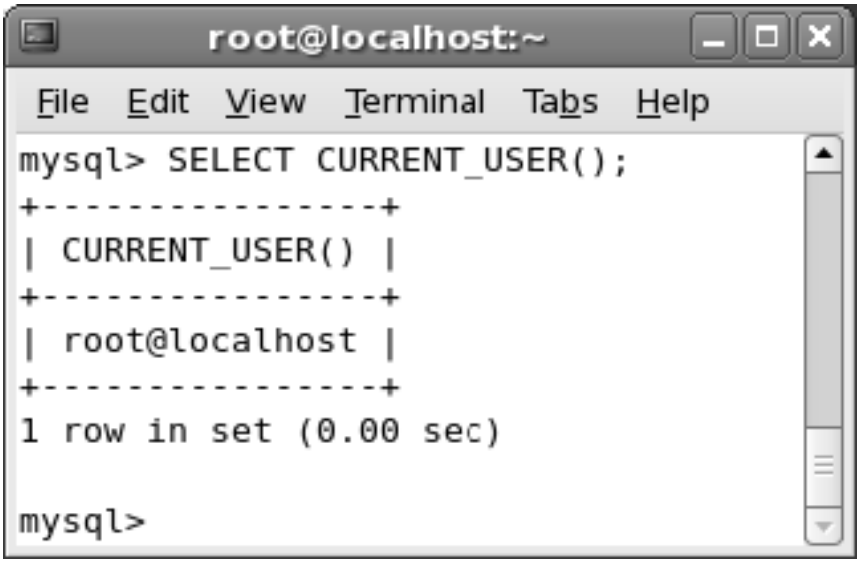


Figure 15.54: CURRENT_USER Function

4. **To view the database currently in use, enter the following command at the command prompt:**

```
SELECT DATABASE();
```

Figure 15.55 displays the output of the command.

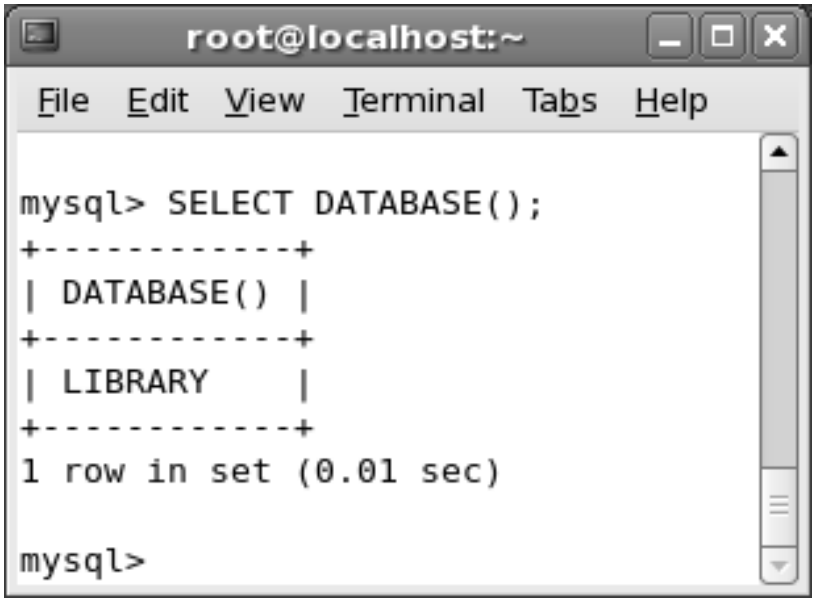


Figure 15.55: DATABASE Function

5. **To display the version of MySQL in use, enter the following command at the command prompt:**

```
SELECT VERSION();
```

Figure 15.56 displays the output of the command.

Figure 15.56: VERSION Function

Do It Yourself

1. Display the current date.
2. Display the current time.
3. Add an interval of `1` year to the date `2011-01-20`.
4. Subtract `2` days from the current date using the `DATE_SUB` function.
5. Display the day of month for the date `2009-06-25`.
6. Display the day of week for the date `2007-08-11`.
7. Display the day of year for `2009-10-20`.
8. Display the microseconds from `09:15:45.123456`.
9. Display the minute part from `05:30:45`.
10. Display the month from the date `2010-12-05`.
11. Calculate the difference in the number of months between the periods `199905` and `199604`.
12. Calculate the quarter for the date `2010-04-05`.
13. Display the seconds from `05:30:45`.
14. Display the current date and time value.
15. Convert the time expression `10:30:00` into seconds.
16. Display the time part from the expression `2010-04-15 20:20:30`.
17. Calculate the time difference between `21:30:00` and `23:30:00`.
18. Display the current timestamp value.
19. Convert the date `2010-04-05` into seconds.

Do It Yourself

20. Display the current UTC DATE.
21. Display the current UTC TIME.
22. Calculate the week number for the year `2005-04-20`.
23. Display the year from the date `2010-12-05`.
24. Calculate the length of string `ABC` in bits.
25. Calculate the character length of the string `XYZ`.
26. Join the strings `DATA` and `BASE` with a comma as the separator.
27. Display the index of the field `C` in the string `A`, `B`, `A`, `C`, `U`, `S`.
28. Calculate the hexadecimal value of the string `DATA`.
29. Convert the string `DYNAMICS` to lowercase.
30. Delete the leading whitespace from the string `'  DATA'`.
31. Calculate and display the ASCII value of the first character of `JOHN`.
32. Replicate the string `D` five times.
33. Display only the last five characters of the string `TRIANGLE` as output.
34. Delete the trailing whitespace of the string `'XYZ  '`.
35. Display a string with only five spaces.
36. Convert the string `abc` to uppercase.
37. Display the character for the hexadecimal string `EF`.
38. Use the `BENCHMARK` function to execute 1,000 nine times.

Do It Yourself

39. Display the thread ID of the current connection.

40. Display the row count.

41. Display the current username and hostname.

Onlinevarsity

TECHNO-WISE

THE TECHNOLOGIES OF TOMORROW

Session
16

Controlling and Managing MySQL Database

Concepts

Objectives

At the end of this session, the student will be able to:

- *Describe the creation of user accounts in MySQL.*
- *Identify the privileges in MySQL.*
- *Explain the privileges present in MySQL.*
- *Explain the commands for setting up of restrictions in MySQL.*

16.1 Introduction

A user account enables you to access a database. You require a user account to manage a database. User accounts enable you to add new users, create, modify, and delete tables and databases, and set privileges or access rights to other user accounts.

In this session, you will learn to create a user account, assign, and alter privileges to users. You will also learn to use privileges and control user access to databases. In addition, you will learn to rename a user account and delete a user account.

16.2 User Accounts

In MySQL, a user is a record present in the user table of the MySQL server. The table stores the username and password. After the installation of MySQL server is complete, the only existing user account is `root` user account. MySQL uses the `root` user account to execute administrative commands. A `root` user has unrestricted access across the database. All users are required to have a user account to access the database. MySQL server does not require a password for the `root` user, if you have not set a password at configuration. MySQL allows the `root` user to set a password using the `mysqladmin` command. MySQL stores user accounts and privileges as tables in a default database named `mysql`. In MySQL, you can create a new user account by using the `CREATE USER` or `GRANT` command. In addition, you can also create user accounts by manipulating the MySQL grant tables directly.

After an account is created, you can set the password, modify and revoke the privileges, rename the account, and delete the account. MySQL provides different commands and options to execute these tasks. Table 16.1 lists the commands for user accounts in MySQL.

MySQL Command	Description
`CREATE USER`	Allows the creation of a new account

Concepts

MySQL Command	Description
SET PASSWORD	Allows to assign a password
GRANT	Allows to assign privileges
RENAME USER	Allows to change the name of the account
REVOKE	Allows to cancel or remove privileges
DROP USER	Allows to delete an account

Table 16.1: User Account Tasks and MySQL Commands

You must login as the `root` user to create user accounts in MySQL.

Note: To connect as the `root` user, enter the following command at the command prompt:

```
mysql – u root –p
```

The MySQL Grant tables contain information about the privileges to user accounts. These tables are stored in the `mysql` database and their names are as follows:

- `columns _ priv` – contains access details for columns in tables
- `db` – contains information of user accounts that can access the database
- `host` – contains list of user accounts that have access to the database
- `tables _ priv` – contains access details for tables in a database
- `user` – contains access details for a user account

16.2.1 Adding User Accounts Using the CREATE USER Command

MySQL server provides the `CREATE USER` command to add new accounts. You must have the global CREATE USER privilege for the `mysql` database to execute the command. MySQL server creates a new row in the user table of `mysql` database and assigns privileges when you add a new user account.

You must login as the root user and connect to the `mysql` database to create user accounts.

To view the list of available databases, enter the following command at the command prompt:

```
SHOW DATABASES;
```

Figure 16.1 displays a list of available databases, including `mysql`, the default database.

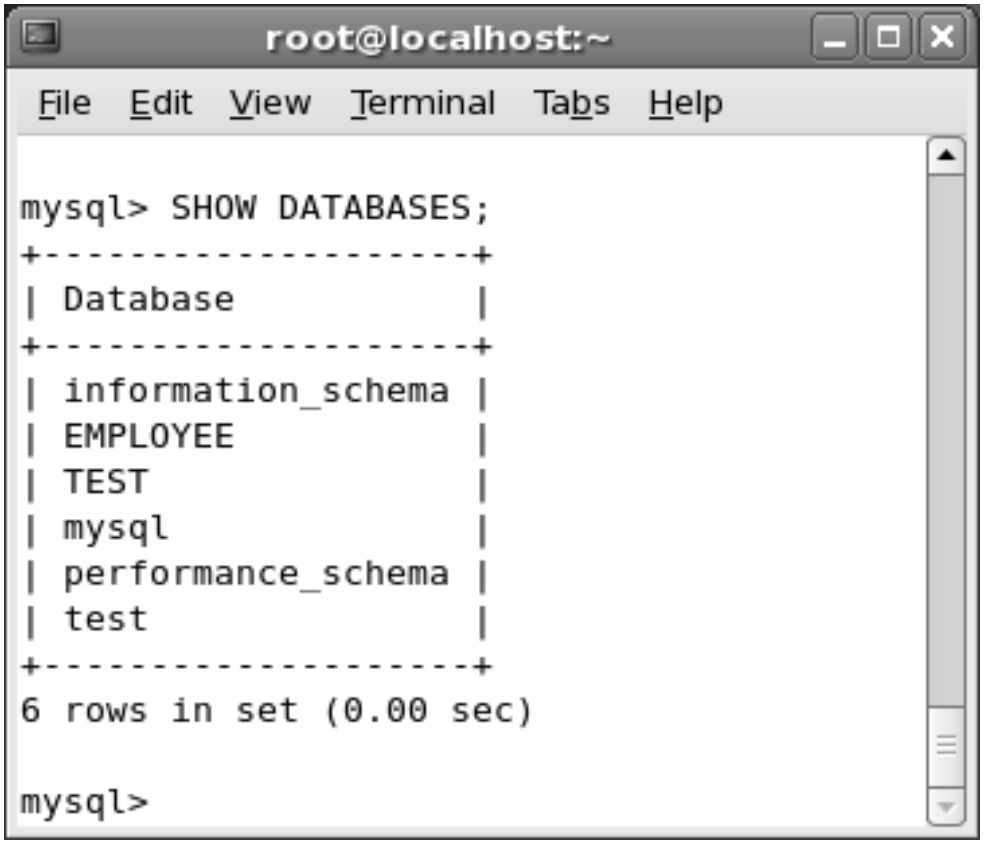


Figure 16.1: Default mysql Database

Note: Ensure that the root user has the `INSERT` and `RELOAD` administrative privileges.

To load the default `mysql` database, enter the following command at the command prompt:

```
USE mysql;
```

This command will change the current database to `mysql`.

Figure 16.2 displays the output of the command.

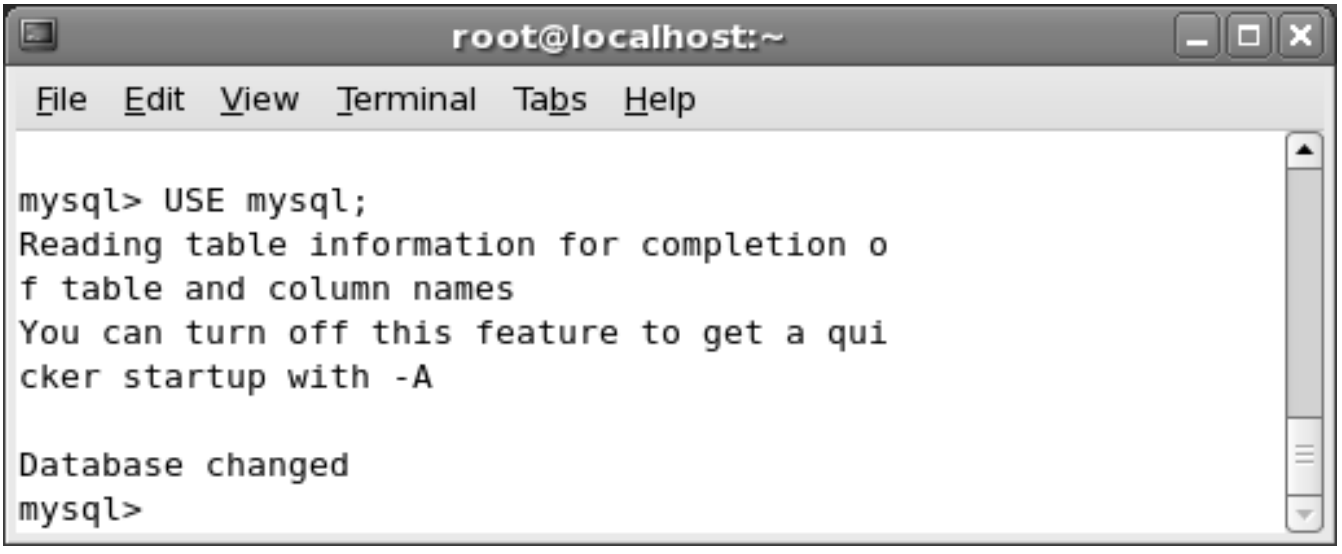


Figure 16.2: Activating mysql Database

To list the tables present in the default database `mysql`, enter the following command at the command prompt:

```
SHOW TABLES;
```

This command, lists all the available tables in the default database.

Figure 16.3 displays the output of the command.

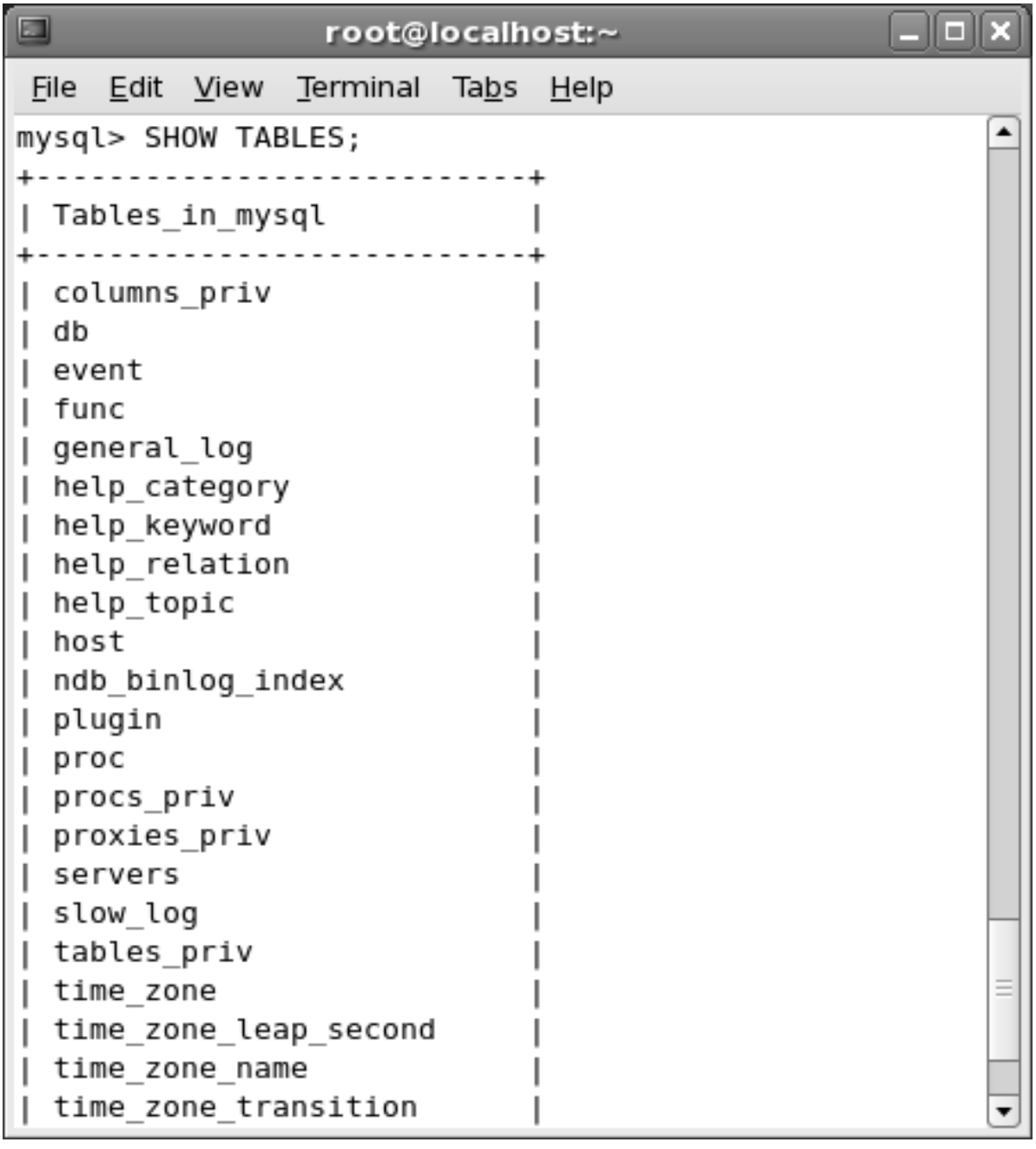


Figure 16.3: Tables in mysql Database

In figure 16.3, the `mysql` database lists the tables that describe the user access privileges. Table 16.2 lists the functions of some of the tables in the `mysql` database.

Table Name	Description
`columns_priv`	Contains access details for columns in tables

Concepts

Table Name	Description
db	Contains information of user accounts that can access the database
func	Contains list of functions and is updated at MySQL server restart
host	Contains list of user accounts that have access to the database
proc	Contains information about database and table modifications
tables_priv	Contains access details for tables in a database
user	Contains access details for a user account

Table 16.2: Default Tables in mysql Database

There are different commands for creating a user account. However, all these commands require you to connect as the `root` user and change the database to `mysql`. The `CREATE USER` command allows you to add a user account.

To add a user account using the `CREATE USER` command, enter the following command at the command prompt:

```
CREATE USER 'temp_user'@'localhost';
```

This command will add a new user named `temp_user` for the instance `localhost` of MySQL server.

16.2.2 Creating User Account Using the GRANT Command

You can also create a new user account using the `GRANT` command. In addition, you can also use the `GRANT` command to assign rights and privileges.

You can grant and revoke rights to a MySQL user account at four levels that are as follows:

- **Global Level** - grants and revokes privileges that apply to all the databases present on the server. These privileges are stored in the `user` table of the `mysql` database.
- **Database Level** - grants and revokes privileges that apply to all the tables of a given database. These privileges are stored in the `db` and host tables of the `mysql` database.
- **Table Level** - grants and revokes privileges that apply to all the columns of a given table. These privileges are stored in the `tables _ priv` table of the `mysql` database.
- **Column Level** - grants and revokes privileges that apply to a single column in a given table. These privileges are stored in the `columns _ priv` table of the `mysql` database.

Table 16.3 lists the privilege types that can be specified in the `GRANT` and `REVOKE` statements.

Privilege	Description
ALL [PRIVILEGES]	Assigns all rights except GRANT OPTION. The user cannot set access controls.
ALTER	Allows the use of the ALTER TABLE command
CREATE	Allows the use of the CREATE TABLE command
CREATE TEMPORARY TABLES	Allows the use of the CREATE TEMPORARY TABLE command
DELETE	Allows the use of the DELETE command
DROP	Allows the use of the DROP TABLE command
EXECUTE	Allows the user to run stored procedures
INDEX	Allows the use of the CREATE INDEX and DROP INDEX commands
INSERT	Allows the use of the INSERT command
SELECT	Allows the use of the SELECT command
SHOW DATABASES	Allows the use of the SHOW DATABASES command
SHUTDOWN	Allows the use of the mysqladmin shutdown privilege
UPDATE	Allows the use of the UPDATE command
USAGE	Allows the use of the 'no privileges' user account
GRANT OPTION	Allows privileges to be granted or revoked from other accounts

Table 16.3: Privileges in GRANT and REVOKE commands

Note: You will have to execute the mysql_upgrade script before using some of these commands.

The syntax for creating a new user with all privileges for all the tables in the database and who can connect to the localhost or any other host is as follows:

```
GRANT ALL [PRIVILEGES] ON *.* TO user_name [IDENTIFIED BY [PASSWORD] 'password'
[WITH GRANT OPTION]
```

where,

user_name – specifies the name for the new account

IDENTIFIED BY [PASSWORD] – sets the password for the new account

password – specifies the password for the account

[WITH GRANT OPTION] – allows the user to set privileges

In the syntax, the PASSWORD and the GRANT OPTION clauses are optional.

Concepts

Note: The `GRANT` command supports table, database, column, and routine names upto 64 characters in length. Length of user name cannot be more than 16 characters long, while host names cannot be more than 60 characters long.

To create a new user with all the privileges and access to MySQL from `localhost`, enter the following command at the command prompt:

```
GRANT ALL PRIVILEGES ON *.* TO 'paul'@'localhost' IDENTIFIED BY 'pass' WITH
GRANT OPTION;
```

The command will assign all privileges to the user `paul` present in the instance, `localhost`. The clause `WITH GRANT OPTION` will allow the user to set privileges.

Figure 16.4 displays the output of the `GRANT` command.

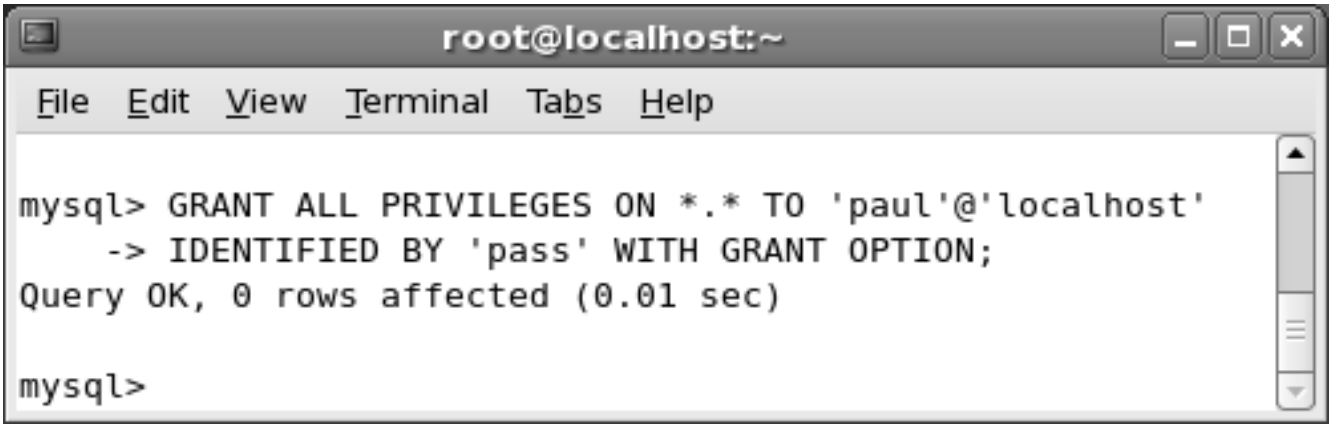


Figure 16.4: Create User Using the GRANT command

In figure 16.4, a new user named `paul` with the password pass is created. MySQL server has assigned all the rights on all the databases present on this instance of MySQL server named `localhost`, to the user. Such a user is referred to as a super user.

Similarly, to create a new user with all the privileges without specifying the `localhost` or any other host, enter the following command at the command prompt:

```
GRANT ALL PRIVILEGES ON *.* TO 'paul'@'%' IDENTIFIED BY 'pass' WITH GRANT
OPTION;
```

Figure 16.5 displays the output of the `GRANT` command.

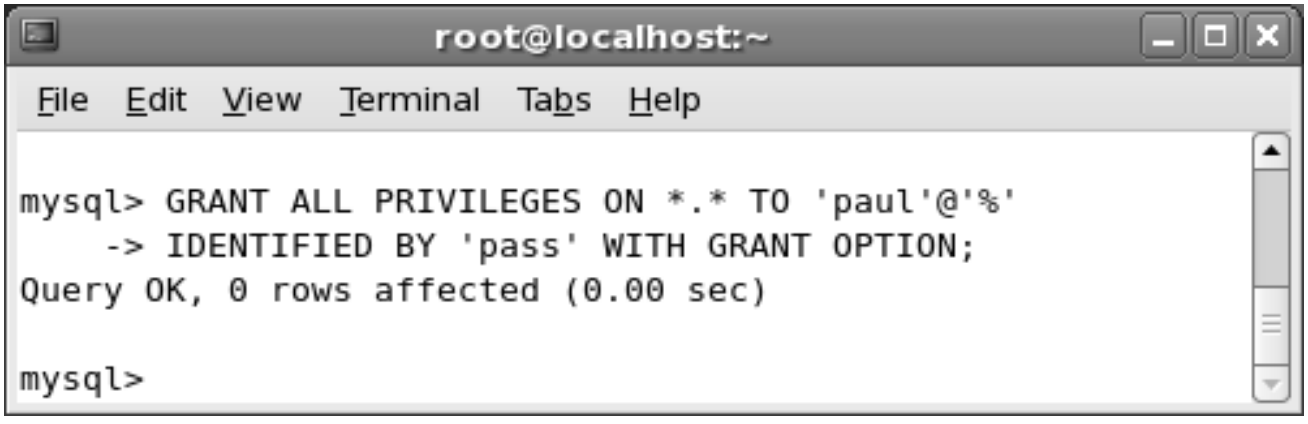


Figure 16.5: Creating a User Without Specifying the Localhost

In figure 16.5, MySQL creates a new user named `paul` with the password set as pass. This user is also a super user who has all the rights on all hosts.

16.2.3 Create User Account Using the INSERT command

MySQL also provides the `INSERT` command to create user accounts. The `INSERT` command adds a new entity to the `user` table of the `mysql` database. This table contains information of all user accounts in the database with their hostnames, passwords, and privileges. The newly created user can be assigned the rights using the `GRANT` command.

Note: To grant rights to the new user, set the privilege columns to `Y`; where `Y` stands for `Yes`. Thus, the user is granted only those privileges, which are set to `Y`.

To create a user by the name `martin` using the `INSERT` command, enter the following command at the command prompt:

```
INSERT INTO user VALUES ('localhost','martin', PASSWORD ('pass'),'Y','Y','Y
','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y', 'Y','Y','Y','Y','Y','Y','Y',
'Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y');
```

The command will add a record into the user table with the values specified in the `VALUES` clause.

Figure 16.6 displays the output of the command.

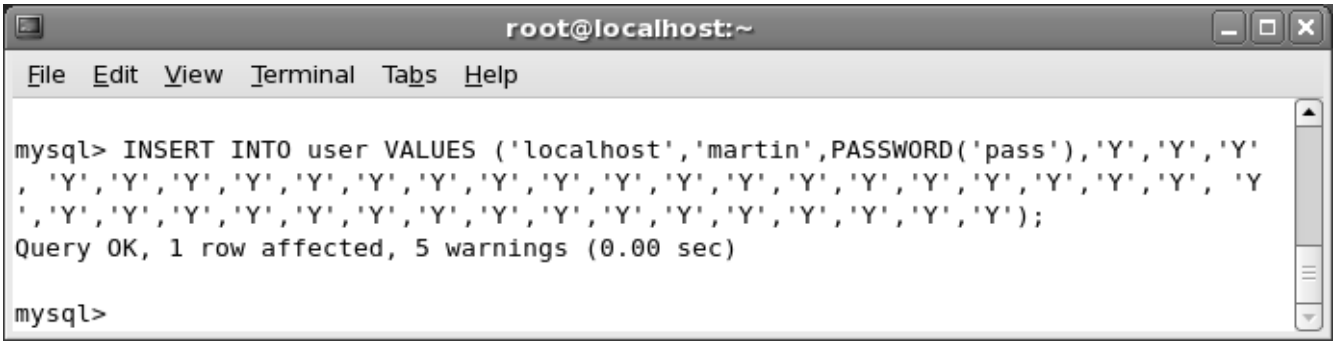


Figure 16.6: Creating User Using INSERT Command

In figure 16.6, `user` is a table in the `mysql` default database that stores details of the user accounts and the privileges granted to the user. The `localhost` is the instance for the newly created user called `martin`. The password for the user has been set to `pass`. The values in the privilege columns have been set as `Y`.

Note: The command used in figure 16.6 will not execute in MySQL installed on the Microsoft Windows operating system.

Table 16.4 lists some of the column names in the user table of `mysql` database.

Column Name
Select_priv
Insert_priv
Update_priv
Delete_priv
Index_priv
Alter_priv
Create_priv
Drop_priv
Grant_priv
Create_view_priv
Show_view_priv
Create_routine_priv
Alter_routine_priv
Execute_priv
Create_tmp_table_priv
Lock_tables_priv
References_priv
Reload_priv
Shutdown_priv

Column Name
Process_priv
File_priv
Show_db_priv
Super_priv
Repl_slave_priv
Repl_client_priv
Create_user_priv

Table 16.4: Privilege Columns in user Table

After creating a user with the `INSERT` command, you must run the `FLUSH PRIVILEGES` command. This command reloads all the grant tables and assigns the new privileges.

The syntax for the `FLUSH PRIVILEGES` command is:

```
FLUSH PRIVILEGES;
```

where,

`FLUSH` – specifies the reloading of the object

`PRIVILEGES` – specifies to refresh and reload the privileges

Note: MySQL will not assign the new privileges until you execute the `FLUSH PRIVILEGES` command.

Figure 16.7 displays the output of the command.

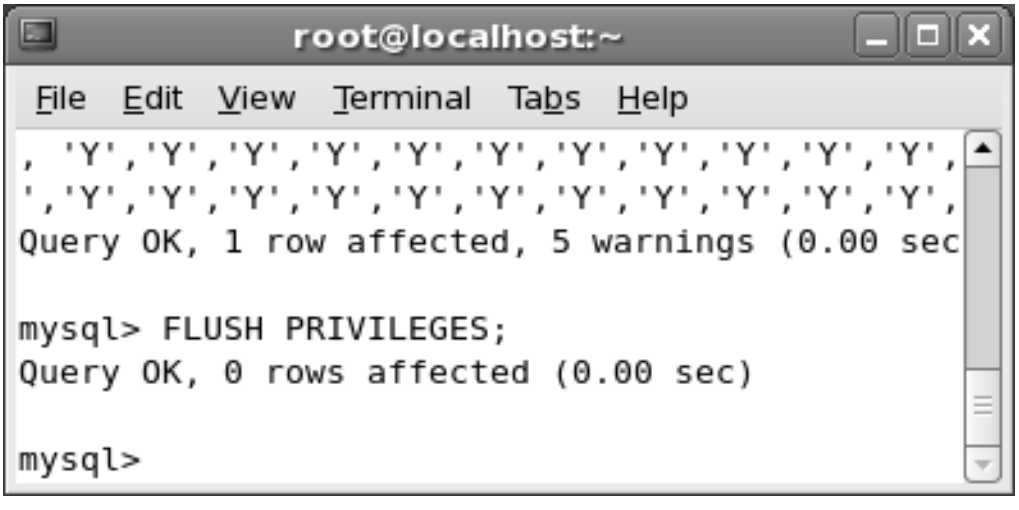


Figure 16.7: FLUSH Privileges

To create a user having only RELOAD and PROCESS privileges, enter the following command at the command prompt:

```
INSERT INTO user SET Host='localhost', User='john', Reload_priv='Y', Process_
priv='Y';
```

The command will add an entry into the user table and will set the value in the Host, User, Reload_priv, and Process_priv columns only. The Host column contains the instance name of MySQL. The Reload_priv and Process_priv assigns the RELOAD and PROCESS privileges.

Figure 16.8 displays the output of the command.

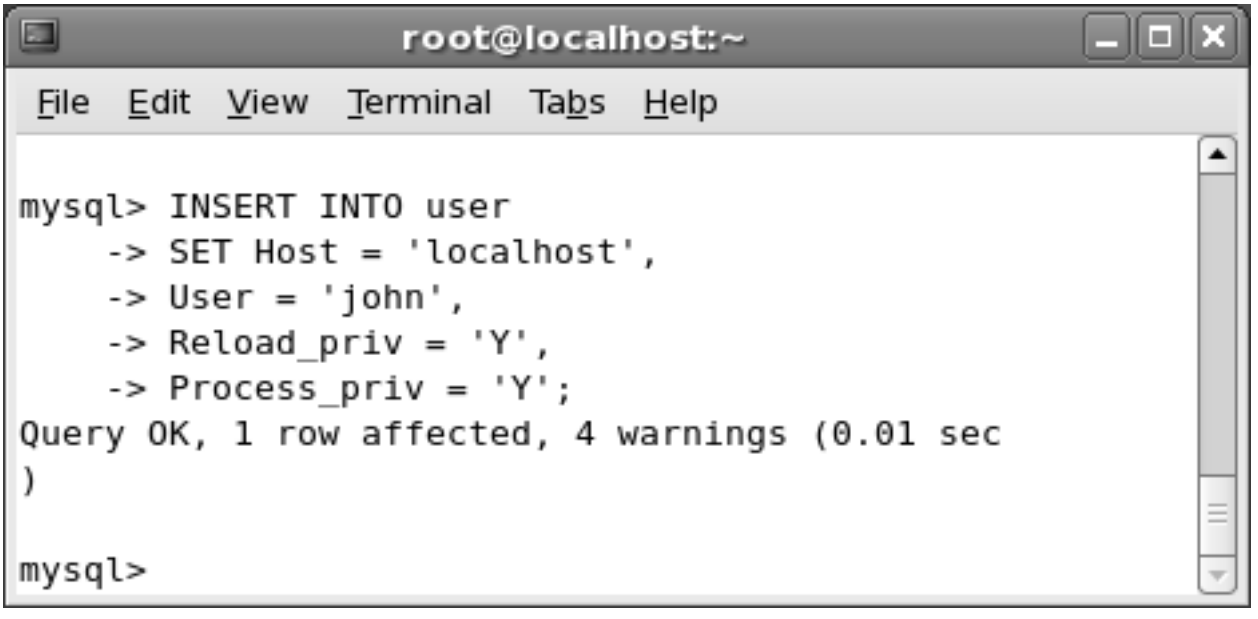


Figure 16.8: Specific Privileges For User

In figure 16.8, the hostname is set as localhost for the user named john with the help of SET command. The user is created without any password. Only the Reload_priv and the Process_priv privilege columns are set to Y.

Note: The command used in figure 16.8 will not work in MySQL installed on Microsoft Windows operating system.

To create an account without any access and password privileges, enter the following command at the command prompt:

```
INSERT INTO user (Host, User, Password) VALUES ('localhost','martin', ' ');
```

The command adds a new entry into the user table and specifies the hostname, user name, and password.

Figure 16.9 displays the output of the command.

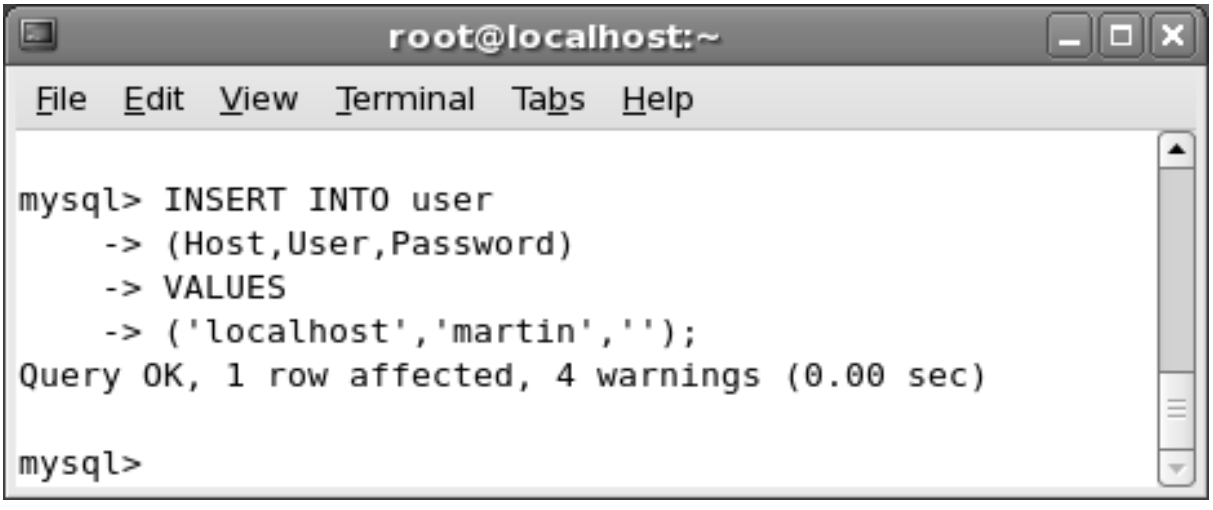


Figure 16.9: Create User Without Setting Any Rights

In figure 16.9, the `Host`, `User`, and `Password` columns in the `user` table are updated. By default, MySQL assigns 'N' to all privilege columns if you do not specify the values for the privilege columns.

Note: Ensure to execute the `FLUSH PRIVILEGES` command after making changes to the grant tables. Also, the command used in figure 16.9 will not work in MySQL installed on Microsoft Windows operating system.

Ensuring Valid Connections

MySQL uses security for all connections and queries. The server controls access by ensuring that the following checks are performed:

- Checks whether the user has privileges to connect to the server. If the user has valid credentials, the connection is established.

- Verifies the privileges assigned to the user after the connection is established. MySQL executes the requested action if and only if the user has the necessary privileges.

Note: To control the users accessing the database, use administrative commands such as `mysql` or `mysqladmin`. To use the administrative commands, remember to login as the `root` user.

Concepts

16.2.4 Rename User Account

MySQL enables you to change the name of an existing user account. You can edit the name of the account using the `RENAME USER` command. MySQL enables you to change the name of the account without modifying the privileges. However, you must have the `UPDATE` or the `CREATE USER` privilege for the `mysql` database to execute the `RENAME USER` command.

For example, to rename a user account, enter the following command at the command prompt:

```
RENAME USER 'temp_user'@'localhost' TO 'temp-user1'@'localhost';
```

The command changes the name of the object by specifying the type of object to rename.

16.3 Altering Privileges

Managing privileges of users involves assigning and revoking user rights and privileges. You can use database-independent SQL commands, such as `GRANT` and `REVOKE` to manage user privileges. You can assign rights and privileges to an account using the `GRANT` command. The `REVOKE` command enables you to remove the rights granted to the user.

To alter privileges for the user, use `GRANT` command with selective privileges. For example, the command to grant permission to a user to read data from a specified table is as follows:

```
GRANT SELECT ON table_name TO user_name;
```

where,

`GRANT` – assigns privileges or rights

`SELECT` – specifies the type of privilege or right to assign

`table_name` – specifies the name of the table

`user_name` – specifies the name of the user or account

The command assigns the `SELECT` permission to the user specified in the `user_name` clause of the statement. The user can read data from the table specified in the `table_name` clause of the statement.

For example, to create a user who has the permission to read data from the `EMP_DETAILS` table, enter the following command at the command prompt:

```
GRANT SELECT ON EMP_DETAILS TO 'carol'@'localhost';
```

The command will assign the SELECT privilege to the user, carol.

Figure 16.10 displays the output of the command.

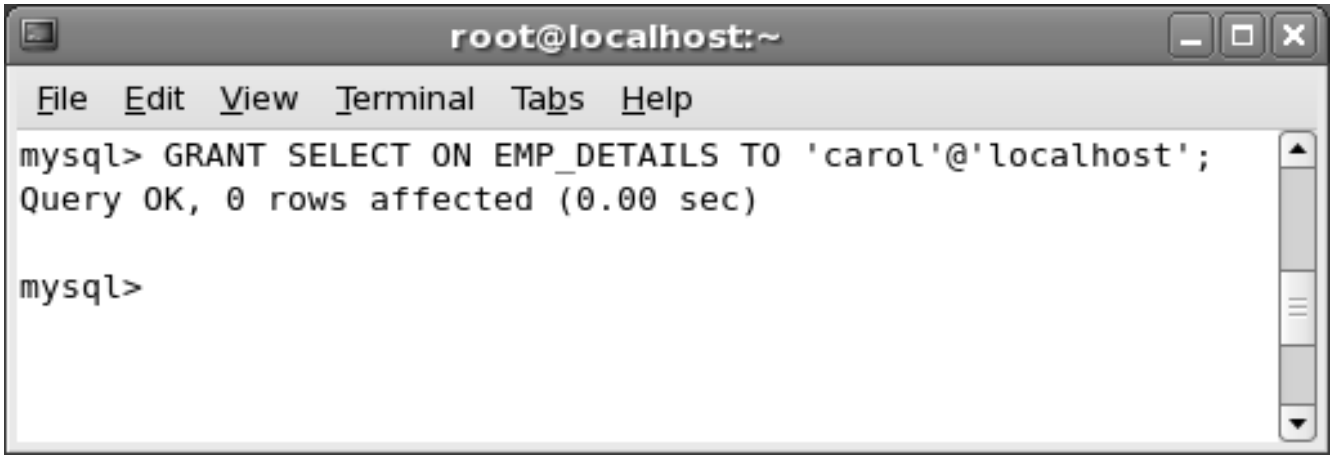


Figure 16.10: Create User Using GRANT command

In figure 16.10, the user named carol is created and granted the privilege of reading data from the EMP_DETAILS table.

Note: The user has not been assigned any password and has access from all hosts.

Similarly, to create a user who is granted only with the RELOAD and the PROCESS rights for the entire database, with no password and has access only from the host named, localhost, enter the following command at the command prompt:

```
GRANT RELOAD,PROCESS ON *.* TO 'admin'@'localhost';
```

Figure 16.11 displays the output of the command.

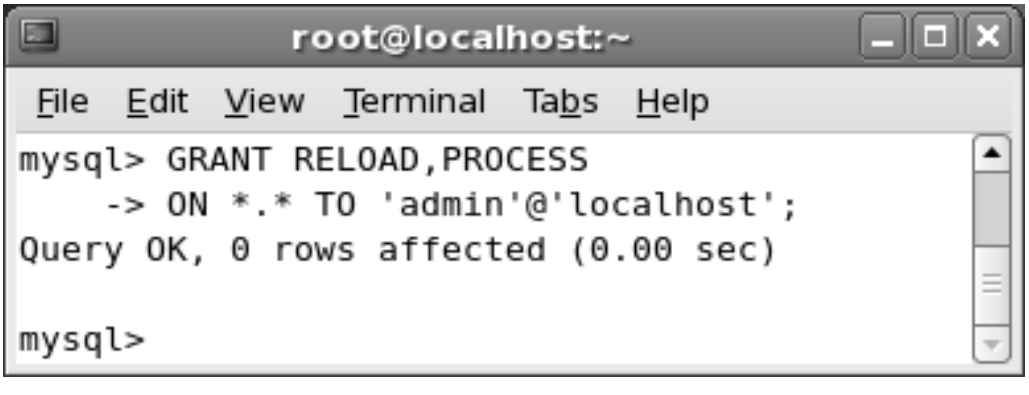


Figure 16.11: Create User with RELOAD and PROCESS Privileges

In figure 16.11, the RELOAD and PROCESS privileges are granted to the admin user for executing the mysqladmin reload, mysqladmin refresh, mysqladmin flush-* , and mysqladmin processlist commands.

To grant the USAGE privilege to the user, enter the following command at the command prompt:

```
GRANT USAGE ON *.* TO 'carol'@'localhost';
```

Figure 16.12 displays the output of the command.

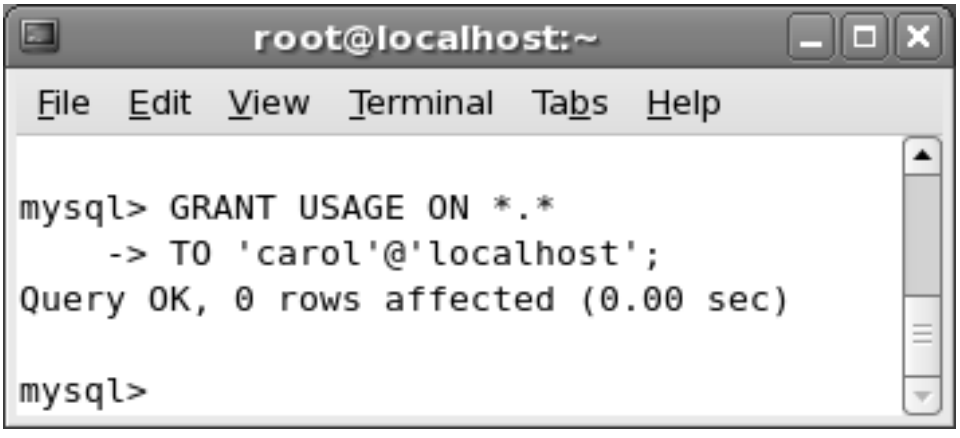


Figure 16.12: Creating User with the USAGE Privilege

In figure 16.12, MySQL assigns only the USAGE privilege to the new user named carol.

The USAGE privilege allows you to create an account without assigning any privileges. You can use such accounts only to establish a connection. The USAGE option sets all the global privileges to N. This privilege assumes that the user will grant specific privileges to the account later.

To create a user who has access to the EMPLOYEE database from localhost only and with selective rights, enter the following command at the command prompt:

```
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP ON EMPLOYEE.* TO carol@
localhost IDENTIFIED BY 'pass';
```

Figure 16.13 displays the output of the command.

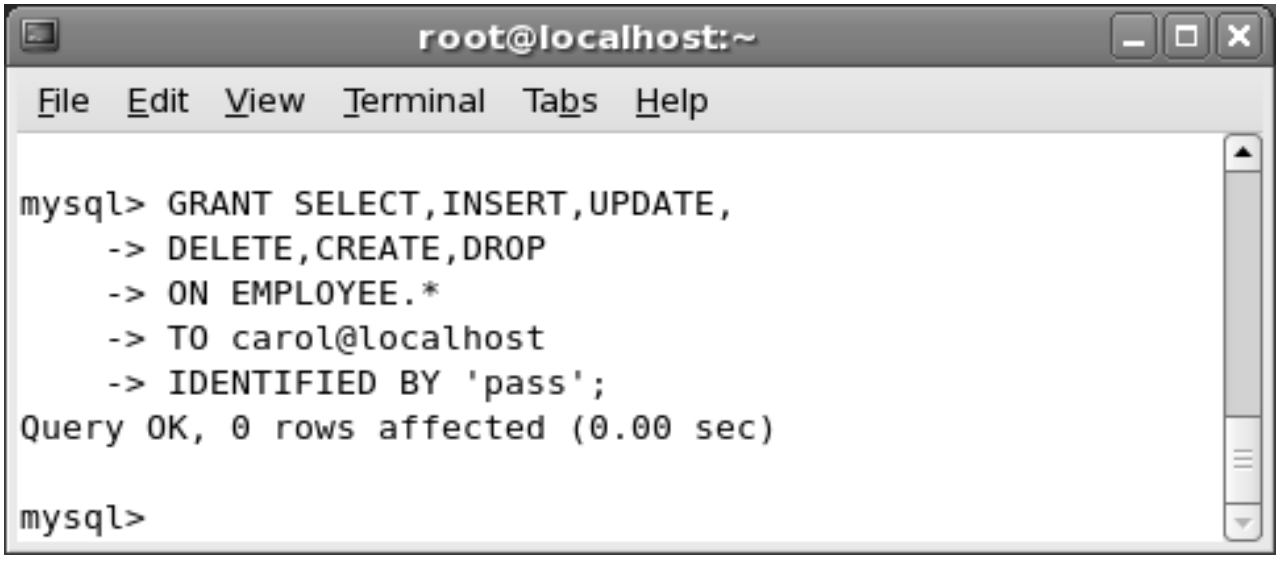


Figure 16.13: Create User with Selective Rights

In figure 16.13, MySQL assigns limited rights, such as select, insert, update, delete, create, and drop tables to the user name `carol` in the `EMPLOYEE` database.

The syntax to grant access to a specific database is:

```
GRANT privilege ON {db_name.*} TO user_name [IDENTIFIED BY [PASSWORD]
'password' [WITH GRANT OPTION]
```

where,

`GRANT` – specifies to assign privileges

`privilege` – specifies the type of permission to assign

`db_name` – specifies the name of the database

`user_name` – specifies the name of the account

`IDENTIFIED BY [PASSWORD]` – sets a password for the user account

`[WITH GRANT OPTION]` – allows the account to set privileges

For example, to create a user who has access to the LIBRARY database from the `server.domain` host, enter the following command at the command prompt:

```
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP ON LIBRARY.*
TO 'carol'@'server.domain' IDENTIFIED BY 'pass';
```

Figure 16.14 displays the output of the command.

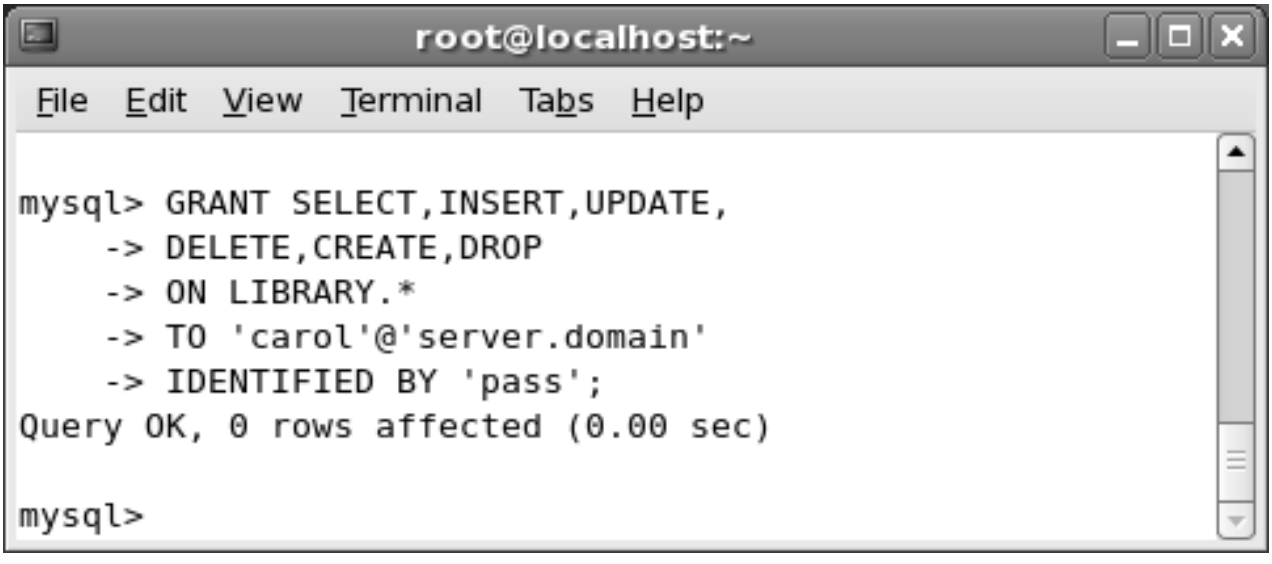


Figure 16.14: Create User with Access to Specific Database from a Specific Host

In figure 16.14, MySQL assigns limited rights and allows the user `carol` to access the `LIBRARY` database from the `server.domain` host.

To give access to a specific user from any machine in a given domain, enter the following command at the command prompt:

```
GRANT ALL ON *.* TO 'carol'@'%.mydomain.com' IDENTIFIED BY 'pass';
```

Figure 16.15 displays the output of the command.

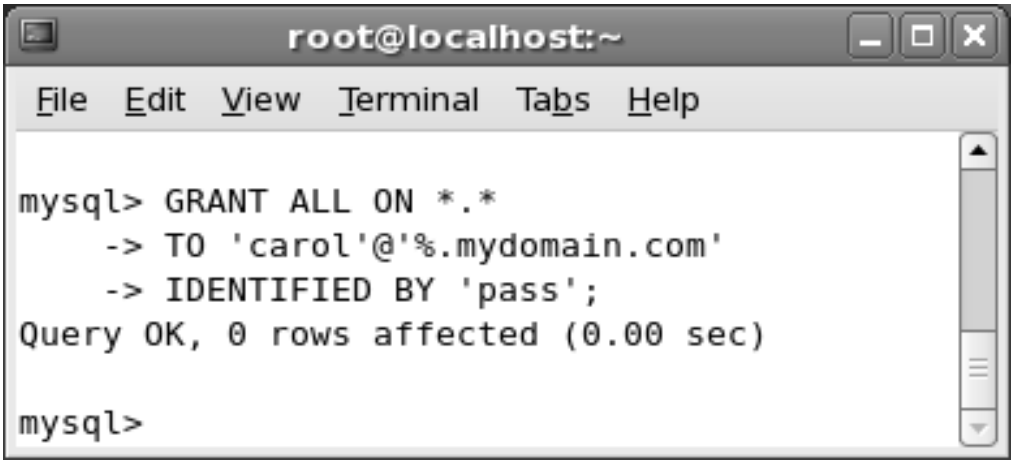


Figure 16.15: Create User with Access to any Machine in a Given Domain

In figure 16.15, the user `carol` is assigned access from any machine included in the `mydomain.com` host. The user is provided with all rights and privileges.

16.3.1 Revoking Privileges

MySQL provides the REVOKE command to remove rights or privileges from user accounts.

Note: The `REVOKE` command only removes privileges. It does not delete `mysql.user` table entries.

For example, to cancel or revoke the `INSERT` privilege for a user, enter the following command at the command prompt:

```
REVOKE INSERT ON *.* FROM 'temp_user' @ 'localhost';
```

The command cancels the `INSERT` privilege assigned to `temp_user` on the `localhost` instance.

Note: To use the `REVOKE` command, you must have the `GRANT OPTION` privilege. Also, you must have the privileges that you are revoking.

16.3.2 Deleting User Account

MySQL provides the `DROP USER` command to remove a user account. You can also use this command to remove rows from the grant tables that contain privileges for user accounts.

To delete a user account, enter the following command at the command prompt:

```
DROP USER user_name;
```

where,

`DROP` – specifies to remove the object from the database

`USER` – specifies the type of object to be removed from the database

`user_name` – specifies the name of the account to be removed from the database

Note: You must be careful when executing the `DROP USER` command.

You can also specify the domain to which the user account belongs while using the `DROP USER` command.

For example,

```
DROP USER 'carol'@'localhost';
```

This command will delete the user account `carol` on the `localhost` instance of MySQL.

MySQL also enables you to remove a user account with the `DELETE` command. This command removes the entry for the account that has been created in the `user` table of `mysql` database.

To delete the user using the `DELETE` command, enter the following command at the command prompt:

```
DELETE FROM user WHERE Host='localhost' AND User='carol';
```

The command will remove the specified account from the `user` table. The `Host` and the `User` contains the instance name and the account name to delete.

Figure 16.16 displays the output of the command.

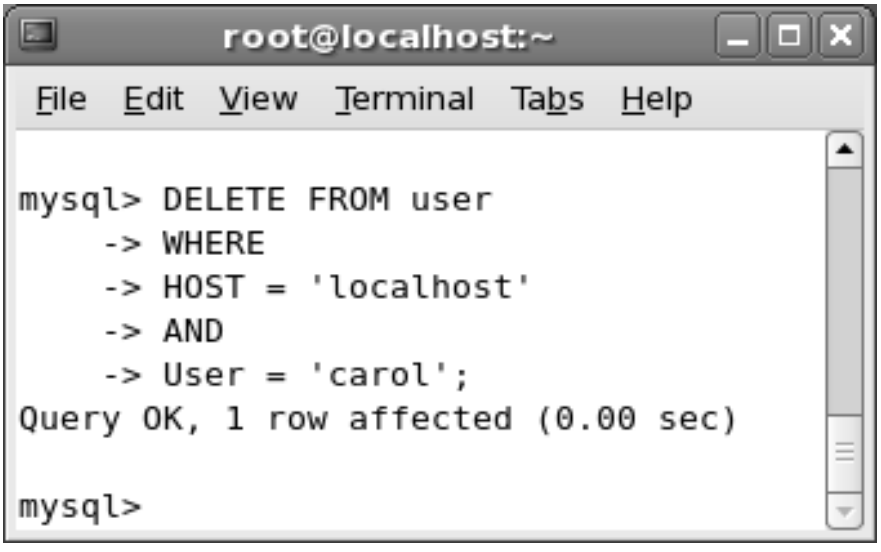


Figure 16.16: Deleting a User

In figure 16.16, MySQL removes the entry from the `user` table where the user is `carol` and the instance is `localhost`.

16.4 Setting Restrictions

You can ensure that single user do not take over the resources by setting a limit on the number of:

- Queries per account per hour
- Updates per account per hour
- Connections per account per hour
- Number of simultaneous connections per account

You can set these restrictions using the `GRANT` command as shown:

```
GRANT … WITH MAX_QUERIES_PER_HOUR R1
             MAX_UPDATES_PER_HOUR R2
             MAX_CONNECTIONS_PER_HOUR R3
             MAX_USER_CONNECTIONS R4;
```

`R1`, `R2`, `R3` and `R4` are integer values that specify the limits. For example, once the user reaches the limit specified in R3, no further connections will be accepted till the hour elapses. Similarly, once the user reaches the limit of the queries specified in R1, no queries are accepted within the hour. Appropriate error

messages are displayed to indicate that the limit has been reached.

Note: You can specify any of the defined limits.

For example, to grant all privileges to the user for all the databases but with resource restrictions, enter the following command at the command prompt:

```
GRANT ALL PRIVILEGES ON *.* TO 'macky'@'localhost'
WITH MAX_QUERIES_PER_HOUR 5
MAX_UPDATES_PER_HOUR 3
MAX_CONNECTIONS_PER_HOUR 3
MAX_USER_CONNECTIONS 1;
```

Figure 16.17 displays the output of the command.

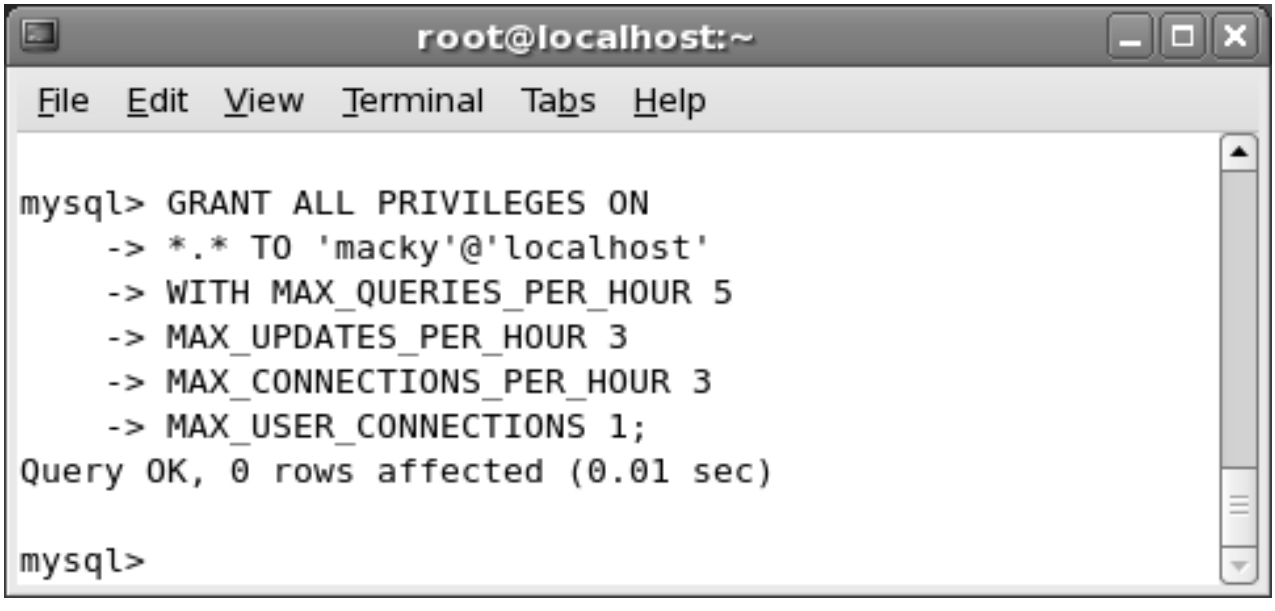


Figure 16.17: Setting Restrictions

Figure 16.17 displays the privileges assigned to the user `'macky'`. The `MAX_QUERIES_PER_HOUR` defines the number of queries that the user can execute in an hour. The `MAX_UPDATES_PER_HOUR` defines the number of updates the user can perform. In, addition, the `MAX_CONNECTIONS_PER_HOUR` defines the number of connections the user account can use to connect simultaneously to the server.

Summary

- MySQL stores details of user accounts and privileges in the `columns _ priv`, `db`, `host`, `tables _ priv`, and `user` tables under the default database named `mysql`.
- MySQL enables you to grant privileges to user accounts at four levels. They are global, database, table, and column level.
- A new user account can be created either by using `GRANT` command or by using `INSERT` command.
- The root user must have `INSERT` and `RELOAD` administrative privileges for creating user.
- The `USAGE` option creates a user without any privileges. An account with just `USAGE` privilege can only be used to establish a connection.
- The `GRANT OPTION` enables you to grant or revoke privileges to another user.
- A super user has all the administrative rights and can access the server from anywhere.
- MySQL provides the `GRANT` and `REVOKE` commands to assign and remove privileges to user accounts. To alter privileges for the user, `GRANT` command is used with selective privilege command.
- You can execute administrative commands, such as `mysql` or `mysqladmin` to control a user's access to the database. For executing administrative commands, the user has to login as the root user.
- You can ensure that no single user takes over the resources by setting a limit on the number of queries per account per hour, updates per account per hour, and so forth.

Check Your Progress

1. For using the default database, the administrator has to log in as _____ user.

   a. super

   b. root

   c. global

   d. virtual

2. The _____ command grants and revokes privileges to all the columns of a given table.

   a. Column level

   b. Global level

   c. Database level

   d. Table level

3. A new user is added in the _____ table of _____ database.

   a. func, test

   b. db, default

   c. user, mysql

   d. host, user

4. When a user connects to the MySQL server, the location of the user gets detected from the _____ and by the _____ name.

   a. Server, path

   b. Hostname, user

Check Your Progress

c. Client machine, user

d. Table level, target

5. Which of the following options help to set the maximum number of simultaneous connections per account?

a. MAX_QUERIES_PER_HOUR

b. MAX_UPDATES_PER_HOUR

c. MAX_CONNECTIONS_PER_HOUR

d. MAX_USER_CONNECTIONS

BE AHEAD OF EVERYONE ELSE

READ ARTICLES

Onlinevarsity

Session

17 Controlling and Managing MySQL Database (Lab)

Objectives

At the end of this session, the student will be able to:

- *Create user accounts in MySQL.*
- *Use the* `GRANT` *command to assign privileges.*
- *Use the* `REVOKE` *command to alter privileges.*

The steps given in the session are detailed, comprehensive and carefully thought through. This has been done so that the learning objectives are met and the understanding of the tool is complete. Please follow the steps carefully.

Part I - For the first 1.5 hours:

Creating user accounts in MySQL

You can create new user accounts by using the `GRANT` command or by manipulating the MySQL grants tables directly. You will use the user table of the `mysql` database.

1. **Activate the mysql database.**

2. **The administrator wants to create a user having all the privileges and should also be able to connect through the localhost only. To do this using the GRANT command, enter the following command at the command prompt:**

```
GRANT ALL PRIVILEGES ON *.* TO 'milton'@'localhost' IDENTIFIED

BY 'password' WITH GRANT OPTION;
```

Figure 17.1 displays the output of the command.

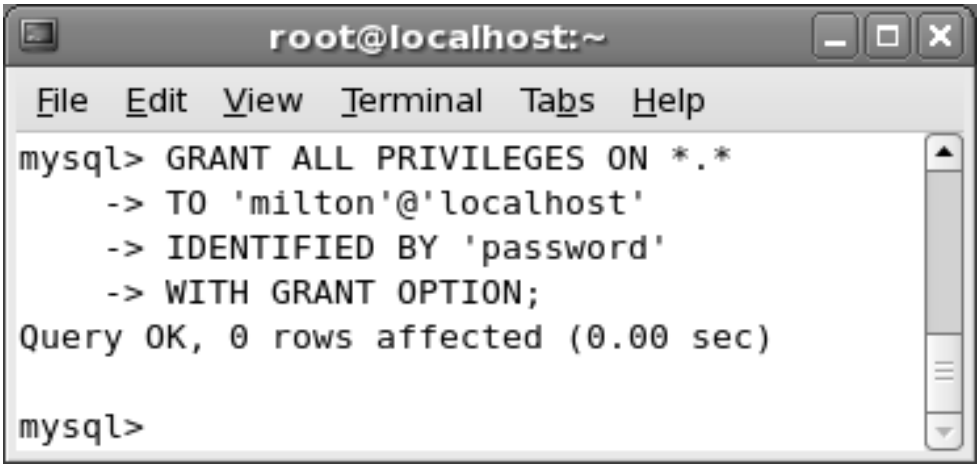


Figure 17.1: Creating a User and Connecting through Localhost

3. **The administrator wants to create a user having all the privileges and should be able to connect through any host. To perform this using the GRANT command, enter the following command at the command prompt:**

```
GRANT ALL PRIVILEGES ON *.* TO 'milton'@'%' IDENTIFIED

BY 'password' WITH GRANT OPTION;
```

Figure 17.2 displays the output of the command.

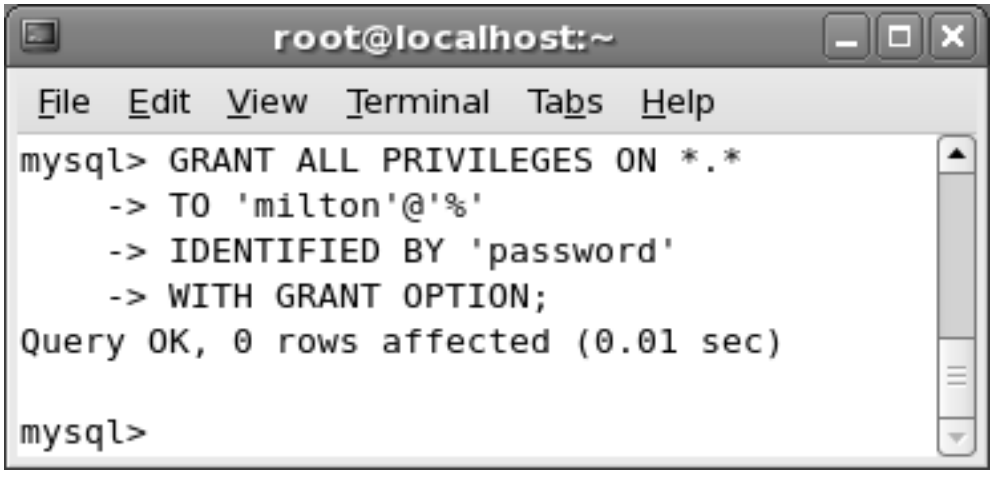


Figure 17.2: Creating a User Connecting Through any Host

4. **The administrator wants to create an account for Julie with all privileges that can connect to MySQL from the localhost instance. To create a user using the INSERT command with**

all the privileges and to connect through the local host, enter the following command at the command prompt:

```
INSERT INTO user VALUES (Host='localhost',User='julie',PASSWORD ('pass
word'),'Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','
,'Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','
Y','Y','Y');
```

Figure 17.3 displays the output of the command.

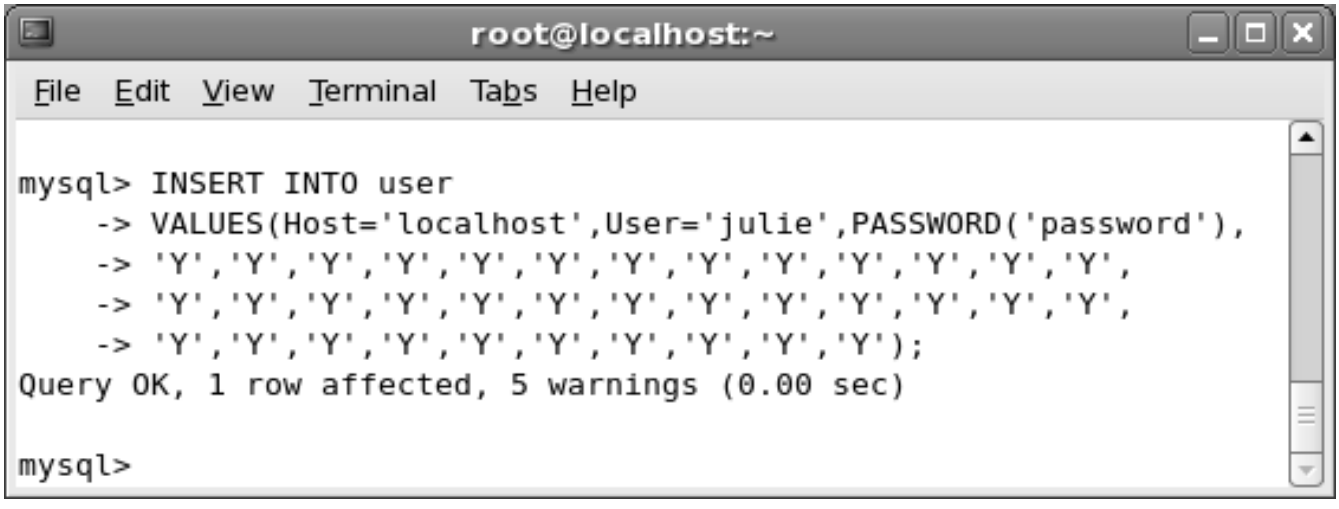


Figure 17.3: Creating a User Connecting to the Localhost Using `INSERT` command

5. **The administrator wants to create a user account for the employee bill who will have unrestricted access to MySQL and can connect from any client machine on the network. The user will have the password set as 'pass'. To create a user with all privileges and to connect through any host using the INSERT command, enter the following command at the command prompt:**

```
INSERT INTO user VALUES (Host='%',User='bill',PASSWORD ('pass'),'Y','Y'
,'Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','
Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y','Y');
```

Figure 17.4 displays the output of the command.

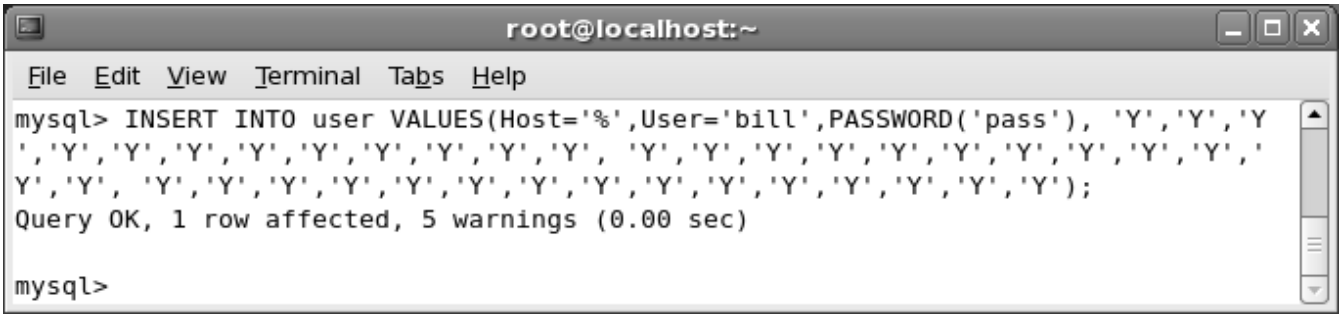


Figure 17.4: Creating a User Connecting to any Host Using `INSERT` command

6. **The administrator wants to create a user with the File_priv and the Show_db_priv privileges only. The user should be able to connect through the localhost using the INSERT command. To do this, enter the following command at the command prompt:**

```
INSERT INTO user SET Host='localhost', User='jonathan',

File_priv='Y', Show_db_priv='Y';
```

Figure 17.5 displays the output of the command.

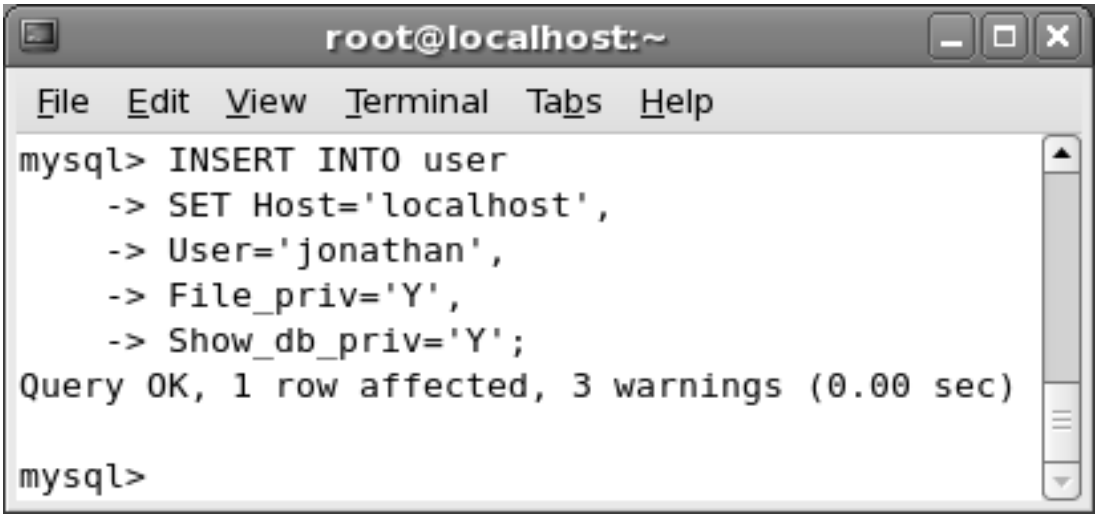


Figure 17.5: Creating a User with the `File_priv` and the `Show_db_priv` Privileges

7. **The administrator wants to create a user account for the employee, katie, using the INSERT command. To create a user account using the INSERT command, and allowing connection through localhost only, enter the following command at the command prompt.**

```
INSERT INTO user (Host, User, Password) VALUES

('localhost','katie','');
```

Figure 17.6 displays the output of the command.

Figure 17.6: Creating a User Using `INSERT` command

The command will create a user in the `localhost` with no password and with no privileges granted to it. After using all the commands, use the `FLUSH PRIVILEGE` command to make the changes effective to the `user` table of `mysql` database.

Using GRANT command to assign privileges to users

A `GRANT` command is used for adding rights and privileges to the user. Perform an addition of privileges to the user, `'katie'` using the `GRANT` command.

1. **Activate the LIBRARY database.**

2. **The administrator wants to assign the SELECT privilege using the GRANT command for BOOK_TABLE of LIBRARY database to the user katie. To assign the SELECT privilege using the GRANT command for BOOK_TABLE of LIBRARY database, enter the following at the command prompt:**

```
GRANT SELECT ON BOOK_TABLE TO katie;
```

Figure 17.7 displays the output of the command.

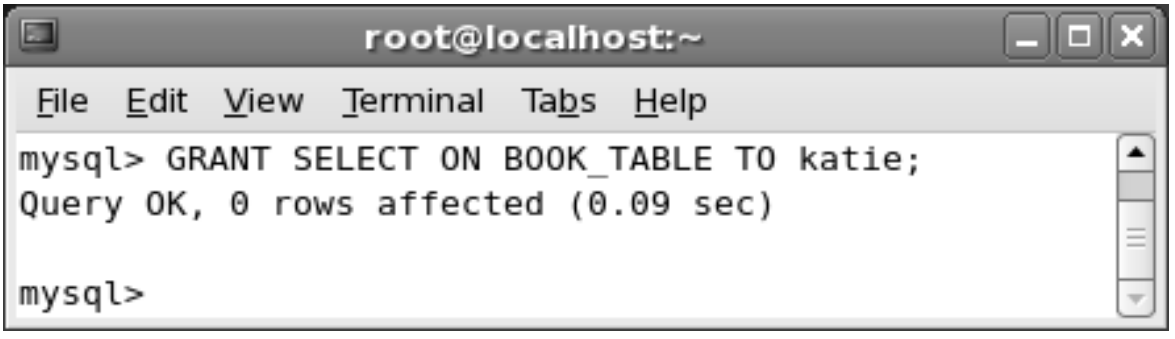


Figure 17.7: Assigning the `SELECT` Privilege

3. **The administrator wants to assign the SELECT and the PROCESS privileges to the user katie using the GRANT command, for all the tables of LIBRARY database. To assign the SELECT and the PROCESS privileges using GRANT command, enter the following command at the command prompt:**

```
GRANT SELECT,PROCESS ON *.* TO 'katie'@'localhost';
```

Figure 17.8 displays the output of the command.

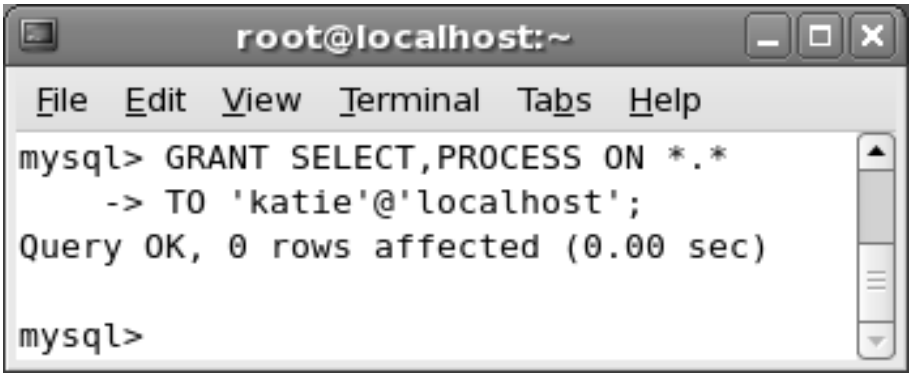


Figure 17.8: Assigning the `SELECT` and the `PROCESS` Privileges to the User

4. **The administrator wants to allow the user katie to access the MySQL database from any machine on the domain using the GRANT command. To provide access to the user from any machine in a given domain by using the GRANT command, enter the following command at the command prompt:**

```
GRANT ALL ON *.* TO 'katie'@'%.mydomain.com' IDENTIFIED BY
'password';
```

Figure 17.9 displays the output of the command.

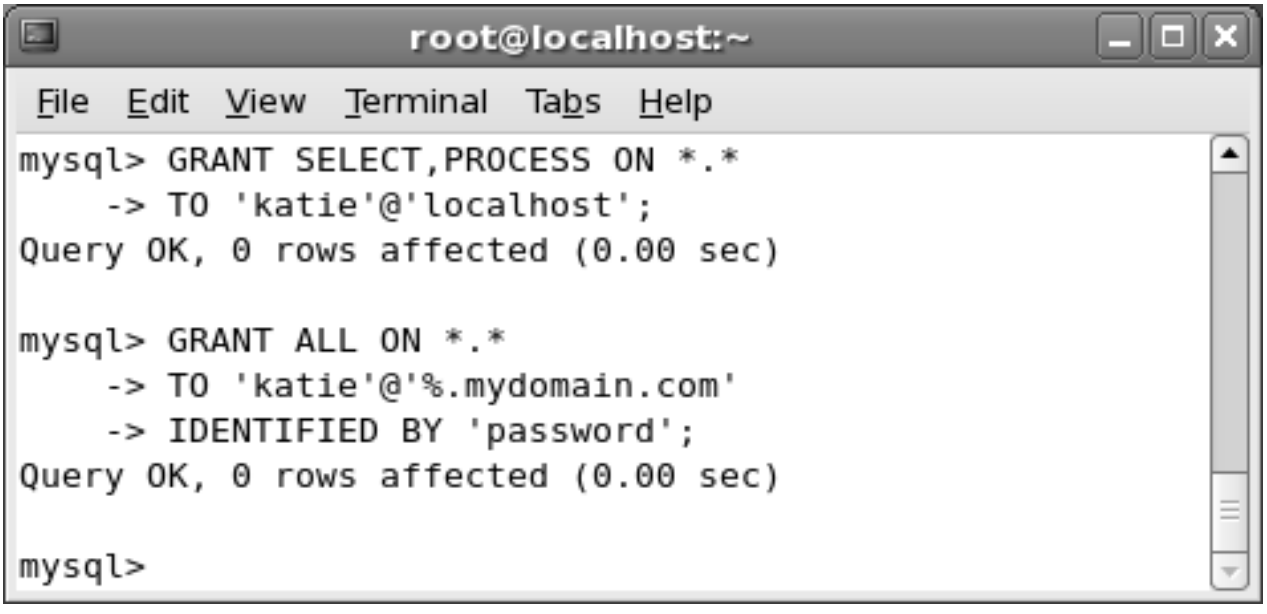


Figure 17.9: Granting Access to the User from any Machine in a Given Domain

In Figure 17.9, the user `katie` is given access to any machine included in the `mydomain.com` host with all rights and privileges.

Using REVOKE command to alter privileges granted to users

The `REVOKE` command is used to remove the privileges from the user. This command drops all databases, tables, and columns level privileges from the user.

1. **Katie has submitted her resignation to the manager. As per the company policy, employees on notice period will not have any privileges on the MySQL database. Therefore the administrator wants to view the privileges assigned to katie. To check the privileges of the user, enter the following command at the command prompt:**

```
SHOW PRIVILEGES;
```

Figure 17.10 displays the output of the command.

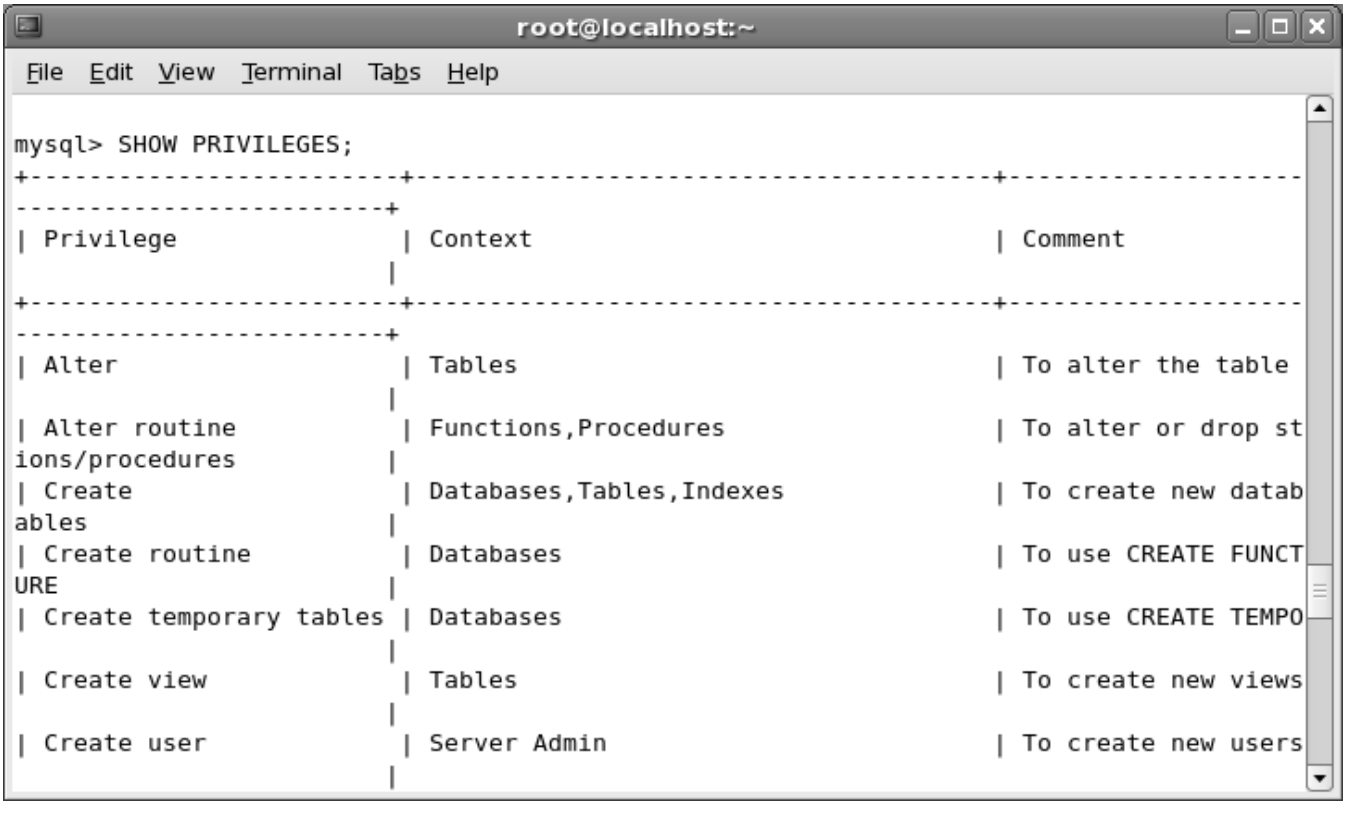


Figure 17.10: Displaying Privileges of a User

2. **As per the company policy, employees on notice period will not have any privileges on the MySQL database. The administrator wants to remove all privileges assigned to katie. To remove all the privileges for the user katie, enter the following command at the command prompt:**

```
REVOKE ALL PRIVILEGES ON *.* FROM 'katie'@'localhost';
```

Figure 17.11 displays the output of the command.

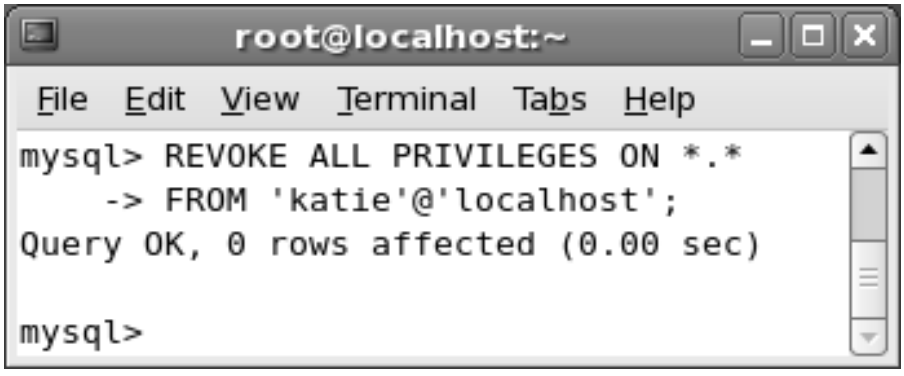


Figure 17.11: Removing privileges from the user

3. **Activate the mysql database.**

4. **Katie is no longer working in the organization. Therefore, the administrator wants to delete the user account, katie. To delete the user by using the DELETE command, enter the following command at the command prompt:**

```
DELETE FROM user WHERE Host='localhost' AND User='katie';
```

Figure 17.12 displays the output of the command.

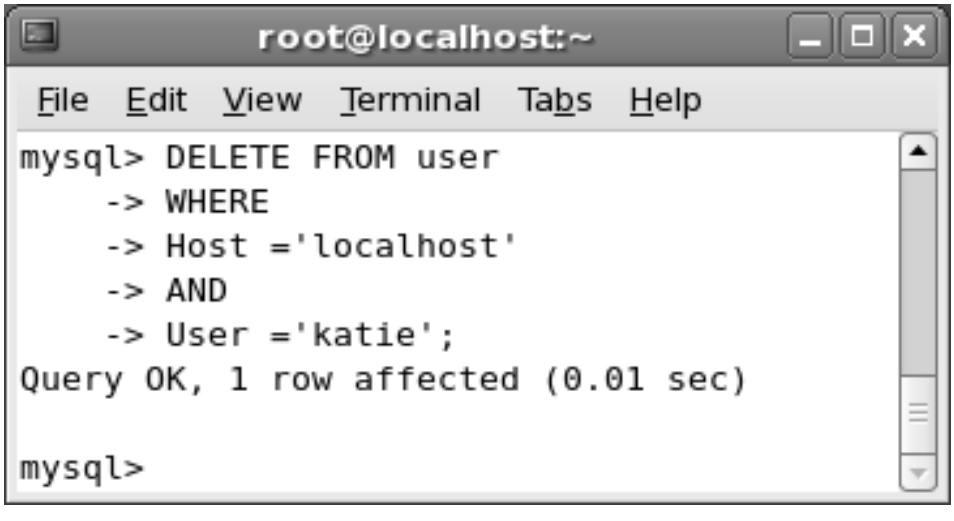


Figure 17.12: Deleting User From Database

Do It Yourself

1. Create a user account having the name as `Betsy` with the help of `GRANT` command and assign it into `localhost` instance.
2. Assign all privileges to the user.
3. Create the same user with the help of `GRANT` command and assign it any host.
4. Add a new user `Helen` in the `user` table of `mysql` database. Assign the user in `server.domain` host.
5. Assign `SELECT`, `PROCESS`, and `DROP` privileges to the user `Helen` using `GRANT` command.
6. Create a new user `Kathy` in `localhost` host and assign only the `Update_priv` and the `Create_priv` privileges to the user.
7. Assign the `INSERT` and `DROP` privileges to `'Kathy'` user for `Stock_Details` table using the `GRANT` command.
8. Check the privileges of user `'Betsy'`.
9. Remove the privileges of user `'Betsy'`.
10. Delete the user `Betsy` from `user` table of `mysql` database.

Session

18 An Overview of MySQL 5.7

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain the process to upgrade MySQL 5.1 to MySQL 5.7.*
- *Describe the new features and enhancements added to MySQL 5.7.*
- *Explain geospatial data and how to use it in MySQL.*
- *Explain various types of MySQL programs.*
- *List and describe spatial data types in MySQL.*
- *Identify JavaScript Object Notation (JSON) data types.*

18.1 Migrating from MySQL 5.1 to MySQL 5.7

In the world of computing, it is common to upgrade a software application. Such an upgrade is released by vendors at regular intervals chiefly to address security vulnerabilities in the prevalent versions. Software updates usually contain fixes for these issues as well as new advanced features. Well, this is also the reason behind updating or upgrading the current version of MySQL.

You perform an upgrade/migration first on a test system to ensure that everything is functioning well. Thereafter, you perform the procedure on the actual production system. Following are the two methods available for upgrading from one instance of MySQL to another:

- **In-place Upgrade**: Includes shutting down the version in use, replacing the version's packages or binaries with new ones, restarting the instance on the prevalent data directory, and executing `mysql _ upgrade`.
- **Logical Upgrade**: Includes exporting the prevalent data from the existing version through mysqldump, installing the new version and loading it with the dump file, and executing `mysql _ upgrade`. Consider this method for skipping several major versions. The only issue with this method is that it does not include users and privileges.

Note: If you are upgrading an instance originally installed by installing several RPM Package Manager (RPM) packages, it is essential to upgrade all the packages. For instance, if you have installed the client and server RPMs, it is wise to upgrade both.

Following are the steps to perform an in-place upgrade:

1. Prior to migration, it is essential to review the changes specified in the installation documentation for determining upgrade issues related to the existing MySQL installation. Changes that are indicated as either Incompatible change or Known issue are incompatibilities with the prior versions. They might need your consideration prior to upgrade.

2. Set up MySQL to perform a gradual shutdown. For instance:

```
shell> bin/mysql -u root -p dbpass123 --execute="set global
innodb_fast_shutdown=0"
```

   `InnoDB` carries out a complete removal and merging of change buffer prior to shutting down. Doing so makes the data files ready, especially if there are differences in file format between releases. The gradual shutdown may take hours if sizeable data is buffered.

3. Shutdown the MySQL server. For instance, you may give the following command:

```
shell> bin/mysqladmin -u root -p dbpass123 shutdown
```

4. Upgrade packages or binaries by replacing the existing ones with new ones.

5. Start MySQL 5.7 server via the prevalent data directory. For instance:

```
shell> bin/mysqld_safe --user=mysql --datadir=/path/to/existing-datadir
```

6. Execute `mysql _ upgrade`. For instance, you may give the following command:

```
shell> bin/mysql_upgrade -u root -p dbpass123
```

   The `mysql_upgrade` program inspects all tables for incompatibilities. It also upgrades the system tables for offering new features or privileges.

Following are the steps for performing a logical upgrade:

1. Prior to migration, it is essential to review the changes specified in the installation documentation for determining upgrade issues related to the existing MySQL installation.

2. Changes that are indicated as either Incompatible change or Known issue are incompatibilities with the prior versions. They might need your consideration prior to upgrade.

3. Export the data from the former MySQL instance or version. The syntax for this is as follows:

```
shell> mysqldump --add-drop-table --routines --events -> --all-databases
--force > data-for-upgrade.sql
```

where,

`--add-drop-table`: Recreates the table with the load to prevent a failure if the current definition differs from that of the load file.

`--routines`: Includes procedures and functions in the dump of user schema, which is required as they are included in mysql.proc dump.

`--events`: Includes events in the dump.

`--all-databases`: Includes all tables and schemas, except for `INFORMATION_SCHEMA and PERFORMANCE_SCHEMA`.

`--force`: Compels the dump to persist in case an error occurs.

4. Shutdown the existing MySQL server. For instance:

```
shell> bin/mysqladmin -u root -p dbpass123 shutdown
```

5. Install the 5.7 version of MySQL.

6. Initialize a new data directory. The syntax for this is as follows:

```
shell> mysqld --initialize --datadir=/path/to/5.7-datadir
```

where,

`--initialize`: Refers to installation with by default security, wherein a random initial root password is generated by marking the existing password as expired and choosing a new one.

`--datadir`: Refers to the path to directory.

7. Keep a copy of temporary password printed on screen or inputted to an error log, for 'root'@'localhost' account, for later use.

8. Start the new MySQL server through the new data directory. Following is the syntax for this:

```
shell> bin/mysqld_safe --user=mysql --datadir=/path/to/5.7-datadir
```

where,

`--user`: Allows running mysqld as the specified user.

`--datadir`: Refers to the path to directory.

9. Reset the root password by connecting to the instance as root user.

```
shell> mysql -u root –p
Enter password: ****  <- provide temporary root password
mysql> ALTER USER USER() IDENTIFIED BY 'new password';
```

10. Load the formerly generated dump file into the new server.

```
shell> bin/mysql -u root -p dbpassch1 --execute="source data-for-upgrade.
sql" –force
```

11. Execute mysql_upgrade. You should not use this program when the server is in `--gtid-mode=ON`, an option indicating that Global Transaction Identifiers (GTIDs) aid in identifying transactions. It is vital to note that the program also does not upgrade the help tables.

12. Set up MySQL to perform a gradual shutdown. For instance:

```
shell> bin/mysql -u root -p dbpassch1 --execute="set global
innodb_fast_shutdown=0"
```

13. Restart MySQL server for a clean startup following a clean shutdown.

```
shell> bin/mysqladmin -u root -p dbpassch1 shutdown
shell> bin/mysqld_safe --user=mysql --datadir=/path/to/5.7-datadir
```

Once you upgrade to MySQL 5.7, you can take the advantage of its enticing new features and enhancements. A few major enhancements include Performance Schema for better insights, security by default, Geographic Information System (GIS) and spatial data support, and JavaScript Object Notation (JSON) implementation. These new features or enhancements offer the benefits of boosted usability, security, and manageability.

18.2 MySQL Performance Schema

In MySQL 5.7, Performance Schema refers to a feature for supervising the execution of the server at a low level. It is referred to as `PERFORMANCE_SCHEMA`. Details related to status and system variables are now available in the tables of Performance Schema. Thus, Performance Schema chiefly focuses on performance data, unlike `INFORMATION_SCHEMA` focusing on metadata. It is executed using the `PERFORMANCE_SCHEMA` database and `PERFORMANCE_SCHEMA` storage engine.

Following are the characteristics of Performance Schema:

- **Server Execution Monitoring**: It offers a way to examine the server's inner execution at runtime.

- **Event Monitoring**: It monitors all events, which the server performs by taking considerable amount of time, such as parsing, a function call, or an SQL statement execution. Collection of all such events gives access to details regarding table I/O, synchronization calls, table locks, and so on.

- **Distinct Events**: Performance Schema events are different from those written to the binary log for stating data alterations in detail as well as from Event Scheduler events (stored programs). They are specific to a given MySQL server instance. Current events are accessible along with history and summary records for finding out the number of times the intended activities were performed and time taken for the same. Event information reveals the activities of particular threads or even of objects such as a file or mutex.

- **Data Collection via Instrumentation Points**: The storage engine gathers event data through 'instrumentation points' present in the source code of the server. Instrumentation points include debugging, exception handling, code tracing, and performance counters. They report events for measuring different parameters such as execution time, function call, data access time, and other things relating to profiling.

- **Local, Temporary Tables but with Event Details**: The tables of the `PERFORMANCE _ SCHEMA` database contain the gathered events. You can query these tables just as other tables. Performance Schema tables are treated as local tables to the server and alterations in them do not get stored in the binary log. These tables are temporary tables or views that do not use any permanent storage on the disk.

- **Dynamically Modifiable Configuration**: The configuration of Performance Schema is subject to dynamic changes by updating tables in its database via SQL queries.

- **Data Collection via Instrumentation**: Data collection occurs by altering the source code of server for inserting instrumentation. No separate threads are linked to the Performance Schema, unlike the Event Scheduler.

- **Support on MySQL-supported Platforms**: Event monitoring is possible on all MySQL-supported platforms. However, the kind of timers may be different for each platform and that instruments applicable to storage engines may not be incorporated in all storage engines. Employing a third-party storage engine is in the hands of the engine caretaker.

Performance Schema aims to offer access to valuable information about execution of server without significantly affecting its performance. Its usage is focused on the following design goals:

- **No Impact on Server's Behavior**: Using the Performance Schema ensures intact server's behavior. For instance, it does not change the schedule of a thread or plans for executing a query.

- **Non-stop Monitoring with Least Overhead**: Server monitoring takes places incessantly as well as unobtrusively with least overhead. Using the schema does not render the server inaccessible.

- **Intact Parser**: There is no change made to the parser; no new statements or keywords are added or used.

- **Ongoing Execution of Server Code**: The server code executes normally in case the Performance Schema fails to run internally.

- **Faster Event Collection**: Priority is to collect events more quickly when there is a choice between initial processing at the time of gathering events or while retrieving events afterward. While collection is ongoing, retrieval happens on demand.

- **Easy Addition of Instrumentation Points**: It is simple and effortless to insert new instrumentation points.

- **Versioned Instrumentation**: If there is a modification in instrumentation implementation, the formerly instrumented code still works well. This is beneficial for the developers who design third-party plugins, as it is now not mandatory to upgrade each plugin for ensuring synchronization.

18.2.1 Performance Schema Tables

The Performance Schema has 52 tables, which include both data and configuration tables. Following are the types of tables available in this schema:

- **Configuration Tables**: Contain logistical details about entities that are monitored.

- **Object Instance Tables**: Contain instances of instrumented objects, such as files, mutexes, and locks.

- **Events Tables**: Contain the details of gathered events that are monitored.

- **History Tables**: Contain details of events occurred in a database.

- **Summary Tables**: Contain details about completed events.
- **Other Tables**: Are remaining tables.

You can query these tables for retrieving the performance details of `PERFORMANCE_SCHEMA` database. For example, you can check performance of an executed query by checking the indexes.

Code Snippet 1 fetches the indexes that are unused.

Code Snippet 1:

```
SELECT object_schema, object_name, index_name
FROM performance_schema.table_io_waits_summary_by_index_usage
WHERE index_name IS NOT NULL
AND count_star = 0
ORDER BY object_schema, object_name;
```

In the query, `table_io_waits_summary_by_index_usage` is a Summary table and gathers all I/O wait events by index. The `index_name` column refers to the Index name, which can be a primary or NULL index. The `count_star` column contains the number indicating the summarized timed as well as non-timed events. For viewing the outcome only for a specific schema, you only need to include `AND object_schema = 'tableName'` after the `WHERE` clause.

In this way, the Performance Schema allows troubleshooting many issues related to server performance by using regular SQL without implementing any third-party solutions.

18.3 sys Schema

Versions 5.7.7 and higher of MySQL include the sys schema. This schema is a collection of objects, which aids the database administrators as well as developers in deciphering the data collected by Performance Schema. The objects are useful for usual tuning and analysis of use cases. These objects include:

- Views summarizing the data of Performance Schema in a more comprehensible way
- Stored procedures carrying out tasks such as configuring Performance Schema and preparing diagnostic reports
- Stored functions offering formatting services and querying configuration of Performance Schema

In case of a new installation of MySQL, the sys schema is installed while initializing data directory by using `mysql_install_db` or `mysqld`. This installation of schema occurs by default. To suppress this default installation, you should use `mysql_install_db` with the `--skip-sys-schema` option. If you are using `mysqld`, you need to drop the schema manually once you initialize the data directory.

In case of an upgrade, the `mysql_upgrade` program installs the schema in case it is not installed. Otherwise, it upgrades the schema to the current version. To hold back this behavior, the installation program offers the `--skip-sys-schema` option. The program gives an error if the schema has no version view, as the nonexistence of this view implies a user-created sys schema. In this case, the administrator is required to rename or remove the sys schema for successful upgrade.

From MySQL 5.7.9 onwards, the objects in the sys schema possess `'mysql.sys'@'localhost'` as the DEFINER, instead of `'root'@'localhost'`.

Additionally, a committed `mysql.sys` account is used to prevent issues occurring upon the renaming or removal of the root account by the administrator.

18.3.1 Prerequisites for sys Schema Usage

Following are prerequisites to be fulfilled for using the sys schema:

- The administrator must enable the Performance Schema, as the sys schema offers an alternative way of accessing the same.
- A user needs to have the following privileges for gaining complete access to the schema:
  - `SELECT` for the schema's all tables and views
  - `EXECUTE` for the schema's all stored procedures and functions
  - `INSERT` and UPDATE for `sys _ config` table for making changes
  - Extra privileges for a few functions and stored procedures, as mentioned in their descriptions, such as `ps_setup_save()`
- A user should also have the following privileges for objects under the schema objects:
  - `SELECT` and `UPDATE` for Performance Schema tables through objects of sys schema
  - `PROCESS` for `INFORMATION _ SCHEMA INNODB _ BUFFER _ PAGE` table
- The administrator must enable the following consumers and instruments of Performance Schema, which also should be timed for tapping in the benefits of schema capabilities:
  - **Instruments**: `Wait`, `statement`, and `stage`
  - **Consumers**: `xxx _ current` and `xxx _ history _ long` for all events

18.3.2 Using the sys Schema

You can set the sys schema as the default one. Doing so eliminates the need to qualify the references to its objects with the schema name.

Figure 18.1 sets the sys schema as the default one.

mysql

```
mysql> USE sys;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM version;
+-------------+---------------+
| sys_version | mysql_version |
+-------------+---------------+
| 1.5.0       | 5.7.11        |
+-------------+---------------+
1 row in set (0.00 sec)
```

Figure 18.1: Setting the sys Schema as the Default One

If you wish to access the schema when it is not set as default (explicit access), it is essential to qualify the references.

Figure 18.2 qualifies the object reference by using the schema name.

mysql

```
mysql> SELECT * FROM sys.version;
+-------------+---------------+
| sys_version | mysql_version |
+-------------+---------------+
| 1.5.0       | 5.7.11        |
+-------------+---------------+
1 row in set (0.00 sec)
```

Figure 18.2: Qualifying the sys Object Reference

The sys schema has several views summarizing the tables of Performance Schema in different ways. A majority of these views are in pairs. A member of a pair possesses the same name as that of the other member, but with the x$ prefix.

Figure 18.3 shows the `host_summary_by_file_io` view with a summary of file I/O by host. It also shows latencies in the form of more legible time values converted from picoseconds.

```
mysql
mysql> SELECT * FROM host_summary_by_file_io;
+------------+------+------------+
| host       | ios  | io_latency |
+------------+------+------------+
| background | 2691 | 269.33 ms  |
| localhost  | 1046 | 170.76 ms  |
+------------+------+------------+
2 rows in set (0.01 sec)
```

Figure 18.3: Summary of File I/O without Prefix in View Name

Figure 18.4 shows the same data but with latencies in unformatted, non-legible picoseconds. This happens when the member is used with the prefix.

```
mysql
mysql> SELECT * FROM x$host_summary_by_file_io;
+------------+------+--------------+
| host       | ios  | io_latency   |
+------------+------+--------------+
| background | 2697 | 269365365278 |
| localhost  | 1046 | 170762214133 |
+------------+------+--------------+
2 rows in set (0.00 sec)
```

Figure 18.4: Summary of File I/O with Prefix in View Name

It is easy to conclude that the view without the prefix offers more user friendly and readable output. The one with the prefix is, however, useful for other tools performing their own data processing procedure.

18.4 Security Enhancements

Following security improvements are added to MySQL 5.7:

- It is now mandatory for the account records in the `mysql.user` table to hold a `plugin` column value or else the account having an empty value in that column is disabled. The database administrators need to convert accounts using the `mysql_old_password plugin` for authentication so that they can use `mysql_native_password`, as the former is not supported.

- Administrators can now set up a policy for automatic expiry of a password. This makes it mandatory for any user to change the account password if the stated period for it to last is over.
- Administrators can now lock and unlock accounts to gain a stricter control over who can or cannot log in.
- The OpenSSL compiled server now easily supports secure connections. It does so by automatically creating the missing Secure Sockets Layer (SSL) and Rivest Shamir Adleman (RSA) certificate as well as the key at startup.
    - Whether compiled with yaSSL or OpenSSL, MySQL server enables SSL at startup when it finds SSL files in the directory. The server does so when it is not explicitly configured for SSL.
    - The server also comes with the `mysql _ ssl _ rsa _ setup` utility for creating SSL and RSA certificate and key files manually.
- MySQL upgrades or through `mysqld --initialize` are, by default, secure. Following modifications are the default features of such deployments:
    - Only one root account is created during installation, which is `'root'@'localhost'`. Similarly, a random password is generated for it and is indicates it as expired. For specifying a new password, the administrator needs to log in as root user by using the random password. This random password is written in the error log.
    - No test database or anonymous-user accounts are created during installation.

18.5 MySQL Programs

There are several command-line programs that MySQL provides to its users for installation. Most MySQL distributions have all of them, except for a few platform-specific ones. This means that you might need to install another package for your particular MySQL distribution.

Each program has several different options. A majority of programs come with the `--help` option for obtaining a description of various options. For instance, you can enter `mysql –help` to see the options it offers. You can override the default value of an option in an option file or on the command line.

Table 18.1 lists the MySQL server and its startup programs.

Server and its Startup Program	Usage	Description
`mysqld`	`shell> mysqld --verbose –help`	The key program or SQL daemon that is responsible for performing most MySQL installation tasks and is called MySQL Server. It gives access to client programs and has many scripts associated with it.

Server and its Startup Program	Usage	Description
mysqld_safe	shell> cd mysql_installation_directory shell> bin/mysqld_safe &	A server startup script, which starts mysqld. It is recommended for starting the server on Unix.
mysqld_server	shell> mysql.server start shell> mysql.server stop	A startup script used on systems, such as Solaris, which run V-style run directories having scripts for starting system services for specific run levels. It starts and stops the server. It can also start mysqld_safe.
mysqld_multi	shell> mysqld_multi start 13 shell> mysqld_multi stop 7,11-13	A startup script useful for starting or halting multiple servers installed on a system. It accepts a GNR value as an option group number or a set of group numbers. This value is the number at the group name's end in the option file.

Table 18.1: Server Startup Programs

Table 18.2 lists the programs that carry on setup operations at the time of upgrade or installation.

Installation- or Upgrade-related Program	Usage	Description
comp_err	shell> comp_err --help	Compiles errmsg.sys that mysqld uses to identify the error message to be shown as per the error code. It creates this file from the location of distribution's source. The program also creates related header files and runs on its own.
mysql_install_db	shell> bin/mysql_install_db --datadir=path/of/directory	Initializes the data directory, produces the mysql database and system tables, configures the InnoDB system tablespace, installs sys schema, and generates an administrative account. The program is executed only once, during installation.
mysql_plugin	shell> mysql_plugin plugin1 ENABLE	Configures server plugins and allows enabling or disabling plugins.

Installation- or Upgrade-related Program	Usage	Description
mysql_secure_installation	shell> mysql_secure_installation	Enables boosting the security of installation by allowing you to set the root account password and remove root accounts reachable from outside the local host, anonymous-user accounts, and test database.
mysql_ssl_rsa_setup	shell> mysql_ssl_rsa_setup	Produces SSL certificate and keys along with RSA key pairs, if they are missing.
mysql_tzinfo_to_sql	shell> mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -u root mysql	Loads the tables of time zone in the mysql database by referring to the contents of the host's zoneinfo database. This database describes time zones and is usually on hosts such as OS X, Linux, and Solaris.
mysql_upgrade	shell> mysql_upgrade --protocol=tcp -P 3309	Examines tables for incompatibility and fix them if required. The program also updates the grant tables by incorporating the changes made in new MySQL versions.

Table 18.2: Installation- or Upgrade-related Programs

Table 18.3 lists the client programs.

Client Program	Usage	Description
mysql	shell> mysql db_name OR shell> mysql --user=admin --password=admin122 db_name	Is a SQL command-line tool. When you utilize it interactively, the outcome of a query is in the form of an American Standard Code for Information Interchange (ASCII) table. If used non-interactively such as a filter, the output is separated by tab. In case of insufficient memory, the --quick option is useful. It helps in retrieving results one row at a time and buffering it, instead of full result set.

Client Program	Usage	Description
mysqladmin	shell> mysqladmin password "passdbtb12"	Carries out different administrative tasks, such as creating or deleting databases, setting new passwords, flushing tables to disk, and reopening log files. You can use the program for fetching status, version, and process details from the server.
mysqlcheck	shell> mysqlcheck db1 book	Is a table-maintenance client for examining, fixing, and optimizing tables.
mysqldump	shell> mysqldump --all-databases > all_databases.sql	Dumps single or multiple databases into an XML, SQL, or a text file so that you can have a backup or transfer to any other SQL server.
mysqlimport	shell> mysqlimport test test.txt	Imports text files into their corresponding tables through LOAD DATA INFILE statement that scans rows from a text file and writes at a high speed into a table. The program ignores extension from the file name and finds out the table name into which the file's data is imported. For instances, the content of files test.txt and test.text would be stored into the test table.
mysqlpump	shell> mysqlpump db1 book authors > academicdump.sql shell> mysqlpump --include-databases=db1,db2 --exclude-tables=db1.book,db2.students > backupdump.sql	Dumps a single or multiple databases into an SQL file.
mysqlshow	mysqlshow -u root –schoolhigh1 HighSchool	Displays details about databases, indexes, columns, and tables.

Client Program	Usage	Description
mysqlslap	mysqlslap --delimiter=";" --create="CREATE TABLE CustAge (age int);INSERT INTO CustAge VALUES (25)" --query="SELECT * FROM CustAge" --concurrency=60 --iterations=100	Imitates client load and state timings for each stage. There are three stages it considers namely, creating objects such as schema and table, running the load, and disconnecting with the server. The utility functions as if several clients are connecting to the server. For example, you can create a table and query it as if 60 clients (concurrency) are querying, and run this test 100 times (iterations).

Table 18.3: Client Programs

Table 18.4 lists the administrative and utility programs.

Administrative and Utility Program	Usage	Description
innochecksum	shell> innochecksum ./data/*.ibd	Is an offline tool that outputs checksums for InnoDB files, including even those that are over two GB.
myisam_ftdump	shell> myisam_ftdump testtable 1	Reveals details about full-text indexes that are in MyISAM tables by referring to index numbers.
myisamchk	shell> myisamchk *.MYI	Expresses, tests, fixes, and optimizes MyISAM tables containing .MYD and .MYI files.
myisamlog	shell> myisamlog myisam.log	Displays the matter existing in a MyISAM log file.
myisampack	shell> myisampack mytest.MYI	Squeezes MyISAM tables for having read-only tables that are smaller.
mysql_config_editor	shell> mysql_config_editor set --login-path=client --host=localhost --user=localuser –password Enter password: pass123	Saves authentication credentials in .mylogin.cnf, which is an encrypted file.
mysqlbinlog	shell> mysqlbinlog binarylog.00005	Reads from a binary log for recovering from a crash.
mysqldumpslow	shell> mysqldumpslow	Reads and sums up the matter existing in a slow query log.

Table 18.4: Administrative and Utility Programs

Table 18.5 lists the program development utilities.

Program Development Utility	Usage	Description
mysql_config	shell> mysql_config	Is a script that generates the option values required while compiling client programs.
my_print_defaults	shell> my_print_defaults mysqlcheck client --user=host1 --password=hostsecret --host=localhost	Displays options that are in the option files' groups. For instance, mysqlcheck reads the two option groups namely, [client] and [mysqlcheck].
resolve_stack_dump	shell> resolve_stack_dump -s /mysqld.sym -n mysqld.stack	Resolve a numeric dump file to symbols. The file must have a numeric stack track from nm --numeric-sort mysqld.

Table 18.5: Program Development Utilities

Table 18.6 lists the miscellaneous utilities.

Miscellaneous Utility	Usage	Description
lz4_decompress	shell> mysqlpump --compress-output=LZ4 > sqldump.lz4 shell> lz4_decompress sqldump.lz4 sqldump.txt	Decompresses the output of mysqlpump, which was produced using LZ4 compression.
perror	shell> perror 12 65	Infers the MySQL error codes or system.
replace	shell> replace b c c b -- charfile1 charfile2...	Replaces string in the input text. For example, it can swap 'b' and 'c' in specified files.
resolveip	shell> resolveip bing.com	Transforms an IP address to its corresponding host name and vice-versa.
zlib_decompress	shell> mysqlpump --compress-output=ZLIB > sqldump.zlib shell> zlib_decompress sqldump.zlib sqldump.txt	Decompresses the output of mysqlpump, which was produced using ZLIB compression.

Table 18.6: Miscellaneous Utilities

18.6 Introduction to Spatial Data

A geographic aspect is something that has a location. This aspect can be a region such as district, an entity such as a lake, or a definable place such as a crossroad. These aspects are also known as geospatial features utilized to map the world. They are also denoted by the most commonly used term, Geometry. It refers to a point or a collection of points representing any location in the world. MySQL provides data types that represent such geometrical or geospatial data.

The Open Geospatial Consortium (OGC) has released the OpenGIS standard specification proposing many conceptual ways for expanding an SQL relational database system to handle spatial data. The specification explains SQL geometry types along with the functions for generating and examining geometry values. Following this specification, MySQL employs spatial types as a subset of an SQL environment that includes a collection of Geometry Types. A geometry-valued column has a geometry data type in SQL. These can be used in the real world to map locations, create open street maps, and so on.

MySQL spatial extensions allow creating, analyzing, and storing data types for spatial values, spatial indexing for boosted access times to relevant columns, and functions for controlling spatial values. The data types as well as functions are for NDB, InnoDB, MyISAM, and ARCHIVE tables. For indexing columns holding spatial values, InnoDB and MyISAM accept even non-SPATIAL indexes.

18.7 Spatial Data Types in MySQL

MySQL provides data types compatible with OpenGIS classes and a few of them contain solo geometry values. Following are the compatible data types:

- `GEOMETRY`, storing values of any type
- `LINESTRING`, `POINT`, and `POLYGON` limiting their values to a specific geometry type

Following are other data types that contain collections of values:

- `GEOMETRYCOLLECTION`, storing a set of any objects
- `MULTILINESTRING`, `MULTIPOINT`, and `MULTIPOLYGON`, limiting the collection members to a specific geometry type

These spatial data types in MySQL are established upon the OpenGIS geometry model.

18.8 OpenGIS Model

In the OpenGIS Geometry model, a geometric object has two generic properties. First, it is linked to the Spatial Reference System, which depicts the coordinate space wherein the object is defined. Second, the object is a part of some geometry class.

In MySQL, `Geometry` is the abstract class; it is a base class. Following are its subclasses:

- `Point` denoting zero-dimensional objects.
- `Curve` denoting one-dimensional objects. It possesses the `LineString` subclass, which has two sub-subclasses namely, `LinearRing` and Line.
- `Surface` denoting two-dimensional (2D) objects and possesses the Polygon subclass.
- `GeometryCollection` denoting zero- to two-dimensional objects. It possesses classes namely, `MultiLineString`, `MultiPoint`, and `MultiPolygon` for modeling objects corresponding to set of `LineStrings`, `Points`, and `Polygons`, respectively. There are also two abstract classes namely, `MultiSurface` and `MultiCurve`, for simplifying the collection interfaces for dealing with `Surfaces` and `Curves`.

Following is the hierarchy of the geometry classes:

```
Geometry (noninstantiable)
```

- `Point`
- `Curve`
  - `LineString`
    - `Line`
    - `LinearRing`
- `Surface`
  - `Polygon`
- `GeometryCollection`
  - `MultiPoint`
  - `MultiCurve`
    - `MultiLineString`
  - `MultiSurface`
    - `MultiPolygon`

Following is the classification of the geometry classes into noninstantiable and instantiable.

- **Noninstantiable**: `Geometry`, `Surface`, `MultiCurve`, `Curve`, and `MultiSurface`, as they contain common methods defined for their corresponding subclasses

- **Instantiable**: `LineString`, `Point`, `GeometryCollection`, `Polygon`, `MultiLineString`, `MultiPoint`, and `MultiPolygon`

You cannot create objects for a noninstantiable class, although it is possible to do so for an instantiable class. While all of these classes contain properties, instantiable ones might also contain assertions, which are rules defining legitimate class instances. The instantiable subclasses are limited to zero- to two-dimensional objects prevalent in 2D coordinate space. They are defined such that the legitimate instances of a geometry class have their boundary.

18.9 Working with Spatial Data

In SQL queries, two standard data formats signify geometry objects. These formats are Well-Known Text (WKT) and Well-Known Binary (WKB). However, MySQL internally does not save geometry values in any of these formats. This means that you are required to use functions for converting data to the desired format.

The WKT format aims to exchange geometry values in ASCII form. The official rules for entering WKT values are specified in the Backus-Naur grammar mentioned in the OpenGIS specification.

Table 18.7 shows WKT format for different geometric objects.

Geometry Object	WKT Representation	Description
`Point`	`POINT(5 10)`	A point is represented with X and Y coordinates separated with space and not comma like in the SQL `Point()` function. You need to use the syntax appropriately as per the spatial operation's context.
	`mysql> SELECT ST_X(Point(5, 10));`	Generates the Point object through the `Point()` function and fetches its X-coordinate.
	`mysql> SELECT ST_X(ST_GeomFromText('POINT(5 10)'));`	Uses the WKT format, which is converted to a Point object by using the spatial function namely, `GeomFromText()`.
`Linestring`	`LINESTRING(0 0, 10 10, 20 25, 50 60)`	Is represented using point coordinate pairs, each followed by a comma.

Geometry Object	WKT Representation	Description
Polygon	POLYGON((0 0,20 0,20 20,0 20,0 0),(7 7,9 7,7 9,7 9,7 7))	Is represented using an exterior ring as well as an interior ring.
MultiPoint	MULTIPOINT(0 0, 30 30, 70 70)	Is represented using point coordinate pairs.
	ST_MPointFromText('MULTIPOINT (4 4, 5 5, 6 6)') ST_MPointFromText('MULTIPOINT ((4 4), (5 5), (6 6))') OR mysql> SET @mulp = 'MULTIPOINT(4 4, 5 5, 5 6)'; mysql> SELECT ST_AsText(ST_GeomFromText(@mulp)); +---------- ----+ | ST_AsText(ST_GeomFromText(@mulp)) +---------- ----+ | MULTIPOINT((1 1),(2 2),(3 3)) | +---------- ----+	Accepts WKT format of MultiPoint values, wherein individual point pair is in parentheses and separated by a comma.
MultiLineString	MULTILINESTRING((20 20, 30 30), (25 25, 40 25))	Accepts two LineString values.
MultiPolygon	MULTIPOLYGON(((0 0,20 0,20 20,0 20,0 0)),((7 7,9 7,7 9,7 9,7 7)))	Accepts two Polygon values.
GeometryCollection	GEOMETRYCOLLECTION(POINT(20 20), POINT(50 50), LINESTRING(25 25, 30 30))	Accepts two Point values along with a LineString value.

Table 18.7: WKT Representations

The WKB format exchanges spatial data in the form of binary streams denoted by values of BLOB, a large binary object holding WKB details. WKB is defined in the specification of OpenGIS. It accepts 1-byte and 4-byte unsigned integers along with 8-byte double-precision numbers. For instance, in WKB, POINT(1 0) is represented as a series of 21 bytes, which is split into the following components:

- **Byte Order**: 01
- **WKB Type**: 01000000

- **X –coordinate with Hexadecimal Digit and Double-precision Value**: 000000000000F03F
- **Y-coordinate with Hexadecimal Digit and Double-precision Value**: 0000000000000000

The byte order is 0 or 1 and denotes little- or big-endian storage. A little-endian byte order is called Network Data Representation (NDR), while a big-endian order is called External Data Representation (XDR). The geometry type is indicated by the WKB type in the form of a value between 1 and 7. Following is the list of what these values represent:

- **1**: Point
- **2**: LineString
- **3**: Polygon
- **4**: MultiPoint
- **5**: MultiLineString
- **6**: MultiPolygon
- **7**: GeometryCollection

The WKB values, if representing complex spatial data, tend to possess more complex data structures.

18.9.1 Creating Columns Holding Spatial Data

`CREATE TABLE` or `ALTER TABLE` statements are used to create columns holding geometric values in MySQL.

Code Snippet 2 creates the `mobile` table with a spatial column named `geodata`.

Code Snippet 2:

```
CREATE TABLE mobile (geodata GEOMETRY);
```

Code Snippet 3 adds or drops a spatial column to or from the `mobile` table.

Code Snippet 3:

```
ALTER TABLE mobile ADD locpt POINT;
ALTER TABLE mobile DROP locpt;
```

Concepts

18.9.2 Adding Values to Spatial Columns

You should store spatial values in internal geometry form. However, you need to convert the values into that internal form from the WKT or WKB format.

Code Snippet 4 converts `WKT` values directly into the internal format in the `INSERT` statement.

Code Snippet 4:

```
INSERT INTO mobile VALUES (ST_GeomFromText('POINT(10 10)'));
```

```
SET @pt = 'POINT(10 10)';
INSERT INTO mobile VALUES (ST_GeomFromText(@pt));
```

Code Snippet 5 performs the conversion before the `INSERT` statement.

Code Snippet 5:

```
SET @pt = ST_GeomFromText('POINT(10 10)');
INSERT INTO mobile VALUES (@pt);
```

Code Snippet 6 shows how to use `ST_GeomFromText()` for generating geometry values.

Code Snippet 6:

```
SET @pt = ST_GeomFromText('POINT(10 10)');
INSERT INTO mobile VALUES (@pt);
```

```
SET @pt = 'LINESTRING(1 1,2 2,3 3)';
INSERT INTO mobile VALUES (ST_LineStringFromText(@pt));
```

```
SET @pt = 'POLYGON((0 0,20 0,20 20,0 20,0 0),(7 7,9 7,7 9,7 7))';
INSERT INTO mobile VALUES (ST_PolygonFromText(@pt));
```

```
SET @pt =
'GEOMETRYCOLLECTION(POINT(20 20), POINT(50 50), LINESTRING(25 25, 30 30))';
INSERT INTO mobile VALUES (ST_GeomCollFromText(@pt));
```

A client program aiming to use WKB format needs to send precisely formed WKB values to the server. Well, many ways exist for fulfilling this prerequisite. For instance, you can insert `POINT(1 0)` in the hexadecimal literal syntax as shown in Code snippet 7.

Code Snippet 7:

```
mysql> INSERT INTO mobile VALUES(ST_
GeomFromWKB(0x0101000000000000000000F03F0000000000000000));
```

Now, an Open Database Connectivity (ODBC) program sends a WKB value by binding it to a placeholder with a BLOB type argument as shown in Code Snippet 8.

Code Snippet 8:

```
INSERT INTO mobile VALUES (ST_GeomFromWKB(?))
```

Even other programming interfaces can implement such a placeholder mechanism.

18.9.3 Retrieving Spatial Data

You can retrieve the stored geometry values in the internal format. You can even convert these spatial values into WKT or WKB.

Code Snippet 9 retrieves data in internal format, which is useful for transferring to a spatial column of other table.

Code Snippet 9:

```
CREATE TABLE mobile1 (geo GEOMETRY);
SELECT geo FROM mobile1;
```

Code Snippet 10 retrieves data in the WKT format by using the ST_AsText() function.

Code Snippet 10:

```
SELECT ST_AsText(geo) FROM mobile1;
```

Code Snippet 11 retrieves data in the WKB format by using the ST_AsBinary() function outputting a BLOB object holding the WKB value.

Code Snippet 11:

```
SELECT ST_AsBinary(geo) FROM mobile1;
```

Concepts

18.10 Hands-On Exercise: Using Spatial Data Type

You can create a separate spatial database in MySQL. In this practice, you will create a spatial database called SPATIAL_DATA. This database shall store details about the places to hang out in the neighborhood such as hotels and restaurants.

1. Open the Linux terminal and run MySQL.

2. To create a database, enter the following command on the command prompt:

```
CREATE DATABASE SPATIAL_DATA;
```

Figure 18.5 shows the output of the command.

```
mysql> CREATE DATABASE SPATIAL_DATA;
Query OK, 1 row affected (0.01 sec)

mysql>
```

Figure 18.5: Creating Spatial Database

3. In the created database, you want to create a table GEO _ SPATIAL _ LOCATIONS for storing details related to different hangouts. This table shall have four columns namely, ID, ORG _ NAME, ORG _ TYPE, and LOCATION. ID will be the primary key, while ORG _ NAME is the hangout name. ORG _ TYPE is the type or category of hangout, which can be a hotel, café, or restaurant, while LOCATION indicates location in the form of Point value. To begin with, set the default database as follows:

```
USE SPATIAL_DATA;
```

Figure 18.6 shows the output of the command.

```
mysql> USE SPATIAL_DATA;
Database changed
mysql>
```

Figure 18.6: Activating the Newly Created Spatial Database

4. Enter the following command on the command prompt to create the table named `GEO _ SPATIAL _ LOCATIONS`:

```
CREATE TABLE GEO_SPATIAL_LOCATIONS(
        ID int(11) NOT NULL,
        ORG_NAME VARCHAR(255) NOT NULL,
        ORG_TYPE VARCHAR(255) NOT NULL,
        LOCATION POINT NOT NULL,
        PRIMARY KEY(ID));
```

Figure 18.7 shows the output of the command.

```
mysql
mysql> CREATE TABLE GEO_SPATIAL_LOCATIONS(
    -> ID int(11) NOT NULL,
    -> ORG_NAME VARCHAR(255) NOT NULL,
    -> ORG_TYPE VARCHAR(255) NOT NULL,
    -> LOCATION POINT NOT NULL ,
    -> PRIMARY KEY(ID));
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Figure 18.7: Creating a Table with a Column Holding Spatial Values

5. Enter the following command on the command prompt to view the table structure:

```
DESC GEO_SPATIAL_LOCATIONS;
```

Figure 18.8 shows the output of the command.

```
mysql> DESC GEO_SPATIAL_LOCATIONS;
+----------+--------------+------+-----+---------+-------+
| Field    | Type         | Null | Key | Default | Extra |
+----------+--------------+------+-----+---------+-------+
| ID       | int(11)      | NO   | PRI | NULL    |       |
| ORG_NAME | varchar(255) | NO   |     | NULL    |       |
| ORG_TYPE | varchar(255) | NO   |     | NULL    |       |
| LOCATION | point        | NO   |     | NULL    |       |
+----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Figure 18.8: Viewing the Structure of a Table Holding Spatial Values

6. Execute the following command on the command prompt for inserting the data into the newly created table:

```
INSERT  INTO  GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE,  LOCATION  )
VALUES (1, 'TEA HOUSE','TEA' , ST_GeomFromText('POINT(1 1)'));
```

Figure 18.9 shows the output of the command.

```
mysql> INSERT INTO GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE, LOCATION )
VALUES (1, 'TEA HOUSE','TEA' , ST_GeomFromText('POINT(1 1)'));
Query OK, 1 row affected (0.00 sec)
```

Figure 18.9: Populating the Table by Adding the First Row with Spatial Value

7. Similarly, enter the following commands to add three more rows to the table:

```
INSERT  INTO  GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE,  LOCATION  )
VALUES (2, 'NOVELTY','HOTEL' , ST_GeomFromText('POINT(2 1)'));
```

```
INSERT  INTO  GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE,  LOCATION  )
VALUES (3, 'SPATIAL','HOTEL' , ST_GeomFromText('POINT(1 2)'));
```

```
INSERT  INTO  GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE,  LOCATION  )
VALUES (4, 'COFFEE ZONE','COFFEE', ST_GeomFromText('POINT(4 5)'));
```

Figure 18.10 shows the output of these commands.

```
mysql> INSERT INTO GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE, LOCATION )
    -> VALUES (2, 'NOVELTY','HOTEL' , ST_GeomFromText('POINT(2 1)'));
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE, LOCATION )
    -> VALUES (3, 'SPATIAL','HOTEL' , ST_GeomFromText('POINT(1 2)'));
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO GEO_SPATIAL_LOCATIONS(ID,ORG_NAME,ORG_TYPE, LOCATION )
    -> VALUES (4, 'COFFEE ZONE','COFFEE', ST_GeomFromText('POINT(4 5)'));
Query OK, 1 row affected (0.00 sec)
```

Figure 18.10: Populating the Table with Three More Rows

Querying a Table Having a Spatial Data Type

You can query the newly created table having spatial data by using the `SELECT` query.

1. Enter the following command on the command prompt:

```
SELECT * FROM GEO_SPATIAL_LOCATIONS;
```

Figure 18.11 shows the output of the command.

```
mysql> SELECT * FROM GEO_SPATIAL_LOCATIONS;
+----+-------------+----------+--------------------------+
| ID | ORG_NAME    | ORG_TYPE | LOCATION                 |
+----+-------------+----------+--------------------------+
|  1 | TEA HOUSE   | TEA      |              ??      ??  |
|  2 | NOVELTY     | HOTEL    |               @      ??  |
|  3 | SPATIAL     | HOTEL    |              ??       @  |
|  4 | COFFEE ZONE | COFFEE   |              @      @ |
+----+-------------+----------+--------------------------+
4 rows in set (0.01 sec)
```

Figure 18.11: Using the SELECT Query

In the output, the values of `LOCATION` column are not readable. This is because MySQL stores the spatial data internally in a different format.

2. Consider that you now need to convert the spatial data into a readable format. To perform this, use the `ST _ asText` function and execute the following query on the command prompt:

```
SELECT ID, ORG_NAME, ORG_TYPE, ST_asText (Location) FROM
GEO_SPATIAL_LOCATIONS;
```

Figure 18.12 shows the output of the command. As you can observe, the spatial data is now represented in a more readable format.

```
mysql> SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION)  FROM GEO_SPATIAL_LOCATIONS;
+----+-------------+----------+---------------------+
| ID | ORG_NAME    | ORG_TYPE | ST_asText(LOCATION) |
+----+-------------+----------+---------------------+
|  1 | TEA HOUSE   | TEA      | POINT(1 1)          |
|  2 | NOVELTY     | HOTEL    | POINT(2 1)          |
|  3 | SPATIAL     | HOTEL    | POINT(1 2)          |
|  4 | COFFEE ZONE | COFFEE   | POINT(4 5)          |
+----+-------------+----------+---------------------+
4 rows in set (0.00 sec)
```

Figure 18.12: Fetching Spatial Values into Readable Format

3. Next, you fetch records of a place that is at a particular point, say, at `POINT(4 5)`. To fetch records of a hangout at this point using the `ST _ Equals` function, execute the following on the command prompt:

```
SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION) FROM GEO_SPATIAL_
LOCATIONS WHERE ST_Equals(ST_GeomFromText('POINT(4 5)'),LOCATION) = 1;
```

Figure 18.13 shows the output of the command.

```
mysql> SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION) FROM GEO_SPATIAL_
LOCATIONS WHERE ST_Equals(ST_GeomFromText('POINT(4 5)'),LOCATION) = 1;
+----+-------------+----------+---------------------+
| ID | ORG_NAME    | ORG_TYPE | ST_asText(LOCATION) |
+----+-------------+----------+---------------------+
|  4 | COFFEE ZONE | COFFEE   | POINT(4 5)          |
+----+-------------+----------+---------------------+
1 row in set (0.00 sec)
```

Figure 18.13: Fetching Details of a Hangout at Specific Point

The `ST_Equals` function takes `two Point` values returns `1` or `0` to indicate whether the two given coordinates are spatially equal or not. 1 indicates equality and 0 indicates inequality.

4. Enter the following command for fetching hangouts that are not at `Point (4 5)`:

```
SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION) FROM GEO_SPATIAL_
LOCATIONS WHERE ST_Equals(ST_GeomFromText('POINT(4 5)'),LOCATION) != 1;
```

Figure 18.14 shows the output of the command.

```
mysql> SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION) FROM GEO_SPATIAL_
LOCATIONS WHERE ST_Equals(ST_GeomFromText('POINT(4 5)'),LOCATION) != 1;
+----+----------+----------+---------------------+
| ID | ORG_NAME | ORG_TYPE | ST_asText(LOCATION) |
+----+----------+----------+---------------------+
|  1 | TEA HOUSE | TEA      | POINT(1 1)          |
|  2 | NOVELTY   | HOTEL    | POINT(2 1)          |
|  3 | SPATIAL   | HOTEL    | POINT(1 2)          |
+----+----------+----------+---------------------+
3 rows in set (0.00 sec)
```

Figure 18.14: Fetching Details of Hangouts Not at a Specific Point

5. Next, you will fetch records of a hangout that is less than five units from a given Point value, say, `POINT(4 5)`. To fetch these records using the `ST _ Distance` function, execute the following on the command prompt:

```
SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION) FROM GEO_SPATIAL_
LOCATIONS WHERE ST_Distance(ST_GeomFromText('POINT(4 5)'),LOCATION) <
5;
```

Figure 18.15 shows the output of the command.

```
mysql> SELECT ID,ORG_NAME,ORG_TYPE, ST_asText(LOCATION)
    -> FROM GEO_SPATIAL_LOCATIONS WHERE
    -> ST_Distance(ST_GeomFromText('POINT(4 5)'),LOCATION) < 5;
+----+-------------+----------+---------------------+
| ID | ORG_NAME    | ORG_TYPE | ST_asText(LOCATION) |
+----+-------------+----------+---------------------+
|  2 | NOVELTY     | HOTEL    | POINT(2 1)          |
|  3 | SPATIAL     | HOTEL    | POINT(1 2)          |
|  4 | COFFEE ZONE | COFFEE   | POINT(4 5)          |
+----+-------------+----------+---------------------+
3 rows in set (0.00 sec)
```

Figure 18.15: Fetching Details of Hangouts at Specific Distance

The `ST_Distance` takes two locations function returns the distance between the two.

18.11 JavaScript Object Notation (JSON) Data Type

As of version 5.7.8, MySQL implements the native JavaScript Object Notation (JSON) data type for efficiently accessing data from a JSON document. Unlike JSON-format strings, the JSON data type allows automatic validation of JSON files inserted in JSON columns and ensures optimized format of storage.

Typically, a JSON document in a JSON column is converted into an internal binary format for quickly reading the document elements. Upon reading a JSON value saved in the binary format, there is no need to parse it from a text format. This binary format facilitates the server to look up nested values or sub-objects by array index or key without navigating through all values preceding or succeeding them in the document.

The size of a stored JSON document is restricted to the value of the system variable, `max_allowed_packet`. This limit is only for storing and not for processing a JSON value in memory. A JSON column neither can be indexed nor can possess a default value. To overcome the index restriction, you can have an index for a generated column fetching a scalar value from the column storing JSON data.

Concepts

18.11.1 Creating and Fetching JSON Values

You can create a JSON array and a JSON object. A JSON array has a list of values within square brackets and followed by commas.

Code Snippet 12 illustrates a JSON array.

Code Snippet 12:

```
[null, 10, "Bob", true, false]
```

A JSON object has a collection of key/value pairs, each followed by a comma and included in curly brackets.

Code Snippet 13 illustrates a JSON object.

Code Snippet 13:

```
{"Name": "Bob", "Age": 30}
```

Both the arrays and objects are capable of holding scalar values, such as numbers, strings, Boolean, and JSON null literal. In a JSON object, a key is always a string. In a JSON array, a temporal scalar value is also allowed.

Code Snippet 14 illustrates an array with temporal scalar values of time, date, and datetime types.

Code Snippet 14:

```
["13:19:30.000000", "2016-04-27", "2016-04-27 13:19:30.000000"]
```

You can even incorporate nesting within the elements and key values.

Code Snippet 15 illustrates nesting in JSON.

Code Snippet 15:

```
[100, {"ID": "P450", "Price": 75.99}, ["pass", "failed"]]
{"ID": "P780", "Point1": [20, 30]}
```

JSON values in MySQL are stored as strings. The server parses any string that should be a JSON value and gives an error in case it is invalid. This string should be a value for a column of JSON data type or a parameter to a function requiring a JSON value.

Code Snippet 16 inserts a value into a JSON column successfully.

Code Snippet 16:

```
mysql> CREATE TABLE Book (bdoc JSON)
```

```
mysql> INSERT INTO Book VALUES('{"Edition": "V1.1", "Author": "James Robert"}');
```

MySQL provides several functions for fetching JSON values. You can also consider casting the values to the JSON type. The syntax for this is as follows:

Syntax:

```
CAST(value AS JSON)
```

Table 18.8 lists a few functions used for fetching JSON values.

Function	Usage	Description

JSON_TYPE()	mysql> SELECT JSON_TYPE('["x", "y", 4]');
	+----------+
	| JSON_TYPE('["x", "y", 4]') |
	+----------+
	| ARRAY |
	+----------+
	mysql> SELECT JSON_TYPE('"Bye"');
	+----------+
	| JSON_TYPE('"Bye"') |
	+----------+
	| STRING |
	+----------+
	mysql> SELECT JSON_TYPE('Hi');
	ERROR 3146 (22032): Invalid data type for JSON data in argument 1 to function json_type; a JSON string or JSON type is required.
		Takes a JSON argument and parses it to give its JSON type. If the argument is invalid, the function generates an error message.
		
		MySQL processes JSON strings by considering the utf8mb4 character set as well as the utf8mb4_bin collation. Those belonging to other character set are transformed into the aforementioned one. Unicode Transformation Format (UTF)-8 and ASCII strings need no conversion, as they are subsets of this supported set.

Function	Usage	Description
JSON_ARRAY()	mysql> SELECT JSON_ARRAY('f', 3, NOW()); +--------------------------+ | JSON_ARRAY('a', 1, NOW()) | +--------------------------+ | ["f", 3, "2016-04-25 10:45:50.000000"] | +--------------------------+	Accepts a list of values and returns them as elements of a JSON array.
JSON_OBJECT()	mysql> SELECT JSON_OBJECT('k1', 2, 'k2', 'y'); +--------------------------+ |JSON_OBJECT('k1', 2, 'k2', 'y')| +--------------------------+ | {"k1": 2, "k2": "y"} | +--------------------------+	Accepts a list of key/value pairs and returns them as pairs in a JSON object.
JSON_MERGE()	mysql> SELECT JSON_MERGE('["c", 2]', '{"k1": "3"}'); +--------------------------+ |JSON_MERGE('["c", 2]', '{"k1": "3"}') | +--------------------------+ | ["c", 2, {"k1": "3"}] | +--------------------------+	Accepts more than one JSON document and gives a combined outcome.

Table 18.8: MySQL Functions for Fetching JSON Values

18.11.2 Searching and Editing JSON Values

You can select a value in a JSON file with the help of a JSON path expression. A path expression is handy while using the functions that fetch a part of or change data in a JSON document. It states where to operate or process within that document.

Code Snippet 17 extracts the object value for the name key from a JSON document.

Code Snippet 17:

```
mysql> SELECT JSON_EXTRACT('{"ID": 590, "name": "Lucifer"}', '$.name');
```

Figure 18.16 shows the output.

```
mysql> SELECT JSON_EXTRACT('{"ID": 590, "name": "Lucifer"}', '$.name');
+-------------------------------------------------------+
| JSON_EXTRACT('{"ID": 590, "name": "Lucifer"}', '$.name') |
+-------------------------------------------------------+
| "Lucifer"                                             |
+-------------------------------------------------------+
1 row in set (0.00 sec)
```

Figure 18.16: Output of JSON_Extract

In Code Snippet 17, the path syntax uses a prefix character, $, which denotes the JSON document to be considered. Following are the optional selectors denoting the document's more specific parts:

- **Period**: Is succeeded with a key name, which denotes an object member. The key name needs to be in double quotation marks in case the name has space, such as "first name".
- **[N]**: Is added to a path and denotes the value at position N in the selected array. In an array, the position count starts with zero.
- **Wildcards**: Are indicated by symbol `[*]` or `[**]`. The `.[*]` wildcard refers to all the values of all object members, while `[*]` refers to values of all array elements. Having `prefix**suffix` refers to all the paths starting with the mentioned prefix name and ending with the mentioned suffix name.

A path, if not existing in a JSON document, returns `NULL` (refers to non-existent data). For example, if $ refers to the following JSON array with three elements:

```
4, {"v": [6, 8], "w": 20}, [20, 30]]
```

Then,

- `$[0]` returns `4`.
- `$[1]` returns `{"v": [6, 8], "w":20}`.
- `$[2]` returns `[20, 30]`.
- `$[3]` returns `NULL` (as no fourth element exists).

At positions 1 and 2, there are non-scalar values. You can use them for more specific expressions for selecting nested values. Following are some examples of such expressions:

- `$[1].v` evaluates to `[6, 8]`.
- `$[1].v[1]` evaluates to `6`.
- `$[1].w` evaluates to `20`.
- `$[2][0]` evaluates to `20`.

Few MySQL functions accept an existing JSON document as the parameter, modify it in its own way, and output the altered document. These functions rely on path expressions to know from where the change has to be made in the document.

Table 18.9 lists a few functions using path expressions. Let us assume that these functions are referring to the following JSON document:

```
mysql> SET @json = '["p", {"q": [true, false]}, [50, 25]]';
```

Function	Usage	Description
JSON_SET()	mysql> SELECT JSON_SET(@json, '$[1].q[0]', 1, '$[2][2]', 2); +------------------------------+ |JSON_SET(@json, '$[1].q[0]', 1, '$[2][2]', 2) | +------------------------------+ |["p", {"q": [1, false]}, [50, 25, 2]] | +------------------------------+	Replaces values for existing paths and adds values for non-existing paths.
JSON_INSERT()	mysql> SELECT JSON_INSERT(@json, '$[1].q[0]', 1, '$[2][2]', 2); +------------------------------+ |JSON_INSERT(@json, '$[1].q[0]', 1, '$[2][2]', 2) | +------------------------------+ |["p", {"q": [true, false]}, [50, 25, 2]] | +------------------------------+	Inserts new values without replacing the current ones.

Function	Usage	Description
JSON_REPLACE()	mysql> SELECT JSON_REPLACE(@json, '$[1].q[0]', 1, '$[2][2]', 2); +------------------------------+ |JSON_REPLACE(@json, '$[1].q[0]', 1, '$[2][2]', 2) | +------------------------------+ |["p", {"q": [1, false]}, [50, 25]] | +------------------------------+	Replaces the current values and overlooks the new ones.
JSON_REMOVE()	mysql> SELECT JSON_REMOVE(@json, '$[2]', '$[1].q[1]','$[1].q[1]'); +------------------------------+ \|JSON_REMOVE(@json, '$[2]', '$[1].q[1]', '$[1].q[1]') | +------------------------------+ | ["p", {"q": [true]}] | +------------------------------+	Accepts single or multiple paths along with the values to be discarded.

Table 18.9: MySQL Functions with Path Expressions

18.12 Hands-On Exercise: Using JSON Data Type

In this practice, you will create a different database in MySQL for storing JSON data. The database shall be named `JSON_DATA` and it shall store details about the movie directors in JSON format.

1. Open the Linux terminal and run MySQL.

2. To create a database, enter the following command on the command prompt:

```
CREATE DATABASE JSON_DATA;
```

3. In the created database, you will now create a table named `JSON _ DATA _ USER` for storing details of different Hollywood directors. This table shall have two columns namely, `ID` and `JSON _ DATA`. `ID` will be the primary key, while `JSON _ DATA` stores the details of directors in JSON format. For creating this table, enter the following command for activating the database:

```
USE JSON_DATA;
```

4. Enter the following command on the command prompt to create the table JSON_DATA_USER:

```
CREATE TABLE JSON_DATA_USER(
     ID int(11) NOT NULL,
     JSON_DATA JSON,
     PRIMARY KEY(ID));
```

Figure 18.17 shows the output of the command.

```
mysql> CREATE TABLE JSON_DATA_USER(
    -> ID int(11) NOT NULL,
    -> JSON_DATA JSON,
    -> PRIMARY KEY(ID));
Query OK, 0 rows affected (0.02 sec)
```

Figure 18.17: Creating a Table for Storing JSON Data

5. Enter the following command on the command prompt for viewing the table structure:

```
DESC JSON_DATA_USER;
```

Figure 18.18 shows the output of the command.

```
mysql> DESC JSON_DATA_USER;
+-----------+---------+------+-----+---------+-------+
| Field     | Type    | Null | Key | Default | Extra |
+-----------+---------+------+-----+---------+-------+
| ID        | int(11) | NO   | PRI | NULL    |       |
| JSON_DATA | json    | YES  |     | NULL    |       |
+-----------+---------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

Figure 18.18: Viewing the Structure of a Table Accepting JSON Data

6. Execute the following command on the command prompt for inserting the data into the newly created table:

```
INSERT INTO JSON_DATA_USER(ID,JSON_DATE) VALUES (1, '{"a" : "b"}');
```

Figure 18.19 shows the output of the command.


```
mysql> INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES (1, '{"a" : "b"}');
Query OK, 1 row affected (0.00 sec)
```

Figure 18.19: Adding the First Row to the Table

7. Similarly, enter the following commands to add four more rows to the table:

```
INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES (2, '{"firstName" :
"James","lastName" : "Cameron","Occupation" : "Direction","Movies" : [
"Terminator" , "Aliens" , "Titanic" , "Avatar" ]}');
```

```
INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES (3, '{"firstName" :
"Jeffrey","lastName" : "Abrams","Occupation" : "Direction","Movies" : [
"StartTrek" , "Star Wars" , "Super 8" , "Cloverfield" ]}');
```

```
INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES (4, '{"firstName" :
"Steven","lastName" : "Spielberg","Occupation" : "Direction","Movies" :
[ "ET" , "Indiana Jones" , "BFG" , "TINTIN" ]}');
```

```
INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES (5, '{"firstName" :
"Christopher","lastName" : "Nolan","Occupation" : "Direction","Movies"
: [ "Memento" , "INCEPTION" ]}');
```

Figure 18.20 shows the output of the command.

```
mysql> INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES
    -> (2, '{"firstName" : "James", "lastName" : "Cameron",
    '> "Occupation" : "Direction",
    '> "Movies" : [ "Terminator" , "Aliens" , "Titanic" , "Avatar" ]}');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES
    -> (3, '{"firstName" : "Jeffrey", "lastName" : "Abrams",
    '> "Occupation" : "Direction",
    '> "Movies" : [ "StartTrek" , "Star Wars" , "Super 8" , "Cloverfield" ]}')
;
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES
    -> (4, '{"firstName" : "Steven", "lastName" : "Spielberg",
    '> "Occupation" : "Direction",
    '> "Movies" : [ "ET" , "Indiana Jones" , "BFG" , "TINTIN" ]}');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO JSON_DATA_USER(ID,JSON_DATA) VALUES
    -> (5, '{"firstName" : "Christopher", "lastName" : "Nolan",
    '> "Occupation" : "Direction",
    '> "Movies" : [ "Memento" , "INCEPTION" ]}');
Query OK, 1 row affected (0.00 sec)
```

Figure 18.20: Populating the Table with Four More Rows

Querying a Table Having a JSON Data Type

You can query the newly created table having JSON data by using the SELECT query.

1. Enter the following command on the command prompt:

```
SELECT * FROM JSON_DATA_USER;
```

Figure 18.21 shows the output of the command.

```
mysql> SELECT * FROM JSON_DATA_USER;
+----+-------------------------------------------------------------------------
-------------------------------------------------------------------------------+
| ID | JSON_DATA
                                                                               |
+----+-------------------------------------------------------------------------
-------------------------------------------------------------------------------+
|  1 | {"a": "b"}
                                                                               |
|  2 | {"Movies": ["Terminator", "Aliens", "Titanic", "Avatar"], "lastNam
e": "Cameron", "firstName": "James", "Occupation": "Direction"}                |
|  3 | {"Movies": ["StartTrek", "Star Wars", "Super 8", "Cloverfield"], "
lastName": "Abrams", "firstName": "Jeffrey", "Occupation": "Direction"}        |
|  4 | {"Movies": ["ET", "Indiana Jones", "BFG", "TINTIN"], "lastName": "
Spielberg", "firstName": "Steven", "Occupation": "Direction"}                  |
|  5 | {"Movies": ["Memento", "INCEPTION"], "lastName": "Nolan", "firstNa
me": "Christopher", "Occupation": "Direction"}                                 |
+----+-------------------------------------------------------------------------
-------------------------------------------------------------------------------+
5 rows in set (0.00 sec)
```

Figure 18.21: Fetching JSON Values Using Select Query

The `SELECT` query fetches all the JSON data. However, this is not a good practice as JSON data can get huge.

2. To fetch only some fields from the JSON database, you can use the `JSON _ EXTRACT` function and execute the following query on the command prompt:

```
SELECT ID , JSON_EXTRACT(JSON_DATA,'$.firstName'),
JSON_EXTRACT(JSON_DATA,'$.Movies') FROM JSON_DATA_USER;
```

Figure 18.22 shows the output of the command.

```
mysql> select ID, JSON_EXTRACT(JSON_DATA,'$.firstName'), JSON_EXTRACT(JSON_DATA,'$.Movies')  from JSON_DATA_USER;
+----+-----------------------------------------+-----------------------------------------------------+
| ID | JSON_EXTRACT(JSON_DATA,'$.firstName') | JSON_EXTRACT(JSON_DATA,'$.Movies')                  |
+----+-----------------------------------------+-----------------------------------------------------+
|  1 | NULL                                    | NULL                                                |
|  2 | "james"                                 | ["terminator", "Aliens", "Titanic", "Avatar"]       |
|  3 | "Jeffrey"                               | ["StartTrek", "Star wars", "Super 8", "cloverfield"] |
|  4 | "Steven"                                | ["ET", "Indiana Jones", "BFG", "TINTIN"]            |
|  5 | "Christopher"                           | ["Memento", "INCEPTION"]                            |
+----+-----------------------------------------+-----------------------------------------------------+
5 rows in set (0.00 sec)
```

Figure 18.22: Fetching Specific Values

The `JSON_EXTRACT` function returns data from the stored JSON document by selecting it from the document's parts as matched by the path arguments. It delivers `NULL` if no path can find a value in the document or if any argument is `NULL`.

3. To fetch records whose `firstName` field has JSON value as James using the `JSON _ CONTAINS` function, execute the following on the command prompt:

```
SELECT * FROM JSON_DATA_USER WHERE JSON_CONTAINS(JSON_DATE,'"James"','$.
firstName') = 1
```

Figure 18.23 shows the output of the command.

```
mysql> SELECT * FROM JSON_DATA_USER WHERE
    -> JSON_CONTAINS(JSON_DATA,'"James"','$.firstName') = 1;
+----+-------------------------------------------------------------------
-------------------------------------------------------------------+
| ID | JSON_DATA
                                                                     |
+----+-------------------------------------------------------------------
-------------------------------------------------------------------+
|  2 | {"Movies": ["Terminator", "Aliens", "Titanic", "Avatar"], "las
tName": "Cameron", "firstName": "James", "Occupation": "Direction"} |
+----+-------------------------------------------------------------------
-------------------------------------------------------------------+
1 row in set (0.00 sec)
```

Figure 18.23: Fetching Details Conditionally

The `JSON_CONTAINS` function takes in the document name, value, and path as parameters and returns `0` or `1` to show whether the mentioned value exists or whether a path argument exists at a specific path in the document. It returns `NULL` if the path cannot look up for the mentioned argument or if any argument is `NULL`. However, the function gives an error if the document is invalid, path has any invalid expression, or path contains a wildcard.

4. To update the `Occupation` field of James Cameron from `Director` to `Producer` in the JSON document, use the `JSON _ SET` function and `UPDATE` statement, execute the following on the command prompt:

```
update JSON_DATA_USER
set      JSON_DATA      =      JSON_SET(JSON_DATA,"$.Occupation"      ,
'["producer","Direction"]')
WHERE JSON_CONTAINS(JSON_DATA,'"james"','$.firstName') = 1;
```

Figure 18.24 shows the output of the command.

```
mysql> update JSON_DATA_USER
    -> set JSON_DATA =  JSON_SET(JSON_DATA,"$.Occupation" , '["producer","Direction"]')
    -> WHERE JSON_CONTAINS(JSON_DATA,'"james"','$.firstName') = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 18.24: Fetching Details of Hangouts at Specific Distance

Summary

- You can upgrade to MySQL 5.7 from MySQL 5.1 either through in-place or logical upgrade.
- Performance Schema is a feature that monitors all server events, stores event details in temporary tables, and holds all system and status variables.
- The sys Schema allows viewing the summary of Performance Schema data.
- MySQL 5.7 features security enhancements such as policy for automatic password expiry, locking and unlocking accounts, creation of SSL and RSA certificates, and by default secure initialization of server instance.
- MySQL offers several types of programs such as server startup, installation- or upgrade-based, client, administrative, development, and miscellaneous programs.
- The spatial data types and their functions are available only for NDB, InnoDB, MyISAM, and ARCHIVE tables.
- Geometry is the base class and has four subclasses namely, Point, Curve, Surface, and GeometryCollection.
- WKT and WKB formats denote geometry objects and are converted into internal format for storing the objects in the database.
- JSON data is stored either as an array or as an object.
- Various MySQL functions are used to obtain JSON values from a JSON document. A few of them use path expressions for fetching or changing data in the document.

Check Your Progress

1. Which of the following programs is used in logical upgrade for starting new server instance via newly created data directory?

   a. `mysqld`

   b. `mysqldadmin`

   c. `mysql_upgrade`

   d. `mysqld_safe`

2. Which of the following tables exists in Performance Schema?

   a. Archive

   b. System

   c. Object Instance

   d. System Config

3. Which of the following is the prerequisite for using sys schema?

   a. Enabled instruments of Performance Schema

   b. EXECUTE for InnoDB

   c. DELETE for INFORMATION_SCHEMA INNODB_BUFFER_PAGE table

   d. Disabled consumers of Performance Schema

Check Your Progress

4. Which of the following programs stops multiple servers at once?

   a. `mysqld_stop`

   b. `mysqld_multiserver`

   c. `mysqld_multiple`

   d. `mysqld_multi`

5. Which of the following codes converts WKT representation of a point into a Point object?

   a. `SELECT ST_X(ST_PointFromText('POINT(5 10)'));`

   b. `SELECT ST_X(ST_GeomFromText('POINT(5 10)'));`

   c. `SELECT ST_X(ST_MPointFromText('POINT(5 10)'));`

   d. `SELECT ST_X(ST_PointFromWKT('POINT(5 10)'));`

6. Identify the output of the following code.

```
SET @json = '["d", {"e": [true, false]}]';

SELECT JSON_REPLACE(@json, '$[1].e[2]', 1);
```

   a. `["d", {"e": [true, false, 1]}]`

   b. `["d", {"e": [1, false]}]`

   c. `["d", {"e": [true, 1]}]`

   d. `["d", {"e": [true, false]}]`

Onlinevarsity

BLOGS

WHERE THE EXPERTS SPEAK THE EXPERIENCE

Session 19

Transaction Management and Globalization

Concepts

Objectives

At the end of this session, the student will be able to:

- *Define and describe stored procedures.*
- *Explain transactions and how to handle them.*
- *Explain the impact of transactions on performance.*
- *Describe MySQL support for different languages and time zones.*

19.1 Introduction

This session introduces you to stored procedures and explains their implementation. The session also defines and describes a transaction in MySQL, lifecycle of a transaction, and explains how it can be controlled and implemented. The session then explores how transactions can play an important role in evaluating performance. Finally, the session explains globalization concepts through MySQL support for different languages and time zones supported by MySQL.

19.2 Stored Procedures

One of the most substantial features supported by MySQL 5.0 and later versions is stored routines. A stored routine is defined as a collection of SQL commands that are stored and executed in the server. A stored routine can be either a procedure or a function. Stored routines are created with the `CREATE PROCEDURE` and `CREATE FUNCTION` statements respectively.

Following are some advantages of using stored procedures:

- **Greater speed**: While performing a database operation, a large amount of data is transferred between the program and the database server. For example, a PHP program runs a `SELECT` command, processes the result, runs an `UPDATE` command depending on the results, and returns `LAST _ INSERT _ ID`, and so on. If all these steps can be performed on the server in a stored procedure, you can suppress a large amount of overhead in transferring data. The database server in stored procedures have the ability to preprocess the code, which is same as a compiler creating a binary file from the source text.

- **Reduced code redundancy, results in better maintenance**: It is very common for multiple applications to access the same database. This is due to same or similar code sections in each application, such as code for validating input values, inserting new data records, and updating tables. If these code sections are devolved to a stored procedure, then, the redundant code can be avoided and all the applications that are impinged on can be easily maintained. For example, when the database schema is modified, just modifying a single stored procedure is adequate and the code in every affected application need not be modified.

- **Increased database security**: In many businesses, such as banking, user programs are restricted from accessing the tables directly due to security issues. Alternatively, stored procedures must be created by the client programs to access the tables. Therefore, there are stored procedures to retrieve data (`SELECT`), add new data, modify existing data, and so on. This may appear complex and definitely the process of creating and maintaining stored procedures in huge numbers is an extensive task.

  It is equally important to note that stored procedures also have some drawbacks. The main one is that stored procedures cannot be imported from one database system to another. The syntax or syntax extension for stored procedures changes from one database to another. Therefore, it is complex to import stored procedures to MySQL from the Microsoft SQL server or Oracle and vice-versa.

19.2.1 Stored Procedures Implementation

In MySQL, stored procedures are saved in the table `mysql.proc`. The name of the database, the attributes, the name and type (Procedure or Function) of the stored routine, the actual code, and other attributes are saved in the columns of table `mysql.proc`.

Stored Procedure Syntax and Language Elements

Stored routines including stored procedures and functions are formed using a group of SQL commands.

Table 19.1 lists the prominent differences between the two types of stored routines.

	Procedures	Functions
Calling	Only with `CALL`	Possible in all SQL commands, For example, `SELECT` and `UPDATE` commands
Return	Returns one or more `SELECT` results	Returns a single value (command `RETURN`); the return values data type must be mentioned in the declaration with `RETURNS`
Parameters	Value and reference parameters possible (`IN`, `OUT`, and `INOUT`)	Only value parameters are allowed, thus, no identification with `IN` and so on
Commands allowed in the code	All SQL commands including `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and `CREATE TABLE`	No commands that access tables
Calling other functions and procedures	May call other procedures and functions	May call only functions but not procedures

Table 19.1: Differences Between Procedures and Functions

There are some commands, such as those used for creating loops and conditional branches, and so on that can be used in stored procedures and functions. Table 19.2 summarizes some of these commands.

Statement	Syntax	Example

IF	IF expression THEN
	statements;
	END IF;
		IF p_country = 'Japan' THEN
		SET v_price=8.15;
		END IF;

IF ELSE	IF expression THEN
	statements;
	ELSE
	else-statements;
	END IF;
		IF p_city = 'Tokyo'
		SET v_price=7.85;
		ELSE
		SET v_price=9.75;
		END IF;

IF ELSEIF ELSE	IF expression THEN
	statements;
	ELSEIF elseif-expression THEN
	elseif-statements;
	...
	ELSE
	else-statements;
	END IF;
		IF p_city = 'Tokyo'
		SET v_price=7.85;
		ELSEIF p_city = 'Osaka'
		THEN
		SET v_price=8.5;
		ELSE
		SET v_price=9.75;
		END IF;

CASE	CASE case_expression WHEN when_expression_1 THEN commands WHEN when_expression_2 THEN commands ... ELSE commands END CASE;	CASE customerCountry WHEN 'Japan' THEN SET shipping = '2-day Shipping'; WHEN 'Canada' THEN SET shipping = '3-day Shipping'; ELSE SET shipping = '5-day Shipping'; END CASE;

WHILE	WHILE expression DO
	statements
	END WHILE
		WHILE y <= 500 DO
		SET str = CONCAT(str,y,',');
		SET y = y + 1;
		END WHILE;

Table 19.2: Statements Used in Procedures and Functions

General Syntax Rules

Following are the syntax rules for stored procedures:

- **Semicolon**: Stored procedures can contain numerous SQL commands and they must be delimited by semicolons. Control structures used for branches and loops must also be concluded with a semicolon.

- **BEGIN–END**: Included between `BEGIN` and `END` are the commands that are not enclosed within keywords such as `THEN` and `END IF`. Therefore, stored procedure code that includes more than one instruction must be placed between `BEGIN` and `END`.

- **New lines**: A new line is syntactically the same as a space character. It is thus acceptable to place an `IF–THEN–ELSE–END-IF` construction on a single line or over multiple lines.

- **Variables**: Local variables (that is, variables that are internal to a stored procedure) and parameters are used with no @ character prefixed to them. In a stored procedure, you can access the ordinary SQL variables, which are generally recognized with the @ character.

- **Case distinction**: Stored procedures are case-insensitive, for example, length is the same whether it is typed `LENGTH` or length. Due to this, it is not possible to describe functions or procedures having the same name except for the case. However, as MySQL can identify a function from a procedure when it is invoked, you can still have a procedure and a function of the same name.

- **Special characters**: International characters such as ä, à, and ñ are avoided in stored procedure names. Such characters are accepted in the `CREATE` command but they can cause problems while managing and using functions and procedures.

- **Comments**: Comments are written using a double hyphen (—) and extend up to the end of the line.

Example:

Consider an example to understand the concept of stored procedures. To know if a given number is odd or even, you will create a user-defined stored procedure, `oddOrEven`.

Step 1: Launch MySQL

To launch the command-line interpreter MySQL in Linux, open a console window and run the command **mysql -u root –p**. Enter the password.

Step 2: Initial setup to create a Stored Procedure

Create and activate a database named `MathTest` and change the delimiter character from ; to $$. These two characters indicate the end of input. As the stored procedure code might use ; (semicolon), it is not a suitable character for termination. Code Snippet 1 shows the code for these steps.

Code Snippet 1:

```
CREATE DATABASE MathTest;
USE MathTest;
delimiter $$
```

Step 3: Defining a procedure oddOrEven

Now, define a stored procedure named `oddOrEven`. When a parameter is passed to the procedure, it must return if the number is even or odd. Code Snippet 2 shows the code for `oddOrEven`.

Code Snippet 2:

```
CREATE PROCEDURE oddOrEven(in n INT, out data varchar(100))
BEGIN
IF ISNULL(n) THEN
SET DATA = 'null';
ELSEIF (n%2 = 0) THEN
SET DATA = 'Even Number';
ELSE
SET DATA = 'odd Number';
END IF;
END$$
```

Now, change the delimiter back to ; by executing the following query:

```
mysql> delimiter ;
```

Step 4: Using Stored Procedures

After defining the procedure, you can execute the series of commands to run it as shown in Code Snippet 3.

Code Snippet 3:

```
SET @input = 11231234;
CALL oddOrEven(@input, @output);
SELECT @input, @output;
The output will be as follows:
+----------+-------------+
| @input | @output |
+----------+-------------+
| 11231234 | Even Number |
+----------+-------------+
1 row in set (0.00 sec)
```

Note: Rather than inserting the code directly into MySQL, you could use a text editor such as Notepad to create the procedure.

It is recommended to check if the procedure already exists. Hence, you can create a text file as follows:

```
DROP PROCEDURE IF EXISTS oddOrEven;
DELIMITER $$
CREATE PROCEDURE oddOrEven ... $$
```

19.2.2 Using Parameters with Stored Procedures

As you have observed, `CREATE PROCEDURE` is the command to create a procedure. Providing a parameter list to this command is optional. However, a pair of parentheses must be added even if there are no parameters. Thus, the syntax is as follows:

```
CREATE PROCEDURE name ([parameterlist])
[options] sqlcode
```

If there are multiple parameters, then they are separated by a comma. The syntax for specifying each parameter is as follows:

```
[IN or OUT or INOUT] <parametername> <datatype
```

The three keywords IN, OUT, and INOUT determine the following:

- **IN**: If the parameter is used only for input
- **OUT**: If the parameter is used only for output
- **INOUT**: If the parameter is used for data transferring in both directions

The default is IN.

All MySQL data types, such as INT, VARCHAR(n), and DOUBLE are allowed. However, contrary to the definition of table columns, you cannot provide additional attributes for types such as NULL and NOT NULL. Currently, there is no provision in MySQL for type checking when passing parameters, however, they may be included in the later versions.

ALTER PROCEDURE

Sometimes, you may need to modify the characteristics of a stored procedure, where characteristics may mean comment, language, security, and so on. You can modify these using ALTER PROCEDURE statements. However, note that you cannot change the parameters or content body of a stored procedure using this statement. You must drop and create the procedure again using DROP PROCEDURE and CREATE PROCEDURE respectively.

Consider an example. Enter the following code at MySQL command prompt.

```
delimiter //
CREATE PROCEDURE countOFEMP(out data int)
COMMENT 'countOFEMP PROC'
BEGIN
SELECT COUNT(*) INTO data FROM emp;
END //
delimiter ;
CALL countOFEMP(@a);
SELECT (@a);
```

The output will be as follows:

```
+----------+
| count(*) |
+----------+
| 2999 |
+----------+
```

Now to modify the characteristics of the procedure, enter the following command:

```
delimiter $$
ALTER PROCEDURE countOFEMP
COMMENT 'countOFEMP Changed Comment';
$$
delimiter ;
CALL countOFEMP(@a);
SELECT (@a);
```

The output will be as follows:

```
+----------+
| count(*) |
+----------+
| 2999 |
+----------+
1 row in set (0.01 sec)
```

19.2.3 Deleting Stored Procedures

After a stored procedure has been defined and used, it may happen that you no longer need it. To delete an existing procedure or function, use the following commands:

- **DROP PROCEDURE [IF EXISTS] name**

  The keyword `IF EXISTS` is optional and allows you to execute the command without returning an error in case the stored procedure does not exist.

  To delete a stored procedure, users need the `ALTER` routine privilege. This is given by default to the creator of the function or procedure, that is the user who executed the `CREATE FUNCTION/PROCEDURE` command.

19.2.4 Security

Only users who have the privilege or access rights to create a routine and alter a routine can create, modify, or delete stored procedures. The `Execute` privilege determines users who are allowed to execute stored procedures. All the three privileges can be set through user administration.

Though a user can execute stored procedures, the database operations that can be performed on the stored procedure is not known. For example, is it possible to delete a record from a table using a particular stored procedure?

MySQL identifies the following two `SECURITY` characteristics:

- **SQL SECURITY INVOKER**: Stored procedures with this option have access privileges that are same as the MySQL user who executed the stored procedure. (The stored procedure can delete records if the creator of the stored procedure is allowed to).

- **SQL SECURITY DEFINER**: The privileges of the MySQL user who defined the stored procedure are also granted to the stored procedure. If `SQL SECURITY INVOKER` is not invoked, then this option is taken by default.

Example:

```
CREATE DEFINER = 'root'@'localhost' PROCEDURE countOFEMP(counter int)

SQL SECURITY DEFINER

BEGIN

 UPDATE emp SET id = counter + 1 where id =counter;

END;
```

Note: Stored procedures that are described at the root level acquire unrestricted privileges on all databases.

19.3 Transactions

A transaction is a sequence of one or more SQL statements that together form a logical unit of work. These statements are generally connected and are used to execute interrelated actions. Every statement that forms a transaction executes certain portions of an operation but all the statements are required to be successful to conclude the operation. Combining the statements to form a transaction informs the DBMS that the complete sequence must be processed such that it passes a test called the Atomicity, Consistency, Isolation, and Durability (ACID) test, which checks if a transaction is atomic, consistent, isolated, and durable.

19.3.1 ACID Properties

According to the ACID test, following are the four characteristics that a transaction must adhere to:

- **Atomic**: A transaction has to be atomic by nature, which means either none of the tasks in a transaction is executed or all of them are. In case of a failure in a transaction after a few statements are processed, the actions of the statements that were processed must be rolled back. A transaction is termed complete only after all the statements are processed and the results of the transaction are applied to the database.

- **Consistent**: A transaction must maintain consistency of the database even if there is a failure. When each transaction ends, the database should be dependable, which means all the rules must be adhered to before the transaction comes to an end. Also, users must not notice any data inconsistency because of modifications done by transactions that have failed.

- **Isolated**: A transaction must work independently without affecting other transactions. No transaction can modify data from other transactions until those transactions have executed completely.

- **Durable**: After a transaction is completed, all the modifications done by it must be saved. There must be consistency in data even if there is any hardware or application failure after execution of the transaction.

19.3.2 Uses of Transaction

A database serves as a data hub that houses information used by real-life sectors such as banking, healthcare, and finance. Consider the scenario in a bank. All the necessary information pertaining to money stored in banks and shares of Dematerialized (Demat) account is saved in the database. Numerous applications continuously work on such data. To maintain the integrity of this data, any modifications to be done to the data must be performed using a transaction. In case of a failure, using a transaction rolls back the data to the state before which the failure occurred.

Let us understand this concept with an example. Consider an Automated Teller Machine (ATM). An ATM is basically used to withdraw and transfer cash. The steps performed to enable withdrawal of cash are as follows:

1. Verifying account details

2. Accepting withdrawal request

3. Checking balance

4. Updating balance

5. Dispensing cash

Consider you make a withdrawal request of US Dollars (USD) 7,000 and available balance is USD 20,000. At Step 4, your balance is updated to USD 7,000 and assume the ATM stops working due to a power outage. Once the power is up you try to withdraw cash and you will be surprised to see your available balance as just USD 13,000 instead of USD 20,000. This seems illogical to any person. This is where a transaction can come to rescue. If SQL statements were executed as a transaction in database, then the balance would have been either USD 13,000 if the cash was dispensed or USD 20,000 if it were not dispensed.

19.3.3 Lifecycle of a Transaction

Following are the six states in which a transaction can exist:

- **Active**: This is the state from which a transaction begins execution.
- **Partially Committed**: At any time, if the transaction is processing correctly, then it is moving towards `COMMIT POINT`. The values generated during processing are all saved in a volatile storage.
- **Failed**: If a transaction failure occurs, the transaction is rolled back and the temporary values are of no use. This means that any modification to the database performed until the point of the transaction failure must be undone. For example, if the unsuccessful transaction has withdrawn USD 500, then the `ROLLBACK` operation must add back USD 500 to that account.
- **Aborted**: Once a transaction has reached the `ROLLBACK` state, the database reaches the Before Image (BFIM) state. In such cases, the transaction is termed aborted.
- **Committed**: When the transaction reaches the `COMMIT POINT` successfully, the temporary values are saved in a stable storage and the transaction is termed committed.
- **Terminated**: After a transaction is committed or aborted, it reaches this state.

The complete process can be illustrated using figure 19.1.

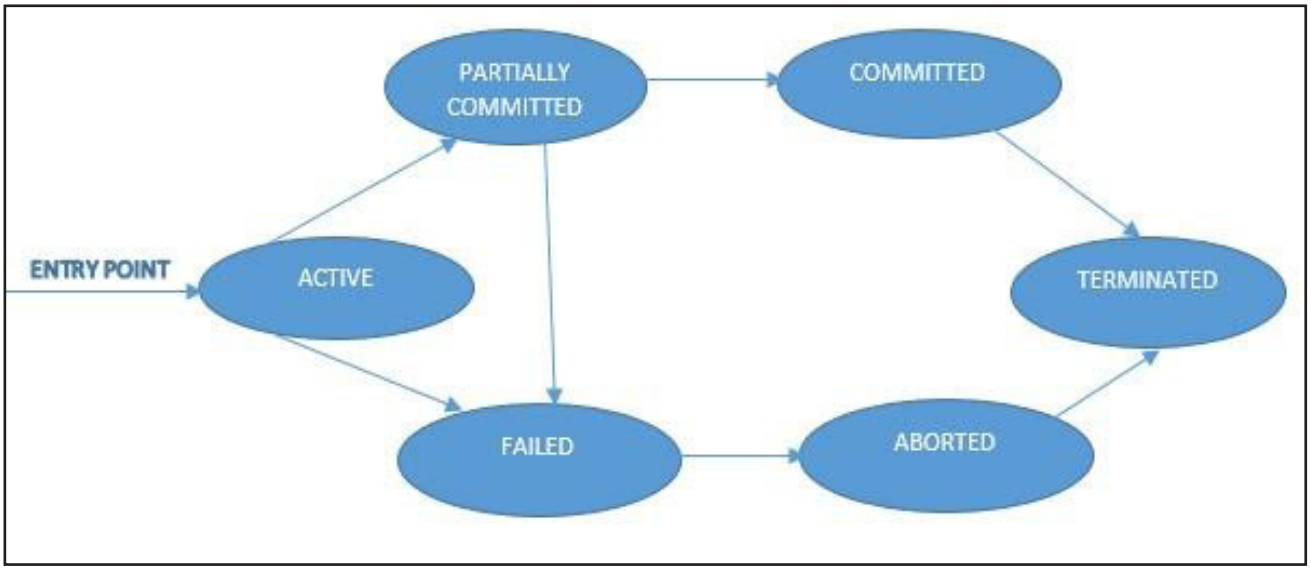


Figure 19.1: Process Diagram

19.3.4 ANSI/ISO SQL Transaction Model

The ANSI/ISO SQL standard describes a standard SQL transaction model and also defines following seven statements that support processing of a transaction:

- **START TRANSACTION**: It sets the attributes of a new transaction. Then it begins the transaction.
- **SET TRANSACTION**: It sets the attributes of the next transaction to be executed. It is independent of any executing transactions.
- **SET CONSTRAINTS**: It sets the constraint mode within a current transaction. This mode allows you to decide if a constraint needs to be applied to the data while it is modified or scheduled to be applied later.
- **SAVEPOINT**: It sets a savepoint in a transaction. A savepoint is a position in a transaction that serves as an intermediate retrieval point. In case of a transaction failure, the transaction can be rolled back to the savepoint rather than starting from the beginning.
- **RELEASE SAVEPOINT**: It allows you to free a savepoint and also release all the resources it may be storing.
- **COMMIT**: It ends a successful transaction and saves all the modifications done to the database.
- **ROLLBACK**: It can be used with or without a savepoint. Without a savepoint, it ends a failed transaction and returns it to the beginning of a transaction. With a savepoint, it takes back the transaction to the savepoint that has been specified, but allows it to proceed from there on.

A MySQL transaction starts with a statement or program and ends when a `COMMIT` or `ROLLBACK` is executed. After a transaction is terminated, it implicitly begins a new one. Successful completion of a transaction is like executing a `COMMIT` statement and an unsuccessful one is like executing a `ROLLBACK` statement.

19.3.5 Controlling Transactions

In MySQL, statements to control transactions are as follows:

START TRANSACTION and SET TRANSACTION Statements

Some basic differences between the two statements are listed in table 19.3.

START TRANSACTION	SET TRANSACTION
Begins a new transaction and allows a user to set a few attributes of the transaction.	Sets attributes to be used by the next transaction. It cannot affect any existing transaction as these attributes cannot be used within the current transaction.
Does not have any `LOCAL` option	Has a `LOCAL` option, which allows to set options for local server execution of a transaction that ranges over multiple servers.

Table 19.3: Differences Between START TRANSACTION and SET TRANSACTION

Following are the three attributes that can be set by these statements:

- **Isolation level**: It describes how independent a transaction will be from other transactions. The default is taken as `SERIALIZABLE`, if no levels are mentioned.

- **Access level**: It describes if the transaction can include statements to make any changes to the database (`READ WRITE`) or just read (`READ ONLY`) it. The default is based on the isolation level, but if no isolation level is mentioned, the default is taken as `READ WRITE`.

- **Diagnostics size**: It sets the diagnostics area that is used for diagnostic information raised by SQL statements. Diagnostic information can be a warning, exception, or any type of message generated while the SQL statements are processed. For example, if this size is set to 5, then five conditions can be saved for that statement.

Note: Currently, this option is not supported by SQL Server, Sybase ASE, Oracle, MySQL, DB2, or many other recent SQL implementations.

Following is an example of a `START TRANSACTION` statement defining an isolation level of `READ UNCOMMITTED`, a diagnostics size of 10, and an access level of `READ ONLY`:

```
START TRANSACTION
ISOLATION LEVEL READ UNCOMMITTED,
READ ONLY,
DIAGNOSTICS SIZE 10;
```

SAVEPOINT, ROLLBACK TO SAVEPOINT, and RELEASE SAVEPOINT

As stated earlier, the `SAVEPOINT` is a position in a transaction that serves as an intermediate retrieval point.

Following is the syntax followed to define, rollback, and release a savepoint respectively. The only attribute that must be specified is a name for the savepoint.

Syntax

```
SAVEPOINT identifier
ROLLBACK [WORK] TO [SAVEPOINT] identifier
RELEASE SAVEPOINT identifier
```

InnoDB has provision for all these three SQL statements and an optional WORK keyword for ROLLBACK.

Let us understand the three statements in detail:

- **SAVEPOINT**: Sets a savepoint with a unique name. If the ongoing transaction already has a savepoint sharing the same name, the old one is removed and the new one is set.

- **ROLLBACK TO SAVEPOINT**: Rolls back a transaction to the specified savepoint without aborting the transaction. Any changes performed to the ongoing transaction after setting the savepoint are cancelled in the rollback. Savepoints defined after the named savepoint are deleted.

  Following error specifies that no savepoint with the stated name exists:

  ```
  ERROR 1305 (42000): SAVEPOINT identifier does not exist
  ```

- **RELEASE SAVEPOINT**: Takes out the specified savepoint from the list of savepoints that are set for the existing transaction. COMMIT or ROLLBACK does not take place because it is an error for a nonexistent savepoint. If they are executed without a savepoint, all savepoints of the current transaction are deleted.

Calling a stored function or activating a trigger causes a new savepoint level to be set. You cannot access the savepoints set on the preceding levels. It is recommended that you do not interfere with savepoints on the new level. When the function or trigger ceases, the set savepoints are released and the previous savepoint level takes charge.

So far, the session explored how savepoints play a positive role in transactions. However, they also have a disadvantage. Savepoints consume a lot of storage space. After a transaction is terminated, all the savepoints are released by default, but practically it is explicit to release all the savepoints that are not required anymore. To achieve this, use the `RELEASE SAVEPOINT` statement.

COMMIT and ROLLBACK Statements

SQL supports the following two SQL transaction-processing statements for explicitly ending transactions:

```
COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
```

where,

- **WORK**: This keyword is added in the standard to make it compatible with some SQL implementations that support it. Otherwise, this does not cause any change.

- **AND [NO] CHAIN**: This keyword specifies whether a new transaction is to be automatically started with the same attributes as the transaction that just terminated. Currently, this is not supported by Oracle, SQL Server, DB2 UDB, or MySQL databases.

- **TO SAVEPOINT**: This keyword calls a `ROLLBACK` to a savepoint that was set previously in a transaction rather than starting from the beginning. This is used only with the `ROLLBACK` statement.

Statements that cannot be rolled back

Not all statements can be rolled back. It includes DDL statements, such as those used for creating or dropping databases, creating, dropping, or modifying tables, or stored routines. It is recommended that you do not use such statements in a transaction. If such a statement has been defined in the beginning of a transaction and then another statement causes a failure, the complete transaction cannot be rolled back using a `ROLLBACK` statement.

Statements that cause an implicit commit

All the statements discussed so far implicitly terminate the transaction in a manner similar to issuing `COMMIT` before executing the statement. Many of these statements also induce an implicit commit after they are executed. The main purpose here is to manage such statements within the transaction itself because they cannot be rolled back. Though there are a few exceptions such as transaction-control and locking statements, if an implicit commit takes place before execution, another certainly cannot occur.

Following are the statements that cause an implicit commit:

- DDL statements that define or alter database objects. This includes `ALTER DATABASE`, `ALTER EVENT`, `ALTER PROCEDURE`, `CREATE DATABASE`, `CREATE INDEX`, `CREATE PROCEDURE`, `CREATE SERVER`, `CREATE TABLE`, `CREATE VIEW`, `DROP DATABASE`, and so on. In addition, `INSTALL PLUGIN` and `UNINSTALL PLUGIN` as of MySQL 5.7.6 are also included. An implicit commit takes place when `ALTER FUNCTION`, `CREATE FUNCTION`, and `DROP FUNCTION` are used in combination with stored functions but not when used with user-defined functions.

Concepts

Note: ALTER FUNCTION cannot be used with user-defined functions but used only with stored functions.

The statements CREATE TABLE and DROP TABLE when executed with the TEMPORARY keyword do not cause a commit. This is not relevant to other actions performed on temporary tables, for example, CREATE INDEX and ALTER TABLE, which cause a commit. Though implicit commit does not occur, the statement cannot be rolled back. If these statements form a transaction, then the transactional atomicity is violated. For instance, issuing CREATE TEMPORARY TABLE and then rolling back the transaction, causes the table to continue to exist.

In InnoDB, CREATE TABLE is executed as one transaction, which implies that ROLLBACK will not undo CREATE TABLE statements executed during that transaction.

When working with non-temporary tables, the CREATE TABLE ... SELECT statement triggers an implicit commit both prior and after the statement is processed. However, commit does not take place when using CREATE TEMPORARY TABLE ... SELECT. This prevents a particular problem during replication. When a table is generated in the master but not logged in the binary log, then it cannot be produced in the slave.

- Statements that implicitly use or alter tables in the MySQL database. ALTER USER, CREATE USER, DROP USER, GRANT, RENAME USER, REVOKE, and SET PASSWORD.

- Transaction-control and locking statements such as BEGIN, LOCK TABLES, SET autocommit = 1 (if the value is not already 1), START TRANSACTION and UNLOCK TABLES. UNLOCK TABLES causes a commit only if LOCK TABLES is used to lock the tables for non-transactional table locks. UNLOCK TABLES following FLUSH TABLES WITH READ LOCK function do not induce a commit as the function does not have table-level locks. It is not possible for transactions to be nested. The transaction being executed must be committed before starting another transaction. Statements that cause an implicit commit must be avoided while a transaction is in action. The BEGIN statement is different from the BEGIN keyword that starts a BEGIN ... END statement. The BEGIN keyword does not induce an implicit commit.

- Data loading statements such as LOAD DATA INFILE triggers an implicit commit only for tables that use the NDB storage engine.

- Administrative statements such as ANALYZE TABLE, CACHE INDEX, CHECK TABLE, FLUSH, LOAD INDEX INTO CACHE, OPTIMIZE TABLE, REPAIR TABLE, and RESET.

- Replication control statements such as START SLAVE, STOP SLAVE, RESET SLAVE, and CHANGE MASTER TO.

19.4 Transaction and Performance

The performance of a database should not decrease with the number of users supported. A database has to provide service to multiple users with many actions such as update, delete, and insert.

Multiuser Processing

When a database is accessed by multiple users at the same time, the transaction processing should be viewed from a different angle. Along with recovering accurately after a failure, now the DBMS must also ensure that the users' actions are independent of each other. Each user must be able to use the database without being interrupted by the actions of other users. The SQL transaction model offers a SQL-based DBMS to isolate users from one another.

Concurrent Transactions

When multiple users are accessing a database and one of the users tries to modify the data, the database is corrupted. To avoid such instances, MySQL utilizes its transaction mechanisms. Along with the all-or-nothing rule followed for the statements in a transaction, MySQL ensures this commitment takes place.

Anytime during a transaction, the database view remains consistent to a user. In mid-transaction, a user:

- Cannot see the uncommitted changes done by others.
- Cannot be affected by the committed changes made by others.

Transactions play a vital role in both recovery and concurrency control in a SQL database.

The three main take-away here as follows:

- Keep the transactions short and simple.
- Use `COMMIT` early and as often as possible.
- Refer the product documentation as DBMS products handle transactions in numerous ways.

19.4.1 Locking

MySQL support sophisticated locking techniques to manage multiuser SQL transactions. Let us understand the concept of locking with an example. When Transaction A performs any operation on the database, then MySQL instantly locks sections of the database that the transaction retrieves or alters. Similarly, when Transaction B executes, MySQL locks sections that it accesses. In case Transaction B attempts to use a section of the database locked by Transaction A, MySQL blocks Transaction B from accessing it. Transaction B has to wait until the data is released.

MySQL frees the locks imposed by Transaction A only after it terminates in a `COMMIT` or `ROLLBACK` operation. Then, MySQL permits Transaction B to proceed by unblocking it. Transaction B can obtain a lock on that section to prevent concurrent transactions.

By using this locking technique, a transaction can obtain exclusive access temporarily to the required sections of a database. This prevents other transactions from using the locked data simultaneously. Though this technique solves concurrent transaction issues, it presents a new concern; locking may introduce a delay because sometimes the required sections of a transaction may be locked by different transactions leading to a long wait.

Shared and Exclusive Locks

To provide enhanced support for concurrent transactions, MySQL utilizes a locking technique with two types of locks namely, shared and exclusive.

- **Shared lock**: This type is used when a transaction only needs to read data from the database. This allows another concurrent transaction to also obtain a shared lock to read the same data.

- **Exclusive lock**: This type is used when a transaction needs to update data in the database. This does not allow any other transaction to obtain either shared or exclusive lock on the data.

Locking can be introduced at various stages of the database such as table-level, page-level, and row-level. However, in simple terms, MySQL can acquire lock on the complete database itself for each transaction.

19.4.2 Deadlocks

Deadlock is a condition in which none of the transactions can proceed because each one holds a lock on the data that the other one needs. The lock will not be released by either of the transactions as they are waiting for the data to become available. A deadlock can arise when the transactions acquire locks on rows in many tables using statements such as `SELECT...FOR UPDATE` or `UPDATE` but in the reverse order. It can also arise when such statements acquire lock on indexes with multiple transactions because of which other transactions continue to wait for the index to be freed.

Deadlock Detection and Rollback

Deadlocks are automatically identified by InnoDB and the respective transactions are rolled back to break the deadlock. InnoDB starts with small transactions to roll back. The size of a transaction is decided by the number of rows that are inserted, deleted, or modified.

InnoDB detects table locks if innodb_table_locks is 1 (the default) and autocommit is 0 and MySQL layer on top of InnoDB is familiar with row-level locks.

InnoDB cannot identify deadlocks if the following are involved:

- A table lock acquired by MySQL LOCK TABLES statement
- A storage engine, which is not InnoDB

These issues can be fixed by assigning the value of the `innodb_lock_wait_timeout` system variable.

When InnoDB rolls back a transaction, all the locks acquired by the transaction are released. However, if only SQL statement is rolled back due to a failure or error, some of the locks acquired by the statement may be retained. This happens because InnoDB saves row-level locks in a format that it later cannot identify which lock was acquired by which statement.

If `SELECT` calls a stored function in a transaction and a statement in that stored function fails, then that particular statement rolls back. If `ROLLBACK` is executed following this, then the complete transaction rolls back.

19.5 Globalization

MySQL includes character set support that enables you to store data using a variety of character sets and perform comparisons according to a variety of collations. You can specify character sets at the server, database, table, and column level.

19.5.1 Character Sets and Collations

A character set comprises a collection of symbols and encodings. A collation comprises a set of rules that can be used to compare characters in a character set. To understand the distinction, let us consider an example.

Assume we have a word consisting of four letters: ‘P’, ‘Q’, ‘p’, and ‘q’. Let us assign a number to each letter: ‘P’ = 0, ‘Q’= 1, ‘p’ = 2, ‘q’ = 3. The letter ‘P’ is a symbol and the value 0 is the encoding for the letter ‘P’. So the combination of all the four letters along with their respective encodings forms a character set.

If we want to compare the two string values, say ‘P’ and ‘Q’, a simple approach is to equate the encodings: 0 for ‘P’ and 1 for ‘Q’. As 0 is lesser than 1, we say ‘P’ is lesser than ‘Q’. The approach that we just followed is to apply a collation to the character set. We have applied one rule (in this scenario) which is ‘equating the encodings’. This simplest form of collation is termed as a binary collation.

Let us consider that we want the lowercase letters and uppercase letters to be considered equal. For this, we need a minimum of two rules:

- The lowercase letters 'p' and 'q' must be treated as equal to uppercase letters 'P' and 'Q'.
- Both the encodings must be equated.

This is termed as a case-insensitive collation and is complicated when compared to a binary collation.

In practice, it is not just 'P' and 'Q' that are included, instead a lot more characters are included in the character sets. It can comprise multiple alphabets with numerous characters, special symbols, and punctuation marks. Collations does not apply rules only for distinguishing cases but also for distinguishing accents and for multi-character mappings.

MySQL offers the following features:

- Save strings with the help of different character sets.
- Compare strings with the help of different collations.
- Combine strings with different collations on the same server, character sets, database, or table.
- Validate the requirements of a character set and collation at all stages.

To effectively utilize these features, you must be familiar with different character sets and collations, how to modify the standards, and how this modification impacts the performance of string operators and functions.

19.5.2 Time Zones Supported by MySQL

Following are different time zone settings used by MySQL:

- **System time zone**: On startup, the server sets the `system _ time _ zone` variable after deciding the host machine's time zone. Once set, this value will not be modified. To set MySQL's system time zone, set the option `--timezone=timezone _ name` to `mysqld _ safe`. It can also be set using the variable `TZenvironment` before starting mysqld. The values accepted for `--timezone` or TZ depends on the system.
- **Server's current time zone**: The variable global `time _ zone` shows the time zone that the server is following. If `time _ zone` is set to 'SYSTEM', it means that the server and the system time zone are the same. Use the option `--default-time-zone=timezone` in the command line to initially set the global time zone value.

- It can also be specified using the following in an option file:

```
default-time-zone='timezone'
```

It can be set at runtime using the following statement, if a user has SUPER privilege:

```
SET GLOBAL time_zone = timezone
```

- **Per-connection time zones**: Every client connecting to the server has its own preferred time zone setting specified by the variable session `time _ zone`. Initially, this variable picks its value from the global `time _ zone` variable. However, the client has the provision to modify the value using the following statement:

```
 SET time_zone = timezone
```

The time zone setting that is being used impacts how the zone-sensitive time values are displayed and stored. It affects the values displayed by NOW() and `CURTIME()` functions and the values that are saved in and obtained from the `TIMESTAMP` columns. To store values in the `TIMESTAMP` columns, they are converted to Coordinated Universal Time (`UTC`) from the current time zone, and for retrieval, its vice-versa.

The current setting has no impact on the values displayed by `UTC_TIMESTAMP()` or the values in `TIME`, `DATE`, or `DATETIME` columns. The columns' values are not stored in UTC format. The time zone changes them only when converting from `TIMESTAMP` values. To obtain the locale-specific arithmetic for these values, they must be converted to UTC, then the arithmetic performed, and converted back.

To view the existing values of the client-specific time zone, use the following statement:

```
SELECT @@global.time_zone
```

To view the existing values of global time zone, use the following statement:

```
SELECT @@session.time_zone;
```

The values for `time_zone` can be specified in multiple formats and they are case-insensitive:

- The value 'SYSTEM' implies that the time zone matches the system time zone.

- The value can be specified as a string representing an offset from UTC, for example, '+11:00' or '-7:00'.

- The value can be specified as names, for example, 'Europe/Helsinki', 'US/Eastern', or 'MET'.

Note: Named time zones can only be used if the tables consisting of time zone information are created and the corresponding data is filled in the MySQL database.

Populating the Time Zone Tables

The MySQL database has numerous tables to maintain data related to time zone information. The time zone tables are created during the installation, but are not populated. It must be done manually using the following instructions:

To populate time zone tables using the `mysql_tzinfo_to_sql` program, on the command line, provide the `zoneinfo` directory path name to `mysql_tzinfo_to_sql` and pass the output into the mysql program. Let us understand this with the following sample code:

```
shell> mysql_tzinfo_to_sql /usr/share/zoneinfo | mysql -u root mysql
```

Here, `mysql_tzinfo_to_sql` scans the files on your system pertaining to time zone and creates SQL statements from them. Next, MySQL executes the statements and populates the time zone tables.

Then, run the following query to check if timezones have been populated from the system after the script has run on the shell:

```
SELECT COUNT (*) FROM mysql.time_zone_name;
```

Figure 19.2 shows the output of the query.

mysql

```
mysql> SELECT COUNT(*) FROM mysql.time_zone_name;
+----------+
| COUNT(*) |
+----------+
|      583 |
+----------+
1 row in set (0.00 sec)
```

Figure 19.2: Checking Populating of Time Zone Table

Staying Updated with Time Zone Changes

When time zone rules changes, applications following the old rules become outdated. To stay updated, it is essential for the system to use the current time zone information. For this to happen, the following factors must be considered:

- The time used by the operating system impacts the MySQL server time if its time zone is set to `SYSTEM`. Ensure that the operating system is on par with the latest time zone information. In many operating systems, the latest update or service pack makes the system ready for the time changes. Refer the operating system vendor's Web site for the update that fixes the time change issues.

- If the `/etc/localtime timezone` file on the system is replaced with an outdated rule at `mysqld` startup, `mysqld` must be restarted to allow it to reflect the updated rules.

- If using named time zones, then time zone tables in MySQL database must be the updated ones. If the system is using a private `zoneinfo` database, MySQL time zone tables must be reloaded every time the `zoneinfo` database is modified.

To determine if named time zones are available, either as time zone setting of the server or by client applications that specify their own time zone, check if time zone tables have any data.

Use the following query to check if the table `time_zone_names` has any rows:

```
SELECT COUNT(*) FROM mysql.time_zone_name;
```

Figure 19.3 shows the output of the query.

mysql

```
mysql> SELECT COUNT(*) FROM mysql.time_zone_name;
+----------+
| COUNT(*) |
+----------+
|        0 |
+----------+
1 row in set (0.00 sec)
```

Figure 19.3: Output Showing the Number of Rows in time_zone_names Table

The query may return the following output:

- **Zero**: It means that the table is empty. In this case, the table need not be updated as nobody can use the named time zones.

- **Anything other than zero**: It means that the table's contents can be utilized for named time zone support. In this case, ensure reloading of tables so that anyone using named time zones has accurate query results.

To ensure if MySQL is updated for a change in Daylight Saving Time (DST) rules, use the following two queries:

Note: The example uses values that reflect the 2016 DST 1-hour change in the United States as on March 14 at 4 A.M.

```
SELECT CONVERT_TZ('2016-03-14 4:00:00','US/Eastern','US/Central'); SELECT
CONVERT_TZ('2016-03-14 5:00:00','US/Eastern','US/Central');
```

The usage of the same value twice implies the time during which the DST happens and the named time zones requires the time zone tables to be used. The expected output is the same (the input time converted to the analogous 'US/Central' time zone) from both the queries.

Prior to updating the time zone tables, an incorrect result would be displayed as shown in figure 19.4:

mysql

```
mysql> SELECT CONVERT_TZ('2007-03-11 2:00:00','US/Eastern','US/Central');
+-----------------------------------------------------------+
| CONVERT_TZ('2007-03-11 2:00:00','US/Eastern','US/Central') |
+-----------------------------------------------------------+
| NULL                                                      |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CONVERT_TZ('2007-03-11 3:00:00','US/Eastern','US/Central');
+-----------------------------------------------------------+
| CONVERT_TZ('2007-03-11 3:00:00','US/Eastern','US/Central') |
+-----------------------------------------------------------+
| NULL                                                      |
+-----------------------------------------------------------+
1 row in set (0.00 sec)
```

Figure 19.4: Output Prior to Updating Time Zone Tables

After updating the tables, the correct information is displayed as shown in figure 19.5:

```
mysql
mysql> SELECT CONVERT_TZ('2007-03-11 2:00:00','US/Eastern','US/Central');
+-----------------------------------------------------------+
| CONVERT_TZ('2007-03-11 2:00:00','US/Eastern','US/Central') |
+-----------------------------------------------------------+
| 2007-03-11 01:00:00                                       |
+-----------------------------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CONVERT_TZ('2007-03-11 3:00:00','US/Eastern','US/Central');
+-----------------------------------------------------------+
| CONVERT_TZ('2007-03-11 3:00:00','US/Eastern','US/Central') |
+-----------------------------------------------------------+
| 2007-03-11 01:00:00                                       |
+-----------------------------------------------------------+
1 row in set (0.00 sec)
```

Figure 19.5: Output after Updating Time Zone Tables

MySQL Server Locale Support

The `lc_time_names` system variable implies the locale that governs the language used for displaying the day, month, and abbreviations. This variable has an impact on the output from the functions: `DATE_FORMAT()`, `DAYNAME()`, and `MONTHNAME()`.

`lc_time_names` has no impact on the `STR_TO_DATE()` or `GET_FORMAT()` function.

This variable has no impact on the `FORMAT()` function but it accepts a third parameter, which is optional. The optional parameter enables a locale, which is used for the decimal point, grouping between separators, and thousands separator of the result number. The acceptable locale values are equivalent to the values for `lc_time_names` system variable.

Language and region sub-tags such as ‘ja_JP’ or ‘pt_BR’ that are part of locale names are registered by IANA (http://www.iana.org/assignments/language-subtag-registry).

‘en_US’ is set as the default irrespective of the system’s locale setting. However, the value can also be specified during the server startup or by setting the `GLOBAL` value if granted with SUPER privilege.

Any client can view the value of `lc_time_names` or set its `SESSION` value to influence the locale for its own connection as shown in figure 19.6.

mysql

```
mysql> SET NAMES 'utf8';
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @@lc_time_names;
+-----------------+
| @@lc_time_names |
+-----------------+
| en_US           |
+-----------------+
1 row in set (0.00 sec)
```

Figure 19.6: Output of lc_time_names

19.6 Hands-On Exercise: Creating Stored Procedures

You will now create a database `INVENTORY` for an online store. This database will have two tables: USERS and `USER_ORDER`. The `USERS` table stores user related information and `USER_ORDER` table stores the data related to users' orders for a particular product. The `USERS` table has four columns: `ID`, `USER_NAME`, `EMAIL`, and `PHONE`, with `ID` being the unique `KEY`. Using this database, we shall demonstrate how to:

- Create and use stored procedures
- Use transactions and understand deadlocks

1. Open the Linux terminal and start MySQL.
2. Create a database INVENTORY and set it as the current database using the following commands at the command prompt:

```
CREATE DATABASE INVENTORY;
USE INVENTORY;
```

3. Create a table USERS by executing the following query:

```
CREATE TABLE USERS(
     ID int(11) NOT NULL,
     USER_NAME VARCHAR(255) NOT NULL,
     EMAIL VARCHAR(255) NOT NULL,
     PHONE INT NOT NULL ,
     PRIMARY KEY(ID));
```

Figure 19.7 shows the output of the command.

```
mysql> CREATE TABLE USERS(
    -> ID int(11) NOT NULL,
    -> USER_NAME VARCHAR(255) NOT NULL,
    -> EMAIL VARCHAR(255) NOT NULL,
    -> PHONE INT NOT NULL ,
    -> PRIMARY KEY(ID));
Query OK, 0 rows affected (0.02 sec)
```

Figure 19.7: Creating Table USERS

4. To view the structure of the table, use the following command:

```
DESC USERS;
```

Figure 19.8 shows the output of the command.

```
mysql> DESC USERS;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   | PRI | NULL    |       |
| USER_NAME | varchar(255) | NO   |     | NULL    |       |
| EMAIL     | varchar(255) | NO   |     | NULL    |       |
| PHONE     | int(11)      | NO   |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Figure 19.8: Structure of the USERS Table

5. Create a table `USER _ ORDER` by executing the following query:

```
CREATE TABLE USER_ORDER(
     ID int(11) NOT NULL,
     USER_ID int(11) NOT NULL,
     PRODUCT_ID int(11) NOT NULL,
     PRIMARY KEY(ID),
 FOREIGN KEY (USER_ID) REFERENCES USERS(ID));
```

Figure 19.9 shows the output of the command.

```
mysql> CREATE TABLE USER_ORDER(
    -> ID int(11) NOT NULL,
    -> USER_ID int(11) NOT NULL,
    -> PRODUCT_ID int(11) NOT NULL,
    -> PRIMARY KEY(ID),
    ->     FOREIGN KEY (USER_ID) REFERENCES USERS(ID));
Query OK, 0 rows affected (0.02 sec)
```

Figure 19.9: Creating Table USER_ORDER

6. To view the structure of the table, use the following command:

```
DESC USER_ORDER;
```

Figure 19.10 shows the output of the command.

```
mysql> DESC USER_ORDER;
+------------+---------+------+-----+---------+-------+
| Field      | Type    | Null | Key | Default | Extra |
+------------+---------+------+-----+---------+-------+
| ID         | int(11) | NO   | PRI | NULL    |       |
| USER_ID    | int(11) | NO   | MUL | NULL    |       |
| PRODUCT_ID | int(11) | NO   |     | NULL    |       |
+------------+---------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

Figure 19.10: Structure of the USER_ORDER Table

Inserting data into Table

7. Insert sample data into both the tables using the following commands. We shall add two user related data and add two orders for user with ID as 1 and one order for user with ID as 2:

```
INSERT INTO USERS(ID,USER_NAME,EMAIL,PHONE) VALUES ( 1, 'JAMES' ,'JAMES@
HP.COM', 12345);
INSERT INTO USERS(ID,USER_NAME,EMAIL,PHONE) VALUES (2, 'TONY', 'TONES@
HONS.COM', 345344);
```

```
INSERT INTO USER_ORDER(ID,USER_ID,PRODUCT_ID) VALUES (1,1,1);
INSERT INTO USER_ORDER(ID,USER_ID,PRODUCT_ID) VALUES (2,1,2);
INSERT INTO USER_ORDER(ID,USER_ID,PRODUCT_ID) VALUES (3,2,3);
```

Creating a Stored Procedure

8. To create a stored procedure that will return the total number of orders made by a user, first, change the delimiter to $$ using the following command:

```
 delimiter $$
```

9. To create the stored procedure, execute the following query:

```
 CREATE PROCEDURE COUNTOFORDERS(in N INT, out ordercount int)
BEGIN
SELECT COUNT(*) INTO ordercount FROM USER_ORDER WHERE USER_ID = N;
END
```

Figure 19.11 shows the stored procedure creation.

```
mysql> CREATE PROCEDURE COUNTOFORDERS
    -> (in N INT, out ordercount int)
    -> BEGIN
    -> SELECT COUNT(*) INTO ordercount FROM USER_ORDER
WHERE USER_ID = N;
    -> END
    -> $$
Query OK, 0 rows affected (0.00 sec)
```

Figure 19.11: Creating the Stored Procedure

10. To change the delimiter to ;, use the following command:

```
delimiter ;
```

Using the Stored Procedure in Queries

11. To obtain all the data from the `USERS` table and to show the count of order done by each user, execute the following query:

```
SET @input =2;
CALL COUNTOFORDERS(@input, @output);
SELECT @output;
```

Figure 19.12 shows the output of the command.

```
mysql> SET @input =2;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL COUNTOFORDERS(@input, @output);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @output;
+---------+
| @output |
+---------+
|       1 |
+---------+
1 row in set (0.00 sec)
```

Figure 19.12: Obtaining Data from USERS Table

The output returns the total number of orders for the particular user. We have passed userID as a parameter to the CountOFOrders function and it returned the count.

Transaction Management

We will explore how transactions are managed in MySQL. In order to perform this, we need two connections connected to the same local instance of MySQL. Run `mysqld` on two different terminals.

Here, we will continue to use the tables created earlier in the `INVENTORY` database.

1. Execute the following query in both connection1 and connection2 terminals:

```
USE INVENTORY;
```

Figures 19.13 and 19.14 show the output of the command.

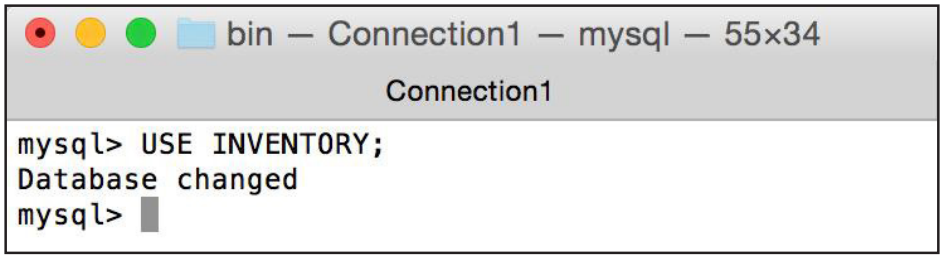


Figure 19.13: Output on Connection1 Terminal

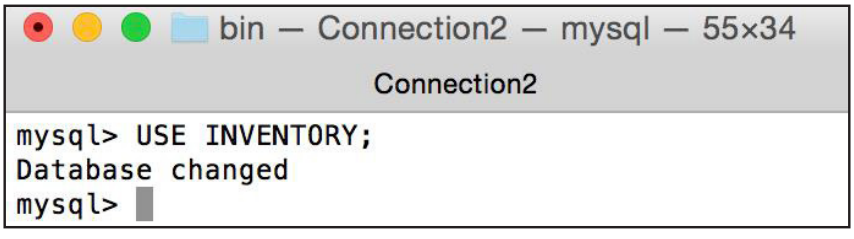


Figure 19.14: Output on Connection2 Terminal

2. Execute the following query on both the connections in order to check the state of the table USERS. The output will be the same on both the connections.

```
SELECT * FROM USERS;
```

Figure 19.15 shows the output of the command.

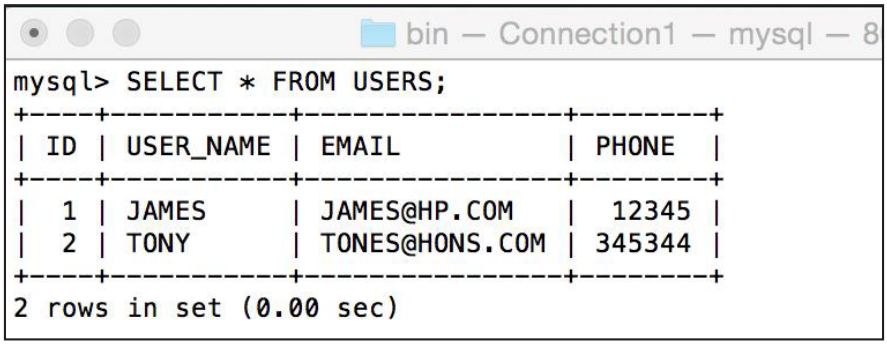


Figure 19.15: Checking Table USERS

Now, let us see how a transaction works in MySQL.

3. To start a transaction, run the following query only on connection1.

```
START TRANSACTION
```

Figure 19.16 shows the output of the command.

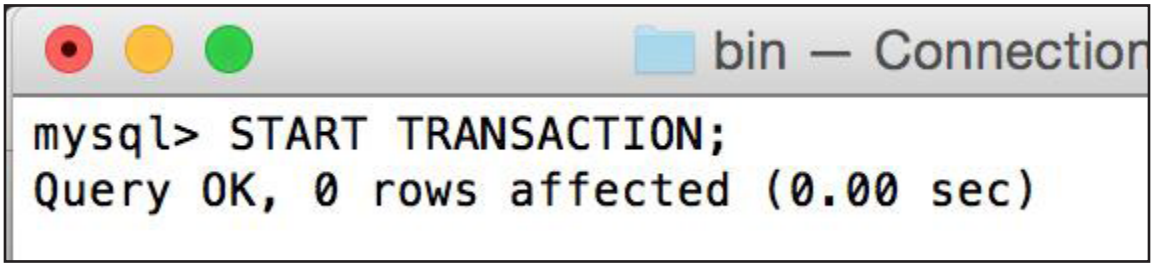


Figure 19.16: Starting a Transaction

4. Insert a row into the USERS table using the following query:

```
INSERT INTO USERS(ID,USER_NAME,EMAIL,PHONE) VALUES (3,'Mary','MARY@
JEEPS.COM',88778);
```

Figure 19.17 shows the output of the command.

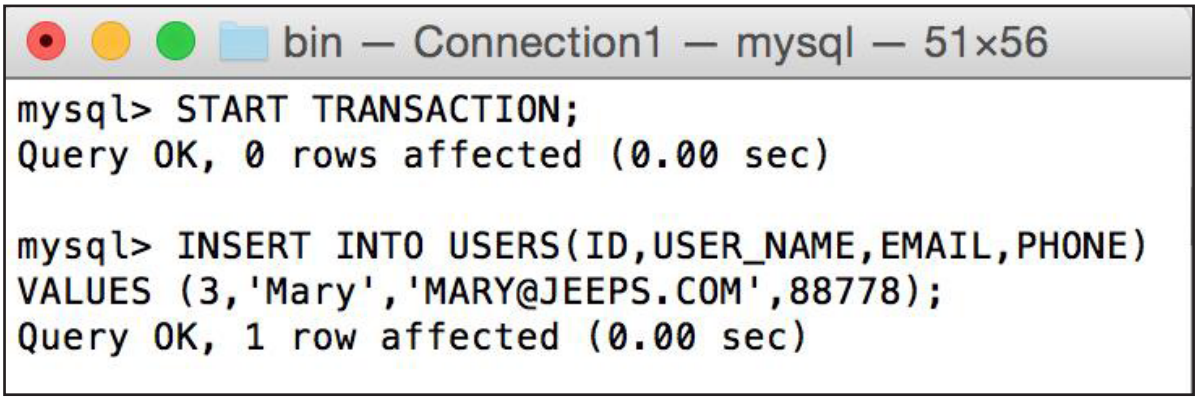


Figure 19.17: Inserting Rows into the Table

5. Run the following query on both connection1 and connection2:

```
SELECT * FROM USERS;
```

Figures 19.18 and 19.19 show the output of the command.

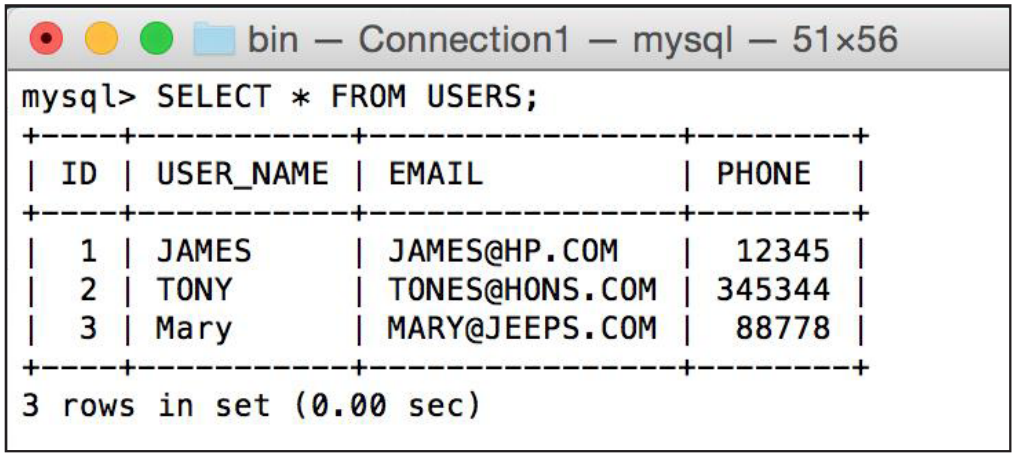


Figure 19.18: Output on Connection1 Terminal

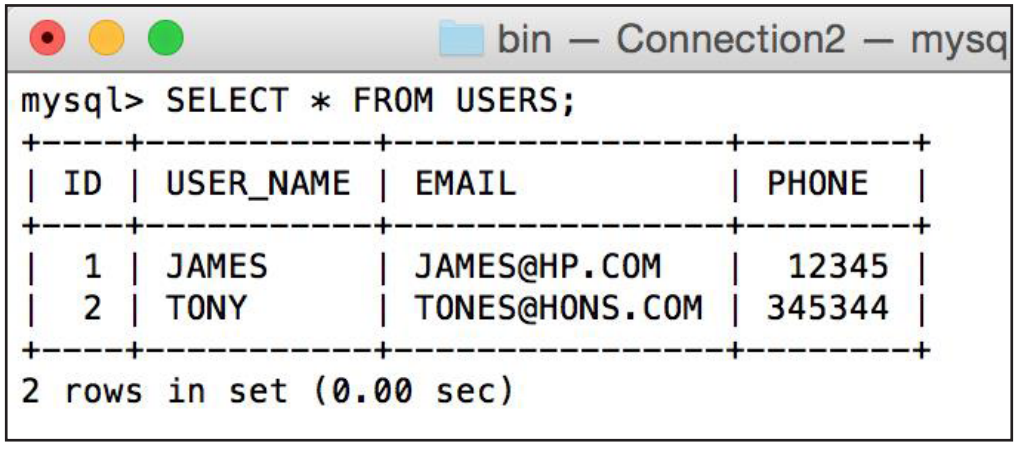


Figure 19.19: Output on Connection2 Terminal

We can see that output of connection1 is returning all three rows, including the row that was inserted in Step 4. However, the output of connection2 is returning only two rows, which means the data inserted in connection1 is not yet visible to connection2.

6. To commit the transaction that we have started in the connection1, run the following query in the connection1 terminal:

```
commit;
```

Figure 19.20 shows the output of the command.

```
mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

Figure 19.20: Committing the Transaction

7. Run the following query on connection1 and connection2.

```
SELECT * FROM USERS;
```

Figures 19.21 and 19.22 show the output of the command.

```
bin — Connection1 — mysql — 51×56
mysql> SELECT * FROM USERS;
+----+-----------+----------------+--------+
| ID | USER_NAME | EMAIL          | PHONE  |
+----+-----------+----------------+--------+
|  1 | JAMES     | JAMES@HP.COM   |  12345 |
|  2 | TONY      | TONES@HONS.COM | 345344 |
|  3 | Mary      | MARY@JEEPS.COM |  88778 |
+----+-----------+----------------+--------+
3 rows in set (0.00 sec)
```

Figure 19.21: Output After Commit on Connection1 Terminal

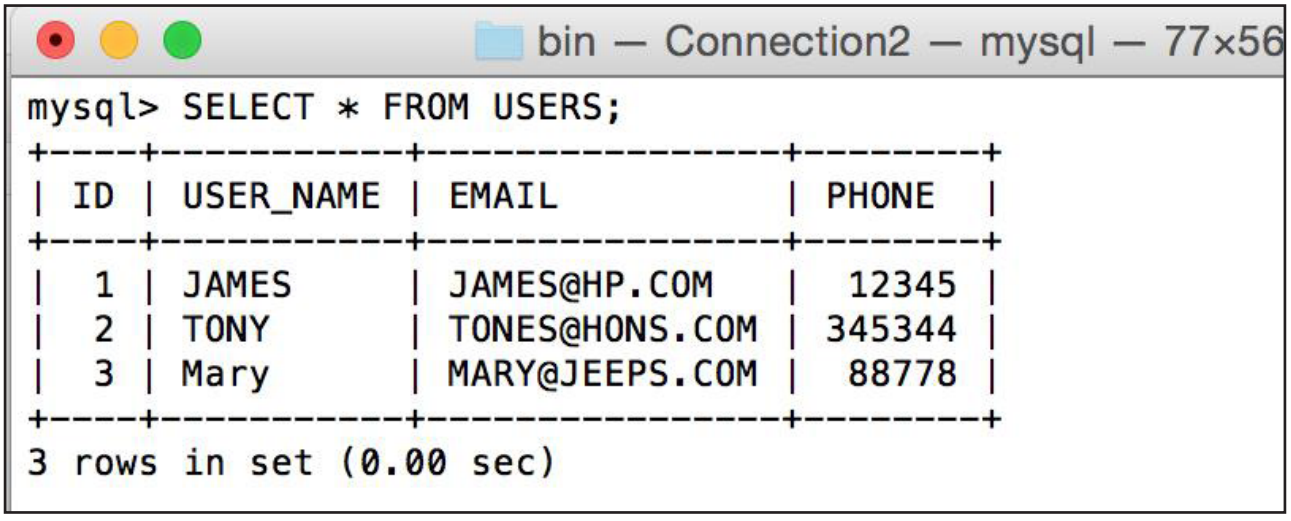


Figure 19.22: Output After Commit on Connection2 Terminal

As the transaction in connection1 is committed to the database, data is available in connection2 and so the output of the `SELECT` query is same in both the connections.

Similarly, let us try to understand the transaction with the Update query.

8. Start the transaction and execute the following update query in connection1. The query will update the e-mail domain to Gmail.

```
START TRANSACTION;
UPDATE USERS SET EMAIL='MARY@GMAIL.COM' WHERE ID = 3;
```

Figure 19.23 shows the output of the commands.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE USERS SET EMAIL='MARY@GMAIL.COM' WHERE ID = 3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Figure 19.23: Starting the Transaction and Executing Update

9. Run the following `SELECT` query on both the connections:

```
SELECT * FROM USERS;
```

Figures 19.24 and 19.25 shows the output of the command.

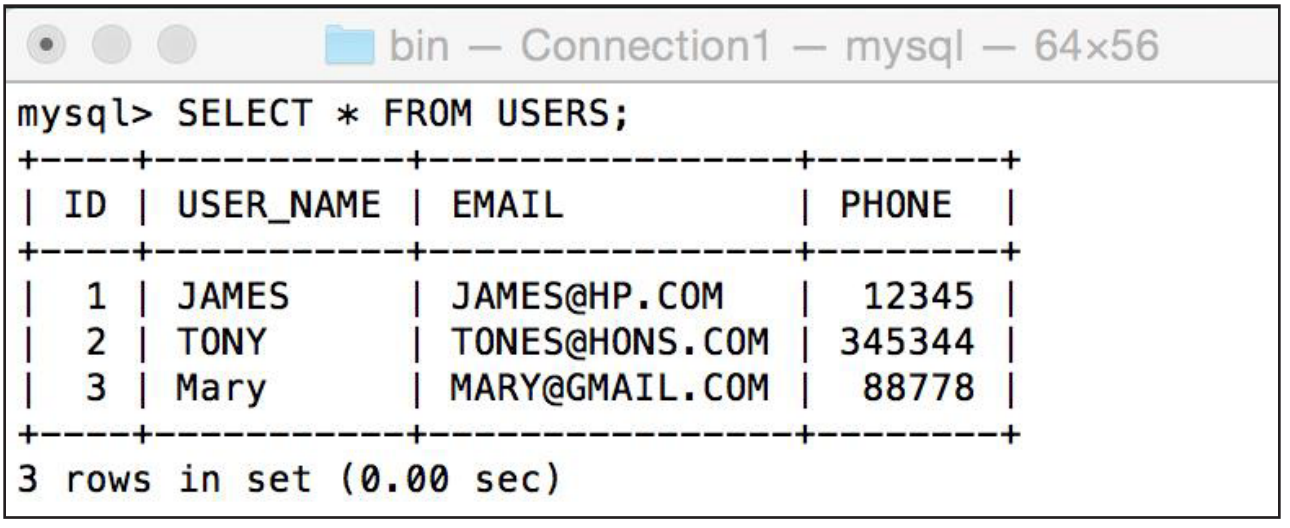


Figure 19.24: Output After Starting Transaction on Connection1 Terminal

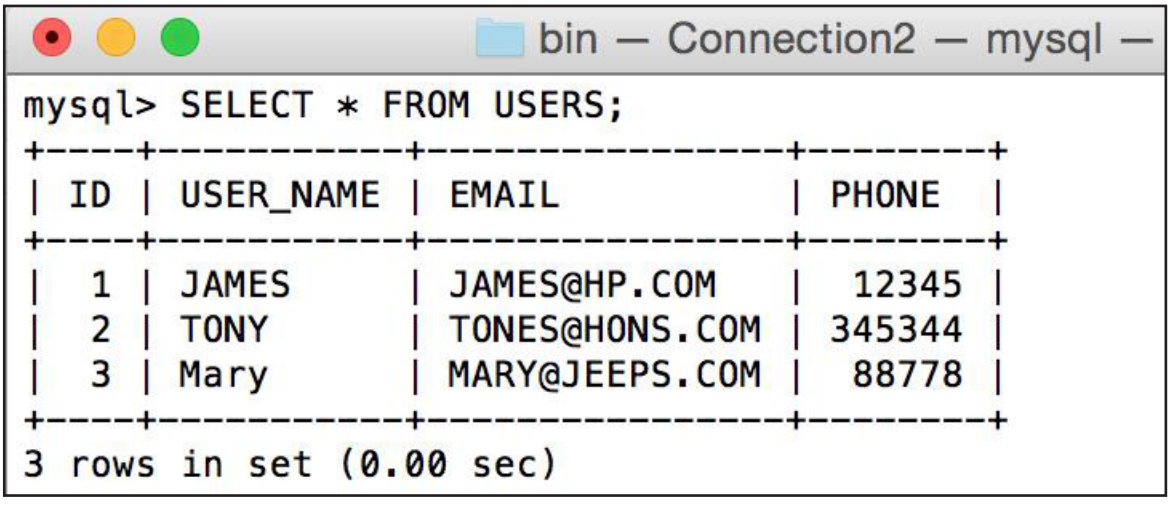


Figure 19.25: Output After Starting Transaction on Connection2 Terminal

As the transaction in connection1 is not committed, the `UPDATE` query changes are not effected in connection2. If we commit the transaction in connection1, then the changes will be shown in connection2. However, we do not want to commit the changes; instead, we want to revert the changes that have been done using the `UPDATE` query.

10. To revert, use the `ROLLBACK` command. This will move the state of the database to the point when the transaction is started.

```
ROLLBACK;
```

Figure 19.26 shows the output of the command.

Figure 19.26: Output of Rollback

11. Run the following select query on both the connections:

```
SELECT * FROM USERS;
```

Figures 19.27 and 19.28 show the output of the command.

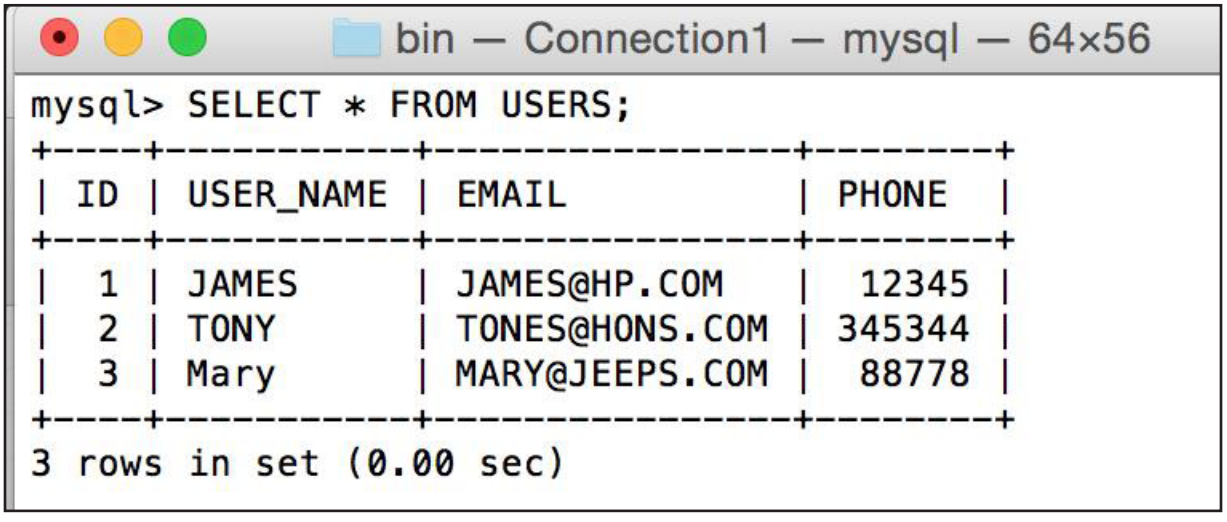


Figure 19.27: Output After Rollback on Connection1 Terminal

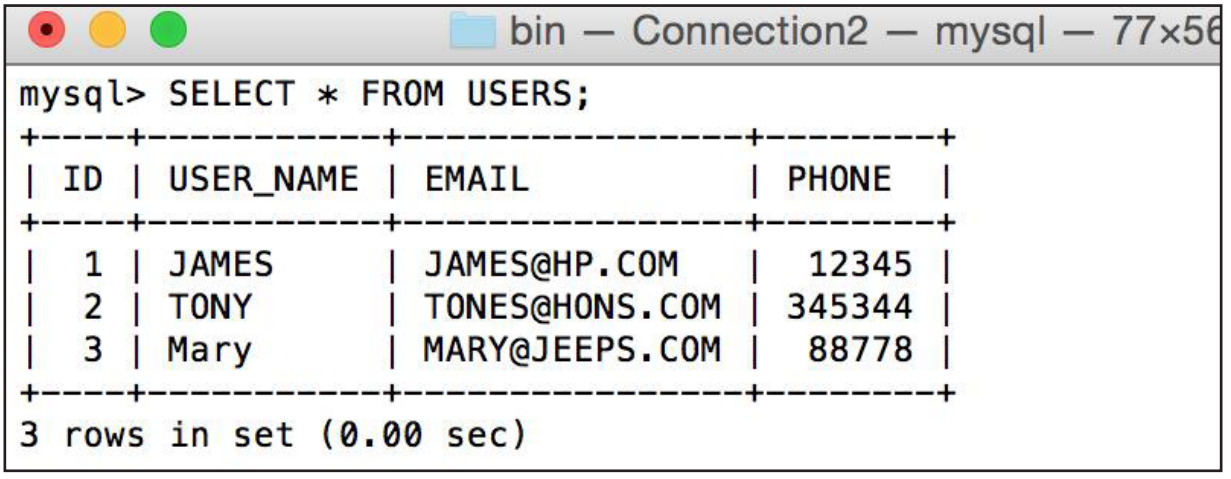


Figure 19.28: Output After Rollback on Connection1 Terminal

As we have executed `ROLLBACK` on the transaction, the data in connection1 is not updated.

Understanding Deadlocks

12. Start the transaction and execute the following update query in connection1. The query updates the e-mail domain to Gmail.

```
 START TRANSACTION;
UPDATE USERS SET EMAIL='MARY@GMAIL.COM' WHERE ID = 3;
```

Figure 19.29 shows the output of the command.

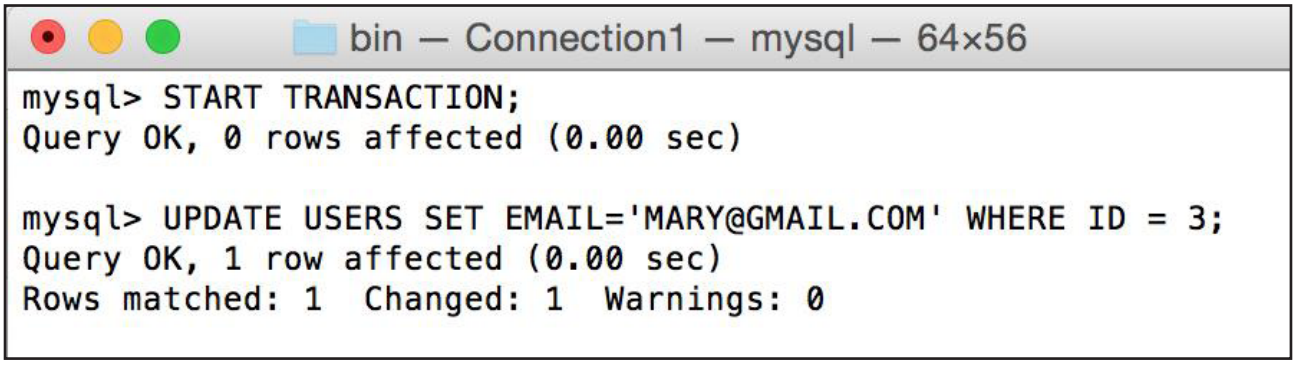


Figure 19.29: Output After Transaction with Update on Connection1 Terminal

Do not commit or rollback the transaction in connection1.

13. Start the transaction and execute the following update query in connection2. The query updates the email domain to Yahoo.

```
START TRANSACTION;
UPDATE USERS SET EMAIL='MARY@YAHOO.COM' WHERE ID = 3;
```

Figure 19.30 shows the output of the command.

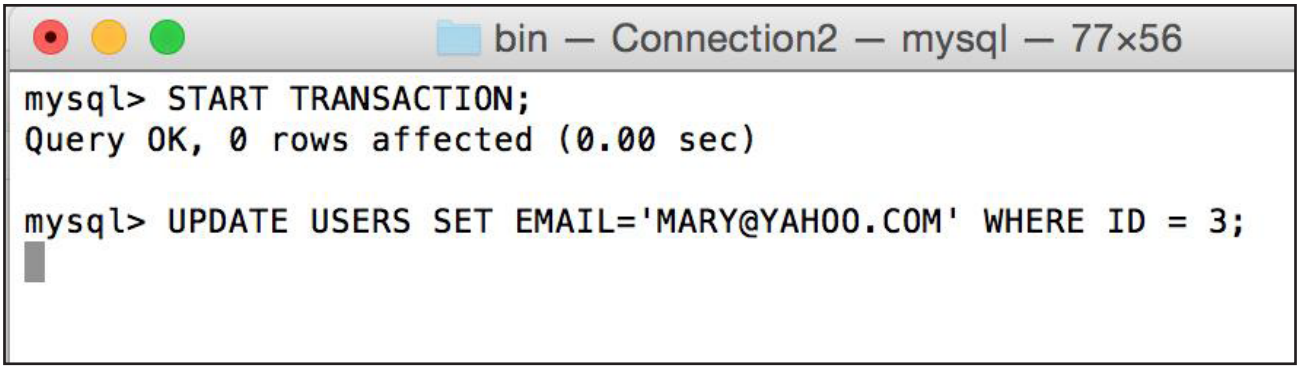


Figure 19.30: Output After Transaction with Update on Connection2 Terminal

When you execute the Update query on conenction2 for the same row, the query does not execute because the transaction of connection1 acquired a lock on that row. The Update in connection2 will fail after some time as shown in the figure 19.31.

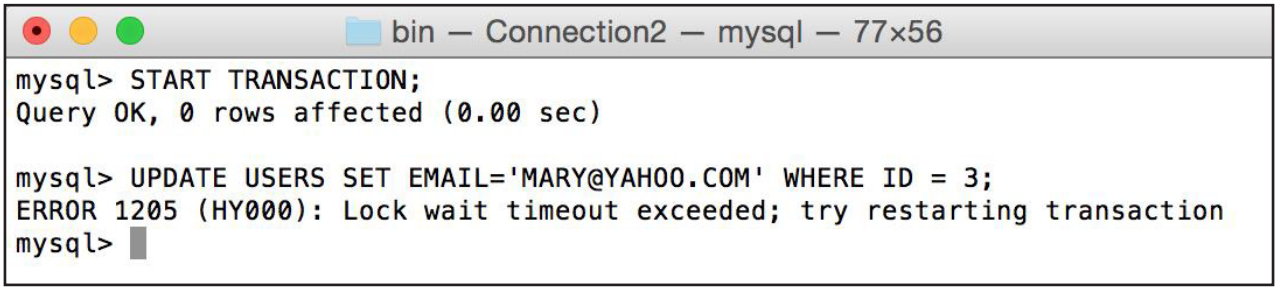


Figure 19.31: Error on Connection2 Terminal

14. Let us execute the query in Step 13 again on connection2 and immediately commit the changes in connection1. The Update query in connection2 will be waiting and if we commit the changes in connection1 before the timeout, the Update query will execute immediately in connection2.

Figure 19.32 shows the output of the command.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE USERS SET EMAIL='MARY@YAHOO.COM' WHERE ID = 3;
Query OK, 1 row affected (12.30 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM USERS;
+----+-----------+----------------+--------+
| ID | USER_NAME | EMAIL          | PHONE  |
+----+-----------+----------------+--------+
|  1 | JAMES     | JAMES@HP.COM   |  12345 |
|  2 | TONY      | TONES@HONS.COM | 345344 |
|  3 | Mary      | MARY@YAHOO.COM |  88778 |
+----+-----------+----------------+--------+
3 rows in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

Figure 19.32: Output of Update Query on Connection2 Terminal

Similarly, if multiple threads are waiting to acquire lock then this condition is called deadlock.

Summary

- Stored procedure is defined as a collection of SQL commands that are stored and executed in MySQL server. They offer advantages such as greater speed, reduced code redundancy, resulting in better maintenance and increased database security.
- A transaction is a sequence of one or more SQL statements that together form a logical unit of work. The life cycle of a transaction includes six stages: active, partially committed, failed, aborted, committed, and terminated.
- Using locking technique, a transaction can obtain exclusive access temporarily to the required sections of a database. This prevents other transactions from using the locked data simultaneously.
- Deadlock is a condition in which none of the transactions can proceed because each one holds a lock on the data that the other one needs. The lock will not be released by both the transactions as they are waiting for the data to become available.
- A character set comprises a collection of symbols and encodings. A collation comprises a set of rules, which can be used for comparison of characters in a character set.
- The different time zone settings used by MySQL are System time zone, Server's current time zone and Per-connection time zones.
- The lc_time_names system variable implies the locale that governs the language used for displaying the day, month, and abbreviations.

Check Your Progress

1. Match the following statements with their description.

Statements		Description	
a.	SET TRANSACTION	1.	Ends a failed transaction and returns it to the beginning of a transaction
b.	SAVEPOINT	2.	Sets the constraint mode and allows you to decide if a constraint needs to be applied to the data while it is modified or scheduled to be applied later
c.	COMMIT	3.	Serves as an intermediate retrieval point
d.	SET CONSTRAINTS	4.	Sets the attributes of the next transaction to be executed
e.	ROLLBACK	5.	Ends a successful transaction and saves all the modifications done to the database

2. Which of the following statements cause an implicit commit?

   a. CREATE TEMPORARY TABLE

   b. LOAD INDEX INTO CACHE

   c. UPGRADE DATA DIRECTORY NAME

   d. CREATE PROCEDURE

3. Stored procedures with which of the following option have access privileges that are same as the MySQL user who executed the stored procedure?

   a. SQL SECURITY DEFINER

   b. SQL SECURITY EXECUTER

   c. SQL SECURITY INVOKER

   d. SQL SECURITY CREATOR

Check Your Progress

4. Which of the following statements are true with respect to stored procedures?

   a. Stored procedures are precompiled

   b. 'Call' statement cannot be used to invoke stored procedures

   c. Stored procedures have a statement for raising application errors

   d. Stored procedures are fast

5. How does transaction/commit affect the performance?

   a. Increases performance when the changes are committed immediately

   b. Increases performance when the changes are committed after a valid number of updates

   c. The atomicity aspect of the ACID model is for all storage engines

   d. Implicit commit statements can be used within a transaction that is, between starting and committing a transaction

Onlinevarsity

GROUPS

COLLABORATIVE LEARNING

Session 20

Performance Optimization and Scalability

Concepts

Objectives

At the end of this session, the student will be able to:

- *Describe performance optimization.*
- *Describe query related functions.*
- *Explain capability of MySQL for scaling and availability.*

20.1 Performance Optimization

While working on a database, the common performance related issues that a user faces are as follows:

- To monitor if a server is performing optimally.
- To identify if a query is being processed quickly.
- To troubleshoot unexplained breakdowns, commonly referred as stalls, pileups, or freezes.

In this session, you will primarily explore solutions for these three issues and understand the tools and techniques that will help to accelerate the server's performance, and fix intermittent failures.

20.1.1 Introduction to Performance Optimization

The word 'performance' can spark different meanings in a database user's mind, varying from 'queries per second', 'CPU utilization' to 'scalability'. For consistency purposes, we shall consider 'performance' as a measure of time that is required to complete a task. In simpler words, it is the response time. It is measurable in terms of tasks and time and not by resources that are used. A database server basically executes SQL statements, so the tasks involved are execution of queries such as `SELECT`, `UPDATE`, and `INSERT`. A database server's performance is thus measured by the time it takes to process the query. The unit of measurement is time taken per query. Performance optimization is the process of using techniques to reduce the response time for a given workload. You may assume that this process pertains to reducing CPU usage, which is limiting resource consumption, however, that is not so. There are instances when an old version of MySQL is replaced with an ancient version of InnoDB and there is a remarkable increase in CPU utilization. This implies that InnoDB is utilizing the time to complete the tasks, rather than being hostile. In such instances, it is sensible to view the query response time to understand if the upgrade caused any improvement. Sometimes, an upgrade causes a bug to be introduced, for example, not using an index. This can also result in increased CPU utilization. Therefore, CPU utilization can be considered as only a symptom and not the main purpose and it is always reasonable to measure the latter.

Similarly, if you think performance optimization enhancing the number of queries processed per second, then it is throughput optimization. Increased throughput is nothing but a result of performance optimization. Optimizing queries helps the server to process more queries per second, as each query takes lesser process time when the server is optimized (throughput is measured in terms of queries per time, which is the inverse of the performance definition).

In order to reduce response time, you should analyze the time taken by the server to execute a query and reduce the surplus work it is performing to achieve the same result. In simple words, analyze how time is being utilized.

20.1.2 Indexing for High Performance

Indexes in MySQL (also called 'keys') are data structures used by storage engines to find rows quickly. They have numerous other constructive properties that will be explored here.

Indexes are helpful to achieve good performance and play a critical role when the data used is large. Small databases can execute reasonably even without using indexes but as the size increases, performance can reduce significantly. If indexes are used effectively, most of the leading performance related problems could be resolved.

Index optimization is the most effective technique to increase query performance. Indexes can increase performance by a large scale and optimal indexes can double the performance than indexes that are just good. Creating optimal indexes requires the queries to be rewritten.

To understand how an index works, let us consider a simple example of a book. To find where a specific term is discussed in a book, we view the index of the book, and it gives us the page number/numbers where that term appears. Similarly, in MySQL, a storage engine looks up the index's data structure for a given value. When a match is located, it finds the row containing that match. Consider an example of an employee table. When you execute the following query, assuming there is an index on the emp_id column, MySQL uses the index to find rows whose emp_id is 100955:

```
mysql> SELECT first_name FROM org.emp WHERE emp_id = 100955;
```

An index can include values from one or multiple columns in a table. When more than one column is indexed, the column order is very crucial as MySQL looks up efficiently on a leftmost prefix of the index.

Indexes created on two columns ([ProductId], [ProductName]) separately are not equal to the index that is created by both the columns together ([ProductId, ProductName]).

Let us understand how indexes enhance the performance with an example.

Example:

First, set up a test environment by performing the following steps:

1. Create a table using the following steps:

   a. Create a database named org and activate it using the following query:

```
mysql> CREATE DATABASE ORG;
USE ORG;
```

   b. Create a table emp with no indexes using the following query:

```
mysql> CREATE TABLE emp( id int(11) NOT NULL, firstname varchar(255)
NOT NULL, lastname varchar(255) NOT NULL);
```

   c. Insert data into the table. Assume that around 9999 records are inserted.

   d. Check the count of entries in the database using the following query. There are 9999 records in the emp table:

```
mysql> select count (*) from emp;
```

   Figure 20.1 shows the output of the query:

```
mysql> select count(*) from emp;
+----------+
| count(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.00 sec)
```

Figure 20.1: Output Showing Count of Entries

2. Use `EXPLAIN` to check the count of records with or without an index:

```
mysql> EXPLAIN SELECT * from emp where id = 155;
```

As we are performing a non-indexed search, using the `EXPLAIN` clause preceding the `SELECT` query will display which indexes (if any) MySQL would use.

This is shown in figure 20.2.

```
mysql> EXPLAIN select * from emp where id = 155;
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | emp   | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 9897 |    10.00 | Using where |
+----+-------------+-------+------------+------+---------------+------+---------+------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)

mysql> select * from emp where id = 155;
+-----+--------------+-------------+
| id  | firstname    | lastname    |
+-----+--------------+-------------+
| 155 | firstName155 | lastname155 |
+-----+--------------+-------------+
1 row in set (0.00 sec)
```

Figure 20.2: Output Showing the Result of Using EXPLAIN

From the preceding result, we can see that MySQL has searched through 9897 rows to select one row. This shows that the MySQL optimizer did not find any keys to use. All the key values here are 'NULL'.

3. Create a simple index and perform a query with `SELECT`. Create an index on the 'id' column and run the following statement:

```
mysql> ALTER TABLE emp ADD INDEX (id);
```

The output is displayed as shown in figure 20.3:

```
mysql> ALTER TABLE emp ADD INDEX (id);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Figure 20.3: Output Showing the Result of SELECT

Run the same `EXPLAIN` query as in Step 2:

```
mysql> EXPLAIN select * from emp where id = 155;
```

The output is displayed as shown in figure 20.4:

```
mysql> EXPLAIN select * from emp where id = 155;
+----+-------------+-------+------------+------+---------------+------+---------+-------+------+----------+-------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref   | rows | filtered | Extra |
+----+-------------+-------+------------+------+---------------+------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | emp   | NULL       | ref  | id            | id   | 4       | const |    1 |   100.00 | NULL  |
+----+-------------+-------+------------+------+---------------+------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.4: Output Showing the Result of EXPLAIN

We can now notice that after creating an index, it searches through only one row. This example clearly demonstrates how performance optimization can be achieved using indexes. Let us now understand the `EXPLAIN` statement.

EXPLAIN Syntax

Following is the syntax for EXPLAIN:

```
{EXPLAIN | DESCRIBE | DESC}
tbl_name [col_name | wild]
{EXPLAIN | DESCRIBE | DESC}
[explain_type]
{explainable_stmt | FOR CONNECTION connection_id}
explain_type: {
EXTENDED
| PARTITIONS
| FORMAT = format_name }
format_name: {s
TRADITIONAL
| JSON }
explainable_stmt: {
SELECT statement
| DELETE statement
| INSERT statement
| REPLACE statement
| UPDATE statement
}
```

The `DESCRIBE` and `EXPLAIN` statements are considered synonyms. Generally, `DESCRIBE` is used to get data related to table structure and `EXPLAIN` to get a query execution plan (a description of how MySQL processes a query).

Concepts

MySQL and Indexes

B-trees are used to save most MySQL indexes (`PRIMARY KEY`, `UNIQUE`, `INDEX`, and `FULLTEXT`). Only indexes on spatial data types are stored as R-trees and MEMORY tables support hash indexes.

Some scenarios when indexes are used by MySQL are as follows:

- To eliminate rows: When there are multiple indexes, MySQL generally picks the index that obtains least number of rows (the most selective index).
- Looking up rows in a table with a multiple-column index; the optimizer can use any leftmost prefix of the index to obtain rows.
- To obtain rows from different tables when using joins. MySQL can handle indexes on columns effectively when they are of the same type and size. `VARCHAR` and `CHAR` are judged as the same if both are of the same size. For example, `VARCHAR(5)` and `CHAR(5)` are considered as the same size, but not `VARCHAR(5)` and `CHAR(15)`.
- To obtain rows that match a `WHERE` clause faster.
- To compare non-binary string columns, when both columns are using the same character set. For example, comparing a UTF-16 column with a UTF-32 column prevents using an index.
- To compare columns of same type as the values need not be converted. When comparing columns of different types (such as, a string column to a temporal or numeric column) indexes cannot be used as the values must be converted before comparing. For example, a value 1 in the numeric column is considered equal to '2', ' 2', '00002', or '02.e2' values in the string column. This eliminates using indexes for the string column.
- To obtain the `MIN()` or `MAX()` value for a particular indexed column `key _ column`. Consider a multiple-part index defined as `key1` (`key _ part1`, `key _ part2`, `key _ part3`). The preprocessor identifies if `WHERE key _ part _ N` = constant is applied on all key parts that occur prior to `key _ column` in the index. `Here`, MySQL performs a single key lookup for each `MIN()` or `MAX()` field and changes it to a constant. If all the fields are changed to constants, the query returns at once.

  For example,

```
SELECT MIN(key_part2),MAX(key_part2) FROM tbl_name WHERE key_part1=10;
```

- To sort or group a table when the operation is performed on a leftmost prefix of an index (for example, `ORDER BY key _ part1`, `key _ part2`). If all key parts are proceeded by the keyword `DESC`, then the key is read in descending order.

20.1.3 Indexing Strategies for High Performance

In order to enhance the performance, it is vital to create the correct indexes and use them appropriately. Following are the strategies to do this:

- **Isolating the Column**: It is very common for queries to defeat the purpose of indexes or stop MySQL from using the existing ones. Usually, MySQL cannot apply indexes on columns except when they are isolated in the query. The term isolating means that the columns must not be included in an expression or in a function in the query. For example, following is a query that cannot use the index on emp_id:

```
mysql> SELECT * FROM EMPLOYEE WHERE emp_id + 1 = 10;
```

  In the example, we can simplify the WHERE clause as emp_id = 9 but MySQL cannot solve the equation for emp_id (emp_id + 1 = 10). Therefore, even though 'emp_id' is already indexed, MySQL will not fetch the results using indexes. This must be performed by the user by providing a simple WHERE criteria, so that only the indexed column forms one side of the comparison operator.

- **Prefix Indexes and Index Selectivity**: Indexing columns with long characters makes indexes large and inefficient. In order to save storage space and to enhance performance, index the first few characters rather than the complete value. By doing this, you can reduce the space and also make them less selective. A prefix of the column is generally sufficient to increase the performance. When indexing BLOB, TEXT, or very long VARCHAR columns, declare prefix indexes, as MySQL does not allow indexing the complete length. The best approach is to select a prefix that is of enough length to offer correct selectivity but is small enough to reduce space. The prefix must be selected such that the index is as helpful as indexing the full column. To identify a prefix of good length, obtain a list of the most common values and relate it to a list of the most common prefixes.

Multicolumn Indexes

Single indexes on multiple columns will not support to enhance the performance for most queries. MySQL 5.0 and newer versions can cope up with such poor indexing by employing index merge technique.

The Index Merge methodology helps to obtain rows with numerous range scans and to merge their output into a single file. This type of merge can generate unions, intersections, or unions-of-intersections of the result. This access method merges data from a single table and not across multiple tables.

In EXPLAIN output, this merge is displayed as `index_merge` in the type column. In this case, the key column includes a record of all the indexes utilized, and `key_len` includes a record of the longest key parts for those indexes.

Column Order

The order of columns in an index is one of the main causes of uncertainty. The proper order is dependent on the queries utilizing the index and it must be selected such that the rows are ordered and combined in a manner that will benefit the query.

For example, in a multicolumn B-Tree index, the order of columns is such that the index is sorted beginning with the leftmost column, then by the next column, and so on. This allows to scan the index in either frontward or backward order to support queries with `ORDER BY`, `GROUP BY`, and `DISTINCT` clauses and match the column order perfectly.

Clustered Indexes

All tables in InnoDB have a unique index to store rows' data that is termed as clustered index. Mostly, this index is the same as the primary key. To enhance the performance of queries and other database operations, analyzing how InnoDB utilizes this type of index to improve the most common lookup and DML operations for each table is important:

- When a PRIMARY KEY is declared for a table, InnoDB considers it as the clustered index. For every table created, set a primary key. If the table does not have a logically unique and not null column, then add an auto increment column to the value.

- When a PRIMARY KEY is not declared for a table, MySQL identifies the first UNIQUE index in which all the key columns are NOT NULL and InnoDB considers it as the clustered index.

- When a PRIMARY KEY or suitable UNIQUE index is not found for a table, InnoDB creates a clustered index on a column that consists of row ID values and this index is hidden. The rows in such tables are arranged by the ID with which InnoDB is associated. The size of row ID field is monotonic 6-byte and it expands as and when new rows are added.

  Finding a row using the clustered index is faster as the search results take you to the page that includes all the row data. In case of large tables, the clustered index architecture saves a disk I/O operation (for example, storage engines such as MyISAM saves data rows in one file and index records in a different file).

Covering Indexes

An index that includes (covers) all the data that is necessary for a query is called a covering index. In a query, it is generally recommended to create indexes for the `WHERE` clause but that is not the end. Indexes must be created for the full query and not just the `WHERE` clause. Indexes not only help in finding rows efficiently, they can also be used to obtain column's data, thus preventing the need to read rows.

These indexes are a powerful tool and when used correctly can dramatically increase the performance. Imagine the outcome of going through only the index rather than the complete data.

Following are the benefits of using covering indexes:

- Reading index entries is much quicker than reading the complete row data. The data to be read is much smaller if it reads only the index. This plays a vital role when working with the following:
  - **Cached workloads**: As majority of the response time comes from replicating the data.
  - **I/O-bound workloads**: As the indexes are comparatively smaller than the data and can be stored in a better way. (In MyISAM, it can make the indexes even smaller by packing them).
- Indexes are arranged based on their index values (within the page), thus I/O-bound accesses perform lesser transactions when compared to retrieving each row from a random disk location. In MyISAM and Percona XtraDB, one can OPTIMIZE the table to obtain completely sorted indexes, thus allowing simple queries to perform sequential index accesses.
- MyISAM caches only the index in the memory. As caching is performed by the operating system, a system call is needed to access it. This leads to on overhead on the performance, especially for cached workloads because the system call is the most pricy part of data access.
- Covering indexes are beneficial for InnoDB tables because of clustered indexes. The secondary indexes maintain the primary key values of the row at their leaf nodes. This index search in the primary key is prevented by the secondary index that covers a query.

  In all the preceding scenarios, the expense is much lesser to satisfy a query from an index rather than reading through the rows.

Using Index Scans for Sorts

MySQL follows the following two approaches to generate ordered results:

- Use a sort operation
- Scan an index in order

  You can identify if MySQL is scanning an index by searching in the type column for ‘index’ in EXPLAIN. The second approach is faster as it simply involves searching through index entries. If MySQL has not employed an index to cover the query, then it has to scan every row it finds in the index.

Redundant and Duplicate Indexes

MySQL does not stop you from creating many indexes on the same column. This requires the duplicate indexes to be managed separately. As the query optimizer must validate each of them when it optimizes queries, it impacts the performance.

Concepts

Note: Indexes of the same type, which are declared on the same set of columns in the same order, are called duplicate indexes. It is recommended to avoid creating such indexes, and must be removed if found.

Unused Indexes

Along with duplicate indexes, there are some indexes that the server simply does not utilize them. These are unwanted and it is better to discard them. Following two tools can help to accomplish this:

- `INFORMATION _ SCHEMA.INDEX _ STATISTICS` table in Percona Server
- MariaDB

Enable the `userstats` server variable, which is disabled by default. Allow the server to run for a while, and you can view how much each index is utilized.

Disadvantages of Indexes

Though indexes help in avoiding duplicate row data, there are some disadvantages with indexes, listed as follows:

- When creating indexes on column/columns, MySQL also generates a distinct ordered file that holds only the field(s) that need to be sorted.
- Indexes need plenty of disk space. Generally, the space consumed is significant but due to the creation of indexes on every column in all possible combinations, the file requires more space than a data file. When using indexes with larger tables, the index file can touch the maximum file size allowed on the operating system.
- Indexes reduce the performance of queries such as `INSERT`, `UPDATE`, and `DELETE`. In a data file, MySQL has to internally manage pointers to the inserted rows. This causes an overhead on the performance.

20.2 Query Caching

The query cache saves the data of a `SELECT` statement along with the result sent to the client. Later, if a matching statement is issued, instead of processing the statement again, the server simply obtains the results from the query cache. The query cache is shared by multiple sessions, thus, allowing a result set generated by one client to be sent to another client when the same query is executed.

The query cache is suitable for tables that are not modified very often and for which the server gets many identical queries. This situation is common among many Web servers that create numerous dynamic pages based on the database content. When tables are updated, any matching entries in the query cache are removed. Thus, the query cache does not save any irrelevant data.

To know whether the query cache is present in your MySQL server, execute the following command:

```
SHOW VARIABLES LIKE 'have_query_cache';
```

Performance Data

Once the server starts to deactivate the query cache, ensure the `query_cache_size` system variable is set to 0. By deactivating the query cache option, there will not be any noticeable overhead.

Using query cache increases the performance but not under all circumstances. Some query cache designs or server workloads actually reduce the performance, such as the following:

- When the query cache is extremely large, the overhead involved to manage the cache increases, possibly beyond the benefit of enabling it. Favorable sizes are in tens of megabytes whereas hundreds of megabytes are not.

- Server workload impacts the efficiency of query cache. A query that includes mostly a stable set of `SELECT` statements benefits more from using the query cache than a query in which frequent `INSERT` statements result in continuous invalidation of results. In some instances, this can be overcome by using the `SQL _ NO _ CACHE` option. This stops the results from using the cache for `SELECT` statements that read from frequently altered tables.

In order to validate if the query cache is advantageous, test the functioning of the MySQL server with the cache activated and deactivated. Perform this validation frequently as the query cache efficiency can change depending on the server workload.

20.2.1 Query Cache Operation

You will now understand how the query cache operates. Any query is first compared with those existing in the query cache before processing. The query cache considers the following two queries as different:

- `SELECT * FROM tbl _ name`

- `Select * from tbl _ name`

The queries that are precisely the same (byte to byte) are considered identical. Sometimes, query strings are considered as different, though they are identical. The reasons may be because the queries read dissimilar databases, protocol versions, or default character sets. In such instances, they are cached separately.

The cache cannot be used for the following query types:

- Queries that are a sub-query of another query

- Queries processed within a stored function, trigger, or event

Before a result is read from the query cache, MySQL examines if the user is provided with SELECT privilege for all the databases and tables. If this condition is not satisfied, the cached result is not utilized.

If a result is obtained from the query cache, the variable `Qcache_hits` status is incremented and not the `Com_select` variable. The `Qcache_hits` server status variable represents the number of query cache hits. The `Com_select` variable indicates the number of times each `SELECT` statement has been executed.

When a table is modified, all cached queries referencing the table become unusable and are dropped from the cache. For example, queries that use `MERGE` tables map to the modified table. A table can be modified by numerous statements such as `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE TABLE`, `ALTER TABLE`, `DROP TABLE`, or `DROP DATABASE`.

Following are instances when the data can be saved in query cache:

- In transactions with InnoDB tables.
- The output from a `SELECT` query on a view.
- `SELECT SQL _ CALC _ FOUND _ ROWS` ... queries and the value returned by `SELECT FOUND _ ROWS()` query. `FOUND _ ROWS()` returns the accurate value even if the previous query was obtained from the cache as the number of found rows is also saved in the cache. It is not possible to cache the `SELECT FOUND _ ROWS()` query itself.

Following are instances when there is a limitation on using query cache:

- Prepared statements that use the binary protocol by executing `mysql _ stmt _ prepare()` and `mysql _ stmt _ execute()`. In a query cache, an issued query is matched with the text of the existing queries only after the ? parameter markers are expanded. The query comparison is performed only with other queries in the cache that were processed using the binary protocol. In the query cache, the prepared statements executed with the binary protocol are separated from the ones executed using the text protocol.
- If the query includes any of the functions listed in table 20.1:

Function Name	Function Name	Function Name
AES_DECRYPT() (as of 5.7.4)	AES_ENCRYPT() (as of 5.7.4)	BENCHMARK()
CONNECTION_ID()	CONVERT_TZ()	CURDATE()
CURRENT_DATE()	CURRENT_TIME()	CURRENT_TIMESTAMP()
CURRENT_USER()	CURTIME()	DATABASE()
ENCRYPT() with one parameter	FOUND_ROWS()	GET_LOCK()
IS_FREE_LOCK()	IS_USED_LOCK()	LAST_INSERT_ID()

Function Name	Function Name	Function Name
LOAD_FILE()	MASTER_POS_WAIT()	NOW()
PASSWORD()	RAND()	RANDOM_BYTES()
RELEASE_ALL_LOCKS()	RELEASE_LOCK()	SLEEP()
SYSDATE()	UNIX_TIMESTAMP() with no parameters	USER()
UUID()	UUID_SHORT()	

Table 20.1: Functions that are not Cached

- Additionally, a query will not be cached if any of these conditions are met:
  - If any User-Defined Functions (UDFs) or stored functions are a part of the query.
  - If user variables, local stored program variables, or partitioned tables are part of the query.
  - If tables in `mysql`, `INFORMATION _ SCHEMA`, or `performance _ schema` database are referred to.
  - If it is of the following forms:

```
SELECT ... LOCK IN SHARE MODE SELECT ... FOR UPDATE SELECT ... INTO
OUTFILE ... SELECT ... INTO DUMPFILE ... SELECT * FROM ... WHERE
autoincrement_col IS NULL
```

  - If statements within transactions have `SERIALIZABLE` isolation level (because `LOCK IN SHARE MODE` locking is used).
  - If `TEMPORARY` tables are referred.
  - If tables are not used.
  - If warnings are generated.
  - If column-level privilege is provided to the user on any of the referred tables.

20.2.2 Query Cache SELECT Options

In `SELECT` statements, the following two options can be specified with respect to the query cache:

- **SQL_CACHE**: When this option is used, the query result is cached and the system variable `query _ cache _ type` is set to `ON` or `DEMAND`.

- **SQL_NO_CACHE**: When this option is used, the query result is not cached and the server does not even attempt to check the cache for the result.

Following is an example that shows how these two types are used:

```
SELECT SQL_CACHE id, name FROM customer; SELECT SQL_NO_CACHE id, name
FROM customer;
```

The functioning of the cache is controlled by various other system variables that start with `query_cache`. To set these variables, you can use either the option file or the command line on mysqld startup.

20.2.3 Query Analysis

Developers and administrators can enhance the performance of the database by using the MySQL Query Analyzer to monitor query performance. This analyzer is capable of identifying the SQL code that is causing a slowdown. It uses graphs that provide a detailed insight into the database performance issues. Using the MySQL Query Analyzer helps developers to improve the code during the development stage and to periodically monitor and fine-tune queries.

The MySQL Query Analyzer offers the following advantages to the developers:

- Determines expensive queries affecting the performance of applications.
- Visualizes the complete query activity to obtain an insight into the performance beyond the scope of query statistics.
- Filters issues such as poor indexing and whole table scans with the help of advanced global search options.
- Identifies and fixes the SQL code that is causing a performance slowdown.

Developers and administrators can simultaneously compare MySQL server activity with the executing queries using the correlated graphs. To perform this:

1. Select an area on a graph to view the queries being processed at the chosen period.
2. Use the timeframes with other filtering options such as Query Type, Execution Counts, and First Seen to fine-tune the queries.

Following detailed information helps in identifying the queries that can be tuned for better performance:

- **Query Analyzer Table**: It maintains details of all the executing queries.
- **Query Response Time Index (QRTi)**: It recognizes queries with slow response times.
- **Response Statistics**: It provides information such as Execution Time and Row Statistics, Time Span, and First Seen.
- **Example Query**: It is a sample query that helps in further review process.
- **Explain Query**: It maintains the execution plan used for the query.
- **Graphs**: It provides key metrics such as Execution Time, Executions, Rows, and Kilobytes at a given time for a specific query.

The Query Analyzer monitors details of SQL statements such as the number of executions and execution times. Queries that are similar but use different literal values are merged (normalized) for reporting purposes. The analyzer gathers data about SQL statements sent by a MySQL client application to the MySQL server.

Following are the different ways through which a Query Analyzer can receive this data:

- Using MySQL Enterprise Monitor Agent and other Third-party Agents that gather data from Performance Schema of MySQL Server without any additional configuration.
- Using the Proxy and Aggregator, the client application transmits its database requests (queries). The Proxy then directs requests to both the MySQL server and the Aggregator. The requests are normalized by the Aggregator and sent to the Service Manager.
- Using a MySQL Enterprise Monitor Plugin that is installed for a Connector, the data is directly sent to MySQL Enterprise Service Manager.

Once the MySQL client application is set up to transmit data through the MySQL Enterprise Monitor Agent, the queries are supervised and sent to the MySQL Enterprise Monitor Agent after they are normalized.

After obtaining the data, the queries are monitored and the execution statistics are validated. The queries are compared with the server graphs in order to correlate query execution with the server status.

Providing Query Analyzer Data

MySQL Query Analyzer can receive data from numerous different sources. The statistical data about the queries, execution times, and result counts are sent to the analyzer along with other information to present and evaluate on the Query Analyzer page.

Following are the different methods to provide query information to MySQL Enterprise Service Manager:

- Using the feature Performance Schema statement digests with MySQL Server 5.6.14 and later, information can be directly obtained from MySQL Server without employing any further configurations.

- Using a MySQL connector with a corresponding MySQL Enterprise Monitor Plugin that offers tracking and statistical data to MySQL Enterprise Service Manager. This method involves using a connector to gather and transmit the statistical data to MySQL Enterprise Service Manager. The connectors gather statistical data of the query, such as the processing time for each query and the row counts, and provide the same to be analyzed by MySQL Enterprise Service Manager. Note: Proxy component is not necessary for this implementation.

- Using the MySQL Enterprise Monitor Proxy and Aggregator. The MySQL Enterprise Monitor Aggregator collects and summarizes the raw query statistics sent from the client application. This data is sent to the MySQL Enterprise Service Manager where it populates the Query Analyzer.

20.2.4 Optimizing Queries with EXPLAIN

The `EXPLAIN` statement gives information about how a statement is processed in MySQL. Following are some of the characteristics:

- Allowed statements for `EXPLAIN` are `SELECT`, `DELETE`, `INSERT`, `REPLACE`, and `UPDATE`.

- When `EXPLAIN` is used with any of the preceding allowable statements, MySQL obtains the statement execution plan from the optimizer. The execution plan provides information about how MySQL executes the statement, including details about how tables are combined and in which order.

- When `EXPLAIN` is used with `FOR CONNECTION connection _ id` instead of an allowed statement, it provides the execution plan for the statement mentioned.

- For versions prior to MySQL 5.7.3, `EXTENDED` keyword is required to get additional details about execution plan. For MySQL 5.7.3 and later, this keyword is not required as extended output is enabled by default.

- For versions prior to MySQL 5.7.3, `PARTITIONS` keyword is required to analyze queries consisting of partitioned tables. For MySQL 5.7.3 and later, this keyword is not required as partition information and is enabled by default.

- Use `FORMAT` option to choose the output format. Using the `TRADITIONAL` keyword displays output in tabular format. This is considered as the default in case no `FORMAT` option is specified. When the `FORMAT` option is set to JSON, the output displays both extended and partition information.

Using EXPLAIN statement, one can identify where to add indexes to find rows in order to process the statement faster. EXPLAIN can also be used to validate if the optimizer merges the tables in the best possible order. To merge a table based on the order in which the tables are named in a SELECT statement, start the statement with SELECT STRAIGHT_JOIN instead of just SELECT. However, the former may not allow indexes to be used as it disables semi-join transformations.

In case indexes are not utilized when they are supposed to be, execute ANALYZE TABLE to update statistical data of a table, such as cardinality of keys, which might impact the selection made by the optimizer.

Optimizing Multi-table Queries

In real world, data cannot be stored in one single table for an application. Typically, the data is too large and it must be normalized into multiple tables. To retrieve such data stored in multiple tables we must use join function. Let us understand this with a scenario when table 'P' is left-joined with table 'Q'.

MySQL executes a P LEFT JOIN Q join_condition as follows:

- Table Q is set to depend on table P and all other tables on which P is dependent.
- Table P is set to depend on all tables other than Q that are part of the LEFT JOIN condition.
- The LEFT JOIN condition is used to determine how to obtain rows from table Q. (In simple terms, you cannot use any condition in the WHERE clause).
- All Join optimizations are performed expect if there is a circular dependency. In this case, MySQL issues an error.
- WHERE conditions are optimized.
- If there is a row in P matching the WHERE clause but there is no row in Q matching the ON condition, then an additional Q row is created with NULL column values.
- When using LEFT JOIN to obtain rows which are not part of some table and you have the test col _ name IS NULL in the WHERE clause, where col _ name defines a column defined as NOT NULL, MySQL does not proceed to find more rows (for a specific key pattern) after it has obtained a row that matches the LEFT JOIN condition.

The implementation of RIGHT JOIN is similar to that of LEFT JOIN but the roles of the tables are swapped. The join optimizer determines the order for joining the tables. The option LEFT JOIN or STRAIGHT_JOIN supports the join optimizer to achieve this faster, because there are lesser table alternatives to validate.

Let us understand this with the following example:

```
SELECT * FROM p JOIN q LEFT JOIN r ON (r.key=p.key) LEFT JOIN s ON (s.key=p.
key) WHERE q.key=s.key;
```

With a query of this type, MySQL performs a complete scan on q because the `LEFT JOIN` compels it to be read before s.

The solution is to change the order in which `p` and `q` are listed in the `FROM` clause:

```
SELECT * FROM q JOIN p LEFT JOIN r ON (r.key=p.key) LEFT JOIN s ON (s.key=p.
key) WHERE q.key=s.key;
```

If the `WHERE` clause always returns false for the `NULL` row produced while using a `LEFT JOIN`, it is changed to a normal join. For the following query, the `WHERE` clause will be false if t2.column1 is NULL:

```
SELECT * FROM t1 LEFT JOIN t2 ON (column1) WHERE t2.column2=5;
```

Therefore, it is preferable to change the query to a normal join as follows:

```
SELECT * FROM t1, t2 WHERE t2.column2=5 AND t1.column1=t2.column1;
```

In the previous query, MYSQL uses the column1 to fetch the data from t2. Hence, the visibility of t2's data depends on t1. However, this query can be processed faster because MySQL can use table t2 before table t1, if doing so would result in a better query plan. To add a hint regarding the table join order, utilize the option `STRAIGHT_JOIN`. However, this option may stop you from using indexes as it deactivates semi-join transformations.

Table Design

As a database designer, it is important to choose the most efficient methods to categorize the schemas, tables, and columns. For example, when tweaking the application code, it is preferable to minimize I/O, place connected items together, and plan beforehand to achieve high performance even as the information volume increases. Using an efficient database design allows to develop high-performing application code, and ensures the database endures evolving and rewritten applications.

Optimizing Data Size

Designing the tables such that they reduce the storage space results in huge performance improvements as the data written to and read from the disk is also reduced. Smaller tables use less memory while their data is actively executed during query processing phase. Also, with smaller tables come smaller indexes thus reducing the processing time.

Session 20

MySQL supports numerous storage engines (table types) and row formats. For every table, different storage and indexing methods can be chosen. However, selecting the appropriate table format based on the application offers a huge performance boost. Following are the techniques that help to achieve better performance and reduce storage space:

- **Table Columns**: Choose the most efficient, preferably the smallest data types. MySQL offers many data types that reduce storage and memory. For example, use MEDIUMINT instead of INT as the former uses 25% less space. Preferably, define NOT NULL columns as it increases the performance by promoting better usage of indexes and eliminating the overhead for checking if the value is NULL. However, sometimes using NULL is unavoidable. In such cases, ensure not to use the default setting which accepts NULL values in every column.

- **Row Format**: In MySQL 5.7.8 and earlier versions, InnoDB creates tables using COMPACT format by default. However, as of MySQL 5.7.9, the default is DYNAMIC and the default row format can be modified using the innodb_default_row_format configuration option.

  To change the row format to anything other than `DYNAMIC`, configure innodb_default_row_format or use the `ROW_FORMAT` option in a `CREATE TABLE` or `ALTER TABLE` statement. Using COMPACT format reduces the row storage space by about 20% thus allowing the CPU to be utilized for other operations. If the amount of work is of general type which is restricted by cache hit rates and disk speed, it will likely be faster. If it is of rare type, then it is restricted by CPU speed, and might be slower.

- **Indexes**: Keeping the primary index as short as possible ensures the rows are identified easily and efficiently. In InnoDB tables, the primary key columns are replicated in each secondary index entry, thus keeping it small reduces significant space if there are many secondary indexes. Ensure the created indexes enhance the query performance.

  Though indexes are handy for retrieving data, they cutback the insert and update operations. If a table is retrieved typically by searching on a combination of columns, instead of creating individual index for every column, create a single composite index. The initial part of the index must include the column used frequently.

- **Joins**: For a table scanned frequently, in some instances, it is recommended to break it into two. This holds true if the table is using dynamic-format and a smaller table with static format can be employed to retrieve the required rows. Define columns with identical data in different tables with identical data types, to increase the performance of joins depending on the corresponding columns. Choose clear and short column names as they can be used across different tables thus simplifying join queries. For example, in an employee table, create a column name of name instead of `employee _ name`. To make your names transferrable between other SQL servers, keep the length shorter than 18 characters.

- **Normalization**: Avoid redundant data (third normal form). Allocate unique IDs to lengthy values such as names and addresses and when the values need to be repeated, use the IDs. These IDs can be used across smaller tables and in the join clause to combine tables. In cases where achieving speed is more vital irrespective of the storage space and the maintenance costs, the normalization rules can be relaxed to replicate the data or create summary tables to increase the speed.

Concepts

Using temporary tables for Optimization

In some scenarios, the server generates temporary tables internally while executing queries and the user has no control over this.

Following are the scenarios when the temporary tables are created:

- Assessing `UNION` statements.
- Assessing views that include the `TEMPTABLE` algorithm, `UNION`, or aggregation.
- Assessing statements that use `ORDER BY` clause and a different `GROUP BY` clause, or when the `ORDER BY` or `GROUP BY` includes columns from tables which is not the first table in the join queue.
- Assessing `DISTINCT` used with `ORDER BY`.
- Assessing queries that include the option `SQL _ SMALL _ RESULT`, unless it also includes elements that need on-disk storage.
- Assessing `UPDATE` statements defined on multiple tables.
- Assessing `GROUP _ CONCAT()` or `COUNT(DISTINCT)` expressions.

Use the `EXPLAIN` statement to identify if a statement needs a temporary table. If `USING TEMPORARY` is mentioned in the Extra column, then a temporary table is required.

Storage Engines Used for Temporary Tables

An internal temporary table can be stored in following two ways:

- It is saved in memory and processed by the MEMORY storage engine
- It is saved on disk and processed by the MyISAM storage engine

An internal temporary table, which is an in-memory table, is changed to an on-disk table when the size is too big. The maximum allowable size for in-memory tables is governed by the system variables `tmp_table_size` and `max_heap_table_size` based on the one containing the smallest value. This does not apply to MEMORY tables generated explicitly using CREATE TABLE. For these tables, the maximum size is determined only by `max_heap_table_size` and there is no provision for converting to on-disk format.

Following are the conditions that prevent the usage of in-memory temporary tables and instead the on-disk table is used:

- Using a BLOB or TEXT column in the table.
- Using any string column in a GROUP BY or DISTINCT clause, which exceeds 512 bytes.
- Using any string column with the length exceeding 512 (bytes for binary strings and characters for non-binary strings) in the SELECT list, if UNION or UNION ALL is utilized.
- Using BLOB for some columns along with SHOW COLUMNS and DESCRIBE statements.

When the server creates an internal temporary table (either in memory or on-disk), it increments the `Created_tmp_tables` status variable. If the server creates the table on-disk (either initially or by converting an in-memory table) it increments the `Created_tmp_disk_tables` status variable.

Temporary Table Storage Format

The MEMORY storage engine controls the in-memory temporary tables following the fixed-length row format. VARCHAR and VARBINARY column values are padded to the maximum column length, in turn saving them as CHAR and BINARY columns.

The MyISAM storage engine controls the on-disk temporary tables following the dynamic-width row format. Columns use only the needed space, thus reducing disk I/O, storage, and execution time as compared to on-disk tables that have fixed length. An exception here is that the on-disk temporary tables created for derived tables follow the fixed-length rows format. Version earlier to MySQL 5.5.47 follow the fixed-length rows format for on-disk temporary tables.

Statements that begin with an in-memory temporary table and then change to an on-disk table can gain better performance by starting with an on-disk table. To change disk storage of internal temporary tables use the `big_tables` system variable.

20.3 Scaling MySQL

Scalability is the capability of a system, network, or process to manage a growing amount of work, or its ability to enlarge so as to adapt to that growth.

Scalability is a common problem faced by electronics systems, databases, routers, and networking. A system is termed scalable if it enhances its performance after adding a hardware, by a value, which is proportional to the capacity introduced.

The two types of scaling are horizontal scaling and vertical scaling. Let us look into each of them in detail.

Horizontal scaling (or scale out/in) involves adding more nodes or removal of nodes from a system. For example, scaling out from one Web server system to four Web server systems. Due to a fall in the prices of computers, applications such as seismic analysis and biotechnology are using low-cost commodity systems for tasks that once needed supercomputers to be used. System designers can now add dozens of computers to a configuration to increase the computing power, which is beyond that of computers using a single traditional processor. The introduction of high-performance Gigabit Ethernet, InfiniBand and Myrinet caused a further increase in performance. Such advancements crave for software that effectively controls and supervises multiple nodes and hardware. Size scalability can be defined as the maximum number of processors that a system can handle.

Vertical scaling (or scale up/down) involves adding more resources to or removal of resources from a single node in a system. This generally requires adding CPUs or memory to a single computer. Vertical scaling of existing systems promotes the usage of virtualization technology effectively, as it offers additional resources to be shared among the operating system and the application modules. Using such shared resources can also be termed as scaling up. For example, increasing the count of Apache daemon processes on a system. Application scalability can be defined as the enhanced performance of running applications on a scaled-up version of the system.

The two models require a number of compromises. Following are the issues that arise as result of using larger numbers of computers:

- Increased management complexity
- Complex programming model
- Throughput and latency between nodes
- Some applications do not get adjusted to a distributed computing model.

Earlier, the difference in cost between the two models made scaling to be used by only those applications that comply to its paradigm. However, with recent advancements in virtualization technology, using a new virtual system is considered much lesser than purchasing and connecting a real one. Using an available spare system is always cheaper than purchasing, installing, and configuring a new one, irrespective of the model.

20.3.1 Database Scalability

Numerous methods allow databases to increase in size along with bolstering an ever-increasing rate of transactions per second. One such method supported by most of the DBMS products is the dividing of huge tables depending on the ranges of values in a key field. In this way, the database can be scaled out across a group of different database servers. With the introduction of 64-bit microprocessors, multi-core CPUs, and SMP multiprocessors, DBMS vendors are using multi-threaded applications that significantly increase the transaction execution capacity.

Network-Attached Storage (NAS) and Storage Area Networks (SANs) combined with steady LANS and Fibre Channel technology allow larger, more loosely coupled configurations of databases and distributed computing power. The widely supported X/Open XA standard employs a global transaction monitor to coordinate distributed transactions among semi-autonomous XA-compliant database resources. Oracle RAC employs on a 'shared-everything' model that uses high-speed links between servers to attain scalability.

20.4 Hands-On Exercise Query Optimization

You will now see how to perform the following operations:

- Query optimization with simple queries.
- Query optimization with order by and limit.

The step-by-step procedure for these operations is as follows:

1. Open Linux terminal and start MySQL.
2. Create a database named `QUERY _ OPTIMIZATION` and set it as the active database.
3. Create a table USERS by executing the following query. It is a simple table with no primary key or index.

```
CREATE TABLE USERS (
     ID int(11) NOT NULL,
     USER_NAME VARCHAR(255) NOT NULL,
     EMAIL VARCHAR(255) NOT NULL,
     PHONE INT NOT NULL );
```

Figure 20.5 shows the output of the command.

```
mysql> CREATE TABLE USERS(
    -> ID int(11) NOT NULL,
    -> USER_NAME VARCHAR(255) NOT NULL,
    -> EMAIL VARCHAR(255) NOT NULL,
    -> PHONE INT NOT NULL );
Query OK, 0 rows affected (0.03 sec)
```

Figure 20.5: Creating a Table Named USERS

4. To view the structure of the table, use the following query:

```
EXPLAIN USERS;
```

Figure 20.6 shows the output of the command.

```
mysql> EXPLAIN USERS;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   |     | NULL    |       |
| USER_NAME | varchar(255) | NO   |     | NULL    |       |
| EMAIL     | varchar(255) | NO   |     | NULL    |       |
| PHONE     | int(11)      | NO   |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Figure 20.6: Structure of USERS Table

5. To create another table `USERS _ WITH _ PRI` with the same structure as `USERS` table, enter the following command. Make the `ID` column as the primary key:

```
CREATE TABLE USER_ORDER(
     ID int(11) NOT NULL,
     USER_ID int(11) NOT NULL,
     PRODUCT_ID int(11) NOT NULL,
     PRIMARY KEY(ID),
    FOREIGN KEY (USER_ID) REFERENCES USERS(ID));
```

Figure 20.7 shows the output of the command.

```
mysql> CREATE TABLE USERS_WITH_PRI(
    -> ID int(11) NOT NULL,
    -> USER_NAME VARCHAR(255) NOT NULL,
    -> EMAIL VARCHAR(255) NOT NULL,
    -> PHONE INT NOT NULL ,
    -> PRIMARY KEY(ID));
Query OK, 0 rows affected (0.01 sec)
```

Figure 20.7: Creating a Table Named USERS_WITH_PRI

6. To view the structure of the table, execute the following query:

```
EXPLAIN USERS_WITH_PRI;
```

Figure 20.8 shows the output of the command.

```
mysql> EXPLAIN USERS_WITH_PRI;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   | PRI | NULL    |       |
| USER_NAME | varchar(255) | NO   |     | NULL    |       |
| EMAIL     | varchar(255) | NO   |     | NULL    |       |
| PHONE     | int(11)      | NO   |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Figure 20.8: Structure of USERS_WITH_PRI Table

7. Insert 9999 records into the tables using stored procedures.

8. Use the following commands to check if data is inserted into both the tables respectively:

```
SELECT count(*) FROM USERS;
SELECT count(*) FROM USERS_WITH_PRI;
```

Figure 20.9 shows the output of the commands.

```
mysql> SELECT count(*) FROM USERS;
+----------+
| count(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.01 sec)

mysql> SELECT count(*) FROM USERS_WITH_PRI;
+----------+
| count(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.00 sec)
```

Figure 20.9: Data Count of the Tables

9. Execute the following queries to obtain all the records whose ID is greater than 2000:

```
SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
SELECT * FROM USERS_WITH_PRI WHERE ID > 2000 ORDER BY USER_NAME desc;
```

10. Run the following query to view the structure of USERS table:

```
EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
```

`EXPLAIN` statement helps you to view how MySQL has evaluated the output for these queries.

Figure 20.10 shows the output of the command. We can see that MySQL has scanned 10013 rows and it has used filesort to fetch the records. This query has not used any key and key_len is also NULL.

```
mysql> EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra                      |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10013 |    33.33 | Using where; Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.10: Structure of USERS Table Ordered by User Name

11. To view the structure of USERS_WITH_PRI table, execute following query:

```
EXPLAIN SELECT * FROM USERS_WITH_PRI WHERE ID > 2000 ORDER BY USER_NAME
desc;
```

Figure 20.11 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS_WITH_PRI WHERE ID > 2000 ORDER BY USER_NAME desc;
+----+-------------+---------------+------------+-------+---------------+---------+---------+------+------+----------+----------------------------+
| id | select_type | table         | partitions | type  | possible_keys | key     | key_len | ref  | rows | filtered | Extra                      |
+----+-------------+---------------+------------+-------+---------------+---------+---------+------+------+----------+----------------------------+
|  1 | SIMPLE      | USERS_WITH_PRI | NULL       | range | PRIMARY       | PRIMARY | 4       | NULL | 5008 |   100.00 | Using where; Using filesort |
+----+-------------+---------------+------------+-------+---------------+---------+---------+------+------+----------+----------------------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.11: Structure of USERS_WITH_PRI Table Ordered by User Name

From the output, you can observe that the query has examined only 5008 rows to execute the query, which is half of the output in figure 20.10. Though the table structure, data, and the query are the same, the output of this query has been optimized. Note that the query here is using the primary key to obtain the records. The primary key acts as an index and allows to fetch the data much faster.

12. To alter the USERS table and add an index to it, use the following query:

```
ALTER TABLE USERS ADD INDEX (ID);
```

13. Execute the same EXPLAIN query used in Step 10 on USERS table.

```
EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
```

Figure 20.12 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra                       |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | ID            | NULL | NULL    | NULL | 10013 |    79.89 | Using where; Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+-----------------------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.12: Structure of USERS Table after Adding an Index

We can note that the output is different from the previous one as we have added an index on the ID column of the table. The same query now uses `ID` (possible_keys) key because of which the filtered criteria is increased from 33.33 to 79.89, thus increasing the performance. For the same query, the filtered value is 100 for `USERS_WITH_PRI` table.

14. To add a primary key to the column ID for USERS table, use the following query:

```
ALTER TABLE USERS ADD INDEX (ID);
```

Now execute the same `EXPLAIN` query used in Step 10:

```
EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
```

Figure 20.13 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS WHERE ID > 2000 ORDER BY USER_NAME desc;
+----+-------------+-------+------------+-------+---------------+---------+---------+------+------+----------+-----------------------------+
| id | select_type | table | partitions | type  | possible_keys | key     | key_len | ref  | rows | filtered | Extra                       |
+----+-------------+-------+------------+-------+---------------+---------+---------+------+------+----------+-----------------------------+
|  1 | SIMPLE      | USERS | NULL       | range | PRIMARY,ID    | PRIMARY | 4       | NULL | 5020 |   100.00 | Using where; Using filesort |
+----+-------------+-------+------------+-------+---------------+---------+---------+------+------+----------+-----------------------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.13: Structure of USERS Table after Adding Primary Key

From the output, we can note that after adding the primary key to the table the query output is optimized and now it is scanning only 5020 rows and the filtered output is 100.

Concepts

Query Optimization using order by and limit

15. Drop the primary key that we have created on the USERS table using the following query:

```
ALTER TABLE USERS DROP PRIMARY KEY;
```

Drop the index using the following query:

```
ALTER TABLE USERS DROP INDEX ID;
```

16. To confirm, check the structure of the USERS table using the following command:

```
EXPLAIN USERS;
```

Figure 20.14 shows the output of the command.

```
mysql> EXPLAIN USERS;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| ID        | int(11)      | NO   |     | NULL    |       |
| USER_NAME | varchar(255) | NO   |     | NULL    |       |
| EMAIL     | varchar(255) | NO   |     | NULL    |       |
| PHONE     | int(11)      | NO   |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Figure 20.14: Structure of the USERS Table after Dropping Primary Key and Index

17. Run the following query to obtain the output:

```
SELECT * FROM USERS order by ID limit 5;
```

Figure 20.15 shows the output of the command.

```
mysql> SELECT * FROM USERS order by ID limit 5;
+----+------------+---------------------+-------+
| ID | USER_NAME  | EMAIL               | PHONE |
+----+------------+---------------------+-------+
|  1 | firstName1 | lastname1@gmail.com |   101 |
|  2 | firstName2 | lastname2@gmail.com |   102 |
|  3 | firstName3 | lastname3@gmail.com |   103 |
|  4 | firstName4 | lastname4@gmail.com |   104 |
|  5 | firstName5 | lastname5@gmail.com |   105 |
+----+------------+---------------------+-------+
5 rows in set (0.00 sec)
```

Figure 20.15: Output of SELECT

18. To view the structure of the table, execute the following command:

```
EXPLAIN SELECT * FROM USERS order by ID limit 5;
```

Figure 20.16 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS order by ID limit 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.16: Output of EXPLAIN Ordered by ID

19. To add an index to the ID column, run the following query:

```
ALTER TABLE USERS ADD INDEX (ID);
```

20. To order by the USER ID column and to check the output, use the following query:

```
EXPLAIN SELECT * FROM USERS order by ID limit 5;
```

Figure 20.17 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS order by ID limit 5;
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------+
| id | select_type | table | partitions | type  | possible_keys | key  | key_len | ref  | rows | filtered | Extra |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------+
|  1 | SIMPLE      | USERS | NULL       | index | NULL          | ID   | 4       | NULL |    5 |   100.00 | NULL  |
+----+-------------+-------+------------+-------+---------------+------+---------+------+------+----------+-------+
```

Figure 20.17: Ordering by USER ID Column

After we index the column ID, the output of rows obtained are reduced from 10027 to five rows. This is because, by default, the index which we have created on ID is already stored in incremental order. Therefore, MySQL uses the index, sorts the keys, and then fetches the data. So, it does not examine the rows, that is, the actual data. It simply uses indexes for sorting.

21. To order by USER_NAME column and check the EXPLAIN output, use the following query:

```
EXPLAIN SELECT * FROM USERS ORDER BY USER_NAME DESC LIMIT 5;
```

Figure 20.18 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM USERS order by user_name desc  limit 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.18: Ordering by USER_NAME Column

From the output we can note that the query is not using index for sorting due to which MySQL is again examining 10027 rows for the output.

22. To order the same query by User ID and user_name, use the following query:

```
EXPLAIN SELECT * FROM USERS ORDER BY ID, USER_NAME LIMIT 5;
```

Figure 20.19 shows the output of the command.

```
mysql> EXPLAIN  SELECT * FROM USERS ORDER BY ID,USER_NAME LIMIT 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.19: Ordering by User ID and USER_NAME Columns

Note that the output is not modified by this ordering. MySQL does not use `ID` index because sorting on `ID` is different from sorting on ID and `USER_NAME`.

23. To add an index on ID and USER_NAME, use the following query:

```
ALTER TABLE USERS ADD INDEX (ID, USER_NAME);
```

24. To view the structure of the table, execute the following command:

```
EXPLAIN SELECT *FROM USERS ORDER BY ID, USER_NAME LIMIT 5;
```

Figure 20.20 shows the output of the command.

```
mysql> EXPLAIN  SELECT * FROM USERS ORDER BY USER_NAME,ID LIMIT 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.20: Output of EXPLAIN

25. To change the order of `USER _ NAME` in descending order execute the following query:

```
EXPLAIN SELECT * FROM USERS ORDER BY ID, USER_NAME DESC LIMIT 5;
```

Figure 20.21 shows the output of the command.

```
mysql> EXPLAIN  SELECT * FROM USERS ORDER BY ID,USER_NAME DESC LIMIT 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.21: Ordering of USER_NAME in Descending Order

From the output, we can note that the number of rows examined are again increased to 10027, that is, optimization did not work. Therefore, even if we change the order, the index which we have created will not work.

26. To reverse the order of columns, execute the following query. This time we will sort by `USER _ NAME`, ID instead of ID, `USER _ NAME`.

```
EXPLAIN SELECT * FROM USERS ORDER BY USER_NAME, ID LIMIT 5;
```

Figure 20.22 shows the output of the command.

```
mysql> EXPLAIN  SELECT * FROM USERS ORDER BY USER_NAME,ID LIMIT 5;
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
| id | select_type | table | partitions | type | possible_keys | key  | key_len | ref  | rows  | filtered | Extra          |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
|  1 | SIMPLE      | USERS | NULL       | ALL  | NULL          | NULL | NULL    | NULL | 10027 |   100.00 | Using filesort |
+----+-------------+-------+------------+------+---------------+------+---------+------+-------+----------+----------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 20.22: Reversed Order of Columns

From the output, we can note that output is not optimized because the index that we have created is for `ID` and `USER_NAME` but we are now sorting on `USER_NAME` and `ID`. So the index which we have created will not be used. In order to optimize the fetch query, the order of index creation should be considered accordingly.

Summary

- Performance is defined as a measure of time that is required to complete a task. In simpler words, it is the response time. It is measurable in terms of tasks and time, and not by the resources used.
- Indexes are helpful to achieve good performance, and play a critical role when the data used is large. Index optimization is the most effective technique to increase query performance.
- The Index Merge methodology helps to obtain rows with numerous range scans and to merge their output into a single file. This type of merge can generate unions, intersections, or unions-of-intersections of the result.
- The query cache saves the data of a SELECT statement along with the result sent to the client. Later, if a matching statement is issued, instead of processing the statement again, the server simply obtains the results from the query cache.
- MySQL Query Analyzer is capable of identifying the SQL code that is causing a slowdown. Using the analyzer helps developers to improve the code during the development stage and to periodically monitor and fine-tune queries.
- Choosing appropriate table columns, row format, indexes, joins, and normalization techniques help to achieve better performance and reduce storage space.
- Scalability is the capability of a system, network, or process to manage a growing amount of work, or its ability to enlarge so as to adapt to that growth.

Check Your Progress

1. Which of the following options can optimize a MySQL query?

   a. Ensure a table has a primary key

   b. Index the field which is used more frequently for searching

   c. The order of condition does not matter

   d. WHERE condition has an indexed column in SELECT statement

2. Match the following.

	Data		Description
a.	UNION	1.	SELECT is part of a sub query within a FROM clause
b.	PRIMARY	2.	SELECT is the second or later statement of a UNION
c.	SUBQUERY	3.	SELECT is in the outermost query in a JOIN
d.	DERIVED	4.	Second or later SELECT of a UNION is dependent on an outer query
e.	DEPENDENT UNION	5.	First SELECT in a sub query

3. Which of the following are advantages of using indexes?

   a. Generates a distinct ordered file that holds only the field(s) that need to be sorted.

   b. Requires more space than a data file.

   c. Increases the performance of queries such as INSERT, UPDATE, and DELETE.

   d. Helps to retrieve rows easily when the data used is large.

Check Your Progress

4. In which of the following instances, will the query cache not be used?

   a. Queries that are a sub-query of another query

   b. In transactions with InnoDB tables

   c. Queries processed within a stored function, trigger, or event

   d. If TEMPORARY tables are referred

5. Match the following.

Data		Description	
a.	Query Analyzer Table	1.	Recognizes queries with slow response times
b.	Response Statistics	2.	Provides key metrics such as Execution Time, Executions, Rows, and Kilobytes at a given time for a specific query
c.	Query Response Time Index	3.	Maintains details of all the executing queries
d.	Explain Query	4.	Provides information such as Row Statistics, Time Span, and First Seen
e.	Graphs	5.	Maintains the execution plan used for the query

Onlinevarsity

INDUSTRY BEST PRACTICES

Session 21

Replication, Data Backup, Partitioning, and Storage Systems

Concepts

Objectives

At the end of this session, the student will be able to:

- *Explain replication in MySQL.*
- *Explain how to manage data using replication.*
- *Describe concepts of partitioning in MySQL.*
- *Describe concepts of Storage Systems and Management in MySQL.*

21.1 Replication Basics

Replication refers to the process of copying data from a MySQL server to single or multiple database servers. The server from which the data is copied is called the master server, while the destination server or servers are called slaves. By default, this process is asynchronous, which means the slaves need not remain connected to the master for obtaining updates. According to the configuration, a user can replicate only a few selected tables in a database, some databases on the master database server, or all databases. Following are the benefits of replication in MySQL:

- **Scaled Environment**: By delegating the load to several slaves for boosting performance. Herein, all update and write operations must occur on the master server, while read operations can occur on any one or more slaves. Such a framework tends to boost the performance of writes due to one dedicated server, while noticeably raising the read speed on slaves despite their increasing number.
- **Ensured Security**: Replication helps in running a backup schedule on the slave without disrupting the corresponding copy of master data.
- **Independent Analytics**: Replication allows generating real-time data on the master, while analyzing data on the slave without corrupting or disrupting the master's performance.
- **Data Distribution across a Long Distance**: Replication helps to generate a local data copy that a server at a remote distance can use, without seeking lasting connection with the master.

Several variables and options tend to control replication. You can consider replication for resolving a variety of problems, including those related to performance, data loss due to no backups, and frequent system failures.

Concepts

21.1.1 Methods of Replication

MySQL 5.7 employs different techniques or methods to replicate master data. First, the traditional method depends on log files as well as positions in them, which should be synchronized between the two servers (master and slave). It is based on binary log's replicating events saved on the master.

Second, the newer method is transactional due to which it does not demand for log files. This significantly simplifies several common tasks while replicating the intended data. The newer method uses Global Transaction Identifiers (GTIDs) and ensures consistency between the two servers, provided that the committed transactions on the master have also occurred on the slave.

You can even perform master-slave, master-master replication, and multi-source replications wherein there are two or more master servers and one slave.

While several solutions are available for replicating data between servers, the best one depends on the kind of engine in use and availability of data.

21.1.2 Formats of Replication

Two types of format are employed in MySQL 5.7 for replication, which are Statement Based Replication (SBR) and Row Based Replication (RBR). SBR replicates the whole SQL statements, while RBR replicates only the rows that are modified. You can even employ Mixed Based Replication (MBR).

21.1.3 Types of Synchronization

In MySQL, replication uses a few types of synchronization namely, asynchronous and semisynchronous. Also known as the original synchronization type, built-in asynchronous technique is a one-way replication, wherein there is a single server as the master and single or multiple other servers are slaves. This contradicts to the synchronous replication, which is a feature of Cluster in MySQL 5.7.

In semisynchronous replication, a committed transaction on the master stops prior to returning to the session that carried out the transaction. This happens until a slave accepts that it has obtained and logged the transaction events. There is also support for delayed replication in MySQL 5.7, wherein a slave server purposely stays behind the master by a stated amount of time.

21.2 Managing the Replication Process

Replication relies on the ability of the master server to track all changes across its databases by logging the same in its binary log. This log acts as a record of all events that changed the content or database structure, since the time when server was started. Usually, SELECT statements are not tracked, as they do not modify anything in the database.

Each slave connecting to a master asks for a binary log copy. This means that it pulls data from the master, instead of the master sending the same to the slave. The slave also runs events from the obtained binary log or repeats the same changes as originally occurred on the master.

Each slave is self-reliant, which means that the replay of changes takes place independently on all slaves connected to the master. Further, as each slave obtains a log copy only by requesting the master, the slave can update and read the database copy at its own pace. It can even start and stop replication but this does affect the ability to update to the latest status of database on slave or master.

21.3 Data Backup and Restore

If any data stored in MySQL databases is important, taking frequent backups of that data is essential for preventing its loss. There are three easy ways to take a backup in any MySQL database, which also facilitates moving that data to another server through restore process. Following are these three ways of backup:

- **From the Command Line**: Works by using the `mysqldump` utility.
- **With Compress**: Works by compressing the `mysqldump` output.
- **Through phpMyAdmin**: Works to back up and restore MySQL databases through phpMyAdmin, which is a free tool designed to administer MySQL on the Internet.

21.3.1 Taking a Backup from the Command Line

You should consider using the `mysqldump` utility for taking a backup if connecting to a Web server via shell access or Telnet is possible. This utility generates an SQL dump document after connecting to the MySQL server. This new document has essential SQL statements for re-generating the database. Following is the syntax of the `mysqldump` utility for taking a backup:

Syntax:

```
$ mysqldump --opt -u [username] -p[password] [databasename] > [dbbck.sql]
```

where,

`opt`: Is the mysqldump option.

`username`: Is the database username.

`password`: Is the password following -p without any space in between the two.

`databasename`: Is the database name.

`dbbck.sql`: Is the filename having statements for recreating the database.

Code Snippet 1 shows the command to backup School database with the username root and without a password to a file `sch_backup.sql`.

Code Snippet 1:

```
$ mysqldump -u root -p School > sch_backup.sql
```

Code Snippet 2 shows the command to backup only two tables namely, `Students` and `Teachers` from the `School` database without a password. Each table name is delimited by space.

Code Snippet 2:

```
$ mysqldump -u root -p School Students Teachers > sch_backup.sql
```

Code Snippet 3 shows the command to backup more than one database by using the `--databases` option without a password. This option is then followed by the databases for backup, each separated by space.

Code Snippet 3:

```
$ mysqldump -u root –p --databases School College University > sch_backup.sql
```

Code Snippet 4 shows the command to backup all databases existing on the server by using the `--all-databases` option without a password. This option informs MySQL to dump all its databases.

Code Snippet 4:

```
$ mysqldump -u root -p --all-databases > adb_backup.sql
```

Following are some useful options of the `mysqldump` command:

- **`--add-drop-table`**: Informs MySQL to insert a `DROP TABLE` statement in the dump, prior to each `CREATE TABLE` statement.
- **`--no-data`**: Informs MySQL to dump only the structure of database and not its contained data.
- **`--add-locks`**: Informs MySQL to add two statements namely, `LOCK TABLES` and `UNLOCK TABLES`, in the dump.

The `mysqldump` command has its own benefits and limitations. The benefits of using it are ease of use and ability to handle table locking problems. On the other hand, the only major limitation is locked tables. The utility lock tables due to which if the table size is big, users can be locked out for a long time.

21.3.2 Taking a Backup Through Compression

If a MySQL database for backup is big, it is wise to compress it. This is done by compressing `mysqldump` output and piping it to gzip, which is a file compressor. Doing so gives the output as gzip file. Following is the syntax for compressing the backup file.

Syntax:

```
$ mysqldump -u [username] -p[password] [databasename] | gzip -9 > [bckfile.
sql.gz]
```

Code Snippet 5 shows the command for extracting the zipped file.

Code Snippet 5:

```
$ gunzip [bckupfile.sql.gz]
```

21.3.3 Restoring a Database

Restoring a database involves recreating it from its backup file saved in `.sql` extension. To recreate a database, it is essential to create a database with an appropriate name on the intended machine.

Following is the syntax for restoring a database:

Syntax:

```
$ mysql -u [username] -p[password] [database_for_restoring] < [bckupfile.sql]
```

Code Snippet 6 shows the command to restore the backup file to the `School` database without a password.

Code Snippet 6:

```
$ mysql -u root -p School < sch_backup.sql
```

You can also restore a compressed file for backup. This is done with the help of `gunzip` and `mysql` commands. Following is the syntax for restoring a compressed backup file:

Syntax:

```
gunzip < [bckupfile.sql.gz] | mysql -u [uname] -p[pass] [dbname]
```

You can even restore an already existing database by using the `mysqlimport` command. Following is the syntax for the same:

Syntax:

```
mysqlimport -u [uname] -p[pass] [dbname] [bckupfile.sql]
```

21.4 Hands-On Exercise: Backing up a Database

Consider the database `QUERY_OPTIMIZATION` that had been created earlier. Let us take a backup of this database.

1. Open the Linux terminal and run MySQL.

2. To activate the database, enter the following command on the command prompt:

```
USE QUERY_OPTIMIZATION;
```

Figure 21.1 shows the output of the command.

```
mysql> USE QUERY_OPTIMIZATION;
Database changed
mysql>
```

Figure 21.1: Activating an Existing Database for Backup

3. To view the total number of records in `USERS` table, enter the following command on the command prompt:

```
SELECT COUNT(*) FROM USERS;
```

Figure 21.2 shows the output of the command.

```
mysql> SELECT COUNT(*) FROM USERS;
+----------+
| COUNT(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.02 sec)
```

Figure 21.2: Viewing Total Records in the Table

4. Navigate to the **bin** folder of MySQL installation path and enter the following command on the command prompt:

```
mysqldump -u root -p QUERY_OPTIMIZATION > D:/Users/MySQL/users.sql
```

This will create users.sql in the specified path. You can change the folder path to a path of your choice before executing this command.

5. Upon executing this command, MySQL will prompt you for the password. Provide the password.

Following is the output that is generated into `users.sql` after entering the right password:

```
-- MySQL dump 10.13 Distrib 5.7.11, for osx10.9 (x86_64)
--
-- Host: localhost Database: QUERY_OPTIMIZATION
-- ------------------------------------------------------
-- Server version5.7.11
```

```
/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS
 */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0
*/;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
```

```
--
-- Table structure for table "USERS"
--
```

```
DROP TABLE IF EXISTS “USERS”;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE “USERS” (
 “ID” int(11) NOT NULL,
 “USER_NAME” varchar(255) NOT NULL,
 “EMAIL” varchar(255) NOT NULL,
 “PHONE” int(11) NOT NULL,
 KEY “ID” (“ID”),
 KEY “ID_2” (“ID”,”USER_NAME”)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
/*!40101 SET character_set_client = @saved_cs_client */;
```

```
--
-- Dumping data for table “USERS”
--
```

```
LOCK TABLES “USERS” WRITE;
/*!40000 ALTER TABLE “USERS” DISABLE KEYS */;
INSERT INTO “USERS” VALUES
(1,’firstName1’,’lastname1@gmail.com’,101)--continue
/*!40000 ALTER TABLE “USERS” ENABLE KEYS */;
UNLOCK TABLES;
```

```
--
-- Table structure for table “USERS_WITH_PRI”
--
```

```
DROP TABLE IF EXISTS “USERS_WITH_PRI”;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE “USERS_WITH_PRI” (
 “ID” int(11) NOT NULL,
 “USER_NAME” varchar(255) NOT NULL,
 “EMAIL” varchar(255) NOT NULL,
 “PHONE” int(11) NOT NULL,
 PRIMARY KEY (“ID”)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
/*!40101 SET character_set_client = @saved_cs_client */;
```

```
--
-- Dumping data for table "USERS_WITH_PRI"
--
```

```
LOCK TABLES "USERS_WITH_PRI" WRITE;
/*!40000 ALTER TABLE "USERS_WITH_PRI" DISABLE KEYS */;
INSERT INTO "USERS_WITH_PRI" VALUES
(1,'firstName1','lastname1@gmail.com',101)-- continue
/*!40000 ALTER TABLE "USERS_WITH_PRI" ENABLE KEYS */;
UNLOCK TABLES;
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;
```

```
/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;
/*!40014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;
/*!40014 SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS */;
/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
/*!40111 SET SQL_NOTES=@OLD_SQL_NOTES */;
-- Dump completed on 2016-05-02 18:53:45
```

```
-- Dump completed on 2016-05-02 18:53:45
```

Restoring a Database

Let us now create a new database `MYSQL_BACKUP`, to which the `QUERY_OPTIMIZATION` database will be restored.

1. Navigate to the `mysqld` connection terminal and enter the following command on the command prompt:

```
CREATE DATABASE MYSQL_BACKUP;
```

Figure 21.3 shows the output of the command.

```
mysql> CREATE DATABASE MYSQL_BACKUP;
Query OK, 1 row affected (0.01 sec)
```

Figure 21.3: Creating a Database for Restoring a Backed Up Database

2. To restore the database, enter the following command on the command prompt:

```
mysql -u root -p MYSQL_BACKUP < D:/Users/MySQL/users.sql
```

This will perform the restore action.

3. To open the terminal and activate the `MYSQL _ BACKUP` database, enter the following command on the command prompt:

```
USE MYSQL_BACKUP;
```

Figure 21.4 shows the output of the command.

```
mysql> USE MYSQL_BACKUP;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

Figure 21.4: Activating the Restored Database

4. To list the number of tables in restored database, enter the following command on the command prompt:

```
SHOW TABLES;
```

Figure 21.5 shows the output of the command.

```
mysql> SHOW TABLES;
+------------------------+
| Tables_in_mysql_backup |
+------------------------+
| USERS                  |
| USERS_WITH_PRI         |
+------------------------+
2 rows in set (0.00 sec)
```

Figure 21.5: Viewing Tables of the Restored Database

Earlier, both these tables had been created in the `QUERY_OPTIMIZATION` database and are now restored to a new database. This is how you can perform backup and restore and also import data from one database to another.

5. To view the total number of records in the restored `USERS` table, enter the following command on the command prompt:

```
SELECT COUNT(*) FROM USERS;
```

Figure 21.6 shows the output of the command.

```
mysql> SELECT COUNT(*) FROM USERS;
+----------+
| COUNT(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.00 sec)
```

Figure 21.6: Viewing Total Records of the Restored Table

The total number of rows is equal to whatever the count was at the time of taking the backup. This shows that backup and restore procedures completed successfully.

21.5 Overview of MySQL Partitioning

Partitioning refers to a database design technique, which aims to alleviate the cost of storing huge amount of data, boost performance, and simplify maintenance. As the name suggests, partitioning divides tables, index-organized tables, and indexes into smaller pieces. Partitioning allows distributing parts of a table across a file system, as per the rules set up by a user. Different portions of a table are available from unique locations as separate tables.

Doing so allows queries to access only the desired part of records. This ensures lesser data to scan and faster results. However, it is possible to accomplish partitioning without splitting tables, wherein tables are physically stored on separate disk drives. Following are two primary forms of partitioning in MySQL:

- **Vertical Partitioning**: Splits a table into several tables, each having lesser columns than the original one. This allows any query to scan less data to boost its performance. For instance, a table with several least-addressed BLOB or very wide text columns is split into two tables. While one of these tables contains the least referenced columns, the other one has the most referenced ones.

- **Horizontal Partitioning**: Split table rows into several partitions by employing some logic. All columns exist in each partition, due to which all partitions can be handled either individually or jointly. For instance, a table having the sales revenues for throughout the year is split horizontally into twelve unique partitions, each showing revenues for a unique month.

21.5.1 Partitioning in MySQL

MySQL does not employ vertical partitioning but only basic partitioning. A user-defined rule for dividing the table data is termed as partitioning function. This function can be a hashing function, modulus, or a match for a set of ranges or lists in MySQL. The function is chosen as per the type of partitioning type that the user states, and accepts the value of a user-stated expression as its parameter. This expression can be a set of single or multiple column values, a column value, and a function defined for single or multiple column values. For example, a developer can have a table with 50 partitions done by its primary key.

Following are the benefits of partitioning:

- More control over how to manage data within the database. For instance, a developer can drop a few partitions with least useful data in a partitioned table instead of deleting each row. Similarly, a developer can add new data by creating more new partitions using the `ALTER TABLE` statement.

- More data storage a single file system or disk partition.

- Optimized query search, as search through a `WHERE` clause happens only on the intended partitions, finally leaving behind the remaining or non-matching partitions. This ability to leave behind the unrelated partitions is called partition pruning. A developer can also re-arrange the data by altering an already created partitioned table so that frequently triggered queries are optimized. Moreover, a partition also supports statements such as `INSERT`, `DELETE`, `UPDATE`, `REPLACE`, and `LOAD DATA`.

- Freedom to select partition explicitly in a query for speedy results. For instance, the query can be `SELECT * FROM` Students `PARTITION (p0,p1) WHERE ID < 10` for fetching only those rows in `p0` and `p1` partitions, which fulfill the `WHERE` condition.

In MySQL, a developer first needs to check whether the current server instance supports partitioning or not. To do so, the `SHOW PLUGINS` statement is used.

Figure 21.7 shows the output of this statement, wherein the status of partition is Active.

```
mysql> SHOW PLUGINS;
+----------------------------+----------+--------------------+-----------------+---------+
| Name                       | Status   | Type               | Library         | License |
+----------------------------+----------+--------------------+-----------------+---------+
| keyring_file               | ACTIVE   | KEYRING            | keyring_file.so | GPL     |
| binlog                     | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| mysql_native_password      | ACTIVE   | AUTHENTICATION     | NULL            | GPL     |
| sha256_password            | ACTIVE   | AUTHENTICATION     | NULL            | GPL     |
| CSV                        | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| MEMORY                     | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| InnoDB                     | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| INNODB_TRX                 | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_LOCKS               | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_LOCK_WAITS          | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMP                 | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMP_RESET           | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMPMEM              | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMPMEM_RESET        | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMP_PER_INDEX       | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_CMP_PER_INDEX_RESET | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_BUFFER_PAGE         | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_BUFFER_PAGE_LRU     | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_BUFFER_POOL_STATS   | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_TEMP_TABLE_INFO     | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_METRICS             | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_DEFAULT_STOPWORD | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_DELETED          | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_BEING_DELETED    | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_CONFIG           | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_INDEX_CACHE      | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_FT_INDEX_TABLE      | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_TABLES          | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_TABLESTATS      | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_INDEXES         | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_COLUMNS         | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_FIELDS          | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_FOREIGN         | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_FOREIGN_COLS    | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_TABLESPACES     | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_DATAFILES       | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| INNODB_SYS_VIRTUAL         | ACTIVE   | INFORMATION SCHEMA | NULL            | GPL     |
| MyISAM                     | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| MRG_MYISAM                 | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| PERFORMANCE_SCHEMA         | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| ARCHIVE                    | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| BLACKHOLE                  | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| FEDERATED                  | DISABLED | STORAGE ENGINE     | NULL            | GPL     |
| partition                  | ACTIVE   | STORAGE ENGINE     | NULL            | GPL     |
| ngram                      | ACTIVE   | FTPARSER           | NULL            | GPL     |
+----------------------------+----------+--------------------+-----------------+---------+
45 rows in set (0.01 sec)
```

Partition status is active

Figure 21.7: Checking Whether Partitioning in MySQL is Supported or Not

You can disable partitioning support by using the `--skip-partition` option while starting the server. The value of this option is `DISABLED`.

Concepts

21.5.2 Partitioning a Table in MySQL

In MySQL, a developer can partition a table with the help of `CREATE TABLE` or `ALTER TABLE` command. Following is the syntax for partitioning a table:

Syntax:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] table_name
(definition,...)
[options_for_table]
[options_for_partition]
```

where,

`options_for_ partition` can include:

```
PARTITION BY
 { [LINEAR] HASH(exp)
 | [LINEAR] KEY(col_list)
 | RANGE(exp)
| LIST(exp) }
 [PARTITIONS num]
 [SUBPARTITION BY
 { [LINEAR] HASH(exp)
 | [LINEAR] KEY(col_list) }
 [SUBPARTITIONS num]
 ]
 [(partition_definition [, partition_definition] ...)]
```

`partition_definition` in turn is as follows:

```
PARTITION name_of_partition
 [VALUES
 {LESS THAN {(exp) | MAXVALUE}
 |
 IN (val_list)}]
 [[STORAGE] ENGINE [=] name_of_engine]
 [COMMENT [=] 'text_for_comment']
 [DATA DIRECTORY [=] 'data_dir']
 [INDEX DIRECTORY [=] 'index_dir']
 [MAX_ROWS [=] max_row_number]
 [MIN_ROWS [=] min_row_number]
 [TABLESPACE [=] name_of_tablespace]
 [NODEGROUP [=] id_of_node_group]
 [(subpartition_definition [, subpartition_definition] ...)]
```

where,

`subpartition_definition` in turn can be as follows:

```
SUBPARTITION logical_name
 [[STORAGE] ENGINE [=] name_of_engine]
 [COMMENT [=] 'text_for_comment']
 [DATA DIRECTORY [=] 'data_dir']
 [INDEX DIRECTORY [=] 'index_dir']
 [MAX_ROWS [=] max_row_number]
 [MIN_ROWS [=] min_row_number]
 [TABLESPACE [=] name_of_tablespace]
 [NODEGROUP [=] id_of_node_group]
```

A partition name has the same rules as other MySQL identifiers such as stored procedures, tables, and constraints. It is not case-sensitive.

Code Snippet 7 creates a partitioned table holding employees' data for an organization.

Code Snippet 7:

```
CREATE TABLE ITEmployees (
 id INT,
 firstname VARCHAR(30),
 lastname VARCHAR(30),
 birth DATE
)
PARTITION BY RANGE( YEAR(birth) )
(
 PARTITION ptn0 VALUES LESS THAN (1960),
 PARTITION ptn1 VALUES LESS THAN (1970),
 PARTITION ptn2 VALUES LESS THAN (1980),
 PARTITION ptn3 VALUES LESS THAN (1990)
);
```

The partition `ptn0` will have records of those employees whose are born before 1960, while the partition `ptn1` will store records of employees born between 1960 and 1969. Similarly, partition `ptn2` will store records of employees born between 1970 and 1979, while partition `ptn3` will store records of employees born between 1980 and 1989.

Code Snippet 8 modifies the `employees` table to hold the details of new members born during 1990 to 1999.

Code Snippet 8:

```
ALTER TABLE ITEmployees ADD PARTITION (PARTITION ptn4  VALUES   LESS   THAN
(2000));
```

21.6 Partitioning Types

MySQL supports six types of partitioning. Following are these partitioning types:

- RANGE
- LIST
- COLUMNS
- HASH
- KEY
- SUB

21.6.1 RANGE Partitioning

In MySQL, RANGE partitioning enables a user to split a table as per different ranges of values of a particular column. The ranges need to be contiguous and not coinciding. A developer needs to define the intended ranges using the `VALUES LESS THAN` operator within the `PARTITION BY RANGE` clause. Following is the syntax of this clause:

Syntax:

```
PARTITION BY RANGE (exp)
```

where,

`exp`: Refers to a column name and is also called a partition key.

Code Snippet 9 shows how to partition the `emp` table with three columns namely, `id`, `firstname`, and `lastname`. It splits the table into four partitions, each partitioned by range of `id` values.

Code Snippet 9:

```
CREATE TABLE emp(
 id int(11) NOT NULL,
 firstname varchar(255) NOT NULL,
 lastname varchar(255) NOT NULL)
 PARTITION BY RANGE (id) (
 PARTITION p0 VALUES LESS THAN (1000),
 PARTITION p1 VALUES LESS THAN (3000),
 PARTITION p2 VALUES LESS THAN (10000),
 PARTITION p3 VALUES LESS THAN (15000)
 );
```

The partition `p0` holds records with id values ranging from 1 to 999, `p1` with id values ranging from 1000 to 2999, `p2` from 3000 to 9999, and p3 from 10000 to 14999.

Now, let us assume that 9999 records are added into this newly created table, with the `id` value for each record incremented by 1. Figure 21.8 shows the minimum and maximum ID values and total records inserted in the table.

```
mysql> select min(id), max(id) from emp;
+---------+---------+
| min(id) | max(id) |
+---------+---------+
|       1 |    9999 |
+---------+---------+
1 row in set (0.01 sec)

mysql> select count(*) from emp;
+----------+
| count(*) |
+----------+
|     9999 |
+----------+
1 row in set (0.00 sec)
```

Figure 21.8: Minimum and Maximum ID Values

Figure 21.9 shows the partition status of `emp` table fetched using the `SELECT` query on `INFORMATION_SCHEMA.PARTITIONS`.

```
mysql> SELECT PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.PARTITIONS
WHERE TABLE_NAME = 'EMP';
+----------------+------------+
| PARTITION_NAME | TABLE_ROWS |
+----------------+------------+
| p0             |        999 |
| p1             |       2000 |
| p2             |       7000 |
| p3             |          0 |
+----------------+------------+
4 rows in set (0.00 sec)
```

Figure 21.9: Status of All Partitions of a Table

It can be seen that partition p3 has no rows because the `emp` table has no record with id values above 9999. Similarly, `p0` has 999 records, `p1` has 2000 records (1000 to 2999), and `p3` has 7000 records (3000 to 9999). It is also possible to truncate a single or multiple partitions if the data of in any of them is now not needed.

Figure 21.10 shows how to drop partition `p2` and delete all rows from it by using `TRUNCATE PARTITION` in `ALTER TABLE` statement.

```
mysql> ALTER TABLE EMP TRUNCATE PARTITION P2;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT COUNT(*) FROM EMP;
+----------+
| COUNT(*) |
+----------+
|     2999 |
+----------+
1 row in set (0.00 sec)
```

Figure 21.10: Dropping a Partition

It can be seen that total number of records is now reduced to 2999. We have all four partitions but the data from `P2` is completely deleted.

Figure 21.11 shows the partition status of the emp table after dropping the partition `p2`.

```
mysql> SELECT PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.PARTITIONS WHERE
 TABLE_NAME = 'EMP';
+----------------+------------+
| PARTITION_NAME | TABLE_ROWS |
+----------------+------------+
| p0             |        999 |
| p1             |       2000 |
| p2             |          0 |
| p3             |          0 |
+----------------+------------+
4 rows in set (0.00 sec)
```

Figure 21.11: Table Status after Dropping a Partition

21.6.2 LIST Partitioning

A list partition splits data in a table as per the pre-defined set of values. Such a partition is created using `PARTITION BY LIST` clause and then defining each partition through `VALUES IN` operator. Following is the syntax of this clause and operator:

Syntax:

```
PARTITION BY LIST (exp)
```

where,

`exp`: Refers to a column name and is also called a partition key.

```
VALUES IN (val_list)
```

where,

`val_list`: Refers to a list of integers separated by a comma.

A developer can match even against NULL while using list partitioning. Let us assume that the `emp_list` table with id column has records of nine employees representing three cities namely, Atlanta, Baltimore, and California. Atlanta (partA) employees have id values as 1, 3, and 5; while Baltimore (partB) employees have id values as 2, 7, and 9 and California (partC) ones have id values as 4, 6, and 8.

Code Snippet 10 shows how to split the employee records in three partitions using LIST partitioning.

Code Snippet 10:

```
CREATE TABLE emp_list(
IDNo int(11) NOT NULL,
firstname varchar(255) NOT NULL,
lastname varchar(255) NOT NULL)
PARTITION BY LIST (IDNo) (
PARTITION partA VALUES IN (1,3,5),
PARTITION partB VALUES IN (2,9,7),
PARTITION partC VALUES IN (4,6,8)
);
```

21.6.3 COLUMNS Partitioning

A developer can split a table by referring to more than one column as partition keys, in `COLUMNS` partitioning. Following are the two types of `COLUMNS` partitioning:

- RANGE COLUMNS
- LIST COLUMNS

Both these types of `COLUMNS` partitioning accept non-integer columns for specifying list members or value ranges. Further, both of them support a column holding values of integer type, date type, or string type. They do not support `FLOAT`, `DECIMAL`, `BLOB`, and `TEXT` types.

Unlike `RANGE` partitioning, `RANGE COLUMNS` partitioning accepts a list of single or multiple column names as partition keys. Following is the syntax of `RANGE COLUMNS` partitioning:

Syntax:

```
CREATE TABLE tbl_name
PARTITIONED BY RANGE COLUMNS(col_list) (
 PARTITION part_name VALUES LESS THAN (val_list)[,
 PARTITION part_name VALUES LESS THAN (val_list)][,
 ...]
)
col_list:
 col_name[, col_name][, ...]
val_list:
 val[, val][, ...]
```

where,

`col_list`: Refers to a list of single or multiple columns.

`val_list`: Refers to a list of values and must be specified for each partition.

In this syntax, the order of partitions needs to be the same in both `col_list` and `val_list`. However, this order of column names in both the list need not be identical to the order of columns in `CREATE TABLE` statement.

Code Snippet 11 partitions a table using `RANGE COLUMNS` partitioning.

Code Snippet 11:

```
CREATE TABLE DATA_COL_RANGE(
 id int(11) NOT NULL,
 numericdata int(11) NOT NULL,
 stringdata varchar(255) NOT NULL,
 comment varchar(255) NOT NULL)
 PARTITION BY RANGE COLUMNS (id, numericdata, stringdata) (
 PARTITION pn0 VALUES LESS THAN (100, 150, 'bbbb'),
 PARTITION pn1 VALUES LESS THAN (150, 250,'yyyy'),
 PARTITION pn2 VALUES LESS THAN (MAXVALUE, MAXVALUE, MAXVALUE)
 );
```

In Code Snippet 11, the table is created by defining four columns, of which the first three are used in splitting the table into four partitions. Each value list defining each partition has three column values in the same order namely, in the `(INT, INT, VARCHAR(255))` form.

21.6.4 HASH Partitioning

A HASH partition splits the table data across the specified number of partitions, according to a column value or expression as per a column value to be hashed. Unlike `LIST` or `RANGE` partitioning wherein a developer explicitly states the partition into which single or multiple column values are to be stored, Hash partitioning performs this implicitly. In simple terms, the developer need not state the partition. The developer only has to specify an expression as per a column value or only a column value along with the number of partitions to be formed. Hash partitioning is implemented using the `PARTITION BY HASH` clause as shown in the following syntax:

Syntax:

```
PARTITION BY HASH (exp)
```

where,

`exp`: Refers to an expression outputting an integer, such as a column name holding integer type values.

The clause is then followed by another clause called `PARTITIONS num`, wherein `num` refers to a positive integer indicating the number of partitions to be made for the specified table. If `num` is unspecified, only partition is created implicitly by default.

Code Snippet 12 creates the emp_hash table and splits it into four partitions by hashing on the id column.

Code Snippet 12:

```
CREATE TABLE emp_hash(
id int(11) NOT NULL,
firstname varchar(255) NOT NULL,
lastname varchar(255) NOT NULL)
)
PARTITION BY Hash (id)
PARTITIONS 4;
```

21.6.5 KEY Partitioning

Regarded as a special type of HASH partition, a KEY partition involves MySQL server that provides its hashing function for partitioning. The server uses its internal function, which is derived from the algorithm similar to `PASSWORD()`. Thus, partitioning by key involves server supplied hash function featuring a user-defined expression.

KEY partitioning is implemented using the `PARTITION BY KEY` clause. Herein, KEY accepts a list of zero or multiple names of columns. This is usually the table's primary key, in case there is such key defined in the table. When no other column is stated for KEY partitioning, the primary key is used.

Code Snippet 13 partitions the `ITemp` table using a primary key as the partitioning key.

Code Snippet 13:

```
CREATE TABLE ITemp (IDNo INT NOT NULL PRIMARY KEY,
firstname VARCHAR(25), lastname VARCHAR(25))
PARTITION BY KEY()
PARTITIONS 2;
```

In case there is no primary key, the unique key is considered as the partitioning key. Code Snippet 14 uses a unique key in the table for splitting it into two partitions.

Code Snippet 14:

```
CREATE TABLE ITempNew (IDNo INT NOT NULL, firstname VARCHAR(25),
lastname VARCHAR(25),
UNIQUE KEY (IDNo))
PARTITION BY KEY()
PARTITIONS 2;
```

21.6.6 SUB Partitioning

SUB partitioning splits further each partition in a partitioned table. The `SUBPARTITION` clauses are used for sub partitioning a partition.

Code Snippet 15 further splits a partition of a table.

Code Snippet 15:

```
CREATE TABLE emp_SUB_PAR(
id int(11) NOT NULL,
firstname varchar(255) NOT NULL,
lastname varchar(255) NOT NULL)
PARTITION BY RANGE (id)
SUBPARTITION BY hash (id)
SUBPARTITIONS 4 (
PARTITION p0 VALUES LESS THAN (1000),
PARTITION p1 VALUES LESS THAN (3000),
PARTITION p2 VALUES LESS THAN (10000),
PARTITION p3 VALUES LESS THAN MAXVALUE
);
```

In Code Snippet 15, the `emp_SUB_PAR` table has four `RANGE` partitions namely `p0`, `p1`, `p2`, and `p3`. Each of these partitions is further split into four sub partitions. This divides the whole table into 16 partitions.

Figure 21.12 shows the partition status of the emp_SUB_PAR table.

```
mysql> SELECT PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.PARTITIONS WHERE TABLE_NAME='emp_SUB_PAR';
+----------------+------------+
| PARTITION_NAME | TABLE_ROWS |
+----------------+------------+
| p0             |        249 |
| p0             |        250 |
| p0             |        250 |
| p0             |        250 |
| p1             |        500 |
| p1             |        500 |
| p1             |        500 |
| p1             |        500 |
| p2             |       1750 |
| p2             |       1750 |
| p2             |       1750 |
| p2             |       1750 |
| p3             |          0 |
| p3             |          0 |
| p3             |          0 |
| p3             |          0 |
+----------------+------------+
16 rows in set (0.00 sec)
```

Figure 21.12: Table Status after Sub Partitioning

21.7 Partition Management

MySQL offers several ways for making changes to partitioned tables. A developer can add, drop, split, merge, and redefine the prevailing partitions. To do so, the developer needs to use partitioning extensions to the ALTER TABLE. It is also possible to fetch information about the partitions and its corresponding table.

21.7.1 Managing RANGE and LIST Partitions

The tasks of adding and dropping RANGE and LIST partitions are quite similar. Of the two tasks, dropping is a partition is simpler and is achieved using the DROP PARTITION clause in the ALTER TABLE statement. While dropping a partition, all data stored in that partition is also deleted, which a developer can verify by executing the SELECT statement.

Code Snippet 16 creates a table partitioned by range, populates it with 10 records, and then drops a partition.

Code Snippet 16:

```
CREATE TABLE Products (id INT, name VARCHAR(50), broughtdate DATE)
PARTITION BY RANGE(YEAR(broughtdate) ) (
PARTITION ptn0 VALUES LESS THAN (1990),
PARTITION ptn1 VALUES LESS THAN (1995),
```

```
 PARTITION ptn2 VALUES LESS THAN (2000),
 PARTITION ptn3 VALUES LESS THAN (2005)
 );
INSERT INTO Products VALUES
 (1, 'Desk Organizer', '2003-10-15'),
 (2, 'CD Player', '1993-11-05'),
 (3, 'TV Set', '1996-03-10'),
 (4, 'Bookcase', '1982-01-10'),
 (5, 'Exercise Bike', '2004-05-09'),
 (6, 'Sofa', '1987-06-05'),
 (7, 'Popcorn Maker', '2001-11-22'),
 (8, 'Aquarium', '1992-08-04'),
 (9, 'Study Desk', '1984-09-16'),
 (10, 'Lava Lamp', '1998-12-25');
SELECT * FROM Products WHERE purchased BETWEEN '1995-01-01'
      AND '1999-12-31';
```

The output will be as follows:

```
+------+-----------+------------+
| id | name | purchased |
+------+-----------+------------+
| 3 | TV set | 1996-03-10 |
| 10 | lava lamp | 1998-12-25 |
+------+-----------+------------+
2 rows in set (0.00 sec)
```

Then give the following command:

```
ALTER TABLE Products DROP PARTITION ptn2;
```

Finally, give the following command:

```
SELECT * FROM Products WHERE purchased
 -> BETWEEN '1995-01-01' AND '1999-12-31';
```

Output shows:

```
Empty set (0.00 sec)
```

Concepts

Note: The NDBCLUSTER storage engine does not accept the DROP PARTITION extension.

21.7.2 Managing HASH and KEY Partitions

Tables partitioned by a key or hash value differ significantly from tables partitioned by a list or range. However, tables partitioned using key or hash values are much similar in terms of changing the partitioning arrangement. Dropping HASH or KEY partitions is dissimilar to dropping `RANGE` or `LIST` partitions.

For removing a KEY or HASH partition, a developer should use the `COALESCE PARTITION` clause in the `ALTER TABLE` statement. A number follows this clause, which indicates the number of partitions to be discarded from the table.

Code Snippet 17 uses `COALESCE PARTITION` to reduce 11 partitions to 8.

Code Snippet 17:

```
CREATE TABLE customers (
 ID INT,
 firstname VARCHAR(40),
 lastname VARCHAR(40),
 signed DATE
)
PARTITION BY HASH( MONTH(signed) )
PARTITIONS 11;
ALTER TABLE clients COALESCE PARTITION 3;
```

The COALESCE PARTITION clause functions well even on tables partitioned by `L`INEAR KEY or LINEAR HASH. Code Snippet 18 uses `COALESCE PARTITION` to reduce 11 partitions to 8 with the help of a LINEAR KEY.

Code Snippet 18:

```
CREATE TABLE customers_lk (
 ID INT,
 firstname VARCHAR(40),
 lastname VARCHAR(40),
 signed DATE
 )
PARTITION BY LINEAR KEY(signed)
 PARTITIONS 11;
ALTER TABLE customers_lk COALESCE PARTITION 3;
```

A developer can even increase the number of partitions. To do so, the `ADD PARTITION` clause is used in the `ALTER TABLE` statement.

Code Snippet 19 uses `ADD PARTITION` to increase the number of partitions to 16 of the customers table.

Code Snippet 19:

```
ALTER TABLE customers ADD PARTITION PARTITIONS 5;
```

21.7.3 Maintaining Partitions

In MySQL 5.7, a developer can perform a variety of tasks for maintaining partitions and their tables with the help of relevant SQL statements and extensions. For maintaining partitioned tables, statements such as `REPAIR TABLE`, `ANALYZE TABLE`, `OPTIMIZE TABLE`, and `CHECK TABLE` are used. The same operations such are also available for partitions, via extensions. Following are the maintenance operations to be performed on partitions:

- **Rebuilding**: Regenerates the partition similarly to dropping all rows and then re-inserting them. This operation helps in defragmentation. Code Snippet 20 rebuilds two existing partitions.

  Code Snippet 20:

  ```
  ALTER TABLE Laptops REBUILD PARTITION ptn0, ptn1;
  ```

- **Optimizing**: Optimizes a partition to defragment the partition file as well as regain an unused space. This is essential in case you have removed too many rows or have made several changes to a partitioned table having rows of variable length, which means a table with TEXT, BLOB, INT, and TEXT columns. Code Snippet 21 optimizes two existing partitions.

  Code Snippet 21:

  ```
  ALTER TABLE Laptops OPTIMIZE PARTITION ptn0, ptn1;
  ```

- **Analyzing**: Deciphers and stores the key distributions. Code Snippet 22 analyzes an existing partition.

  Code Snippet 22:

  ```
  ALTER TABLE Laptops ANALYZE PARTITION ptn3;
  ```

- **Repairing**: Fixes corrupted partitions. Code Snippet 23 repairs two existing partitions.

  Code Snippet 23:

```
ALTER TABLE Laptops REPAIR PARTITION ptn0, ptn1;
```

  Usually, `REPAIR PARTITION` is unsuccessful if there are duplicate errors in a partition. In MySQL 5.7.2 and above, using `ALTER IGNORE TABLE` along with this extension removes all rows that were previously irremovable due to duplicate keys.

- **Checking**: Scans partitions for bugs just as the `CHECK TABLE` statement for non-partitioned tables. It informs whether the indexes or data in the partitions are corrupted or not. Code Snippet 24 checks an existing partition.

  Code Snippet 24:

```
ALTER TABLE Laptops CHECK PARTITION ptn1;
```

21.7.4 Fetching Information about Partitions

Following are the different methods for fetching information about the prevalent partitions:

- The `SHOW TABLE STATUS` statement for finding out whether a specific table is partitioned or not
- The `SHOW CREATE TABLE` statement for knowing the partitioning clauses implemented while creating a partitioned table
- The `SELECT` statement on the `INFORMATION _ SCHEMA.PARTITIONS` table
- The `EXPLAIN PARTITIONS SELECT` statement to know the used partitions by a specific `SELECT` statement

21.8 Hands-On Exercise: Creating a Range Partition

Let us create a new database `TABLE_PARTITION` and activate it. This database shall hold a partitioned table along with its partitions.

1. Open the Linux terminal and run MySQL.
2. To create and activate the database, enter the following commands on the command prompt:

```
CREATE DATABASE TABLE_PARTITION;
USE TABLE_PARTITION;
```

Figure 21.13 shows the output of the commands.

```
mysql> CREATE DATABASE TABLE_PARTITION;
Query OK, 1 row affected (0.01 sec)

mysql> USE TABLE_PARTITION;
Database changed
```

Figure 21.13: Creating and Activating a Database for Holding a Partitioned Table

3. In the created database, you will now create the `Employee` table for an organization. This table will have four columns namely, `emp _ id`, `first _ name`, `last _ name`, and `department _ id` without any primary or index key. It will also be split into three partitions. The first partition will hold details of employees whose `empid < 100`, `second with 100 <= empid < 1000`, and third with `empid > 1000`. Enter the following command on the command prompt to create the table `Employee` with the three `RANGE` partitions:

```
CREATE TABLE EMPLOYEE(

EMP_ID int(11) NOT NULL,

FIRST_NAME VARCHAR(255) NOT NULL,

LAST_NAME VARCHAR(255) NOT NULL,

DEPARTMENT_ID INT NOT NULL)

PARTITION BY RANGE(EMP_ID)(

PARTITION PART0 VALUES LESS THAN (100),

PARTITION PART1 VALUES LESS THAN (1000),

PARTITION PART2 VALUES LESS THAN MAXVALUE

);
```

Figure 21.14 shows the output of the command.

```
mysql> CREATE TABLE EMPLOYEE(
    -> EMP_ID int(11) NOT NULL,
    -> FIRST_NAME VARCHAR(255) NOT NULL,
    -> LAST_NAME VARCHAR(255) NOT NULL,
    -> DEPARTMENT_ID INT NOT NULL )
    -> PARTITION BY RANGE(EMP_ID)(
    -> PARTITION PART0 VALUES LESS THAN (100),
    -> PARTITION PART1 VALUES LESS THAN (1000),
    -> PARTITION PART2 VALUES LESS THAN MAXVALUE
    -> );
Query OK, 0 rows affected (0.06 sec)
```

Figure 21.14: Creating a Table with Three Partitions by Range

4. Execute the following commands on the command prompt for inserting data into the new table:

```
INSERT INTO EMPLOYEE VALUES (1, "Jame","Bond",1);
INSERT INTO EMPLOYEE VALUES (32, "Jame","Cameron",3);
INSERT INTO EMPLOYEE VALUES (16, "Harley","Davidson",2);
INSERT INTO EMPLOYEE VALUES (123, "Captain","America",3);
INSERT INTO EMPLOYEE VALUES (1023, "Bruce","Wayne",3);
INSERT INTO EMPLOYEE VALUES (2000, "Karl","Kent",1);
```

Figure 21.15 shows the output of the commands.

```
mysql> INSERT INTO EMPLOYEE VALUES (1, "Jame","Bond",1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES (32, "Jame","Cameron",3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES (16, "Harley","Davidson",2);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES (123, "Captain","America",3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES (1023, "Bruce","Wayne",3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES (2000, "Karl","Kent",1);
Query OK, 1 row affected (0.00 sec)
```

Figure 21.15: Populating the Table for RANGE Partitioning

5. Enter the following command on the command prompt for checking the partitioning state in the `Partitions` table of `INFORMATION _ SCHEMA`:

```
SELECT TABLE_NAME, PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.
PARTITIONS WHERE TABLE_NAME = 'EMPLOYEE';
```

Figure 21.16 shows the output of the command.

```
mysql> SELECT PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.PARTITIONS WHERE
 TABLE_NAME = 'EMPLOYEE';
+----------------+------------+
| PARTITION_NAME | TABLE_ROWS |
+----------------+------------+
| PART0          |          3 |
| PART1          |          1 |
| PART2          |          2 |
+----------------+------------+
3 rows in set (0.00 sec)
```

Figure 21.16: Checking Partition Status

The PART0 partition has three records, PART1 has one record, and PART2 has two records. In this way, there six records or rows in the table are partitioned.

6. Enter the following command for obtaining information about how the specified SELECT statement would execute.

```
EXPLAIN SELECT * FROM EMPLOYEE WHERE EMP_ID = 20;
```

Figure 21.17 shows the output of the command.

```
mysql> EXPLAIN SELECT * FROM EMPLOYEE WHERE EMP_ID = 20;
+----+-------------+----------+------------+------+---------------+------+-------
--+------+------+----------+-------------+
| id | select_type | table    | partitions | type | possible_keys | key  | key_le
n | ref  | rows | filtered | Extra       |
+----+-------------+----------+------------+------+---------------+------+-------
--+------+------+----------+-------------+
|  1 | SIMPLE      | EMPLOYEE | PART0      | ALL  | NULL          | NULL | NULL
  | NULL |    3 |    33.33 | Using where |
+----+-------------+----------+------------+------+---------------+------+-------
--+------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 21.17: Optimized Output of EXPLAIN on SELECT

Although a SELECT query without WHERE clause would examine six rows, herein only three rows are examined. This is because MySQL only searches the PART0 partition as the employee ID that the query is looking for exists, in this partition. This is how partition boosts search performance.

Creating a LIST Partition

Let us now create a new table for employees, EMPLOYEE_LIST, for splitting it by a list of values.

1. In the created database, you will now create the EMPLOYEE _ LIST table. It will be the same four columns namely, emp _ id, first _ name, last _ name, and department _ id without any primary or index key. It will also be split into three partitions. The first partition will hold details of employees whose department _ id is 1 and 4, second with department _ id being 2 and 5, and third with department _ id being 3. Enter the following command on the command prompt to create the table EMPLOYEE _ LIST with the three LIST partitions:

```
CREATE TABLE EMPLOYEE_LIST(
EMP_ID int(11) NOT NULL,
FIRST_NAME VARCHAR(255) NOT NULL,
LAST_NAME VARCHAR(255) NOT NULL,
DEPARTMENT_ID INT NOT NULL)
```

```
PARTITION BY LISt(DEPARTMENT_ID)(
PARTITION PART0 VALUES IN (1,4),
     PARTITION PART1 VALUES IN (2,5),
     PARTITION PART2 VALUES IN (3)
);
```

Figure 21.18 shows the output of the command.

```
mysql> CREATE TABLE EMPLOYEE_LIST(
    -> EMP_ID int(11) NOT NULL,
    -> FIRST_NAME VARCHAR(255) NOT NULL,
    -> LAST_NAME VARCHAR(255) NOT NULL,
    -> DEPARTMENT_ID INT NOT NULL )
    -> PARTITION BY LISt(DEPARTMENT_ID)(
    -> PARTITION PART0 VALUES IN (1,4),
    -> PARTITION PART1 VALUES IN (2,5),
    -> PARTITION PART2 VALUES IN (3)
    -> );
Query OK, 0 rows affected (0.08 sec)
```

Figure 21.18: Creating a Table with Three Partitions by List

2. Execute the following commands for inserting data into the newly created table:

```
INSERT INTO EMPLOYEE_LIST VALUES (1, "Jame","Bond",1);
INSERT INTO EMPLOYEE_LIST VALUES (32, "Jame","Cameron",3);
INSERT INTO EMPLOYEE_LIST VALUES (16, "Harley","Davidson",2);
INSERT INTO EMPLOYEE_LIST VALUES (123, "Captain","America",3);
INSERT INTO EMPLOYEE_LIST VALUES (1023, "Bruce","Wayne",5);
INSERT INTO EMPLOYEE_LIST VALUES (2000, "Karl","Kent",1);
```

Figure 21.19 shows the output of the command.

```
mysql> INSERT INTO EMPLOYEE_LIST VALUES (1, "jame","bond",1);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EMPLOYEE_LIST VALUES (32, "jame","cameron",3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE_LIST VALUES (16, "harley","davidson",2);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE_LIST VALUES (123, "captain","america",3);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE_LIST VALUES (1023, "Bruce","wayne",5);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE_LIST VALUES (2000, "karl","kent",1);
Query OK, 1 row affected (0.00 sec)
```

Figure 21.19: Inserting Values in the Table for LIST Partitioning

3. Enter the following command for checking the partitioning state in the Partitions table of INFORMATION _ SCHEMA:

```
SELECT TABLE_NAME, PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.
PARTITIONS WHERE TABLE_NAME = 'EMPLOYEE_LIST';
```

Figure 21.20 shows the output of the command.

```
mysql> SELECT TABLE_NAME, PARTITION_NAME, TABLE_ROWS FROM INFORMATION_SCHEMA.PART
ITIONS WHERE TABLE_NAME = 'EMPLOYEE_LIST';
+---------------+----------------+------------+
| TABLE_NAME    | PARTITION_NAME | TABLE_ROWS |
+---------------+----------------+------------+
| employee_list | PART0          |          2 |
| employee_list | PART1          |          2 |
| employee_list | PART2          |          2 |
+---------------+----------------+------------+
3 rows in set (0.00 sec)
```

Figure 21.20: Checking Status of LIST Partitions

Each partition has two records. In this way, there six records or rows in the table are partitioned.

4. Enter the following command on the command prompt for obtaining information about how the specified SELECT statement would execute.

```
EXPLAIN SELECT * FROM EMPLOYEE_LIST WHERE EMP_ID = 20;
```

Figure 21.21 shows the output.

```
mysql> EXPLAIN SELECT * FROM EMPLOYEE_LIST WHERE EMP_ID = 20;
+----+-------------+---------------+-------------------+------+-------------
-+------+---------+------+------+----------+-------------+
| id | select_type | table         | partitions        | type | possible_keys
 | key  | key_len | ref  | rows | filtered | Extra       |
+----+-------------+---------------+-------------------+------+-------------
-+------+---------+------+------+----------+-------------+
|  1 | SIMPLE      | EMPLOYEE_LIST | PART0,PART1,PART2 | ALL  | NULL
 | NULL | NULL    | NULL |    6 |    16.67 | Using where |
+----+-------------+---------------+-------------------+------+-------------
-+------+---------+------+------+----------+-------------+
1 row in set, 1 warning (0.00 sec)
```

Figure 21.21: Non-optimized Output of EXPLAIN on SELECT through Employee ID Criterion

5. Herein, all six rows are examined. This is because partitioning is done according to the `department _ id`, while the fetch criterion in the query is based on `EMP _ ID`. Thus, there is no optimization. However, if the fetch criterion is on `department _ id`, the outcome would be optimized.

21.9 Storage Systems

In computer science, a storage system is referred to as a disk array or a storage array. It uses disk drives along with distinctive hardware and software for ensuring quick and reliable storage for processing purpose. Usually, storage systems are complex. Thus, you can think of them as special purpose processors, which offer not only storage capacity but also advanced utilities for protecting data.

21.9.1 Storage Engines

A storage engine refers to a software component designed for creating, reading, storing, and updating the data in a database. In MySQL, two types of storage engines exist namely, Transactional and Non-transactional. While `MyISAM` is the default engine for up to version 5.5, `InnoDB` is the default general-purpose one for version 5.5 and above.

The server employs a pluggable architecture for storage engines. This architecture loads and unloads the engines from a running instance. It is vital to select the most suitable storage engine, as an inappropriate one tends to degrade performance, result in data loss, and affect future development.

To find out the storage engines supported by MySQL server, the `SHOW ENGINES` statement should be used on the command prompt. Figure 21.22 shows the output of this statement.

```
mysql> SHOW ENGINES\G
*************************** 1. row ***************************
      Engine: InnoDB
     Support: DEFAULT
     Comment: Supports transactions, row-level locking, and foreign keys
Transactions: YES
          XA: YES
  Savepoints: YES
*************************** 2. row ***************************
      Engine: MRG_MYISAM
     Support: YES
     Comment: Collection of identical MyISAM tables
Transactions: NO
          XA: NO
  Savepoints: NO
*************************** 3. row ***************************
```

Figure 21.22: Output of SHOW ENGINES Statement

In Figure 21.22, the Support column value determines whether a developer can use the corresponding engine or not. For this column, the value can be `DEFUALT`, `YES`, or `NO`.

Following are the storage engines available in MySQL 5.7:

- **InnoDB**: Is transaction-safe complying with Atomicity, Consistency, Isolation, Durability (ACID) properties through crash-recovery, rollback, and commit abilities for ensuring data protection. Its row-level locking and uniform non-locking reads boost performance and concurrency in the multi-user environment. The engine stores data from users in clustered indexes for alleviating Input/Output (I/O) for usual, primary key-based queries. For ensuring data integrity, this storage engine is capable of employing FOREIGN KEY constraints. The `InnoDB` engine aims to strike a balance between high performance and high reliability.

- **MyISAM**: Contains tables with a small footprint. The performance is restricted during the loads of reads and writes due to table-level locking. This is why it is preferable for read-only or mostly-read loads in data warehousing and Web setups.

- **Memory**: Ensures quick access for rapid lookups of non-crucial data by storing data in Random Access Memory (RAM). Previously termed as the HEAP engine, the engine is now hardly used. This is because `InnoDB` offers a durable way to retain all data in memory through the featured buffer pool memory. Similarly, `NDBCLUSTER` facilitates swift key-value searches for huge and distributed data.

- **CSV**: Contains text file tables wherein values are separated by comma. These tables allow importing data in the Comma-separated Values (CSV) format, which you can exchange with applications performing write and read operations in that format. The data is retained usually in InnoDB tables at the time of normal operation because the CSV tables are un-indexed. The CSV tables are only used for importing or exporting purpose.

- **Archive**: Contains unindexed but compact tables for retaining and retrieving huge amounts of information that is hardly referenced but is useful for security audit.

- **Blackhole**: Accepts data but without storing it. The contained tables are still useful for duplicating configurations wherein statements for modification are transferred to slave servers without the master retaining its own data copy. Queries to these tables always give the outcome of empty set.

- **NDB (NDBCLUSTER)**: Is a clustered engine designed for applications demanding maximum availability as well as uptime.

- **Merge**: Allows a developer to categorize a set of alike MyISAM tables logically and refer to them as a single object. This engine is suitable for Very Large Database (VLDB) setups such as data warehousing.

- **Federated**: Allows linking different MySQL servers for forming a single logical database from a variety of physical servers. This engine is ideal for distributed environments.

- **`Example`**: Acts as an example in MySQL's source code, which shows how to code for new engines. This engine is quite significant for developers. It is much like a stub that does not perform anything. While it is fine to create tables using `Example` engine, it is impossible to store or fetch data from them.

You can use the same engine for a full schema or server as well as mention a specific engine for a table. For instance, an application may be set to use `InnoDB` tables, some `MEMORY` tables as temporary terminals, and a CSV table for fetching data to an Excel spreadsheet. You can state the desired storage engine while creating a table. This is done by using the Engine keyword. Code Snippet 25 specifies MyISAM storage engine for the table `CarModels`.

Code Snippet 25:

```
CREATE TABLE CarModels(MID INTEGER PRIMARY KEY, Name VARCHAR(60), Price
INTEGER) ENGINE='MyISAM';
```

If a developer does not explicitly state the engine, the default one is used. To know which is the default engine, the `storage_engine` variable is used. Code Snippet 26 uses this variable to know the default engine.

Code Snippet 26:

```
 SHOW VARIABLES LIKE 'storage_engine';
+----------------+--------+
| Variable_name | Value |
+----------------+--------+
| storage_engine | InnoDB |
+----------------+--------+
1 row in set (0.00 sec)
```

Code Snippet 27 shows how to change the engine of a specific table.

Code Snippet 27:

```
 SELECT ENGINE FROM information_schema.TABLES
 -> WHERE TABLE_SCHEMA='Vechiclesdb'
 -> AND TABLE_NAME='CarModels';
+--------+
| ENGINE |
+--------+
| MyISAM |
+--------+
1 row in set (0.00 sec)
```

```
ALTER TABLE CarModels ENGINE='InnoDB';
```

In MySQL 5.7, the `CREATE TABLE` statement, be default, generates `InnoDB` tables. Even if the `ENGINE` keyword is not specified, the table is created as an `InnoDB` table.

21.10 Transaction Model and Other Concepts

In `InnoDB` engine, transactions and locking allow tuning server performance and implementing a highly reliable large scale database application. The transaction model of this storage engine aims to merge the conventional 2-phase locking (2PL) ability with the characteristics of a multi-versioning database.

In this default engine, all user actions take place within a transaction. In the enabled autocommit mode, an SQL statement is considered as one transaction. By default, the session begins with the enabled autocommit mode for each connection.

Such a session can execute a transaction having multiple statements. This is done by initiating the transaction with an explicit `BEGIN` or `START TRANSACTION` statement and concluding it with a `ROLLBACK` or `COMMIT` statement. The server performs the commit action after each statement is executed, provided the statement did not give an error. In case of an error, the MySQL server performs a commit or rollback action, as per the error.

The `COMMIT` statement indicates that the changes done through the present transaction are permanent. These changes are then noticeable to other users or sessions. On the other hand, the `ROLLBACK` statement revokes all changes. Both these statements set all `InnoDB` locks free, which were applied at the time of executing the present transaction.

In case of disabled autocommit mode `(SET autocommit = 0)`, the transaction is always open in the session. A `ROLLBACK` or `COMMIT` statement finishes the present transaction after which a new one begins.

There are transaction isolation levels (SQL:1992), which aim to fine tune both reliability and performance along with consistency when several transactions are modifying and querying the same data simultaneously. These isolation levels are `REPEATABLE READ`, `SERIALIZABLE`, `READ UNCOMMITTED`, and `READ COMMITTED`; all of which are supported by `InnoDB`. The default one for this storage engine is `REPEATABLE READ`. This means that several users can keep this isolation level for all operations. Herein, data representation (snapshot) is the same since the first read operation, even if the changes occur permanently through simultaneous execution of other transactions.

However, it is possible to alter the isolation level for all succeeding connections or for only one session. This is done by using the `SET TRANSACTION` statement. For using the default isolation level, the `--transaction-isolation` option is used.

Code Snippet 28 sets the isolation level to `READ COMMITTED` for all upcoming sessions, with the help of the `GLOBAL` keyword.

Code Snippet 28:

```
SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

The READ COMMITTED isolation level ensures that a transaction's each consistent read reads its own updated snapshot. This means the users can see all committed changes prior to the execution of the read query.

Locking is performed on the row level and it executes queries by default as non-locking consistent reads, just as the Oracle style. The lock details are stored in InnoDB such that efficient space is utilized to keep lock escalation at bay. Usually, multiple users can lock every row or any arbitrary subset of rows in tables of InnoDB, without any kind of memory exhaustion.

InnoDB employs next-key locking mechanism for row-level locking. This mechanism involves locking an index record along with the existing gap just above it, with the help of next-key lock. This helps in hindering insertions by other sessions, which can attempt to add indexed values in that gap.

21.11 Other Storage Engines

Community members as well as third parties implementing the Custom Storage Engine interface may offer other storage engines. However, MySQL does not support the third party engines. Following are the available third party engines:

- **PrimeBase XT (PBXT)**: Is ideal for a Web environment with high concurrency.
- **RitmarkFS**: Enables using SQL queries for manipulating and replicating the file system.
- **Distributed Data Engine**: Stores distributed data as per the workload statistics. It is an open source engine.
- **Mdbtools**: Is a pluggable engine giving access to Microsoft Access files only for reading purpose.
- **solidDB for MySQL**: Is an open source, multi-threaded engine suitable for mission-critical tasks backed by a transactional database. It complies with isolation levels, ACID properties, Multi-Version Concurrency Control (MVCC), and row-level locking.
- **BLOB Streaming Engine (MyBS)**: Is capable of transforming MySQL server into a server for scalable media. This transformation facilitates in streaming audio and video content along with other Binary Large Objects (BLOBs).
- **InnoDB Plugin**: Allows substituting the built-in InnoDB version with a new one. It offers new features such as new INFORMATION _ SCHEMA tables, quick index creation, file management, and compression of tables and indexes.

Summary

- Replication refers to the process of copying data from a MySQL master server to single or multiple slave servers.
- Traditional replication is based on events of master's binary log, while the transactional replication is dependent upon GTIDs, providing to be a simpler method.
- You can back up and restore a database by using the mysqldump utility, compressing its output, or via PHPMyAdmin tool.
- Partitioning divides tables and indexes into smaller pieces for decreasing the cost of storing huge amount of data, enhancing performance, and simplifying maintenance.
- A user-defined rule for splitting the table data is termed as partitioning function, which can be a hashing function, modulus, or a match for a list or range.
- The SHOW PLUGINS statement allows checking whether or not the current server instance supports partitioning.
- You can partition a table by ranges, lists, hash function, columns, and keys.
- You can rebuild, optimize, analyze, check, and fix partitions.
- InnoDB is the default general-purpose storage engine for version MySQL 5.7.
- MySQL 5.7 supports different storage engines such as MyISAM, CSV, NDB, Archive, Merge Memory, Federated, and Example.
- The default isolation level of InnoDB is REPEATABLE READ.

Check Your Progress

1. Which of the following is the term used to signify the source server from which the data is to be replicated?

   a. base

   b. original

   c. master

   d. None of these

2. Which of the following utilities helps in taking a backup of a database?

   a. `mysqldump`

   b. `mysql`

   c. `mysqld`

   d. `mysqlbackup`

3. Which of the following commands helps to restore a compressed backup file?

   a. `gunzip, mysql`

   b. `gunzip, mysqld`

   c. `gunzip, mysqldump`

   d. `gunzip, mysqladmin`

Check Your Progress

4. Identify the output of the following code snippet after adding the fifth partition.

```
ALTER TABLE Books ADD PARTITION (PARTITION p4 VALUES LESS THAN (2000));
```

a. Modifies the table to hold the details of new books costing between 1000 and 2000

b. Modifies the table to hold the details of new books costing between 1000 and 1999

c. Modifies the table to hold the details of new books costing between 1 and 1999

d. None of these

5. How many partitions will the following code generate?

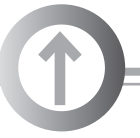
```
mysql> CREATE TABLE emp_SUB_PAR(
 id int(11) NOT NULL,
 firstname varchar(255) NOT NULL,
 lastname varchar(255) NOT NULL)
 PARTITION BY RANGE (id)
 SUBPARTITION BY hash (id)
 SUBPARTITIONS 3 (
 PARTITION partn0 VALUES LESS THAN (1000),
 PARTITION partn1 VALUES LESS THAN (3000),
 PARTITION partn2 VALUES LESS THAN (10000)
 );
```

a. 3

b. 6

c. 12

d. 9



Check Your Progress

6. Which of the following is a stored engine supported by MySQL 5.7?

   a. Blackhole

   b. MDB

   c. MyINAM

   d. Examples

Storing and Manipulating Data with MySQL

Answers to Check Your Progress

Session 1: Introduction to MySQL as an Open Source Database

1 The mysqlshow command displays information about the _____ _.

a. Databases and tables on the server
b. Tables and columns in a database
c. Columns and rows in a table
d. Version of MySQL Server

2 _____is an administrative interface for MySQL databases.

a. mysqladmin
b. PHPMyAdmin
c. admin
d. SQL

3 To execute a PHP script, you will need to install _____.

a. Web Server, PHP, MySQL
b. PHP, MySQL
c. MySQL, Web Server
d. PHP, Web Server

4 MySQL allows upto _____ indexes per table.

a. 16
b. 32
c. 46
d. 64

5 _____ tool manages users of MySQL.

a. mysqladmin
b. mysqlaccess
c. mysql
d. mysqlshow

Answers

1. a, b and c
2. b
3. a
4. d
5. b

Session 2: Installing and Configuring MySQL

1. To start MySQL client session in MS-DOS prompt, type _____ _.

   a. mysql –u root
   b. mysql
   c. mysqld-shareware –standalone
   d. mysql -u

2. Which of the following environment variable locates the MySQL history file?

   a. HOME
   b. MYSQL_PSL
   c. MYSQL_HOST
   d. MYSQL_HISTFILE

3. Which of the following options for MySQL can be specified in the **my.cnf** file?

   a. Global
   b. User
   c. Server
   d. Comment

4. In binary distribution format, the installation is configured using _____.

   a. System Variables
   b. Environment variables
   c. Server Variables
   d. Scripts

5. Which command is used to configure the installation of MySQL in Red Hat Enterprise Linux?

   a. ./configure
   b. ./configure -help
   c. configure
   d. vi configure

Answers

1. b
2. d
3. a, b, c, and d
4. b and d
5. a

Session 4: Using MySQL

1. The Boolean data type is similar to_____ _.

   a. TINYTEXT
   b. MEDIUMTEXT
   c. TINYINT
   d. MEDIUMINT

2. A foreign key in a table can also be a _____ _.

   a. Primary key in another table
   b. Composite key in another table
   c. Primary key and composite key in another table
   d. Foreign key in another table

3. An entity is said to be in the first normal form if _____ _.

   a. all the attributes are single valued
   b. all the attributes are multiple valued
   c. all the attributes are not null valued
   d. there is no top to bottom ordering of rows

4. Which type of key is used by the child table for referential integrity?

   a. Primary key
   b. Foreign key
   c. Composite key
   d. Indexes

5. Which of the following options helps to remove redundancies from a database?

   a. Referential integrity
   b. Indexing
   c. Normalization
   d. Boyce-Codd Rule

Answers

1. a
2. a
3. a
4. b
5. c

Session 6 - Implementing SQL Queries Using MySQL - I

1. You can view the columns of a table using the _____ command.
   a. SELECT
   b. VIEW
   c. SHOW
   d. USE

2. MySQL displays only the first 100 characters of each process in the SHOW PROCESSLIST command, if you do not specify the clause.
   a. STATUS
   b. VARIABLE
   c. FULL
   d. INDEX

3. Which of the following is similar to the SHOW TABLE command except that it displays detailed information about each table?
   a. SHOW STATUS
   b. SHOW TABLE STATUS
   c. SHOW TABLES
   d. SHOW VARIABLES

4. Identify the command that performs actions on temporary copy of original database.

   a. MODIFY
   b. ALTER
   c. CHANGE
   d. UPDATE

5. In which file of the database directory does MySQL store the global characteristics attributes of a database?
   a. tables_priv
   b. db
   c. user
   d. db.opt

6. Which of the following commands will display all the records of EMPLOYEE table?
   a. SELECT ALL FROM EMPLOYEE;
   b. SELECT COLUMNS FROM EMPLOYEE;
   c. SELECT * FROM EMPLOYEE;
   d. SELECT RECORDS FROM EMPLOYEE;

7. You will use the _____ option of the DROP command to display the dependencies of a table.
   a. IF EXIST
   b. RESTRICT
   c. CASCADE
   d. DROP TABLE

Answers

1. c
2. c
3. b
4. b
5. d
6. c
7. b

Session 8 - Implementing SQL Queries Using MySQL - II

1. While creating a foreign key _____ option prevents the deletion of parent row if a dependent child row exists.
   a. RESTRICT
   b. NO ACTION
   c. ON DELETE
   d. ON UPDATE

2. The _____ option of INSERT command postpones the addition of records unless all the clients finish reading from the table.

   a. LOW_PRIORITY
   b. IGNORE
   c. DELAYED
   d. DEFAULT

3. If you use the _____ option with the UPDATE command then the conflicting rows will not be updated.

   a. IGNORE
   b. WHERE
   c. LIMIT
   d. LOW_PRIORITY

4. The _____ option of the DELETE command suspends the merging of indexes.
   a. LOW_PRIORITY
   b. QUICK
   c. LIMIT
   d. ORDER BY

5. Grouping combines rows with _____ in order to operate on them together.

   a. values for a specific column into multiple rows
   b. values for a specific row
   c. values for a specific column into a single row
   d. values for specific columns into a single column

Answers

1. a
2. a
3. b
4. b
5. c

Session 10 - Using Joins

1. A comparison is made between two columns that carry same values are termed as _____ _.
   a. Self-Join
   b. Equi-Join
   c. Left Join
   d. Sub-Select

2. When there is no match in the columns of the joining table with the source table, then those columns contain _____ values.
   a. Equal
   b. Different
   c. Null
   d. Single

3. When a select query is used inside another select query then the second select query is known as_____.
   a. Multiple query
   b. Subquery
   c. Join query
   d. Single query

4. A _____ function is used to extract string or_____ from a column of a table, specified in the column reference.
   a. IN, Numeric
   b. SUB-STRING, Character literal
   c. EXIST, Decimal
   d. LIKE, Boolean

5. Which of the following specifies the conditional expression that can be used in a WHERE clause?

   a. USING
   b. FROM
   c. ON
   d. HAVING

6. Which of the following clause determine the existence of values in the list or the table?

   a. BETWEEN
   b. LIKE
   c. EXIST
   d. IN

7. Which of the following options displays all the rows from all the tables specified in the FROM clause in the result set, with or without any match with the specified table?

   a. LEFT OUTER JOIN
   b. RIGHT OUTER JOIN
   c. SELF JOIN

Answers

1. a
2. c
3. b
4. b
5. d
6. d
7. b

Session 12 - Using Basic Functions in MySQL - I

1. The output of CEIL (7.86) would be _.
    a. 7
    b. 8
    c. 0.86
    d. 0.06

2. Which function returns the number of distinct non-NULLvalues in a given column?
    a. COUNT
    b. COUNT(DISTINCT)
    c. MAX
    d. LOCATE

3. The output of FLOOR(9.85) would be _.
    a. 9
    b. 10
    c. 8
    d. 0.85

4. Which function returns the remainder of the first argument divided by the second?
    a. LOG
    b. EXP
    c. POWER
    d. MOD

5. Which function returns the non-negative square root of the value specified in the argument?

    a. SIN
    b. CRC
    c. PI
    d. SQRT

Answers

1. b
2. b
3. a
4. d
5. d

Session 14 Using Basic Functions in MySQL - II

1. Which of the following function returns the numeric value of the leftmost character of the string specified in the argument?
   a. CHAR
   b. ASCII
   c. MOD
   d. BIN

2. Which of the following function returns the index of the string in the other strings specified in the argument?
   a. FIELD
   b. CHAR
   c. BIN
   d. BIT_LENGTH

3. Which of the following function returns the length of an argument in bits?
   a. LENGTH
   b. COMPRESS
   c. BIT_LENGTH
   d. COUNT

4. Diana wants to extract the time from a given expression. Which of the following functions will help Diana to achieve this?
   a. TIME_FORMAT
   b. TIME
   c. TIMESTAMP

5. John wants to delete leading and trailing spaces before displaying the records from a table. Which of the following functions will help John to achieve this?
   a. LTRIM
   b. TRIM
   c. RTRIM
   d. TRUNCATE

6. Which of the following function returns the value of the string whose position in the argument is passed as a numeric value?
   a. ELT
   b. FIELD
   c. ORD
   d. FIND_IN_SET

7. John wants to view the username and the hostname. Which of the following functions will help John to display the current username and hostname?
   a. SCHEMA
   b. DATABASE
   c. USER
   d. VERSION

8. Which of the following function returns the name of the month for the date entered as the argument?
   a. MONTH
   b. MONTHNAME
   c. DAYOFMONTH
   d. LAST_DAY

9. Which of the following function changes all the characters of the argument into uppercase?
   a. LOWER
   b. LCASE
   c. UPPER
   d. REPLACE

10. Which of the following function returns the day number for a datetime value?
   a. TO_DAYS
   b. DAY
   c. FROM_DAYS
   d. LAST_DAY

Answers

1. b
2. a
3. c
4. b
5. b
6. a
7. c
8. b
9. c
10. b

Session 16 - Controlling and Managing MySQL database

1. For using the default database, the administrator has to log in as _____ user.

   a. super
   b. root
   c. global
   d. virtual

2. The _____ command grants and revokes privileges to all the columns of a given table.

   a. Column level
   b. Global level
   c. Database level
   d. Table level

3. A new user is added in the _____ table of_____ database.

   a. func, test
   b. db, default
   c. user, mysql
   d. host, user

4. When a user connects to the MySQL server, the location of the user gets detected from the_____ and by the _____name.

   a. Server, path
   b. Hostname, user
   c. Client machine, user
   d. Table level, target

5. Which of the following options help to set the maximum number of simultaneous connections per account?

   a. MAX_QUERIES_PER_HOUR
   b. MAX_UPDATES_PER_HOUR
   c. MAX_CONNECTIONS_PER_HOUR
   d. MAX_USER_CONNECTIONS

Answers

1. b
2. d
3. c
4. b
5. d

Session 18 - An Overview of MySQL 5.7

1. Which of the following programs is used in logical upgrade for starting new server instance via newly created data directory?
   a. mysqld
   b. mysqldadmin
   c. mysql_upgrade
   d. mysqld_safe

2. Which of the following tables exists in Performance Schema?
   a. Archive
   b. System
   c. Object Instance
   d. System Config

3. Which of the following is the prerequisite for using sys schema?
   a. Enabled instruments of Performance Schema
   b. EXECUTE for InnoDB
   c. DELETE for INFORMATION_SCHEMA INNODB_BUFFER_PAGE table
   d. Disabled consumers of Performance Schema

4. Which of the following programs stops multiple servers at once?
   a. mysqld_stop
   b. mysqld_multiserver
   c. mysqld_multiple
   d. mysqld_multi

5. Which of the following codes converts WKT representation of a point into a Point object?
   a. SELECT ST_X(ST_PointFromText('POINT(5 10)'));
   b. SELECT ST_X(ST_GeomFromText('POINT(5 10)'));
   c. SELECT ST_X(ST_MPointFromText('POINT(5 10)'));
   d. SELECT ST_X(ST_PointFromWKT('POINT(5 10)'));

6. Identify the output of the following code.

```
SET @json = '["d", {"e": [true, false]}]';
SELECT JSON_REPLACE(@json,
'$[1].e[2]', 1);
```

   a. ["d", {"e": [true, false, 1]}]
   b. ["d", {"e": [1, false]}]
   c. ["d", {"e": [true, 1]}]
   d. ["d", {"e": [true, false]}]

Answers

1. d
2. c
3. a
4. d
5. b
6. d

Session 19 - Transaction Management and Globalization

1. Match the following statements with their description.

Statements		Description	
a.	SET TRANSACTION	1.	Ends a failed transaction and returns it to the beginning of a transaction
b.	SAVEPOINT	2.	Sets the constraint mode and allows you to decide if a constraint needs to be applied to the data while it is modified or scheduled to be applied later
c.	COMMIT	3.	Serves as an intermediate retrieval point
d.	SET CONSTRAINTS	4.	Sets the attributes of the next transaction to be executed
e.	ROLLBACK	5.	Ends a successful transaction and saves all the modifications done to the database

2. Which of the following statements cause an implicit commit?

   a. CREATE TEMPORARY TABLE
   b. LOAD INDEX INTO CACHE
   c. UPGRADE DATA DIRECTORY NAME
   d. CREATE PROCEDURE

3. Stored procedures with which of the following option have access privileges that are same as the MySQL user who executed the stored procedure?

   a. SQL SECURITY DEFINER
   b. SQL SECURITY EXECUTER
   c. SQL SECURITY INVOKER
   d. SQL SECURITY CREATOR

4. Which of the following statements are true with respect to stored procedures?

   a. Stored procedures are precompiled
   b. 'Call' statement cannot be used to invoke stored procedures
   c. Stored procedures have a statement for raising application errors
   d. Stored procedures are fast

5. How does transaction/commit affect the performance?

   a. Increases performance when the changes are committed immediately
   b. Increases performance when the changes are committed after a valid number of updates
   c. The atomicity aspect of the ACID model is for all storage engines
   d. Implicit commit statements can be used within a transaction that is, between starting and committing a transaction

Answers

1. a-4, b-3, c-5, d-2, e-1
2. a
3. c
4. a, b, and c
5. b

Session 20 - Performance Optimization and Scalability

1. Which of the following options can optimize a MySQL query?

   a. Ensure a table has a primary key
   b. Index the field which is used more frequently for searching
   c. The order of condition does not matter
   d. WHERE condition has an indexed column in SELECT statement

2. Match the following.

Data		Description	
a.	UNION	1.	SELECT is part of a sub query within a FROM clause
b.	PRIMARY	2.	SELECT is the second or later statement of a UNION
c.	SUBQUERY	3.	SELECT is in the outermost query in a JOIN
d.	DERIVED	4.	Second or later SELECT of a UNION is dependent on an outer query
e.	DEPENDENT UNION	5.	First SELECT in a sub query

3. Which of the following are advantages of using indexes?
   a. Generates a distinct ordered file that holds only the field(s) that need to be sorted.
   b. Requires more space than a data file.
   c. Increases the performance of queries such as INSERT, UPDATE, and DELETE.
   d. Helps to retrieve rows easily when the data used is large.

4. In which of the following instances, will the query cache not be used?

   a. Queries that are a sub-query of another query
   b. In transactions with InnoDB tables
   c. Queries processed within a stored function, trigger, or event
   d. If TEMPORARY tables are referred

5. Match the following.

Data		Description	
a.	Query Analyzer Table	1.	Recognizes queries with slow response times
b.	Response Statistics	2.	Provides key metrics such as Execution Time, Executions, Rows, and Kilobytes at a given time for a specific query
c.	Query Response Time Index	3.	Maintains details of all the executing queries
d.	Explain Query	4.	Provides information such as Row Statistics, Time Span, and First Seen
e.	Graphs	5.	Maintains the execution plan used for the query

Answers

1. a, b, and d
2. a-2, b-3, c-5, d-1, e-4
3. d
4. a, c, and d
5. a-3, b-4, c-1, d-5, e-2

Session 21: Replication, Data Backup, Partitioning, and Storage Systems

1. Which of the following is the term used to signify the source server from which the data is to be replicated?
   a. base
   b. original
   c. master
   d. None of these

2. Which of the following utilities helps in taking a backup of a database?
   a. mysqldump
   b. mysql
   c. mysqld
   d. mysqlbackup

3. Which of the following commands helps to restore a compressed backup file?
   a. gunzip, mysql
   b. gunzip, mysqld
   c. gunzip, mysqldump
   d. gunzip, mysqladmin

4. Identify the output of the following code snippet after adding the fifth partition.
   ALTER TABLE Books ADD PARTITION (PARTITION p4 VALUES LESS THAN (2000));
   a. Modifies the table to hold the details of new books costing between 1000 and 2000
   b. Modifies the table to hold the details of new books costing between 1000 and 1999
   c. Modifies the table to hold the details of new books costing between 1 and 1999
   d. None of these

5. How many partitions will the following code generate?
   mysql> CREATE TABLE emp_SUB_PAR(id int(11) NOT NULL,
   firstname varchar(255) NOT NULL, lastname varchar(255) NOT NULL) PARTITION BY RANGE (id) SUBPARTITION BY hash (id)
   SUBPARTITIONS 3 (
   PARTITION partn0 VALUES LESS THAN (1000), PARTITION partn1 VALUES LESS THAN (3000), PARTITION partn2 VALUES LESS THAN (10000));

   a. 3
   b. 6
   c. 12
   d. 9

6. Which of the following is a stored engine supported by MySQL 5.7?
   a. Blackhole
   b. MDB
   c. MyINAM
   d. Examples

Answers

1. c
2. a
3. a
4. b
5. d
6. a